
REVERSE ENGINEERING DATA

From the Central Limit Theorem to TinyGPT

Reverse Engineering Data: A Complete Course in Statistics, Machine Learning, and Deep Learning with PyTorch

Python • PyTorch • MPS • 30 Notebooks

Statistics, Machine Learning, and Deep Learning with PyTorch

September 24, 2026

The complete course, notebooks 00 through 29

Contents

Preface

Who This Book Is For

How to Use This Book

Setting Up Your Environment

The Notebooks

A Note on Code Style

Part I: Foundations

00-CLT: Central Limit Theorem

01-EDA: Penguins Dataset - EDA

02-Clustering: Penguins Dataset - Clustering

Part II: Simple Linear Regression

03-SLR-LR: Penguins Dataset Simple Linear Regression - Linear Relationship

04-SLR-NLR: Penguins Dataset Simple Linear Regression - Non-Linear Relationship

05-SLR-FLF: Penguins Dataset Simple Linear Regression - Fitting Linear Feature

06-SLR-FNLF: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature

07-SLR-FLFP: Penguins Dataset Simple Linear Regression - Fitting Linear Feature Predictions

08-SLR-FNLFP: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature Predictions

09-SLR-MLFP: Penguins Dataset Simple Linear Regression - Measuring Linear Feature Performance

10-SLR-FMM: Penguins Dataset Simple Linear Regression - Fitting Multiple Models

Part III: Multiple Linear Regression and Linear Regression

11-MLR-AF: Multiple Linear Regression - Additive Features

12-MLR-IF: Multiple Linear Regression - Interactive Features

13-LR: Penguins Dataset - Linear Regression

Part IV: Classification and Deep Learning with PyTorch

14-LR: Penguins Dataset - Logistic Regression

15-MLP: Penguins Dataset - MLP

16-CNN: Fashion-MNIST Dataset - CNN

Part V: Sequences, Attention, and Language Models

17-RNN: Tiny Shakespeare Dataset - RNN

18-ATT: Tiny Shakespeare Dataset - Attention

19-GPT: Tiny Shakespeare Dataset - TinyGPT From Scratch

Part VI: Regularization, Evaluation, and Generative Vision

20-REG: Breast Cancer Dataset - Regularization

21-EVAL: Breast Cancer Dataset - Model Evaluation

22-AE: Fashion-MNIST Dataset - Autoencoder

23-GAN: Fashion-MNIST Dataset - GAN

24-VIT: Fashion-MNIST Dataset - Vision Transformer

Part VII: Segmentation, Reinforcement Learning, and Deployment

25-DEPLOY: Fashion-MNIST Dataset - Model Deployment

26-SEG: Oxford-IIIT Pet Dataset - Semantic Segmentation

27-DRL: CartPole Environment - Deep Q-Network

28-LLM: Tiny Shakespeare Dataset - Fine-Tuning a Pretrained LLM

Part VIII: A Kaggle Project from Scratch

29-KAGGLE: Kaggle Titanic Dataset - EDA and Binary Classifier

Preface

This book is the complete *Reverse-Engineering-Data* course, expanded from thirty Jupyter notebooks into a single reference. It begins with the Central Limit Theorem and ends with a decoder-only transformer, TinyGPT, written from scratch. Between those two points it teaches exploratory data analysis, clustering, simple and multiple linear regression, logistic regression, and then the full arc of modern deep learning in PyTorch: multilayer perceptrons, convolutional networks, recurrent networks, attention, transformers, regularization, evaluation, autoencoders, generative adversarial networks, vision transformers, semantic segmentation, deep reinforcement learning, deployment with ONNX, fine-tuning a pretrained language model, and a complete Kaggle project from scratch.

The method of the course is *reverse engineering*: taking a dataset or a model apart, understanding every piece, and rebuilding it from first principles. Nothing is treated as a black box. Every model is defined in code you can read, every metric is computed by hand before it is called from a library, and every result is reproducible.

Who This Book Is For

This book is for the student who wants to understand data and models from the ground up. It assumes only that you can install software, open a terminal, and persist through errors. Every concept is introduced with a concrete, runnable example.

How to Use This Book

Read with a keyboard. Every chapter is a notebook, and every code block in this book is the code from that notebook, followed by the output it produced when it was run. Type the code, run it, and compare your output to the printed output. The exercises and the reasoning matter more than the results.

The chapters are cumulative. Parts I through III build the statistical foundation. Part IV begins deep learning. Parts V through VII build modern architectures and put them to work. You can read the parts in order, or jump to a chapter that interests you and follow the cross-references back.

Setting Up Your Environment

Every notebook in this course runs in a single Conda environment named `prod`. Create it once:

```
conda create --name prod anaconda
conda activate prod
conda update --all
```

Then add the libraries this course needs that are not in the Anaconda bundle:

```
conda install -c conda-forge pytorch torchvision torchaudio z3-solver onnx onnxruntime
pip install maraboupy gymnasium transformers
```

The course was executed on an Apple Silicon Mac using PyTorch with the Metal (MPS) backend. Every notebook selects its device automatically:

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
```

The Notebooks

Notebook	Topic	Dataset
00-CLT	Central Limit Theorem	Simulated
01-EDA	Exploratory Data Analysis	Penguins
02-Clustering	KMeans and PCA	Penguins
03 to 10 SLR	Simple Linear Regression	Penguins
11 to 12 MLR	Multiple Linear Regression	Penguins
13-LR	Linear Regression and inference	Penguins
14-LR	Logistic Regression and ROC	Penguins
15-MLP	Multilayer Perceptron	Penguins
16-CNN	Convolutional Neural Network	Fashion-MNIST
17-RNN	Recurrent Neural Network	Tiny Shakespeare
18-ATT	Attention and Transformer block	Tiny Shakespeare
19-GPT	TinyGPT from scratch	Tiny Shakespeare
20-REG	Regularization and early stopping	Breast Cancer
21-EVAL	Model Evaluation and calibration	Breast Cancer
22-AE	Autoencoder and anomaly detection	Fashion-MNIST
23-GAN	Generative Adversarial Network	Fashion-MNIST
24-VIT	Vision Transformer	Fashion-MNIST
25-DEPLOY	ONNX deployment and CLI	Fashion-MNIST
26-SEG	U-Net semantic segmentation	Oxford-IIIT Pet
27-DRL	Deep Q-Network	CartPole
28-LLM	Fine-tuning DistilGPT-2	Tiny Shakespeare
29-KAGGLE	Kaggle project: EDA and binary classifier	Titanic

A Note on Code Style

Every function, method, and class in this course has a full docstring with both a `Parameters:` section and a `Returns:` section, even when one or both are empty. Signatures carry no type hints; types live in the docstrings. Every function body is fewer than eight lines, there are no blank lines inside a function body, and longer logic is split into small documented helpers. This is the house style of the course, and it is the style a research group will expect.

Part I: Foundations

00-CLT: Central Limit Theorem

Author: [Kevin Thomas](#)

Import Modules

```
import random
import matplotlib.pyplot as plt
```

For Loop

Basic for loop example.

```
numbers = [1,
           2,
           3,
           4,
           5]
```

```
for num in numbers:
    print(num)
```

```
1
2
3
4
5
```

Random Numbers

Generate 100 Random Numbers w/ a For Loop

```
random.seed(42)

for _ in range(100):
    print(random.random())
```

0.6394267984578837
0.025010755222666936
0.27502931836911926
0.22321073814882275
0.7364712141640124
0.6766994874229113
0.8921795677048454
0.08693883262941615
0.4219218196852704
0.029797219438070344
0.21863797480360336
0.5053552881033624
0.026535969683863625
0.1988376506866485
0.6498844377795232
0.5449414806032167
0.2204406220406967
0.5892656838759087
0.8094304566778266
0.006498759678061017
0.8058192518328079
0.6981393949882269
0.3402505165179919
0.15547949981178155
0.9572130722067812
0.33659454511262676
0.09274584338014791
0.09671637683346401
0.8474943663474598
0.6037260313668911
0.8071282732743802
0.7297317866938179
0.5362280914547007
0.9731157639793706
0.3785343772083535
0.552040631273227
0.8294046642529949
0.6185197523642461
0.8617069003107772
0.577352145256762
0.7045718362149235
0.045824383655662215
0.22789827565154686
0.28938796360210717
0.0797919769236275
0.23279088636103018
0.10100142940972912
0.2779736031100921
0.6356844442644002
0.36483217897008424
0.37018096711688264
0.2095070307714877
0.26697782204911336
0.936654587712494
0.6480353852465935
0.6091310056669882

```
0.171138648198097
0.7291267979503492
0.1634024937619284
0.3794554417576478
0.9895233506365952
0.6399997598540929
0.5569497437746462
0.6846142509898746
0.8428519201898096
0.7759999115462448
0.22904807196410437
0.03210024390403776
0.3154530480590819
0.26774087597570273
0.21098284358632646
0.9429097143350544
0.8763676264726689
0.3146778807984779
0.65543866529488
0.39563190106066426
0.9145475897405435
0.4588518525873988
0.26488016649805246
0.24662750769398345
0.5613681341631508
0.26274160852293527
0.5845859902235405
0.897822883602477
0.39940050514039727
... (10 lines omitted) ...
0.38161928650653676
0.9961213802400968
0.529114345099137
0.9710783776136181
0.8607797022344981
```

Generate 100 Random Numbers w/ a For Loop and Store in a List

```
random_numbers = []
```

```
random_numbers
```

```
[]
```

```
random.seed(42)

for _ in range(100):
    random_numbers.append(random.random())
```

random_numbers

[0.6394267984578837,
0.025010755222666936,
0.27502931836911926,
0.22321073814882275,
0.7364712141640124,
0.6766994874229113,
0.8921795677048454,
0.08693883262941615,
0.4219218196852704,
0.029797219438070344,
0.21863797480360336,
0.5053552881033624,
0.026535969683863625,
0.1988376506866485,
0.6498844377795232,
0.5449414806032167,
0.2204406220406967,
0.5892656838759087,
0.8094304566778266,
0.006498759678061017,
0.8058192518328079,
0.6981393949882269,
0.3402505165179919,
0.15547949981178155,
0.9572130722067812,
0.33659454511262676,
0.09274584338014791,
0.09671637683346401,
0.8474943663474598,
0.6037260313668911,
0.8071282732743802,
0.7297317866938179,
0.5362280914547007,
0.9731157639793706,
0.3785343772083535,
0.552040631273227,
0.8294046642529949,
0.6185197523642461,
0.8617069003107772,
0.577352145256762,
0.7045718362149235,
0.045824383655662215,
0.22789827565154686,
0.28938796360210717,
0.0797919769236275,
0.23279088636103018,
0.10100142940972912,
0.2779736031100921,
0.6356844442644002,
0.36483217897008424,
0.37018096711688264,
0.2095070307714877,
0.26697782204911336,
0.936654587712494,
0.6480353852465935,
0.6091310056669882,

```
0.171138648198097,  
0.7291267979503492,  
0.1634024937619284,  
0.3794554417576478,  
0.9895233506365952,  
0.6399997598540929,  
0.5569497437746462,  
0.6846142509898746,  
0.8428519201898096,  
0.7759999115462448,  
0.22904807196410437,  
0.03210024390403776,  
0.3154530480590819,  
0.26774087597570273,  
0.21098284358632646,  
0.9429097143350544,  
0.8763676264726689,  
0.3146778807984779,  
0.65543866529488,  
0.39563190106066426,  
0.9145475897405435,  
0.4588518525873988,  
0.26488016649805246,  
0.24662750769398345,  
0.5613681341631508,  
0.26274160852293527,  
0.5845859902235405,  
0.897822883602477,  
0.39940050514039727,  
... (10 lines omitted) ...  
0.38161928650653676,  
0.9961213802400968,  
0.529114345099137,  
0.9710783776136181,  
0.8607797022344981]
```

List Comprehension

```
<output_list> = [<action> for <variable> in <sequence>]
```

```
random.seed(42)
```

```
random_numbers_lc = [random.random() for _ in range(100)]
```

```
random_numbers_lc
```

[0.6394267984578837,
0.025010755222666936,
0.27502931836911926,
0.22321073814882275,
0.7364712141640124,
0.6766994874229113,
0.8921795677048454,
0.08693883262941615,
0.4219218196852704,
0.029797219438070344,
0.21863797480360336,
0.5053552881033624,
0.026535969683863625,
0.1988376506866485,
0.6498844377795232,
0.5449414806032167,
0.2204406220406967,
0.5892656838759087,
0.8094304566778266,
0.006498759678061017,
0.8058192518328079,
0.6981393949882269,
0.3402505165179919,
0.15547949981178155,
0.9572130722067812,
0.33659454511262676,
0.09274584338014791,
0.09671637683346401,
0.8474943663474598,
0.6037260313668911,
0.8071282732743802,
0.7297317866938179,
0.5362280914547007,
0.9731157639793706,
0.3785343772083535,
0.552040631273227,
0.8294046642529949,
0.6185197523642461,
0.8617069003107772,
0.577352145256762,
0.7045718362149235,
0.045824383655662215,
0.22789827565154686,
0.28938796360210717,
0.0797919769236275,
0.23279088636103018,
0.10100142940972912,
0.2779736031100921,
0.6356844442644002,
0.36483217897008424,
0.37018096711688264,
0.2095070307714877,
0.26697782204911336,
0.936654587712494,
0.6480353852465935,
0.6091310056669882,


```
0.171138648198097,  
0.7291267979503492,  
0.1634024937619284,  
0.3794554417576478,  
0.9895233506365952,  
0.6399997598540929,  
0.5569497437746462,  
0.6846142509898746,  
0.8428519201898096,  
0.7759999115462448,  
0.22904807196410437,  
0.03210024390403776,  
0.3154530480590819,  
0.26774087597570273,  
0.21098284358632646,  
0.9429097143350544,  
0.8763676264726689,  
0.3146778807984779,  
0.65543866529488,  
0.39563190106066426,  
0.9145475897405435,  
0.4588518525873988,  
0.26488016649805246,  
0.24662750769398345,  
0.5613681341631508,  
0.26274160852293527,  
0.5845859902235405,  
0.897822883602477,  
0.39940050514039727,  
... (10 lines omitted) ...  
0.38161928650653676,  
0.9961213802400968,  
0.529114345099137,  
0.9710783776136181,  
0.8607797022344981]
```

Average

1. Get the sum of all the numbers.

```
nums = [1,  
        2,  
        3,  
        4,  
        5]
```

```
a_sum = 0  
for num in nums:  
    a_sum += num
```

```
a_sum
```

```
15
```

2. Count the number of elements in the list.

```
nums_len = len(nums)
```

```
nums_len
```

```
5
```

3. Divide the sum by the number of elements in the list.

```
a_sum / nums_len
```

```
3.0
```

Create a Function to Calculate the Average

```
def my_avg(n):  
    """  
    Calculate the average of a list of numbers.  
  
    Parameters:  
        n (list): A list of numbers.  
  
    Returns:  
        float: The average of the numbers in the list.  
    """  
    return sum(n) / len(n)
```

```
my_avg(nums)
```

```
3.0
```

Getting Help on a Function

```
help(my_avg)
```

Help on function my_avg in module __main__:

```
my_avg(n)
    Calculate the average of a list of numbers.

    Parameters:
        n (list): A list of numbers.

    Returns:
        float: The average of the numbers in the list.
```

Collect 3 Observations on Different Days for 10 People

```
random.seed(42)

day1 = [random.random() for _ in range(10)]
day2 = [random.random() for _ in range(10)]
day3 = [random.random() for _ in range(10)]
```

day1

```
[0.6394267984578837,
 0.025010755222666936,
 0.27502931836911926,
 0.22321073814882275,
 0.7364712141640124,
 0.6766994874229113,
 0.8921795677048454,
 0.08693883262941615,
 0.4219218196852704,
 0.029797219438070344]
```

day2

```
[0.21863797480360336,
 0.5053552881033624,
 0.026535969683863625,
 0.1988376506866485,
 0.6498844377795232,
 0.5449414806032167,
 0.2204406220406967,
 0.5892656838759087,
 0.8094304566778266,
 0.006498759678061017]
```

day3

```
[0.8058192518328079,  
0.6981393949882269,  
0.3402505165179919,  
0.15547949981178155,  
0.9572130722067812,  
0.33659454511262676,  
0.09274584338014791,  
0.09671637683346401,  
0.8474943663474598,  
0.6037260313668911]
```

All 3 Have Different Values, Let's Calculate Their Averages

```
my_avg(day1)
```

```
0.4006685751243019
```

```
my_avg(day2)
```

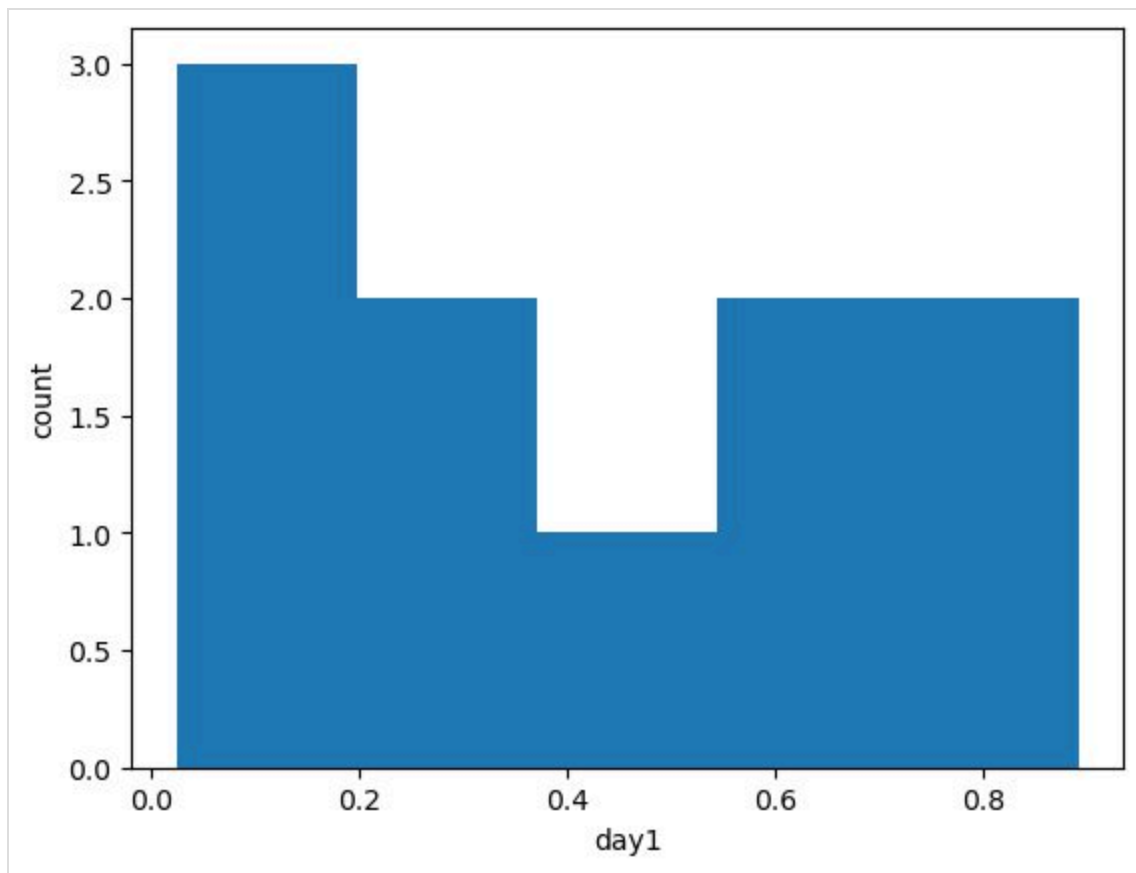
```
0.3769828323932711
```

```
my_avg(day3)
```

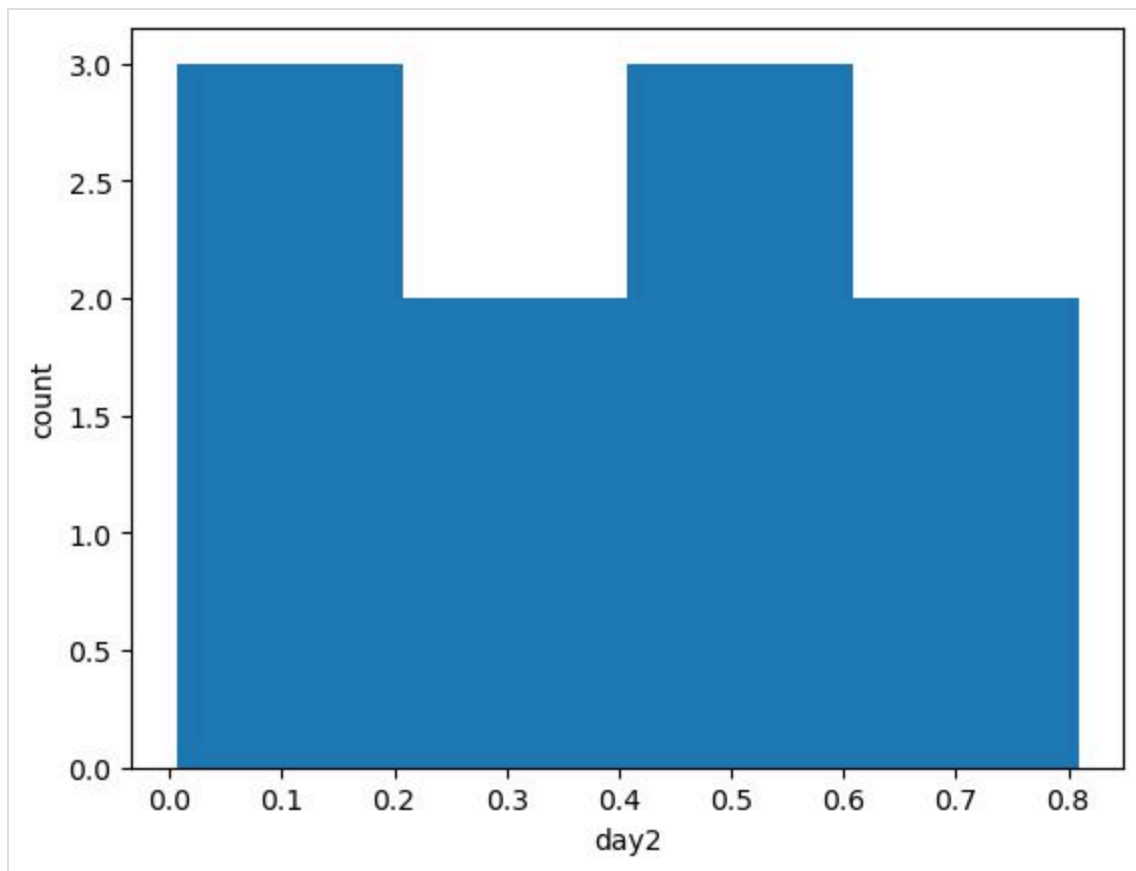
```
0.49341788983981794
```

Visualize the Distribution Using a Histogram

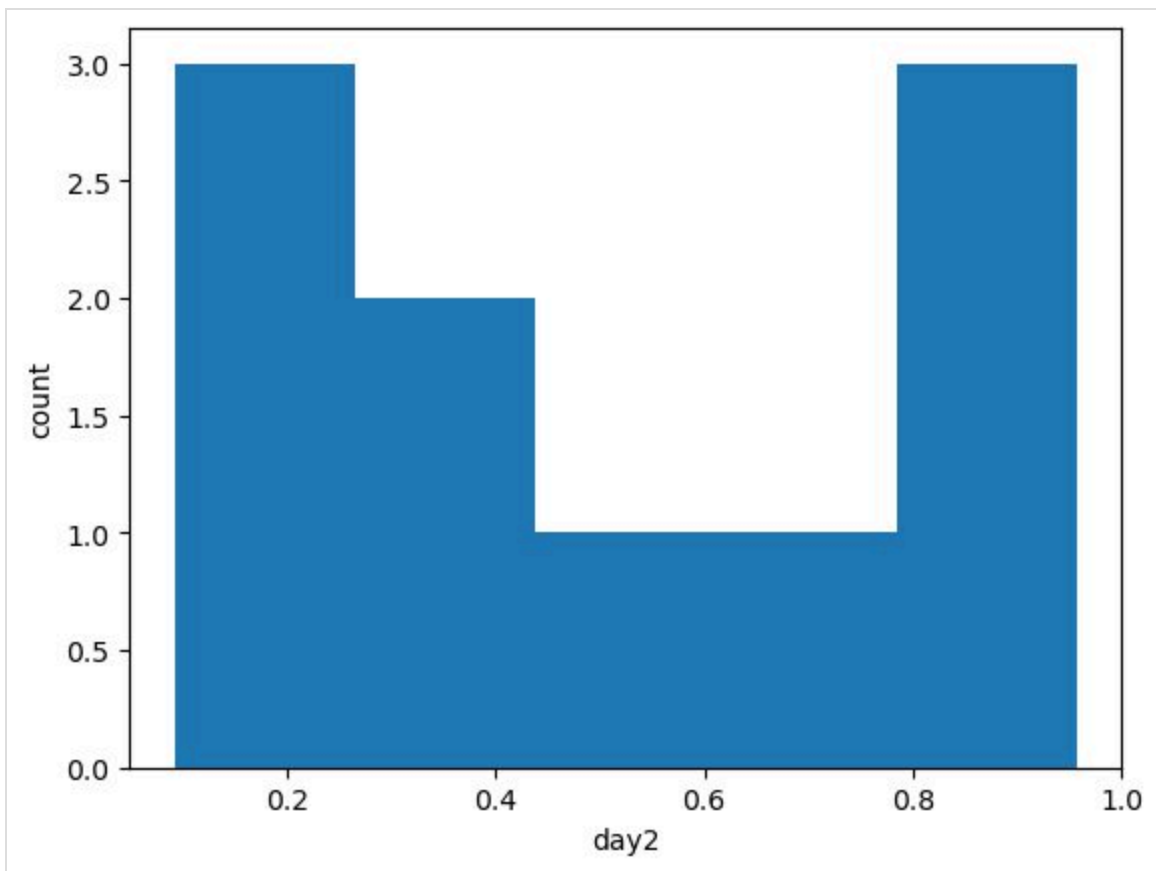
```
plt.hist(day1, bins=5)  
plt.xlabel('day1')  
plt.ylabel('count')  
plt.show()
```



```
plt.hist(day2, bins=4)
plt.xlabel('day2')
plt.ylabel('count')
plt.show()
```



```
plt.hist(day3, bins=5)
plt.xlabel('day2')
plt.ylabel('count')
plt.show()
```



Create a Function for Generating Samples and Calculating the Average

```
def gen_samples_calc_avg(n):  
    """  
    Assume we are generating values from a UNIFORM distribution between 0 and 1  
  
    Parameters:  
        n (int): The number of random numbers to generate.  
  
    Returns:  
        float: The average of the generated random numbers.  
    """  
    return my_avg([random.random() for _ in range(n)])
```

Let's Calculate the Average of 10 Numbers 100 Times

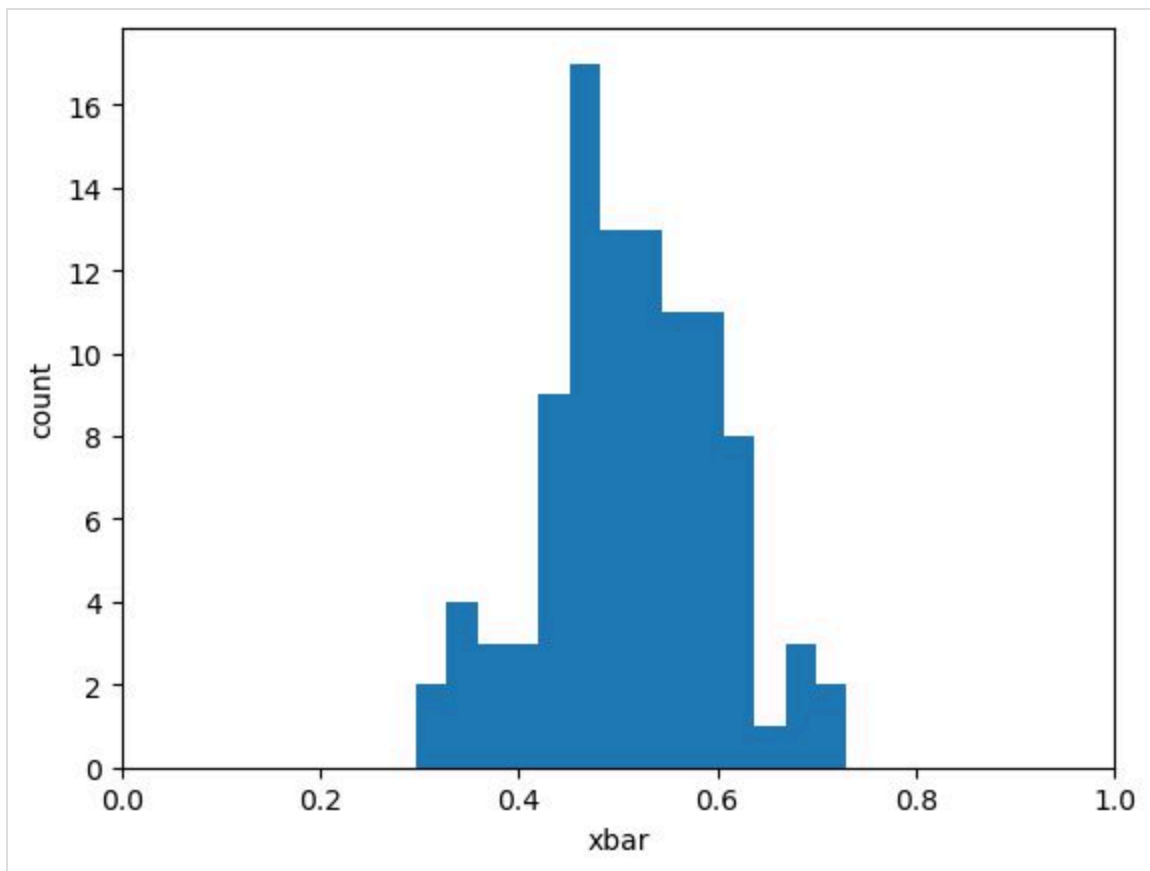
```
random.seed(42)  
  
num_10_100s = [gen_samples_calc_avg(10) for _ in range(100)]
```

```
num_10_100s
```

[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,


```
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,  
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
... (10 lines omitted) ...  
0.6310578737150109,  
0.5747700872274695,  
0.4611341010100567,  
0.5249278209488202,  
0.5015525305823275]
```

```
plt.hist(num_10_100s, bins=14)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Let's Calculate the Average of 10 Numbers 1000 Times

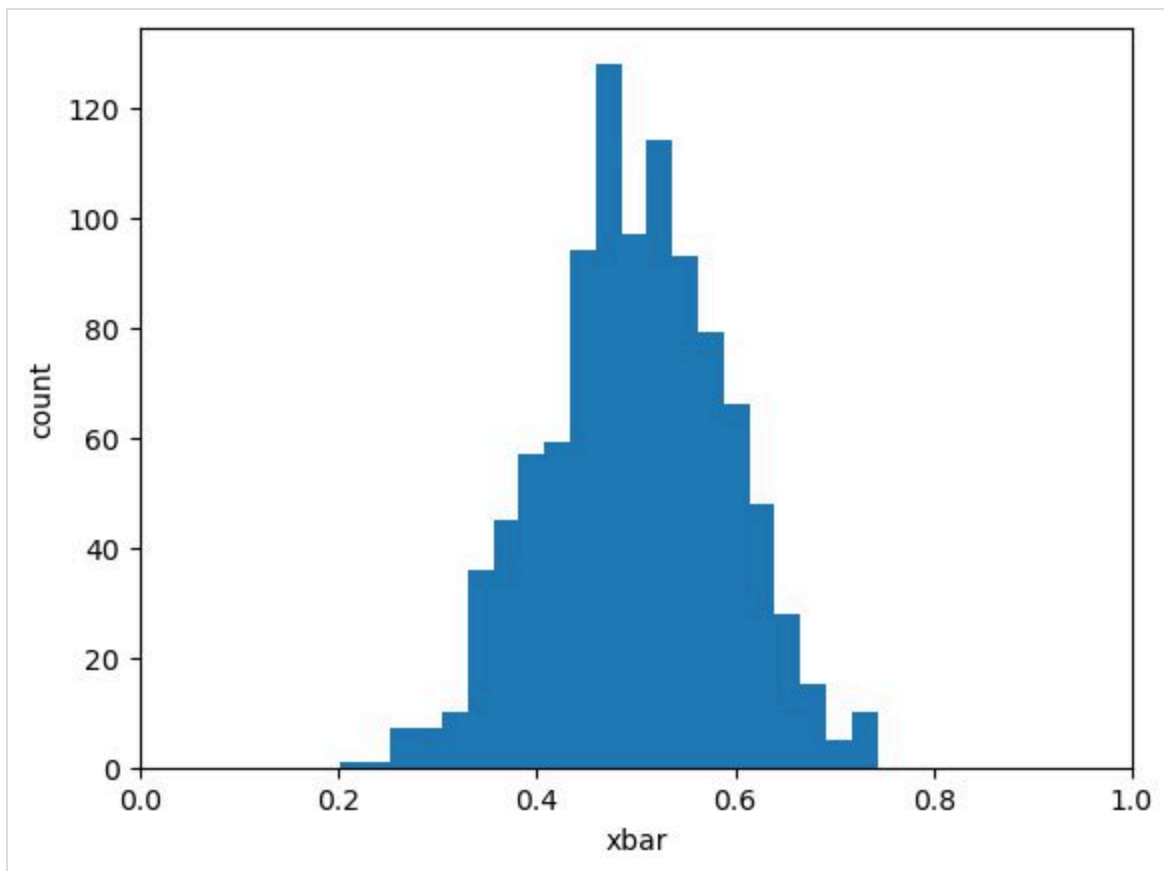
```
random.seed(42)  
num_10_1000s = [gen_samples_calc_avg(10) for _ in range(1000)]
```

```
num_10_1000s
```

[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,

```
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,  
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
... (910 lines omitted) ...  
0.3652910975718219,  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831]
```

```
plt.hist(num_10_1000s, bins=21)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Let's Calculate the Average of 10 Numbers 10,000 Times

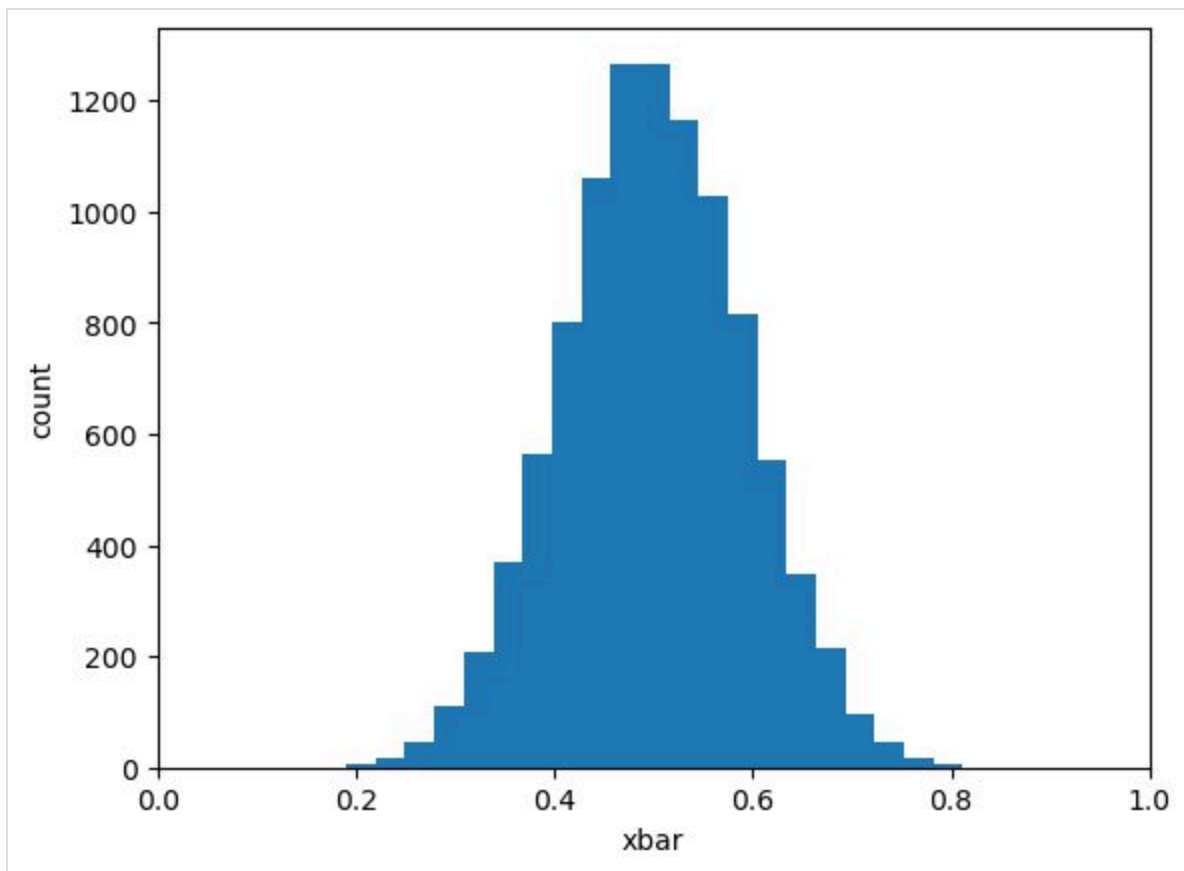
```
random.seed(42)  
num_10_10000s = [gen_samples_calc_avg(10) for _ in range(10000)]
```

```
num_10_10000s
```

[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,

```
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,  
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
... (911 lines omitted) ...  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831,  
...]
```

```
plt.hist(num_10_10000s, bins=21)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Let's Calculate the Average of 10 Numbers 100,000 Times

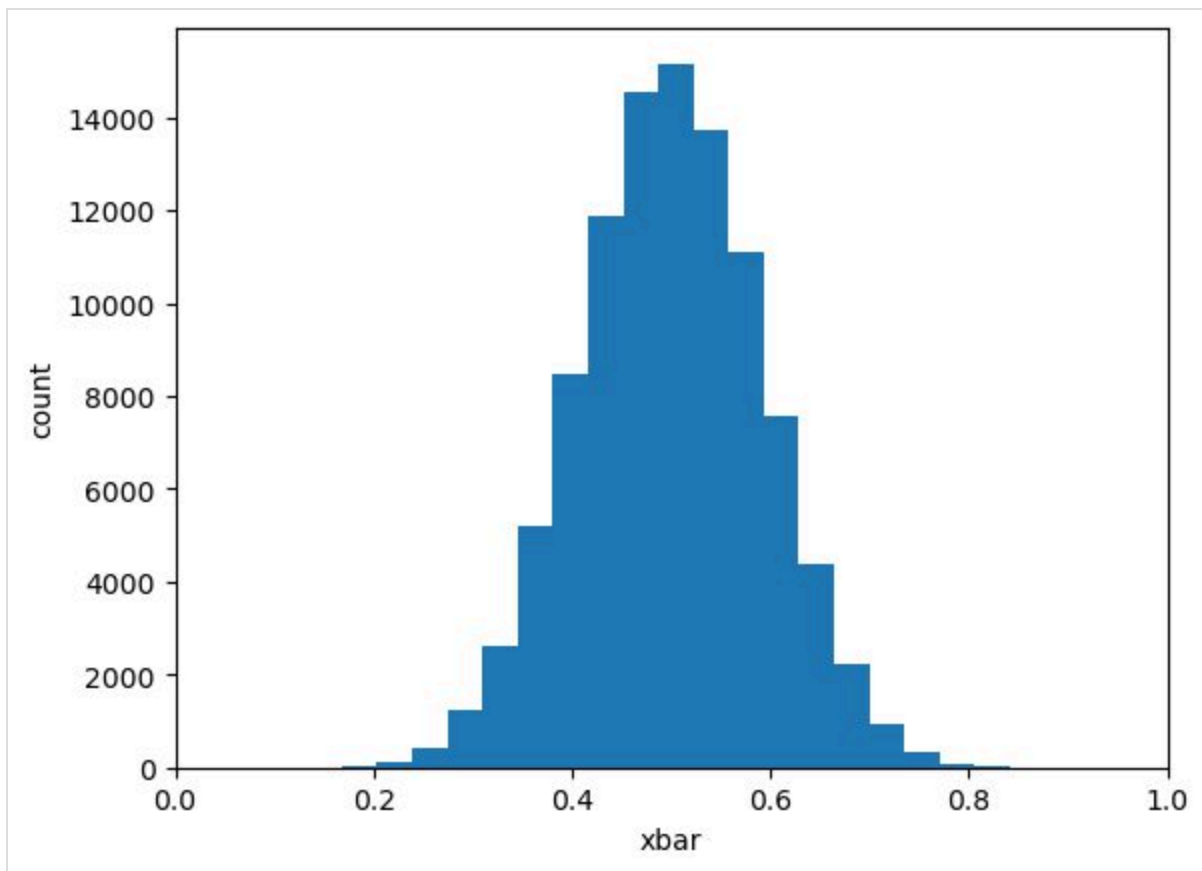
```
random.seed(42)  
num_10_100000s = [gen_samples_calc_avg(10) for _ in range(100000)]
```

```
num_10_100000s
```


[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,

```
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,  
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
... (911 lines omitted) ...  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831,  
...]
```

```
plt.hist(num_10_100000s, bins=21)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Let's Calculate the Average of 10 Numbers 1,000,000 Times

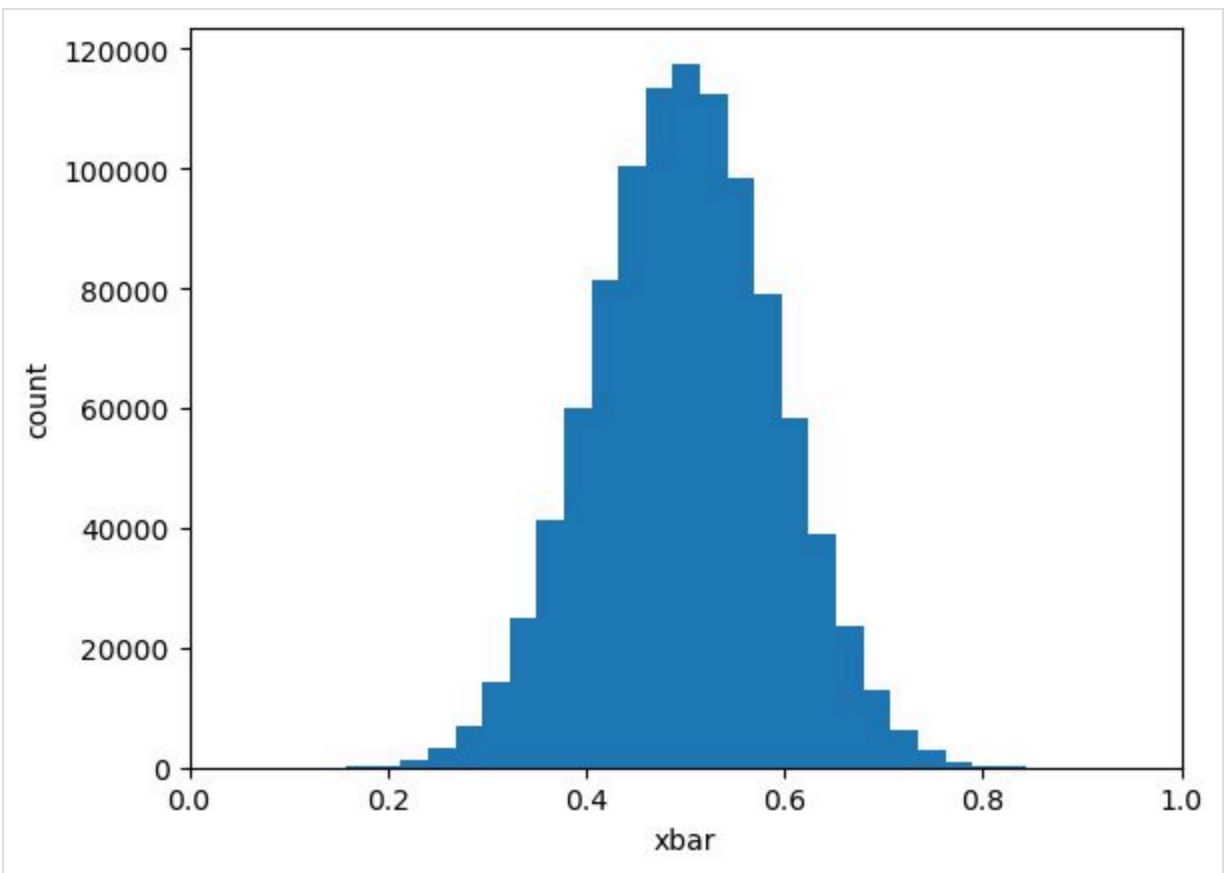
```
random.seed(42)
num_10_1000000s = [gen_samples_calc_avg(10) for _ in range(1000000)]
```

```
num_10_1000000s
```

[0.4006685751243019,
0.3769828323932711,
0.49341788983981794,
0.686376238606863,
0.29597569781632027,
0.44836101802315814,
0.533428117689419,
0.528091574806805,
0.4570329568579887,
0.5753575301062241,
0.4812527587386003,
0.5300830160193748,
0.5594560699987943,
0.4486082829315742,
0.4579606299821986,
0.4323131269803091,
0.4020481658999553,
0.5180339984328821,
0.4976098693315329,
0.5614202376942153,
0.5433716165239272,
0.5605711093272987,
0.595318519108694,
0.42416779210070904,
0.49869962802742684,
0.507829604061832,
0.6085403471214621,
0.45059590489603646,
0.4851957880754007,
0.43623517532803885,
0.6006227477552516,
0.5237048733092099,
0.7223651008715091,
0.5508299621111297,
0.47195243225557065,
0.5491879723111697,
0.5012411998355109,
0.5261024506936252,
0.46012254404717334,
0.4784093951087044,
0.4564321026172258,
0.4333098071191711,
0.48559709819968366,
0.4589175404280795,
0.7307533607570189,
0.5448090085991202,
0.3329021748699117,
0.5799730139025867,
0.5614311179565079,
0.30844719005199556,
0.3304273775552172,
0.4155776220777579,
0.5508967952577224,
0.46814721725259484,
0.5052942116360402,
0.5877682384088321,

```
0.5848666254043279,  
0.6108617556799849,  
0.46457267109406686,  
0.519906974170459,  
0.498734631981905,  
0.505973310643967,  
0.35925702411496985,  
0.3308405771923436,  
0.606734037324806,  
0.3783690795132647,  
0.5339320857345057,  
0.5935450014887813,  
0.3343045623606754,  
0.6745753728979176,  
0.4658750784825184,  
0.4450376099408852,  
0.6153047520461203,  
0.47039242510117374,  
0.5917760985890255,  
0.5363212489382024,  
0.6053232093920518,  
0.46869479717005014,  
0.5822969490633502,  
0.6917193782153774,  
0.47845226531629487,  
0.47377576632527685,  
0.6157262328676482,  
0.5677634969182028,  
0.6419341084315839,  
... (911 lines omitted) ...  
0.4526580600425577,  
0.5797380404798498,  
0.3544587730381279,  
0.5293835478493831,  
...]
```

```
plt.hist(num_10_1000000s, bins=28)  
plt.xlabel('xbar')  
plt.ylabel('count')  
plt.xlim(0,1)  
plt.show()
```



Create a Group of Charts Demonstrating a Normal Distribution Occuring w/ Increased Counts

- The distribution of the average is converging to a gaussian or normal distribution; bell curve.
- This is the basis for the **Central Limit Theorem**.
- The **Central Limit Theorem** tells us that the average is distributed normally, therefore, we can express the variability of the average using a statistic that represents the width of the bell curve.
- The variance is a statistic that describes variation or variability.

```

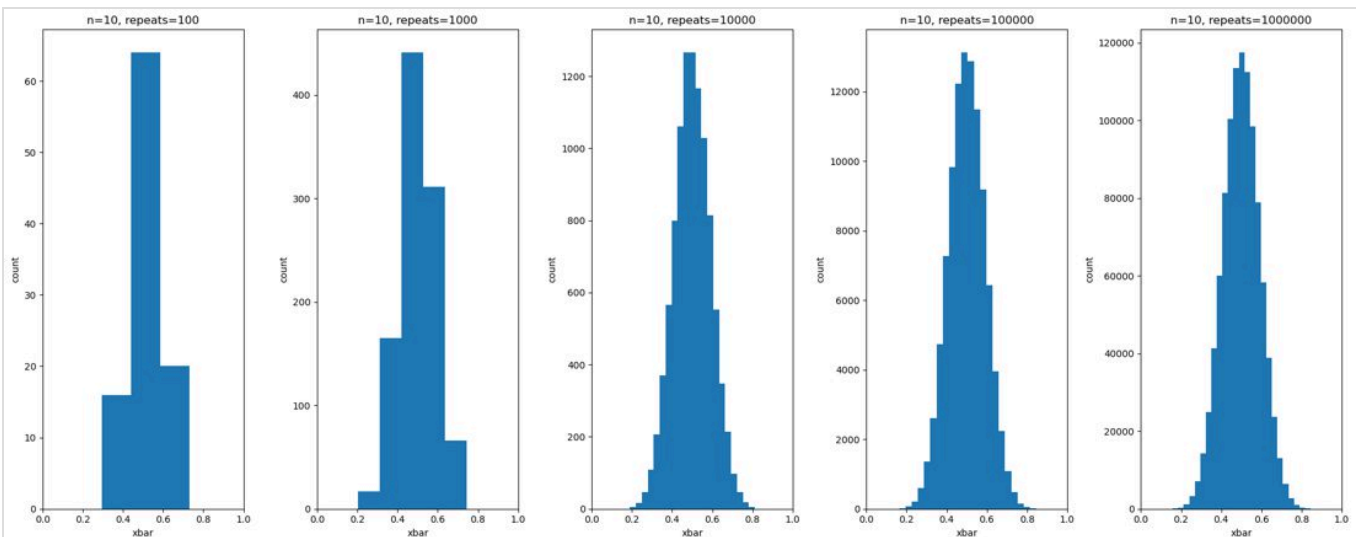
# Data preparation
if 'data_lists' not in locals():
    data_lists = [num_10_100s, num_10_1000s, num_10_10000s, num_10_100000s,
num_10_1000000s]
if 'labels' not in locals():
    labels = [100,
              1000,
              10000,
              100000,
              1000000]

# Create subplots
fig, axes = plt.subplots(1,
                        len(data_lists),
                        figsize=(20, 8))

# Iterate through data lists and axes to create histograms
for i, (ax, data, label) in enumerate(zip(axes,
                                          data_lists,
                                          labels)):

    if i == 0:
        bins = 3
    elif i == 1:
        bins = 5
    elif i == 2:
        bins = 21
    elif i == 3:
        bins = 24
    else:
        bins = 28
    ax.hist(data, bins=bins)
    ax.set_xlabel('xbar')
    ax.set_ylabel('count')
    ax.set_title('n=10, repeats={}'.format(label))
    ax.set_xlim(0, 1)
plt.tight_layout()
plt.show()

```



Central Limit Theorem, Variance, and Standard Deviation

Definition: Let $X_1, X_2, \dots, X_n$ be i.i.d. random variables with mean μ and variance $\sigma^2 < \infty$. The distribution of the sample mean $\bar{X}_n$ approaches a normal distribution as $n \rightarrow \infty$.

- Sample mean: $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$
- CLT (in distribution): $\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$

Equivalently: $\bar{X}_n \xrightarrow{d} \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$

Variance = the average of the squared deviations from the mean.

- Population variance: $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$
- Sample (unbiased) variance: $s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2$

Standard deviation = the square root of the variance.

- Population standard deviation: $\sigma = \sqrt{\sigma^2}$
- Sample standard deviation: $s = \sqrt{s^2}$

THE CENTRAL LIMIT THEOREM STATES THAT THE DISTRIBUTION OF THE SAMPLE MEAN OF A LARGE NUMBER OF INDEPENDENT, IDENTICALLY DISTRIBUTED RANDOM VARIABLES WITH FINITE MEAN AND VARIANCE APPROACHES A NORMAL DISTRIBUTION (WITH MEAN μ AND VARIANCE σ^2/n) AS THE SAMPLE SIZE n GROWS.

VARIANCE = THE AVERAGE OF THE SQUARED VERTICAL DISTANCES FROM EACH DATA POINT TO THE MEAN.

VARIANCE = THE AVERAGE OF THE SQUARED DEVIATIONS FROM THE MEAN.

STANDARD DEVIATION = THE SQUARE ROOT OF THE VARIANCE.

Let's Create a Variance Function

```
def sample_variance(data):  
    """  
    Calculate the sample variance of a list of numbers (Bessel's correction: divide by n-1).  
  
    Parameters:  
        data (list): A list of numbers.  
  
    Returns:  
        float: The sample variance of the numbers.  
    """  
    mean = sum(data) / len(data)  
    squared_deviation = [(x - mean) ** 2 for x in data]  
    return sum(squared_deviation) / (len(data) - 1)
```

Determine the Variance of the Average

```
sample_variance([5,  
                10,  
                15])
```

25.0

Let's Make a Function to Calculate the Sample Variance

```
def sample_sd(data):  
    """  
    Calculate the sample standard deviation of a list of numbers (using Bessel's  
    correction, n-1).  
  
    Parameters:  
        data (list): A list of numbers.  
  
    Returns:  
        float: The sample standard deviation of the numbers.  
    """  
    return sample_variance(data) ** 0.5
```

```
sample_sd([5,  
          10,  
          15])
```

5.0

Visualize How the Sample Variance and Sample Standard Deviation Work

```

# Data
data = [5,
        10,
        15]
mean = sum(data) / len(data)
n = len(data)
x = data

# Calculate squared deviations and SD
sq_diffs = [(val - mean)**2 for val in data]
variance = sum(sq_diffs) / (n - 1)
sample_sd_ = variance ** 0.5

# Plot points and mean
plt.figure(figsize=(12, 6))
plt.scatter(x,
            data,
            color='blue',
            s=120,
            label='Data Points')
plt.axhline(mean,
            color='red',
            linestyle='--',
            lw=2,
            label='Mean = {:.1f}'.format(mean))

# Draw deviation lines and annotate
for i, y in enumerate(data):
    plt.plot([x[i],
              x[i]],
             [mean, y],
             'k:', lw=1)
    diff = y - mean
    # Purple annotation
    plt.text(x[i],
             y+1 if y>mean else y-2,
             '({}-10)={:.1f}'.format(y, diff),
             color='purple',
             fontsize=8)
    # Orange annotation, just 0.5 units above/below the purple one
    plt.text(x[i],
             (y+1.5 if y>mean else y-2.5),
             '({}-10)^2={:.1f}'.format(y, diff**2),
             color='orange',
             fontsize=8)

# SD calculation steps
plt.text(10,
        4.5,
        'n = {}'.format(n),
        fontsize=13,
        color='black',
        ha='center')
plt.text(10,

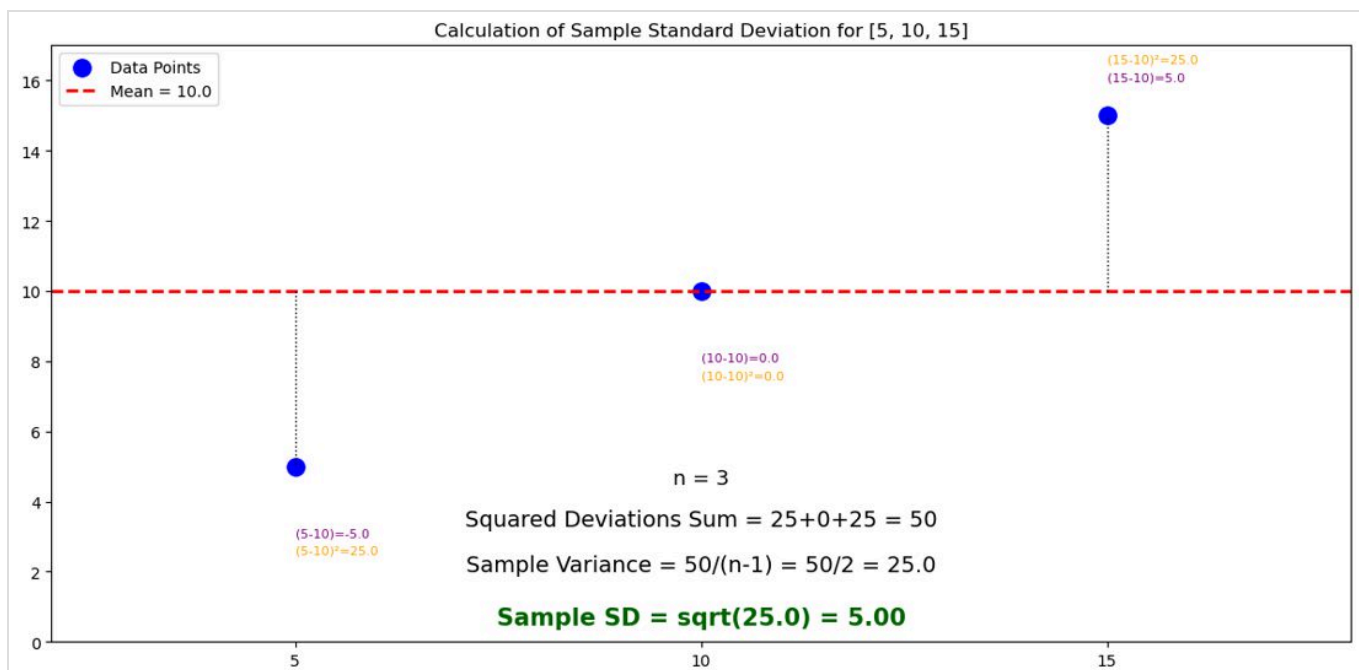
```

```

3.3,
'Squared Deviations Sum = {} = {}'.format("+".join(str(int(sd)) for sd in
sq_diffs),
int(sum(sq_diffs))),
color='black',
fontsize=14,
ha='center')
plt.text(10,
2.0,
'Sample Variance = {}/(n-1) = {}/2 = {:.1f}'.format(int(sum(sq_diffs)),
int(sum(sq_diffs)),
variance),
color='black',
fontsize=14,
ha='center')
plt.text(10,
0.5,
'Sample SD = sqrt({:.1f}) = {:.2f}'.format(variance, sample_sd_),
color='darkgreen',
fontsize=15,
fontweight='bold',
ha='center')

# Calculation of Sample Standard Deviation
plt.title('Calculation of Sample Standard Deviation for [5, 10, 15]')
plt.xticks(data,
[str(v) for v in data])
plt.xlim(2, 18)
plt.ylim(0, 17)
plt.legend(loc='upper left')
plt.grid(False)
plt.tight_layout()
plt.show()

```



```
sample_size_study = [3 * (2 ** ix) for ix in range(15)]
```

```
sample_size_study
```

```
[3, 6, 12, 24, 48, 96, 192, 384, 768, 1536, 3072, 6144, 12288, 24576, 49152]
```

Let's Replicate 2500 Times Calculating the Average from a Sample Size to Ensure a Normal Distribution for the Sample Average

```
sample_sd([1,  
            2,  
            3])
```

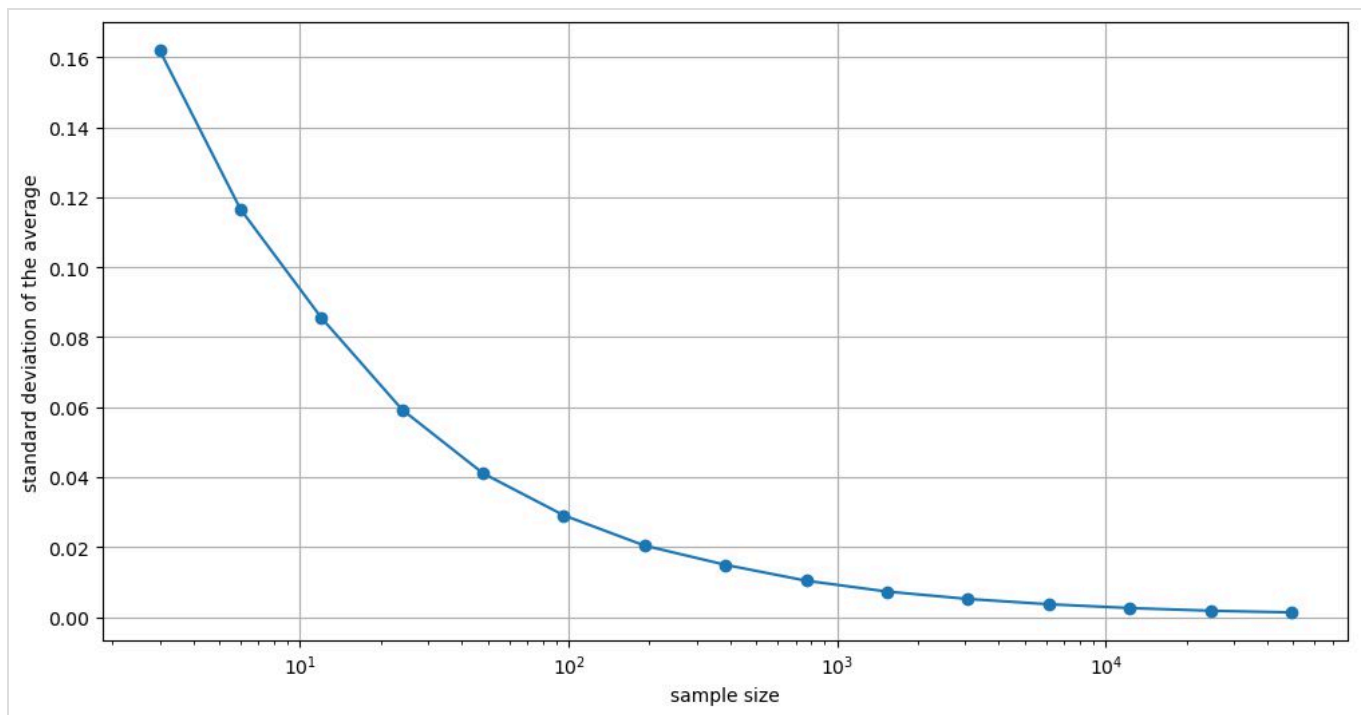
```
1.0
```

```
random.seed(42)
```

```
study_sd_vs_sample_size = [sample_sd([gen_samples_calc_avg(sample) for _ in range(2500)])  
                           for sample in sample_size_study]
```

Show the Standard Deviation of the Average wrt Sample Size

```
fig, ax = plt.subplots(figsize=(12, 6))  
ax.plot(sample_size_study, study_sd_vs_sample_size, '-o')  
ax.set_xscale('log')  
ax.set_xlabel('sample size')  
ax.set_ylabel('standard deviation of the average')  
ax.grid(True)  
plt.show()
```



The standard deviation on the average decreases as the sample size increases.

The standard deviation on the average is called the standard error on the mean or SEM.

SEM = (standard_deviation) / (square root of the number of samples)

$$SEM = \frac{\sigma}{\sqrt{N}}$$

The σ is the standard deviation of the measurements. We are currently using a random number generator that generates **uniform** random numbers. There is a known formula for the **variance** of a **uniform** distribution.

$$SEM = \frac{1}{\sqrt{12 \times N}}$$

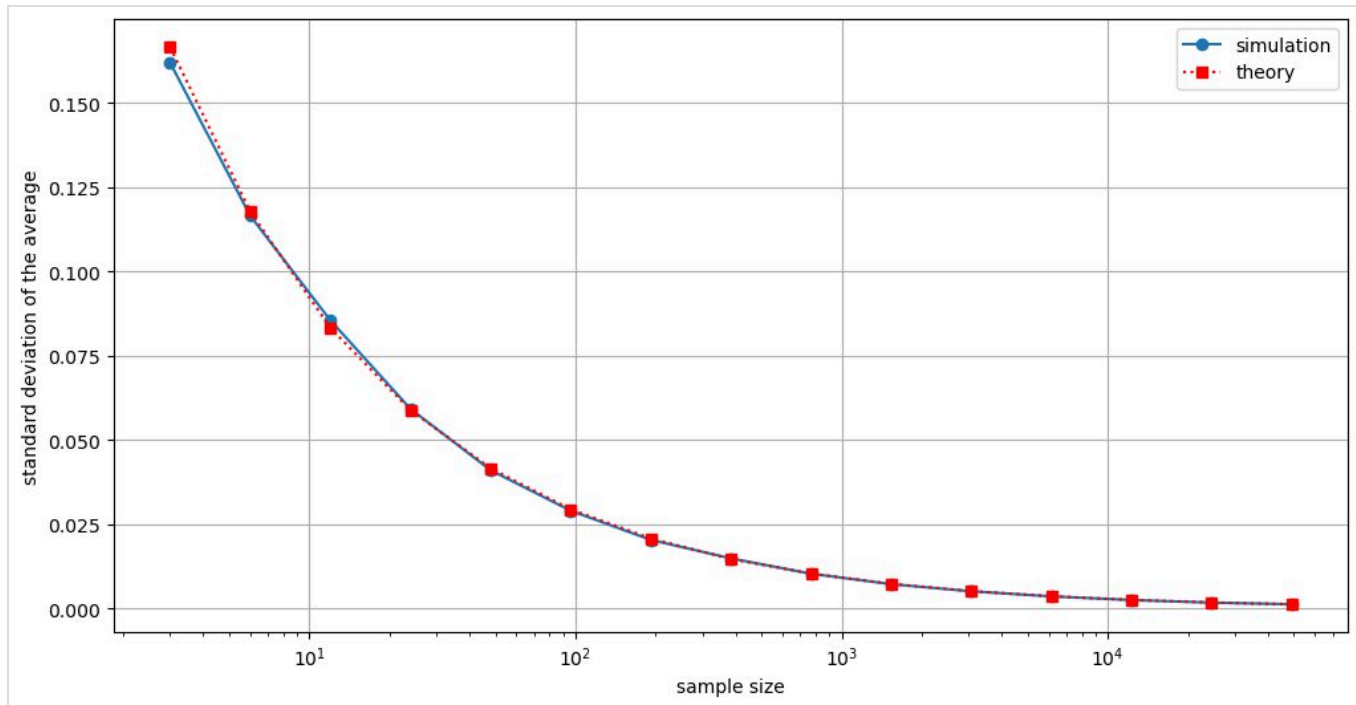
or

$$SEM = \left(\frac{1/12}{N} \right)^{0.5}$$

Let's use this formula and compare this theoretical to our simulation results...

```
sem_theory = [((1/12) / sample) ** 0.5 for sample in sample_size_study]
```

```
fig, ax = plt.subplots(figsize=(12, 6))
ax.plot( sample_size_study, study_sd_vs_sample_size, '-o', label='simulation')
ax.plot( sample_size_study, sem_theory, ':sr', label='theory')
ax.set_xscale('log')
ax.set_xlabel('sample size')
ax.set_ylabel('standard deviation of the average')
ax.grid(True)
ax.legend()
plt.show()
```



WE COLLECT DATA AND MORE OFTEN THAN NOT, WE HAVE RELATIVELY SMALL SAMPLE SIZES.

HOW CONFIDENT ARE WE ON AVERAGE BASED ON THESE SMALL SAMPLE SIZES REPRESENTING THE TOTAL POPULATION?

WE CAN EXPRESS CONFIDENCE USING THE CLT AND THE STANDARD ERROR.

The confidence interval represents where you anticipate the true population average to be, based on your sample data and the chosen confidence level.

The most common confidence interval is 95%.

Small Sample Set

```
random.seed(42)

small_sample = [random.random() for _ in range(10)]
```

```
small_sample
```

```
[0.6394267984578837,  
0.025010755222666936,  
0.27502931836911926,  
0.22321073814882275,  
0.7364712141640124,  
0.6766994874229113,  
0.8921795677048454,  
0.08693883262941615,  
0.4219218196852704,  
0.029797219438070344]
```

```
small_sample_avg = my_avg(small_sample)
```

```
small_sample_avg
```

```
0.4006685751243019
```

```
sample_sd_ = sample_sd(small_sample)
```

```
sample_sd_
```

```
0.3183924797690624
```

```
sem = sample_sd_ / (len(small_sample) ** 0.5)
```

```
sem
```

```
0.10068454259393185
```

```
conf_int = (small_sample_avg - 2 * sem,  
            small_sample_avg + 2 * sem)
```

```
conf_int
```

```
(0.19929948993643817, 0.6020376603121655)
```

```
small_sample_avg
```

```
0.4006685751243019
```

```
conf_int[1] - conf_int[0]
```

```
0.40273817037572734
```

Large Sample Set

```
random.seed(42)  
  
large_sample = [random.random() for _ in range(1000000)]
```

```
large_sample
```


[0.6394267984578837,
0.025010755222666936,
0.27502931836911926,
0.22321073814882275,
0.7364712141640124,
0.6766994874229113,
0.8921795677048454,
0.08693883262941615,
0.4219218196852704,
0.029797219438070344,
0.21863797480360336,
0.5053552881033624,
0.026535969683863625,
0.1988376506866485,
0.6498844377795232,
0.5449414806032167,
0.2204406220406967,
0.5892656838759087,
0.8094304566778266,
0.006498759678061017,
0.8058192518328079,
0.6981393949882269,
0.3402505165179919,
0.15547949981178155,
0.9572130722067812,
0.33659454511262676,
0.09274584338014791,
0.09671637683346401,
0.8474943663474598,
0.6037260313668911,
0.8071282732743802,
0.7297317866938179,
0.5362280914547007,
0.9731157639793706,
0.3785343772083535,
0.552040631273227,
0.8294046642529949,
0.6185197523642461,
0.8617069003107772,
0.577352145256762,
0.7045718362149235,
0.045824383655662215,
0.22789827565154686,
0.28938796360210717,
0.0797919769236275,
0.23279088636103018,
0.10100142940972912,
0.2779736031100921,
0.6356844442644002,
0.36483217897008424,
0.37018096711688264,
0.2095070307714877,
0.26697782204911336,
0.936654587712494,
0.6480353852465935,
0.6091310056669882,

```
0.171138648198097,  
0.7291267979503492,  
0.1634024937619284,  
0.3794554417576478,  
0.9895233506365952,  
0.6399997598540929,  
0.5569497437746462,  
0.6846142509898746,  
0.8428519201898096,  
0.7759999115462448,  
0.22904807196410437,  
0.03210024390403776,  
0.3154530480590819,  
0.26774087597570273,  
0.21098284358632646,  
0.9429097143350544,  
0.8763676264726689,  
0.3146778807984779,  
0.65543866529488,  
0.39563190106066426,  
0.9145475897405435,  
0.4588518525873988,  
0.26488016649805246,  
0.24662750769398345,  
0.5613681341631508,  
0.26274160852293527,  
0.5845859902235405,  
0.897822883602477,  
0.39940050514039727,  
... (911 lines omitted) ...  
0.3705299587344354,  
0.44338308606070165,  
0.950555169851427,  
0.8554501933059546,  
...]
```

```
large_sample_avg = my_avg(large_sample)
```

```
large_sample_avg
```

```
0.5000145439034674
```

```
sample_sd_ = sample_sd(large_sample)
```

```
sample_sd_
```

```
0.28849403201032836
```

```
sem = sample_sd_ / (len(large_sample) ** 0.5)
```

```
sem
```

```
0.00028849403201032834
```

```
conf_int = (large_sample_avg - 2 * sem,  
            large_sample_avg + 2 * sem)
```

```
conf_int
```

```
(0.49943755583944677, 0.5005915319674881)
```

```
conf_int[1] - conf_int[0]
```

```
0.0011539761280413008
```

01-EDA: Penguins Dataset - EDA

Penguins Dataset

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g   2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm  54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Save the cleaned dataset for use in other project notebooks.

```
df_clean.to_csv('penguins-clean.csv', index=False)
```

Convert Data: Long Format

Categorical Names

```
df_clean_category_names = df_clean.select_dtypes(['object',
                                                  'category']). \
    columns.tolist()
```

```
df_clean_category_names
```

```
['species', 'island', 'sex']
```

Numeric Names

```
df_clean_numeric_names = df_clean.select_dtypes('number'). \
    columns.tolist()
```

```
df_clean_numeric_names
```

```
['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
```

Reshape

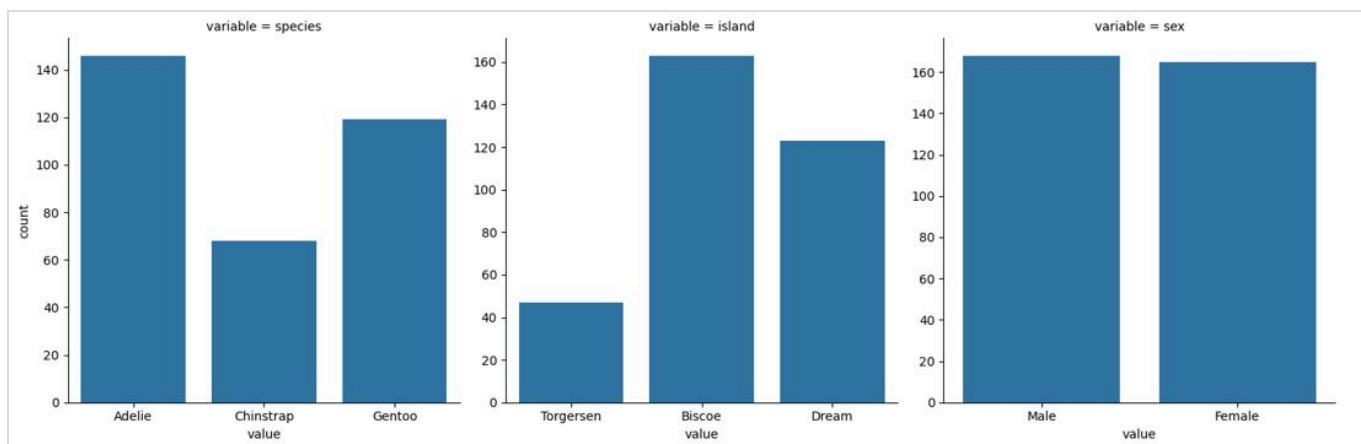
```
df_clean_lf = df_clean.reset_index(). \
    rename(columns={'index': 'rowid'}). \
    melt(id_vars=['rowid'] + df_clean_category_names,
        value_vars=df_clean_numeric_names)
```

Visually Explore Data

Count Plots – Marginal Distributions of Each Categorical Variable, with Facets for Each Categorical Variable

The data reveals a roughly equal distribution of male and female penguins, with Adelie being the most common species and Biscoe the most represented island, while Chinstrap and Torgersen are the least frequent species and island, respectively.

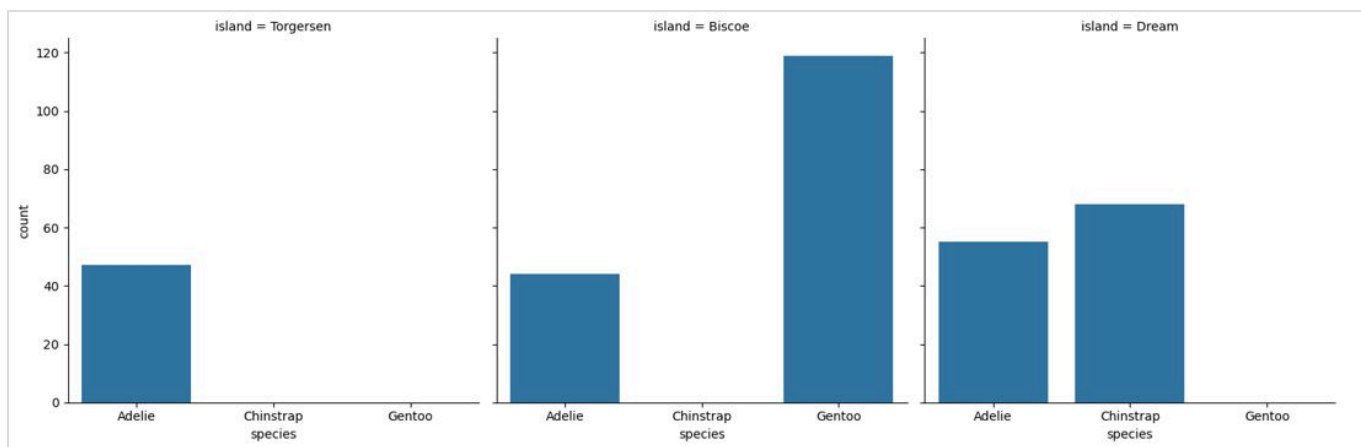
```
sns.catplot(data=df_clean.melt(value_vars=df_clean_category_names),
            x='value',
            col='variable',
            col_wrap=3,
            kind='count',
            sharex=False,
            sharey=False)
plt.show()
```



Count Plots – Marginal Distributions of the species Categorical Variable, with Facets for Each island Categorical Variable

Gentoo penguins are found exclusively on Biscoe island, Chinstrap only on Dream island, while Adelie is present on all three islands.

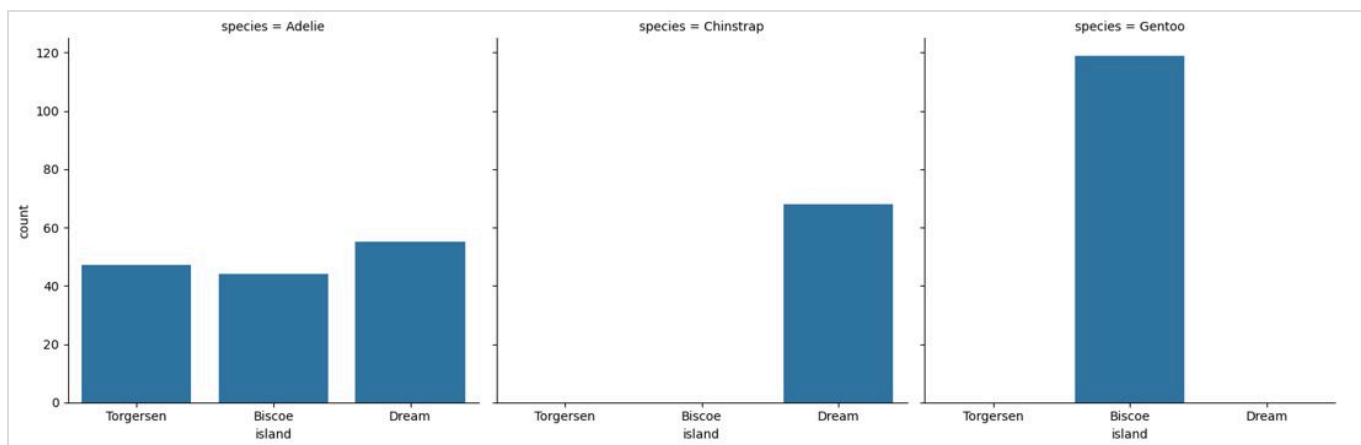
```
sns.catplot(data=df_clean,
            x='species',
            col='island',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `island` Categorical Variable, with Facets for Each species Categorical Variable

Adelie penguins are evenly distributed across all three islands, Chinstrap is found only on Dream island, and Gentoo is present exclusively on Biscoe island.

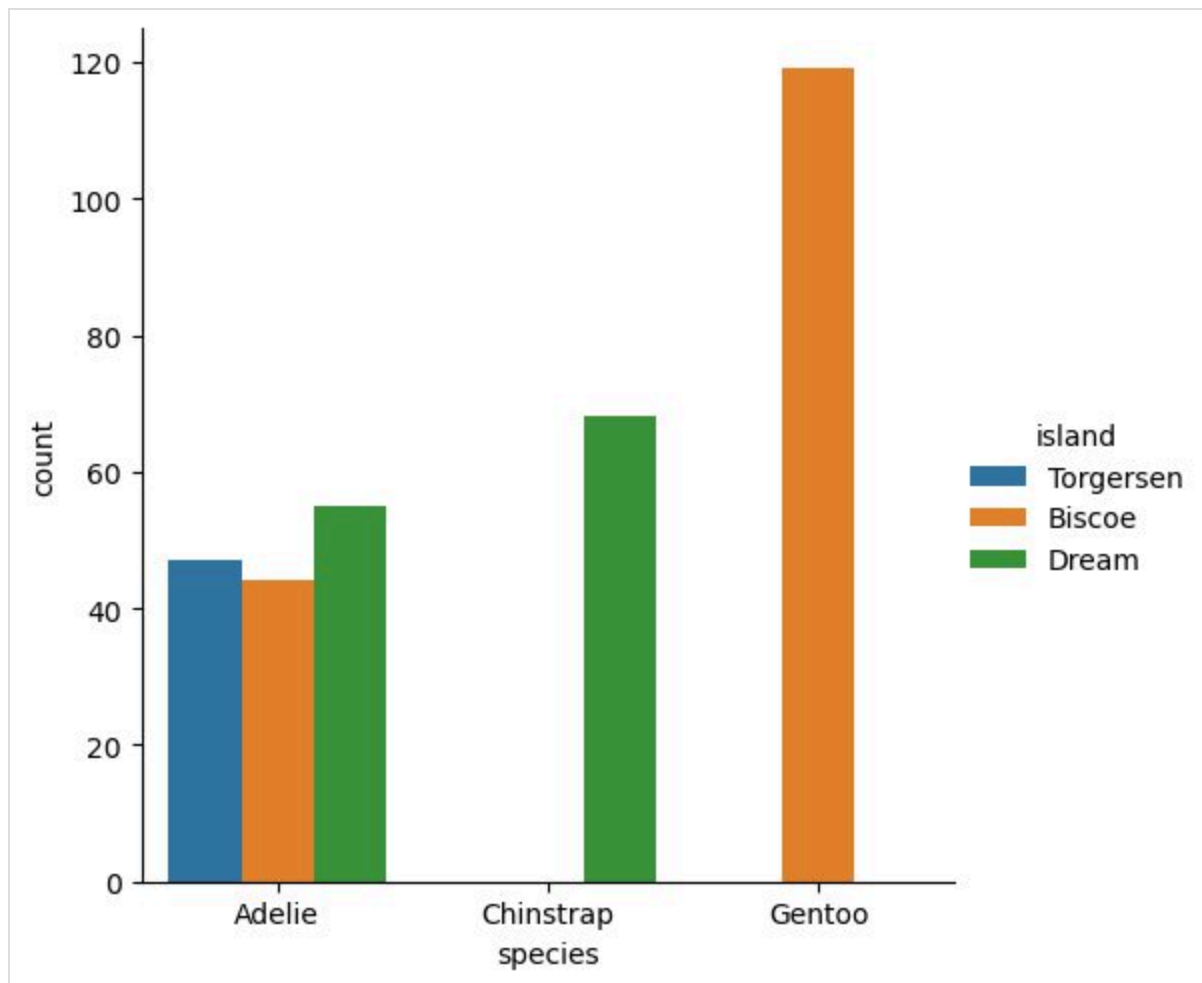
```
sns.catplot(data=df_clean,
            x='island',
            col='species',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, Colored by Each `island` Categorical Variable

Adelie penguins are distributed across all islands, Chinstrap is exclusive to Dream island, and Gentoo is found only on Biscoe island.

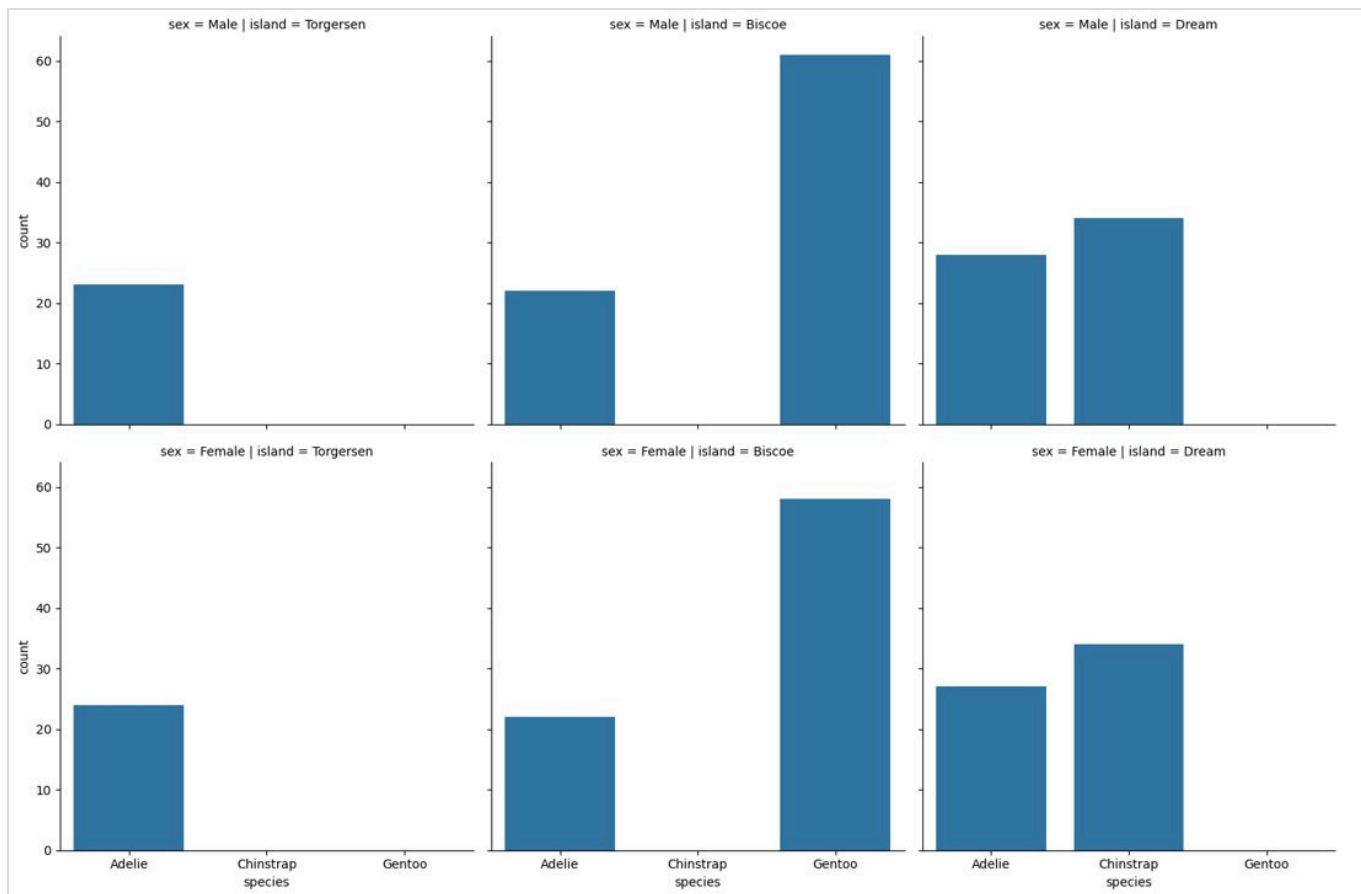
```
sns.catplot(data=df_clean,
            x='species',
            hue='island',
            kind='count')
plt.show()
```

Count Plots – Marginal Distributions of the `species` Categorical Variable, with Facets for Each Combination of `sex` and `island` Categorical Variables

Across all islands, Adelie penguins are evenly distributed between sexes, Chinstrap is found only on Dream island with more females than males, and Gentoo is exclusive to Biscoe island, where males are slightly more numerous than females.

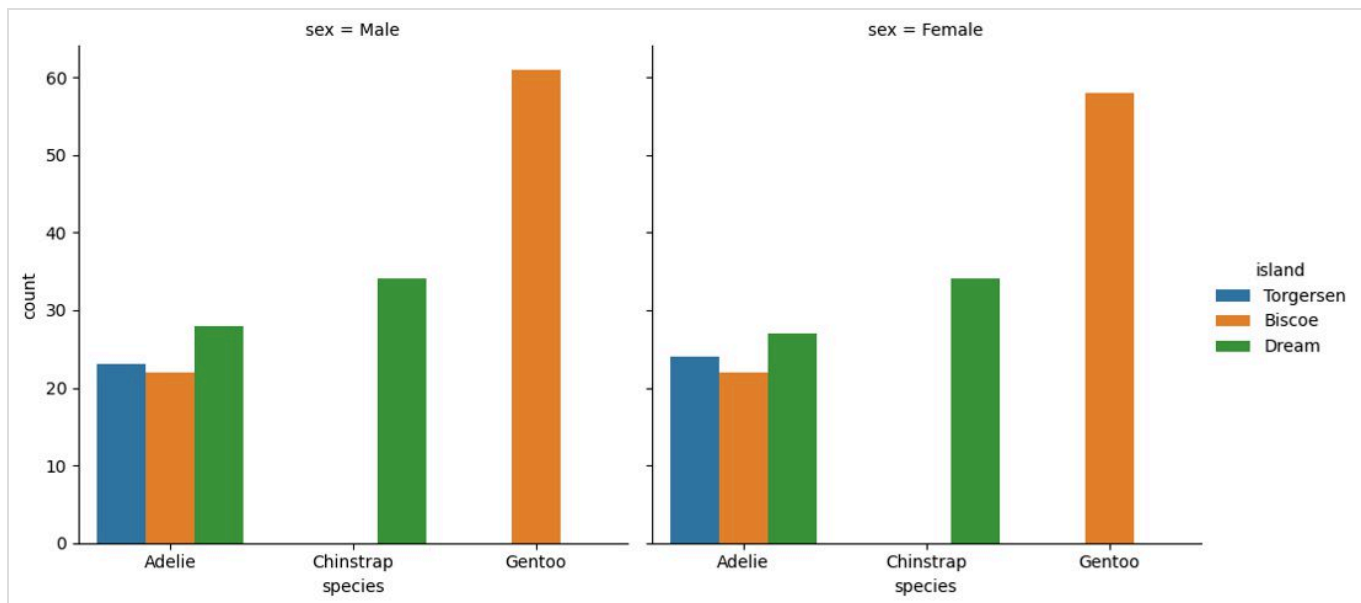
```
sns.catplot(data=df_clean,
            x='species',
            row='sex',
            col='island',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the species Categorical Variable, Colored by Each island Categorical Variable, with Facets for Each sex Categorical Variable

Both males and females show a similar distribution: Adelie penguins are present on all islands, Chinstrap is exclusive to Dream island, and Gentoo is found only on Biscoe island.

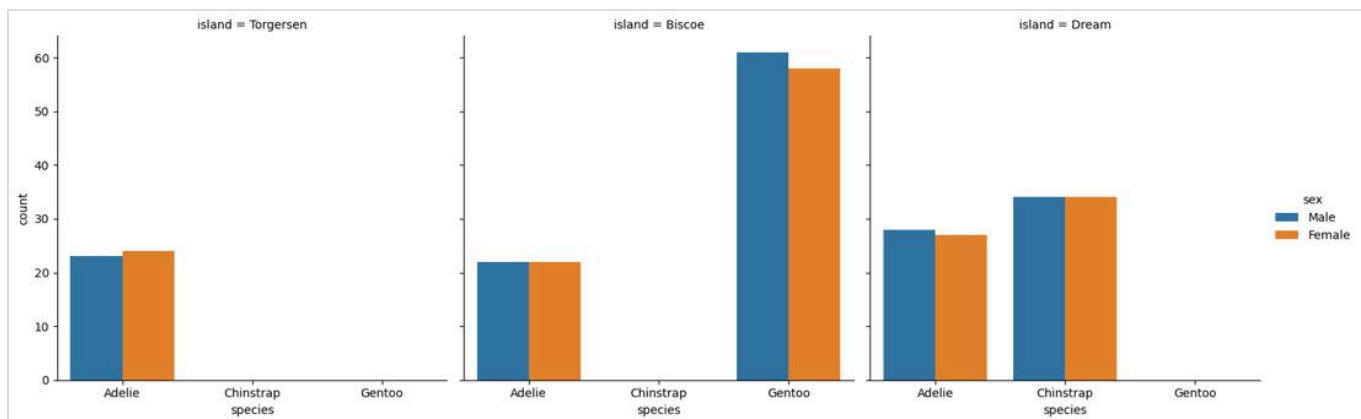
```
sns.catplot(data=df_clean,
            x='species',
            hue='island',
            col='sex',
            kind='count')
plt.show()
```



Count Plots – Marginal Distributions of the `species` Categorical Variable, Colored by Each `sex` Categorical Variable, with Facets for Each `island` Categorical Variable

On each island, Adelie penguins are evenly split between sexes, Gentoo is exclusive to Biscoe with similar male and female counts, and Chinstrap is found only on Dream island with equal representation of both sexes.

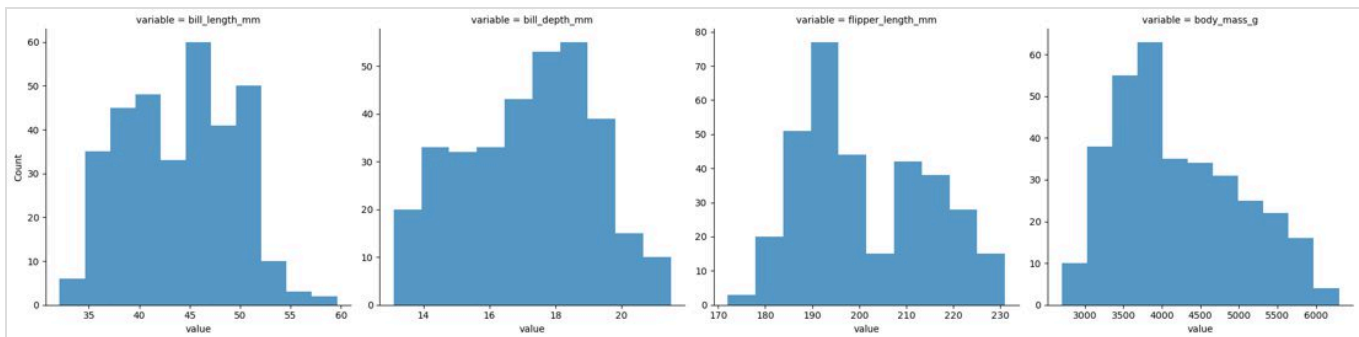
```
sns.catplot(data=df_clean,
            x='species',
            hue='sex',
            col='island',
            kind='count')
plt.show()
```



Histogram Plots – Marginal Distributions of Each Numerical Variable, with Facets for Each Numerical Variable

The distributions of bill length, bill depth, flipper length, and body mass are all approximately unimodal and show moderate variation, with most measurements concentrated around their respective central values.

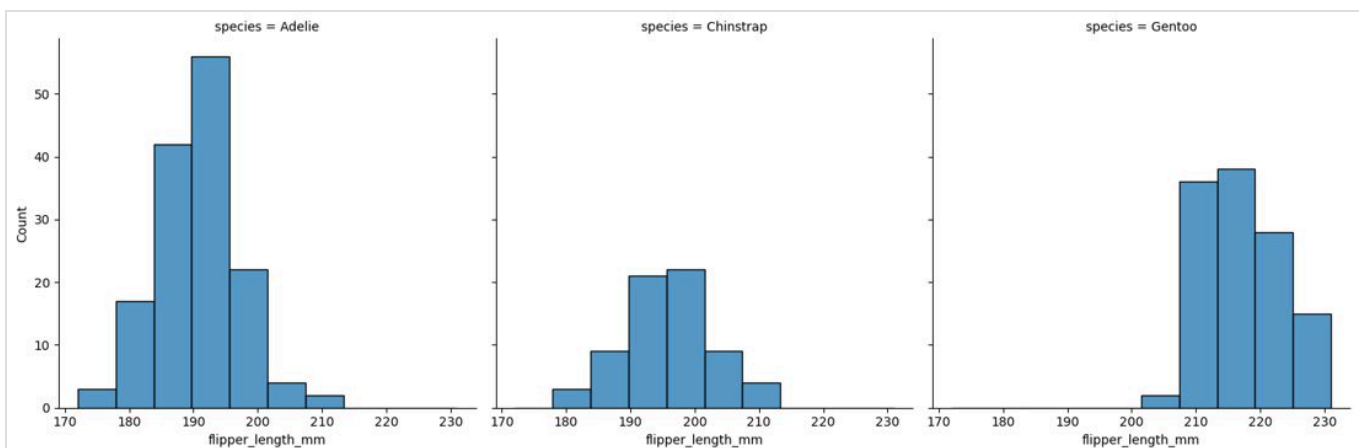
```
sns.displot(data=df_clean_lf,
            x='value',
            col='variable',
            col_wrap=4,
            kind='hist',
            facet_kws={'sharex': False,
                      'sharey': False},
            common_bins=False)
plt.show()
```



Histogram Plots – Marginal Distributions of the `flipper_length_mm` Continuous Variable, with Facets for Each `species` Categorical Variable

Adelie and Chinstrap penguins have overlapping flipper length distributions centered around 190–200 mm, while Gentoo exhibits distinctly longer flipper lengths, mostly between 210 and 230 mm.

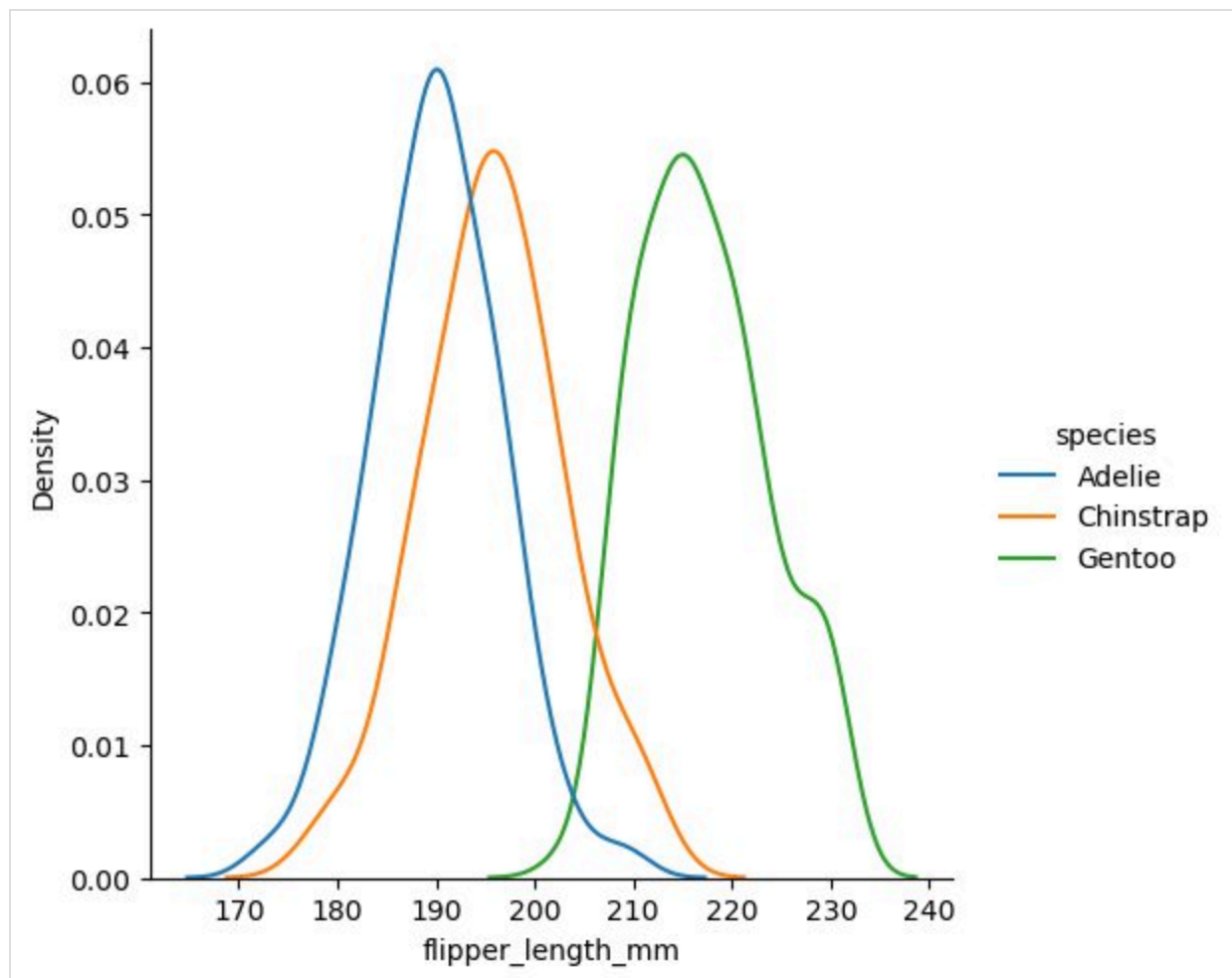
```
sns.displot(data=df_clean,
            x='flipper_length_mm',
            col='species',
            kind='hist')
plt.show()
```



KDE Plot – Marginal Distribution Density of the `flipper_length_mm` Continuous Variable, Colored by Each `species` Categorical Variable

The density plot shows that Adelie and Chinstrap penguins have overlapping flipper length distributions centered around 190–200 mm, while Gentoo exhibits distinctly longer flipper lengths, mostly between 210 and 230 mm.

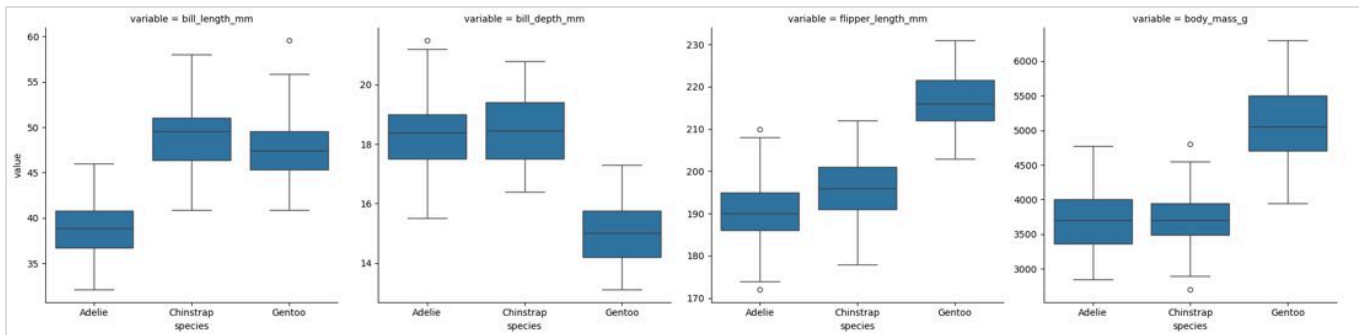
```
sns.displot(data=df_clean,
            x='flipper_length_mm',
            hue='species',
            common_norm=False,
            kind='kde')
plt.show()
```

**Box Plots – Distribution of Each Numerical Variable by `species`, Faceted by the Respective Numerical Variable**

Gentoo penguins show higher values for flipper length and body mass, Chinstrap tend to have the longest and deepest bills, while Adelie generally have the smallest measurements across all variables.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            kind='box',
            sharey=False)

plt.show()
```

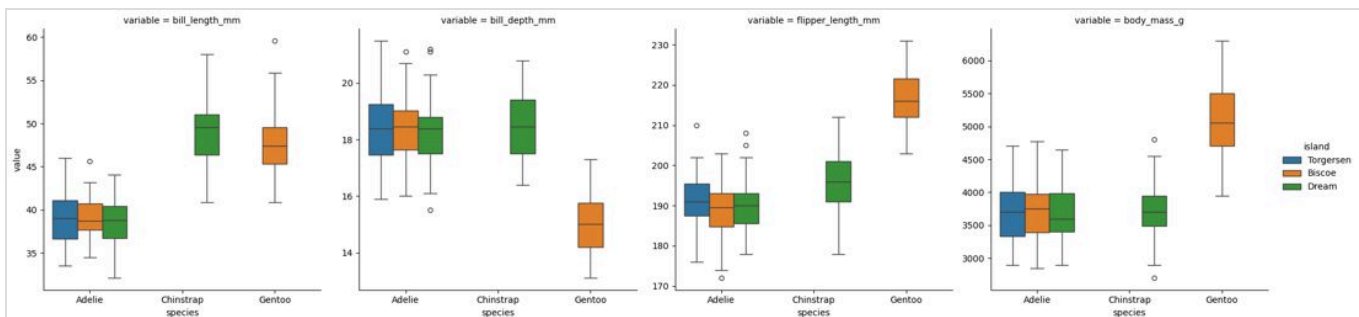


Box Plots – Distribution of Each Numerical Variable by species , Colored by island Categorical Variable, Faceted by the Respective Numerical Variable

Adelie penguins show similar measurements across all islands, Chinstrap are present only on Dream with relatively larger bills, while Gentoo is exclusive to Biscoe and stands out for its greater flipper length and body mass.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='island',
            kind='box',
            sharey=False)

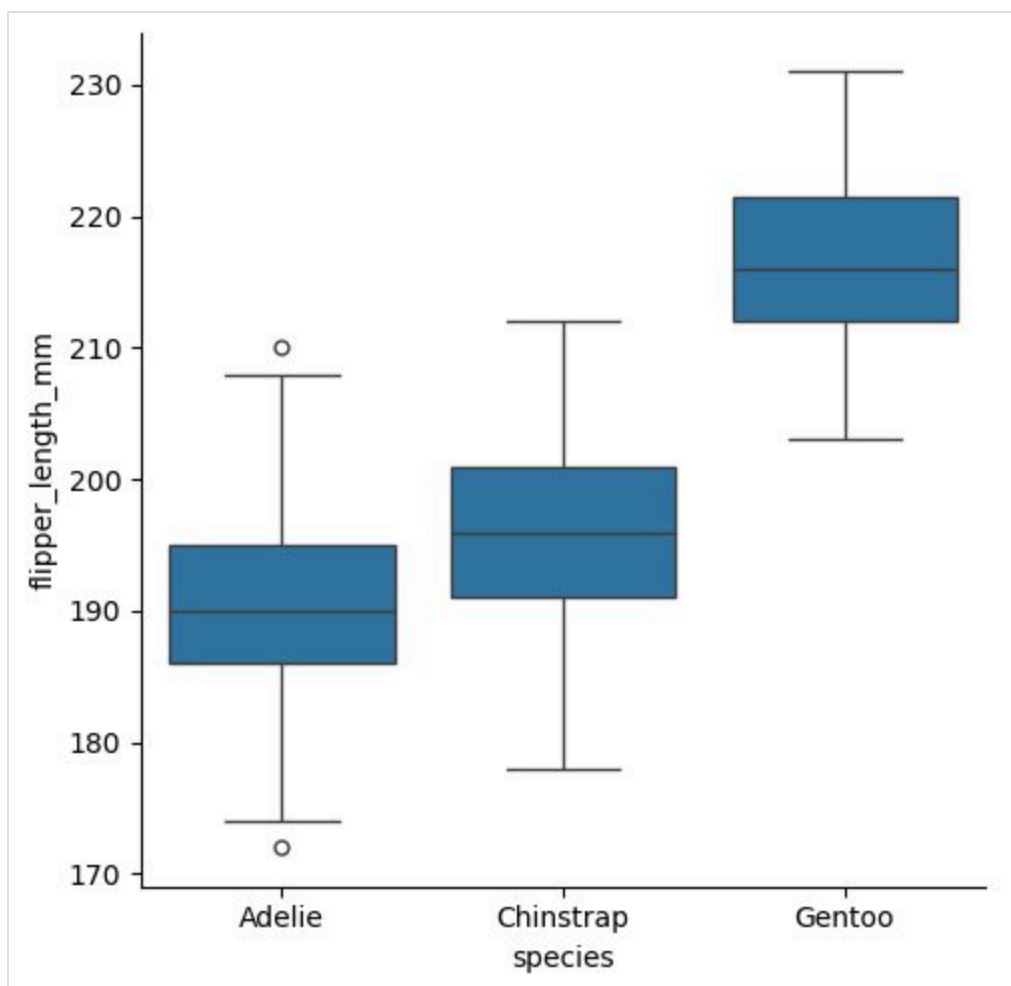
plt.show()
```



Box Plot – Marginal Distributions of the `flipper_length_mm` Continuous Variable, Grouped by Each `species` Categorical Variable

Gentoo penguins have the longest flipper lengths, followed by Chinstrap, and then Adelie. The boxplots show clear separation in flipper length among the three species, with Gentoo consistently measuring higher.

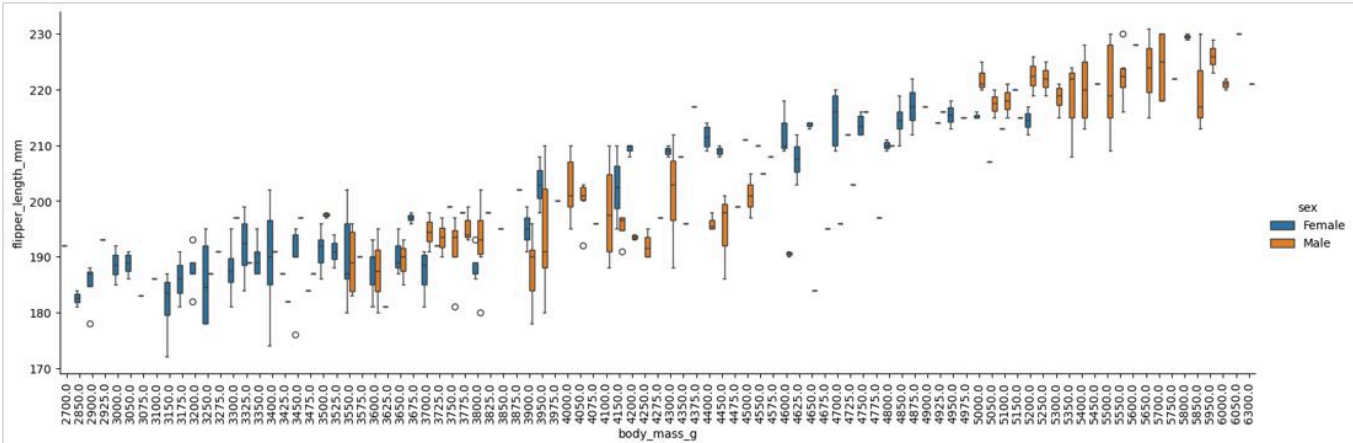
```
sns.catplot(data=df_clean,
            x='species',
            y='flipper_length_mm',
            kind='box')
plt.show()
```



Box Plot – Distribution of the `flipper_length_mm` Continuous Variable Across `body_mass_g` Continuous Variable, Colored by Each `sex` Categorical Variable

The plot shows that as penguins' body mass increases, their flipper length also tends to increase. This trend is consistent for both male and female penguins, with heavier penguins generally having longer flippers.

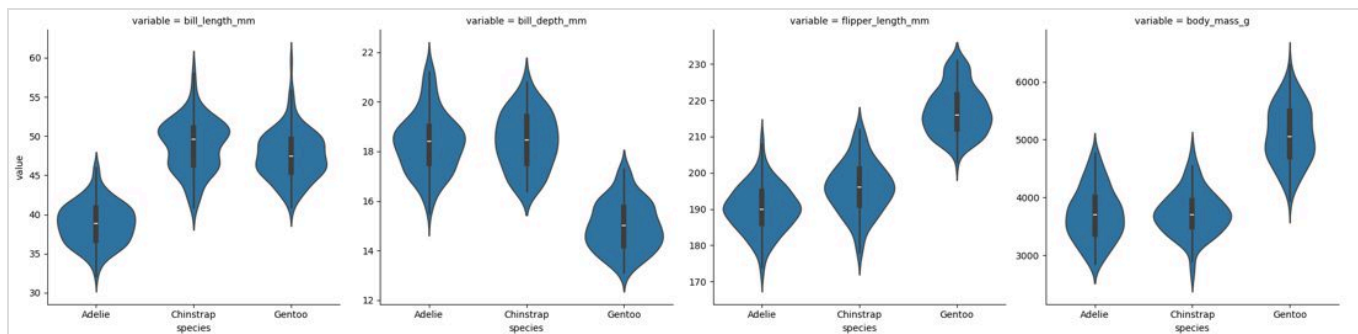
```
sns.catplot(data=df_clean,
            x='body_mass_g',
            y='flipper_length_mm',
            hue='sex',
            kind='box',
            aspect=3)
plt.xticks(rotation=90)
plt.show()
```



Violin Plots – Distribution Density of Each Numerical Variable by species , Faceted by the Respective Numerical Variable

Chinstrap and Gentoo penguins have longer bills compared to Adelie penguins, with Chinstrap showing the greatest bill depth. Gentoo penguins stand out for having the longest flipper lengths and the highest body mass, while Adelie penguins have the smallest measurements across all variables.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            kind='violin',
            sharey=False)
plt.show()
```

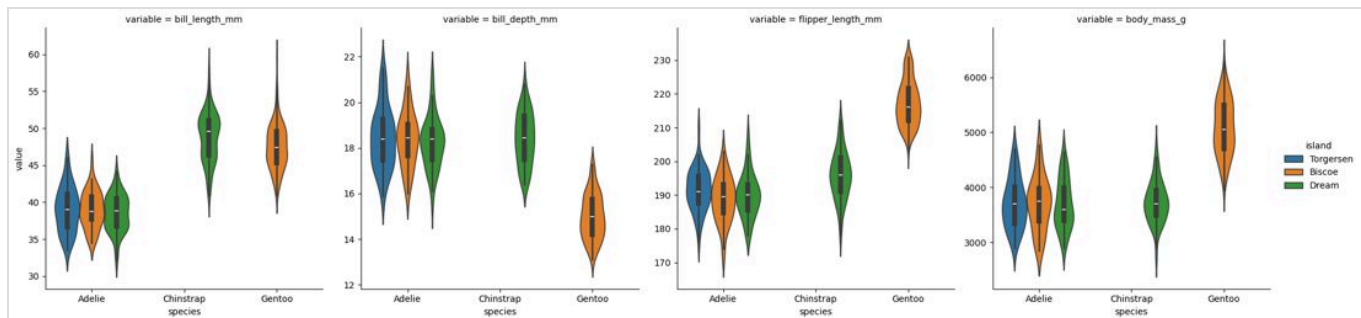


Violin Plots – Distribution Density of Each Numerical Variable by species, Colored by island, Faceted by the Respective Numerical Variable

Adelie penguins have similar measurements across all islands. Chinstrap penguins, only found on Dream, have greater bill length and depth compared to Adelie. Gentoo penguins, exclusive to Biscoe, stand out for their larger flipper length and body mass. This visualization highlights clear differences in physical traits among species and islands.

```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='island',
            kind='violin',
            sharey=False)

plt.show()
```

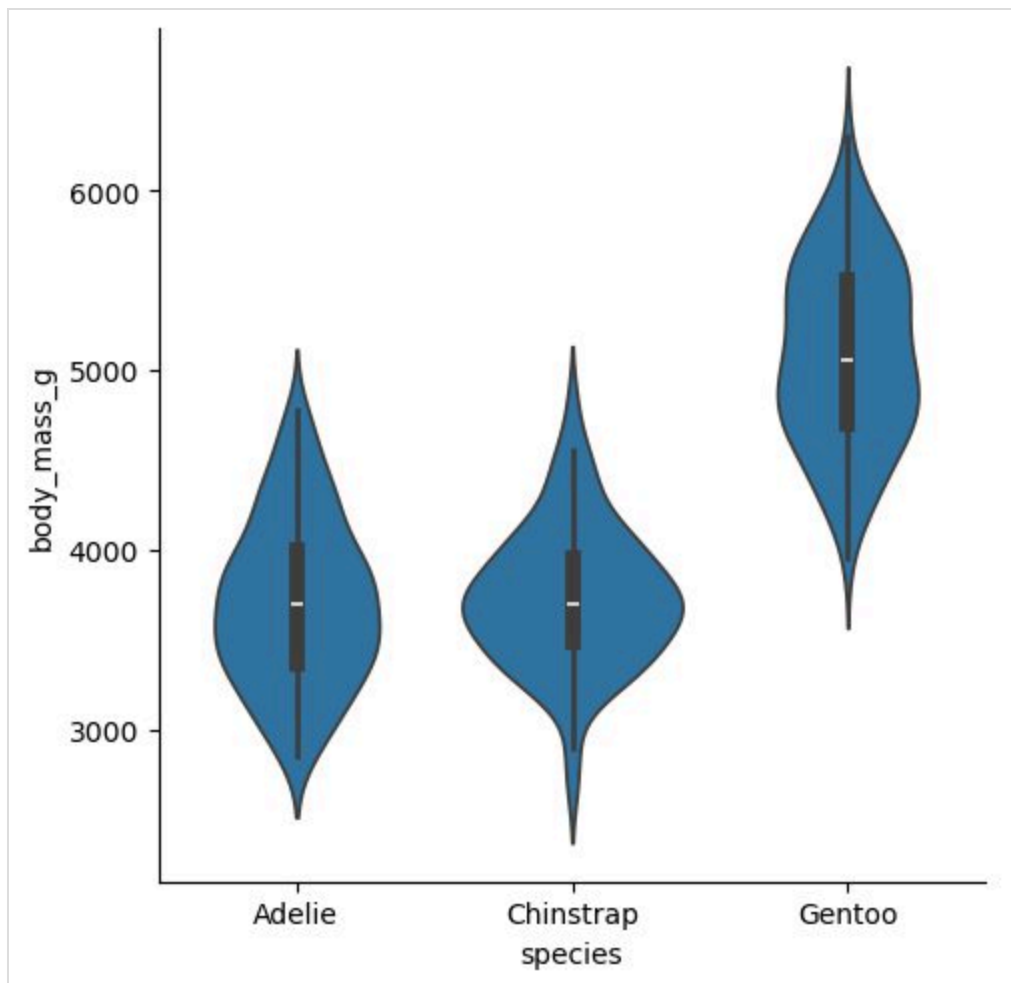


Violin Plot – Marginal Distribution Density of the body_mass_g Continuous Variable, Grouped by Each species Categorical Variable

Gentoo penguins have the highest body mass, followed by Chinstrap, while Adelie penguins have the lowest body mass. Gentoo penguins also display greater variation in body mass compared to the other species.

```
sns.catplot(data=df_clean,
            x='species',
            y='body_mass_g',
            kind='violin')

plt.show()
```

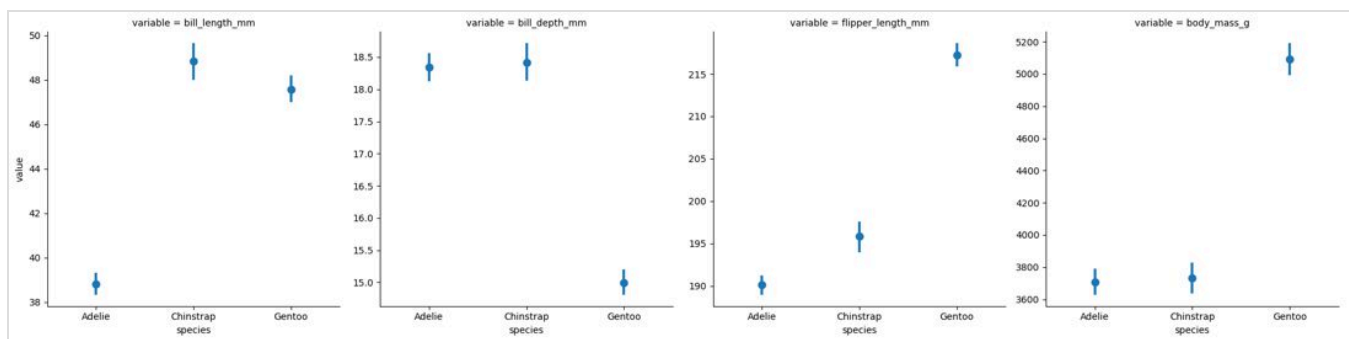


Point Plots – Mean and Confidence Interval of Each Numerical Variable by species , Faceted by the Respective Numerical Variable

Chinstrap penguins have the longest bills and greatest bill depth, Gentoo penguins have the longest flipper length and highest body mass, while Adelie penguins have the smallest measurements for all variables. Error bars indicate the variability around the mean values for each species.

```
sns.catplot(data=df_clean_lf,  
            x='species',  
            y='value',  
            col='variable',  
            col_wrap=4,  
            kind='point',  
            sharey=False,  
            linestyle='none')  
plt.show()
```

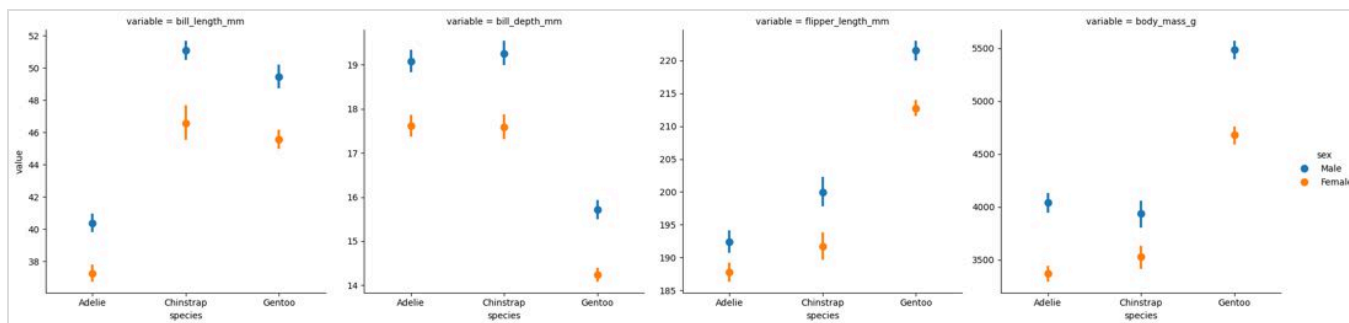
REVERSE ENGINEERING DATA



Point Plots – Mean and Confidence Interval of Each Numerical Variable by species , Colored by sex Categorical Variable, Faceted by the Respective Numerical Variable

For all species, male penguins have higher measurements than female penguins across bill length, bill depth, flipper length, and body mass. Chinstrap males have the largest bills, Gentoo males have the longest flippers and greatest body mass, while Adelie females have the lowest values for all traits. The differences between sexes are consistent and pronounced for each variable.

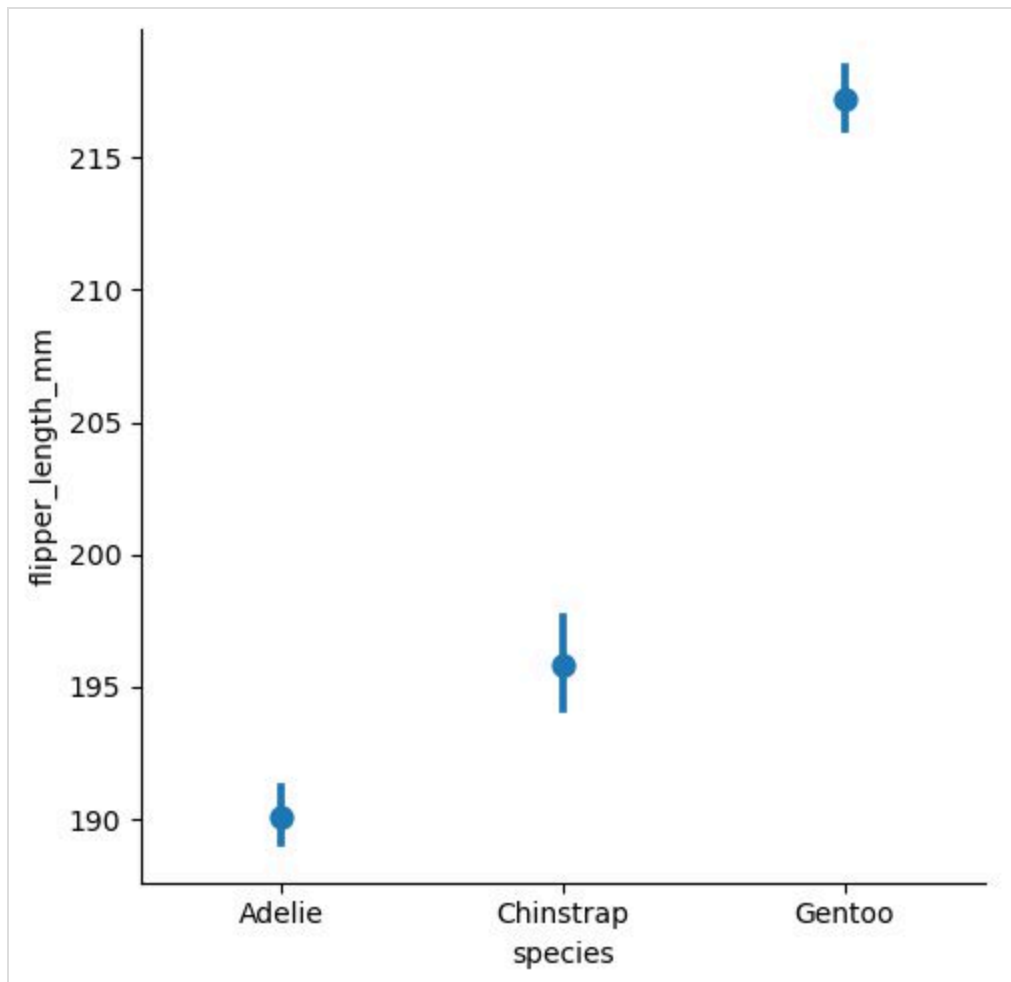
```
sns.catplot(data=df_clean_lf,
            x='species',
            y='value',
            col='variable',
            col_wrap=4,
            hue='sex',
            kind='point',
            sharey=False,
            linestyle='none')
plt.show()
```



Point Plot – Mean and Confidence Interval of the flipper_length_mm Continuous Variable, Grouped by Each species Categorical Variable

Gentoo penguins have the longest average flipper length, followed by Chinstrap penguins, with Adelie penguins having the shortest flippers. The error bars indicate the variability around the mean for each species.

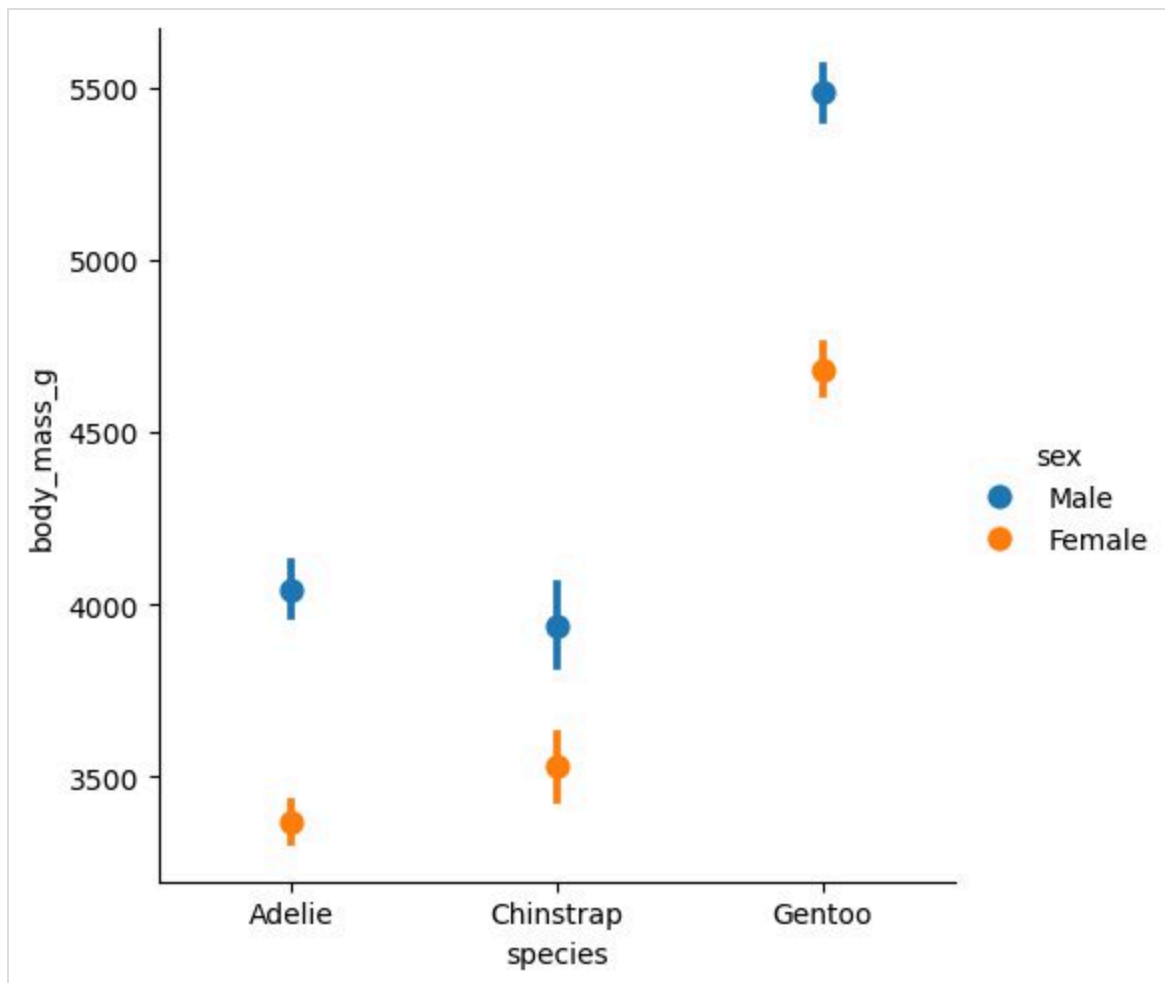
```
sns.catplot(data=df_clean,
            x='species',
            y='flipper_length_mm',
            kind='point',
            linestyle='none')
plt.show()
```



Point Plot – Mean and Confidence Interval of the `body_mass_g` Continuous Variable, Grouped by Each Combination of `species` Categorical Variable, Colored by `sex` Categorical Variable

For each species, male penguins have a higher average body mass than female penguins. Gentoo penguins are the heaviest, followed by Chinstrap and then Adelie. The differences between males and females are consistent across species, with males weighing more.

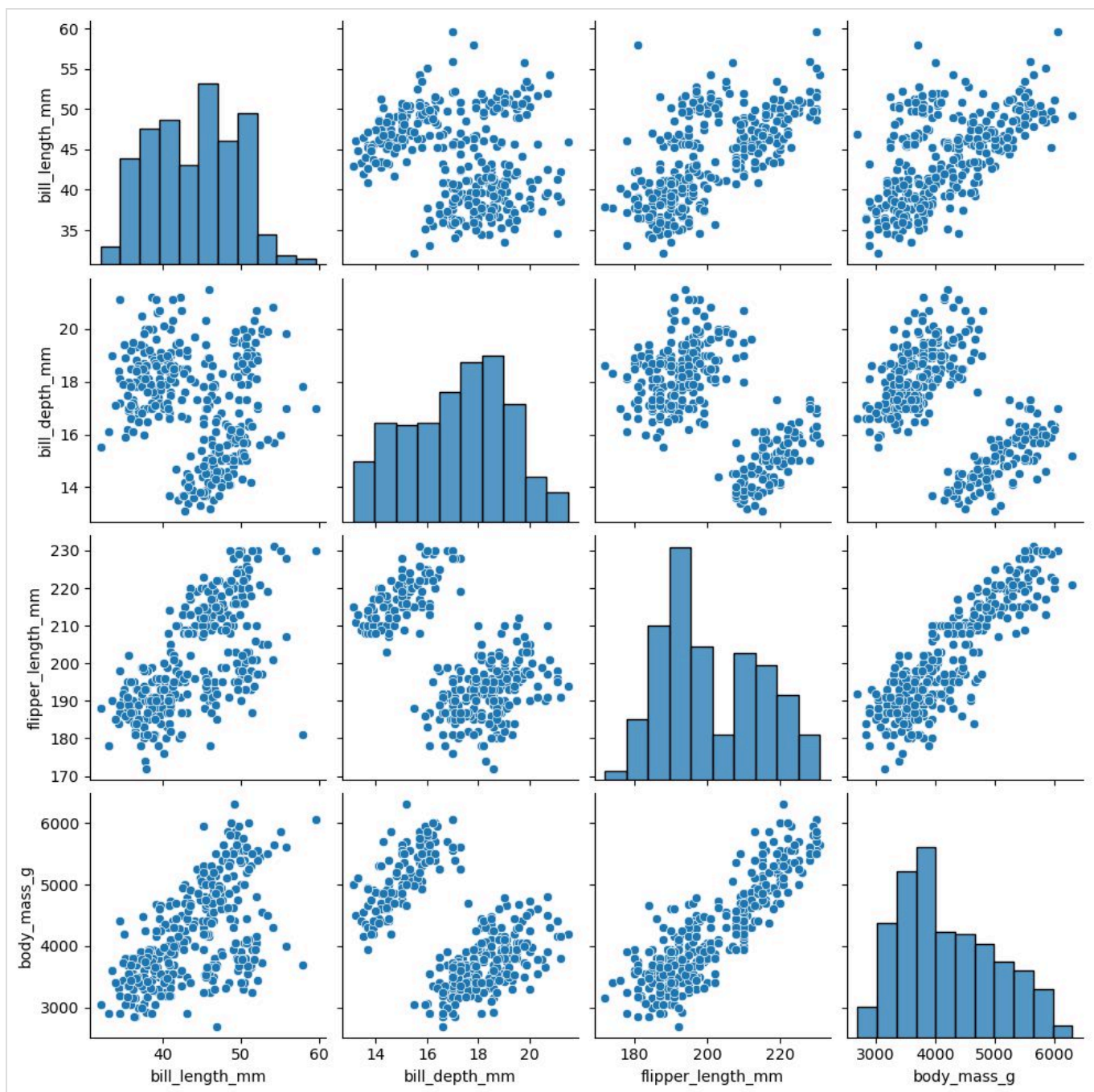
```
sns.catplot(data=df_clean,
            x='species',
            y='body_mass_g',
            hue='sex',
            kind='point',
            linestyle='none')
plt.show()
```



Pair Plots – Pairwise Relationships of All Numerical Variables

There are clear positive relationships among bill length, flipper length, and body mass in penguins—penguins with longer bills tend to have longer flippers and greater body mass. Bill depth displays less pronounced correlation with the other variables. Each variable also shows its own distribution along the diagonal, with flipper length and body mass having wider ranges compared to bill measurements.

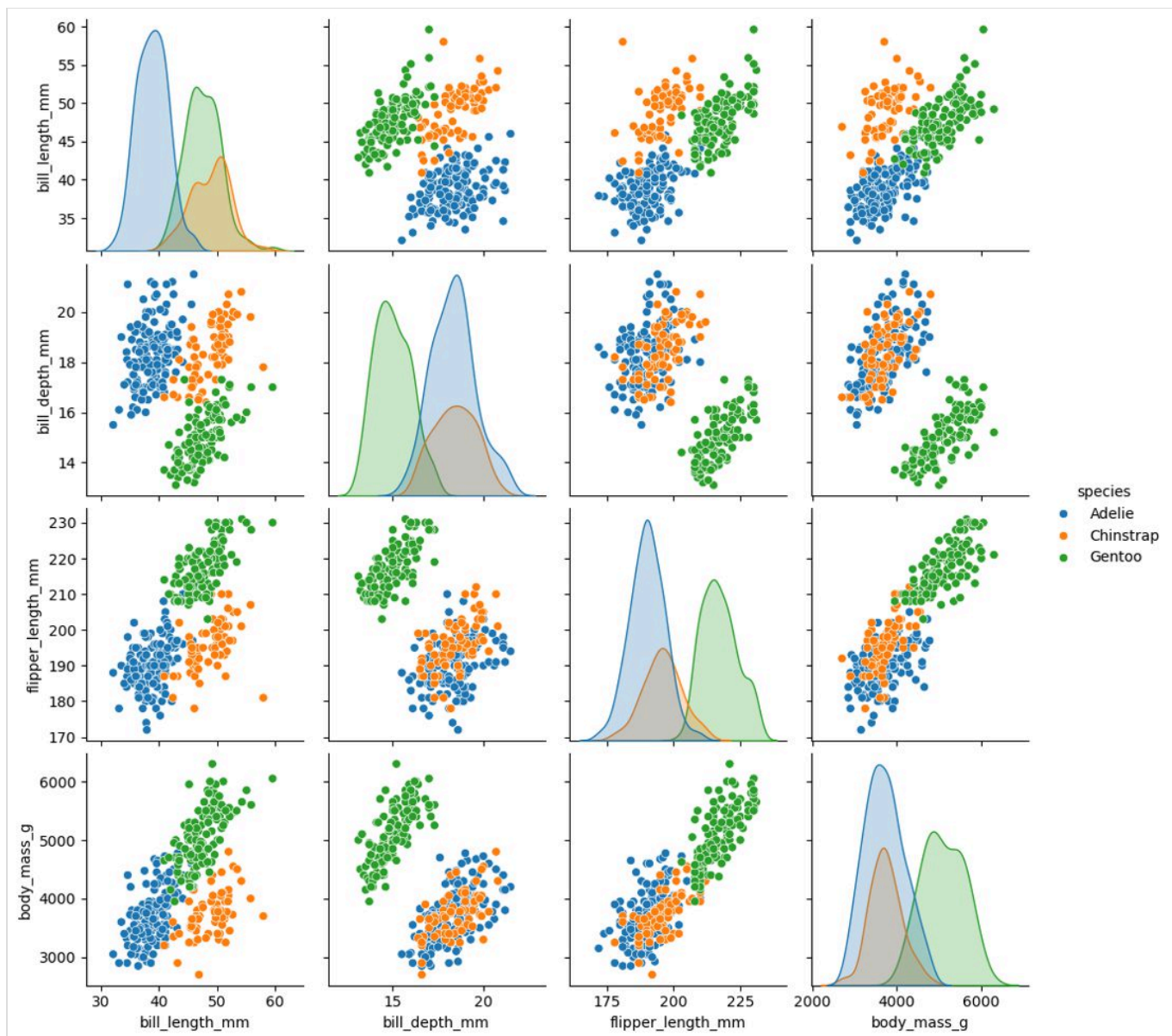
```
sns.pairplot(df_clean)
plt.show()
```



Pair Plots – Pairwise Relationships of All Numerical Variables, Colored by species Categorical Variable

There are clear distinctions among Adelie, Chinstrap, and Gentoo penguins for all measured variables. Gentoo penguins generally have the largest bill length, flipper length, and body mass, while Adelie penguins have the smallest. Chinstrap penguins typically have the greatest bill depth. The scatter plots reveal strong positive relationships between flipper length and body mass, as well as between bill length and flipper length. Each species forms distinct clusters, highlighting the differences in physical traits.

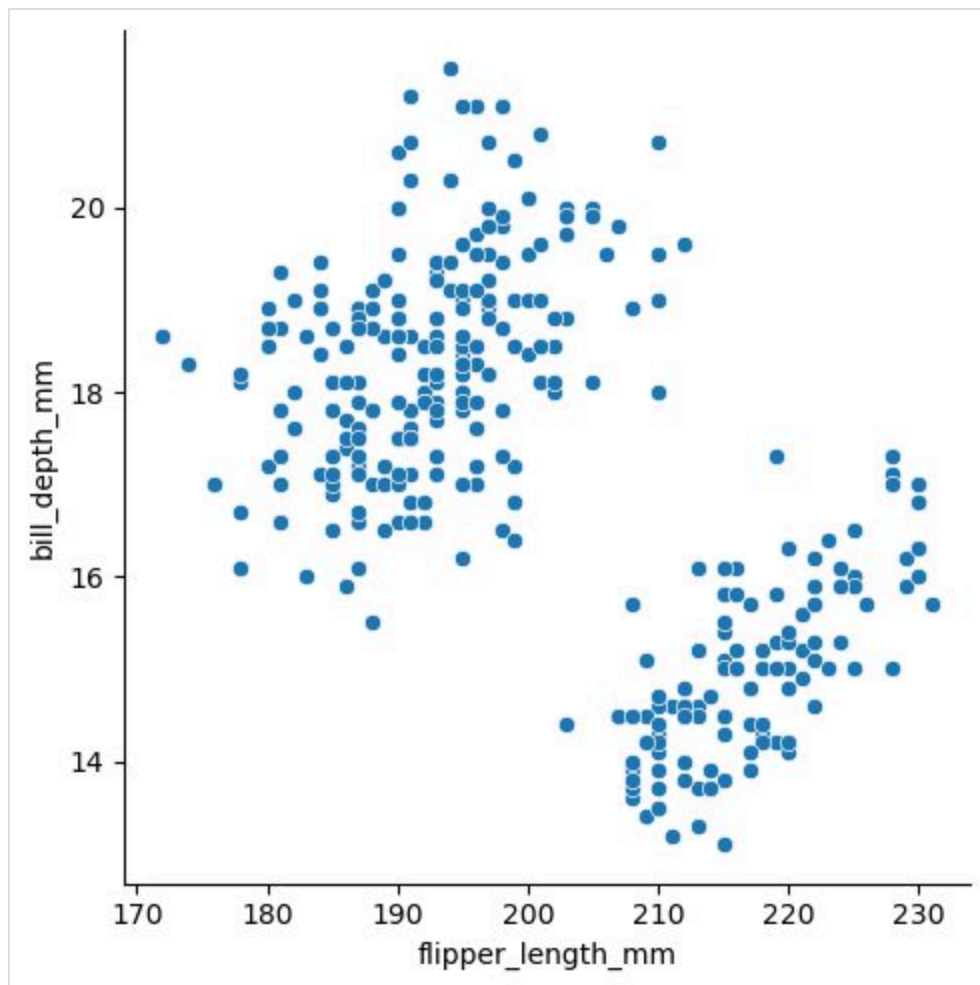

```
sns.pairplot(df_clean,
             hue='species')
plt.show()
```



Scatter Plot – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables

Penguins with longer flippers tend to have lower bill depths, while those with shorter flippers generally have higher bill depths. This suggests a negative relationship between flipper length and bill depth among penguins, with two distinct clusters visible in the data.

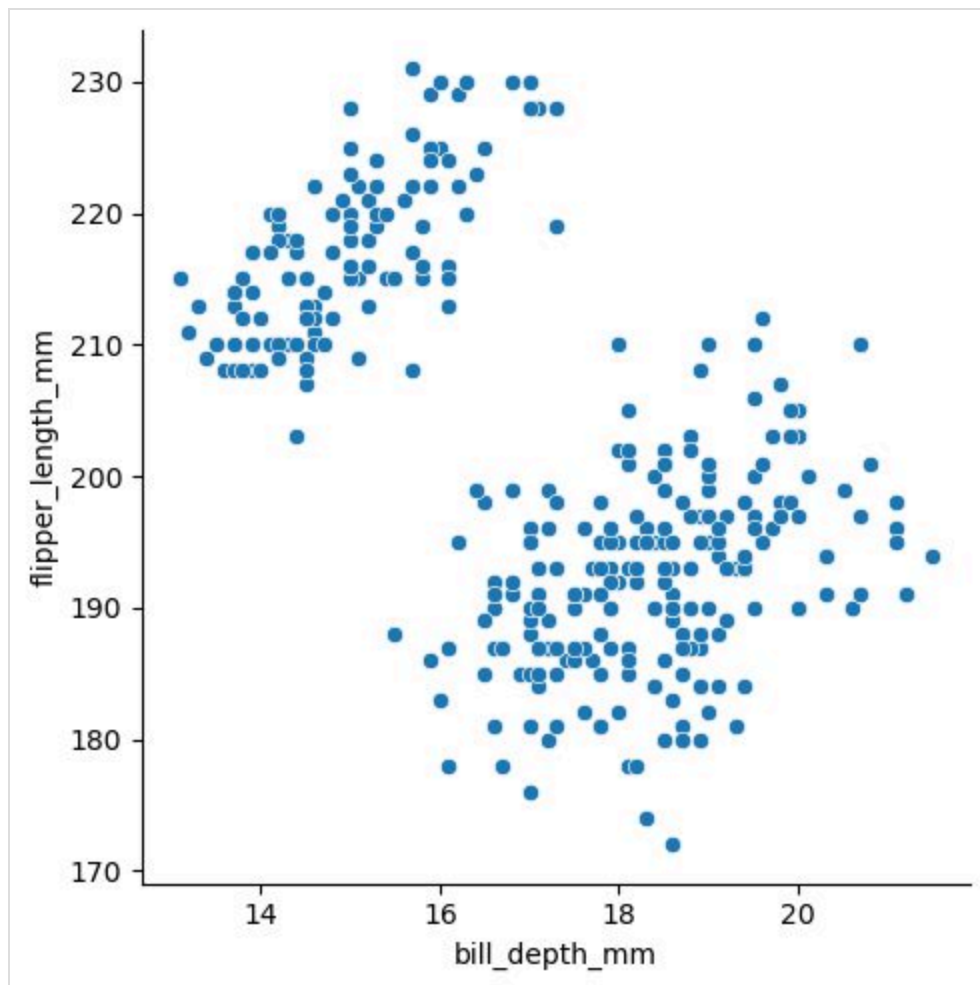
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm')
plt.show()
```



Scatter Plot – Joint Distribution of the `bill_depth_mm` and `flipper_length_mm` Continuous Variables

There is a distinct pattern: penguins with lower bill depth tend to have longer flipper lengths, while those with higher bill depth generally have shorter flipper lengths. There are two clear clusters, indicating a negative relationship between bill depth and flipper length among the penguin data.

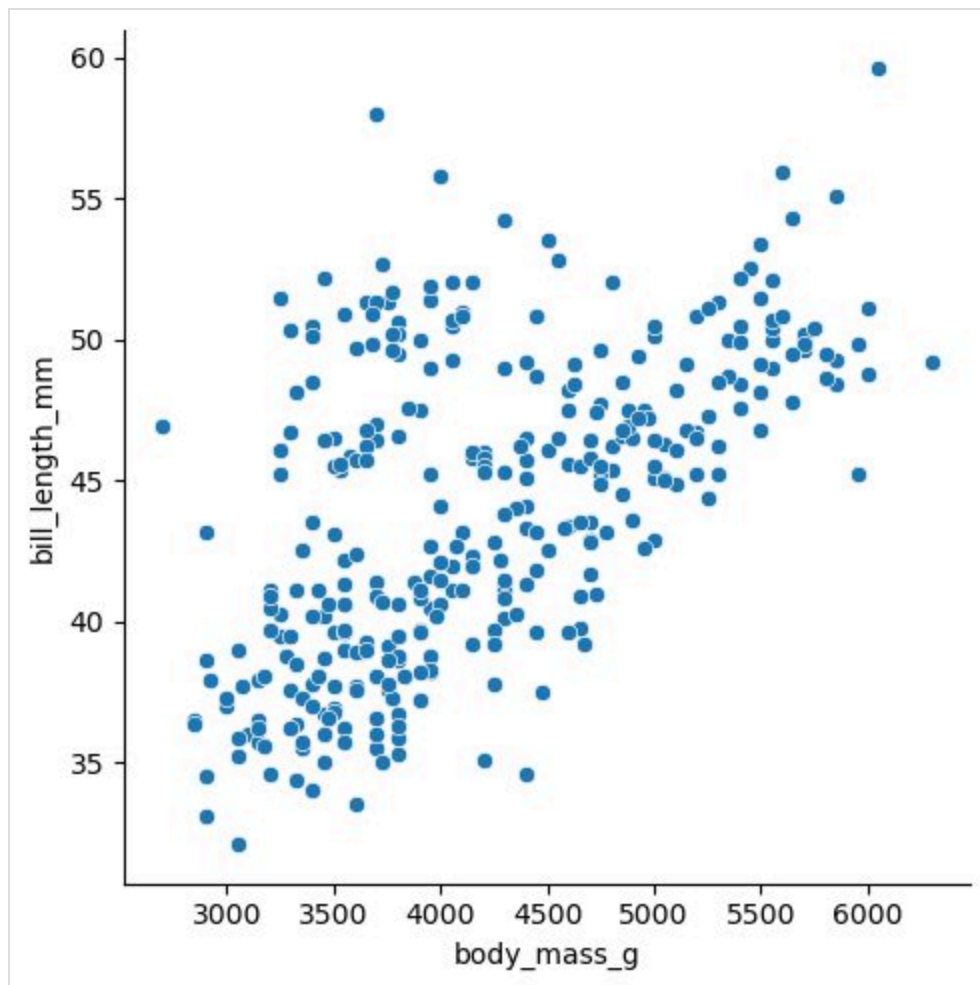
```
sns.relplot(data=df_clean,  
            x='bill_depth_mm',  
            y='flipper_length_mm')  
plt.show()
```

Scatter Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables

There is a clear positive relationship between body mass and bill length in penguins. As body mass increases, bill length also tends to increase. The data points are fairly spread out but show a general upward trend, indicating that larger penguins tend to have longer bills.

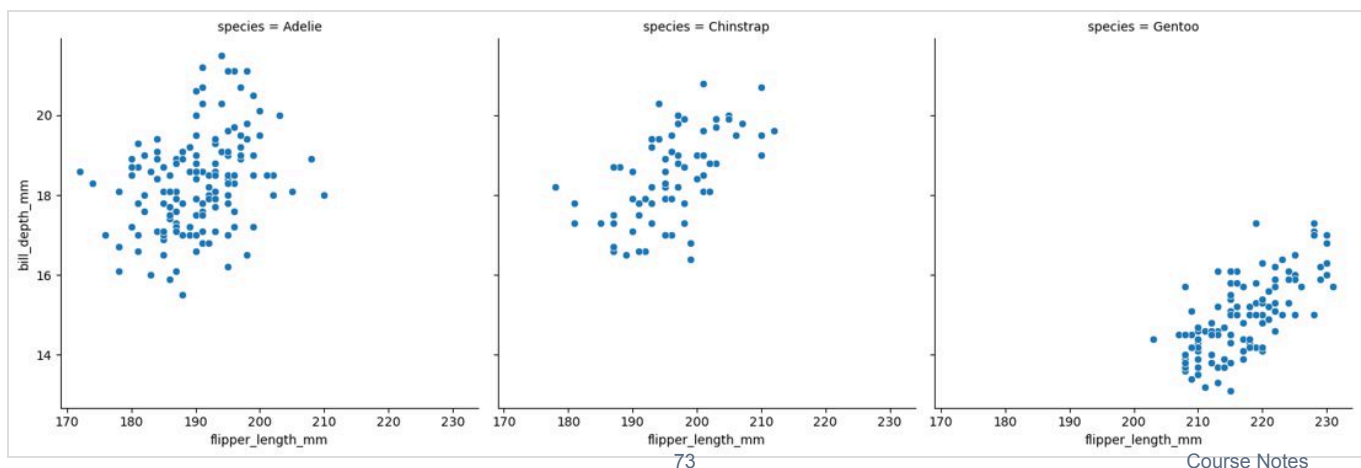
```
sns.relplot(data=df_clean,  
            x='body_mass_g',  
            y='bill_length_mm')  
plt.show()
```



Scatter Plot Facets – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each species Categorical Variable

Gentoo penguins have longer flipper lengths and lower bill depths compared to the other species.

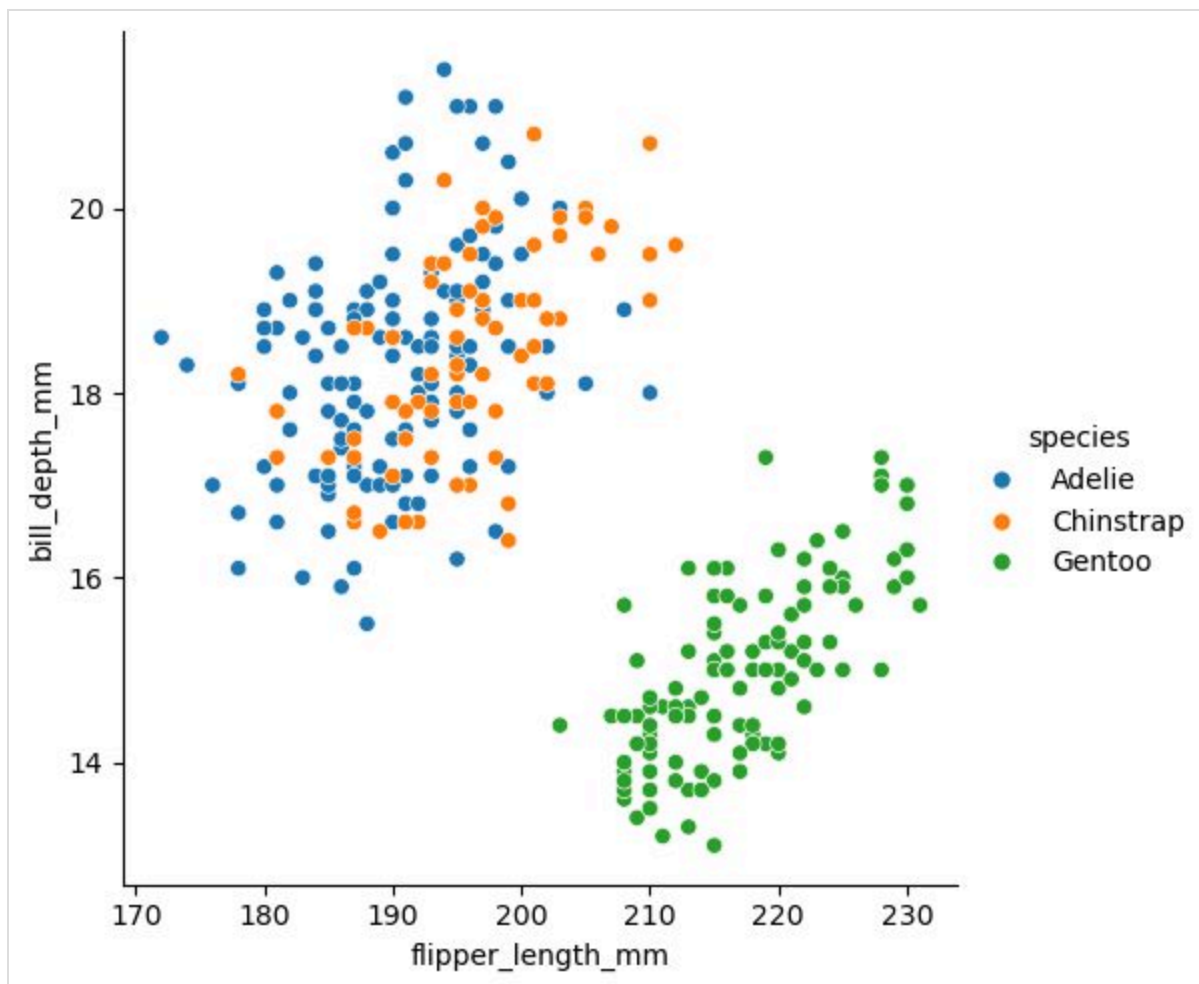
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            col='species')
plt.show()
```



Scatter Plot – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Colored by species Categorical Variable

Adelie and Chinstrap penguins cluster together with higher bill depths and shorter flipper lengths, while Gentoo penguins are clearly separated—they have longer flipper lengths and lower bill depths. The color coding makes it easy to distinguish the species' differences and reveals that bill depth and flipper length are useful for differentiating Gentoo penguins from the other two species.

```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species')
plt.show()
```

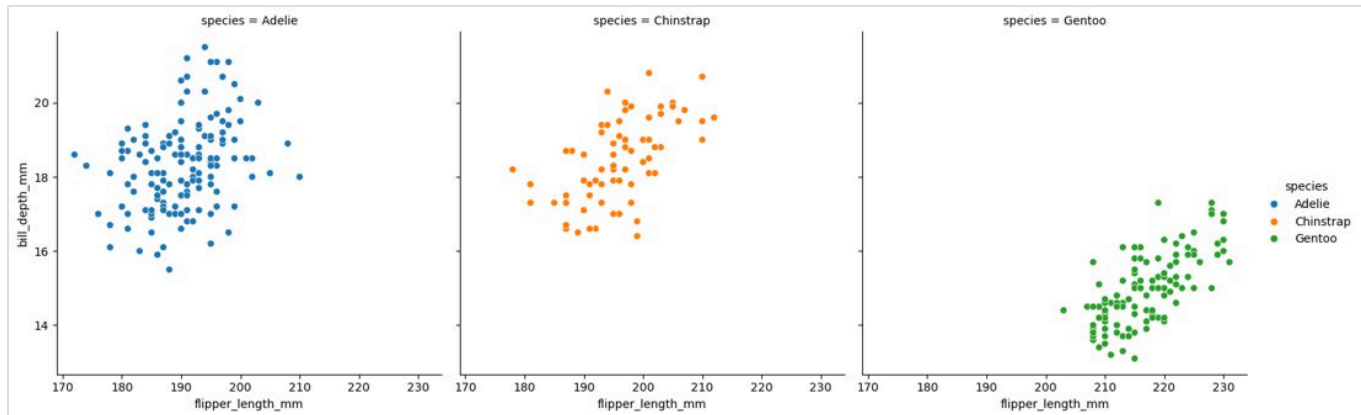


Scatter Plot Facets – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each species Categorical Variable, Colored by species Categorical Variable

Adelie penguins are shown in blue with flipper lengths mostly between 180 and 200 mm, and bill depths ranging from 16 to 21 mm. Chinstrap penguins, represented in orange, overlap with Adelie in flipper length but generally have slightly higher bill depths. Gentoo penguins, shown in green, form a

distinct cluster with much longer flipper lengths (over 210 mm) and lower bill depths, clearly separated from the other two species. The color legend on the right facilitates species identification, and overall, the plot highlights the morphological differences among the three penguin species.

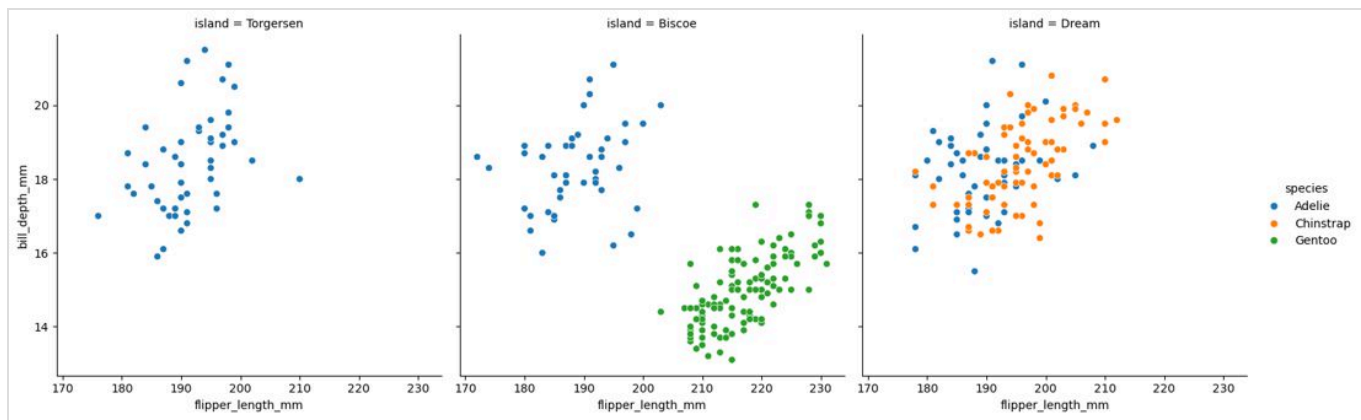
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species',
            col='species')
plt.show()
```



Scatter Plot Facets – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each island Categorical Variable, Colored by species Categorical Variable

On Torgersen island (left), only Adelie penguins are present, and they cluster around shorter flipper lengths and higher bill depths. Biscoe island (middle) hosts both Adelie and Gentoo penguins; Adelie penguins overlap with those from Torgersen, while Gentoo penguins form a distinct group with longer flippers and lower bill depths. Dream island (right) features Adelie and Chinstrap penguins, both with higher bill depths and shorter flipper lengths, but Chinstrap penguins tend to have slightly greater bill depths than Adelie. The color legend clarifies species distribution, and the plots reveal how island location is associated with both species composition and their morphological characteristics.

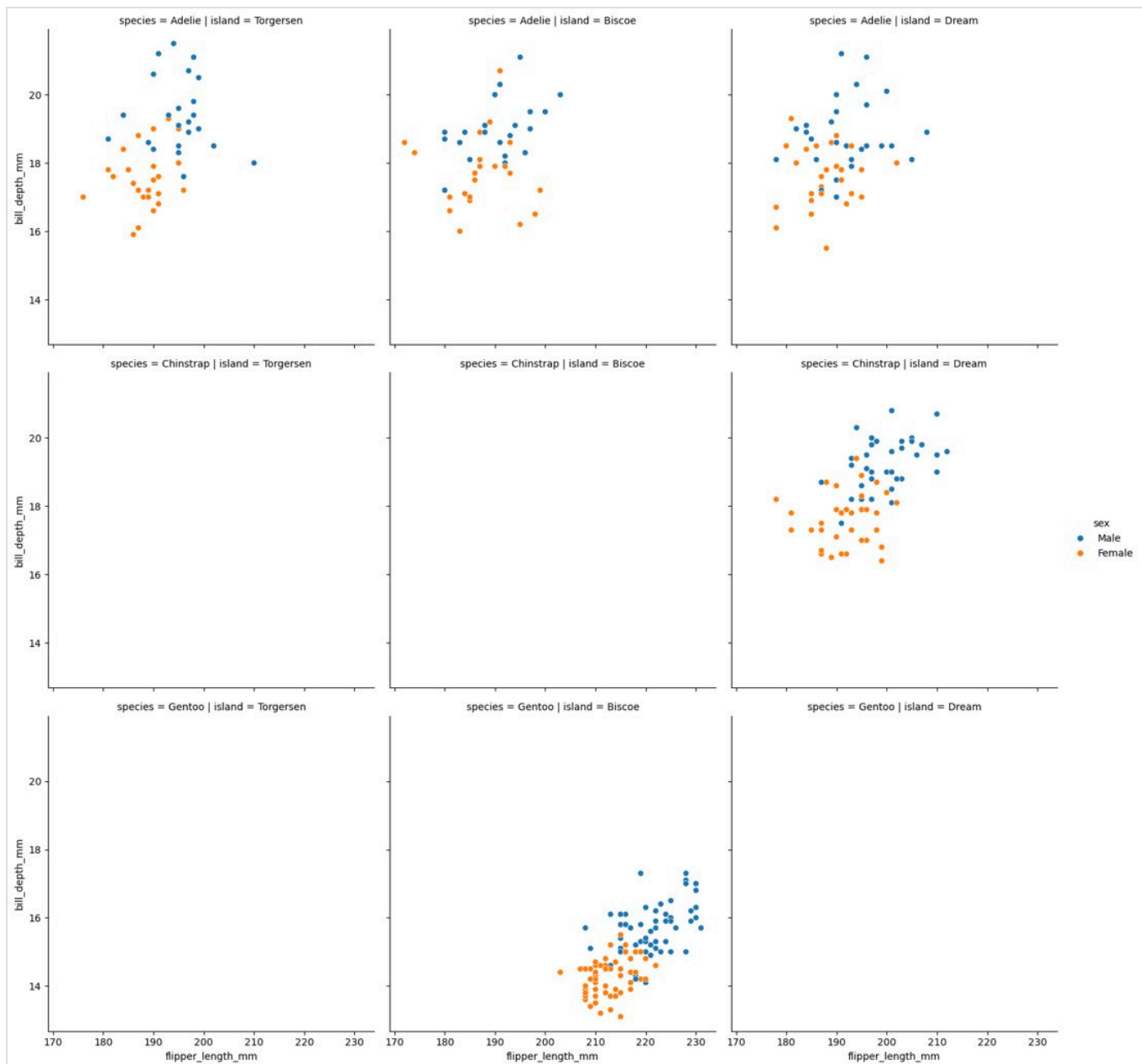
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species',
            col='island')
plt.show()
```



Scatter Plot Facets – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, Faceted by Each Combination of `species` and `island` Categorical Variables, Colored by `sex` Categorical Variable

The Adelie penguin panels show data across all three islands, with some variation in the spread and clustering of male and female measurements. Chinstrap penguins are only present on Dream island, where males tend to have slightly greater flipper lengths and bill depths than females. Gentoo penguins appear only on Biscoe island, with males generally showing longer flipper lengths and higher bill depths compared to females. This visualization makes it easy to compare how sex, species, and island location interact to influence penguin morphology.

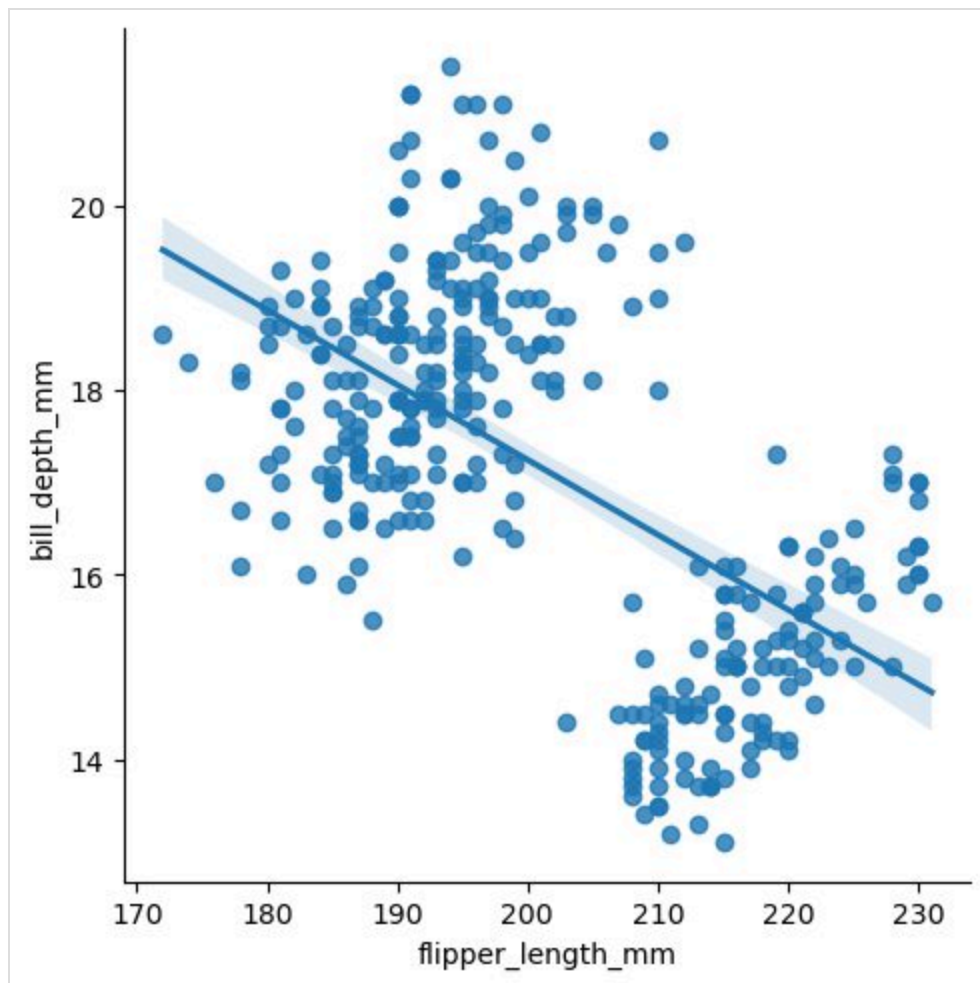
```
sns.relplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='sex',
            col='island',
            row='species')
plt.show()
```



Trend Plot – Joint Distribution of the `flipper_length_mm` and `bill_depth_mm` Continuous Variables, with Fitted Linear Regression and Confidence Interval

The negative slope of the regression line indicates an inverse relationship: as flipper length increases, bill depth tends to decrease. The spread of points suggests that there are distinct clusters, likely representing different species, but overall, longer flippers are associated with shallower bills. The shaded area around the line provides a visual indication of the uncertainty or variability in this trend.

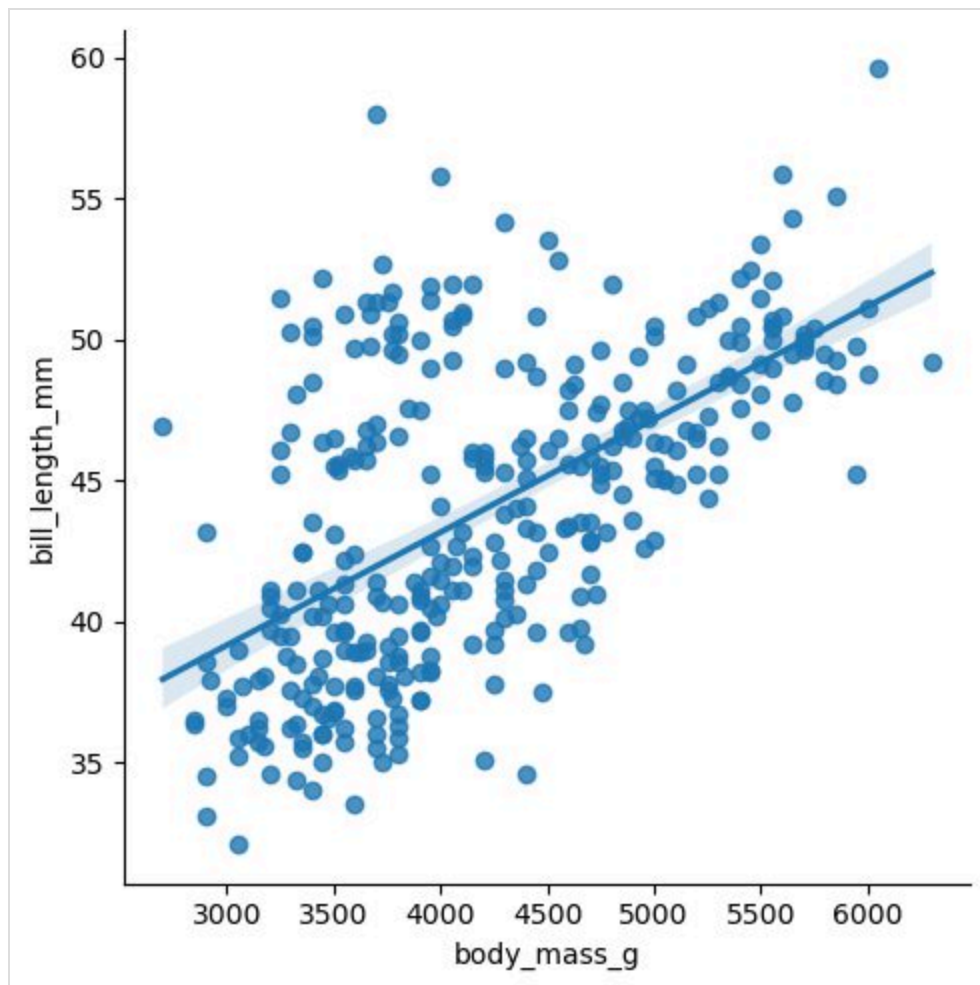
```
sns.lmplot(data=df_clean,
           x='flipper_length_mm',
           y='bill_depth_mm')
plt.show()
```



Trend Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, with Fitted Linear Regression and Confidence Interval

The positive slope of the regression line indicates a direct correlation: as body mass increases, bill length also tends to increase. The data points are spread across a range of values, but the overall trend suggests that larger penguins generally have longer bills. The shaded area around the regression line represents the confidence interval, illustrating the variability in this relationship.

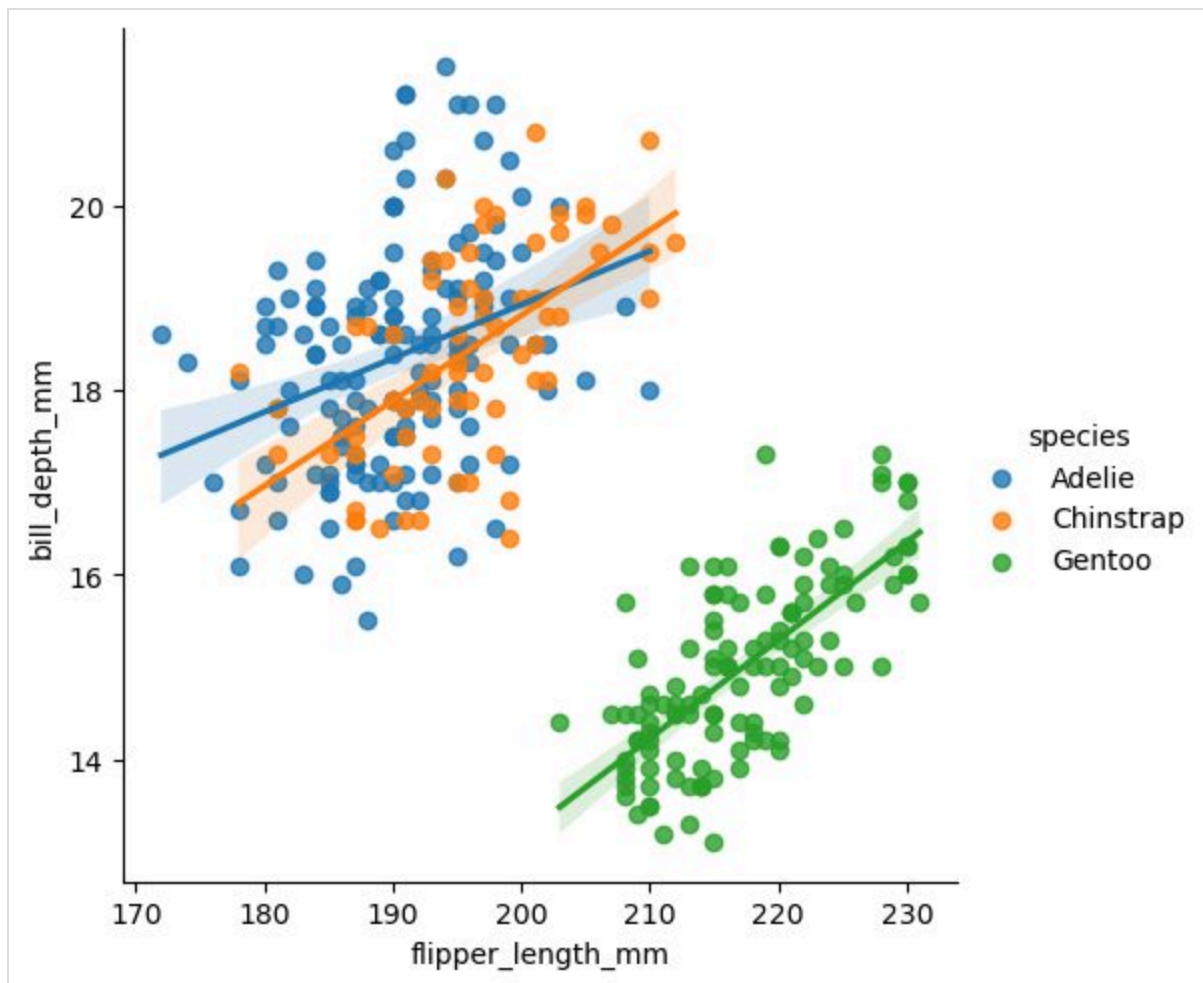
```
sns.lmplot(data=df_clean,  
           x='body_mass_g',  
           y='bill_length_mm')  
plt.show()
```

Trend Plot – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Colored by species Categorical Variable, with Fitted Linear Regression and Confidence Interval

Adelie and Chinstrap regression lines indicate a positive correlation between the two variables for both species. Gentoo penguins form a distinct cluster with longer flipper lengths and lower bill depths, and their regression line also shows a positive association. The shaded regions around each regression line reflect the confidence intervals, highlighting the variability of the data within each species. Overall, the plot emphasizes both the differences in morphology between species and the positive relationship between flipper length and bill depth within each group.

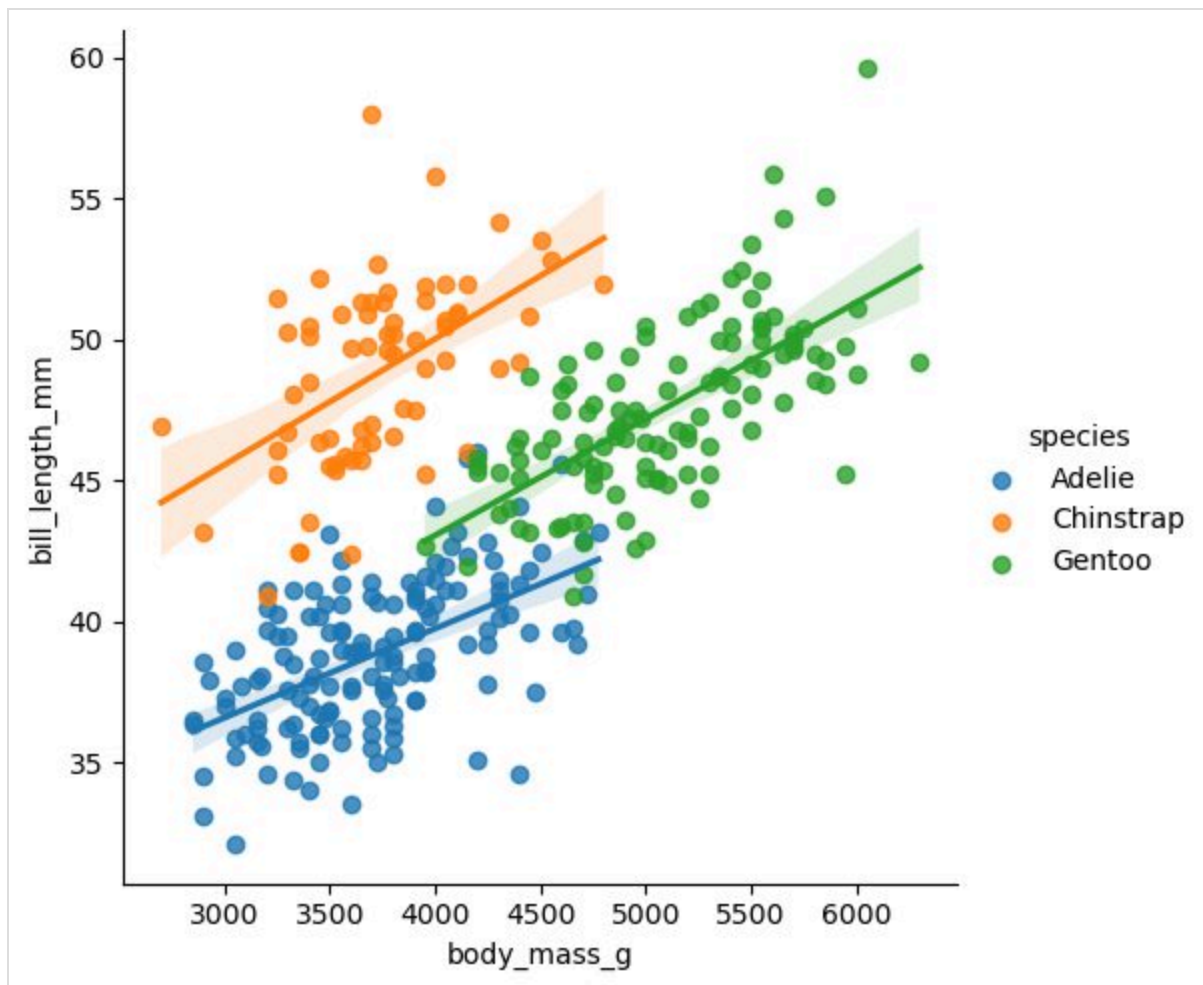
```
sns.lmplot(data=df_clean,
            x='flipper_length_mm',
            y='bill_depth_mm',
            hue='species')
plt.show()
```

Trend Plot – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Colored by `species` Categorical Variable, with Fitted Linear Regression and Confidence Interval

All three species show a positive correlation between body mass and bill length, but their regression lines are vertically separated, indicating clear differences in average bill length among species for a given body mass. Chinstrap penguins (orange) generally have the longest bills for a given body mass, followed by Gentoo (green) and then Adelie (blue). The shaded regions around each regression line represent the variability within each species, and the plot overall highlights both the species-specific trends and the general pattern that heavier penguins tend to have longer bills.

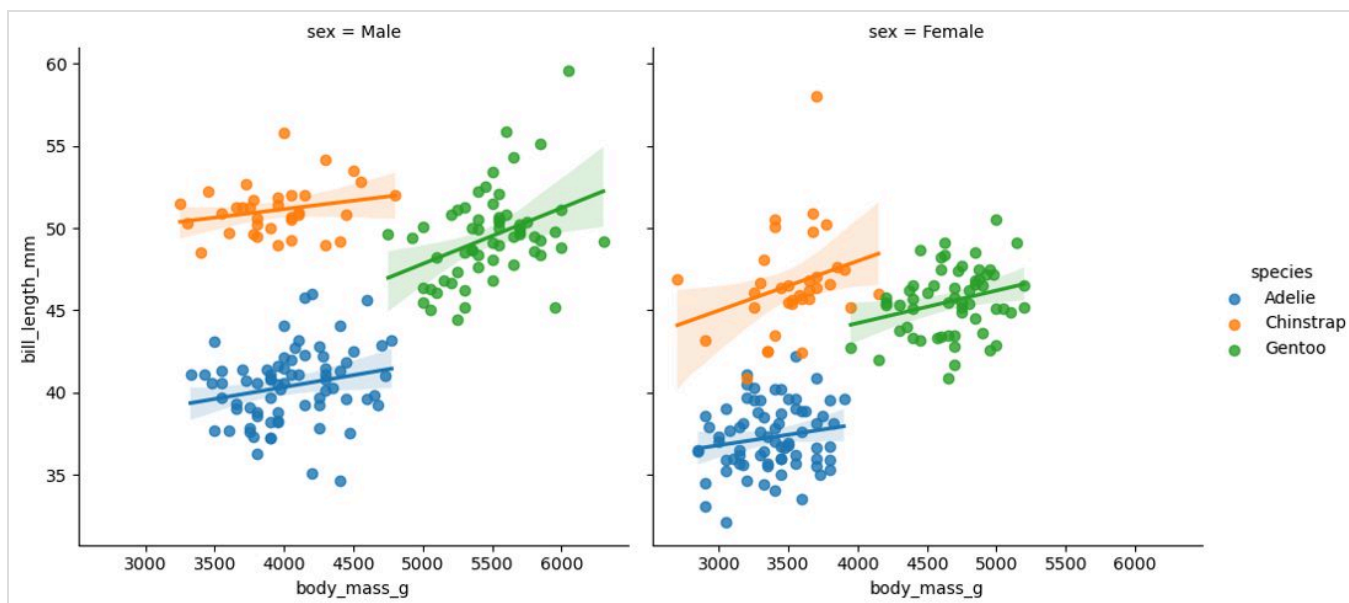
```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='species')
plt.show()
```



Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by `sex` Categorical Variable, Colored by `species` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Males generally have longer bills than females for a given body mass, and that Chinstrap penguins have the longest bills, followed by Gentoo and Adelie.

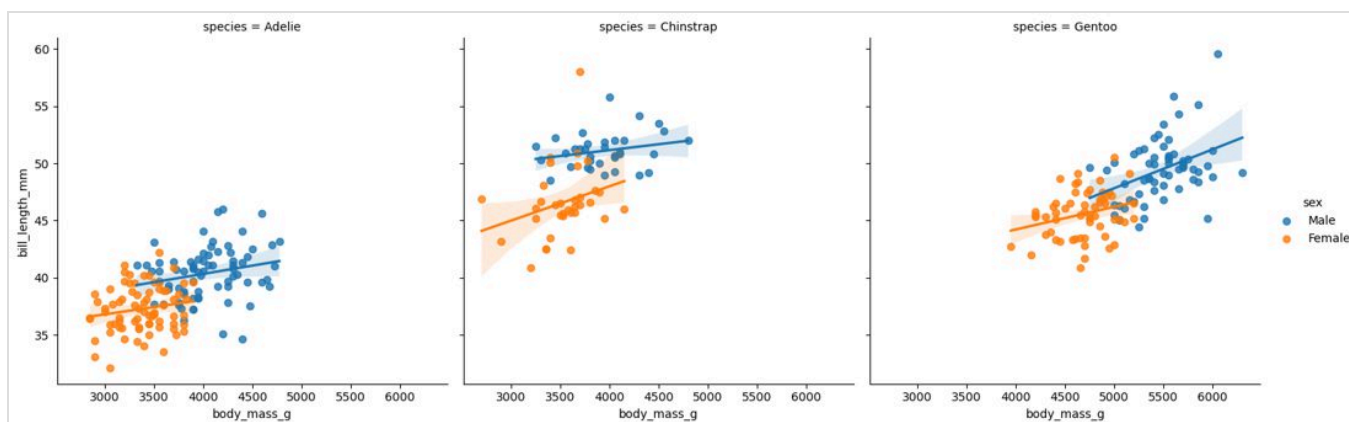
```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='species',
            col='sex')
plt.show()
```



Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by `species` Categorical Variable, Colored by `sex` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Males generally have longer bills than females for a given body mass across all species.

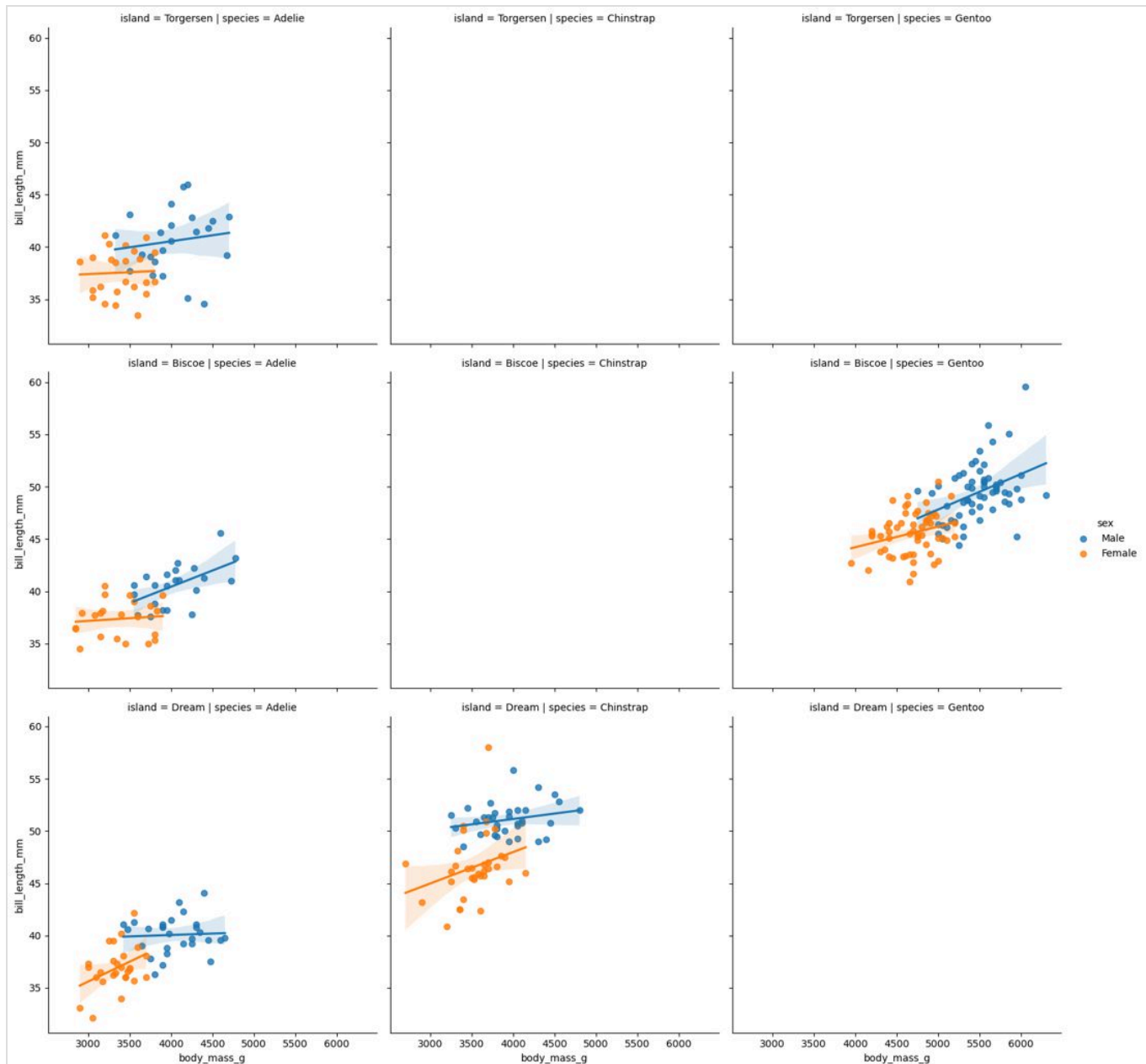
```
sns.lmplot(data=df_clean,
            x='body_mass_g',
            y='bill_length_mm',
            hue='sex',
            col='species')
plt.show()
```



Trend Plots – Joint Distribution of the `body_mass_g` and `bill_length_mm` Continuous Variables, Faceted by Each Combination of `island` and `species` Categorical Variables, Colored by `sex` Categorical Variable, with Fitted Linear Regression and Confidence Interval

Adelie penguins appear on all islands, Chinstrap penguins only on Dream, and Gentoo penguins only on Biscoe. In each group, males generally have longer bills than females for a given body mass, and the regression lines with confidence intervals highlight the differences and variability within each island-species-sex combination.

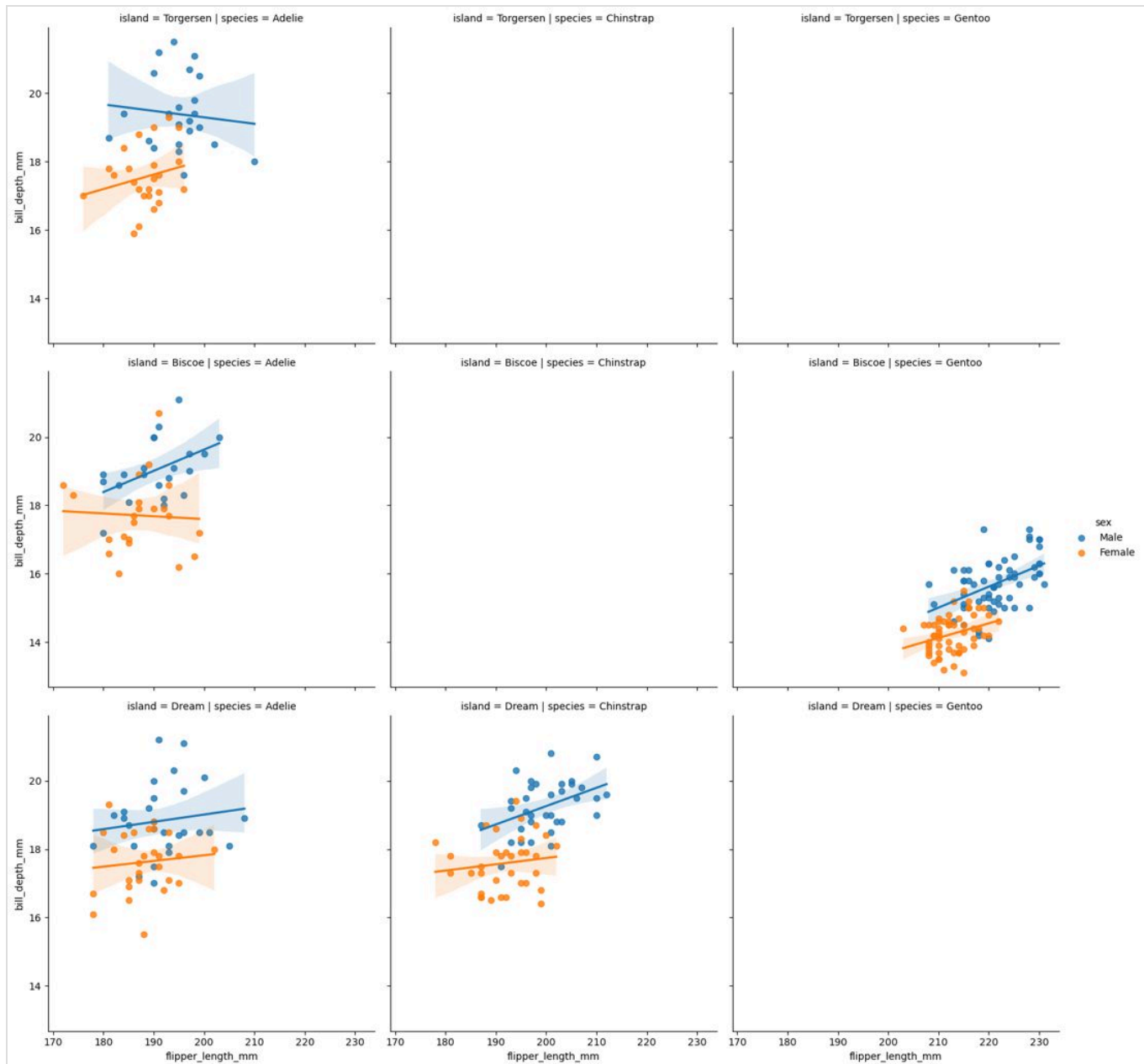
```
sns.lmplot(data=df_clean,
           x='body_mass_g',
           y='bill_length_mm',
           hue='sex',
           col='species',
           row='island')
plt.show()
```



Trend Plots – Joint Distribution of the flipper_length_mm and bill_depth_mm Continuous Variables, Faceted by Each Combination of island and species Categorical Variables, Colored by sex Categorical Variable, with Fitted Linear Regression and Confidence Interval

Adelie penguins appear on all islands, Chinstrap on Dream, and Gentoo on Biscoe and Dream, showing that males generally have deeper bills than females for a given flipper length, and highlighting differences in morphology across islands and species.

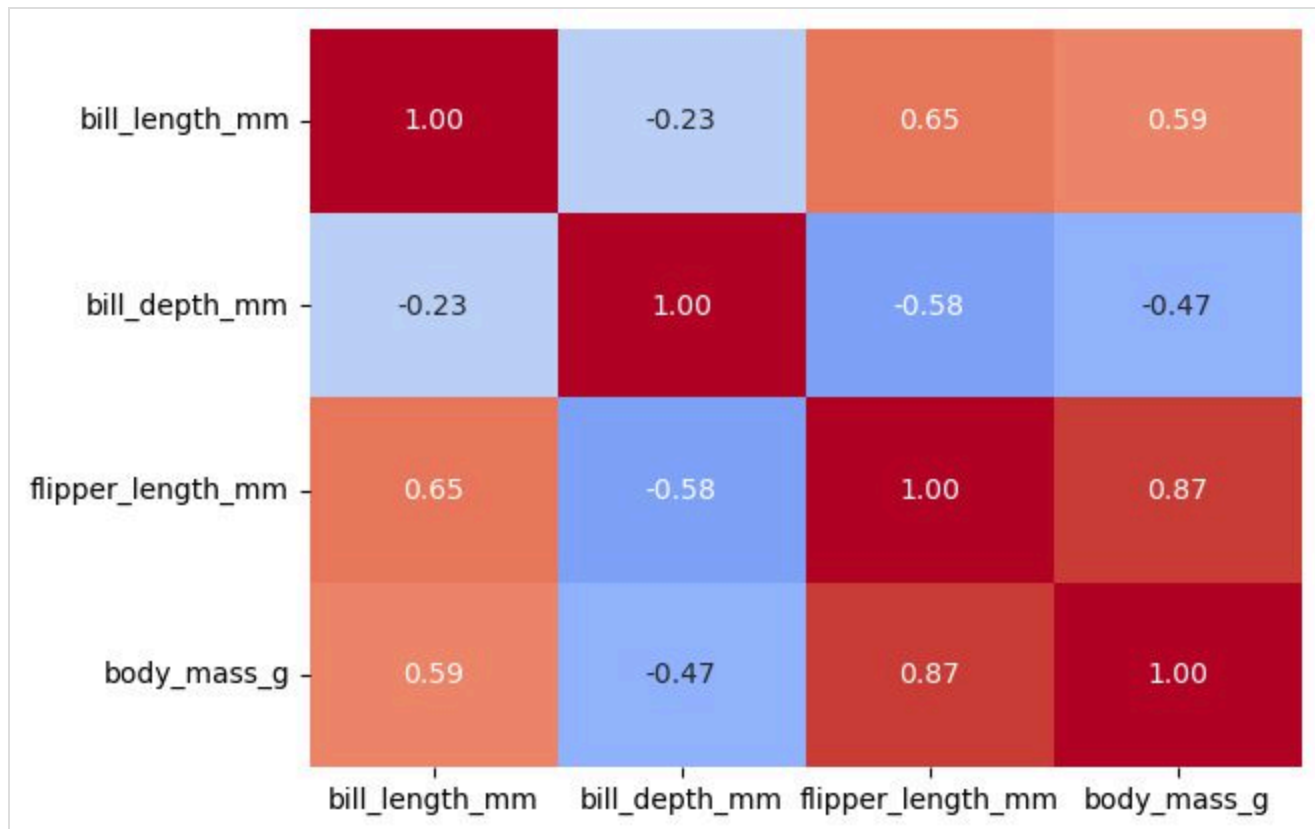
```
sns.lmplot(data=df_clean,
           x='flipper_length_mm',
           y='bill_depth_mm',
           hue='sex',
           col='species',
           row='island')
plt.show()
```



Correlation Plots - Correlation Matrix of All Numerical Variables

Flipper length and body mass are strongly positively correlated (0.87), bill length is moderately correlated with both flipper length (0.65) and body mass (0.59), and bill depth is negatively correlated with both flipper length (-0.58) and body mass (-0.47).

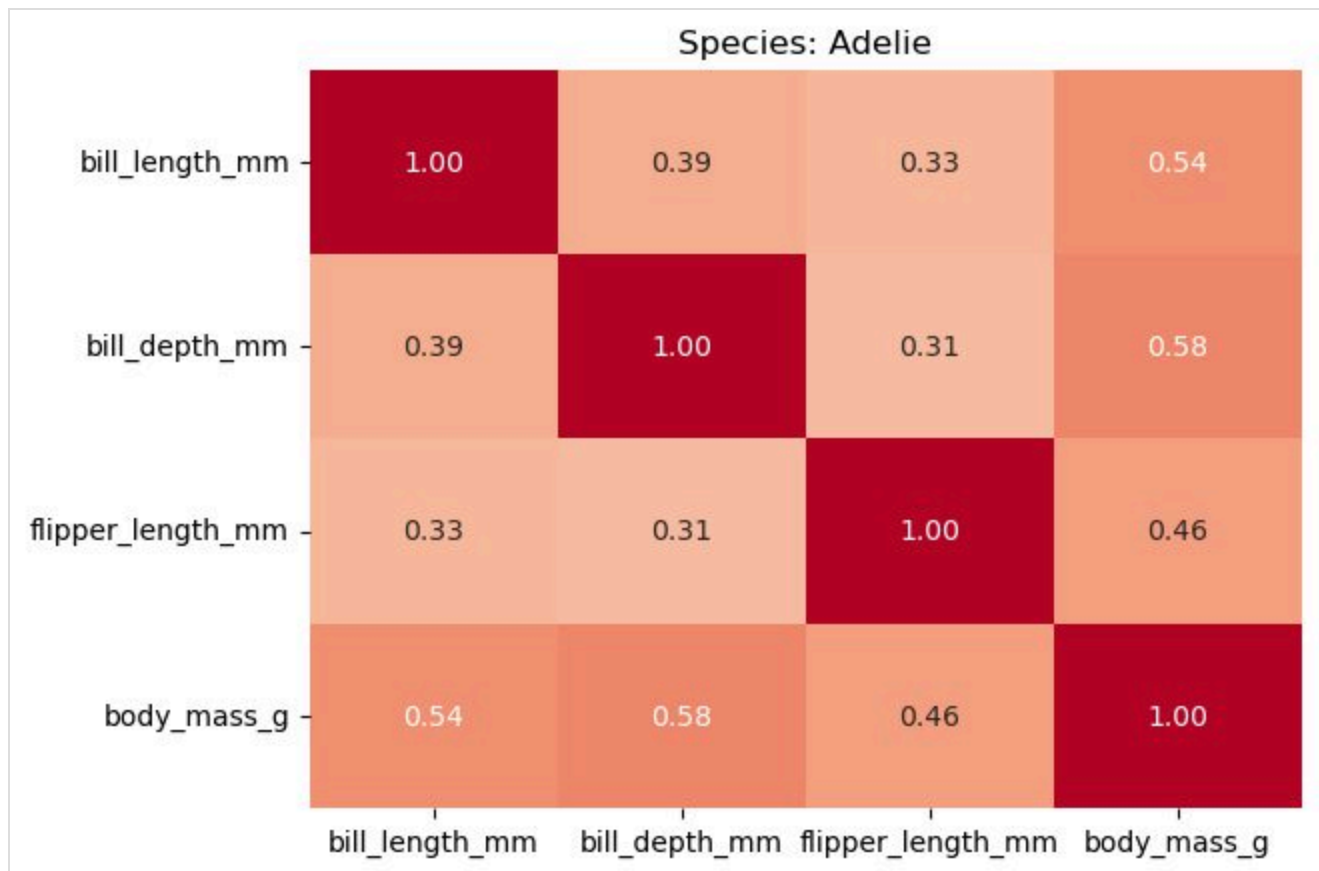
```
fig, ax = plt.subplots()
sns.heatmap(data=df_clean.corr(numeric_only=True),
            vmin=-1,
            vmax=1,
            center=0,
            cmap='coolwarm',
            cbar=False,
            annot=True,
            fmt='.2f',
            annot_kws={'size': 10},
            ax=ax)
plt.show()
```

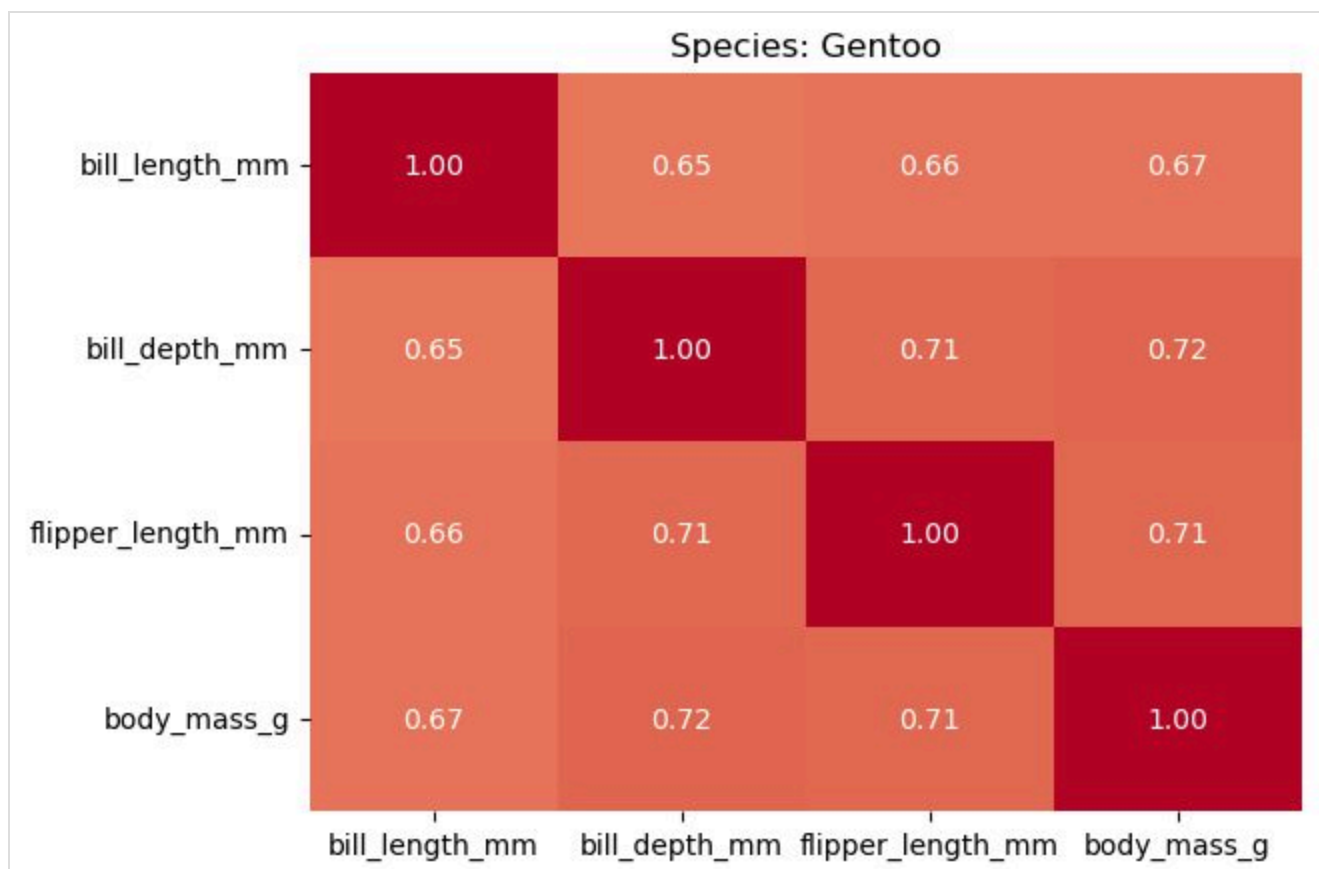
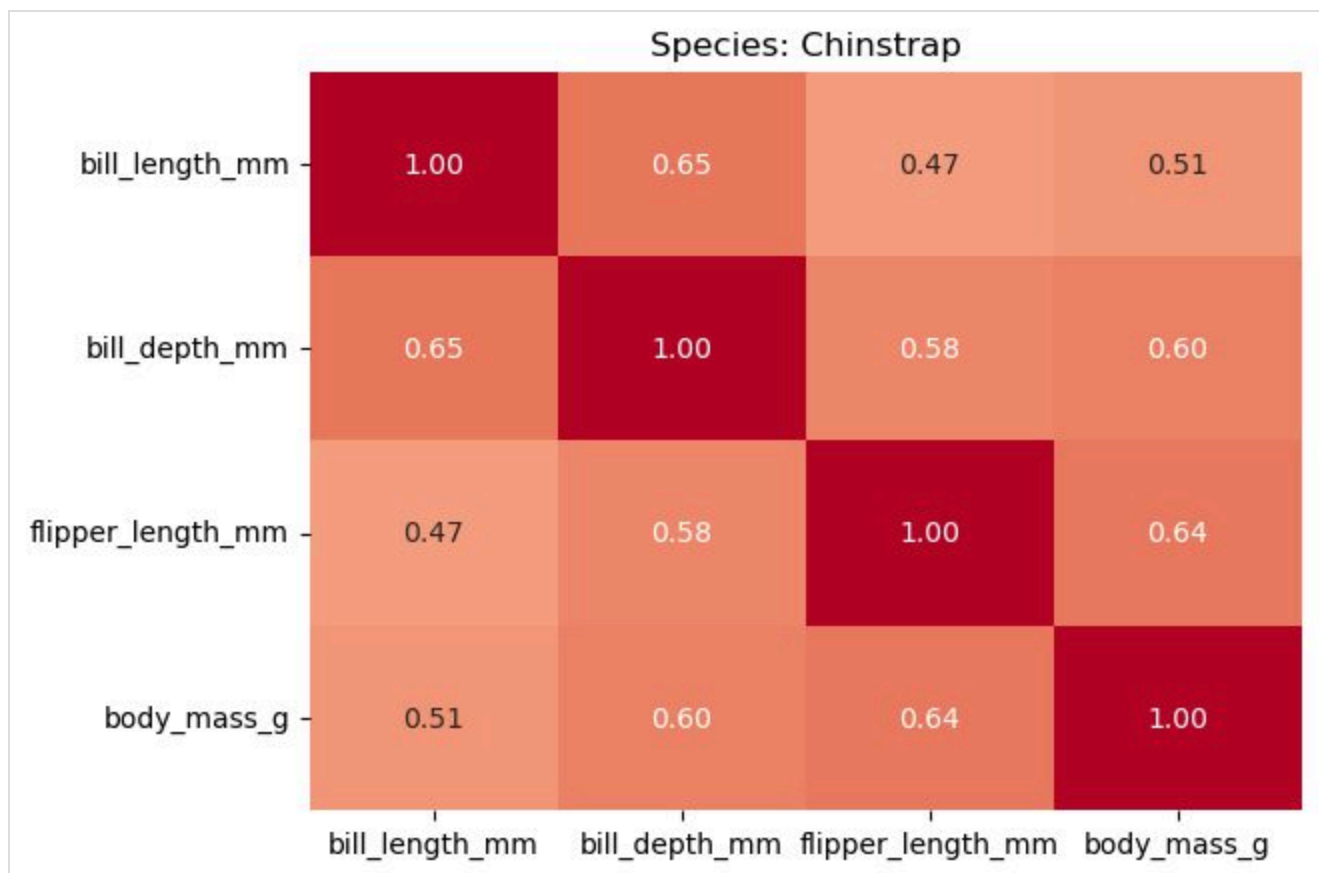


Correlation Plots – Correlation Matrix of All Numerical Variables, Faceted by species Categorical Variable

Correlations among bill length, bill depth, flipper length, and body mass are stronger in Gentoo penguins (all above 0.65), moderate in Chinstrap (mostly 0.47–0.65), and weaker in Adelie (mostly 0.31–0.58), indicating species-specific differences in how morphological traits are related.

```
species_list = df_clean['species'].unique()
for species in species_list:
    fig, ax = plt.subplots()
    subset = df_clean[df_clean['species'] == species]
    sns.heatmap(data=subset.corr(numeric_only=True),
                vmin=-1,
                vmax=1,
                center=0,
                cmap='coolwarm',
                cbar=False,
                annot=True,
                fmt='.2f',
                annot_kws={'size': 10},
                ax=ax)
    ax.set_title(f'Species: {species}')
    plt.show()
```



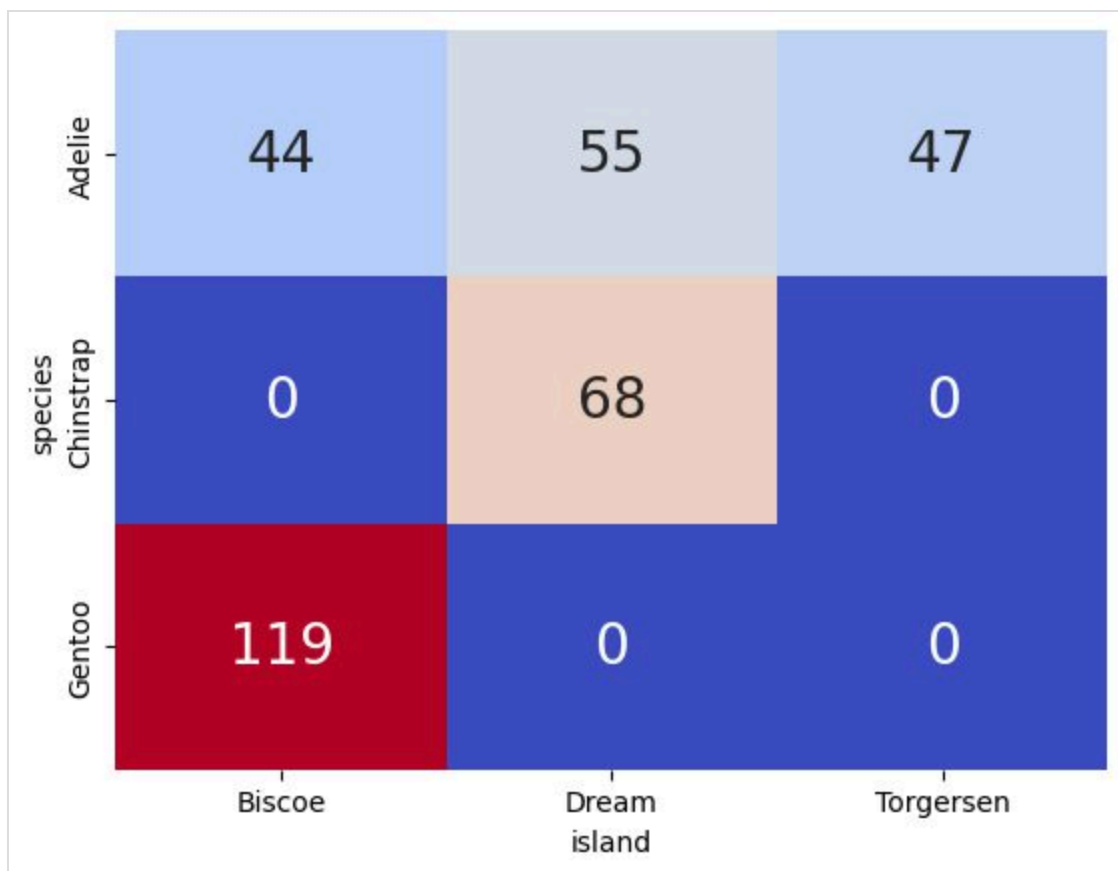


Heat Map – Counts of Observations for Each Combination of species and island Categorical Variables

Adelie penguins are present on all three islands, Chinstrap penguins are found only on Dream island, and Gentoo penguins are found only on Biscoe island.

```
fig, ax = plt.subplots()
sns.heatmap(pd.crosstab(df_clean.species,
                        df_clean.island),
            ax=ax,
            annot=True,
            annot_kws={'size': 20},
            fmt='d',
            cmap='coolwarm',
            cbar=False)

plt.show()
```

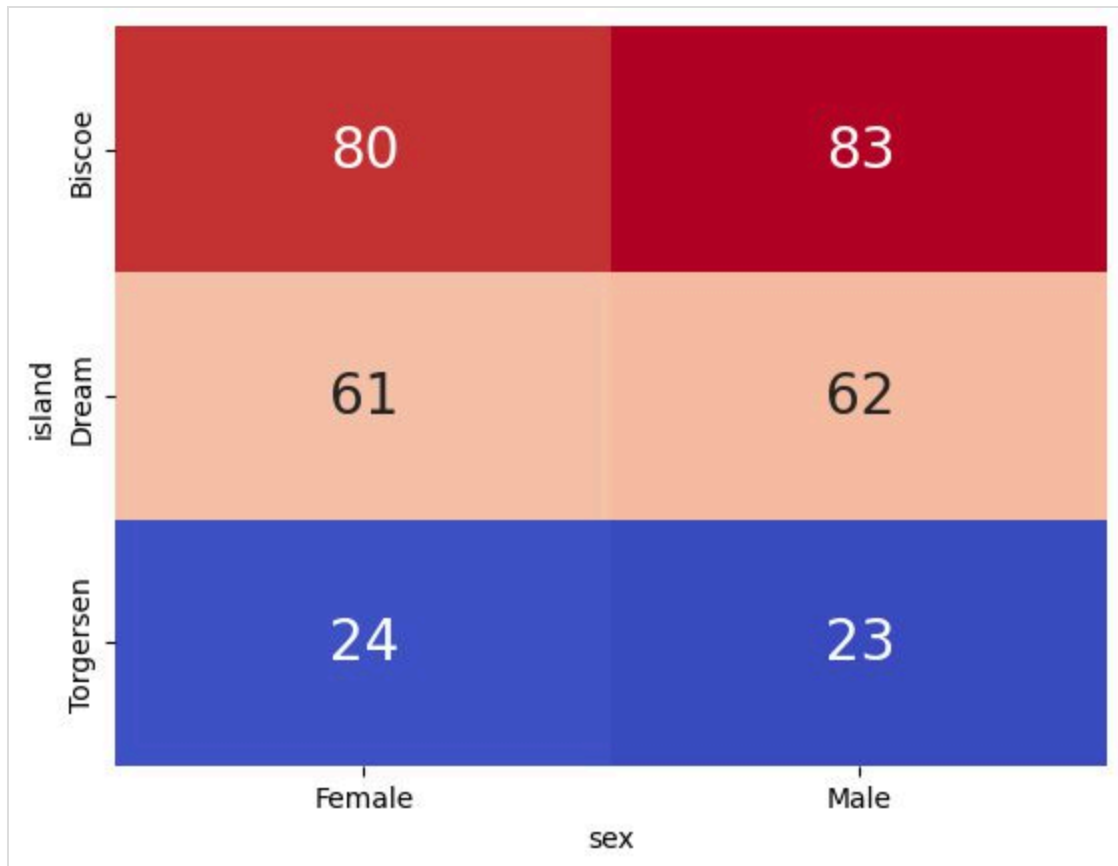


Heat Map – Counts of Observations for Each Combination of island and sex Categorical Variables

Biscoe has the highest numbers of both females (80) and males (83), Dream island also has similar counts of females (61) and males (62), and Torgersen has the lowest counts, with 24 females and 23 males. The distribution of sexes is fairly balanced across all islands.

```
fig, ax = plt.subplots()
sns.heatmap(pd.crosstab(df_clean.island,
                        df_clean.sex ),
            ax=ax,
            annot=True,
            annot_kws={'size': 20},
            fmt='d',
            cmap='coolwarm',
            cbar=False)

plt.show()
```



Final Summation of EDA and Key Features for Modeling

The exploratory data analysis of the Palmer Penguins dataset, comprising 333 cleaned records after removing missing values, revealed distinct morphological differences among the three species: Adelie, Chinstrap, and Gentoo. Univariate analyses showed that `flipper_length_mm` and `body_mass_g` have the widest ranges and are strongly right-skewed, while `bill_length_mm` and `bill_depth_mm` are more normally distributed. Categorical variables like `sex` (balanced male/female) and `island` (Biscoe most common) showed varying distributions, with Gentoo exclusive to Biscoe, Chinstrap to Dream, and Adelie on all islands. Bivariate and multivariate visualizations, including pair plots and correlation matrices, indicated that `flipper_length_mm` (correlation with `body_mass_g` ~0.87 overall) and `body_mass_g` are the strongest linear predictors for distinguishing species, with Gentoo showing the largest values, followed by Chinstrap and Adelie. Bill measurements also differ, with Chinstrap having

the longest bills and Adelie the shortest. For modeling species classification, key features should include `flipper_length_mm`, `body_mass_g`, `bill_length_mm`, and `bill_depth_mm` as primary predictors, while considering `island` and `sex` for interaction effects, and potentially feature engineering to capture non-linear relationships or ratios (e.g., bill length to depth) to enhance model performance.

02-Clustering: Penguins Dataset - Clustering

Penguins Dataset

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island       object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex          object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g   2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm  54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Data Standardization

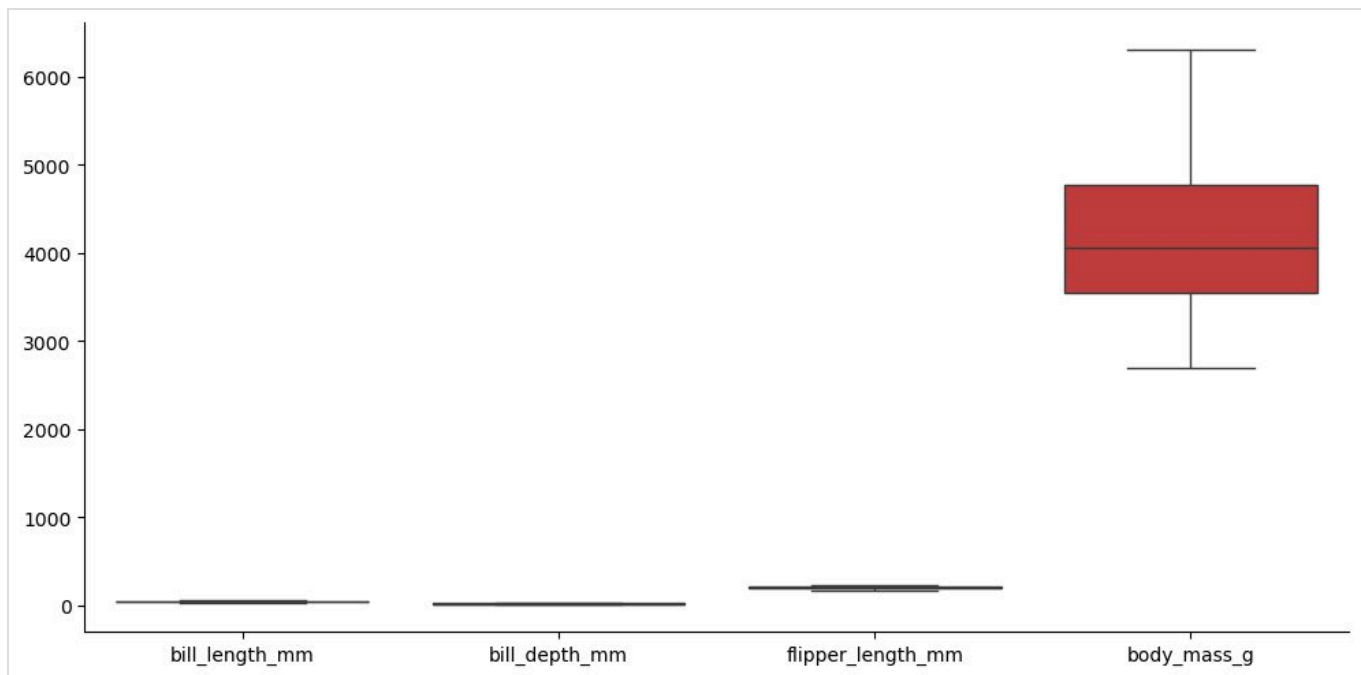
Examine raw numeric features magnitude and scale.

The actual values in the boxplot represent the distributions of the penguin dataset's numeric features: bill length (mm), bill depth (mm), flipper length (mm), and body mass (g). The plot shows that body mass is on a much larger scale than the other features, indicating that its values are much higher (in grams) compared to the millimeter measurements of the other features, which is important for understanding feature scaling and the need for standardization in data analysis.

```
df_clean.describe()
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	333.000000	333.000000	333.000000	333.000000
mean	43.992793	17.164865	200.966967	4207.057057
std	5.468668	1.969235	14.015765	805.215802
min	32.100000	13.100000	172.000000	2700.000000
25%	39.500000	15.600000	190.000000	3550.000000
50%	44.500000	17.300000	197.000000	4050.000000
75%	48.600000	18.700000	213.000000	4775.000000
max	59.600000	21.500000	231.000000	6300.000000

```
sns.catplot(data=df_clean,
            kind='box',
            aspect=2)
plt.show()
```



Prepare the dataset by extracting our numeric features.

```
df_clean_features = df_clean.select_dtypes('number').\
    copy()
```

Standardize the numeric features.

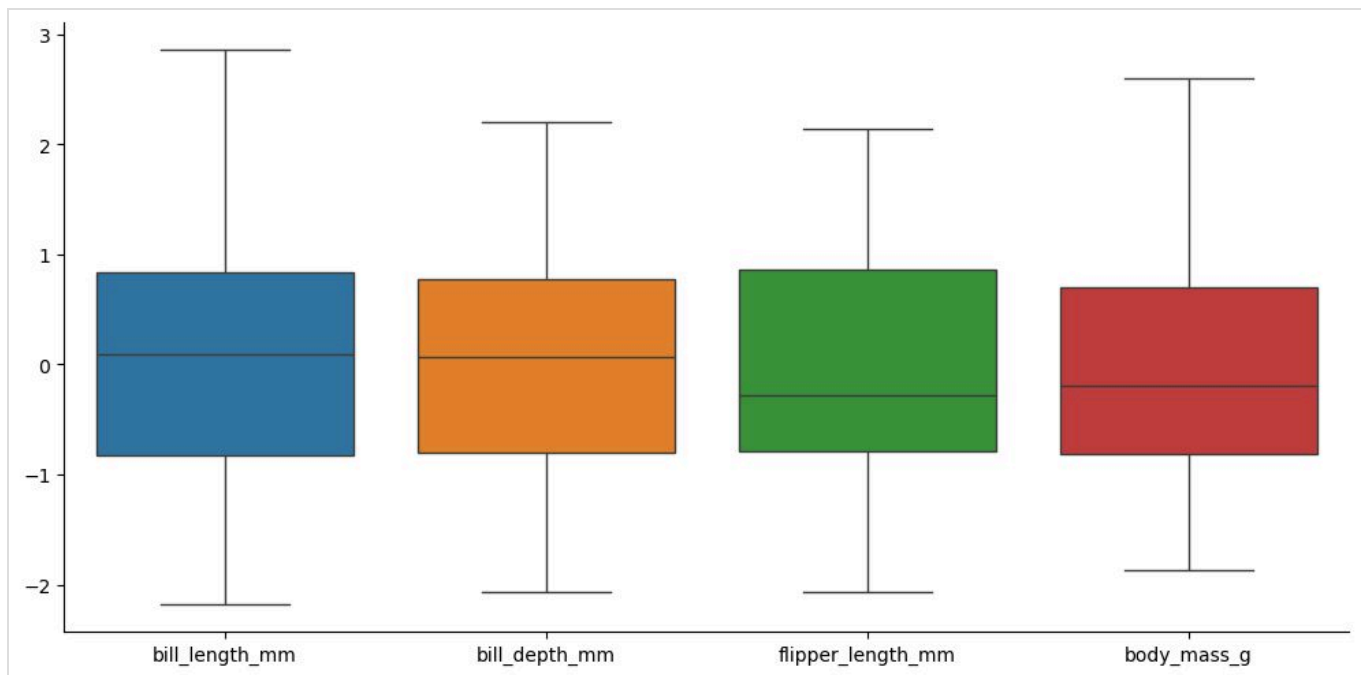
```
X = StandardScaler().fit_transform(df_clean_features)
```

Examine standardized numeric features magnitude and scale.

After standardization, all numeric features (bill length, bill depth, flipper length, and body mass) have similar distributions centered around zero with comparable spread, indicating that differences in scale have been removed and the features are now suitable for clustering and other analyses that assume equal variance.

```
sns.catplot(data=pd.DataFrame(X,
                               columns=df_clean_features.columns),
            kind='box',
            aspect=2)

plt.show()
```



KMeans

Initialize, fit and predict our KMeans by starting with two clusters.

```
clusters_2 = KMeans(n_clusters=2,
                    random_state=42,
                    n_init=25,
                    max_iter=500).\
                    fit_predict(X)
```

Assign cluster labels to a clean copied DataFrame.

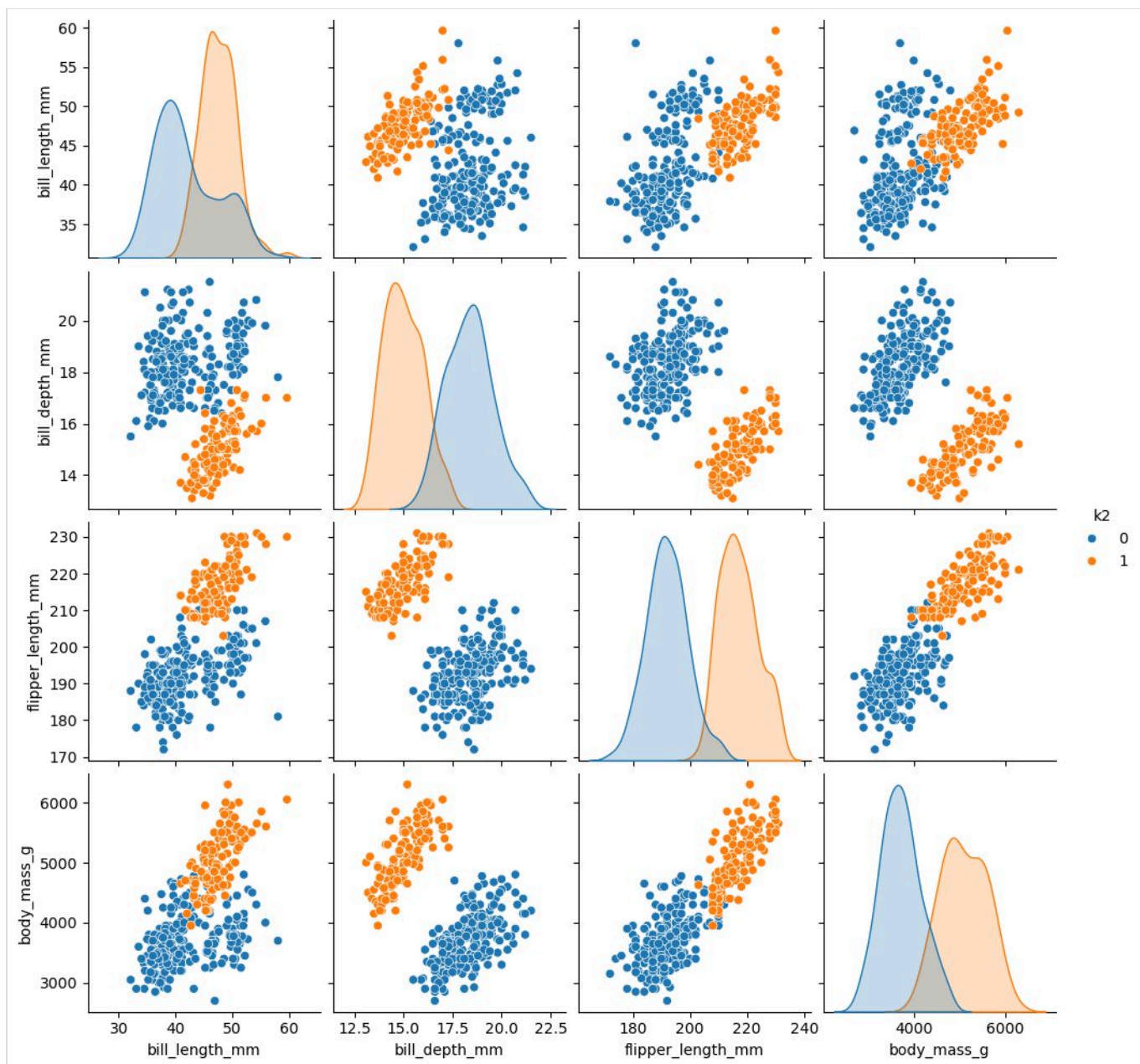
```
df_clean_copy = df_clean.copy()
```

```
df_clean_copy['k2'] = pd.Series(clusters_2,
                               index=df_clean_copy.index). \
                               astype('category')
```

Visualize the clusters using a pair plot.

The pair plot shows that the KMeans clustering with two clusters separates the penguins into two distinct groups based on their physical measurements, with clear differences in bill length, bill depth, flipper length, and body mass between the clusters, indicating strong feature-based separation in the dataset.

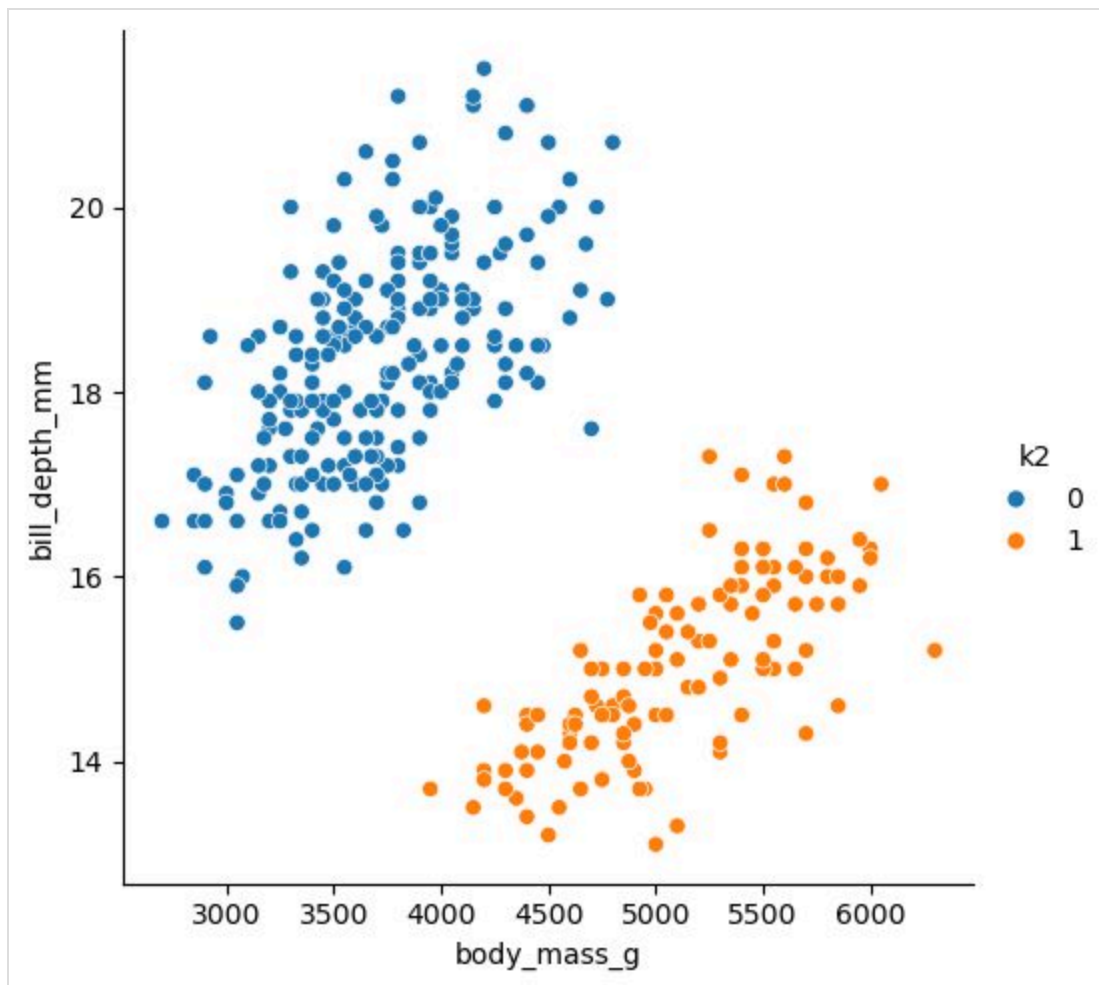
```
sns.pairplot(data=df_clean_copy,
             hue='k2',
             diag_kws={'common_norm': False})
plt.show()
```

Visualize the cluster results on a scatter plot.

This scatter plot demonstrates that the two clusters identified by KMeans are well separated, with one group of penguins having higher body mass and lower bill depth, and the other group having lower body mass and higher bill depth, highlighting distinct physical differences between the clusters.

```
sns.relplot(data=df_clean_copy,
            x='body_mass_g',
            y='bill_depth_mm',
            hue='k2')
plt.show()
```

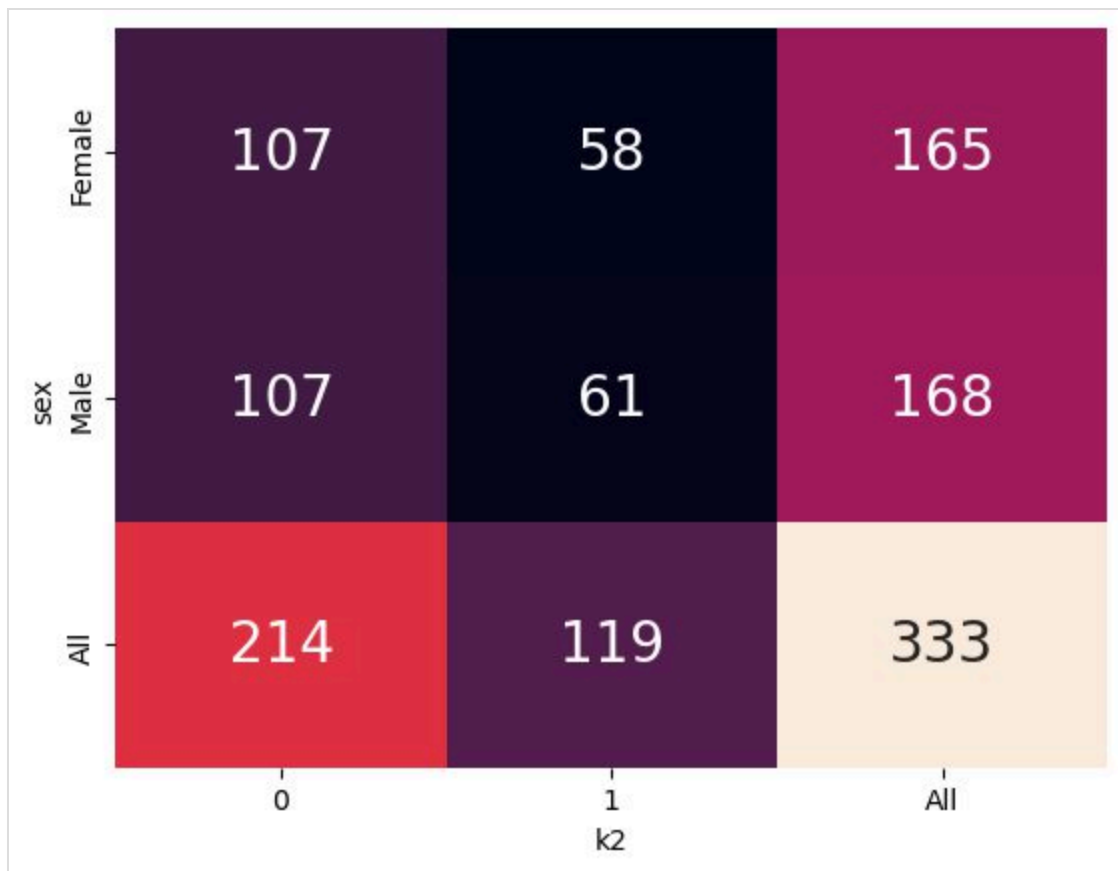


Use a heatmap to visually show how the clusters align with the known groupings.

The heatmap shows that both clusters contain similar numbers of male and female penguins, indicating that the clustering is not strongly driven by sex and that both groups are well represented across this categorical variable.

```
fig, ax = plt.subplots()
sns.heatmap(data=pd.crosstab(df_clean_copy.sex,
                             df_clean_copy.k2,
                             margins=True ),
            annot=True,
            annot_kws={"fontsize": 20},
            fmt='g',
            cbar=False,
            ax=ax)

plt.show()
```



Determine the optimal number of clusters.

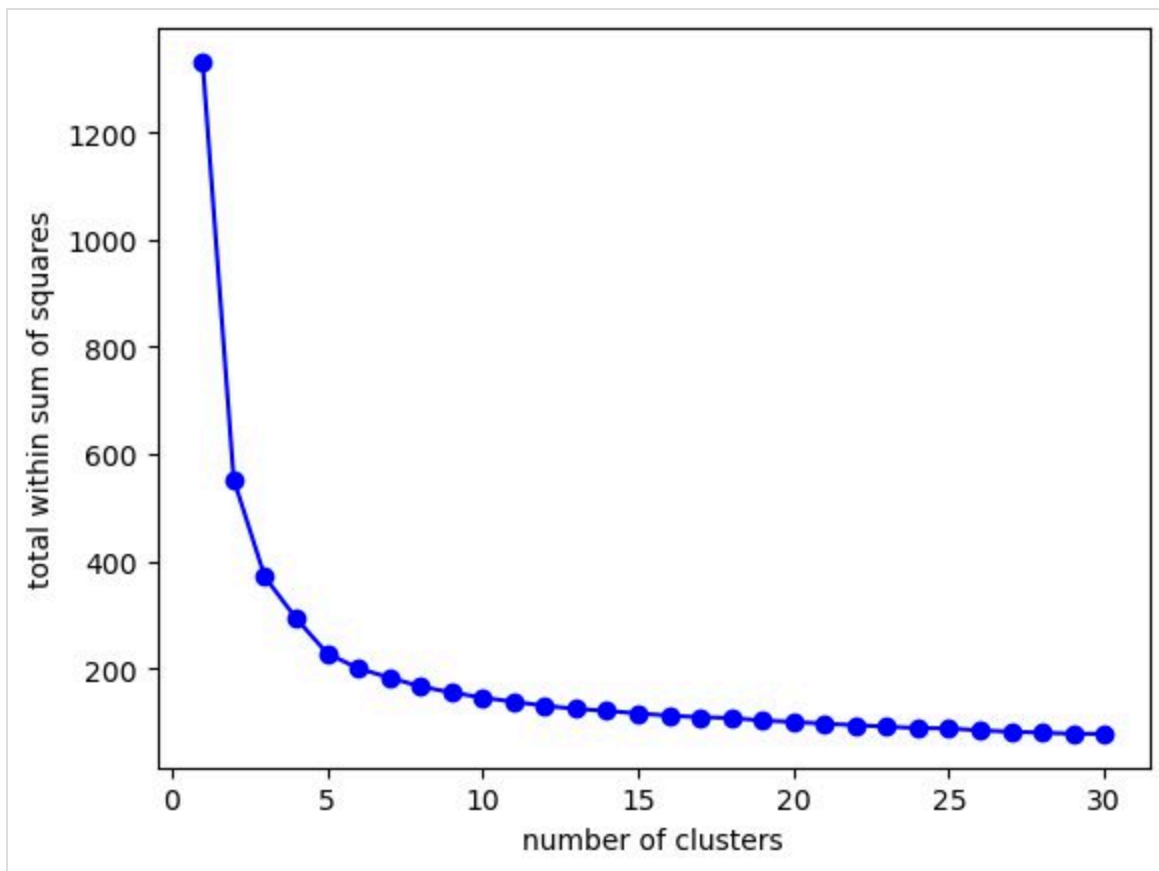
```
tots_within = []
K = range(1, 31)
for k in K:
    km = KMeans(n_clusters=k,
                random_state=42,
                n_init=25,
                max_iter=500)
    km = km.fit(X)
    tots_within.append(km.inertia_)
```

Visualize the optimal number of clusters.

The elbow plot shows a sharp decrease in total within sum of squares as the number of clusters increases, with the curve flattening after about 2–3 clusters, suggesting that two or three clusters are optimal for capturing the main structure in the data.

```
fig, ax = plt.subplots()
ax.plot( K, tots_within, 'bo-' )
ax.set_xlabel('number of clusters')
ax.set_ylabel('total within sum of squares')

plt.show()
```



Use three clusters.

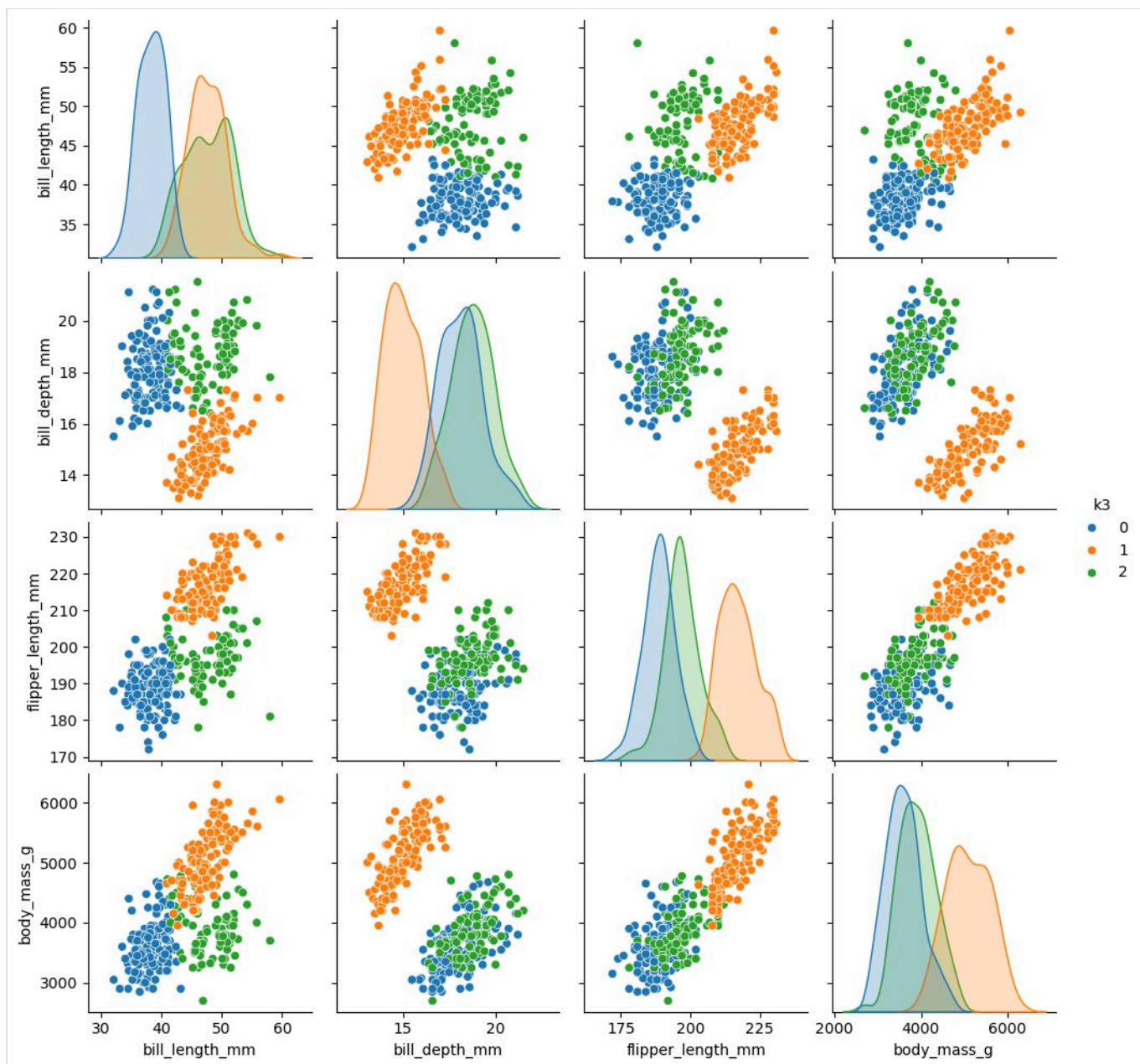
```
clusters_3 = KMeans(n_clusters=3,
                    random_state=42,
                    n_init=25,
                    max_iter=500). \
                    fit_predict(X)
```

```
df_clean_copy['k3'] = pd.Series(clusters_3,
                                index=df_clean_copy.index). \
                                astype('category')
```

Visualize the clusters using a pair plot.

The pair plot with three clusters reveals that the penguins are grouped into three distinct sets based on their physical measurements, with each cluster showing unique combinations of bill length, bill depth, flipper length, and body mass, suggesting the presence of three underlying subpopulations in the dataset.

```
sns.pairplot(data=df_clean_copy,
             hue='k3',
             diag_kws={'common_norm': False})
plt.show()
```



Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean_copy.select_dtypes(exclude='number').nunique()
```

```
species    3
island     3
sex        2
k2         2
k3         3
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

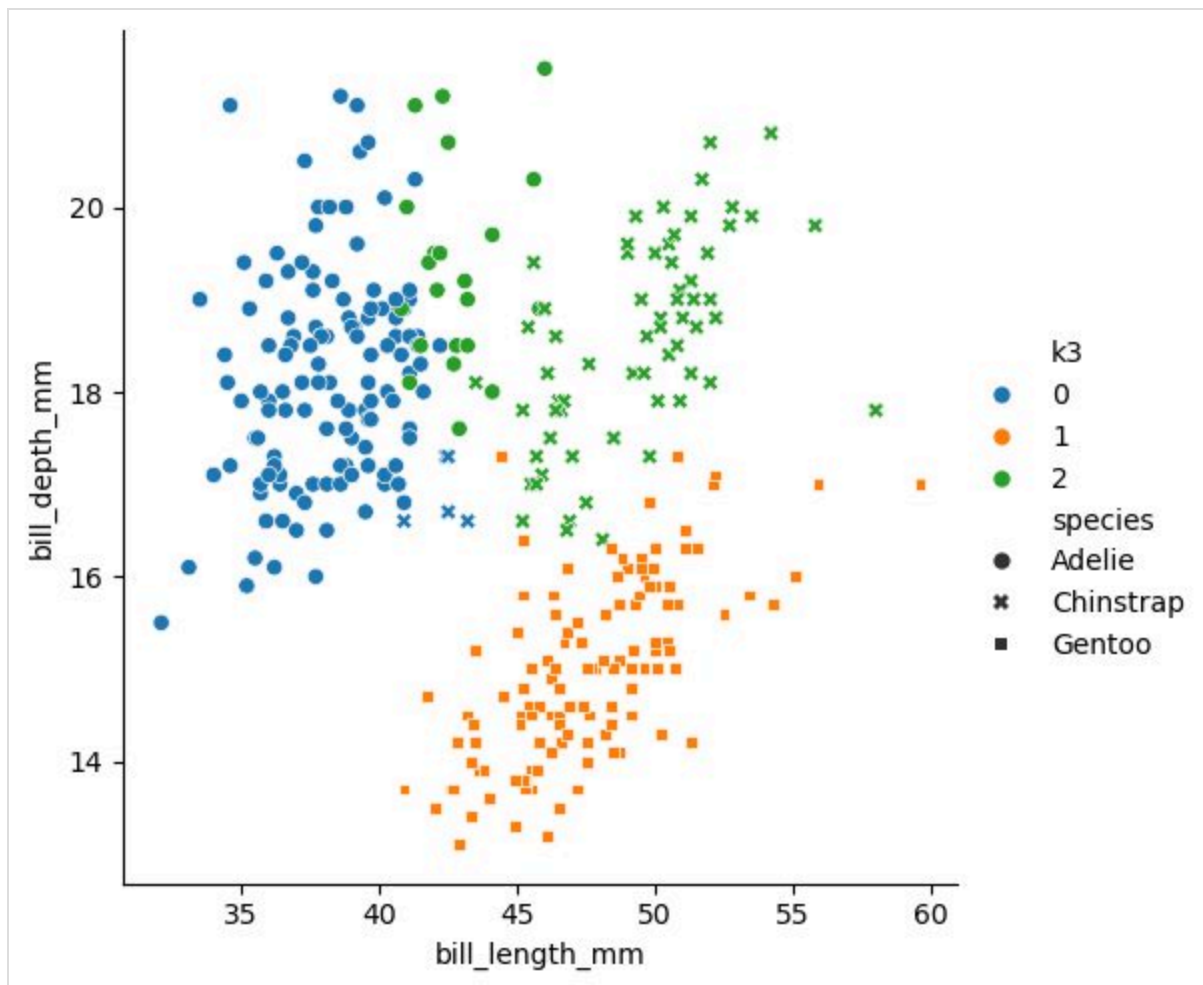
```
df_clean_copy.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Visualize the cluster results on a scatter plot.

This scatter plot shows that the three clusters correspond closely to the three penguin species, with each species occupying a distinct region in the bill length and bill depth space, demonstrating that the clustering effectively captures species-level differences in these physical traits.

```
sns.relplot(data=df_clean_copy,
            x='bill_length_mm',
            y='bill_depth_mm',
            hue='k3',
            style='species')
plt.show()
```

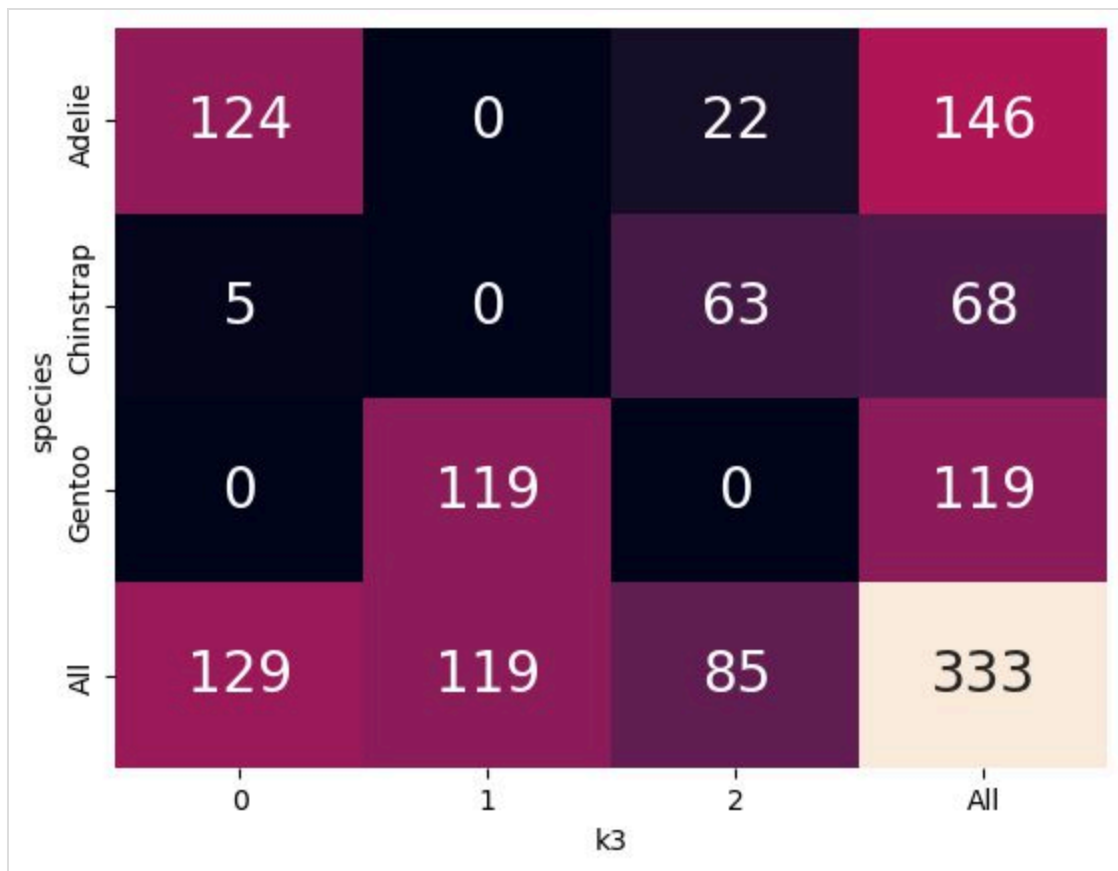



Use a heatmap to visually show how the clusters align with the known groupings.

The heatmap shows a strong correspondence between clusters and species: each cluster almost exclusively contains one species, confirming that the clustering algorithm has successfully separated Adelie, Chinstrap, and Gentoo penguins based on their physical measurements.

```
fig, ax = plt.subplots()
sns.heatmap(data=pd.crosstab(df_clean_copy.species,
                             df_clean_copy.k3,
                             margins=True ),
            annot=True,
            annot_kws={"fontsize": 20},
            fmt='g',
            cbar=False,
            ax=ax)

plt.show()
```



PCA

```
pca_df_clean_copy = PCA(n_components=2). \
    fit_transform(X)
```

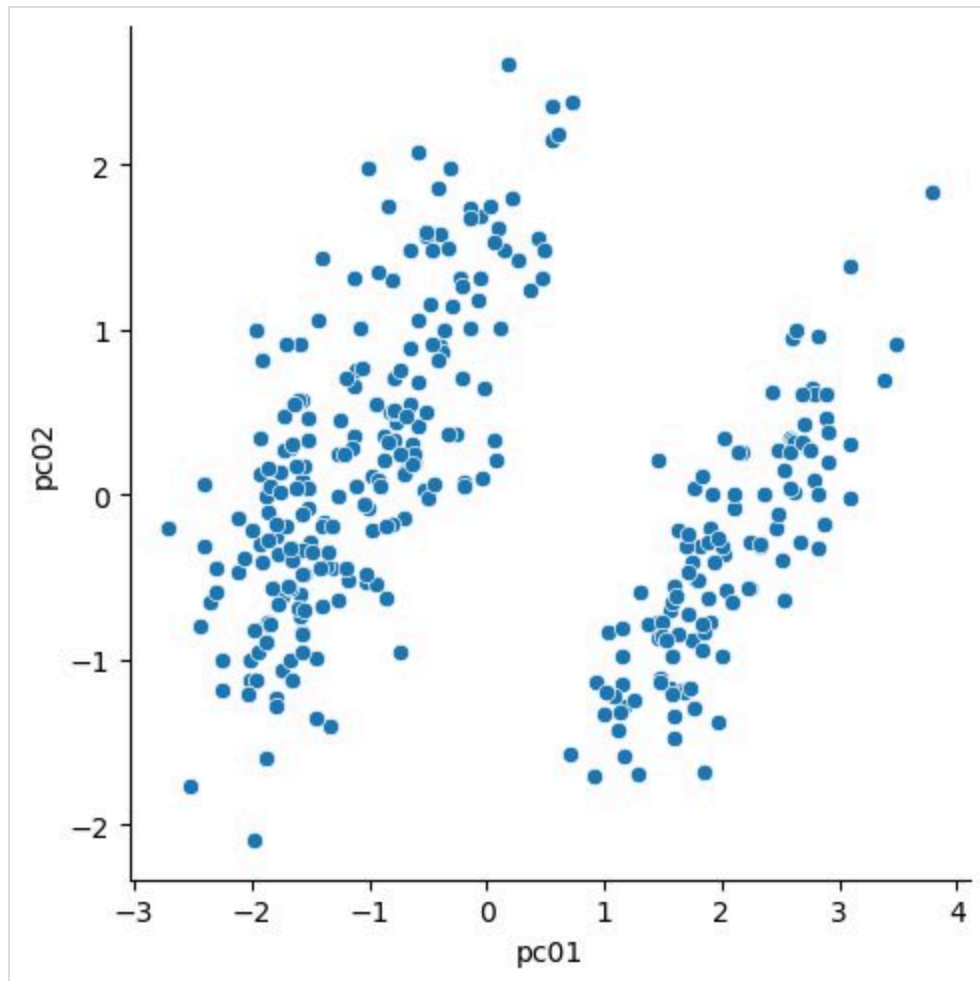
```
pca_df_clean_copy_df = pd.DataFrame(pca_df_clean_copy,
    columns=['pc01',
            'pc02'])
```

```
pca_df_clean_copy_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 333 entries, 0 to 332
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    pc01    333 non-null         float64
1    pc02    333 non-null         float64
dtypes: float64(2)
memory usage: 5.3 KB
```

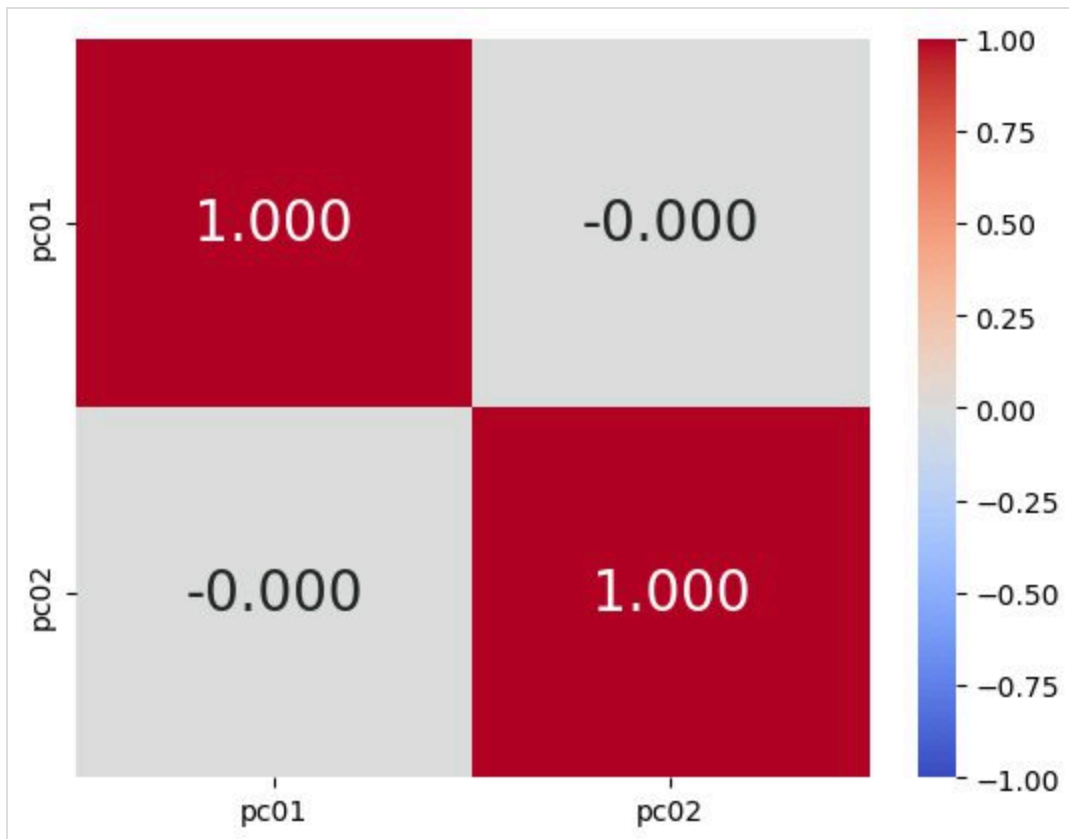

This PCA scatter plot shows that the first two principal components capture clear structure in the data, with the points forming two well-separated groups, indicating strong underlying variation and suggesting that dimensionality reduction effectively reveals major patterns in the penguin measurements.

```
sns.relplot(data=pca_df_clean_copy_df,  
            x='pc01',  
            y='pc02')  
plt.show()
```



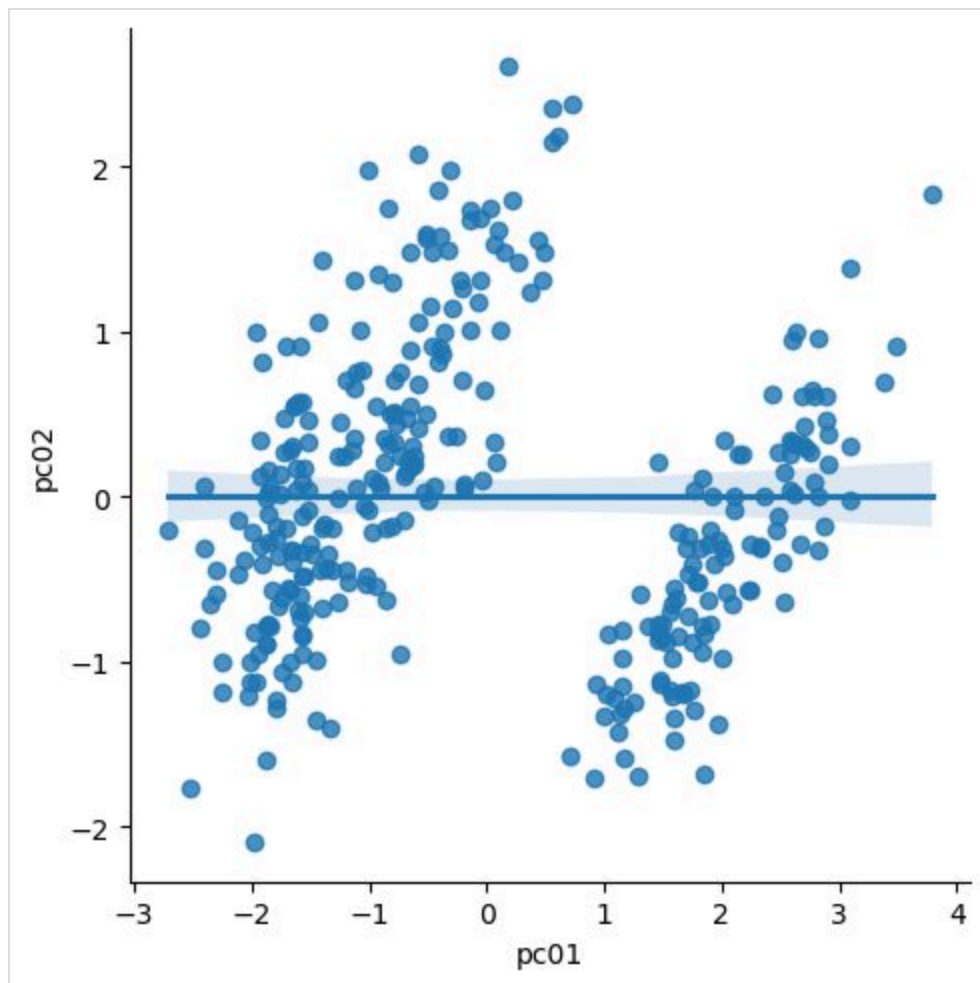
The heatmap of the PCA components' correlation matrix shows that pc01 and pc02 are completely uncorrelated (correlation ≈ 0), confirming that the principal components are orthogonal and capture independent sources of variance in the standardized penguin measurements.

```
fig, ax = plt.subplots()
sns.heatmap(pca_df_clean_copy_df.corr(numeric_only=True),
            vmin=-1,
            vmax=1,
            center=0,
            cmap='coolwarm',
            annot=True,
            annot_kws={'fontsize': 20},
            fmt='.3f',
            ax=ax)
plt.show()
```



This scatter plot with regression line shows that there is no significant linear relationship between the first and second principal components, confirming their statistical independence and the effectiveness of PCA in separating the main sources of variance in the data.

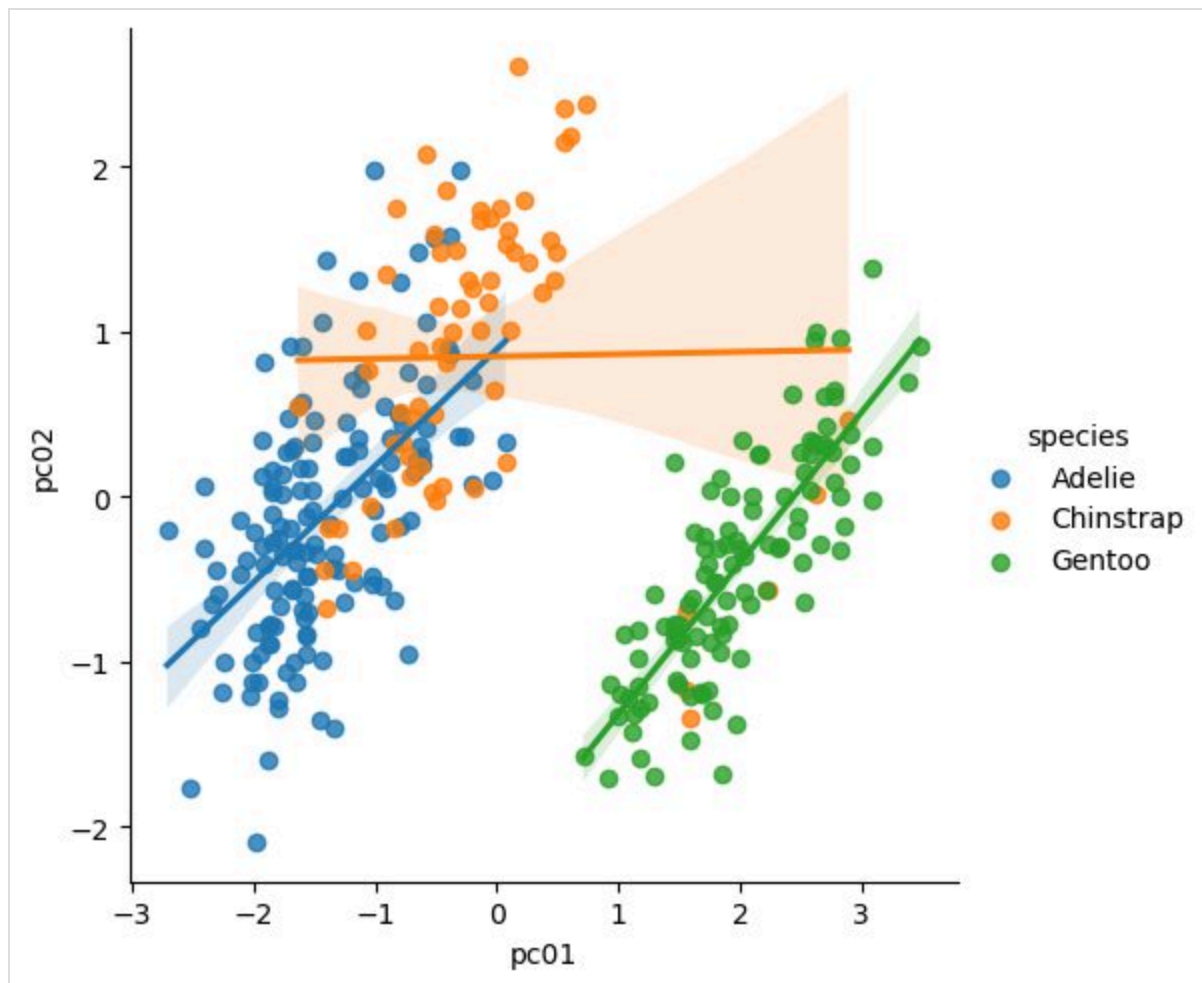
```
sns.lmplot(data=pca_df_clean_copy_df,
           x='pc01',
           y='pc02')
plt.show()
```



The regression lines in this PCA scatter plot show that each penguin species has a distinct linear trend in the principal component space, highlighting species-specific variation and confirming that PCA effectively separates the three species based on their physical measurements.

```
pca_df_clean_copy_df['species'] = df_clean_copy['species']
```

```
sns.lmplot(data=pca_df_clean_copy_df,  
           x='pc01',  
           y='pc02',  
           hue='species')  
plt.show()
```



Clustering & PCA

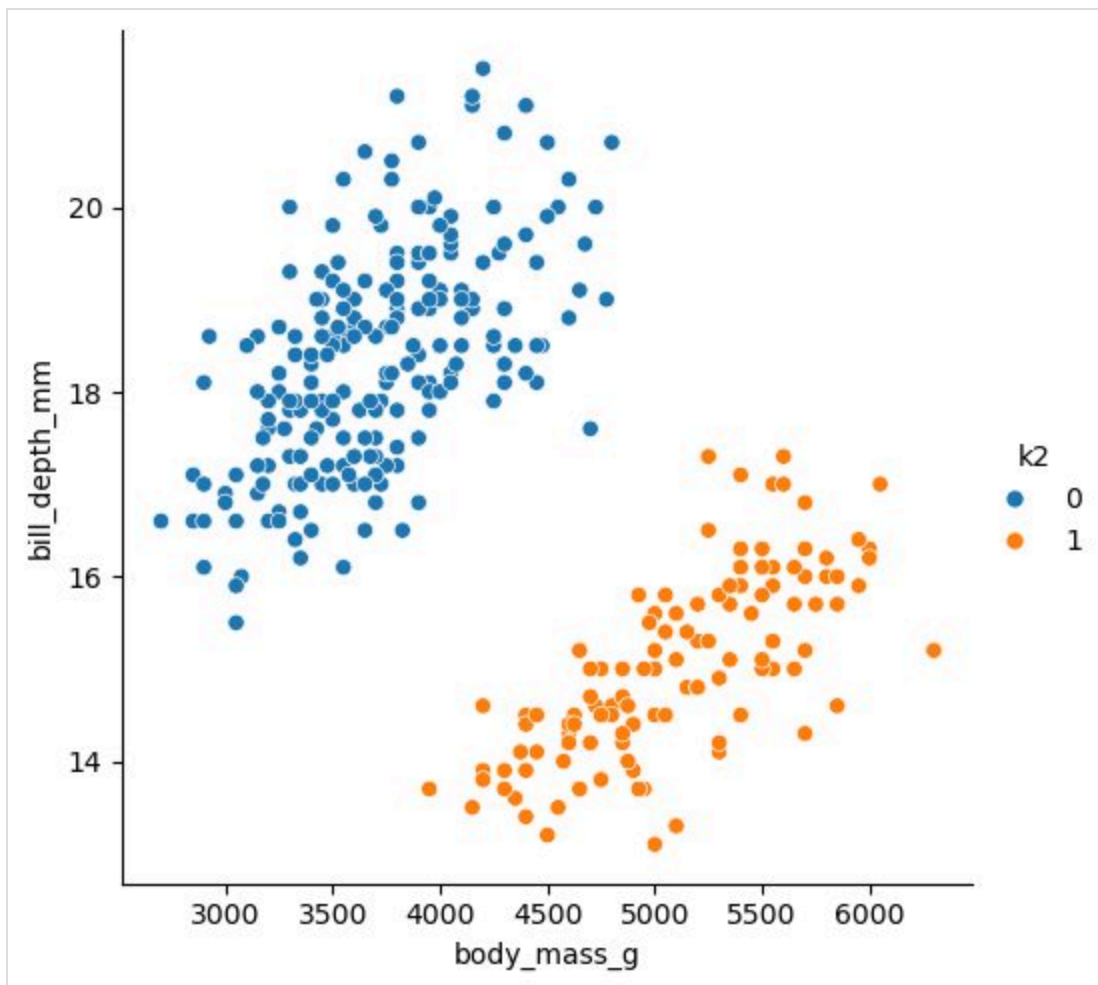
Both scatter plots demonstrate that the KMeans clustering algorithm effectively separates the penguins into two distinct groups, whether visualized in the original feature space (body mass vs. bill depth) or in the reduced PCA space (pc01 vs. pc02). In both cases, the clusters are well separated, indicating that the underlying physical measurements and their principal components capture strong, non-overlapping patterns in the data. The PCA plot further confirms that dimensionality reduction preserves the cluster structure, with each group occupying a distinct region in the principal component space, validating the consistency and robustness of the clustering results across different representations of the data.

```
pca_df_clean_copy_df['k2'] = pd.Series(clusters_2,
                                       index=pca_df_clean_copy_df. \
                                       index). \
                                       astype('category')
```

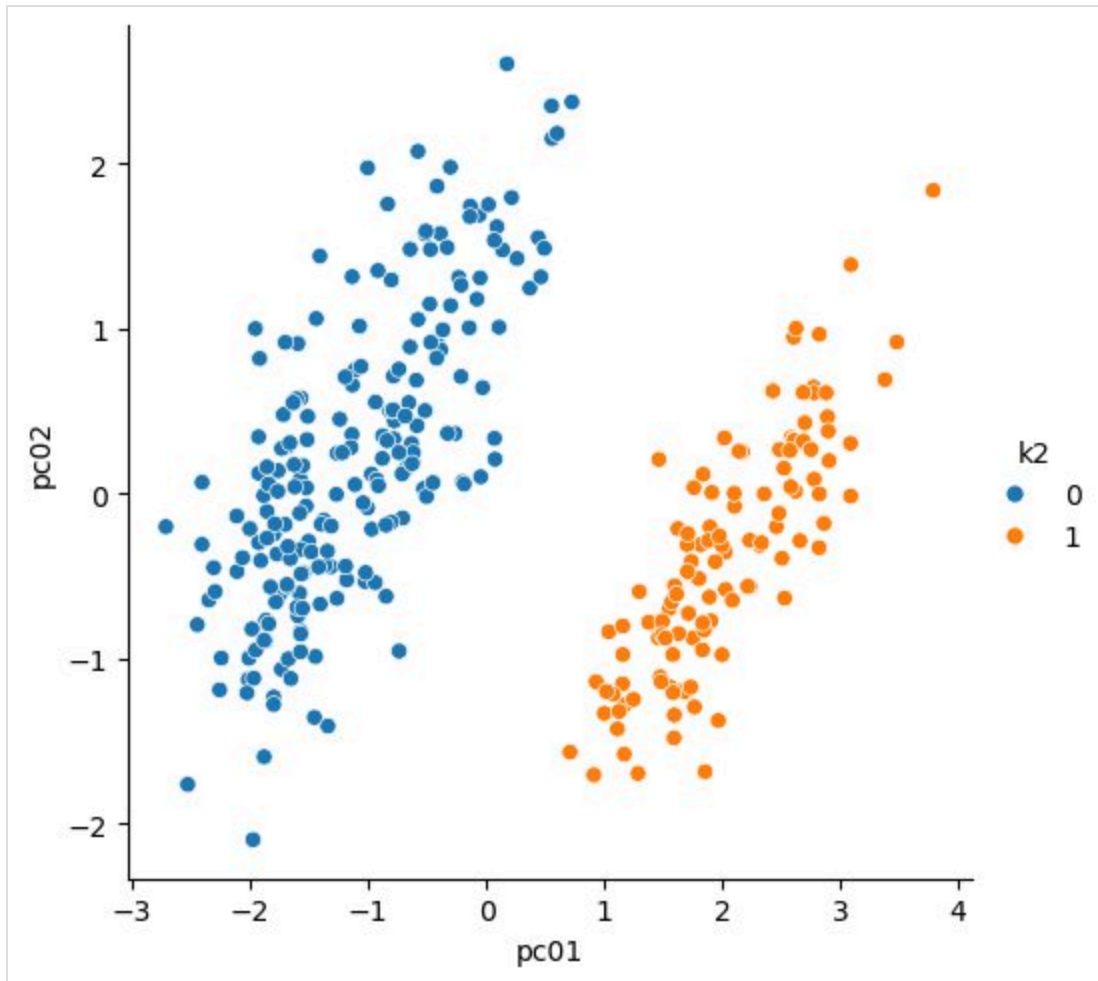
```
pca_df_clean_copy_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 333 entries, 0 to 332  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype    
---  ---        
0    pc01        333 non-null    float64  
1    pc02        333 non-null    float64  
2    species     324 non-null    object    
3    k2          333 non-null    category  
dtypes: category(1), float64(2), object(1)  
memory usage: 8.4+ KB
```

```
sns.relplot(data=df_clean_copy,  
            x='body_mass_g',  
            y='bill_depth_mm',  
            hue='k2')  
plt.show()
```



```
sns.relplot(data=pca_df_clean_copy_df,  
            x='pc01',  
            y='pc02',  
            hue='k2')  
plt.show()
```



Part II: Simple Linear Regression

03-SLR-LR: Penguins Dataset Simple Linear Regression - Linear Relationship

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```


Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island        0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g   2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island        0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex           0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island        3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm  54
body_mass_g   93
sex           2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Linear Relationship

We will start with a single continuous input that is linearly related to the trend. In our case we will attempt to use `flipper_length_mm` to predict `body_mass_g`.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

Visualize the Trend

The trend is increasing as the input increases because we have a positive **slope**.

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

```
my_slope, my_intercept = np.polyfit(x,
                                     y,
                                     1)
```

```
my_slope
```

```
50.153265942241134
```

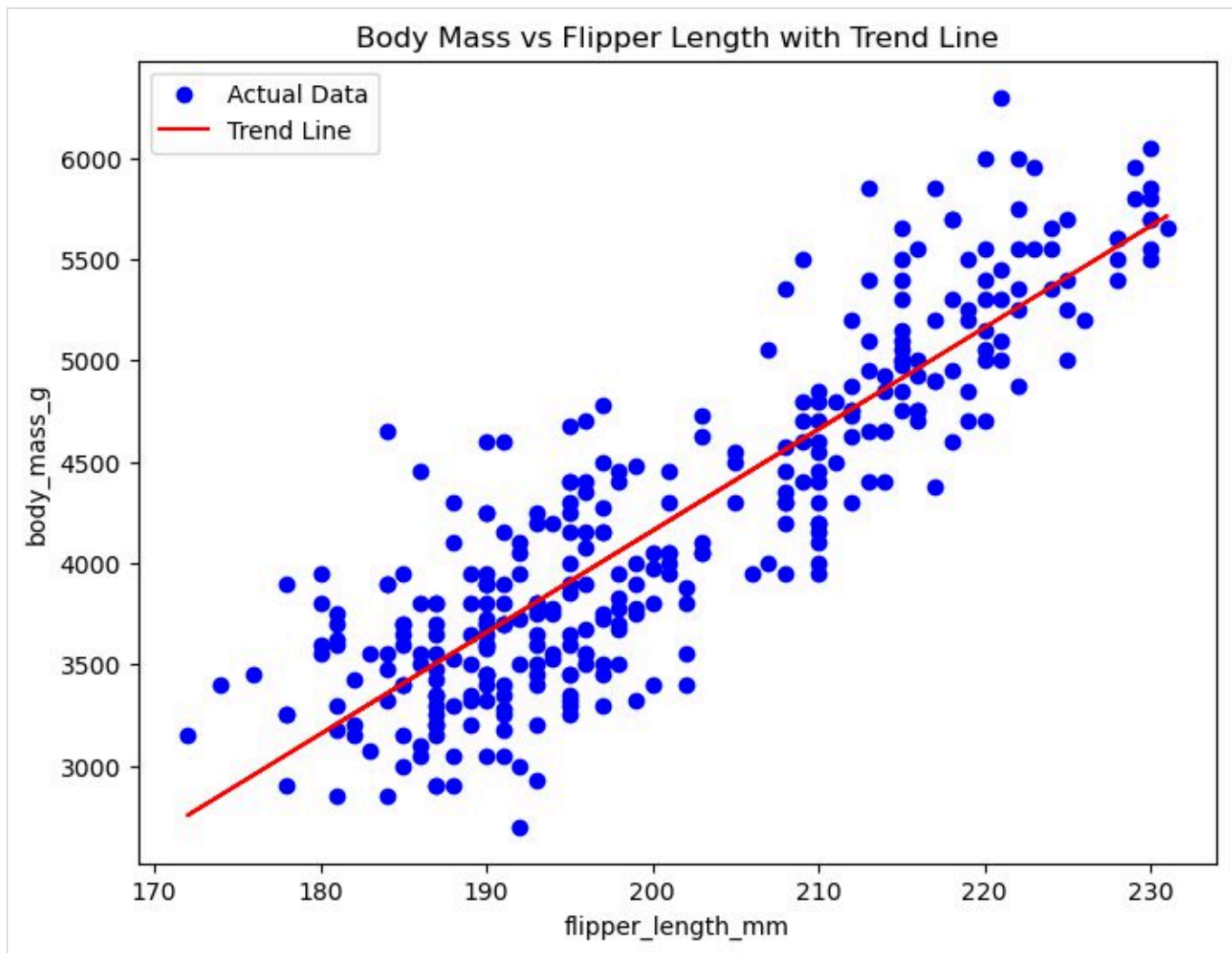
```
my_intercept
```

```
-5872.09268284282
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \  
                  my_slope * \  
                  df_viz['flipper_length_mm']
```

```
plt.figure(figsize=(8,6))  
plt.scatter(df_viz['flipper_length_mm'],  
            df_viz['body_mass_g'],  
            color='blue',  
            label='Actual Data')  
plt.plot(df_viz['flipper_length_mm'],  
         df_viz['trend'],  
         color='red',  
         label='Trend Line')  
plt.xlabel('flipper_length_mm')  
plt.ylabel('body_mass_g')  
plt.title('Body Mass vs Flipper Length with Trend Line')  
plt.legend()  
plt.show()
```



Calculating σ

The likelihood of an output around the trend is a Gaussian distribution and the variation of a Gaussian is specified by the **standard deviation** or σ .

To calculate **sigma** (σ), the **standard deviation** of the residuals in linear regression, use the following formula.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2}$$

where:

- y_i is the actual observed value for the i th point,
- μ_i is the predicted value for the i th point (from your regression line),
- n is the number of data points.

Step-by-step explanation.

1. **Calculate the residuals:** $y_i - \mu_i$.
2. **Square the residuals:** $(y_i - \mu_i)^2$.
3. **Sum squared residuals:** $\sum_{i=1}^n (y_i - \mu_i)^2$.
4. **Divide by the number of points:** $\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2$.
5. **Take the square root:** $\sqrt{\cdot}$.

If you want the **sample standard deviation** (which is common), use $n - 1$ in the denominator.

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - \mu_i)^2}$$

σ measures how much your actual values deviate from the values predicted by the regression line.

```
my_residuals = df_viz['body_mass_g'] - df_viz['trend']
```

```
my_sigma = np.sqrt(np.sum(my_residuals**2) / (len(my_residuals) - 1))
```

```
my_sigma
```

```
392.7504293927682
```

Visualize the variation around the trend or uncertainty of a single output measurement around the average output.

```
df_viz['obs_lwr_68'] = df_viz.trend - my_sigma
df_viz['obs_upr_68'] = df_viz.trend + my_sigma
```

```
df_viz['obs_lwr_95'] = df_viz.trend - 2 * my_sigma
df_viz['obs_upr_95'] = df_viz.trend + 2 * my_sigma
```

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

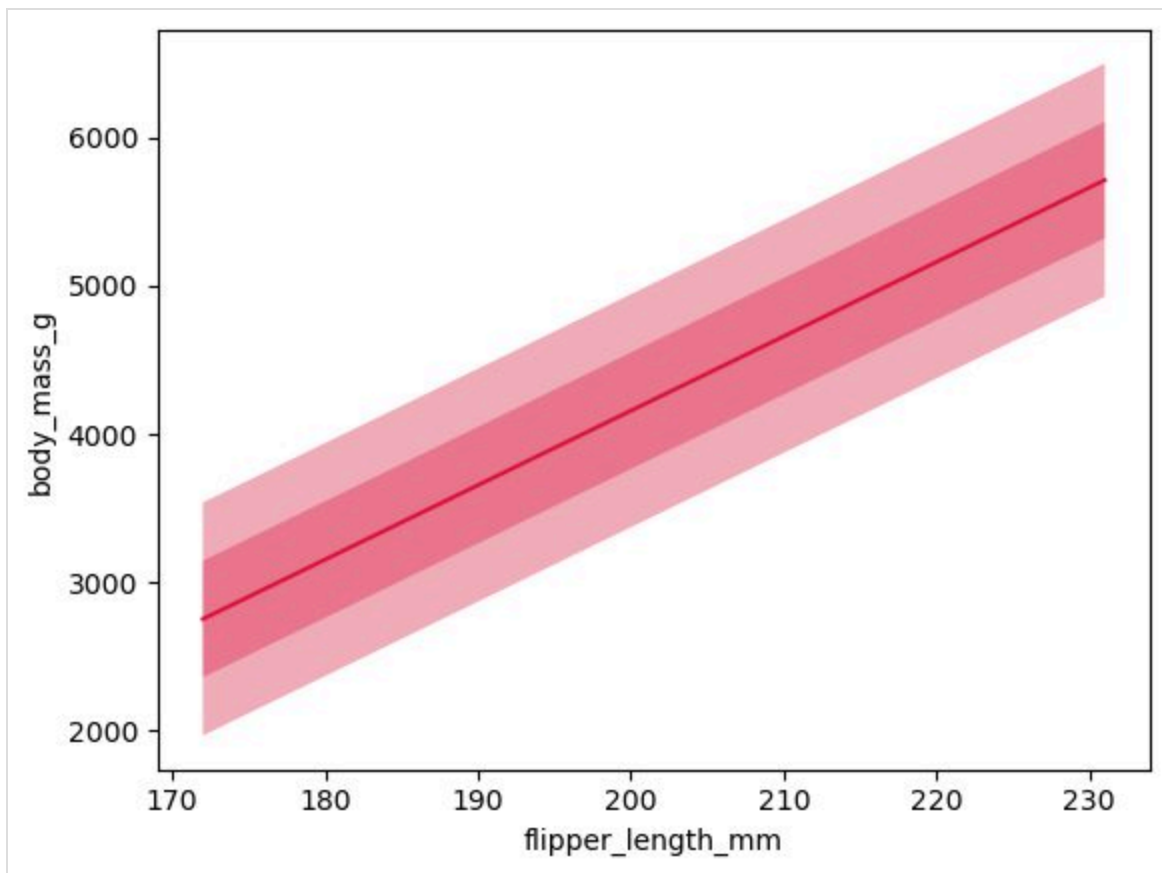
# Create the figure and axis
fig, ax = plt.subplots()

# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_68,
                df_viz_sorted.obs_upr_68,
                facecolor='crimson',
                alpha=0.35)

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```



Calculate the trend given the input values.

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

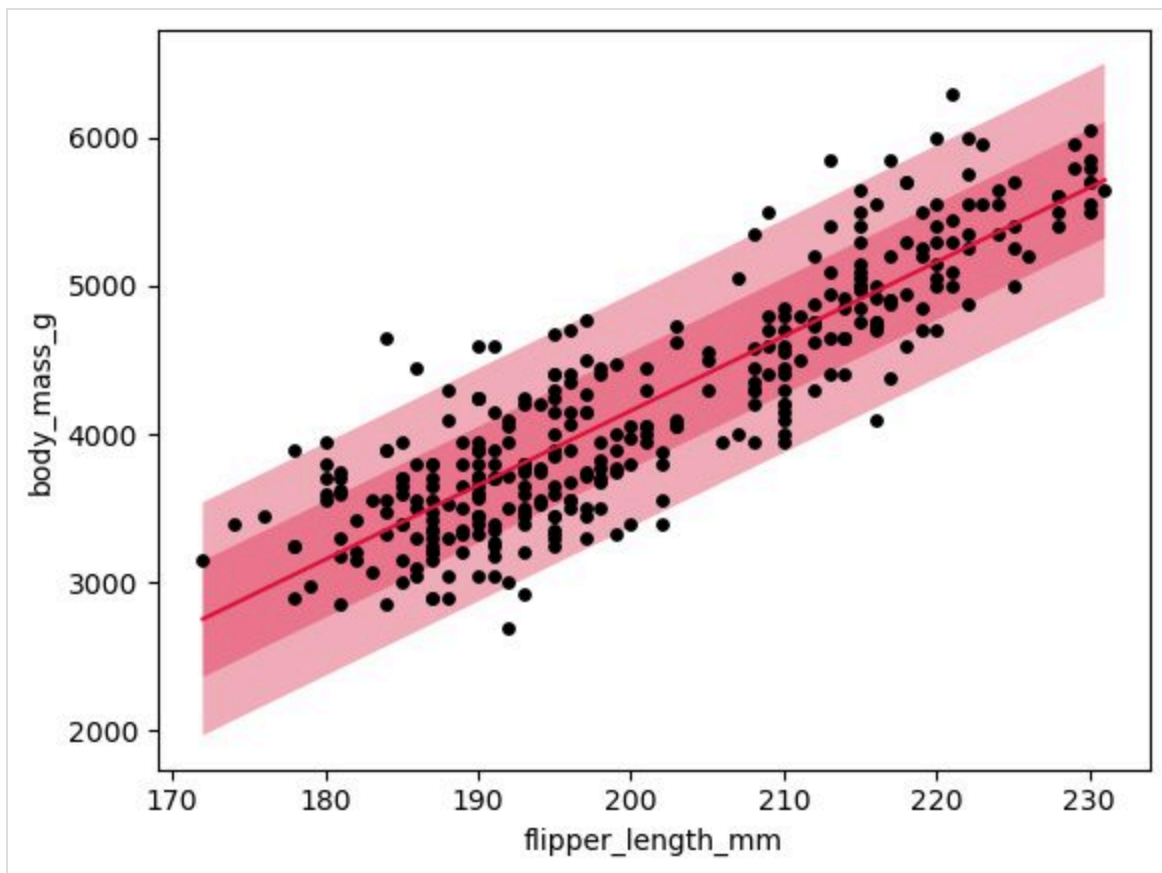
# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_68,
                df_viz_sorted.obs_upr_68,
                facecolor='crimson',
                alpha=0.35)

# Include the randomly generated small data set as a scatter plot
ax.scatter(df.flipper_length_mm,
            df.body_mass_g,
            s=15,
            color='k')

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```

04-SLR-NLR: Penguins Dataset Simple Linear Regression - Non-Linear Relationship

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species          0
island           0
bill_length_mm   0
bill_depth_mm    0
flipper_length_mm 0
body_mass_g      0
sex              0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species          3
island           3
bill_length_mm   163
bill_depth_mm    79
flipper_length_mm 54
body_mass_g      93
sex              2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Non-Linear Relationship

We will start with a single continuous input that is non-linearly related to the trend. In our case we will attempt to use `flipper_length_mm` to predict `body_mass_g`.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ predicted trend

β_0 intercept: value of μ when $x = 0$

β_1 linear effect: change in μ for a one-unit increase in x

β_2 quadratic effect: change in μ for a one-unit increase in x^2

x feature or input variable: independent variable used to predict μ

Visualize the Trend

The trend is increasing as the input increases because we have a positive quadratic **slope**, but the rate of change is not constant.

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

```
my_quadratic, my_slope, my_intercept = np.polyfit(x,
                                                    y,
                                                    2)
```

```
my_quadratic
```

```
0.5449477765724435
```

```
my_slope
```

```
-171.61397647827607
```

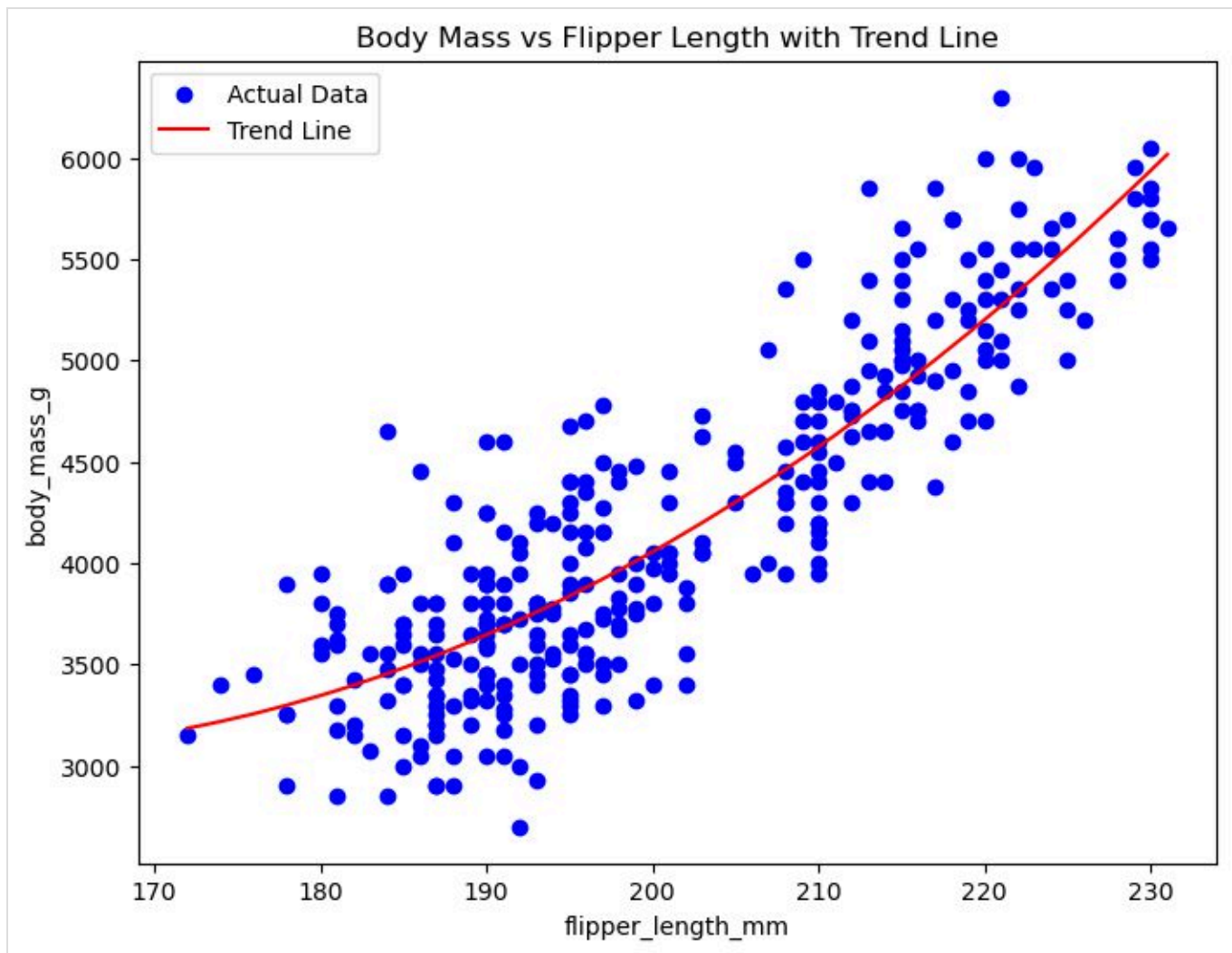
```
my_intercept
```

```
16579.869214753533
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \  
    my_slope * df_viz['flipper_length_mm'] + \  
    my_quadratic * df_viz['flipper_length_mm'] \  
    **2
```

```
sorted_df = df_viz.sort_values('flipper_length_mm')  
plt.figure(figsize=(8,6))  
plt.scatter(df_viz['flipper_length_mm'],  
            df_viz['body_mass_g'],  
            color='blue',  
            label='Actual Data')  
plt.plot(sorted_df['flipper_length_mm'],  
         sorted_df['trend'],  
         color='red',  
         label='Trend Line')  
plt.xlabel('flipper_length_mm')  
plt.ylabel('body_mass_g')  
plt.title('Body Mass vs Flipper Length with Trend Line')  
plt.legend()  
plt.show()
```



Calculating σ

The likelihood of an output around the trend is a Gaussian distribution and the variation of a Gaussian is specified by the **standard deviation** or σ .

To calculate **sigma** (σ), the **standard deviation** of the residuals in linear regression, use the following formula.

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2}$$

where:

- y_i is the actual observed value for the i th point,
- μ_i is the predicted value for the i th point (from your regression line),
- n is the number of data points.

Step-by-step explanation.

1. **Calculate the residuals:** $y_i - \mu_i$.
2. **Square the residuals:** $(y_i - \mu_i)^2$.
3. **Sum squared residuals:** $\sum_{i=1}^n (y_i - \mu_i)^2$.
4. **Divide by the number of points:** $\frac{1}{n} \sum_{i=1}^n (y_i - \mu_i)^2$.
5. **Take the square root:** $\sqrt{\cdot}$.

If you want the **sample standard deviation** (which is common), use $n - 1$ in the denominator.

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - \mu_i)^2}$$

σ measures how much your actual values deviate from the values predicted by the regression line.

```
my_residuals = df_viz['body_mass_g'] - df_viz['trend']
```

```
my_sigma = np.sqrt(np.sum(my_residuals**2) / (len(my_residuals) - 1))
```

```
my_sigma
```

```
379.3340514016102
```

Visualize the variation around the trend or uncertainty of a single output measurement around the average output.

```
df_viz['obs_lwr_68'] = df_viz.trend - my_sigma
df_viz['obs_upr_68'] = df_viz.trend + my_sigma
```

```
df_viz['obs_lwr_95'] = df_viz.trend - 2 * my_sigma
df_viz['obs_upr_95'] = df_viz.trend + 2 * my_sigma
```

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

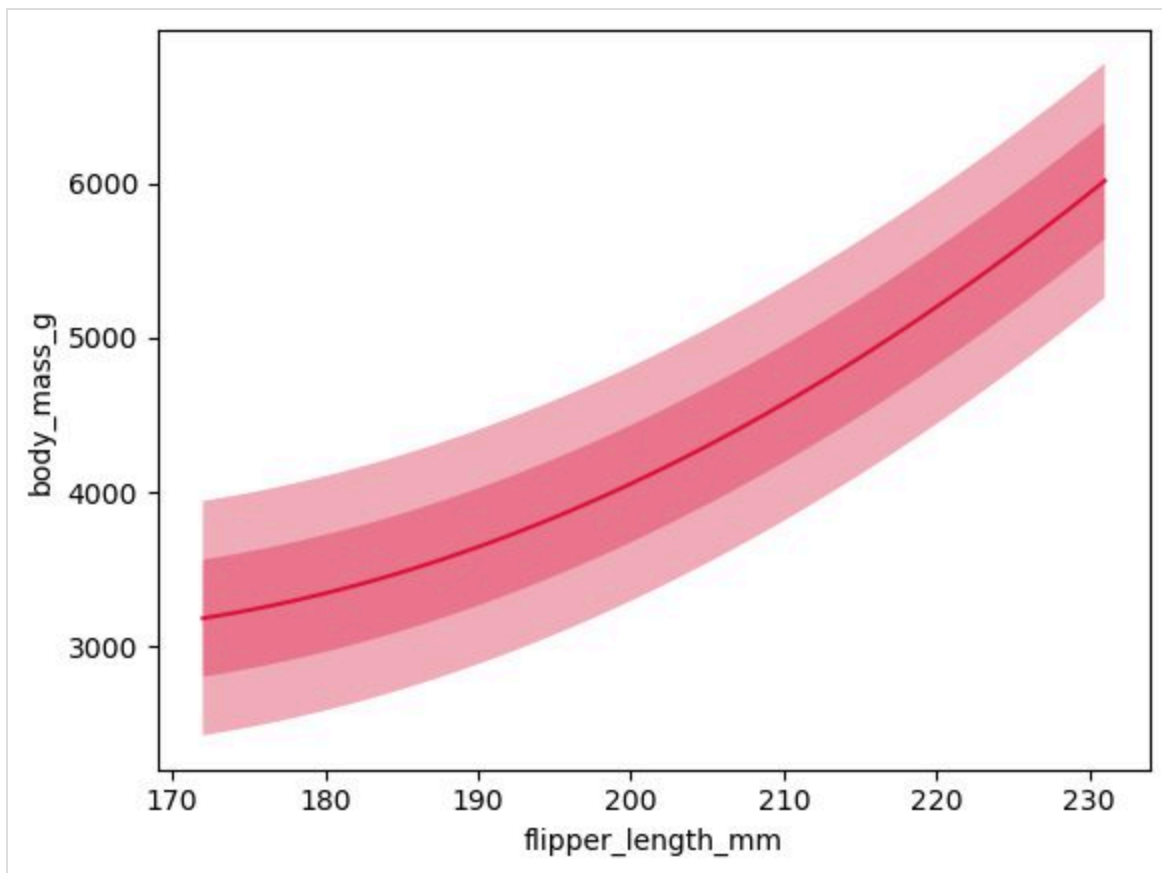
# Create the figure and axis
fig, ax = plt.subplots()

# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_68,
                df_viz_sorted.obs_upr_68,
                facecolor='crimson',
                alpha=0.35)

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

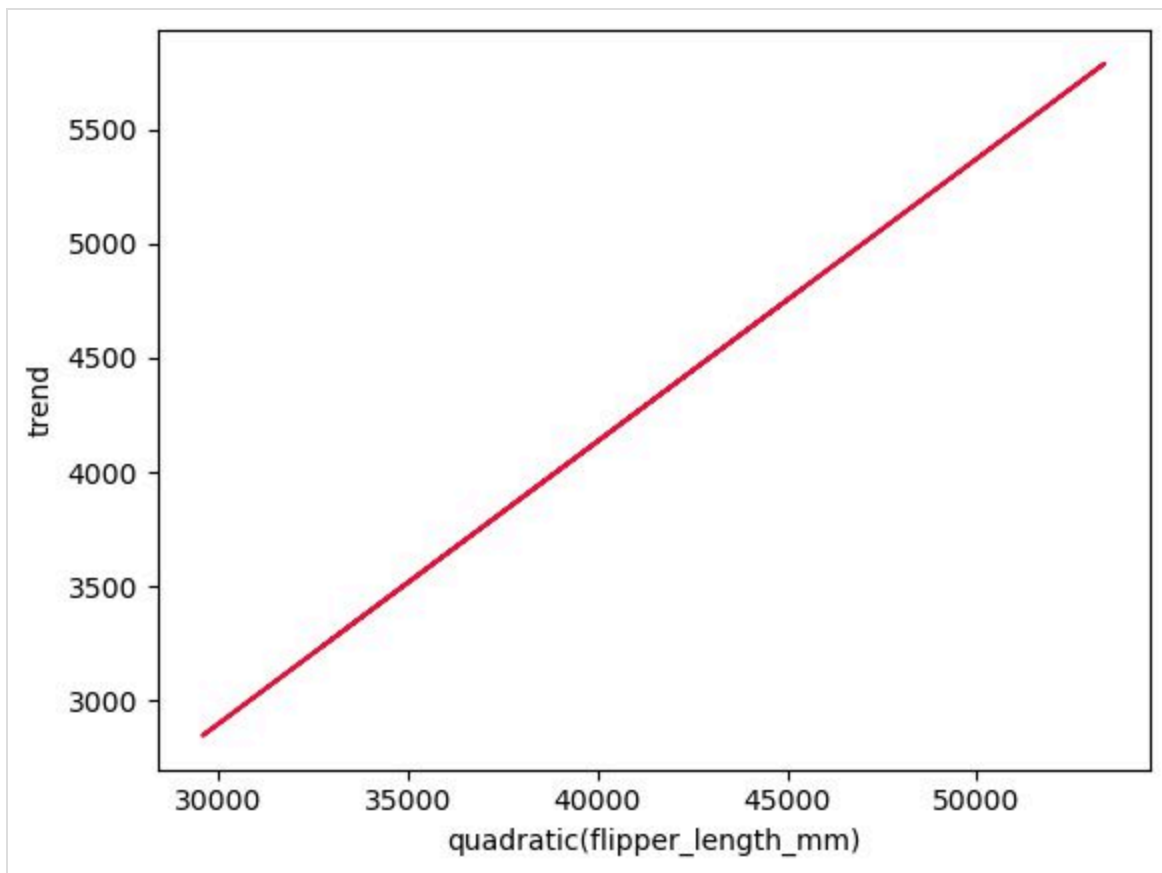
# Show the plot
plt.show()
```

There is a linear relationship between the trend and the non-linear feature.

```
x_quad = np.power(df_viz.flipper_length_mm,
                  2)
slope, intercept = np.polyfit(x_quad,
                              y,
                              1)

trend = slope * \
        x_quad + \
        intercept
fig, ax = plt.subplots()
ax.plot(x_quad,
        trend,
        color='crimson')
ax.set_xlabel('quadratic(flipper_length_mm)')
ax.set_ylabel('trend')
plt.show()
```



Calculate the trend given the input values.

```
# Sort by flipper_length_mm for continuous plotting
df_viz_sorted = df_viz.sort_values('flipper_length_mm')

# Create the figure and axis
fig, ax = plt.subplots()

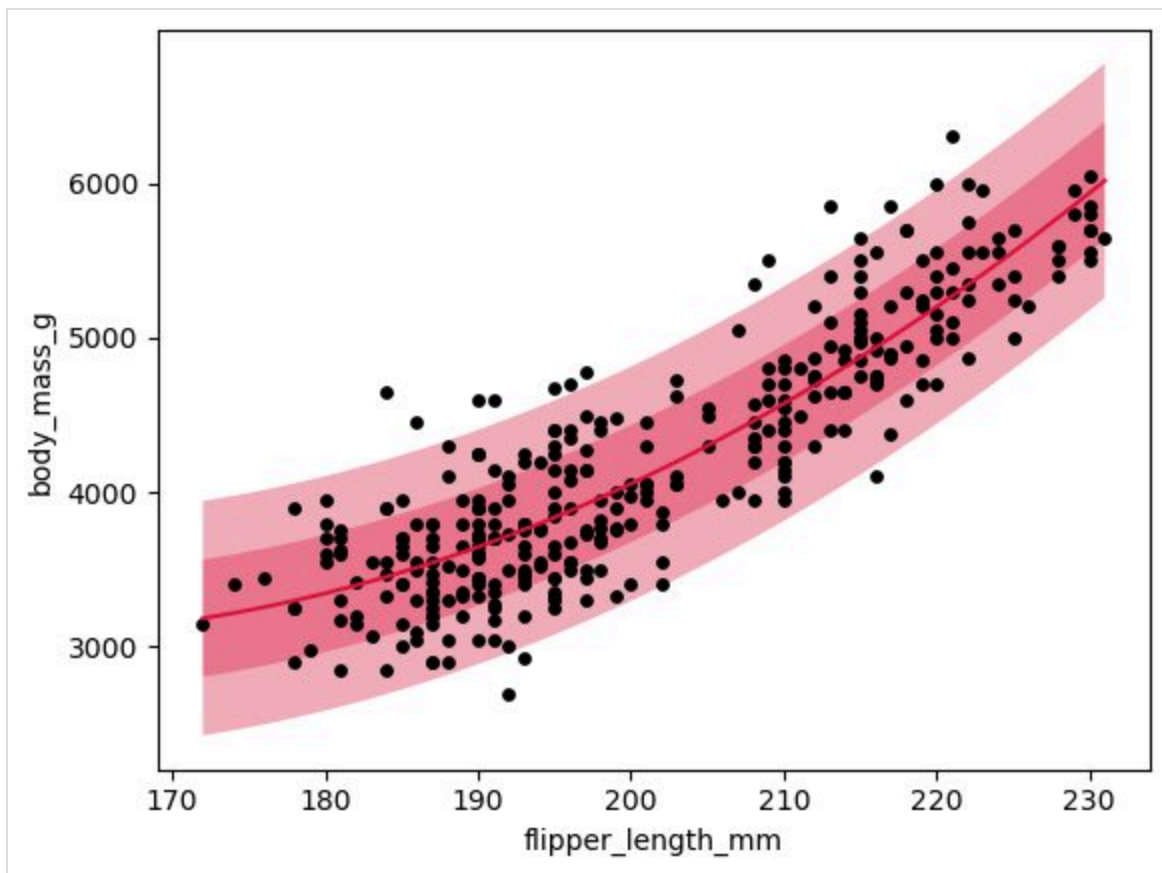
# True trend
ax.plot(df_viz_sorted.flipper_length_mm,
        df_viz_sorted.trend,
        color='crimson',
        linewidth=1.5)

# True variation around the trend showing 2 sigma intervals
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_95,
                df_viz_sorted.obs_upr_95,
                facecolor='crimson',
                alpha=0.35)
ax.fill_between(df_viz_sorted.flipper_length_mm,
                df_viz_sorted.obs_lwr_68,
                df_viz_sorted.obs_upr_68,
                facecolor='crimson',
                alpha=0.35)

# Include the randomly generated small data set as a scatter plot
ax.scatter(df.flipper_length_mm,
           df.body_mass_g,
           s=15,
           color='k')

# Set labels
ax.set_xlabel('flipper_length_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```



05-SLR-FLF: Penguins Dataset Simple Linear Regression - Fitting Linear Feature

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species          0
island           0
bill_length_mm   0
bill_depth_mm    0
flipper_length_mm 0
body_mass_g      0
sex              0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species          3
island           3
bill_length_mm   163
bill_depth_mm    79
flipper_length_mm 54
body_mass_g      93
sex              2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g      R-squared:                0.762
Model:                  OLS              Adj. R-squared:         0.761
Method:                 Least Squares    F-statistic:             1060.
Date:                   Tue, 26 Aug 2025  Prob (F-statistic):       3.13e-105
Time:                   19:46:43         Log-Likelihood:          -2461.1
No. Observations:       333             AIC:                   4926.
Df Residuals:           331             BIC:                   4934.
Df Model:                1
Covariance Type:        nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept              -5872.0927    310.285    -18.925    0.000    -6482.472    -5261.713
flipper_length_mm       50.1533         1.540     32.562    0.000      47.123      53.183
=====
Omnibus:                 5.922    Durbin-Watson:           2.102
Prob(Omnibus):           0.052    Jarque-Bera (JB):         5.876
Skew:                    0.325    Prob(JB):                 0.0530
Kurtosis:                3.025    Cond. No.                  2.90e+03
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.9e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
"""
```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```
Intercept          -5872.092683
flipper_length_mm   50.153266
dtype: float64
```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.

- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.
- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, -5872.09 and 50.15, respectively.
- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept          310.285155
flipper_length_mm   1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept          -6492.662994
flipper_length_mm   47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept          -5251.522372
flipper_length_mm   53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	-6482.472236	-5261.713130
flipper_length_mm	47.123390	53.183142

The P-values for the coefficients are used to identify the statistical significance of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**

```

def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolord=ecolord,
        elinewidth=elinewidth,
        ms=ms)

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

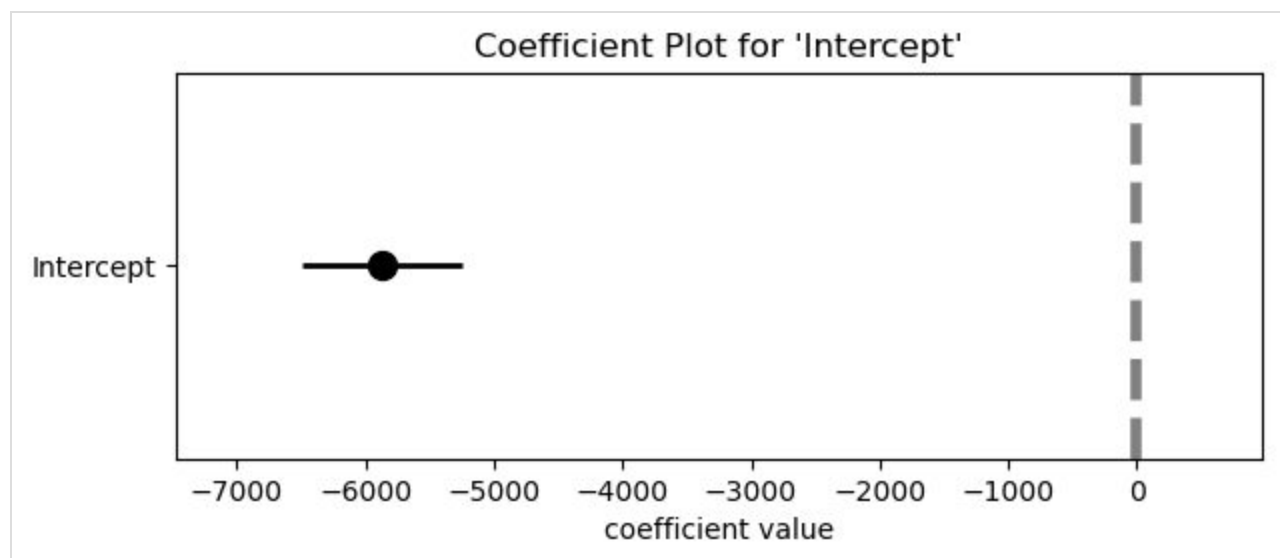
```

```
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

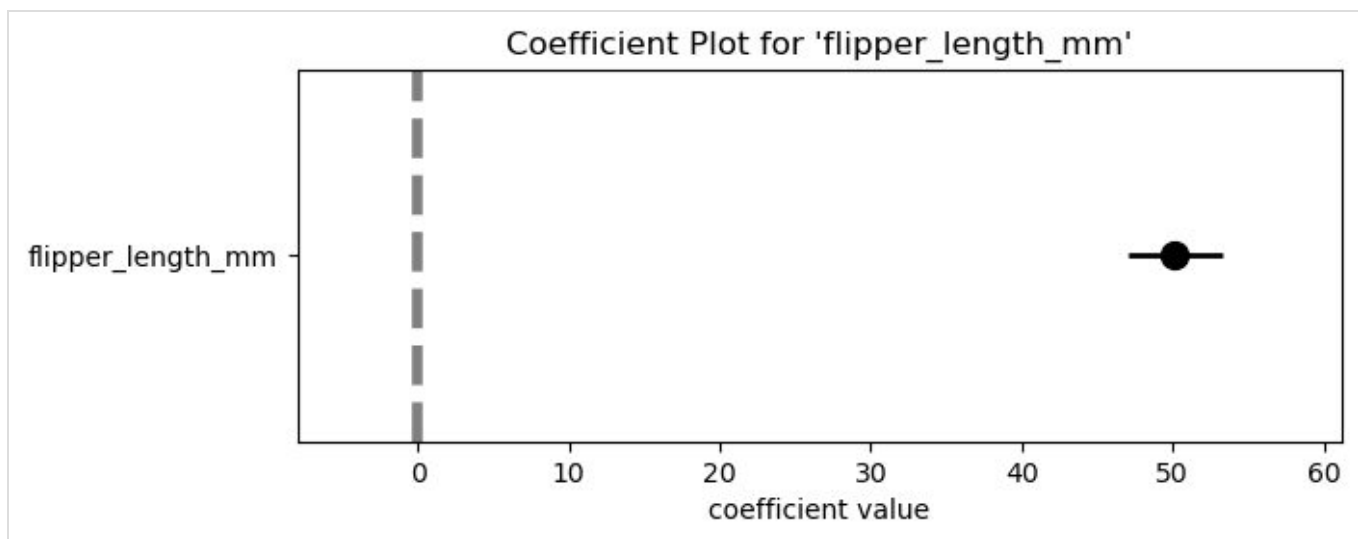
# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

# Show the plot
plt.show()
```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='flipper_length_mm')
```



06-SLR-FNLF: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species          0
island           0
bill_length_mm    0
bill_depth_mm     0
flipper_length_mm 0
body_mass_g       0
sex              0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species          3
island           3
bill_length_mm   163
bill_depth_mm    79
flipper_length_mm 54
body_mass_g      93
sex              2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Non-Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a non-linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **linear effect**: change in μ for a one-unit increase in x

β_2 **quadratic effect**: change in μ for a one-unit increase in x^2

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm + I(flipper_length_mm ** 2)',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```

<class 'statsmodels.iolib.summary.Summary'>
"""
                                OLS Regression Results
=====
Dep. Variable:                body_mass_g    R-squared:                0.778
Model:                        OLS            Adj. R-squared:           0.777
Method:                       Least Squares   F-statistic:             578.5
Date:                         Tue, 26 Aug 2025  Prob (F-statistic):    1.34e-108
Time:                         10:12:55        Log-Likelihood:          -2449.5
No. Observations:             333            AIC:                    4905.
Df Residuals:                 330            BIC:                    4916.
Df Model:                      2
Covariance Type:              nonrobust
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
-----
--
Intercept                    1.658e+04   4616.247      3.592    0.000    7498.887
2.57e+04
flipper_length_mm           -171.6140    45.524     -3.770    0.000   -261.169
-82.059
I(flipper_length_mm ** 2)    0.5449      0.112      4.874    0.000      0.325
0.765
=====
Omnibus:                     10.746    Durbin-Watson:           2.214
Prob(Omnibus):                0.005    Jarque-Bera (JB):        10.985
Skew:                         0.441    Prob(JB):                0.00412
Kurtosis:                     3.119    Cond. No.                9.07e+06
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 9.07e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **intercept** is 16,579.87, which represents the baseline predicted body mass when both flipper length and its square are zero.
- The **coefficient** for `flipper_length_mm` is -171.61, indicating that for every additional millimeter in flipper length, body mass decreases by about 171.61 grams, before considering the quadratic effect.
- The **coefficient for the squared term**, `I(flipper_length_mm ** 2)`, is 0.54, meaning that for every unit increase in the square of flipper length, body mass increases by 0.54 grams, introducing an upward-curving effect.
- The presence of the positive squared term (0.54) shows that the relationship between flipper length and body mass is quadratic and curves upward, rather than being strictly linear.


```
lm_fit.params
```

```
Intercept                16579.869215
flipper_length_mm        -171.613976
I(flipper_length_mm ** 2)    0.544948
dtype: float64
```

The standard error on the estimate is contained in the `.bse` attribute.

- The standard error for the intercept is 4,616.25, indicating the level of uncertainty in estimating the baseline predicted body mass.
- The standard error for the `flipper_length_mm` coefficient is 45.52, quantifying the variability in the estimate of the linear effect of flipper length on body mass.
- The standard error for the squared term, `I(flipper_length_mm ** 2)`, is 0.11, reflecting the precision of the estimate for the quadratic effect.

```
lm_fit.bse
```

```
Intercept                4616.246647
flipper_length_mm        45.524470
I(flipper_length_mm ** 2)    0.111807
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- We are 95% confident that the intercept is somewhere between 7,498.89 and 25,660.85 grams.
- We are 95% confident that the coefficient for `flipper_length_mm` is somewhere between -261.17 and -82.06 grams per millimeter.
- We are 95% confident that the coefficient for the quadratic term `(I(flipper_length_mm ** 2))` is somewhere between 0.33 and 0.76.
- These are exact confidence intervals calculated by `.conf_int()` rather than approximate intervals based on standard errors.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept                7347.375921
flipper_length_mm        -262.662917
I(flipper_length_mm ** 2)    0.321333
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept                25812.362509
flipper_length_mm        -80.565036
I(flipper_length_mm ** 2)    0.768562
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
Intercept                7498.887253  25660.851176
flipper_length_mm        -261.168743  -82.059210
I(flipper_length_mm ** 2)    0.325003    0.764892
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept                0.000379
flipper_length_mm        0.000194
I(flipper_length_mm ** 2) 0.000002
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept                True
flipper_length_mm        True
I(flipper_length_mm ** 2)  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.

- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- We are confident the quadratic coefficient is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.

```

def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolord=ecolord,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

```

```

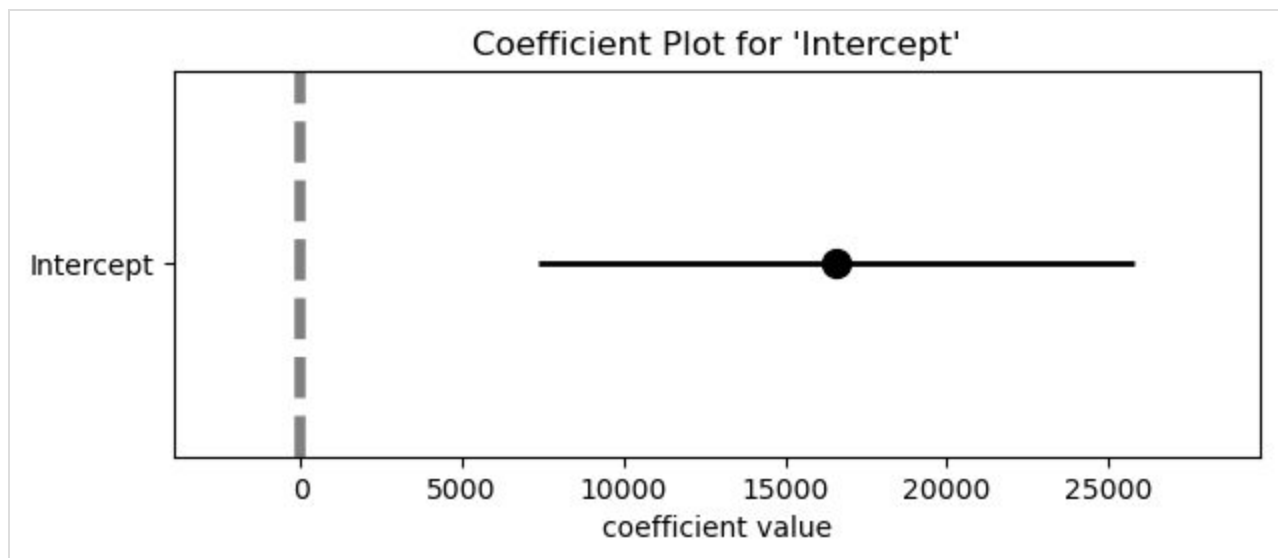
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

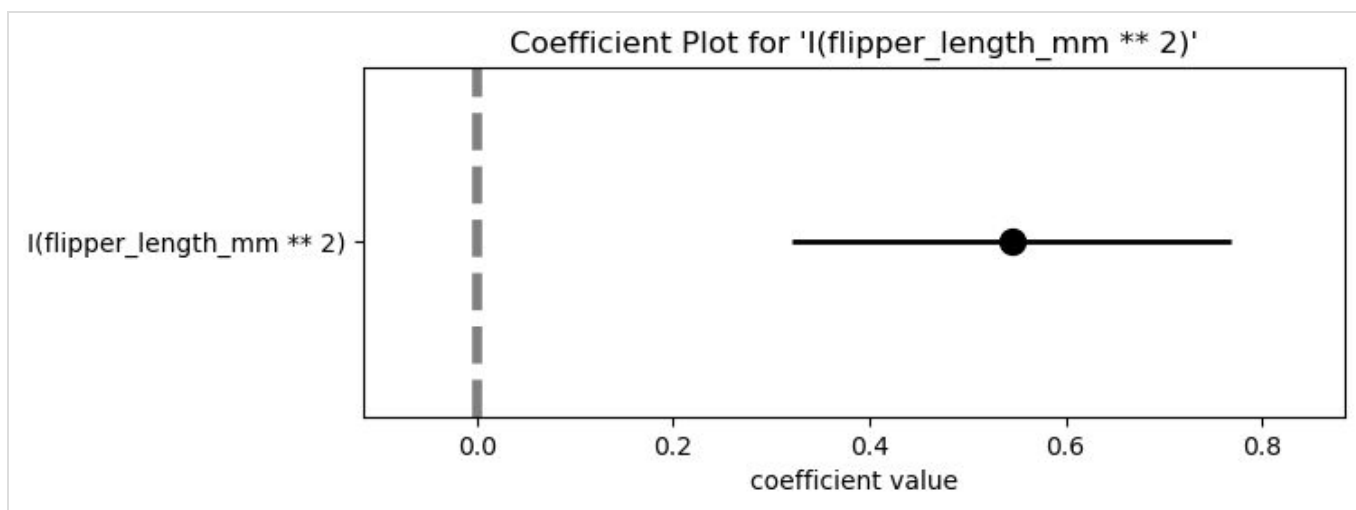
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='I(flipper_length_mm ** 2)')
```



07-SLR-FLFP: Penguins Dataset Simple Linear Regression - Fitting Linear Feature Predictions

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g   2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm  54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                 data=df_clean). \
    fit()
```

```
lm_fit.summary()
```



```
<class 'statsmodels.iolib.summary.Summary'>
"""
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g      R-squared:                0.762
Model:                  OLS              Adj. R-squared:         0.761
Method:                 Least Squares    F-statistic:             1060.
Date:                   Tue, 26 Aug 2025  Prob (F-statistic):       3.13e-105
Time:                   19:54:26         Log-Likelihood:         -2461.1
No. Observations:      333              AIC:                    4926.
Df Residuals:          331              BIC:                    4934.
Df Model:               1
Covariance Type:       nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept             -5872.0927     310.285    -18.925     0.000    -6482.472    -5261.713
flipper_length_mm      50.1533         1.540     32.562     0.000      47.123      53.183
=====
Omnibus:                 5.922    Durbin-Watson:           2.102
Prob(Omnibus):           0.052    Jarque-Bera (JB):         5.876
Skew:                    0.325    Prob(JB):                 0.0530
Kurtosis:                3.025    Cond. No.                 2.90e+03
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.9e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
"""
```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```
Intercept          -5872.092683
flipper_length_mm    50.153266
dtype: float64
```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.

- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.
- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, -5872.09 and 50.15, respectively.
- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept          310.285155
flipper_length_mm   1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept          -6492.662994
flipper_length_mm   47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept          -5251.522372
flipper_length_mm   53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	-6482.472236	-5261.713130
flipper_length_mm	47.123390	53.183142

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.

```

def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolord=ecolord,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

```

```

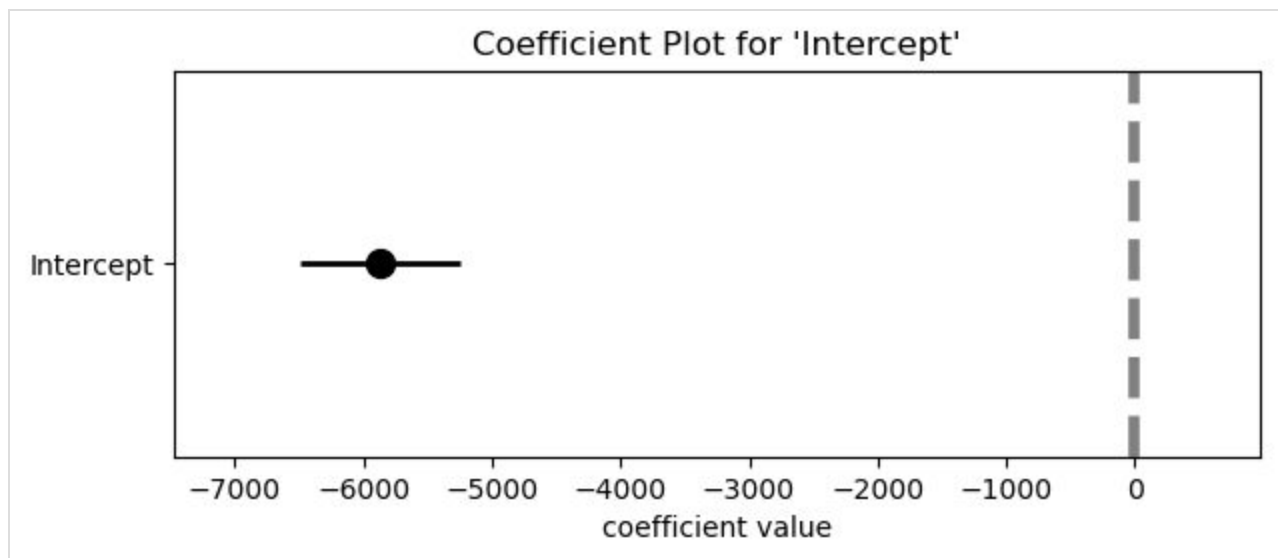
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

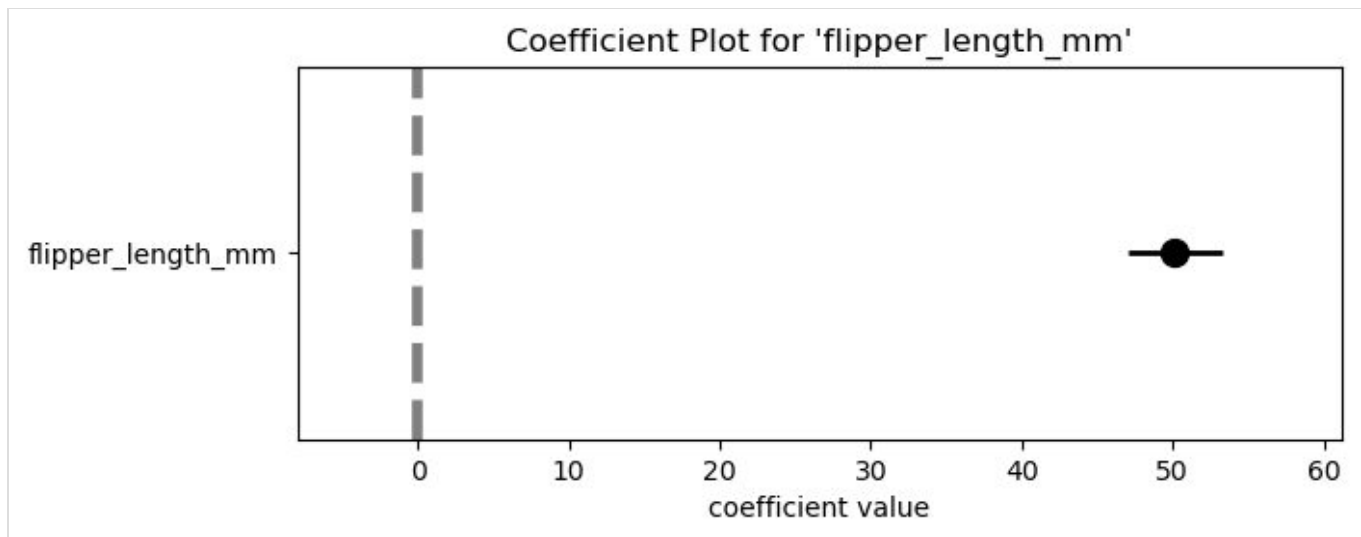
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='flipper_length_mm')
```



Linear Feature Prediction

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

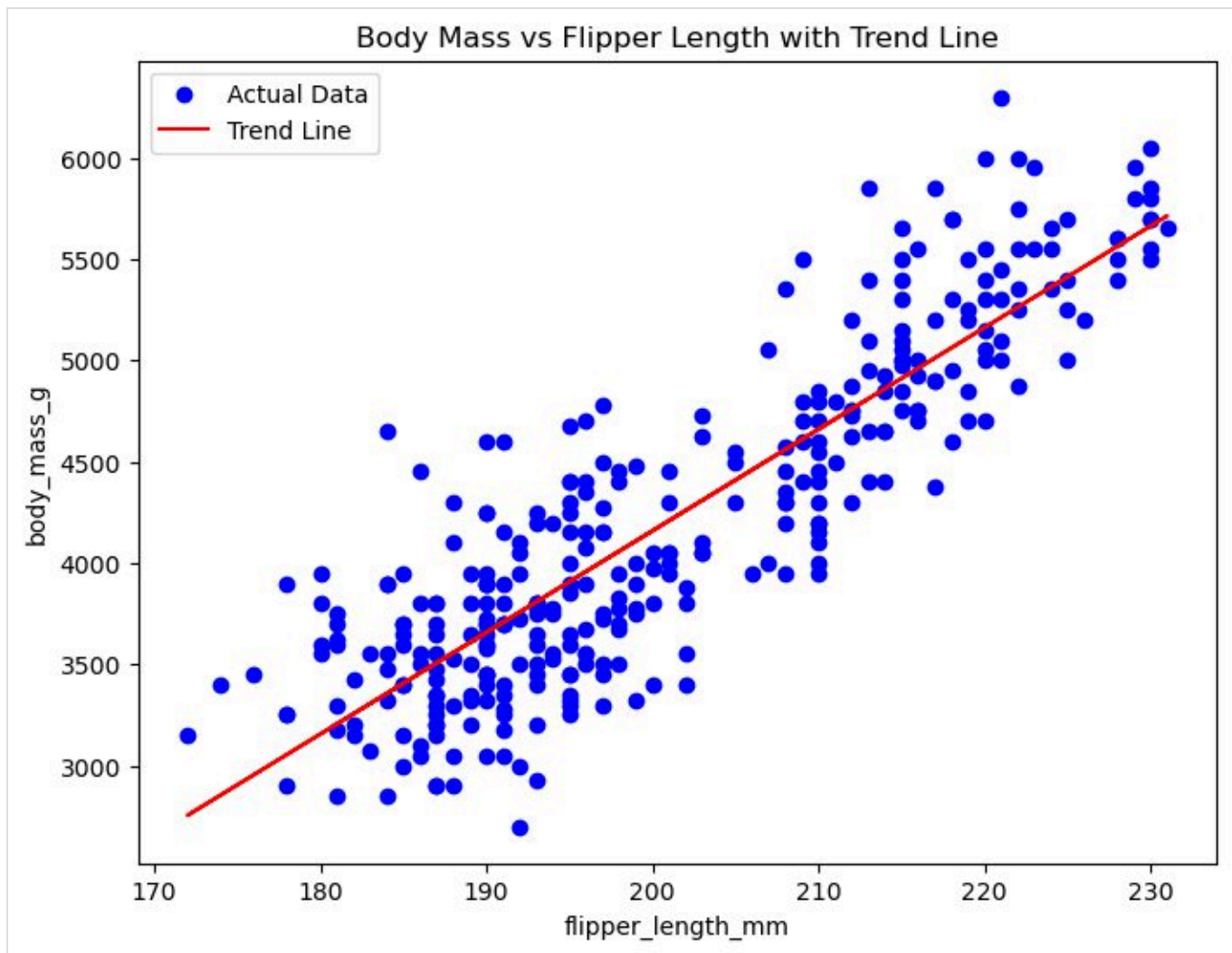
```
my_slope, my_intercept = np.polyfit(x,
                                     y,
                                     1)
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \
                  my_slope * \
                  df_viz['flipper_length_mm']
```

Include the training set as a scatter plot.

```
plt.figure(figsize=(8,6))
plt.scatter(df_viz['flipper_length_mm'],
            df_viz['body_mass_g'],
            color='blue',
            label='Actual Data')
plt.plot(df_viz['flipper_length_mm'],
         df_viz['trend'],
         color='red',
         label='Trend Line')
plt.xlabel('flipper_length_mm')
plt.ylabel('body_mass_g')
plt.title('Body Mass vs Flipper Length with Trend Line')
plt.legend()
plt.show()
```



We will not know the true relationship therefore we must include the confidence intervals on the predicted trend as we must always represent the uncertainty on the trend given the fitted data.

```
predictions = lm_fit.get_prediction(df_viz)
```

```
lm_pred_summary = predictions.summary_frame()
```

```
lm_pred_summary
```

```

      mean    mean_se  mean_ci_lower  mean_ci_upper  obs_ci_lower  \
0  3205.648453  37.555479   3131.770937   3279.525968   2428.361760
1  3456.414782  31.560151   3394.331017   3518.498548   2680.160243
2  3907.794176  23.432601   3861.698575   3953.889777   3132.654498
3  3807.487644  24.803187   3758.695887   3856.279401   3032.182961
4  3657.027846  27.385208   3603.156847   3710.898845   2881.386950
..      ...      ...      ...      ...      ...
328 4860.706229  29.454747   4802.764124   4918.648334   4084.771951
329 4910.859495  30.525256   4850.811531   4970.907459   4134.765123
330 5261.932356  38.911498   5185.387339   5338.477374   4484.387595
331 4760.399697  27.448091   4706.404999   4814.394395   3984.750199
332 4810.552963  28.427409   4754.631792   4866.474134   4034.766979

      obs_ci_upper
0    3982.935146
1    4232.669322
2    4682.933854
3    4582.792327
4    4432.668743
..      ...
328  5636.640507
329  5686.953867
330  6039.477118
331  5536.049195
332  5586.338947

[333 rows x 6 columns]
```

Visualize the 95% confidence interval and 95% prediction interval.

- **Confidence Interval (CI) for the Mean Prediction (Grey Band):**
 - Shows uncertainty in the estimate of the average value (the regression line) for a given x.
 - Typically narrow; with enough data, the mean can be estimated precisely.
 - 95% CI means: "We are 95% confident the true average response falls within this band for each x-value."
 - Most data points will not fall inside this band; it is only about the mean.
- **Prediction Interval (PI) for Individual Observations (Orange Band):**
 - Shows where we expect 95% of future individual data points to fall, at each value of x.
 - Much wider than the CI, because it includes both the uncertainty in the mean and the natural scatter (variance) of the data.
 - 95% PI means: "If we observe a new data point at a given x, there is a 95% chance it will fall inside this band."
- **Key Differences:**
 - CI is about the uncertainty in the average (regression line).
 - PI is about the uncertainty in predicting a single new data point.

- **In the plot:**
 - The grey band is the 95% CI for the mean prediction.
 - The orange band is the 95% PI for individual body mass measurements given flipper length.
 - The black dots are your actual data points.
- Most dots fall inside the orange PI band (as expected), but only a few will fall inside the narrow grey CI band.
- Use the PI to show the expected spread of new data.
- Use the CI to show uncertainty in the estimated average.

```
# Generate x values over the full data range
flipper_range = np.linspace(df_clean.flipper_length_mm.min(),
                             df_clean.flipper_length_mm.max(),
                             200)

df_pred = pd.DataFrame({'flipper_length_mm': flipper_range})
lm_pred_summary_full = lm_fit.get_prediction(df_pred).summary_frame()

# Create the figure and axis
fig, ax = plt.subplots()

# Prediction interval – the uncertainty on a single measurement (observation)
ax.fill_between(flipper_range,
                lm_pred_summary_full.obs_ci_lower,
                lm_pred_summary_full.obs_ci_upper,
                facecolor='orange',
                alpha=0.75,
                edgecolor='orange')

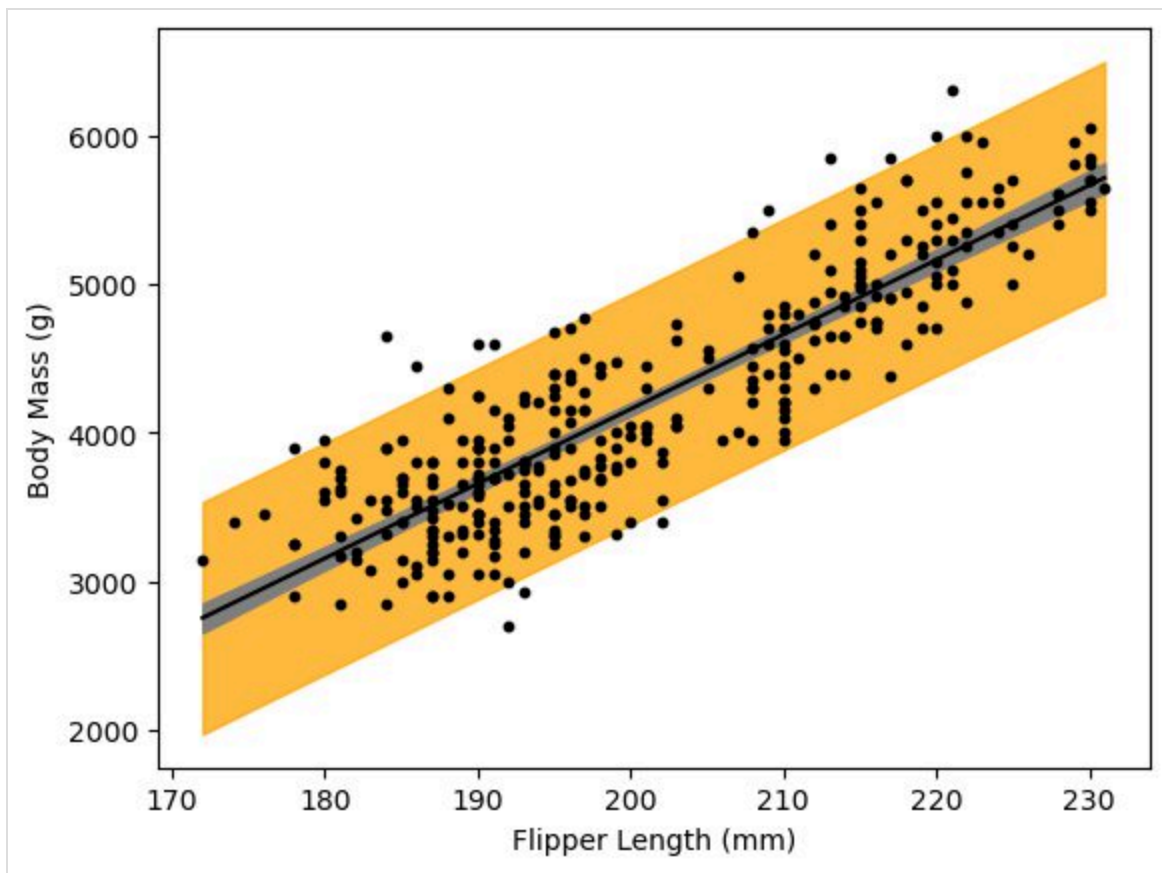
# Confidence interval – the uncertainty on the mean output
ax.fill_between(flipper_range,
                lm_pred_summary_full.mean_ci_lower,
                lm_pred_summary_full.mean_ci_upper,
                facecolor='grey',
                edgecolor='grey')

# Output mean – the predictive trend – the best fit line
ax.plot(flipper_range,
        lm_pred_summary_full['mean'],
        color='k',
        linewidth=1.5)

# Include the training data
ax.scatter(df_clean.flipper_length_mm,
           df_clean.body_mass_g,
           color='k',
           s=10)

# Set the labels
ax.set_xlabel('Flipper Length (mm)')
ax.set_ylabel('Body Mass (g)')

# Show the plot
plt.show()
```



08-SLR-FNLFP: Penguins Dataset Simple Linear Regression - Fitting Non-Linear Feature Predictions

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species          0
island           0
bill_length_mm   0
bill_depth_mm    0
flipper_length_mm 0
body_mass_g      0
sex              0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species          3
island           3
bill_length_mm   163
bill_depth_mm    79
flipper_length_mm 54
body_mass_g      93
sex              2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Non-Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a non-linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 x + \beta_2 x^2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **linear effect**: change in μ for a one-unit increase in x

β_2 **quadratic effect**: change in μ for a one-unit increase in x^2

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm + I(flipper_length_mm ** 2)',
                  data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```

<class 'statsmodels.iolib.summary.Summary'>
"""
                                OLS Regression Results
=====
Dep. Variable:                body_mass_g    R-squared:                0.778
Model:                        OLS           Adj. R-squared:         0.777
Method:                      Least Squares  F-statistic:            578.5
Date:                        Tue, 26 Aug 2025 Prob (F-statistic):      1.34e-108
Time:                        19:56:33       Log-Likelihood:         -2449.5
No. Observations:            333           AIC:                   4905.
Df Residuals:                330           BIC:                   4916.
Df Model:                    2
Covariance Type:            nonrobust
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
-----
--
Intercept                    1.658e+04   4616.247        3.592    0.000    7498.887
2.57e+04
flipper_length_mm           -171.6140    45.524       -3.770    0.000   -261.169
-82.059
I(flipper_length_mm ** 2)    0.5449      0.112        4.874    0.000     0.325
0.765
=====
Omnibus:                    10.746   Durbin-Watson:           2.214
Prob(Omnibus):              0.005   Jarque-Bera (JB):        10.985
Skew:                      0.441   Prob(JB):                0.00412
Kurtosis:                   3.119   Cond. No.                9.07e+06
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 9.07e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
"""

```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **intercept** is 16,579.87, which represents the baseline predicted body mass when both flipper length and its square are zero.
- The **coefficient** for `flipper_length_mm` is -171.61, indicating that for every additional millimeter in flipper length, body mass decreases by about 171.61 grams, before considering the quadratic effect.
- The **coefficient for the squared term**, `I(flipper_length_mm ** 2)`, is 0.54, meaning that for every unit increase in the square of flipper length, body mass increases by 0.54 grams, introducing an upward-curving effect.
- The presence of the positive squared term (0.54) shows that the relationship between flipper length and body mass is quadratic and curves upward, rather than being strictly linear.

```
lm_fit.params
```

```
Intercept                16579.869215
flipper_length_mm        -171.613976
I(flipper_length_mm ** 2)    0.544948
dtype: float64
```

The standard error on the estimate is contained in the `.bse` attribute.

- The standard error for the intercept is 4,616.25, indicating the level of uncertainty in estimating the baseline predicted body mass.
- The standard error for the `flipper_length_mm` coefficient is 45.52, quantifying the variability in the estimate of the linear effect of flipper length on body mass.
- The standard error for the squared term, `I(flipper_length_mm ** 2)`, is 0.11, reflecting the precision of the estimate for the quadratic effect.

```
lm_fit.bse
```

```
Intercept                4616.246647
flipper_length_mm        45.524470
I(flipper_length_mm ** 2)    0.111807
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- We are 95% confident that the intercept is somewhere between 7,498.89 and 25,660.85 grams.
- We are 95% confident that the coefficient for `flipper_length_mm` is somewhere between -261.17 and -82.06 grams per millimeter.
- We are 95% confident that the coefficient for the quadratic term `(I(flipper_length_mm ** 2))` is somewhere between 0.33 and 0.76.
- These are exact confidence intervals calculated by `.conf_int()` rather than approximate intervals based on standard errors.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept                7347.375921
flipper_length_mm        -262.662917
I(flipper_length_mm ** 2)    0.321333
dtype: float64
```



```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept                25812.362509
flipper_length_mm        -80.565036
I(flipper_length_mm ** 2)    0.768562
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
Intercept                7498.887253  25660.851176
flipper_length_mm        -261.168743  -82.059210
I(flipper_length_mm ** 2)    0.325003    0.764892
```

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept                0.000379
flipper_length_mm        0.000194
I(flipper_length_mm ** 2) 0.000002
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept                True
flipper_length_mm        True
I(flipper_length_mm ** 2)  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.

- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.

```

def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolord=ecolord,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

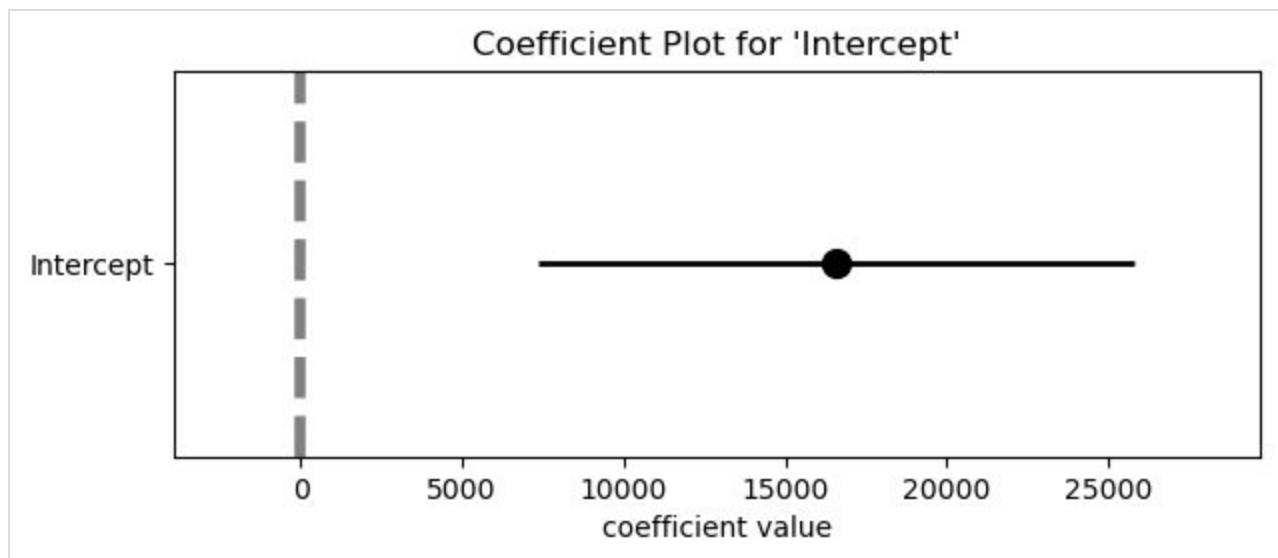
```

```
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

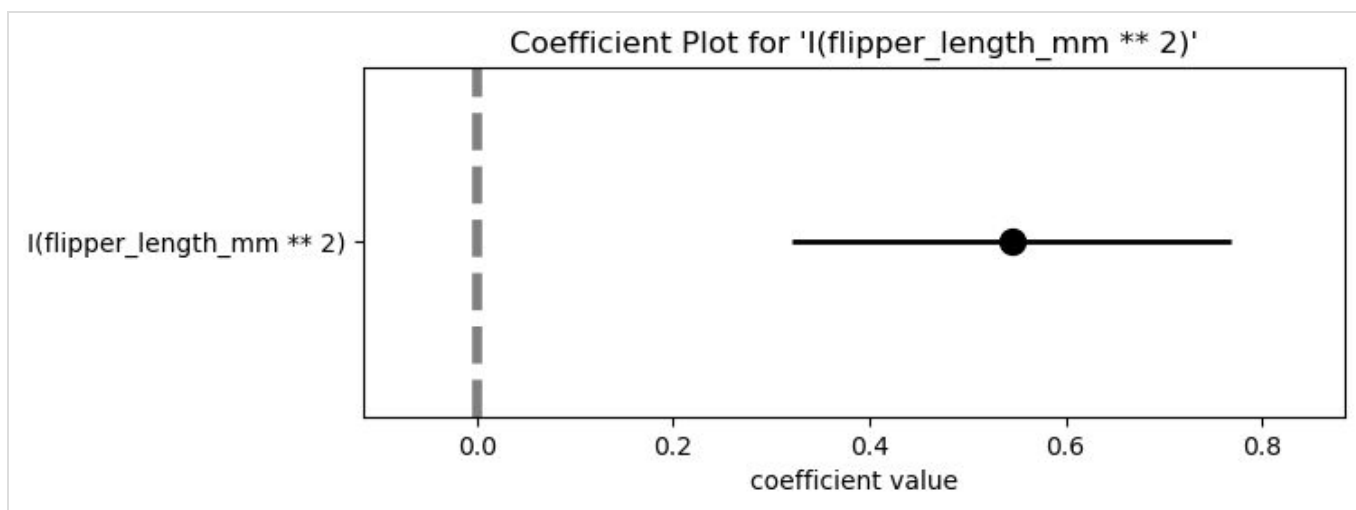
# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

# Show the plot
plt.show()
```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='I(flipper_length_mm ** 2)')
```



Non-Linear Feature Prediction

```
x = df_clean['flipper_length_mm']
y = df_clean['body_mass_g']
```

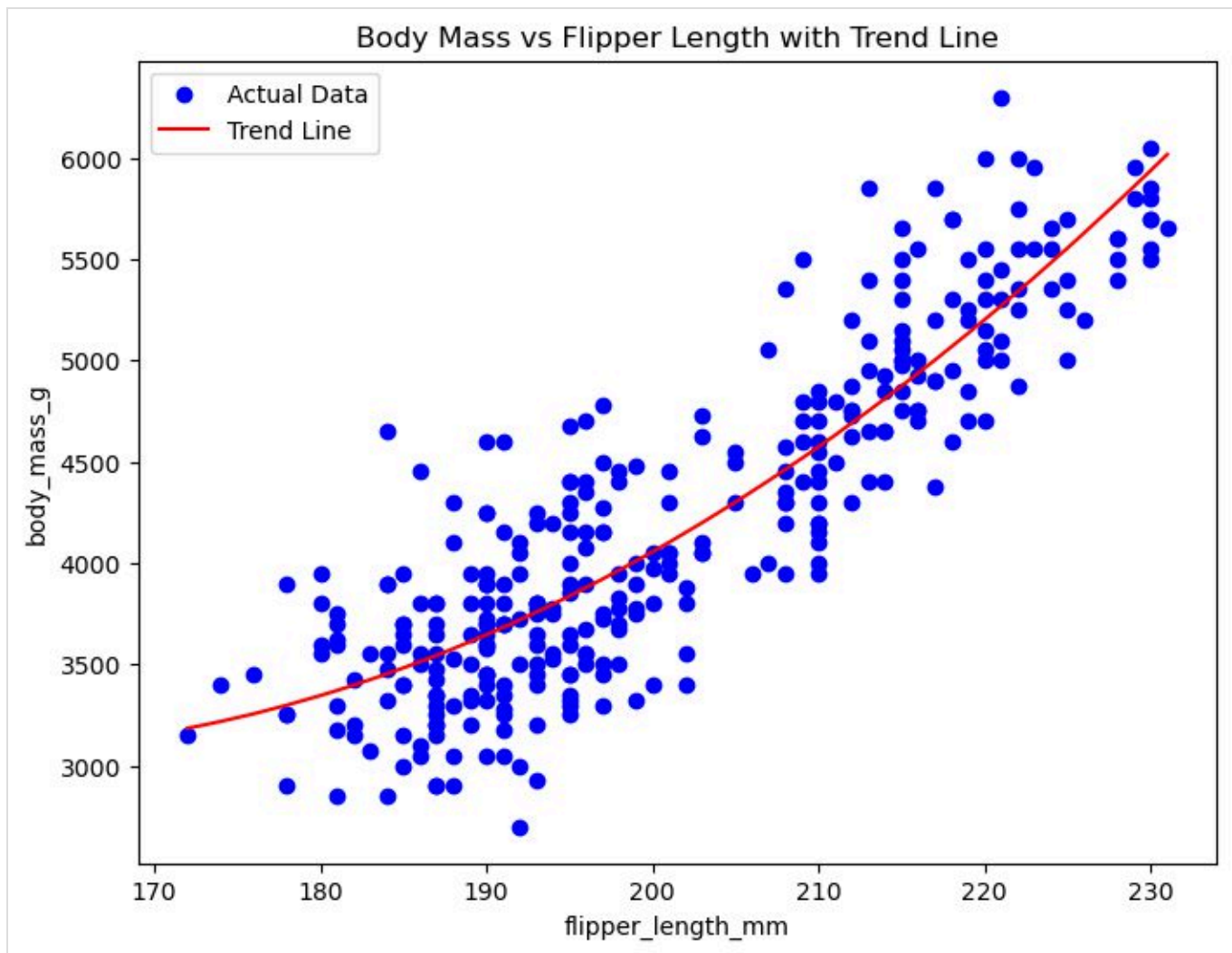
```
my_quadratic, my_slope, my_intercept = np.polyfit(x,
                                                    y,
                                                    2)
```

```
df_viz = df_clean.copy()
```

```
df_viz['trend'] = my_intercept + \
                  my_slope * df_viz['flipper_length_mm'] + \
                  my_quadratic * df_viz['flipper_length_mm'] \
                  **2
```

Include the training set as a scatter plot.

```
sorted_df = df_viz.sort_values('flipper_length_mm')
plt.figure(figsize=(8,6))
plt.scatter(df_viz['flipper_length_mm'],
            df_viz['body_mass_g'],
            color='blue',
            label='Actual Data')
plt.plot(sorted_df['flipper_length_mm'],
         sorted_df['trend'],
         color='red',
         label='Trend Line')
plt.xlabel('flipper_length_mm')
plt.ylabel('body_mass_g')
plt.title('Body Mass vs Flipper Length with Trend Line')
plt.legend()
plt.show()
```



We will not know the true relationship therefore we must include the confidence intervals on the predicted trend as we must always represent the uncertainty on the trend given the fitted data.

```
predictions = lm_fit.get_prediction(df_viz)
```

```
lm_pred_summary = predictions.summary_frame()
```

```
lm_pred_summary
```

```

      mean    mean_se  mean_ci_lower  mean_ci_upper  obs_ci_lower  \
0  3370.773580  49.673534   3273.056864   3468.490297   2615.946156
1  3512.682868  32.638135   3448.477825   3576.887911   2761.458420
2  3836.783006  26.945004   3783.777368   3889.788644   3086.432766
3  3757.131484  26.122145   3705.744557   3808.518411   3006.893855
4  3645.828418  26.589245   3593.522623   3698.134213   2895.527292
..      ...      ...      ...      ...      ...
328 4810.906624  30.268285   4751.363500   4870.449749   4060.066250
329 4873.075244  30.527844   4813.021521   4933.128967   4122.194206
330 5338.772657  40.807527   5258.496960   5419.048355   4586.004385
331 4689.839072  30.240975   4630.349671   4749.328472   3939.002956
332 4749.827900  30.188751   4690.441234   4809.214567   3998.999917

      obs_ci_upper
0    4125.601005
1    4263.907316
2    4587.133245
3    4507.369113
4    4396.129544
..      ...
328  5561.746999
329  5623.956282
330  6091.540929
331  5440.675187
332  5500.655883

[333 rows x 6 columns]
```

Visualize the 95% confidence interval and 95% prediction interval.

- **Confidence Interval (CI) for the Mean Prediction (Grey Band):**
 - Shows uncertainty in the estimate of the average value (the regression line) for a given x.
 - Typically narrow; with enough data, the mean can be estimated precisely.
 - 95% CI means: "We are 95% confident the true average response falls within this band for each x-value."
 - Most data points will not fall inside this band; it is only about the mean.
- **Prediction Interval (PI) for Individual Observations (Orange Band):**
 - Shows where we expect 95% of future individual data points to fall, at each value of x.
 - Much wider than the CI, because it includes both the uncertainty in the mean and the natural scatter (variance) of the data.
 - 95% PI means: "If we observe a new data point at a given x, there is a 95% chance it will fall inside this band."
- **Key Differences:**
 - CI is about the uncertainty in the average (regression line).
 - PI is about the uncertainty in predicting a single new data point.

- **In the plot:**
 - The grey band is the 95% CI for the mean prediction.
 - The orange band is the 95% PI for individual body mass measurements given flipper length.
 - The black dots are your actual data points.
- Most dots fall inside the orange PI band (as expected), but only a few will fall inside the narrow grey CI band.
- Use the PI to show the expected spread of new data.
- Use the CI to show uncertainty in the estimated average.


```
# Generate x values over the full data range
flipper_range = np.linspace(df_clean.flipper_length_mm.min(),
                             df_clean.flipper_length_mm.max(),
                             200)

df_pred = pd.DataFrame({'flipper_length_mm': flipper_range})
lm_pred_summary_full = lm_fit.get_prediction(df_pred).summary_frame()

# Create the figure and axis
fig, ax = plt.subplots()

# Prediction interval – the uncertainty on a single measurement (observation)
ax.fill_between(flipper_range,
                lm_pred_summary_full.obs_ci_lower,
                lm_pred_summary_full.obs_ci_upper,
                facecolor='orange',
                alpha=0.75,
                edgecolor='orange')

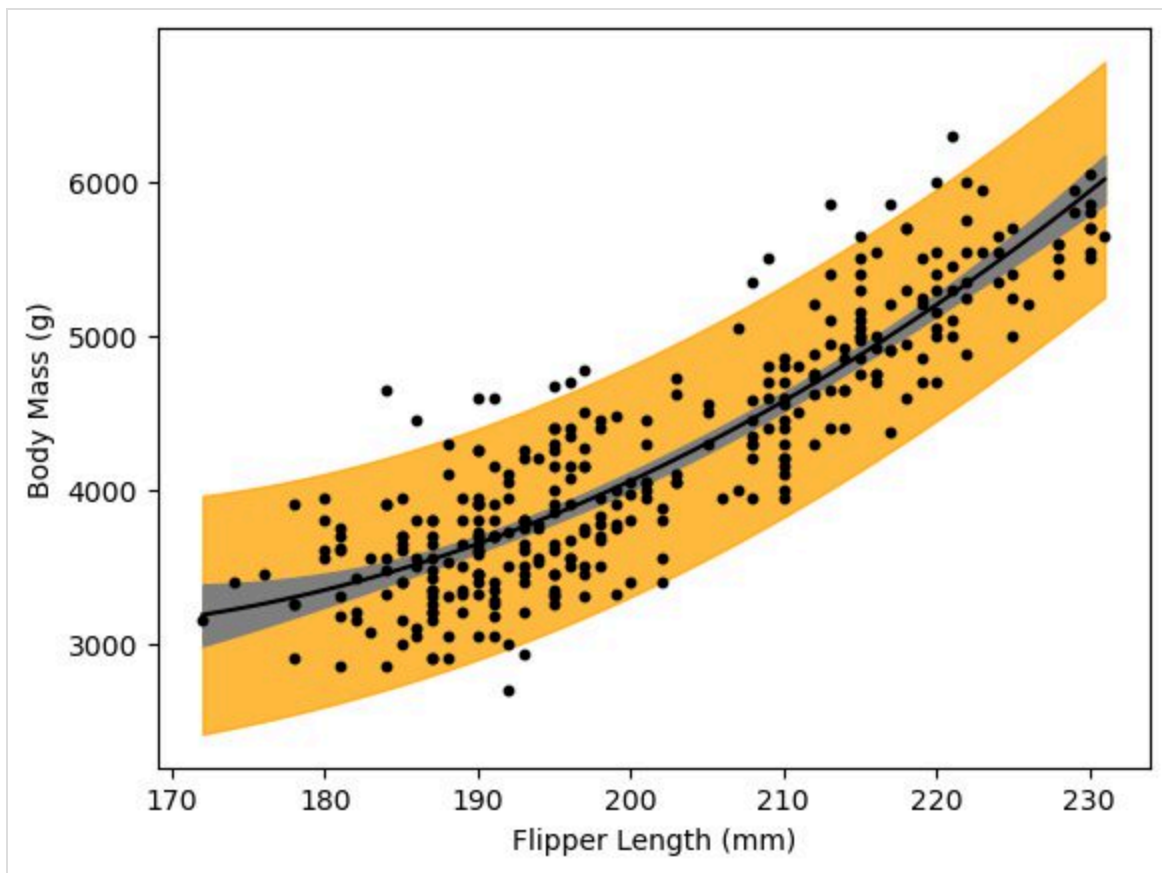
# Confidence interval – the uncertainty on the mean output
ax.fill_between(flipper_range,
                lm_pred_summary_full.mean_ci_lower,
                lm_pred_summary_full.mean_ci_upper,
                facecolor='grey',
                edgecolor='grey')

# Output mean – the predictive trend – the best fit line
ax.plot(flipper_range,
        lm_pred_summary_full['mean'],
        color='k',
        linewidth=1.5)

# Include the training data
ax.scatter(df_clean.flipper_length_mm,
           df_clean.body_mass_g,
           color='k',
           s=10)

# Set the labels
ax.set_xlabel('Flipper Length (mm)')
ax.set_ylabel('Body Mass (g)')

# Show the plot
plt.show()
```



09-SLR-MLFP: Penguins Dataset Simple Linear Regression - Measuring Linear Feature Performance

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species      0
island       0
bill_length_mm  2
bill_depth_mm  2
flipper_length_mm  2
body_mass_g   2
sex          11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species      0
island       0
bill_length_mm  0
bill_depth_mm  0
flipper_length_mm  0
body_mass_g   0
sex          0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species      3
island       3
bill_length_mm 163
bill_depth_mm  79
flipper_length_mm  54
body_mass_g   93
sex          2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species      3
island       3
sex          2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm      163
bill_depth_mm       79
flipper_length_mm   54
body_mass_g         93
dtype: int64
```

Fitting a Linear Feature

We will fit a linear model between the input `flipper_length_mm` and the output `body_mass_g` assuming a linear relationship between the trend and the input.

$$\mu = \beta_0 + \beta_1 \times x$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x

x **feature** or **input variable**: independent variable used to predict μ

```
lm_fit = smf.ols(formula='body_mass_g ~ flipper_length_mm',
                 data=df_clean). \
    fit()
```

```
lm_fit.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                        OLS Regression Results
=====
Dep. Variable:          body_mass_g      R-squared:                0.762
Model:                  OLS              Adj. R-squared:         0.761
Method:                 Least Squares    F-statistic:             1060.
Date:                   Wed, 27 Aug 2025  Prob (F-statistic):       3.13e-105
Time:                   08:47:58         Log-Likelihood:          -2461.1
No. Observations:       333              AIC:                    4926.
Df Residuals:           331              BIC:                    4934.
Df Model:               1
Covariance Type:        nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept              -5872.0927    310.285    -18.925    0.000   -6482.472   -5261.713
flipper_length_mm       50.1533         1.540     32.562    0.000     47.123     53.183
=====
Omnibus:                 5.922    Durbin-Watson:           2.102
Prob(Omnibus):           0.052    Jarque-Bera (JB):         5.876
Skew:                    0.325    Prob(JB):                 0.0530
Kurtosis:                3.025    Cond. No.                  2.90e+03
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.9e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
"""
```

The `.params` attribute stores the coefficient estimates as they minimize the sum of squared errors.

- The **slope**, 50.15, describes how much the predicted body mass changes for each additional millimeter of flipper length.
- The **intercept**, -5872.09, is a mathematical result that helps position the regression line but isn't physically meaningful for penguins, since the data doesn't include flipper lengths near zero.

```
lm_fit.params
```

```
Intercept          -5872.092683
flipper_length_mm   50.153266
dtype: float64
```

The standard error on the estimate is contained in the `.bse` attribute.

- The **standard error** for the **intercept**, 310.29, indicates that, if we repeatedly sampled penguins and fit the same model, the estimated starting body mass, -5872.09, would typically vary by about 310 grams between samples, showing the precision of this baseline estimate.

- The **standard error** for the flipper length coefficient, 1.54, means our estimate that each additional millimeter of flipper length increases a penguin's body mass by 50.15 grams is quite precise, with typical variation of only 1.54 grams across samples.
- These small **standard errors** demonstrate that, for the penguins in the dataset, both the baseline body mass and the effect of flipper length are estimated with high statistical confidence.
- They are considered to have high statistical confidence because the **standard errors** for both the **intercept**, 310.29, and the flipper length coefficient, 1.54, are small relative to the estimated values, -5872.09 and 50.15, respectively.
- This means that if you repeatedly sampled penguins and fit the same regression model, the estimates for the intercept and slope would vary only slightly from sample to sample.

```
lm_fit.bse
```

```
Intercept          310.285155
flipper_length_mm   1.540231
dtype: float64
```

We can use the ± 1 and ± 2 standard error intervals to know how trustworthy the estimate is.

- The most common standard error interval is the ± 2 standard error interval because that is approximately equal to the 95% confidence interval.
- We are 95% confident that the coefficient is somewhere between 47 and 53.
- The below are approximations as we will get the exact values with `.conf_int()`.

```
lm_fit.params - 2 * lm_fit.bse
```

```
Intercept          -6492.662994
flipper_length_mm   47.072804
dtype: float64
```

```
lm_fit.params + 2 * lm_fit.bse
```

```
Intercept          -5251.522372
flipper_length_mm   53.233728
dtype: float64
```

```
lm_fit.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	-6482.472236	-5261.713130
flipper_length_mm	47.123390	53.183142

The P-values for the coefficients are used to identify the statistically significant of the input.

- A **statistically significant** input one that has a **p-value** less than 0.05 which means if this is the case, we have a **statistically significant** input or feature.
- These values are extremely small and much lower than 0.05 so they indicate very strong **statistical significance** for both parameters.

```
lm_fit.pvalues
```

```
Intercept          1.183941e-54
flipper_length_mm  3.132836e-105
dtype: float64
```

```
lm_fit.pvalues < 0.05
```

```
Intercept          True
flipper_length_mm  True
dtype: bool
```

Visualize the coefficient estimates as markers in a scatter plot and the confidence intervals as error bars.

- We are confident that the 95% confidence interval does NOT contain zero as both `flipper_length_mm` and the **intercept** are not touching the zero grey dotted line.
- The **intercept** is **statistically significantly negative**; the **slope** is negative, and the `flipper_length_mm` is **statistically significantly positive**; the **slope** is positive.
- **We are confident the slope is positive for the `flipper_length_mm`, therefore we are confident that the average output is increasing as the input or feature is increasing.**


```

def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolord='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolord: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolord=ecolord,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
    ax.set_xlabel(xlabel)
    if title is None:
        title = f"Coefficient Plot for '{var}'"
    ax.set_title(title)

```

```

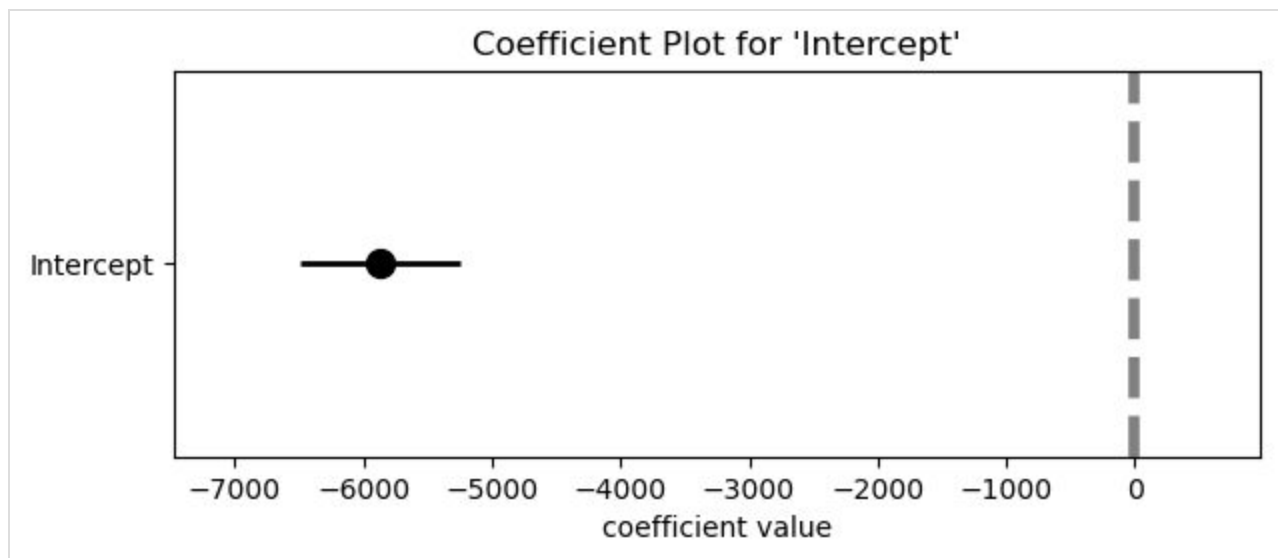
# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

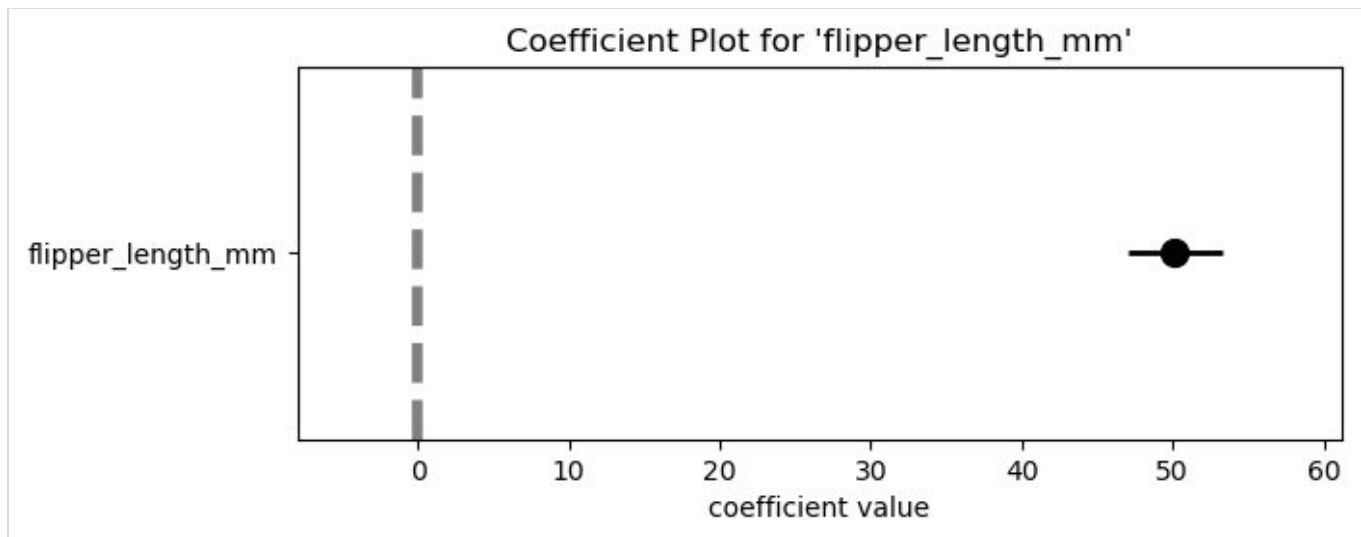
# Show the plot
plt.show()

```

```
my_coefplot(lm_fit, var='Intercept')
```



```
my_coefplot(lm_fit, var='flipper_length_mm')
```



Measure Linear Feature Performance

```
lm_fit.fittedvalues
```

```
0      3205.648453
1      3456.414782
2      3907.794176
4      3807.487644
5      3657.027846
...
338    4860.706229
340    4910.859495
341    5261.932356
342    4760.399697
343    4810.552963
Length: 333, dtype: float64
```

Visually compare the model fitted values or training predictions to the observed outputs.

```
df_clean_copy = df_clean.loc[:,
                             ['flipper_length_mm',
                              'body_mass_g']]. \
    copy()
```

```
df_clean_copy
```

```

      flipper_length_mm  body_mass_g
0          181.0         3750.0
1          186.0         3800.0
2          195.0         3250.0
4          193.0         3450.0
5          190.0         3650.0
..          ...          ...
338         214.0         4925.0
340         215.0         4850.0
341         222.0         5750.0
342         212.0         5200.0
343         213.0         5400.0

```

[333 rows x 2 columns]

```
df_clean_copy['fitted'] = lm_fit.fittedvalues
```

```
df_clean_copy
```

```

      flipper_length_mm  body_mass_g      fitted
0          181.0         3750.0  3205.648453
1          186.0         3800.0  3456.414782
2          195.0         3250.0  3907.794176
4          193.0         3450.0  3807.487644
5          190.0         3650.0  3657.027846
..          ...          ...          ...
338         214.0         4925.0  4860.706229
340         215.0         4850.0  4910.859495
341         222.0         5750.0  5261.932356
342         212.0         5200.0  4760.399697
343         213.0         5400.0  4810.552963

```

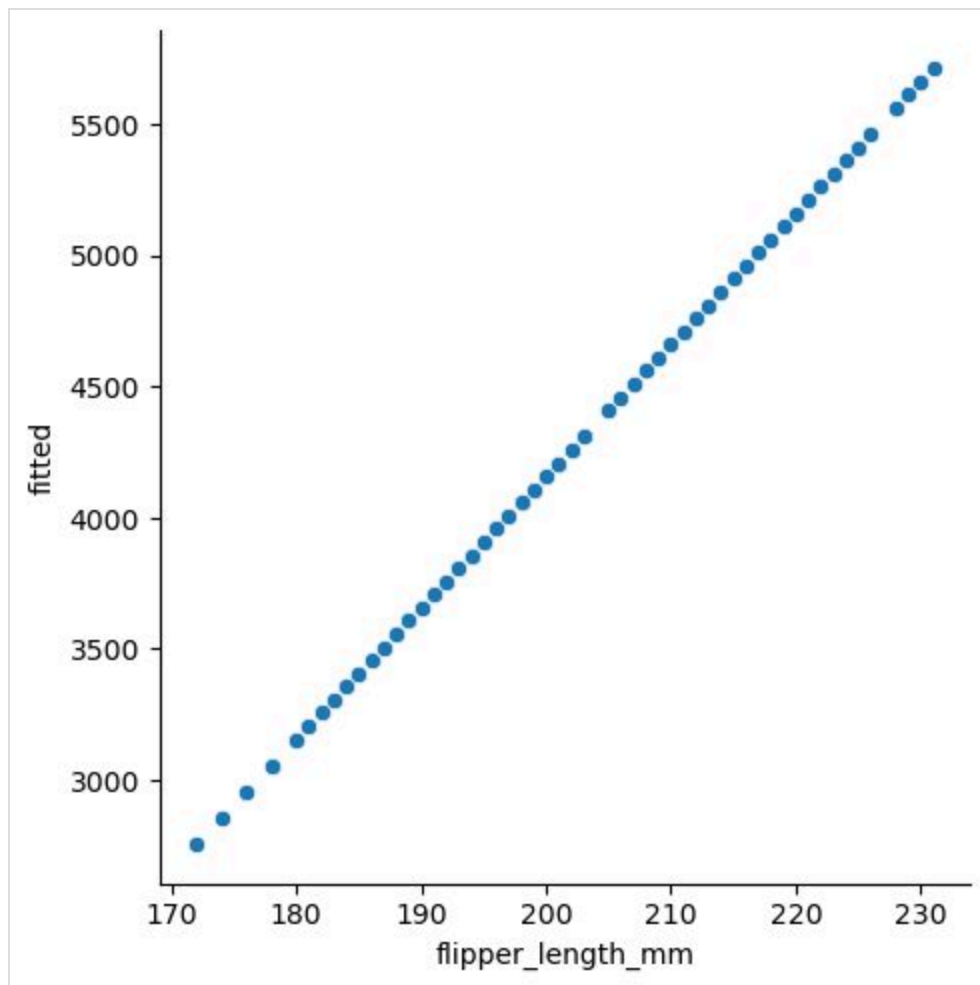
[333 rows x 3 columns]

Visualize the Predictions

```

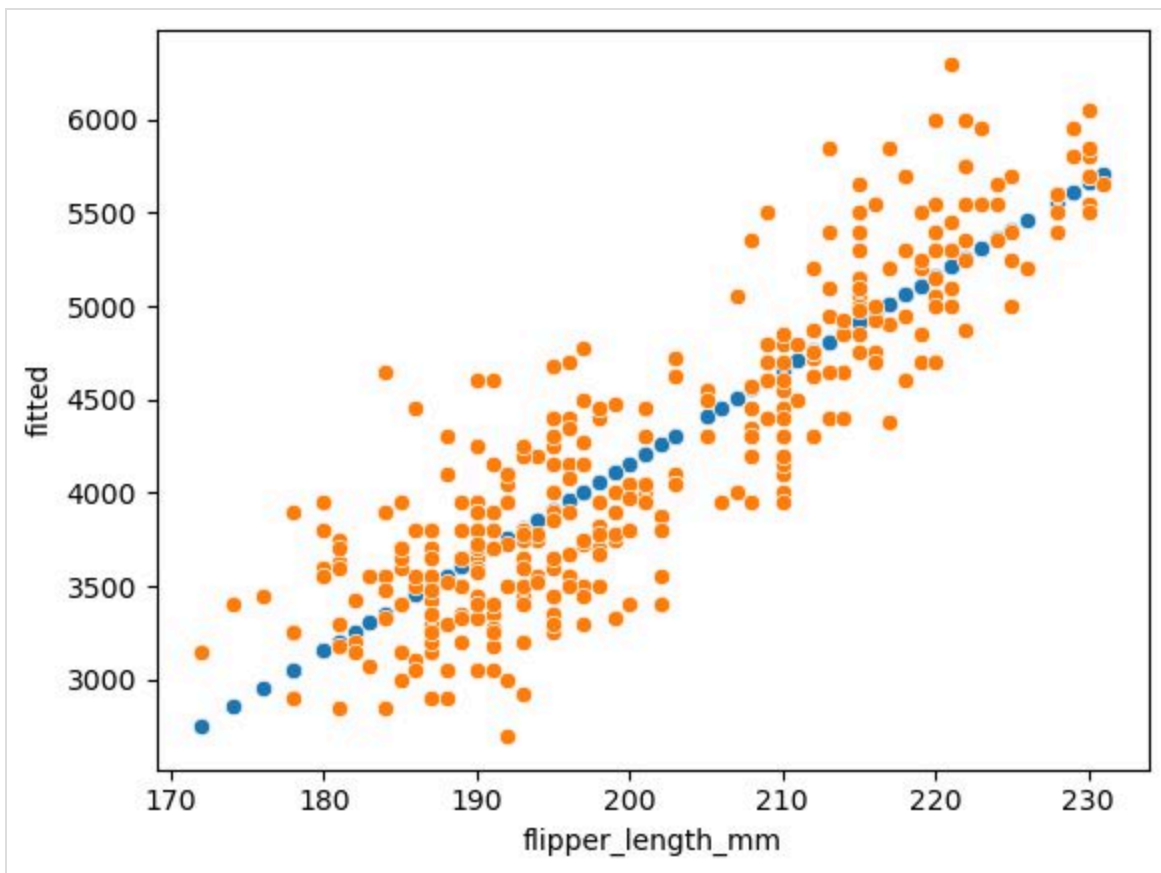
sns.relplot(data=df_clean_copy,
            x='flipper_length_mm',
            y='fitted')
plt.show()

```



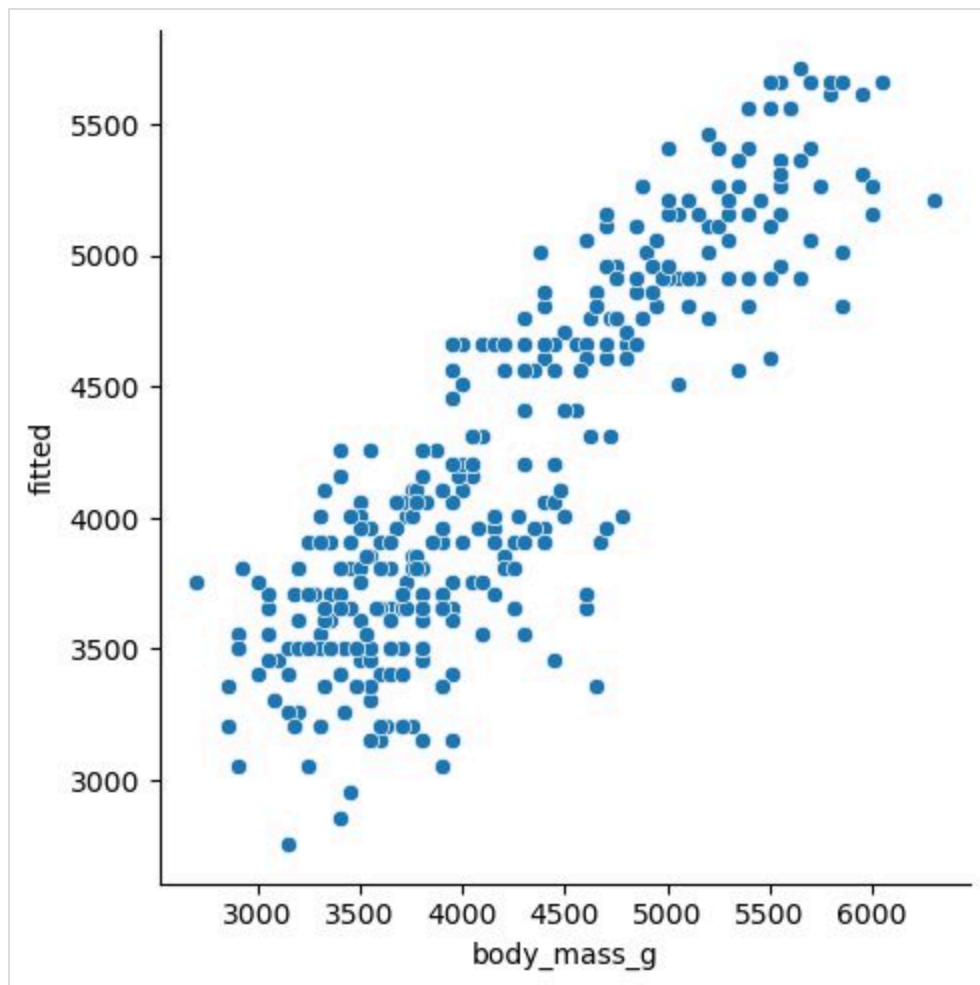
Compare the observed output to the model fits by overlaying the observed output vs the input with the predicted trend or fitted values with the input.

```
fig, ax = plt.subplots()
sns.scatterplot(data=df_clean_copy,
                x='flipper_length_mm',
                y='fitted',
                ax=ax)
sns.scatterplot(data=df_clean_copy,
                x='flipper_length_mm',
                y='body_mass_g',
                ax=ax)
plt.show()
```



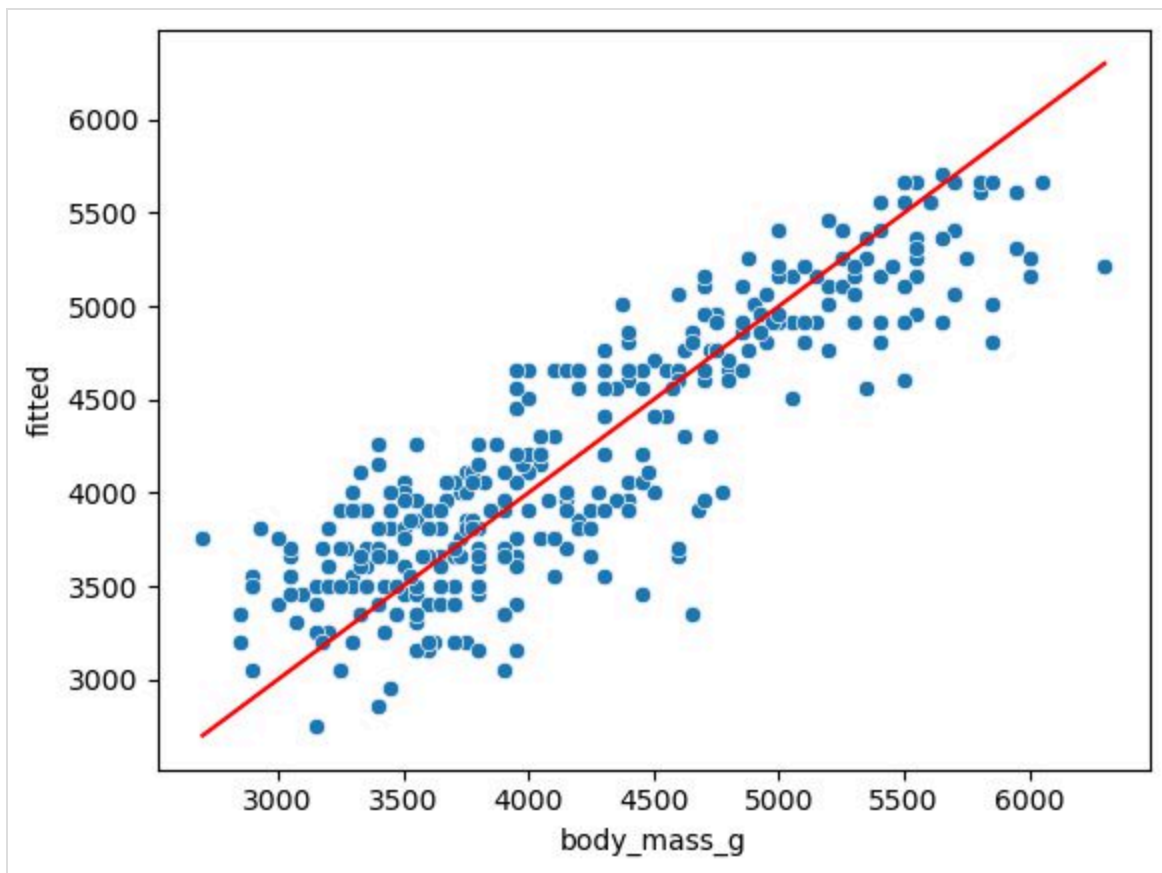
Directly relate the observed output and the model fits or the predicted vs observed figure.

```
sns.relplot(data=df_clean_copy,  
            x='body_mass_g',  
            y='fitted')  
plt.show()
```



Include a "45 degree" reference line with our predicted vs observed figure.

```
fig, ax = plt.subplots()
sns.scatterplot(data=df_clean_copy,
                x='body_mass_g',
                y='fitted',
                ax=ax)
sns.lineplot(data=df_clean_copy,
             x='body_mass_g',
             y='body_mass_g',
             color='red',
             ax=ax)
plt.show()
```



Often we want a number or quantitative metric for the correlation between the model predictions and the observed outputs by calculating the correlation coefficient between the fits and the observed output.

```
df_clean_copy.loc[:,  
                    ['body_mass_g', 'fitted']]. \  
                    corr(numeric_only=True)
```

```
      body_mass_g    fitted  
body_mass_g    1.000000  0.872979  
fitted          0.872979  1.000000
```

```
df_clean_copy.loc[:,  
                    ['body_mass_g', 'fitted']]. \  
                    corr(numeric_only=True). \  
                    iloc[0, 1]
```

```
0.8729788985653614
```


The correlation coefficient can be negative or positive so we will square it.

```
df_clean_copy.loc[:,
                    ['body_mass_g', 'fitted']]. \
    corr(numeric_only=True). \
    iloc[0, 1] ** 2
```

```
0.7620921573403915
```

This calculation is called R-Squared as it comes from the predicted vs observed figure above as it will show whether the observed output increase or decrease with an increase or decrease of the predicted output from the model.

IF THERE IS A GENERAL LINEAR RELATIONSHIP BETWEEN THE OBSERVED AND PREDICTED OUTPUT, R-SQUARED WILL BE HIGH CLOSE TO 1.

```
lm_fit.rsquared
```

```
0.7620921573403914
```

The error is the difference between the observed output and the predicted trend or fitted values as the errors can be positive or negative.

```
df_clean_copy['errors'] = df_clean_copy.body_mass_g - df_clean_copy.fitted
```

```
df_clean_copy
```

	flipper_length_mm	body_mass_g	fitted	errors
0	181.0	3750.0	3205.648453	544.351547
1	186.0	3800.0	3456.414782	343.585218
2	195.0	3250.0	3907.794176	-657.794176
4	193.0	3450.0	3807.487644	-357.487644
5	190.0	3650.0	3657.027846	-7.027846
..	...	...	...	...
338	214.0	4925.0	4860.706229	64.293771
340	215.0	4850.0	4910.859495	-60.859495
341	222.0	5750.0	5261.932356	488.067644
342	212.0	5200.0	4760.399697	439.600303
343	213.0	5400.0	4810.552963	589.447037

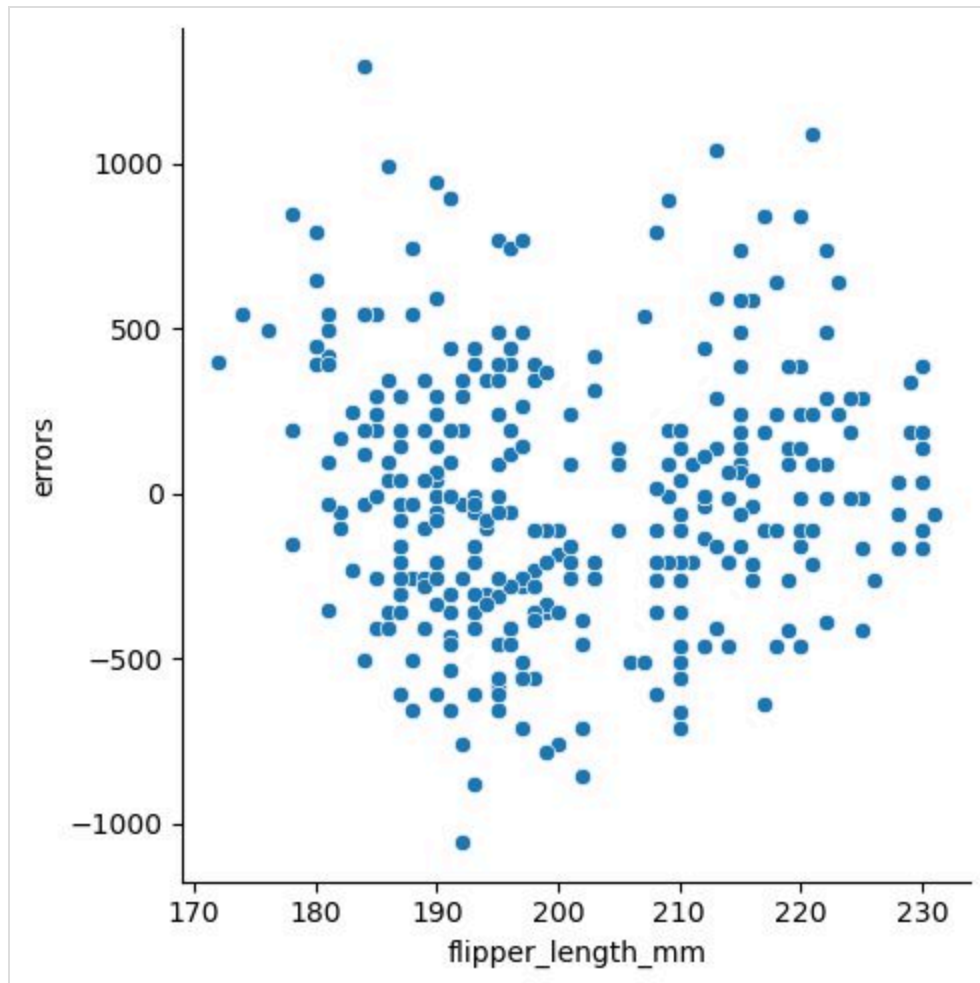
```
[333 rows x 4 columns]
```

We can visualize the errors with respect to an input as ideally there should be no clear correlation between an error and an input as the errors are meant to represent unexplained variability or everything the model can't capture.

IF YOU SEE A LINEAR OR NON-LINEAR RELATIONSHIP THAT MEANS THERE IS SOME FEATURE YOU NEED TO DERIVE FROM YOUR INPUT TO BETTER CAPTURE THE OUTPUT TO INPUT

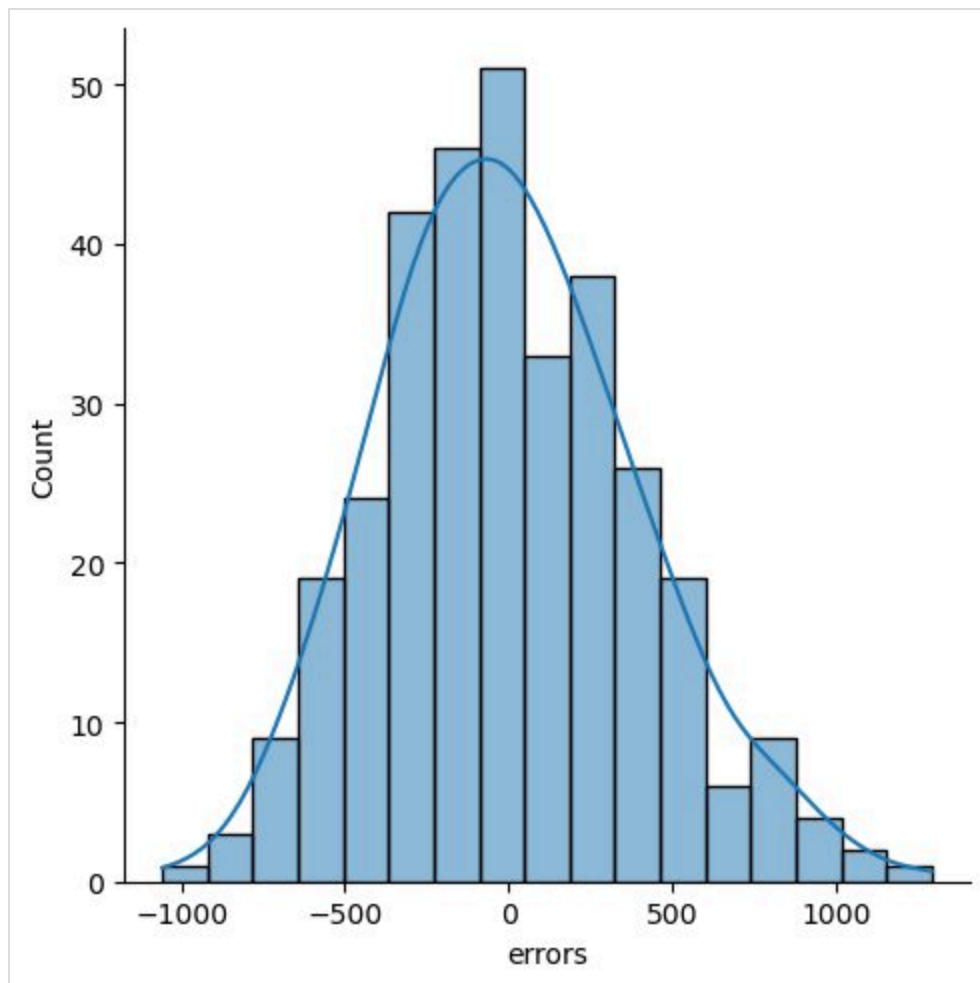
RELATIONSHIP.

```
sns.relplot(data=df_clean_copy,  
            x='flipper_length_mm',  
            y='errors')  
plt.show()
```



The distribution of the errors should be Gaussian.

```
sns.displot(data=df_clean_copy,  
            x='errors',  
            kind='hist',  
            kde=True)  
plt.show()
```



The other term for the error is the residual.

```
lm_fit.resid
```

```
0      544.351547
1      343.585218
2     -657.794176
4     -357.487644
5       -7.027846
...
338     64.293771
340    -60.859495
341     488.067644
342     439.600303
343     589.447037
Length: 333, dtype: float64
```

```
df_clean_copy['residuals'] = lm_fit.resid
```

```
df_clean_copy
```

	flipper_length_mm	body_mass_g	fitted	errors	residuals
0	181.0	3750.0	3205.648453	544.351547	544.351547
1	186.0	3800.0	3456.414782	343.585218	343.585218
2	195.0	3250.0	3907.794176	-657.794176	-657.794176
4	193.0	3450.0	3807.487644	-357.487644	-357.487644
5	190.0	3650.0	3657.027846	-7.027846	-7.027846
..	...	...	...	...	...
338	214.0	4925.0	4860.706229	64.293771	64.293771
340	215.0	4850.0	4910.859495	-60.859495	-60.859495
341	222.0	5750.0	5261.932356	488.067644	488.067644
342	212.0	5200.0	4760.399697	439.600303	439.600303
343	213.0	5400.0	4810.552963	589.447037	589.447037

[333 rows x 5 columns]

To summarize the errors or residuals across all of the data by squaring the error which will convert any negative value to a positive error or SSE, sum of squared errors.

```
(lm_fit.resid ** 2).sum()
```

51211962.72968365

THE SSE INCREASES AS THE NUMBER OF DATA POINTS INCREASES SO WE WANT THE AVERAGE OR THE MSE, MEAN SQUARED ERROR OR AVERAGE SQUARED ERROR PER DATA POINT.

```
(lm_fit.resid ** 2).mean()
```

153789.6778669179

The MSE is not in the same units as the output therefore we must take the square root called RMSE or root mean squared error.

```
np.sqrt((lm_fit.resid ** 2).mean())
```

392.16027063806183

The MAE or mean absolute error makes all values positive and is the average absolute error of the training data and is less sensitive to extreme values because its not squaring the error.

```
np.abs(lm_fit.resid).mean()
```

312.08474571800986

10-SLR-FMM: Penguins Dataset Simple Linear Regression - Fitting Multiple Models

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
```

Load Dataset

```
df = sns.load_dataset('penguins')
```

Perform Essential Exploration & Data Cleaning

Display the initial number of rows and columns.

```
df.shape
```

```
(344, 7)
```

Display the initial variable names and their associated data types.

```
df.dtypes
```

```
species      object
island        object
bill_length_mm  float64
bill_depth_mm  float64
flipper_length_mm float64
body_mass_g    float64
sex           object
dtype: object
```

Display the number of missing values for each column, remove the missing values and create a clean copy of the dataset.

```
df.isna().sum()
```

```
species          0
island           0
bill_length_mm   2
bill_depth_mm    2
flipper_length_mm 2
body_mass_g      2
sex              11
dtype: int64
```

```
df_clean = df.dropna().copy()
```

```
df_clean.isna().sum()
```

```
species          0
island           0
bill_length_mm   0
bill_depth_mm    0
flipper_length_mm 0
body_mass_g      0
sex              0
dtype: int64
```

Display the number of unique values for each column after removing missing data and before initial exploration.

```
df_clean.nunique()
```

```
species          3
island           3
bill_length_mm   163
bill_depth_mm    79
flipper_length_mm 54
body_mass_g      93
sex              2
dtype: int64
```

Display the number of unique values for the categorical variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(exclude='number').nunique()
```

```
species    3
island     3
sex        2
dtype: int64
```

Display the number of unique values for the continuous variables after removing missing data and before initial exploration.

```
df_clean.select_dtypes(include='number').nunique()
```

```
bill_length_mm    163
bill_depth_mm     79
flipper_length_mm  54
body_mass_g       93
dtype: int64
```

Fitting Multiple Models

Create plotting function to be used for multiple models.

```
def my_coefplot(
    model,
    var,
    xerr_mult=2,
    color='k',
    ecolor='k',
    elinewidth=2,
    ms=10,
    vline=0,
    xlabel='coefficient value',
    title=None,
    figsize=(7, 2.5),
    xlim=None):
    """
    Plots a single coefficient with error bars from a statsmodels model.

    Params:
        model: fitted statsmodels model
        var: variable name (str), e.g. 'Intercept' or 'I(flipper_length_mm ** 2)'
        xerr_mult: multiplier for error bars (default 2)
        color: marker color
        ecolor: error bar color
        elinewidth: error bar line width
        ms: marker size
        vline: where to draw the vertical reference line (default 0)
        xlabel: x-axis label (default 'coefficient value')
        title: figure title (default None, uses variable name)
        figsize: tuple, figure size
        xlim: tuple, x-axis limits (optional, auto if None)
    """
    # Get coefficient and error values
    coef = model.params[var]
    err = xerr_mult * model.bse[var]

    # Create the plot
    _, ax = plt.subplots(figsize=figsize)
    ax.errorbar(
        y = [var],
        x = [coef],
        xerr = [err],
        fmt='o',
        color=color,
        ecolor=ecolor,
        elinewidth=elinewidth,
        ms=ms
    )

    # Add vertical line for reference
    ax.axvline(x=vline,
               linestyle='--',
               linewidth=4,
               color='grey')
```



```
ax.set_xlabel(xlabel)
if title is None:
    title = f"Coefficient Plot for '{var}'"
ax.set_title(title)

# Always include zero in xlim, and add padding around error bar
bar_left = min(coef - err, vline)
bar_right = max(coef + err, vline)
pad = (bar_right - bar_left) * 0.15 if (bar_right - bar_left) > 0 else 1
auto_xlim = (bar_left - pad, bar_right + pad)

# Set x-axis limits
if xlim is not None:
    ax.set_xlim(*xlim)
else:
    ax.set_xlim(*auto_xlim)

# Show the plot
plt.show()
```

The simplest possible linear model is an unknown constant average or intercept only model where we are saying `body_mass_g` is a constant.

```
lm_fit_00 = smf.ols(formula='body_mass_g ~ 1',
                    data=df_clean). \
    fit()
```

```
lm_fit_00.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          body_mass_g      R-squared:                0.000
Model:                  OLS              Adj. R-squared:          0.000
Method:                 Least Squares    F-statistic:              nan
Date:                  Wed, 27 Aug 2025   Prob (F-statistic):       nan
Time:                  09:11:44          Log-Likelihood:          -2700.1
No. Observations:      333              AIC:                    5402.
Df Residuals:          332              BIC:                    5406.
Df Model:               0
Covariance Type:       nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    4207.0571     44.126     95.343     0.000     4120.256     4293.858
=====
Omnibus:                 30.464    Durbin-Watson:           0.972
Prob(Omnibus):            0.000    Jarque-Bera (JB):       19.874
Skew:                    0.470    Prob(JB):               4.84e-05
Kurtosis:                 2.260    Cond. No.               1.00
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
"""
```

```
lm_fit_00.params
```

```
Intercept    4207.057057
dtype: float64
```

```
lm_fit_00.bse
```

```
Intercept    44.125554
dtype: float64
```

```
lm_fit_00.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

```
              ci_lwr      ci_upr
Intercept    4120.256133  4293.857981
```

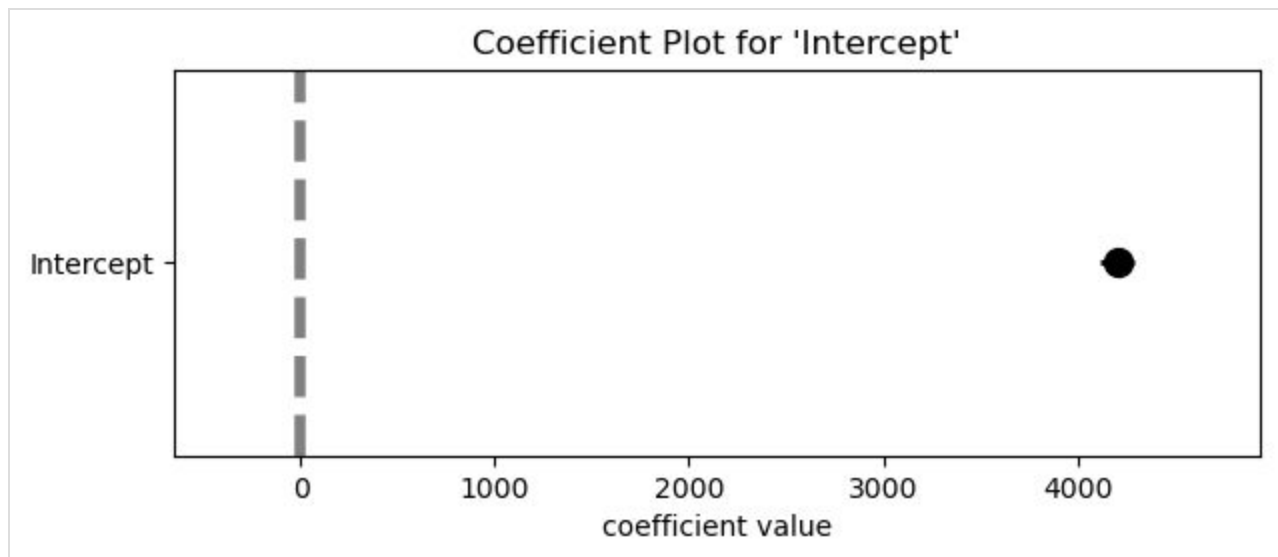
```
lm_fit_00.pvalues
```

```
Intercept    2.805040e-243  
dtype: float64
```

```
lm_fit_00.pvalues < 0.05
```

```
Intercept    True  
dtype: bool
```

```
my_coefplot(lm_fit_00, var='Intercept')
```



```
lm_fit_00.rsquared
```

```
4.440892098500626e-16
```

```
np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
804.005860159563
```

Our next step is to model the linear relationship or classic best fit line.

```
lm_fit_01 = smf.ols(formula='body_mass_g ~ flipper_length_mm',  
                    data=df_clean). \  
                    fit()
```

```
lm_fit_01.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          body_mass_g      R-squared:                0.762
Model:                  OLS              Adj. R-squared:         0.761
Method:                 Least Squares    F-statistic:             1060.
Date:                  Wed, 27 Aug 2025  Prob (F-statistic):       3.13e-105
Time:                  09:11:44          Log-Likelihood:          -2461.1
No. Observations:      333              AIC:                    4926.
Df Residuals:          331              BIC:                    4934.
Df Model:               1
Covariance Type:       nonrobust
=====
```

```
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    -5872.0927    310.285    -18.925    0.000    -6482.472    -5261.713
flipper_length_mm  50.1533      1.540     32.562    0.000      47.123      53.183
=====
```

```
Omnibus:                5.922    Durbin-Watson:                2.102
Prob(Omnibus):           0.052    Jarque-Bera (JB):          5.876
Skew:                    0.325    Prob(JB):                  0.0530
Kurtosis:                3.025    Cond. No.                  2.90e+03
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.9e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
lm_fit_01.params
```

```
Intercept          -5872.092683
flipper_length_mm    50.153266
dtype: float64
```

```
lm_fit_01.bse
```

```
Intercept          310.285155
flipper_length_mm    1.540231
dtype: float64
```

```
lm_fit_01.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	-6482.472236	-5261.713130
flipper_length_mm	47.123390	53.183142

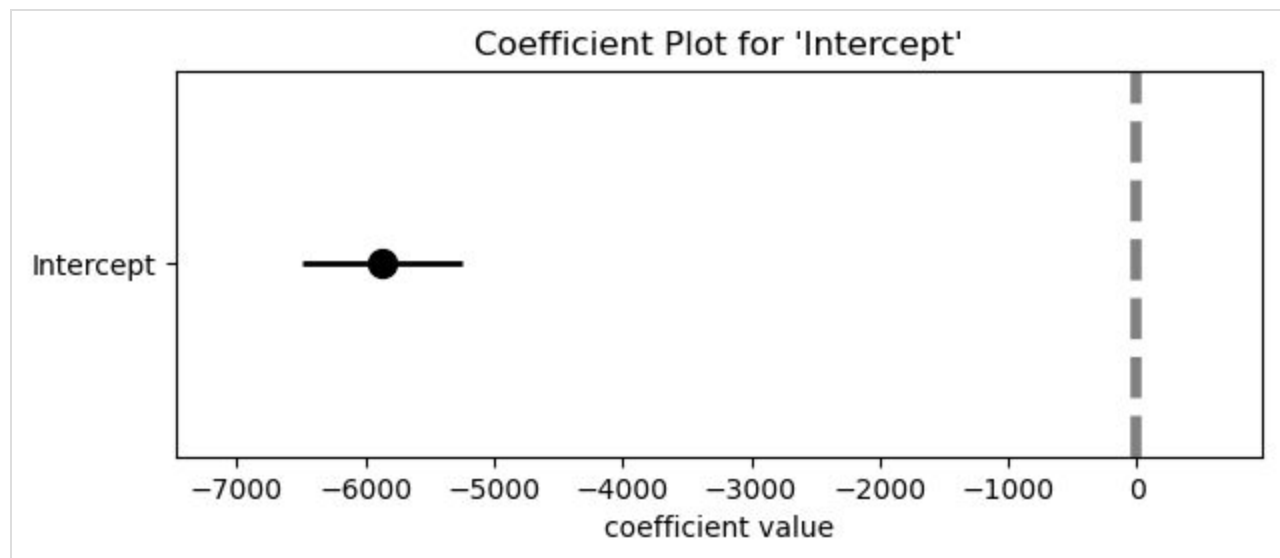
```
lm_fit_01.pvalues
```

```
Intercept          1.183941e-54  
flipper_length_mm  3.132836e-105  
dtype: float64
```

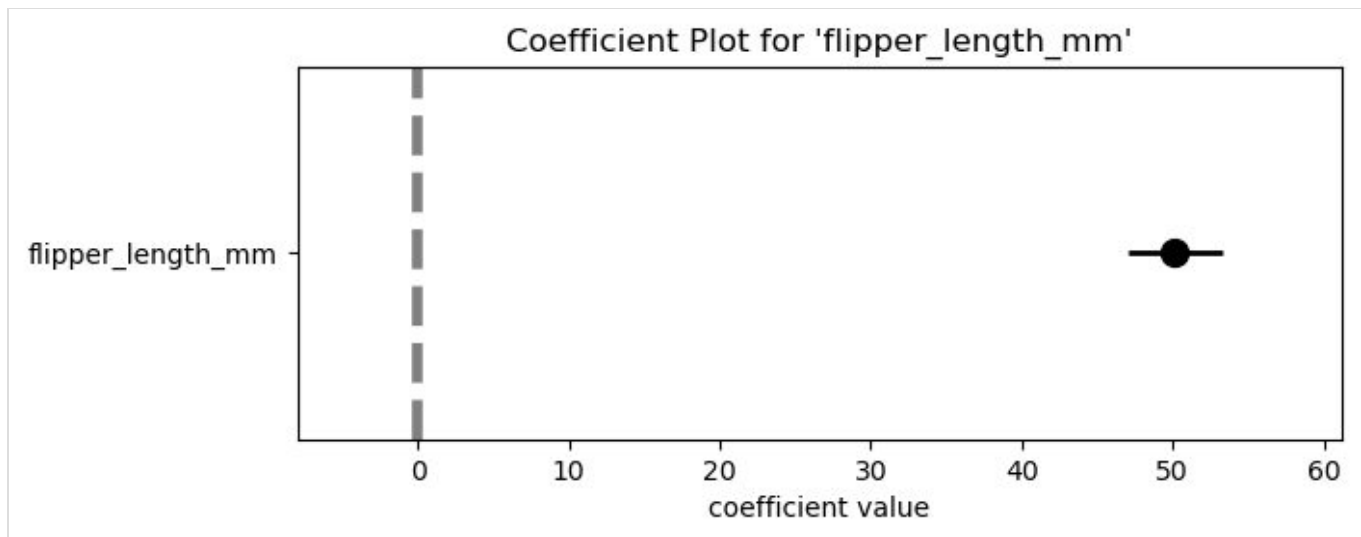
```
lm_fit_01.pvalues < 0.05
```

```
Intercept          True  
flipper_length_mm  True  
dtype: bool
```

```
my_coefplot(lm_fit_01, var='Intercept')
```



```
my_coefplot(lm_fit_01, var='flipper_length_mm')
```



```
lm_fit_01.rsquared
```

```
0.7620921573403914
```

```
np.sqrt((lm_fit_01.resid ** 2).mean())
```

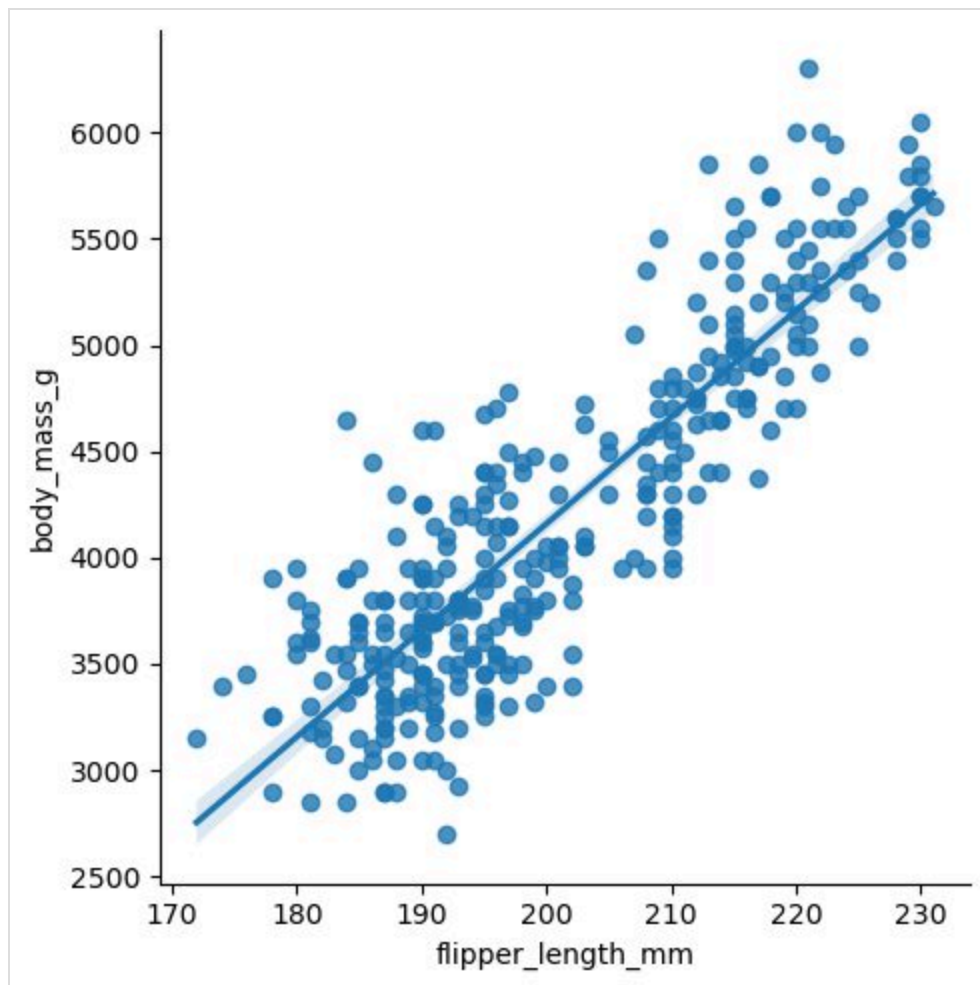
```
392.16027063806183
```

This model has a lower RMSE so it is doing better than the intercept-only model.

```
np.sqrt((lm_fit_01.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
True
```

```
sns.lmplot(data=df_clean,  
           x='flipper_length_mm',  
           y='body_mass_g')  
plt.show()
```



We will try a second degree polynomial or quadratic model.

```
lm_fit_02 = smf.ols(formula='body_mass_g ~ flipper_length_mm + np.power(flipper_length_mm,
2)',
                    data=df_clean). \
                    fit()
```

```
lm_fit_02.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          body_mass_g      R-squared:                0.778
Model:                  OLS              Adj. R-squared:          0.777
Method:                 Least Squares    F-statistic:              578.5
Date:                   Wed, 27 Aug 2025  Prob (F-statistic):      1.34e-108
Time:                   09:11:44         Log-Likelihood:           -2449.5
No. Observations:       333             AIC:                    4905.
Df Residuals:           330             BIC:                    4916.
Df Model:                2
Covariance Type:        nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.025
-----
0.975]
-----
Intercept                    1.658e+04   4616.247      3.592      0.000     7498.887
2.57e+04
flipper_length_mm           -171.6140    45.524     -3.770      0.000    -261.169
-82.059
np.power(flipper_length_mm, 2)  0.5449      0.112      4.874      0.000      0.325
0.765
=====
```

```
Omnibus:                   10.746   Durbin-Watson:              2.214
Prob(Omnibus):              0.005   Jarque-Bera (JB):           10.985
Skew:                       0.441   Prob(JB):                   0.00412
Kurtosis:                   3.119   Cond. No.                   9.07e+06
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 9.07e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
lm_fit_02.params
```

```
Intercept                16579.869215
flipper_length_mm        -171.613976
np.power(flipper_length_mm, 2)  0.544948
dtype: float64
```

```
lm_fit_02.bse
```

```
Intercept                4616.246647
flipper_length_mm        45.524470
np.power(flipper_length_mm, 2)  0.111807
dtype: float64
```



```
lm_fit_02.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	7498.887253	25660.851176
flipper_length_mm	-261.168743	-82.059210
np.power(flipper_length_mm, 2)	0.325003	0.764892

```
lm_fit_02.pvalues
```

Intercept	0.000379
flipper_length_mm	0.000194
np.power(flipper_length_mm, 2)	0.000002

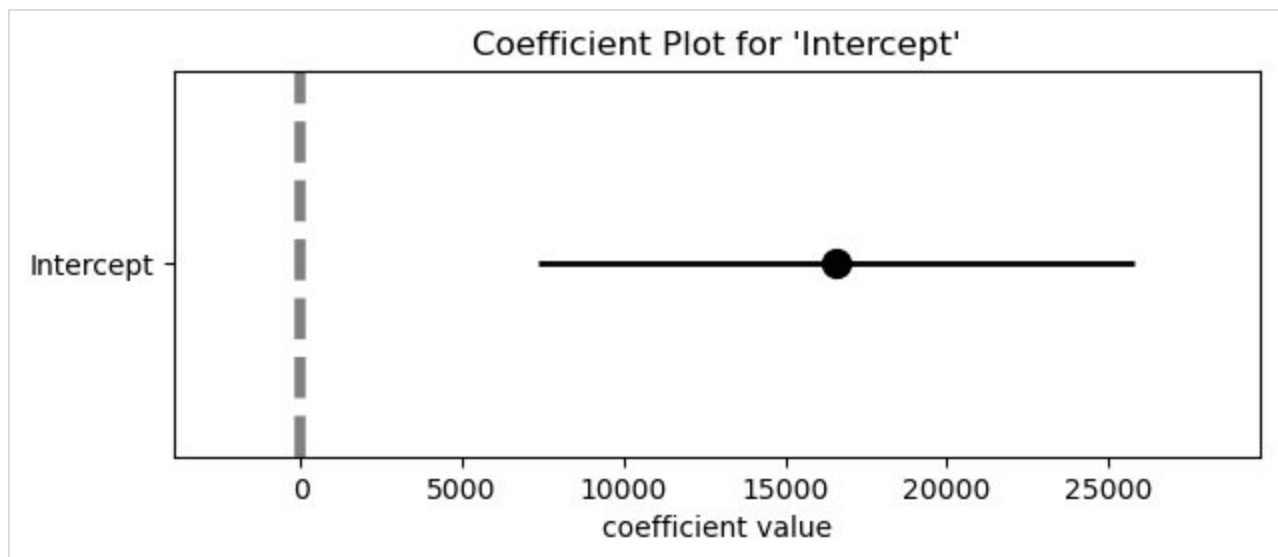
dtype: float64

```
lm_fit_02.pvalues < 0.05
```

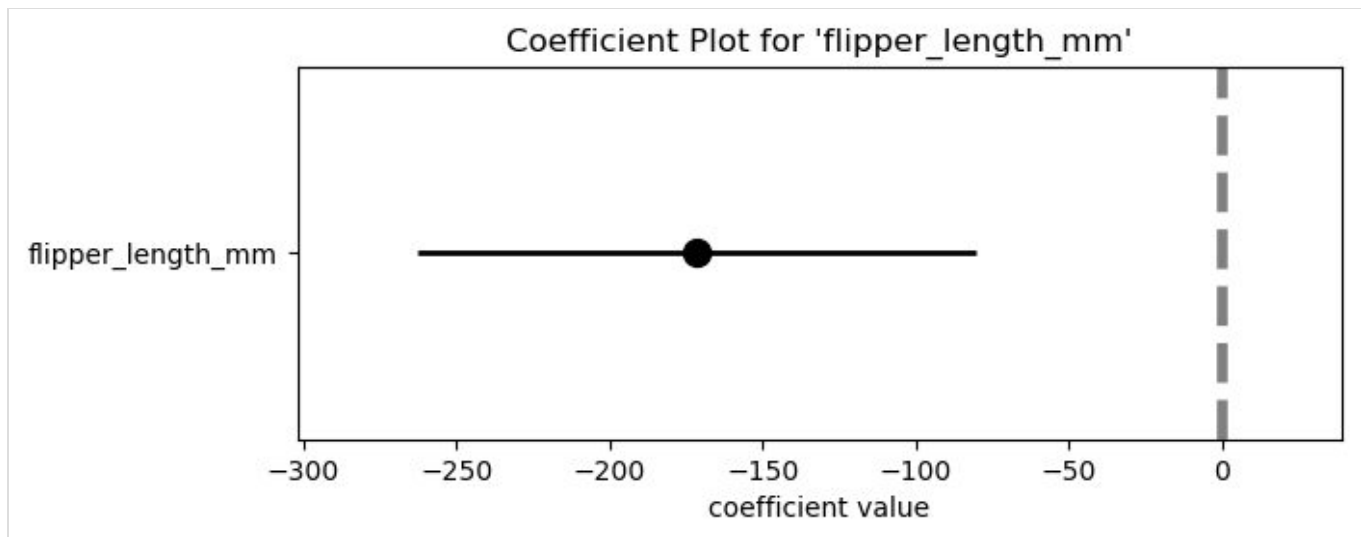
Intercept	True
flipper_length_mm	True
np.power(flipper_length_mm, 2)	True

dtype: bool

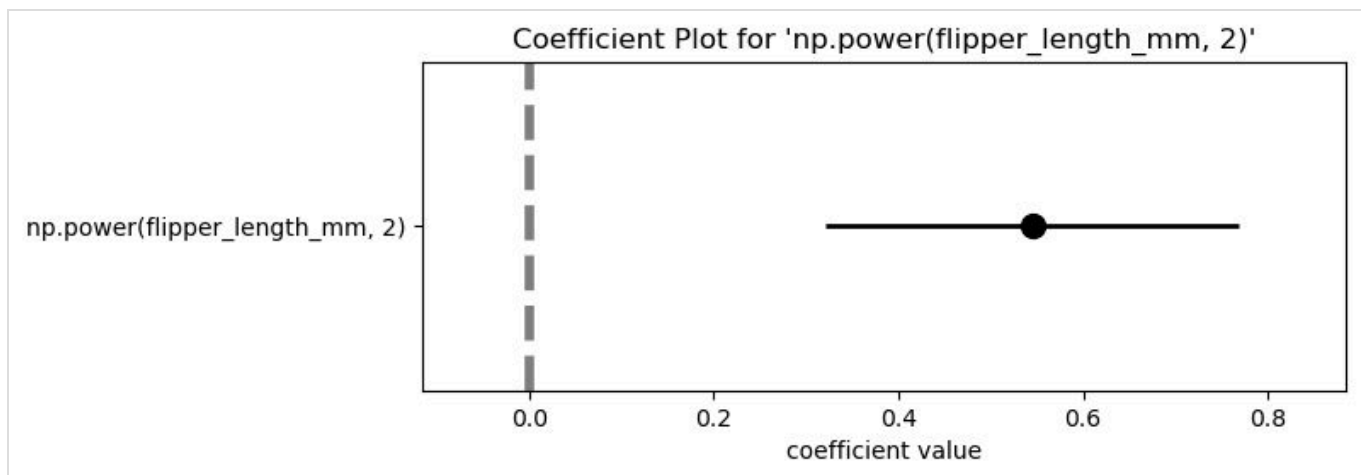
```
my_coefplot(lm_fit_02, var='Intercept')
```



```
my_coefplot(lm_fit_02, var='flipper_length_mm')
```



```
my_coefplot(lm_fit_02, var='np.power(flipper_length_mm, 2)')
```



```
lm_fit_02.rsquared
```

```
0.7780684324476418
```

```
np.sqrt((lm_fit_02.resid ** 2).mean())
```

```
378.7640525050107
```

This model has a lower RMSE than the other two so it is doing better.

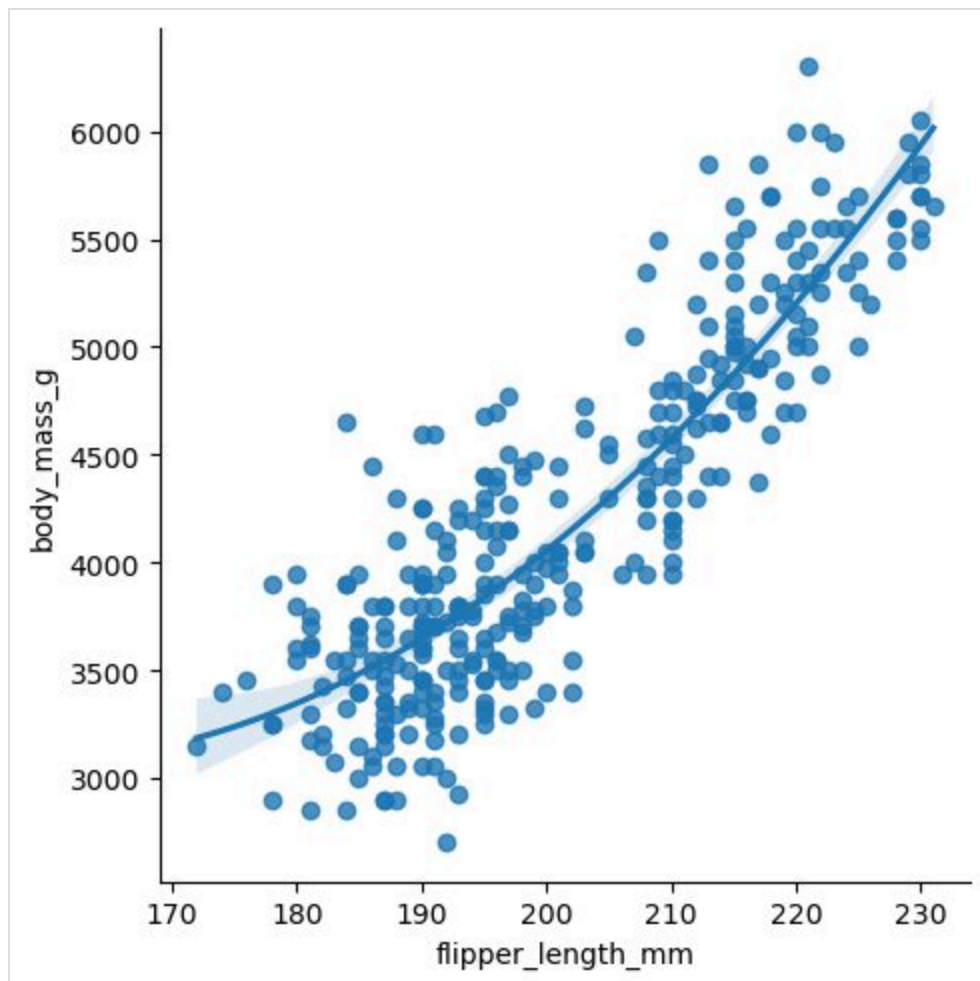
```
np.sqrt((lm_fit_02.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
True
```

```
np.sqrt((lm_fit_02.resid ** 2).mean()) < np.sqrt((lm_fit_01.resid ** 2).mean())
```

True

```
sns.lmplot(data=df_clean,  
           x='flipper_length_mm',  
           y='body_mass_g',  
           order=2)  
plt.show()
```



We will review a cubic model.

```
lm_fit_03 = smf.ols(formula='body_mass_g ~ flipper_length_mm + np.power(flipper_length_mm,  
2) + np.power(flipper_length_mm, 3)',  
                    data=df_clean). \  
                    fit()
```

```
lm_fit_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          body_mass_g      R-squared:                0.784
Model:                  OLS              Adj. R-squared:         0.782
Method:                 Least Squares    F-statistic:             397.5
Date:                   Wed, 27 Aug 2025  Prob (F-statistic):      5.07e-109
Time:                   09:11:44         Log-Likelihood:          -2445.2
No. Observations:       333              AIC:                   4898.
Df Residuals:           329              BIC:                   4914.
Df Model:                3
Covariance Type:        nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.025
-----
0.975]
-----
Intercept                    1.795e+05   5.55e+04     3.232     0.001    7.02e+04
2.89e+05
flipper_length_mm           -2591.3912   823.371    -3.147     0.002   -4211.126
-971.656
np.power(flipper_length_mm, 2)  12.4909     4.060     3.076     0.002     4.504
20.478
np.power(flipper_length_mm, 3)  -0.0196     0.007    -2.943     0.003    -0.033
-0.006
=====
```

```
Omnibus:                    11.383   Durbin-Watson:           2.320
Prob(Omnibus):              0.003   Jarque-Bera (JB):        11.688
Skew:                       0.454   Prob(JB):                0.00290
Kurtosis:                   3.130   Cond. No.                2.27e+10
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.27e+10. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
lm_fit_03.params
```

```
Intercept                    179467.306901
flipper_length_mm           -2591.391180
np.power(flipper_length_mm, 2)  12.490930
np.power(flipper_length_mm, 3)  -0.019598
dtype: float64
```

```
lm_fit_03.bse
```

```
Intercept                55530.217107
flipper_length_mm        823.370574
np.power(flipper_length_mm, 2)    4.060253
np.power(flipper_length_mm, 3)    0.006659
dtype: float64
```

```
lm_fit_03.conf_int(). \
    rename(columns={0: 'ci_lwr',
                    1: 'ci_upr'})
```

	ci_lwr	ci_upr
Intercept	70228.225861	288706.387940
flipper_length_mm	-4211.126337	-971.656024
np.power(flipper_length_mm, 2)	4.503598	20.478261
np.power(flipper_length_mm, 3)	-0.032697	-0.006499

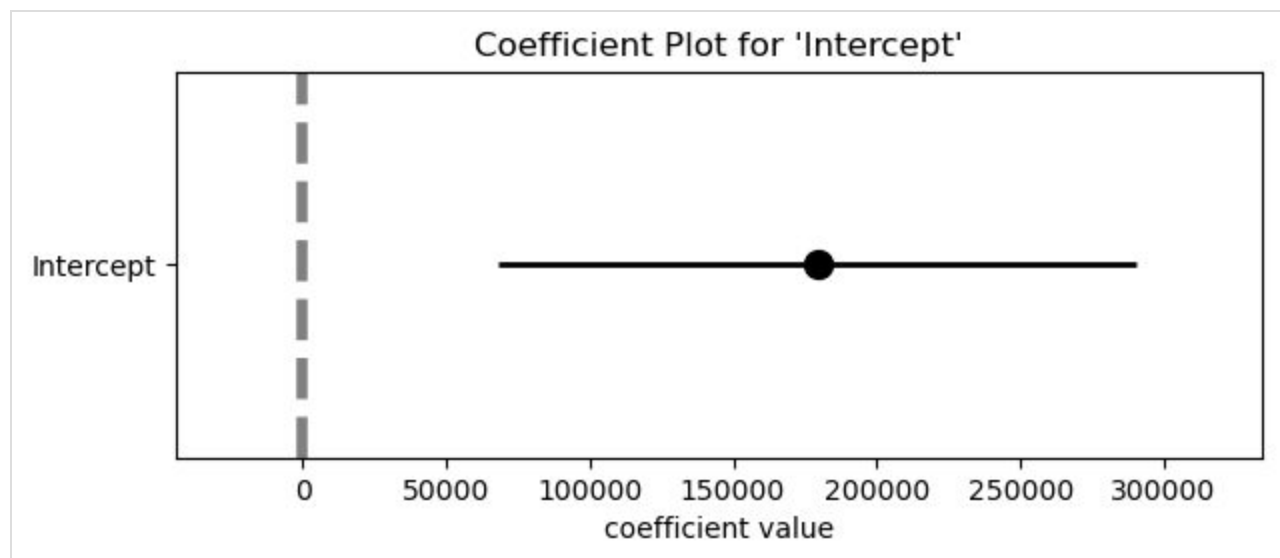
```
lm_fit_03.pvalues
```

```
Intercept                0.001354
flipper_length_mm        0.001799
np.power(flipper_length_mm, 2)    0.002271
np.power(flipper_length_mm, 3)    0.003478
dtype: float64
```

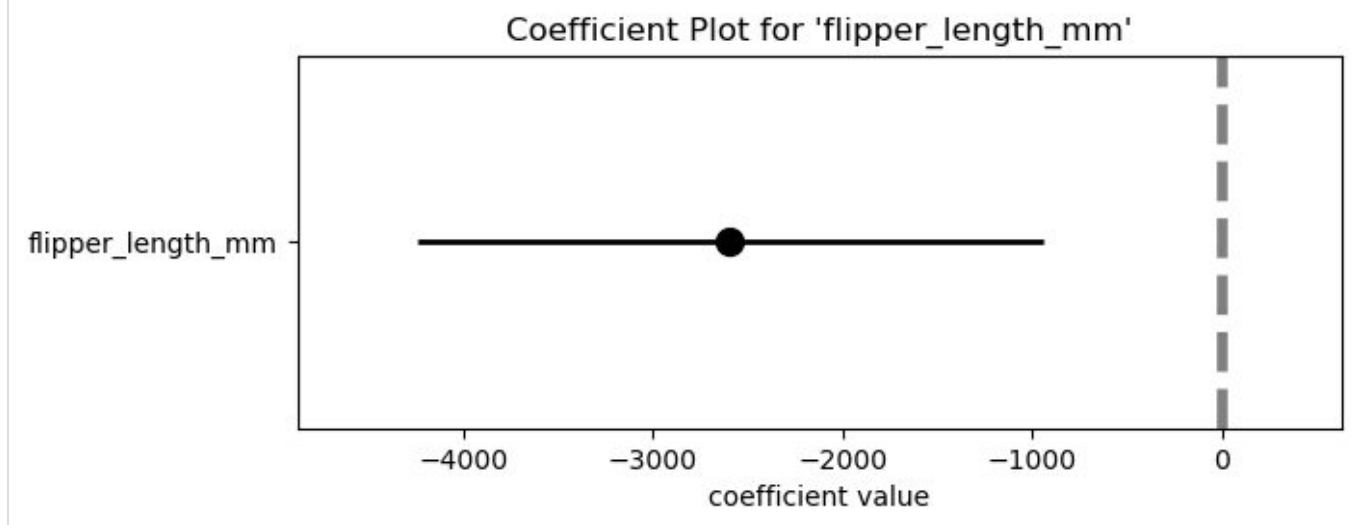
```
lm_fit_03.pvalues < 0.05
```

```
Intercept                True
flipper_length_mm        True
np.power(flipper_length_mm, 2)    True
np.power(flipper_length_mm, 3)    True
dtype: bool
```

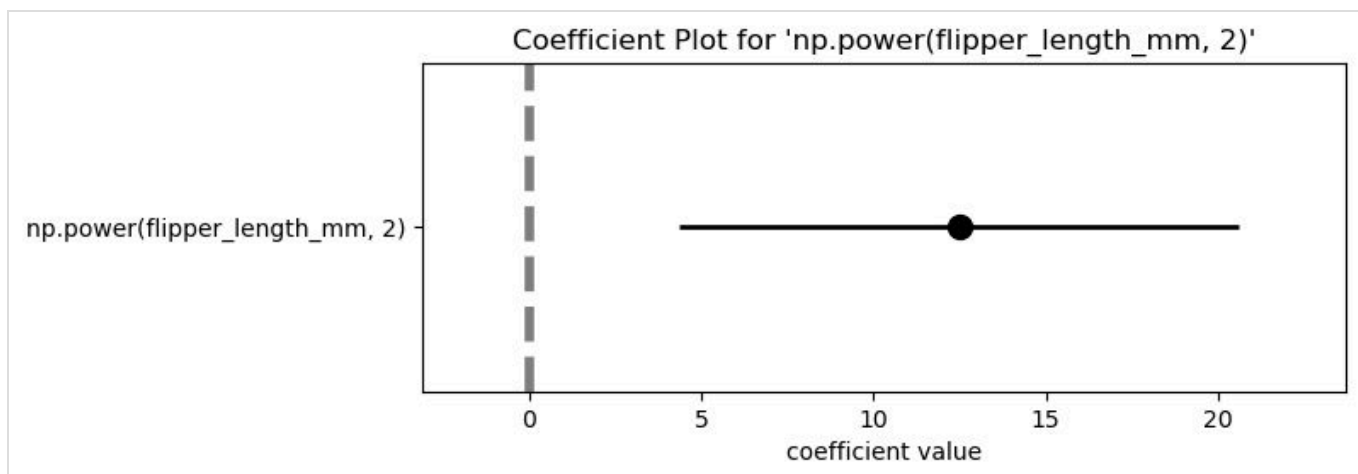
```
my_coefplot(lm_fit_03, var='Intercept')
```



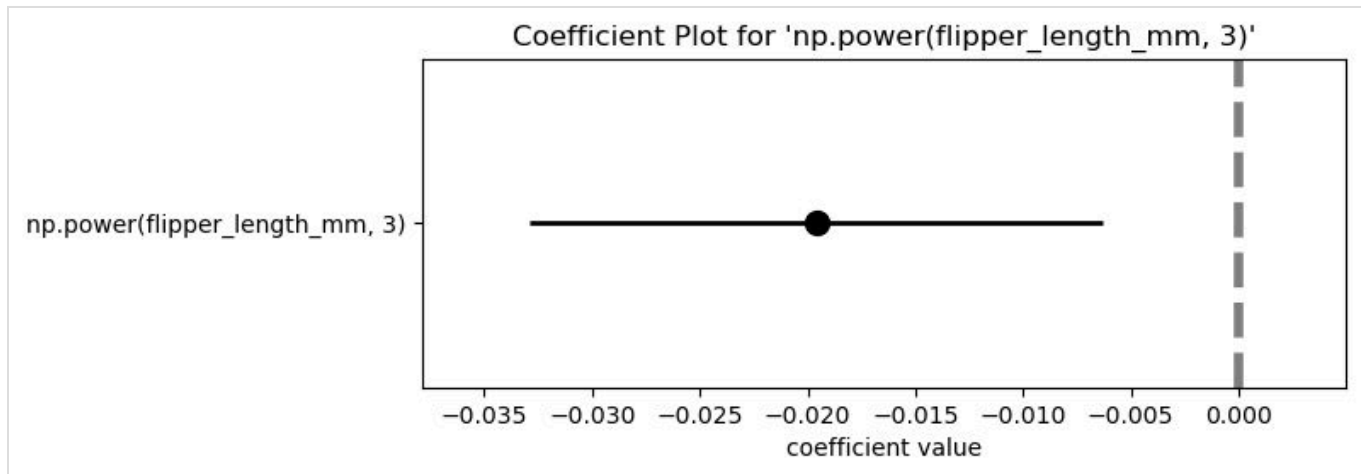
```
my_coefplot(lm_fit_03, var='flipper_length_mm')
```



```
my_coefplot(lm_fit_03, var='np.power(flipper_length_mm, 2)')
```



```
my_coefplot(lm_fit_03, var='np.power(flipper_length_mm, 3)')
```



```
lm_fit_03.rsquared
```

```
0.783762142815277
```

This model has a lower RMSE than the other three so it is doing best so far.

```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_00.resid ** 2).mean())
```

```
True
```

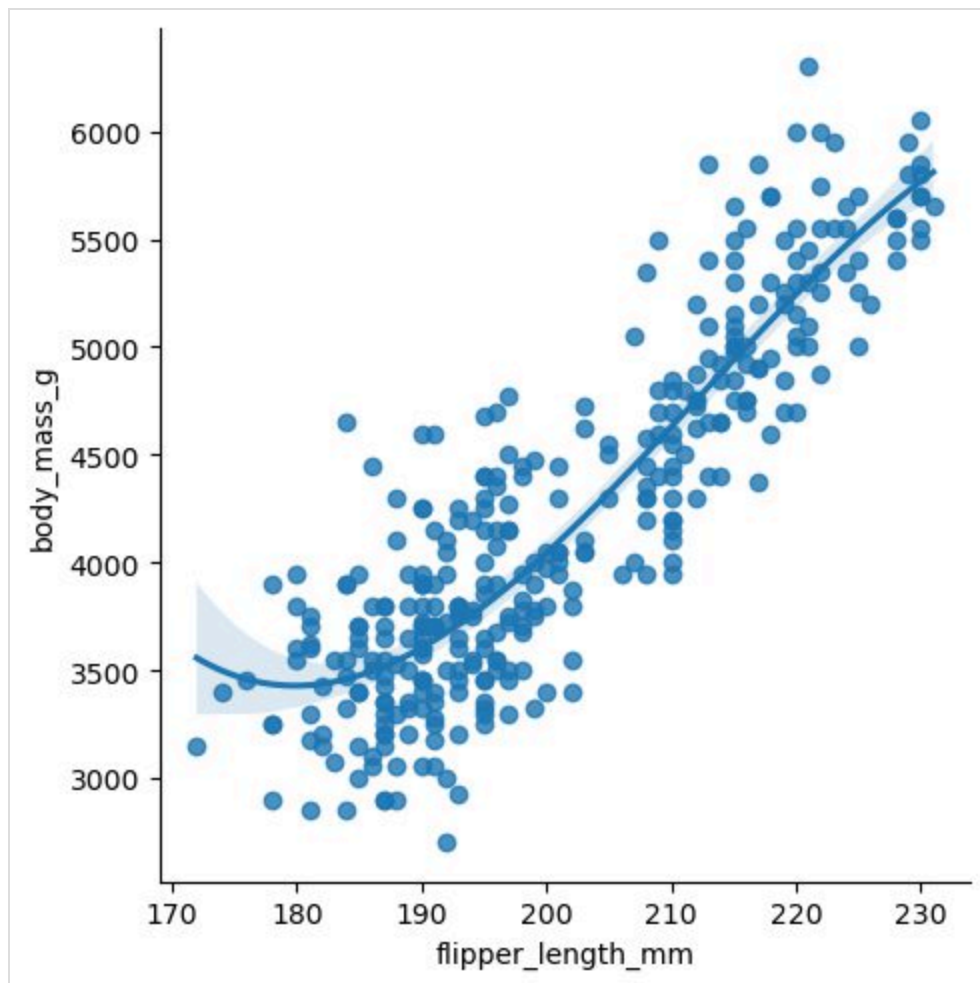
```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_01.resid ** 2).mean())
```

```
True
```

```
np.sqrt((lm_fit_03.resid ** 2).mean()) < np.sqrt((lm_fit_02.resid ** 2).mean())
```

```
True
```

```
sns.lmplot(data=df_clean,
            x='flipper_length_mm',
            y='body_mass_g',
            order=3)
plt.show()
```



Part III: Multiple Linear Regression and Linear Regression

11-MLR-AF: Multiple Linear Regression - Additive Features

Author: [Kevin Thomas](#)

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Understanding How Additive Features Affect Data

Additive features add the effect of input 2 to the effect of input 1.

$$\mu = \beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x_1

x_1 **feature** or **input variable**: one of the independent variables used to predict μ

β_2 **slope**: change in μ for a one-unit increase in x_2

x_2 **feature** or **input variable**: one of the independent variables used to predict μ

Additive linear relationships is where we calculate the average output or trend given input 1 and input 2 and the 3 regression coefficients as we will create a function that will allow visualization of the trend with respect to input 1.

```
def calc_trend_wrt_x1(x1,
                     x2,
                     b0,
                     b1,
                     b2):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the
    provided coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated
        trend.
    """
    res_df = pd.DataFrame({'x1': x1})
    res_df['x2'] = x2
    res_df['trend'] = b0 + b1 * res_df.x1 + b2 * res_df.x2
    return res_df
```

To demonstrate the relationships, let's define the following coefficients.

```
b0 = -0.25
b1 = 1.95
b2 = 0.2
```

Let's define 101 evenly or uniformly spaced values of `x1` between -3 and 3.

```
x1_values = np.linspace(-3,
                        3,
                        101)
```

```
x1_values
```

```
array([-3.   , -2.94, -2.88, -2.82, -2.76, -2.7 , -2.64, -2.58, -2.52,
       -2.46, -2.4 , -2.34, -2.28, -2.22, -2.16, -2.1 , -2.04, -1.98,
       -1.92, -1.86, -1.8 , -1.74, -1.68, -1.62, -1.56, -1.5 , -1.44,
       -1.38, -1.32, -1.26, -1.2 , -1.14, -1.08, -1.02, -0.96, -0.9 ,
       -0.84, -0.78, -0.72, -0.66, -0.6 , -0.54, -0.48, -0.42, -0.36,
       -0.3 , -0.24, -0.18, -0.12, -0.06,  0.   ,  0.06,  0.12,  0.18,
        0.24,  0.3 ,  0.36,  0.42,  0.48,  0.54,  0.6 ,  0.66,  0.72,
        0.78,  0.84,  0.9 ,  0.96,  1.02,  1.08,  1.14,  1.2 ,  1.26,
        1.32,  1.38,  1.44,  1.5 ,  1.56,  1.62,  1.68,  1.74,  1.8 ,
        1.86,  1.92,  1.98,  2.04,  2.1 ,  2.16,  2.22,  2.28,  2.34,
        2.4 ,  2.46,  2.52,  2.58,  2.64,  2.7 ,  2.76,  2.82,  2.88,
        2.94,  3.   ])
```

```
x1_values.size
```

```
101
```

```
x1_values.ndim
```

```
1
```

Let's calculate the average output for a single value of `x2` at 0 where `x2` has 1 unique value 0, and it appears 101 times.

```
calc_trend_wrt_x1(x1_values,
                  0,
                  b0,
                  b1,
                  b2)
```

	x1	x2	trend
0	-3.00	0	-6.100
1	-2.94	0	-5.983
2	-2.88	0	-5.866
3	-2.82	0	-5.749
4	-2.76	0	-5.632
..	...	..	...
96	2.76	0	5.132
97	2.82	0	5.249
98	2.88	0	5.366
99	2.94	0	5.483
100	3.00	0	5.600

```
[101 rows x 3 columns]
```

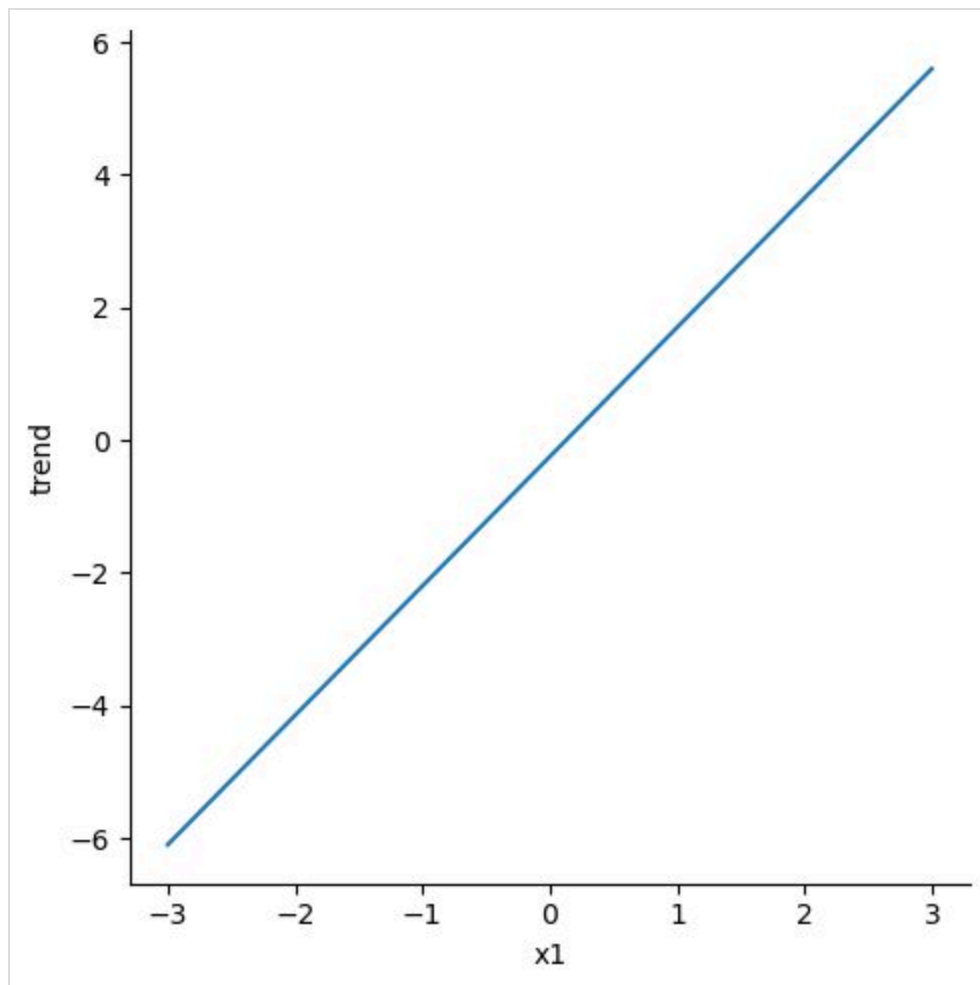
```
calc_trend_wrt_x1(x1_values,
                  0,
                  b0,
                  b1,
                  b2). \
x2.value_counts()
```

```
x2
0    101
Name: count, dtype: int64
```

Let's visualize the relationship between the average output and input 1 where the average output has roughly a change of 2 on the trend for a 1 unit change on the input as the slope is how much does the output change on average for a one unit change of the input.

```
sns.relplot(data=calc_trend_wrt_x1(x1_values,
                                   0,
                                   b0,
                                   b1,
                                   b2),
            x='x1',
            y='trend',
            kind='line')

plt.show()
```



Now we will change x_2 from 0 to -2 as we can visualize the new trend as a 1 unit change in the input produces an almost 2 unit change on the trend but it is hard to see any difference from the prior graphic.

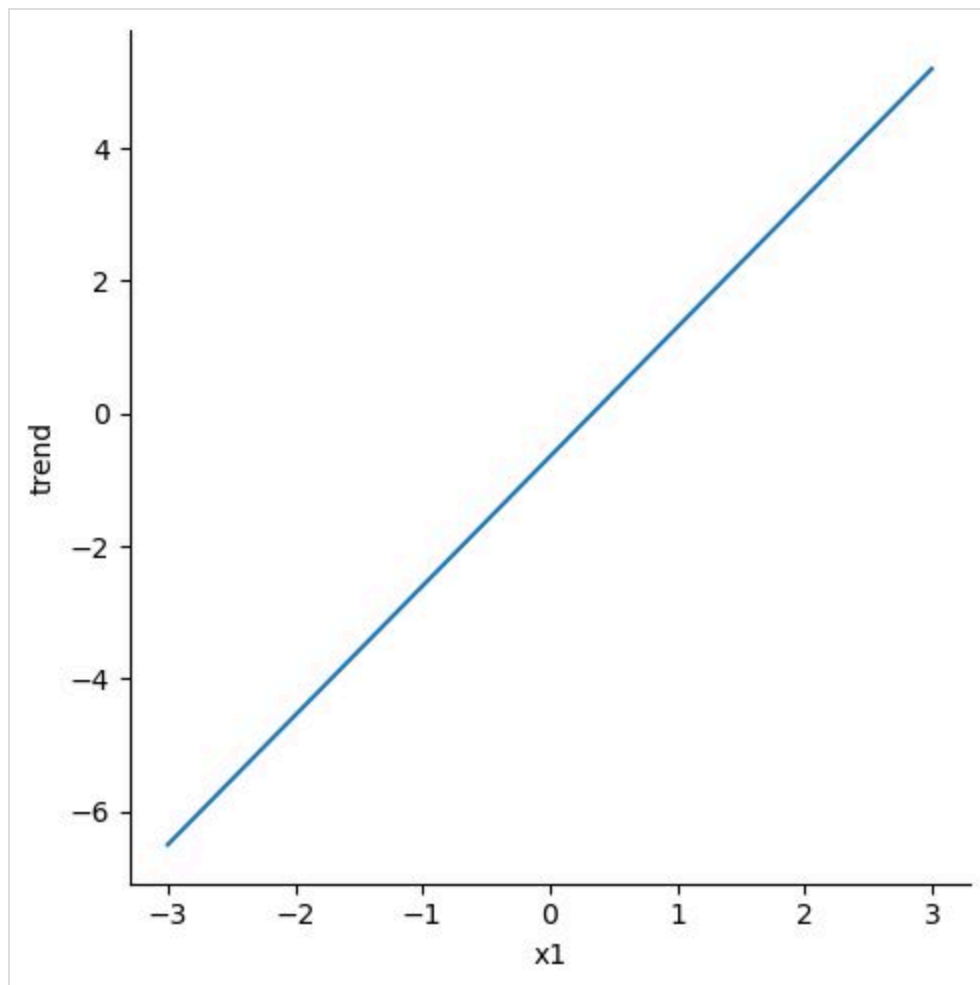
```
calc_trend_wrt_x1(x1_values,  
                  -2,  
                  b0,  
                  b1,  
                  b2)
```

	x1	x2	trend
0	-3.00	-2	-6.500
1	-2.94	-2	-6.383
2	-2.88	-2	-6.266
3	-2.82	-2	-6.149
4	-2.76	-2	-6.032
...	...	...	...
96	2.76	-2	4.732
97	2.82	-2	4.849
98	2.88	-2	4.966
99	2.94	-2	5.083
100	3.00	-2	5.200

[101 rows x 3 columns]

```
sns.relplot(data=calc_trend_wrt_x1(x1_values,
                                   -2,
                                   b0,
                                   b1,
                                   b2),
            x='x1',
            y='trend',
            kind='line')

plt.show()
```



Let's repeat or replicate calculating the trend with respect to `x1` for different values of `x2`.

```
x2_values = np.linspace(-3,  
                        3,  
                        9)
```

```
x2_values
```

```
array([-3. , -2.25, -1.5 , -0.75,  0.  ,  0.75,  1.5 ,  2.25,  3.  ])
```

```
study_wrt_x1_list = [calc_trend_wrt_x1(x1_values, x2, b0, b1, b2) for x2 in x2_values]
```

```
len(study_wrt_x1_list)
```

```
9
```

```
study_wrt_x1_list[0]
```



```

      x1  x2  trend
0  -3.00 -3.0 -6.700
1  -2.94 -3.0 -6.583
2  -2.88 -3.0 -6.466
3  -2.82 -3.0 -6.349
4  -2.76 -3.0 -6.232
..    ...  ...    ...
96   2.76 -3.0  4.532
97   2.82 -3.0  4.649
98   2.88 -3.0  4.766
99   2.94 -3.0  4.883
100  3.00 -3.0  5.000

[101 rows x 3 columns]

```

Combine or concat the separate DataFrames together.

```
study_wrt_x1_df = pd.concat(study_wrt_x1_list, ignore_index=True)
```

```
study_wrt_x1_df
```

```

      x1  x2  trend
0  -3.00 -3.0 -6.700
1  -2.94 -3.0 -6.583
2  -2.88 -3.0 -6.466
3  -2.82 -3.0 -6.349
4  -2.76 -3.0 -6.232
..    ...  ...    ...
904  2.76  3.0  5.732
905  2.82  3.0  5.849
906  2.88  3.0  5.966
907  2.94  3.0  6.083
908  3.00  3.0  6.200

[909 rows x 3 columns]

```

```
study_wrt_x1_df.x2.value_counts()
```

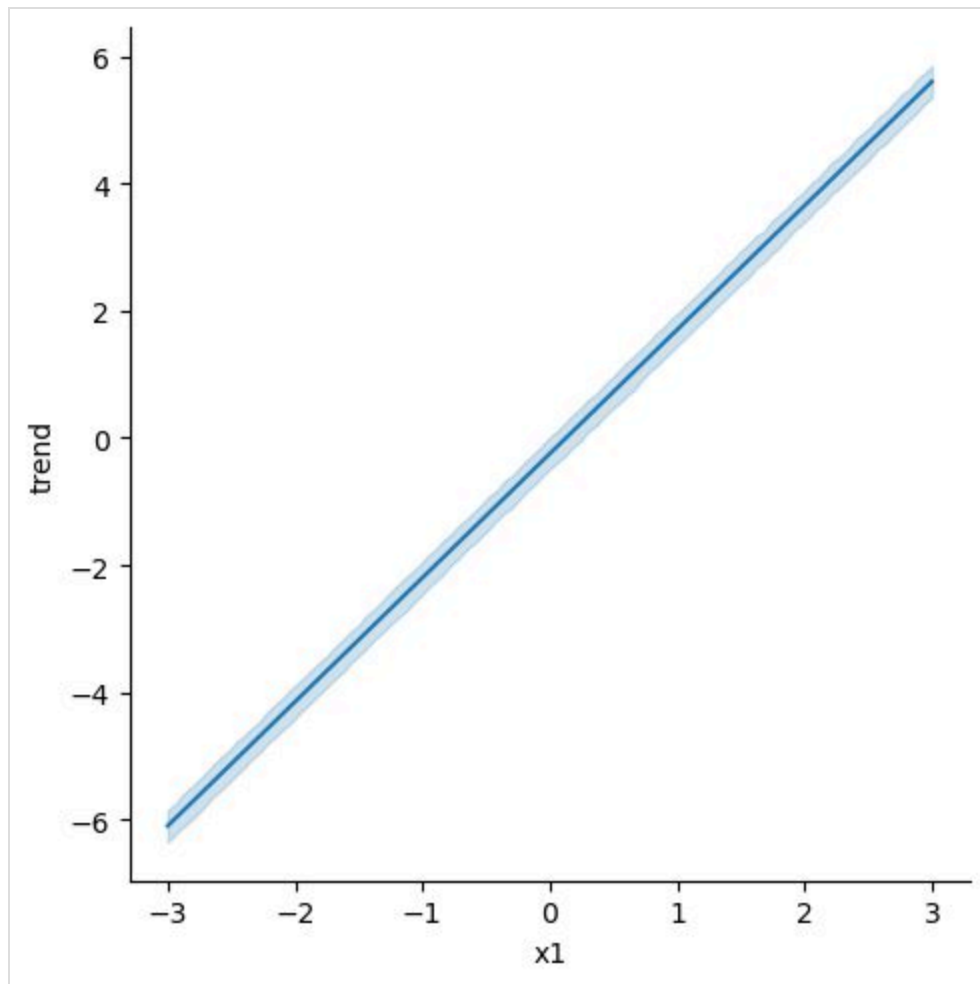
```

x2
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64

```

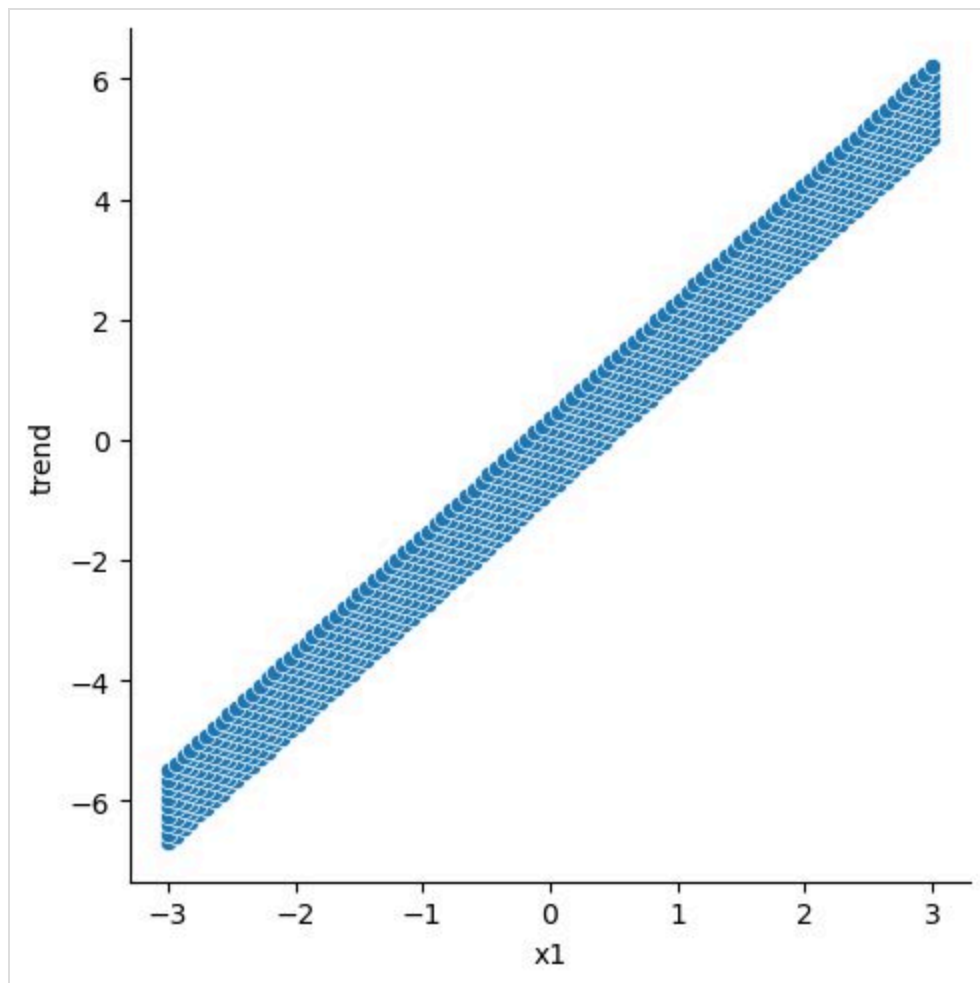
Visualize the trend or average output with respect to `x1` for each unique value of `x2` as a line chart as it will include the 95% confidence interval as this is a summation of the data.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='line')  
plt.show()
```



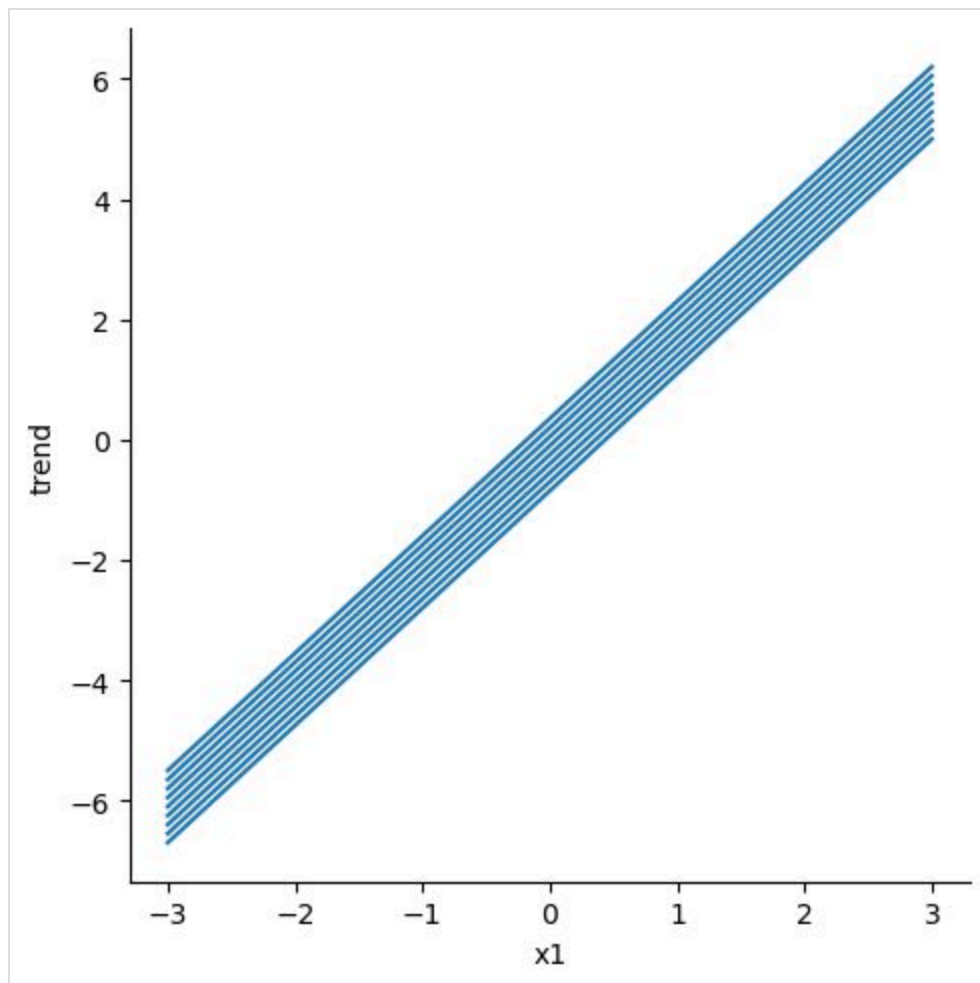
Visualize the raw data.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='scatter')  
plt.show()
```



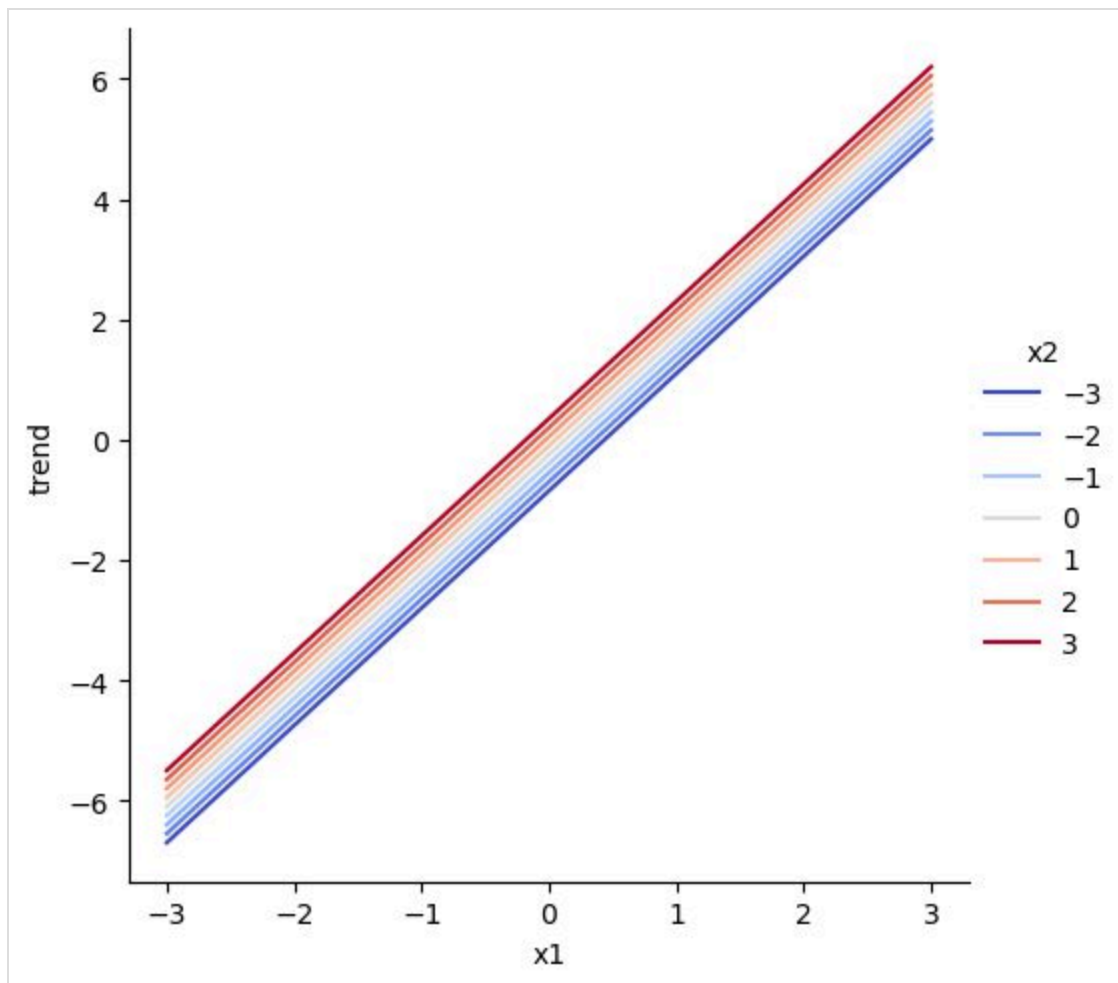
We need to disable Seaborn's default line chart averaging.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            kind='line',  
            estimator=None,  
            units='x2')  
plt.show()
```



We can associate the line color to `x2` via the `hue` argument.

```
sns.relplot(data=study_wrt_x1_df,  
            x='x1',  
            y='trend',  
            hue='x2',  
            kind='line',  
            palette='coolwarm',  
            estimator=None,  
            units='x2')  
plt.show()
```



WHEN WE HAVE ADDITIVE FEATURES WHICH ARE ADDING THE EFFECT OF INPUT 2 TO THE EFFECT OF INPUT 1, WE ARE LITERALLY SHIFTING THE LINE UP OR DOWN.

Additive features add the effect of one input to the other and if the inputs have the same magnitude of and scale, the magnitude of the slope tells you which input causes the greater change on the average output.

b1

1.95

In the above we have the slope β_1 very close to 2, therefore, a 1 unit change in `x1` is producing a 2 unit change in the trend or average output.

Let's focus on the relationship with respect to `x2`.

```
def calc_trend_wrt_x2(x1,
                     x2,
                     b0,
                     b1,
                     b2):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the
    provided coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated
        trend.
    """
    res_df = pd.DataFrame({'x2': x2})
    res_df['x1'] = x1
    res_df['trend'] = b0 + b1 * res_df.x1 + b2 * res_df.x2
    return res_df
```

Let's define 2 new arrays so we can visualize the trends with respect to `x2` for different values of `x1`.

```
x2_values_b = np.linspace(-3, 3, 101)
```

```
x2_values_b.shape
```

```
(101,)
```

```
x1_values_b = np.linspace(-3, 3, 9)
```

```
x1_values_b.shape
```

```
(9,)
```

Iterate over values `x1`.

```
study_wrt_x2_list = [calc_trend_wrt_x2(x1, x2_values_b, b0, b1, b2) for x1 in x1_values_b]
```

```
len(study_wrt_x2_list)
```

```
9
```

```
study_wrt_x2_df = pd.concat(study_wrt_x2_list, ignore_index=True)
```

```
study_wrt_x2_df
```

```

      x2  x1  trend
0  -3.00 -3.0 -6.700
1  -2.94 -3.0 -6.688
2  -2.88 -3.0 -6.676
3  -2.82 -3.0 -6.664
4  -2.76 -3.0 -6.652
..    ...  ...    ...
904  2.76  3.0  6.152
905  2.82  3.0  6.164
906  2.88  3.0  6.176
907  2.94  3.0  6.188
908  3.00  3.0  6.200

```

```
[909 rows x 3 columns]
```

```
study_wrt_x2_df.x1.value_counts()
```

```

x1
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64

```

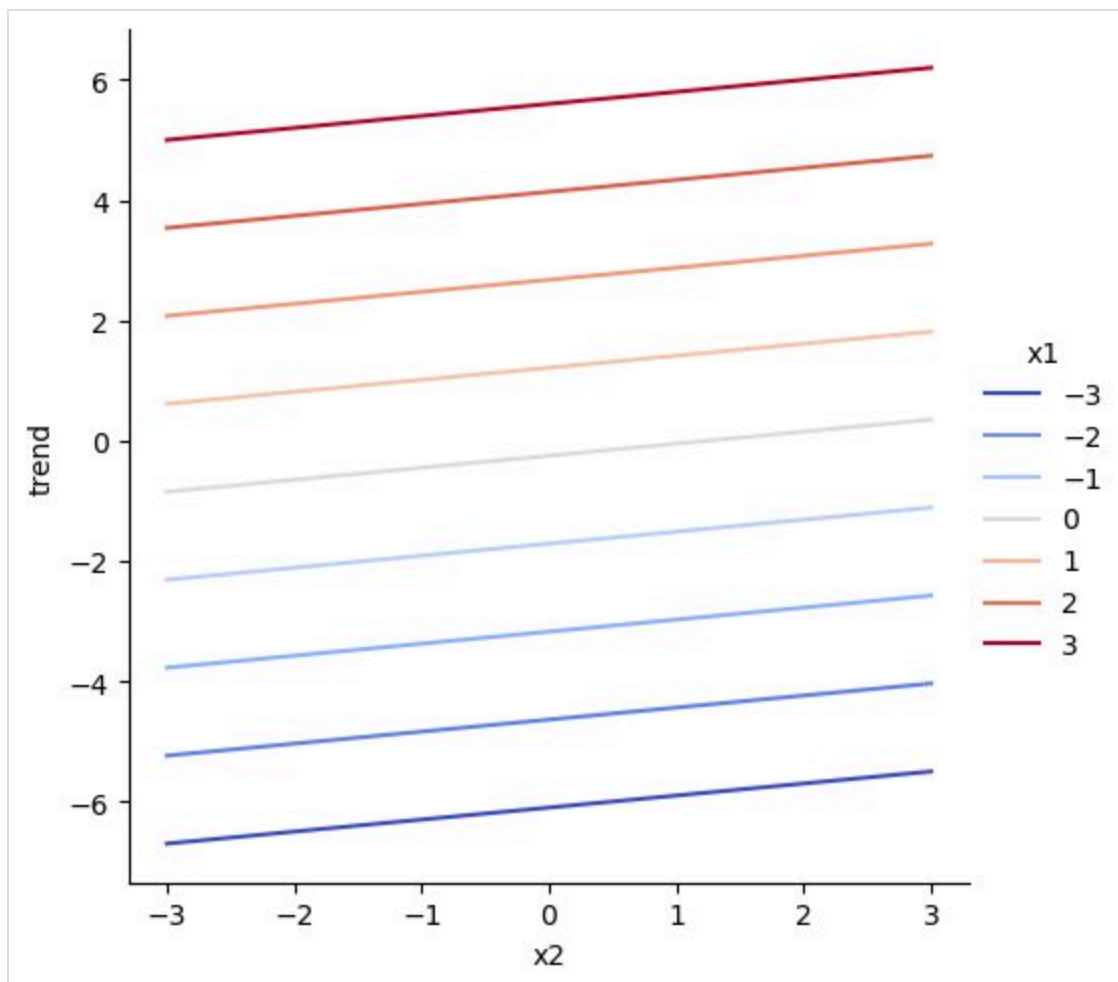
Visualize the trend or average output with respect to `x2` for each unique value of `x1` as we will use the diverging color palette to help distinguish `x1` values above and below the `x1` midpoint.

```
b2
```

```
0.2
```

In the above we have the slope β_2 0.2, therefore, a 1 unit change in x_2 is producing a very small change in the trend or average output.

```
sns.relplot(data=study_wrt_x2_df,  
            x='x2',  
            y='trend',  
            hue='x1',  
            kind='line',  
            palette='coolwarm',  
            estimator=None,  
            units='x1')  
plt.show()
```



12-MLR-IF: Multiple Linear Regression - Interactive Features

Author: [Kevin Thomas](#)

Import Modules

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Understanding How Interactive Features Affect Data

Interactive features include the linear main effects of each input and also the product of the two inputs, which serves as the interaction term.

The interaction feature captures how the effect on the output depends on both input variables simultaneously.

$$\mu = \beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2 + \beta_3 \times x_1 \times x_2$$

μ **predicted trend**

β_0 **intercept**: value of μ when $x = 0$

β_1 **slope**: change in μ for a one-unit increase in x_1

x_1 **feature** or **input variable**: one of the independent variables used to predict μ

β_2 **slope**: change in μ for a one-unit increase in x_2

x_2 **feature** or **input variable**: one of the independent variables used to predict μ

β_3 **slope**: change in μ for a one-unit increase in the product $x_1 \times x_2$

Let's define a function that calculates the trend or average output focusing on the relationship with x_1 where we will have the interaction features which equals the product of the two inputs.

```
def calc_trend_wrt_x1_w_interactions(x1,
                                     x2,
                                     b0,
                                     b1,
                                     b2,
                                     b3):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the
    provided coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.
        b3 (float): Coefficient for the interaction term (x1 * x2).

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated
        trend.
    """
    res_df = pd.DataFrame({'x1': x1})
    res_df['x2'] = x2
    res_df['trend'] = b0 + \
        b1 * \
        res_df.x1 + \
        b2 * \
        res_df.x2 + \
        b3 * \
        res_df.x1 * \
        res_df.x2

    return res_df
```

We need a slope defined that multiplies the interaction feature.

```
b0 = -0.25
b1 = 1.95
b2 = 0.2
b3 = 1
```

```
x1_values = np.linspace(-3,
                        3,
                        101)
```

```
x2_values = np.linspace(-3,
                        3,
                        9)
```

Let's calculate the trend with respect to `x1` for different values of `x2`.

```
study_interaction_wrt_x1_list = [calc_trend_wrt_x1_w_interactions(x1_values, x2, b0, b1,
b2, b3) for x2 in x2_values]
```

```
len(study_interaction_wrt_x1_list)
```

```
9
```

```
study_interaction_wrt_x1_df = pd.concat(study_interaction_wrt_x1_list, ignore_index=True)
```

```
study_interaction_wrt_x1_df
```

```

      x1  x2  trend
0  -3.00 -3.0  2.300
1  -2.94 -3.0  2.237
2  -2.88 -3.0  2.174
3  -2.82 -3.0  2.111
4  -2.76 -3.0  2.048
..    ...  ...    ...
904  2.76  3.0 14.012
905  2.82  3.0 14.309
906  2.88  3.0 14.606
907  2.94  3.0 14.903
908  3.00  3.0 15.200
```

```
[909 rows x 3 columns]
```

```
study_interaction_wrt_x1_df.x2.value_counts()
```

```

x2
-3.00    101
-2.25    101
-1.50    101
-0.75    101
 0.00    101
 0.75    101
 1.50    101
 2.25    101
 3.00    101
Name: count, dtype: int64
```

Let's visualize the average output or trend with respect to x_1 for each x_2 unique value.

We still have 9 unique values of x_2 however the slopes are not shifting up or down as we now have different slopes.

When x_2 is -3, the trend or average output is decreasing for a 1 unit change of input 1 and when x_2 is 3, the trend or average output is increasing for a 1 unit change of input 1.

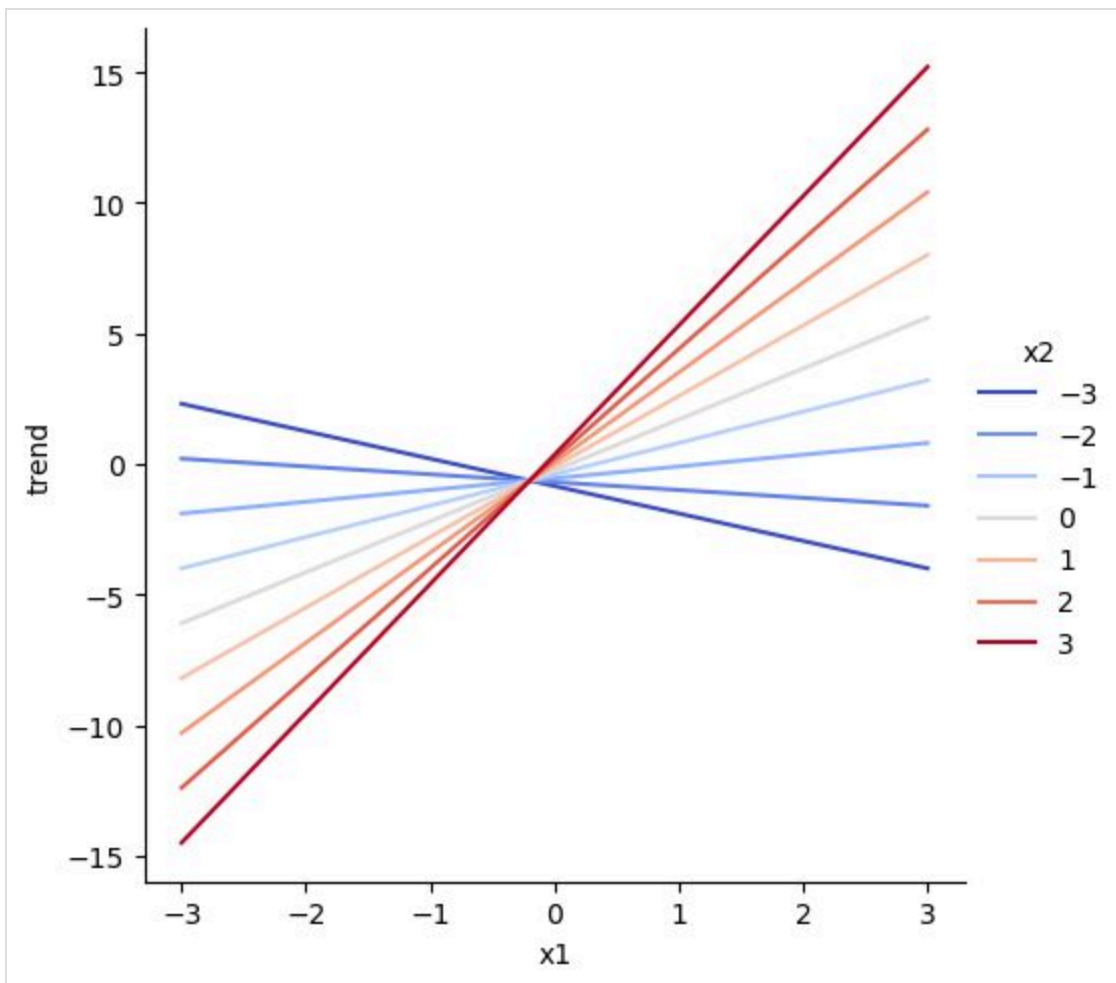
b_1

1.95

We know β_1 is > 0 , positive, and yet we have a negative slope, however, when x_2 is 3 we have a positive slope.

The change when x_2 is 3, it is greater than β_1 .

```
sns.relplot(data=study_interaction_wrt_x1_df,
            x='x1',
            y='trend',
            hue='x2',
            kind='line',
            palette='coolwarm',
            estimator=None,
            units='x2')
plt.show()
```



An interaction represents that the relationship with respect to 1 input depends on the other input and the same idea holds for the relationship with respect to x_2 .

```
def calc_trend_wrt_x2_w_interactions(x1,
                                     x2,
                                     b0,
                                     b1,
                                     b2,
                                     b3):
    """
    Calculates the linear trend based on two independent variables x1 and x2, using the
    provided coefficients.

    Args:
        x1 (array-like): Values for the first independent variable.
        x2 (array-like): Values for the second independent variable.
        b0 (float): Intercept coefficient.
        b1 (float): Coefficient for x1.
        b2 (float): Coefficient for x2.
        b3 (float): Coefficient for the interaction term (x1 * x2).

    Returns:
        pandas.DataFrame: DataFrame containing columns for x1, x2, and the calculated
        trend.
    """
    res_df = pd.DataFrame({'x2': x2})
    res_df['x1'] = x1
    res_df['trend'] = b0 + \
        b1 * \
        res_df.x1 + \
        b2 * \
        res_df.x2 + \
        b3 * \
        res_df.x1 * \
        res_df.x2

    return res_df
```

```
x2_values_b = np.linspace(-3, 3, 101)
```

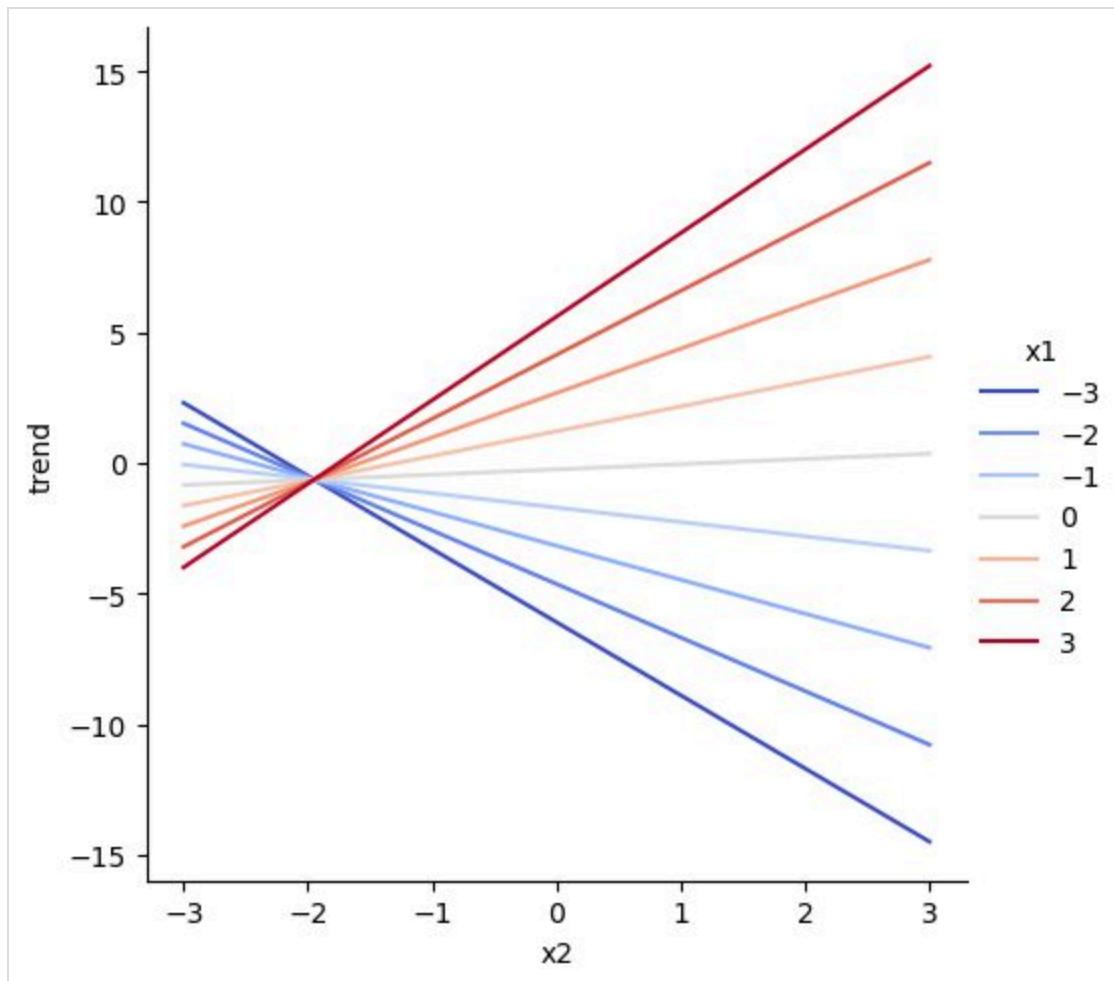
```
x1_values_b = np.linspace(-3, 3, 9)
```

```
study_interaction_wrt_x2_list = [calc_trend_wrt_x2_w_interactions(x1, x2_values_b, b0, b1,
                                                                    b2, b3) for x1 in x1_values_b]
```

```
study_interaction_wrt_x2_df = pd.concat(study_interaction_wrt_x2_list, ignore_index=True)
```

Here we see $\beta_2 < \beta_1$ as the magnitude and sign depend on the value of x_1 .

```
sns.relplot(data=study_interaction_wrt_x2_df,  
            x='x2',  
            y='trend',  
            hue='x1',  
            kind='line',  
            palette='coolwarm',  
            estimator=None,  
            units='x1')  
plt.show()
```



THE RELATIONSHIP OF THE AVERAGE OUTPUT WITH RESPECT TO 1 INPUT DEPENDS ON THE OTHER INPUT AND ROTATES THE LINE BY CHANGING THE SLOPE.

Interactions provide a lot of flexibility and allow the inputs to effect the behavior of other inputs.

13-LR: Penguins Dataset - Linear Regression

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression
from patsy import dmatrices
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
```

Load Dataset

```
df = pd.read_csv('penguins-clean.csv')
```

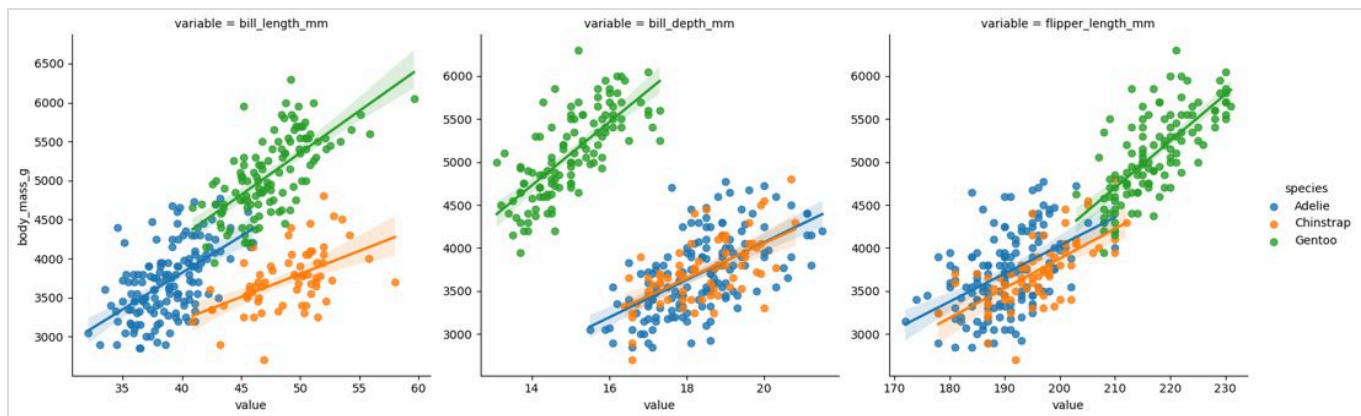
Linear Regression

Visualize Relationship Between the Output and Each Continuous Input, Colored by species

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 3 species form distinct clusters with Gentoo (green) largest, Chinstrap (orange) intermediate, and Adelie (blue) smallest.

There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trends for `flipper_length_mm` and `bill_length_mm` (especially for Gentoo) and a somewhat noisier, more overlapping pattern for `bill_depth_mm` (Adelie and Chinstrap overlap while Gentoo remains separated).


```
sns.lmplot(
    data=df. \
        reset_index(). \
        rename(columns={'index':'rowid'}). \
        melt(id_vars=['rowid',
                      'body_mass_g',
                      'species'],
             value_vars=['bill_length_mm',
                         'bill_depth_mm',
                         'flipper_length_mm']),
    x='value',
    y='body_mass_g',
    col='variable',
    hue='species',
    facet_kws={'sharey': False,
              'sharex': False})
plt.show()
```

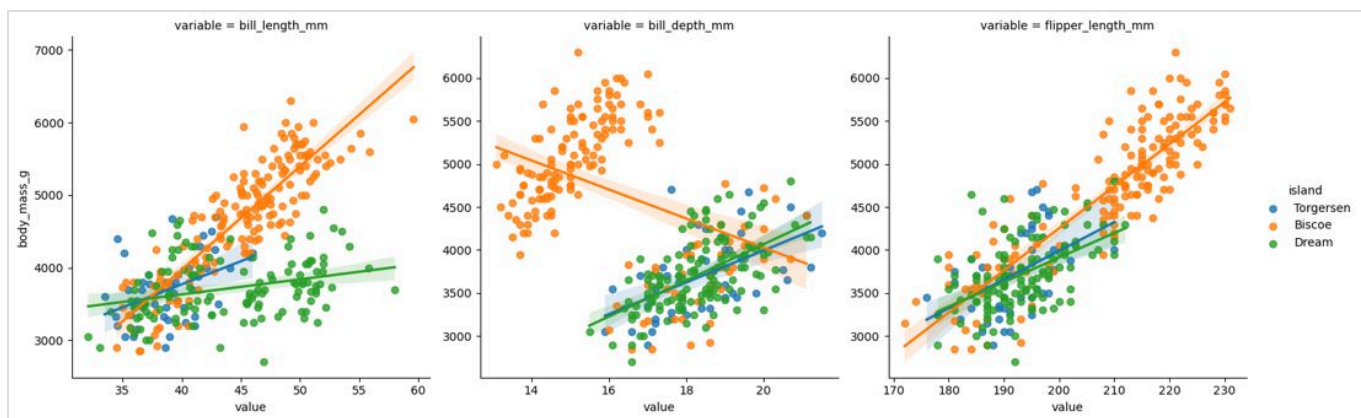


Visualize Relationship Between the Output and Each Continuous Input, Colored by island

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 3 islands show overlapping but distinguishable clusters with Biscoe (orange) largest, Torgersen (blue) intermediate, and Dream (green) smallest.

There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trends for `bill_length_mm` and `flipper_length_mm` (especially on Biscoe) and a weaker, island-specific pattern for `bill_depth_mm`.

```
sns.lmplot(
    data=df. \
        reset_index(). \
        rename(columns={'index':'rowid'}). \
        melt(id_vars=['rowid',
                      'body_mass_g',
                      'island'],
             value_vars=['bill_length_mm',
                         'bill_depth_mm',
                         'flipper_length_mm']),
    x='value',
    y='body_mass_g',
    col='variable',
    hue='island',
    facet_kws={'sharey': False,
               'sharex': False})
plt.show()
```

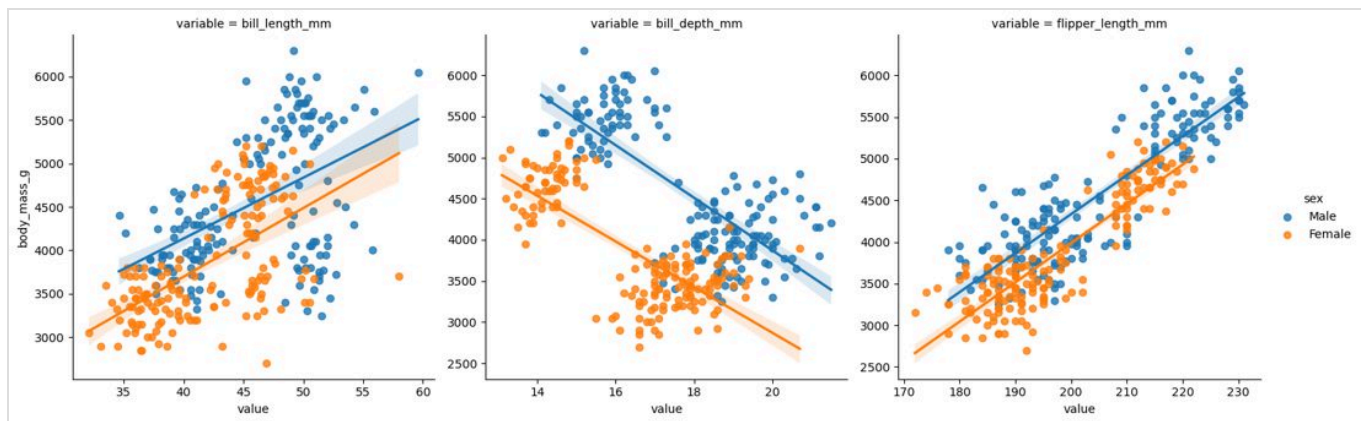


Visualize Relationship Between the Output and Each Continuous Input, Colored by sex

Body mass generally increases with each predictor (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`), and the 2 sexes form overlapping but distinguishable bands with Males (blue) larger and Females (orange) smaller across most of the measurement ranges; the main separation is vertical (mass) rather than horizontal, with notable overlap in the mid-ranges of the predictors.

There are clear relationships between all 3 continuous inputs and the target, with the strongest positive trend for `flipper_length_mm`, a moderate positive trend for `bill_length_mm`, and a contrasting negative within-sex trend for `bill_depth_mm` (body mass declines as `bill_depth_mm` increases), while parallel sex-specific lines indicate a fairly consistent mass offset between Males and Females.

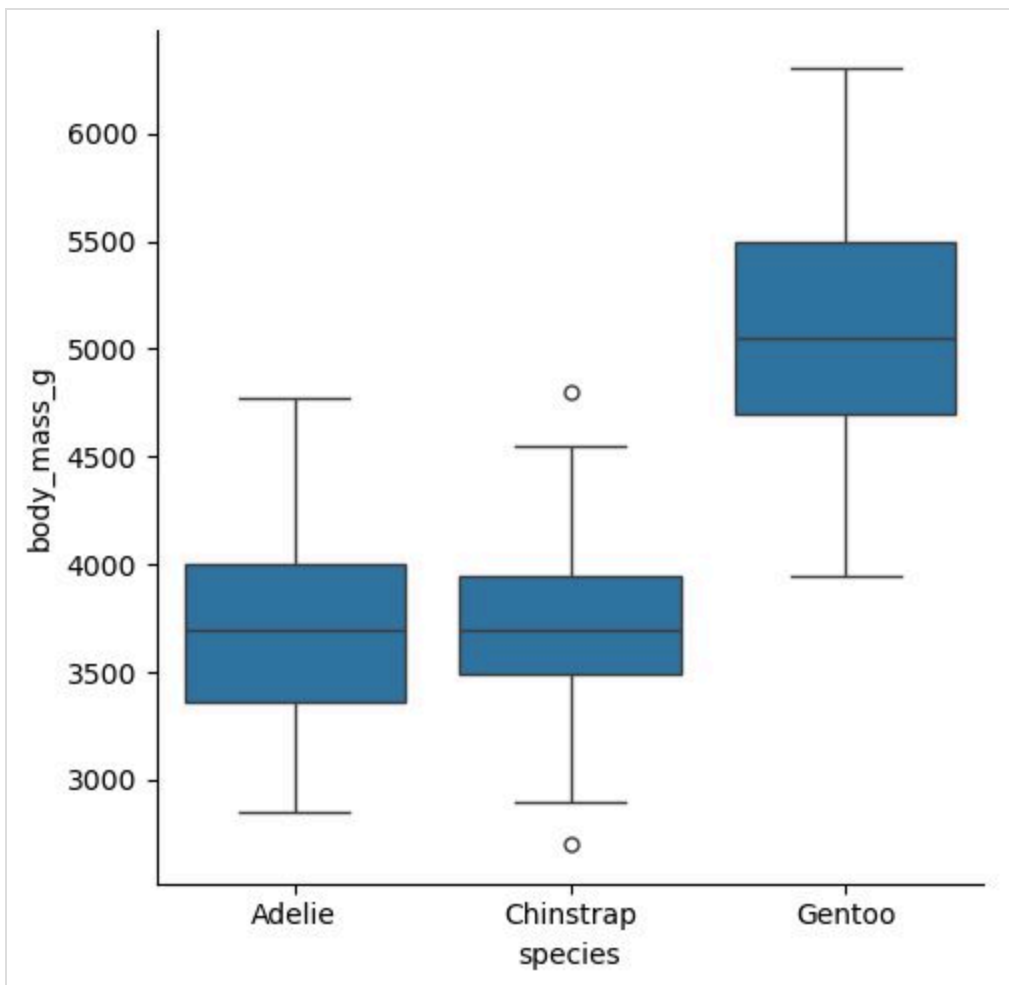
```
sns.lmplot(
    data=df. \
        reset_index(). \
        rename(columns={'index':'rowid'}). \
        melt(id_vars=['rowid',
                      'body_mass_g',
                      'sex'],
             value_vars=['bill_length_mm',
                         'bill_depth_mm',
                         'flipper_length_mm']),
    x='value',
    y='body_mass_g',
    col='variable',
    hue='sex',
    facet_kws={'sharey': False,
               'sharex': False})
plt.show()
```



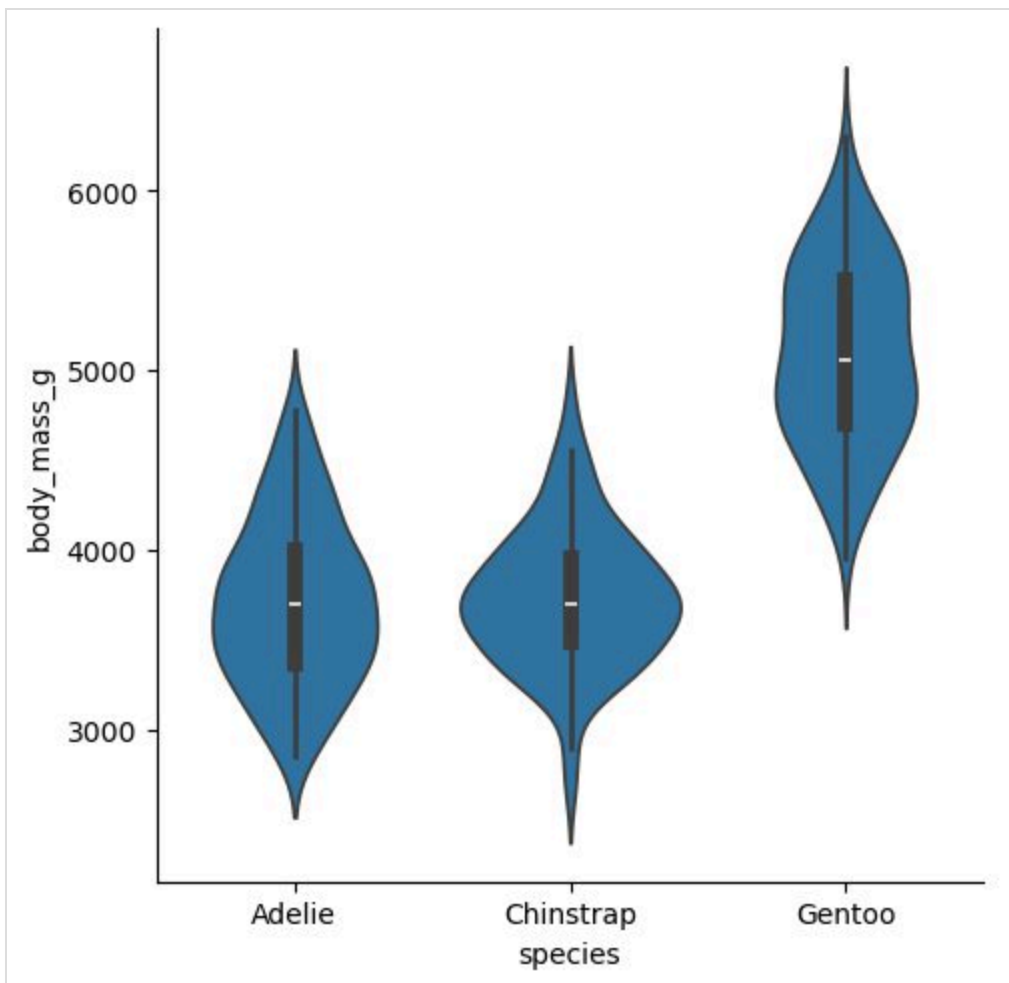
Visualize Relationship Between the Output and species

Across all three visualizations, Gentoo penguins show a clear upward shift in body mass relative to Adelie and Chinstrap as the boxplot emphasizes Gentoo's higher median and broader IQR, the violin plot reveals a bimodal/shouldered density suggesting internal variation (possibly sex or subgroups) while Adelie and Chinstrap have tighter, more centralized distributions with overlapping ranges, and the point plot with error bars (mean $\pm$ likely CI) succinctly confirms the large mean gap (Gentoo far higher, Chinstrap only slightly above Adelie); together they indicate strong inter-species separation driven by uniformly higher and more variable Gentoo masses, with Adelie and Chinstrap largely overlapping in central tendency but Chinstrap extending a bit higher in upper distribution tails.

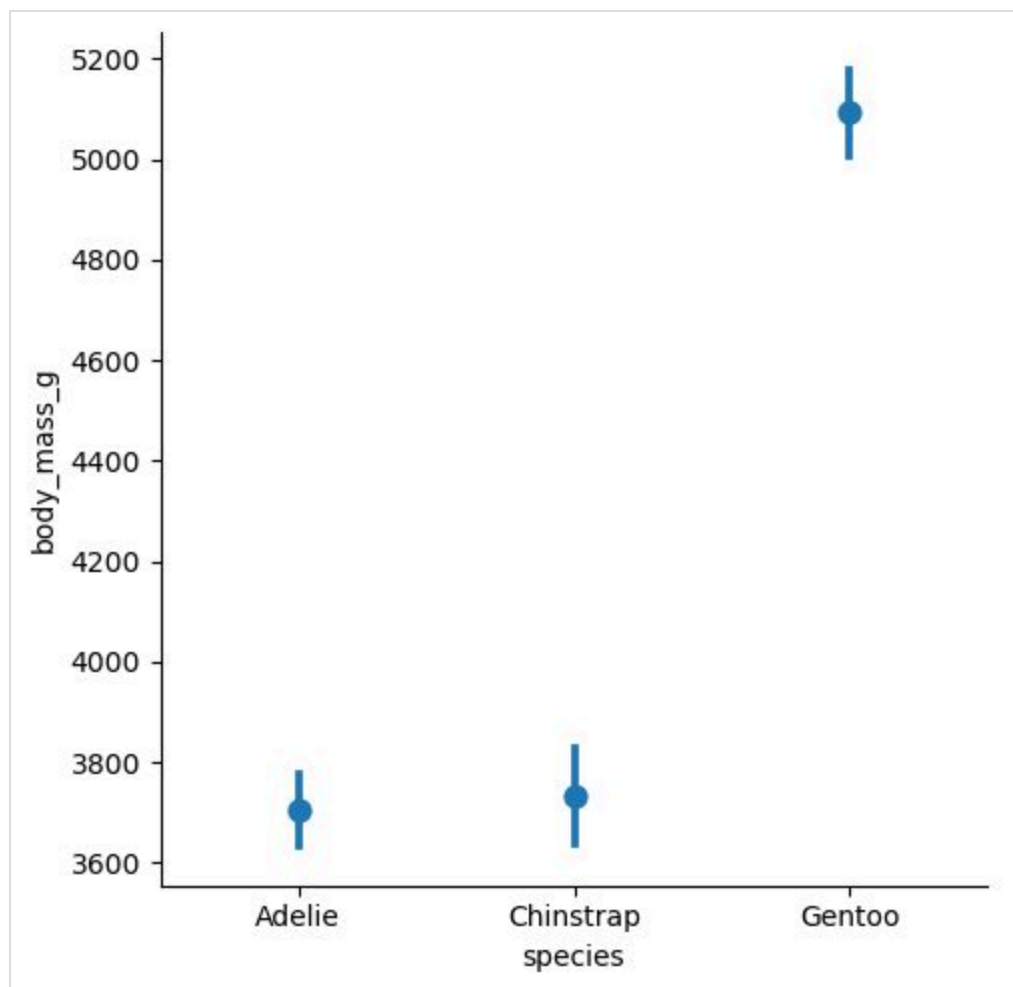
```
sns.catplot(data=df,
            x='species',
            y='body_mass_g',
            kind='box')
plt.show()
```



```
sns.catplot(data=df,  
            x='species',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



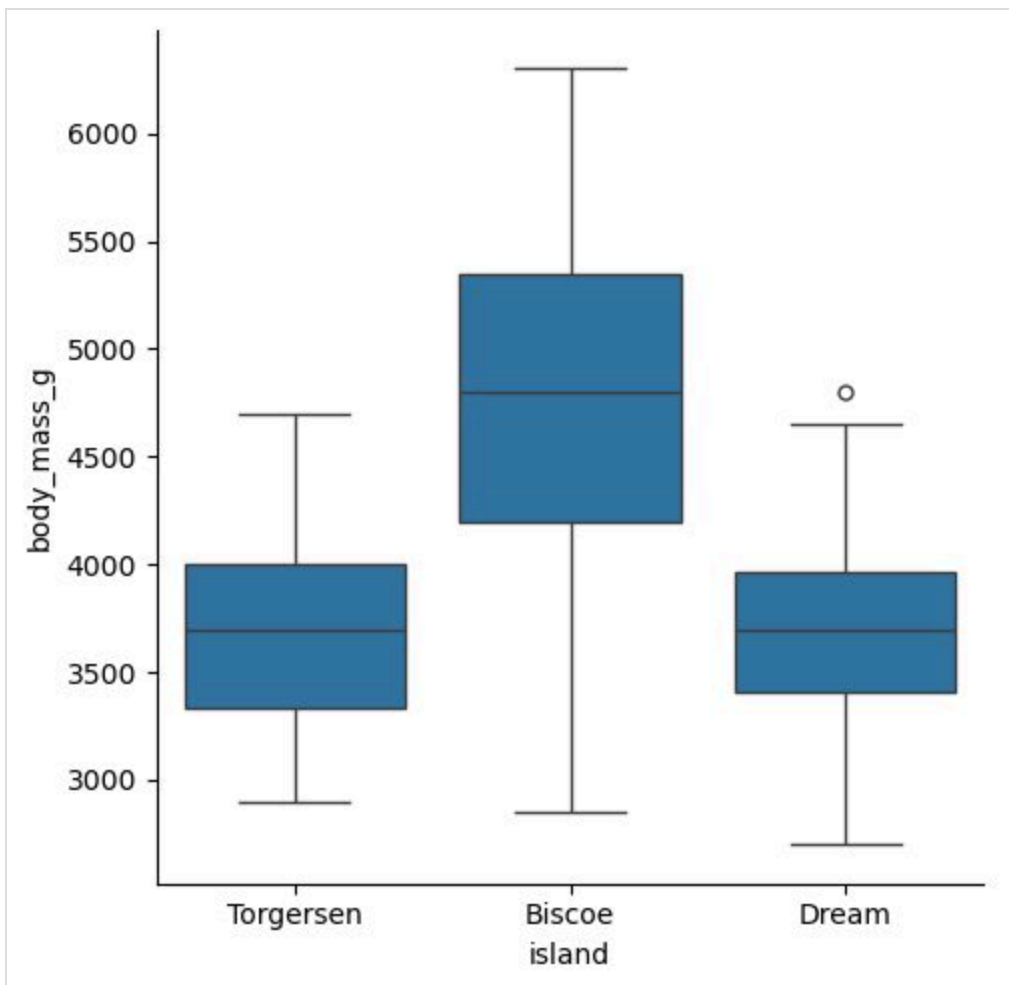
```
sns.catplot(data=df,  
            x='species',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```



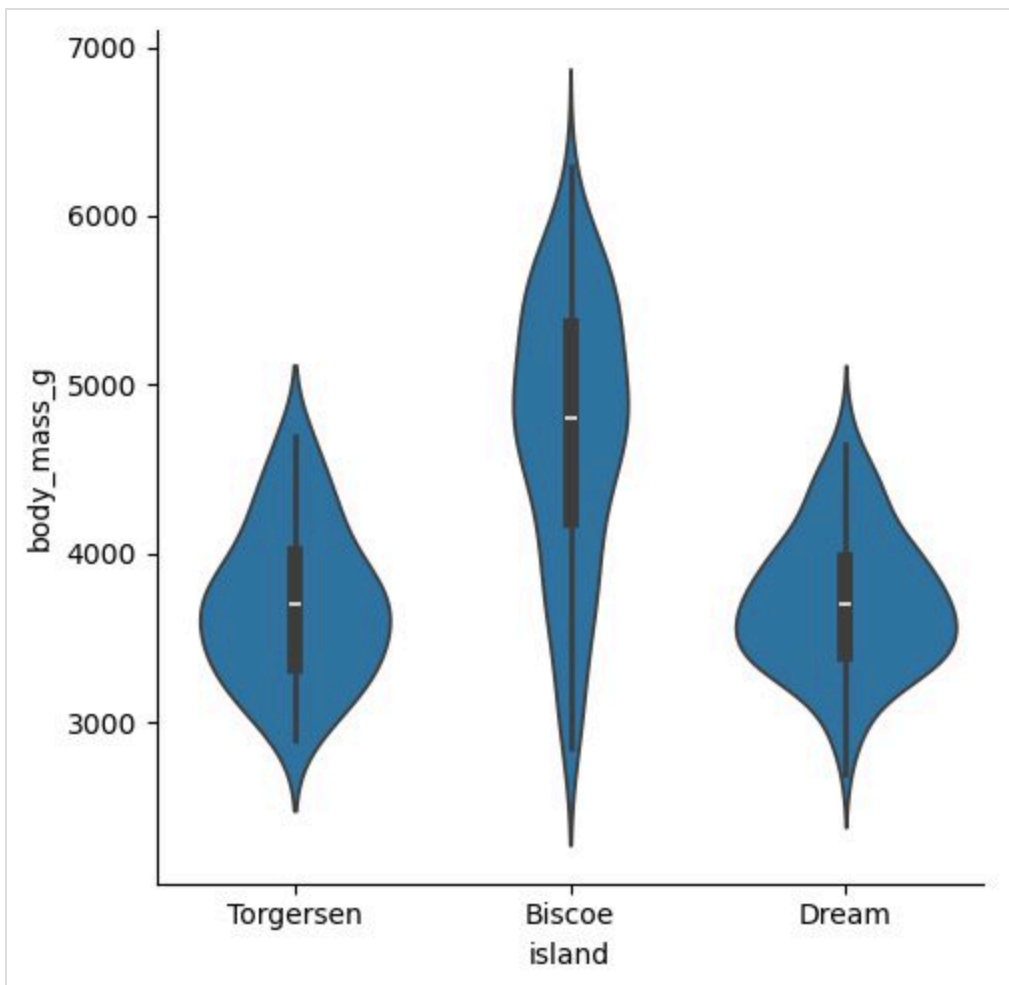
Visualize Relationship Between the Output and island

Across the island visualizations, penguins from Biscoe exhibit a clear upward shift in body mass relative to Torgersen and Dream as the boxplot shows Biscoe's median markedly higher with a much wider IQR and whiskers spanning the broadest range (including the heaviest individuals), while Torgersen and Dream have lower, overlapping medians and tighter boxes; the violin plot reinforces this separation with a tall, multi-shouldered Biscoe density (suggesting internal substructure, likely species or sex mix) contrasted against the more compact, single-peaked shapes for Torgersen and Dream; and the point plot (means with intervals) succinctly confirms the large mean gap—Biscoe well above, Dream only slightly exceeding Torgersen—indicating that inter-island differences in body mass are driven primarily by the consistently higher and more variable masses of penguins sampled on Biscoe, with Torgersen and Dream forming a lower, overlapping baseline.

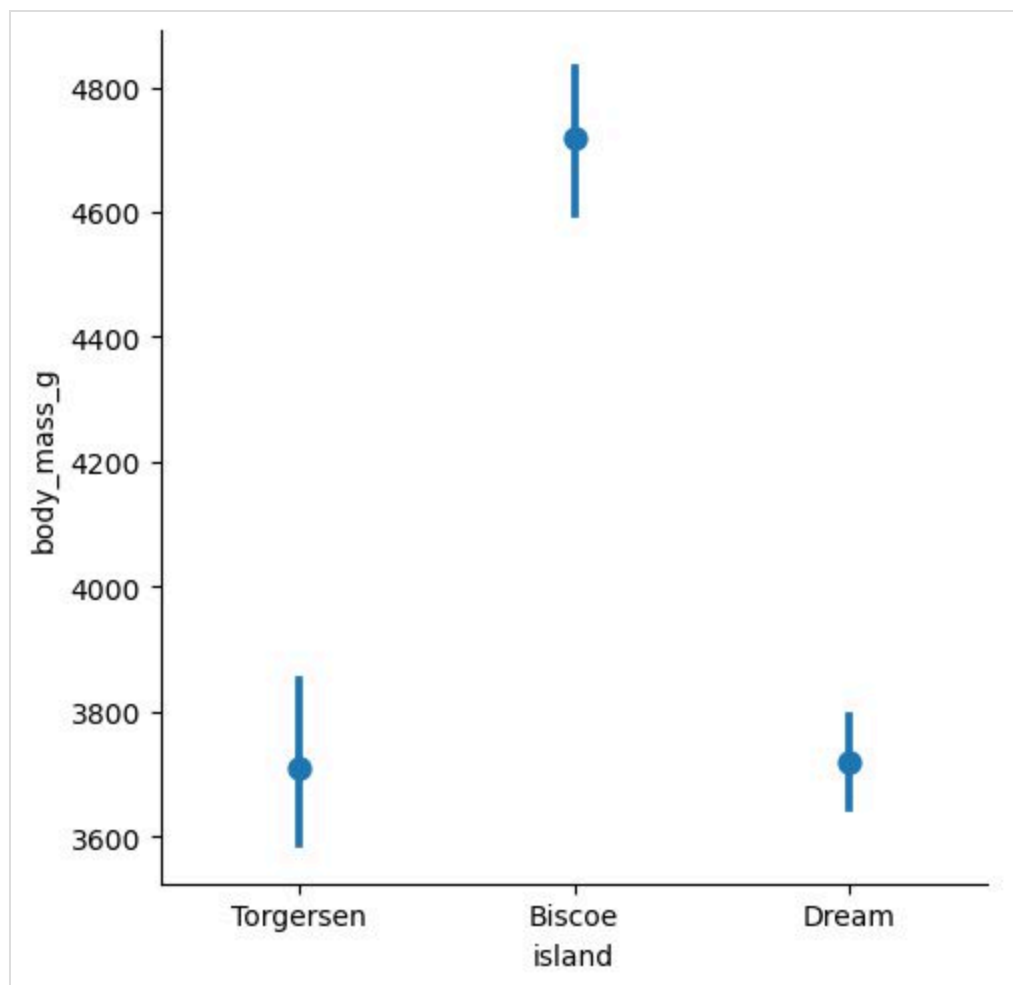
```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='box')  
plt.show()
```



```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



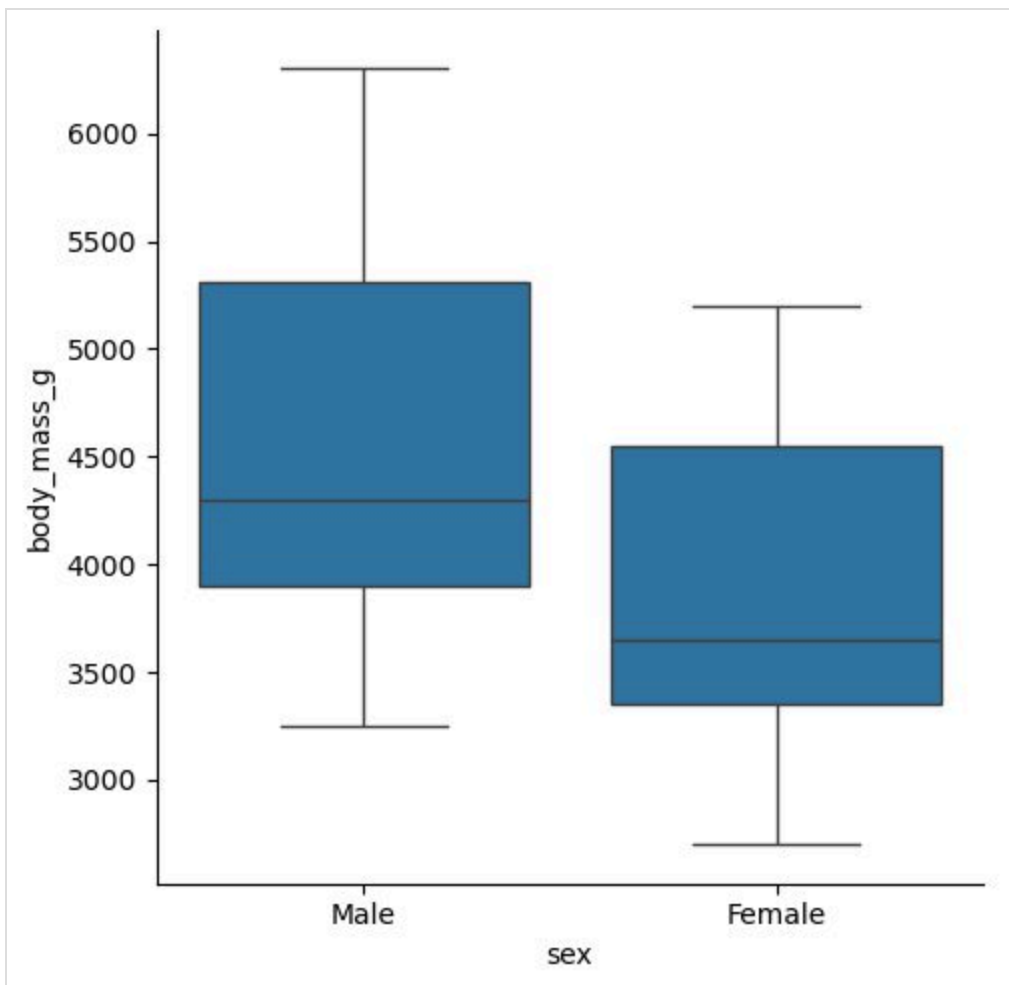
```
sns.catplot(data=df,  
            x='island',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```

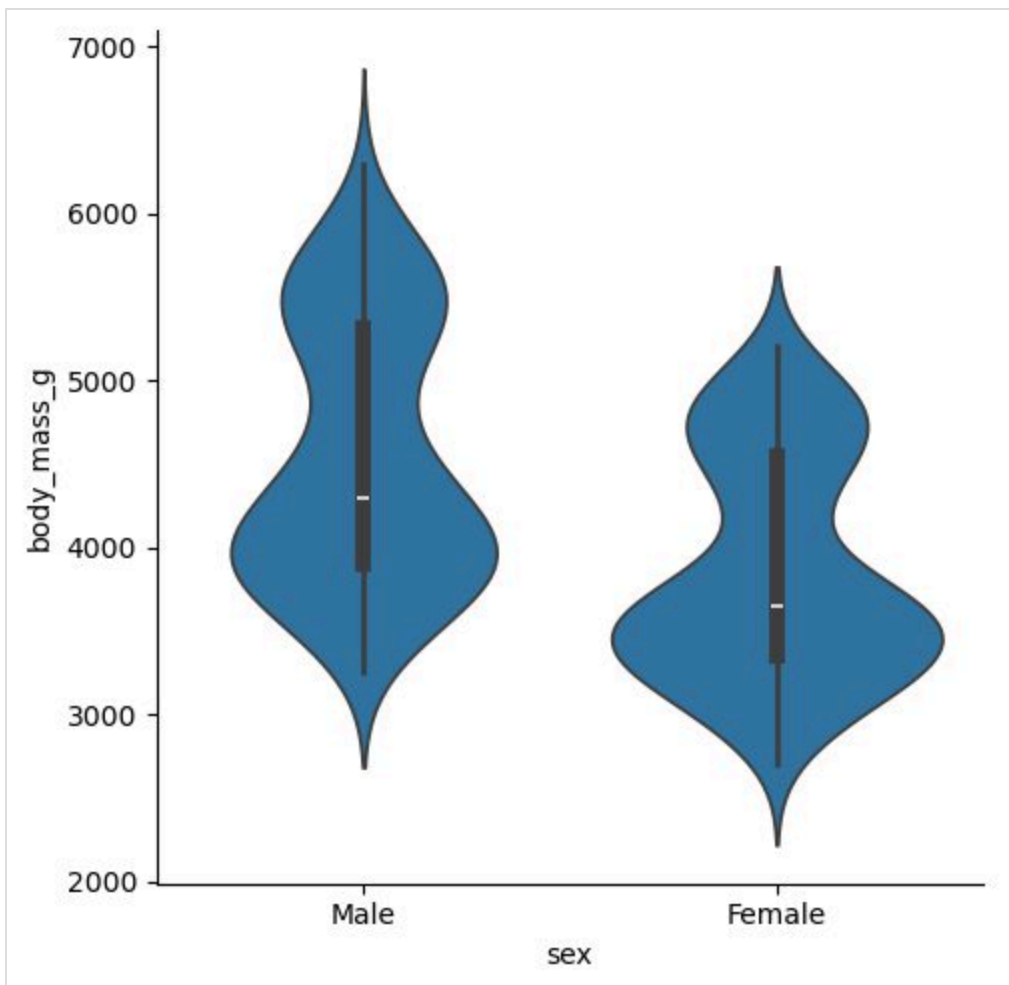
Visualize Relationship Between the Output and sex

Across the sex-focused visualizations, Male penguins show a clear upward shift in body mass versus Females as the boxplot highlights a higher Male median with an IQR spanning substantially larger values and whiskers reaching the heaviest birds, while the Female box sits lower with a narrower upper range; the violin plot reinforces this by displaying a broader, multi-shouldered Male density (suggesting internal subgrouping, e.g., species differences layered atop sex) contrasted with a more compact Female distribution that tapers earlier at the high end; and the point plot (mean with interval) succinctly confirms the gap, with the Male mean well above the Female mean and only modest overlap in confidence intervals—together indicating a consistent vertical (mass) separation where overlap exists mainly in the mid to lower mass range.

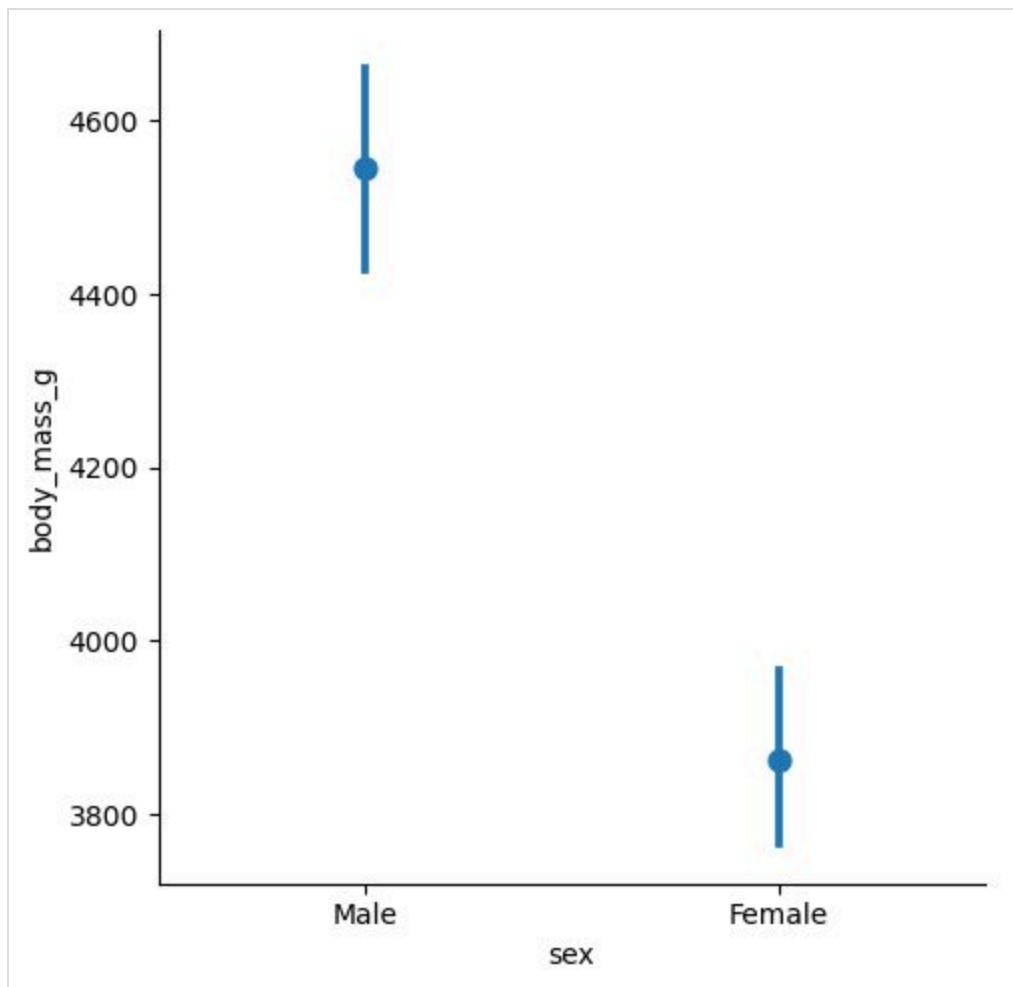
```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='box')  
plt.show()
```



```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='violin')  
plt.show()
```



```
sns.catplot(data=df,  
            x='sex',  
            y='body_mass_g',  
            kind='point',  
            linestyle='none')  
plt.show()
```



Initially Fit Linear Regression Models - Training Data

Functions

```
def fit_and_assess_ols(model,
                      formula,
                      df):
    """
    Fits an Ordinary Least Squares (OLS) regression model using statsmodels and returns
    model metrics.

    Parameters
    -----
    model : str
        A descriptive name for the model.
    formula : str
        A statsmodels formula string specifying the regression model.
    df : pandas.DataFrame
        The input data to fit the model.

    Returns
    -----
    pandas.DataFrame
        A single-row DataFrame containing:
        - 'model_name': Name of the model
        - 'model_formula': Formula used for fitting
        - 'num_coefs': Number of estimated coefficients
        - 'R-squared': Model R-squared statistic
        - 'RMSE': Root Mean Squared Error of residuals
    """
    # Fit the model
    a_model = smf.ols(formula=formula, data=df).fit()

    # Extract model metrics
    results_dict = {'model_name': model,
                    'model_formula': formula,
                    'num_coefs': len(a_model.params),
                    'R-squared': a_model.rsquared,
                    'RMSE': np.sqrt((a_model.resid ** 2).mean())}

    # Return results as DataFrame
    return pd.DataFrame(results_dict, index=[0])
```

```

def my_coefplot(model,
                figsize_use=(10, 8),
                zero_nudge=125):
    """
    Create a coefficient plot of a fitted statsmodels regression model.

    This function visualizes the estimated coefficients of a regression model with their
    confidence intervals (approximated as  $\pm 2$  standard errors).
    Statistically significant coefficients ( $p < 0.05$ ) are shown in red.

    Parameters
    -----
    model : statsmodels.regression.linear_model.RegressionResultsWrapper
        A fitted statsmodels regression model, e.g., the result of smf.ols(...).fit().
    figsize_use : tuple, optional
        Size of the matplotlib figure, default is (10, 4).
    zero_nudge : float, optional
        Amount to nudge significant coefficients away from zero for visual clarity. Default
    is 125.

    Returns
    -----
    None
        Displays a matplotlib plot showing the model coefficients and their error bars.
    """
    # Create the figure and axis
    _, ax = plt.subplots(figsize=figsize_use)

    # Nudge coefficients very close to zero for visual clarity
    coefs = model.params.copy()
    coefs[np.isclose(coefs, 0, atol=zero_nudge)] = zero_nudge

    # Determine colors for coefficients based on significance
    colors = ['red' if p < 0.05 else 'black' for p in model.pvalues]

    # Plot each coefficient with its color
    for _, (y, x, xerr, color) in enumerate(zip(coefs.index, coefs, 2 * model.bse,
colors)):
        ax.errorbar([x],
                    [y],
                    xerr=[xerr],
                    fmt='o',
                    color=color,
                    ecolor=color,
                    elinewidth=2,
                    ms=10)

    # Add a vertical line at x=0
    ax.axvline(x=0,
                linestyle='--',
                linewidth=3.5,
                color='grey')

    # Add labels and title
    ax.set_xlabel('Coefficient Value')

```

```
# Show the plot  
plt.show()
```

Formulas

```
formula_list = [
    # Model 00: Intercept-Only (Baseline)
    'Q("body_mass_g") ~ '
    '1',

    # Model 01: Species (Categorical) Only
    'Q("body_mass_g") ~ '
    'C(Q("species"))',

    # Model 02: Continuous Predictors, Linear Additive
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")',

    # Model 03: Continuous Predictors + Species (Additive)
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + C(Q("species"))',

    # Model 04: Continuous Predictors with Pairwise Interactions and Quadratic Terms
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + Q("bill_length_mm"):Q("bill_depth_mm")'
    ' + Q("bill_length_mm"):Q("flipper_length_mm")'
    ' + Q("bill_depth_mm"):Q("flipper_length_mm")'
    ' + I(Q("bill_length_mm") ** 2)'
    ' + I(Q("bill_depth_mm") ** 2)'
    ' + I(Q("flipper_length_mm") ** 2)',

    # Model 05: Species Interacting with All Continuous Predictors (Allows Species-Specific
    Slopes)
    'Q("body_mass_g") ~ '
    'C(Q("species")) * ('
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    '),'

    # Model 06: Higher-Order Polynomials on Flipper Length (If Non-Linear Scaling
    Suspected)
    'Q("body_mass_g") ~ '
    'Q("bill_length_mm")'
    ' + Q("bill_depth_mm")'
    ' + Q("flipper_length_mm")'
    ' + I(Q("flipper_length_mm") ** 2)'
    ' + I(Q("flipper_length_mm") ** 3)',

    # Model 07: All Predictors, Interactions Among Continuous Terms, and Species (Full
    Candidate)
```



```

'Q("body_mass_g") ~ '
'(Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm"))**2'
' + C(Q("species"))',

# Model 08: Final Model (Species-Specific Slope for Flipper Length)
'Q("body_mass_g") ~ '
'Q("flipper_length_mm")'
' + Q("bill_length_mm")'
' + C(Q("species"))'
' + Q("flipper_length_mm"):C(Q("species"))'
]

```

Model 00: Intercept-Only (Baseline)

```

model_00 = smf.ols(formula=formula_list[0],
                    data=df). \
                    fit()

```

```
model_00.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
''''
```

OLS Regression Results

```

=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.000
Model:              OLS                  Adj. R-squared:           0.000
Method:              Least Squares        F-statistic:              nan
Date:                Wed, 03 Sep 2025     Prob (F-statistic):       nan
Time:                21:28:05             Log-Likelihood:          -2700.1
No. Observations:    333                 AIC:                     5402.
Df Residuals:        332                 BIC:                     5406.
Df Model:            0
Covariance Type:     nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4207.0571	44.126	95.343	0.000	4120.256	4293.858

```

=====
Omnibus:                30.464    Durbin-Watson:                0.972
Prob(Omnibus):           0.000    Jarque-Bera (JB):           19.874
Skew:                    0.470    Prob(JB):                   4.84e-05
Kurtosis:                2.260    Cond. No.                   1.00
=====

```

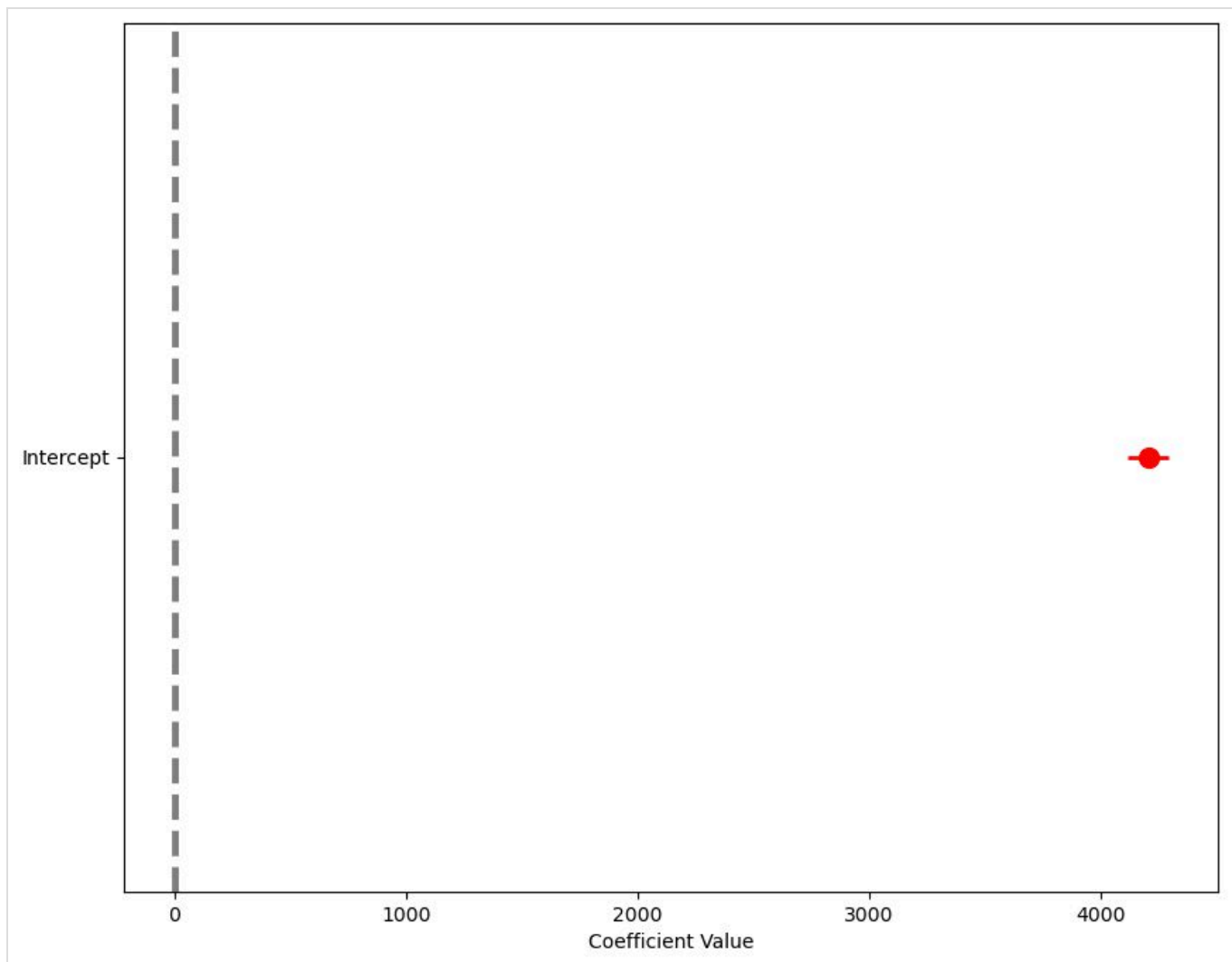
Notes:

```

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
''''

```

```
my_coefplot(model_00)
```



Model 01: Species (Categorical) Only

```
model_01 = smf.ols(formula=formula_list[1],  
                    data=df). \  
                    fit()
```

```
model_01.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
''''
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.674
Model:                  OLS                  Adj. R-squared:           0.673
Method:                 Least Squares        F-statistic:              341.9
Date:                   Wed, 03 Sep 2025     Prob (F-statistic):       3.74e-81
Time:                   21:28:05             Log-Likelihood:           -2513.3
No. Observations:       333                  AIC:                      5033.
Df Residuals:           330                  BIC:                      5044.
Df Model:               2
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025
Intercept	3706.1644	38.136	97.184	0.000	3631.145
C(Q("species"))[T.Chinstrap]	26.9239	67.652	0.398	0.691	-106.161
C(Q("species"))[T.Gentoo]	1386.2726	56.909	24.359	0.000	1274.323

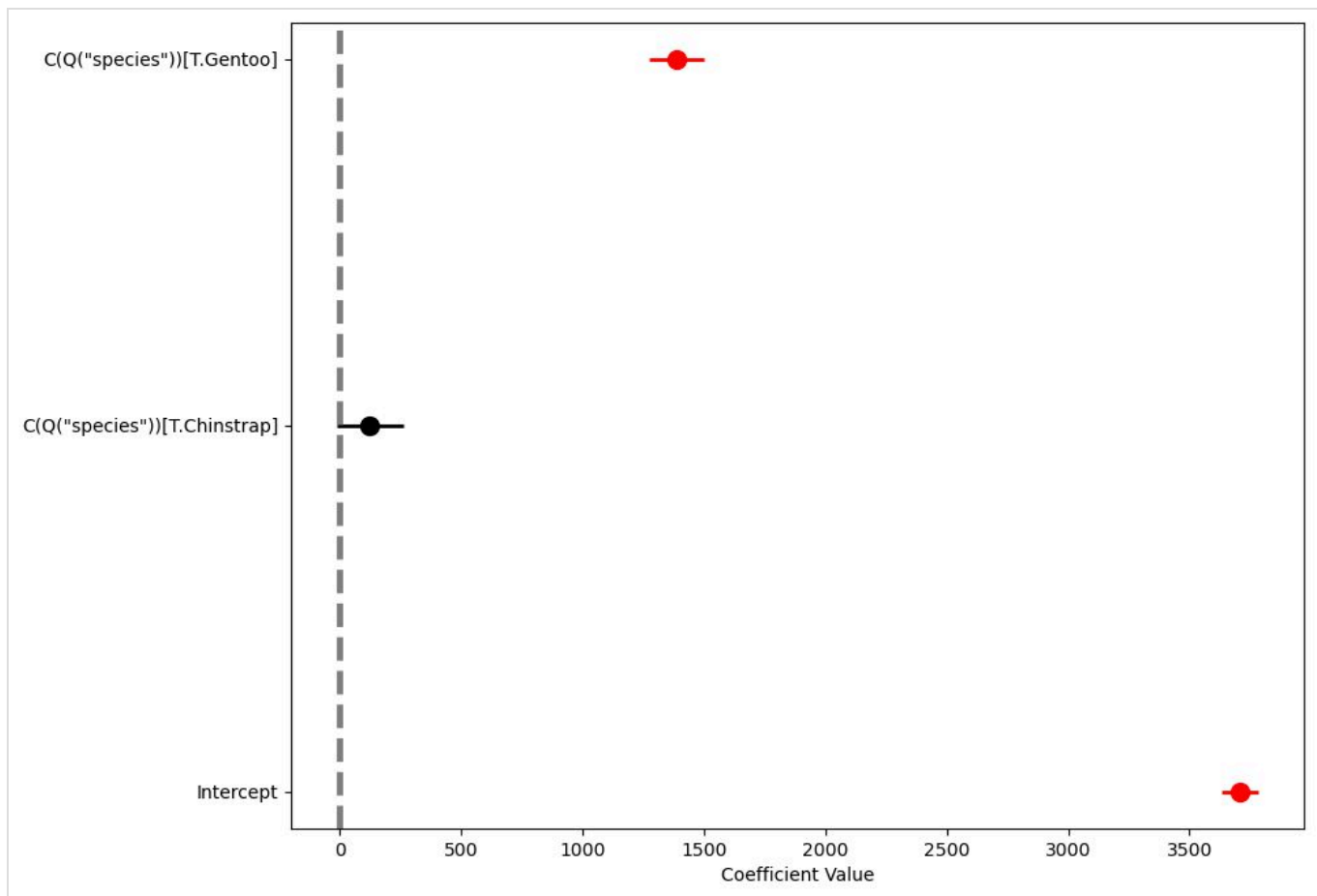
```
=====
Omnibus:                 6.944    Durbin-Watson:              2.983
Prob(Omnibus):            0.031    Jarque-Bera (JB):          4.976
Skew:                     0.169    Prob(JB):                  0.0831
Kurtosis:                 2.506    Cond. No.                  3.45
=====
```

Notes:

```
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
''''
```

```
my_coefplot(model_01)
```



Model 02: Continuous Predictors, Linear Additive

```
model_02 = smf.ols(formula=formula_list[2],  
                    data=df). \  
                    fit()
```

```
model_02.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.764
Model:                  OLS                  Adj. R-squared:           0.762
Method:                 Least Squares        F-statistic:              354.9
Date:                   Wed, 03 Sep 2025     Prob (F-statistic):       9.26e-103
Time:                   21:28:05             Log-Likelihood:           -2459.8
No. Observations:      333                  AIC:                      4928.
Df Residuals:          329                  BIC:                      4943.
Df Model:               3
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-6445.4760	566.130	-11.385	0.000	-7559.167	-5331.785
Q("bill_length_mm")	3.2929	5.366	0.614	0.540	-7.263	13.849
Q("bill_depth_mm")	17.8364	13.826	1.290	0.198	-9.362	45.035
Q("flipper_length_mm")	50.7621	2.497	20.327	0.000	45.850	55.675

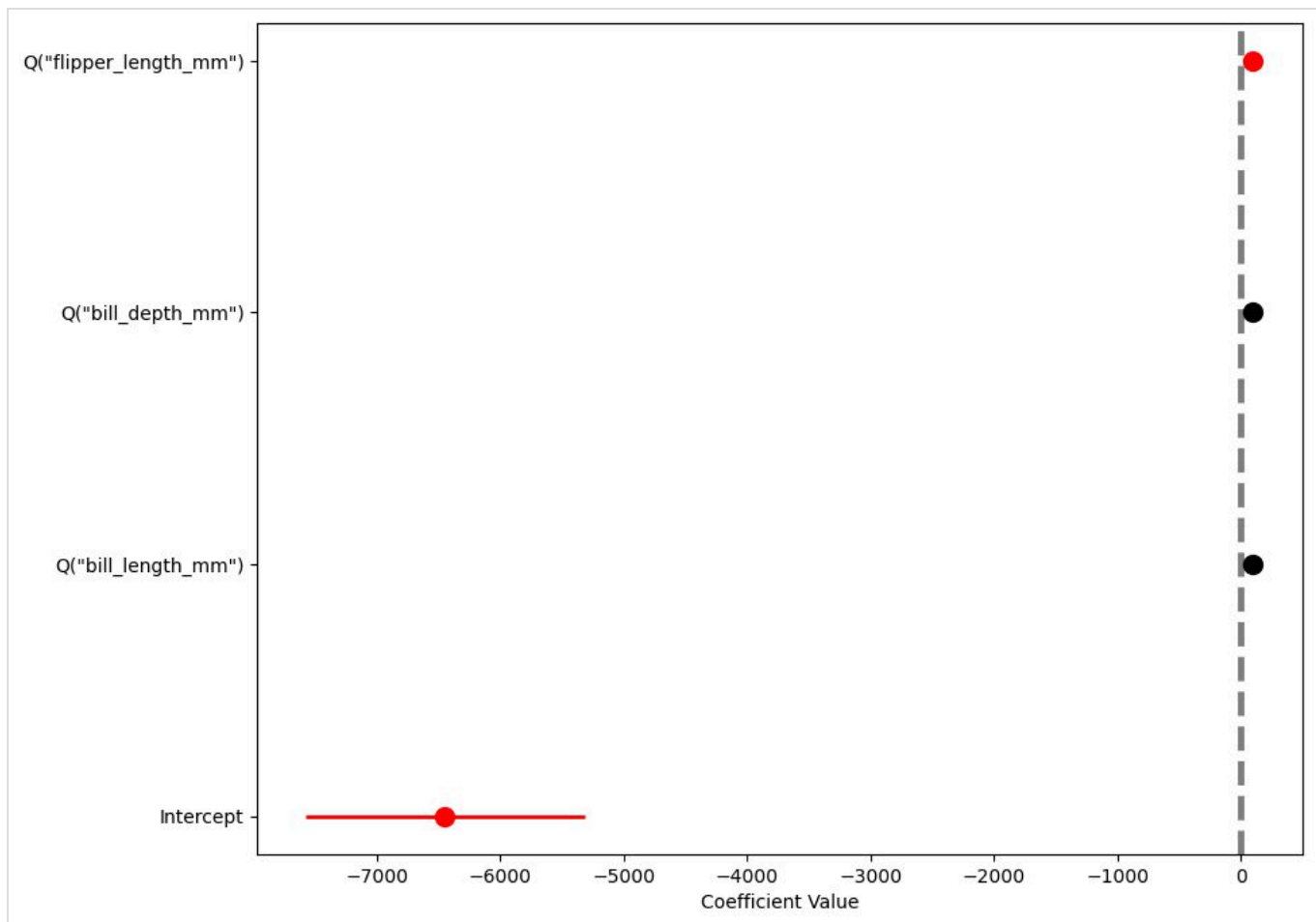
```
=====
Omnibus:                5.596    Durbin-Watson:              1.968
Prob(Omnibus):          0.061    Jarque-Bera (JB):           5.469
Skew:                   0.312    Prob(JB):                   0.0649
Kurtosis:               3.068    Cond. No.                   5.44e+03
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 5.44e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_02, zero_nudge=90)
```



Model 03: Continuous Predictors + Species (Additive)

```
model_03 = smf.ols(formula=formula_list[3],  
                    data=df). \  
                    fit()
```

```
model_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.849
Model:              OLS                  Adj. R-squared:           0.847
Method:             Least Squares        F-statistic:              369.1
Date:               Wed, 03 Sep 2025     Prob (F-statistic):       4.22e-132
Time:               21:28:05             Log-Likelihood:           -2384.8
No. Observations:   333                  AIC:                      4782.
Df Residuals:       327                  BIC:                      4805.
Df Model:           5
Covariance Type:    nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
```

```
-----
Intercept                -4282.0802    497.832     -8.601    0.000   -5261.438
-3302.723
C(Q("species"))[T.Chinstrap] -496.7583    82.469     -6.024    0.000   -658.995
-334.521
C(Q("species"))[T.Gentoo]    965.1983   141.770      6.808    0.000    686.301
1244.096
Q("bill_length_mm")         39.7184     7.227      5.496    0.000     25.501
53.936
Q("bill_depth_mm")         141.7714    19.163      7.398    0.000    104.072
179.470
Q("flipper_length_mm")      20.2264     3.135      6.452    0.000     14.059
26.394
=====
```

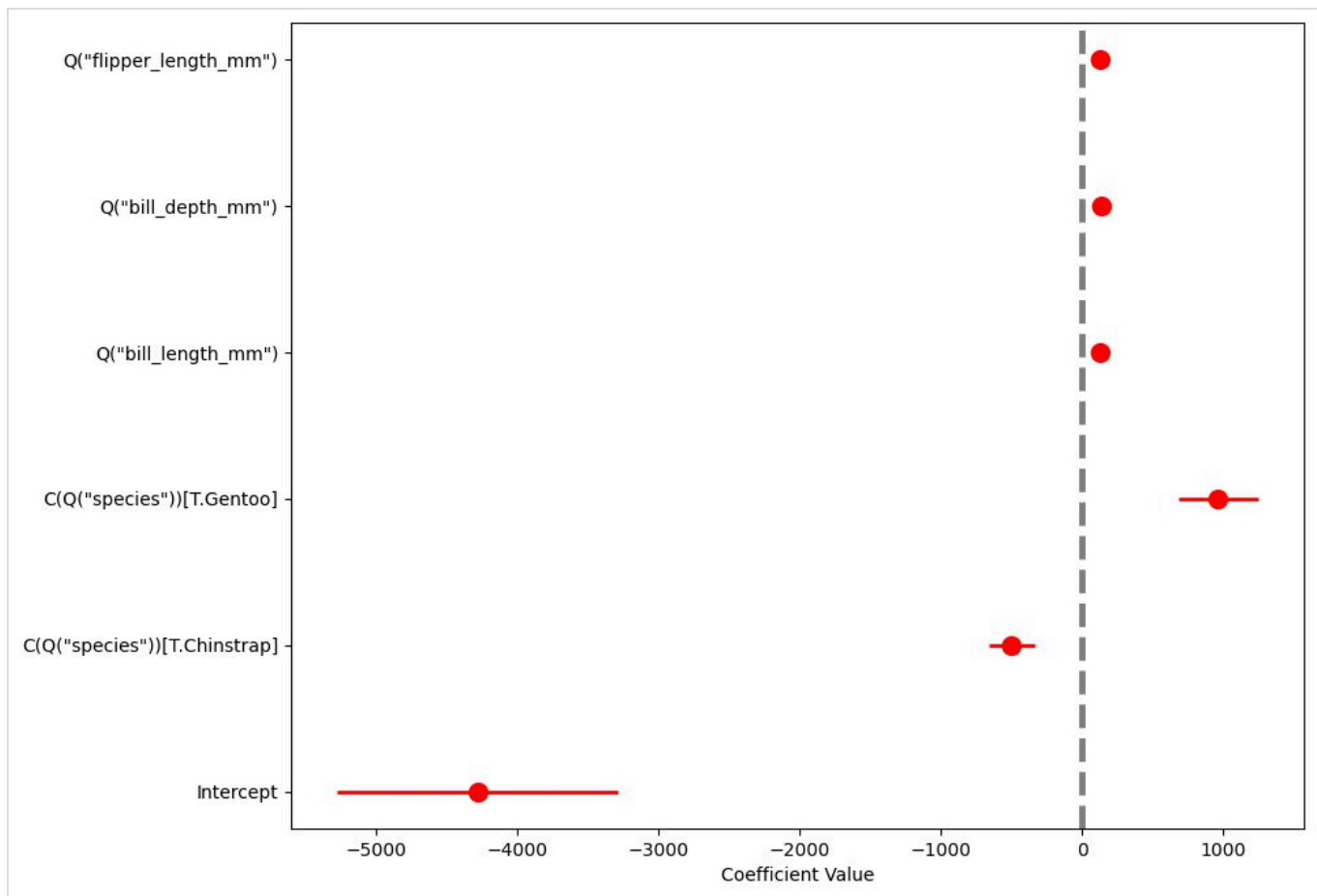
```
Omnibus:                7.321    Durbin-Watson:           2.247
Prob(Omnibus):           0.026    Jarque-Bera (JB):         7.159
Skew:                    0.348    Prob(JB):                 0.0279
Kurtosis:                3.179    Cond. No.                  6.01e+03
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 6.01e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_03)
```



Model 04: Continuous Predictors with Pairwise Interactions and Quadratic Terms

```
model_04 = smf.ols(formula=formula_list[4],  
                    data=df). \  
                    fit()
```

```
model_04.summary()
```



```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.816
Model:                  OLS                  Adj. R-squared:           0.811
Method:                 Least Squares        F-statistic:              159.1
Date:                  Wed, 03 Sep 2025      Prob (F-statistic):       4.75e-113
Time:                  21:28:05              Log-Likelihood:           -2418.4
No. Observations:      333                  AIC:                      4857.
Df Residuals:          323                  BIC:                      4895.
Df Model:               9
Covariance Type:       nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                    -1.001e+04    1.14e+04    -0.878    0.381
-3.25e+04    1.24e+04
Q("bill_length_mm")          203.8636    122.035     1.671    0.096
-36.221     443.948
Q("bill_depth_mm")          1178.9782    539.781     2.184    0.030
117.048     2240.909
Q("flipper_length_mm")       -54.0439     84.854    -0.637    0.525
-220.981     112.893
Q("bill_length_mm"):Q("bill_depth_mm") -10.7817     3.224    -3.344    0.001
-17.125     -4.438
Q("bill_length_mm"):Q("flipper_length_mm") 0.9314     0.564     1.652    0.099
-0.178      2.040
Q("bill_depth_mm"):Q("flipper_length_mm") -4.1625     2.086    -1.996    0.047
-8.266      -0.059
I(Q("bill_length_mm") ** 2)    -2.0777     0.824    -2.522    0.012
-3.699      -0.457
I(Q("bill_depth_mm") ** 2)      4.8728     6.879     0.708    0.479
-8.661      18.406
I(Q("flipper_length_mm") ** 2)  0.3112     0.186     1.673    0.095
-0.055      0.677
=====
```

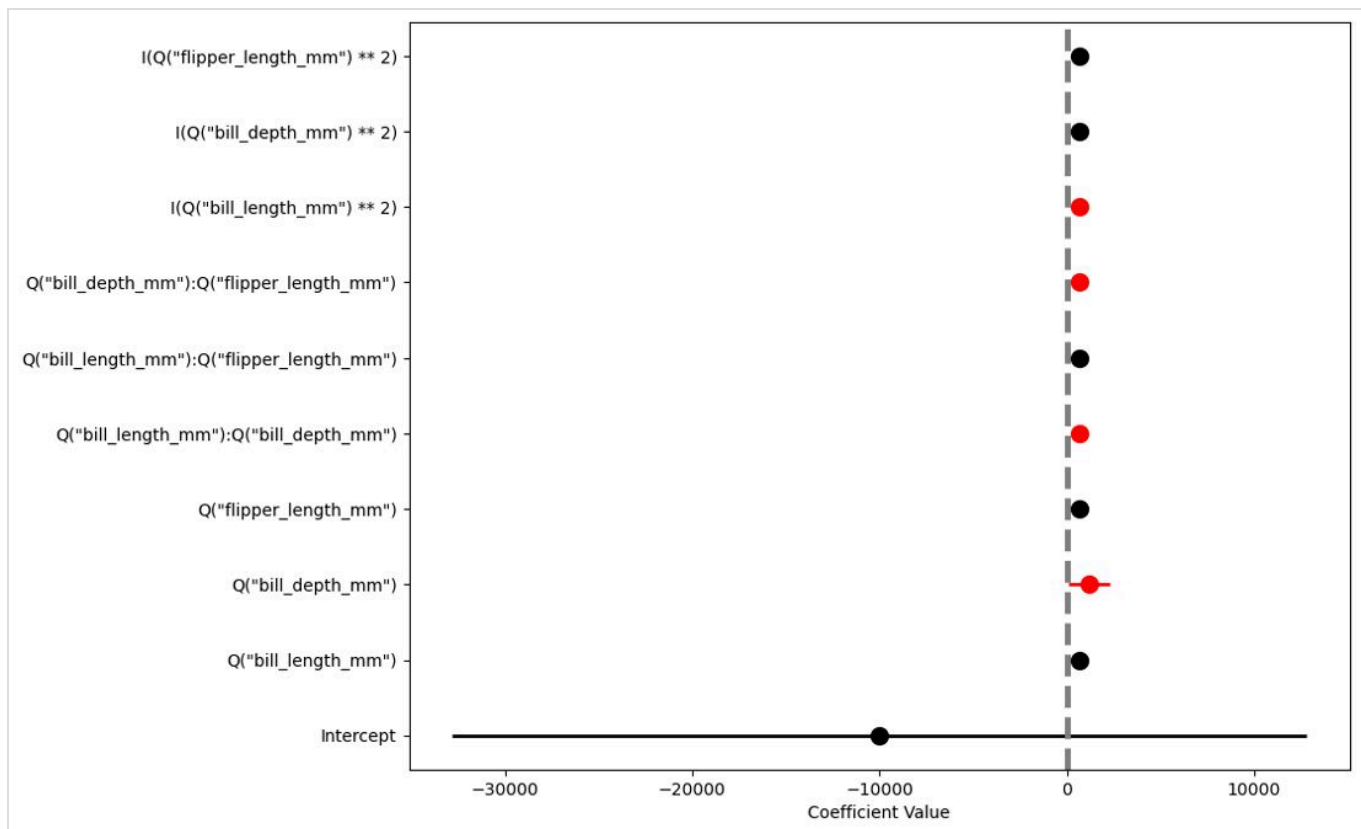
```
=====
Omnibus:                    9.149    Durbin-Watson:              2.059
Prob(Omnibus):              0.010    Jarque-Bera (JB):           9.132
Skew:                      0.395    Prob(JB):                   0.0104
Kurtosis:                   3.181    Cond. No.                   2.51e+07
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.51e+07. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_04, zero_nudge=700)
```



Model 05: Species Interacting with All Continuous Predictors (Allows Species-Specific Slopes)

```
model_05 = smf.ols(formula=formula_list[5],  
                    data=df). \  
                    fit()
```

```
model_05.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.856
Model:              OLS                  Adj. R-squared:           0.851
Method:             Least Squares        F-statistic:              173.4
Date:               Wed, 03 Sep 2025     Prob (F-statistic):       7.27e-128
Time:               21:28:06             Log-Likelihood:           -2377.5
No. Observations:   333                  AIC:                      4779.
Df Residuals:       321                  BIC:                      4825.
Df Model:           11
Covariance Type:    nonrobust
=====
```

```
=====
                                coef    std err          t
P>|t|      [0.025      0.975]
-----
Intercept                                -4270.6550      774.961      -5.511
0.000      -5795.299      -2746.011
C(Q("species"))[T.Chinstrap]             1113.1250      1301.811       0.855
0.393      -1448.034      3674.284
C(Q("species"))[T.Gentoo]                -173.7598      1274.837      -0.136
0.892      -2681.851      2334.332
Q("bill_length_mm")                       54.5124       10.828       5.034
0.000       33.209       75.816
C(Q("species"))[T.Chinstrap]:Q("bill_length_mm") -38.4733       18.657      -2.062
0.040      -75.179      -1.768
C(Q("species"))[T.Gentoo]:Q("bill_length_mm") -16.9957       17.021      -0.999
0.319      -50.482       16.490
Q("bill_depth_mm")                      144.1567       23.465       6.144
0.000       97.993      190.320
C(Q("species"))[T.Chinstrap]:Q("bill_depth_mm") -52.6440       53.766      -0.979
0.328      -158.422       53.134
C(Q("species"))[T.Gentoo]:Q("bill_depth_mm")   35.8488       49.827       0.719
0.472      -62.180      133.877
Q("flipper_length_mm")                   16.9148        4.291       3.942
0.000        8.472      25.357
C(Q("species"))[T.Chinstrap]:Q("flipper_length_mm") 5.6649        7.881       0.719
0.473      -9.840      21.170
C(Q("species"))[T.Gentoo]:Q("flipper_length_mm") 6.3447        7.923       0.801
0.424      -9.242      21.931
=====
```

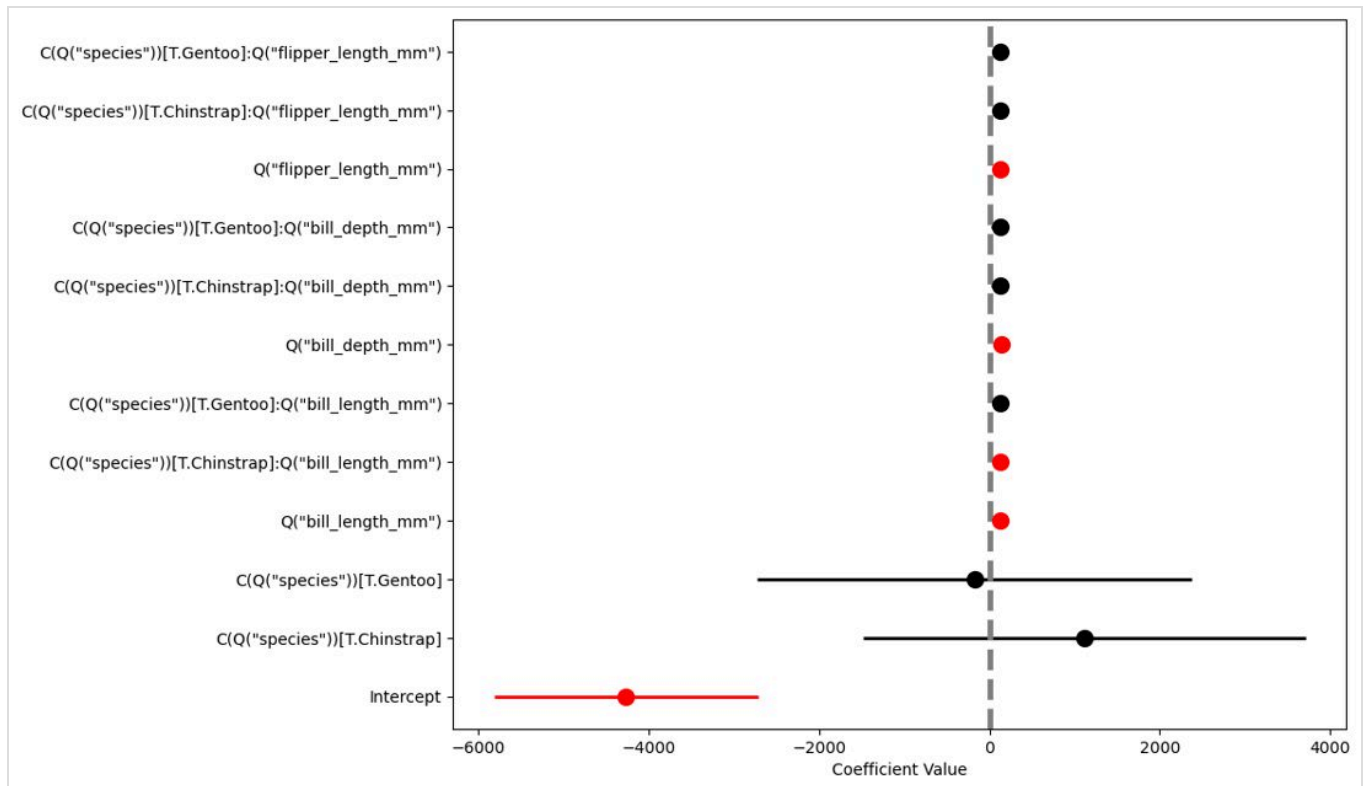
```
Omnibus:                8.160    Durbin-Watson:                2.165
Prob(Omnibus):           0.017    Jarque-Bera (JB):          8.027
Skew:                    0.357    Prob(JB):                  0.0181
Kurtosis:                3.264    Cond. No.                  2.19e+04
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.19e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_05, zero_nudge=120)
```



Model 06: Higher-Order Polynomials on Flipper Length (If Non-Linear Scaling Suspected)

```
model_06 = smf.ols(formula=formula_list[6],
                    data=df). \
                    fit()
```

```
model_06.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:      Q("body_mass_g")    R-squared:                0.793
Model:              OLS                  Adj. R-squared:           0.790
Method:             Least Squares        F-statistic:             250.8
Date:              Wed, 03 Sep 2025      Prob (F-statistic):      1.43e-109
Time:              21:28:06              Log-Likelihood:          -2437.8
No. Observations:  333                  AIC:                     4888.
Df Residuals:      327                  BIC:                     4910.
Df Model:          5
Covariance Type:    nonrobust
=====
```

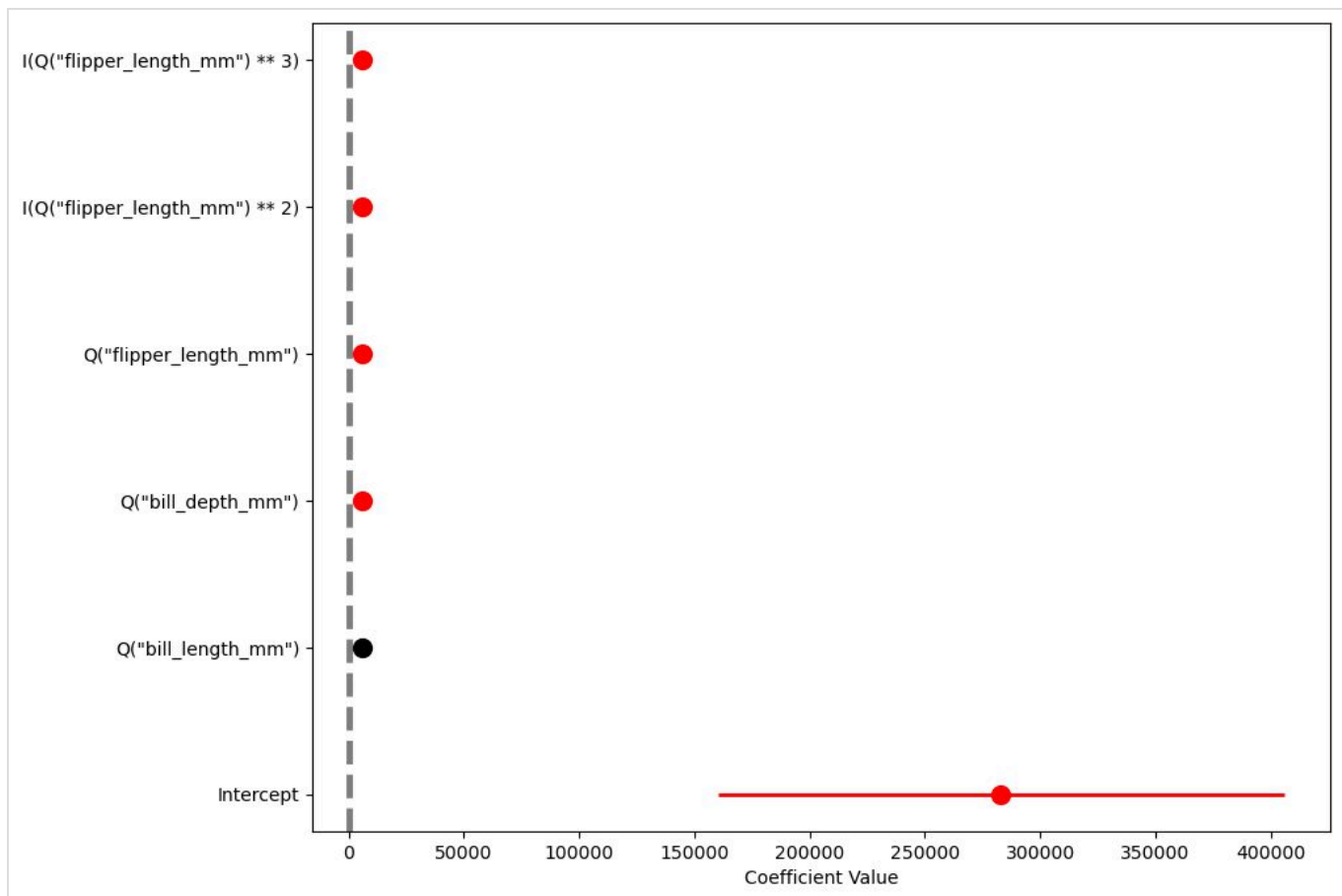
```
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
-----
Intercept                2.831e+05    6.13e+04     4.616    0.000    1.62e+05
4.04e+05
Q("bill_length_mm")         1.4184      5.070     0.280    0.780    -8.555
11.392
Q("bill_depth_mm")         53.4670     14.687     3.640    0.000    24.573
82.361
Q("flipper_length_mm")    -4143.6361    911.346    -4.547    0.000   -5936.478
-2350.795
I(Q("flipper_length_mm") ** 2)  20.1267      4.493     4.479    0.000    11.287
28.966
I(Q("flipper_length_mm") ** 3)  -0.0320      0.007    -4.354    0.000    -0.047
-0.018
=====
Omnibus:                10.214    Durbin-Watson:           2.101
Prob(Omnibus):          0.006    Jarque-Bera (JB):        10.260
Skew:                   0.408    Prob(JB):                0.00592
Kurtosis:               3.272    Cond. No.                2.55e+10
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.55e+10. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_06, zero_nudge=6200)
```

**Model 07: All Predictors, Interactions Among Continuous Terms, and Species (Full Candidate)**

```
model_07 = smf.ols(formula=formula_list[7],  
                    data=df). \  
                    fit()
```

```
model_07.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.852
Model:                  OLS                  Adj. R-squared:           0.849
Method:                 Least Squares        F-statistic:              233.7
Date:                  Wed, 03 Sep 2025      Prob (F-statistic):       1.30e-129
Time:                  21:28:06              Log-Likelihood:           -2381.7
No. Observations:      333                  AIC:                     4781.
Df Residuals:          324                  BIC:                     4816.
Df Model:               8
Covariance Type:       nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                    -6490.4539    5383.685     -1.206     0.229
-1.71e+04    4100.938
C(Q("species"))[T.Chinstrap]    -441.1314     89.228     -4.944     0.000
-616.671    -265.592
C(Q("species"))[T.Gentoo]         908.7480    162.852      5.580     0.000
588.366    1229.130
Q("bill_length_mm")           139.4687     83.530      1.670     0.096
-24.860     303.798
Q("bill_depth_mm")           297.0222    288.078      1.031     0.303
-269.718     863.762
Q("flipper_length_mm")          7.1624     28.747      0.249     0.803
-49.391     63.716
Q("bill_length_mm"):Q("bill_depth_mm")    -6.4401      2.765     -2.329     0.020
-11.880     -1.000
Q("bill_length_mm"):Q("flipper_length_mm")  0.0691      0.294      0.235     0.815
-0.510      0.648
Q("bill_depth_mm"):Q("flipper_length_mm")  0.5889      1.598      0.369     0.713
-2.554      3.732
=====
```

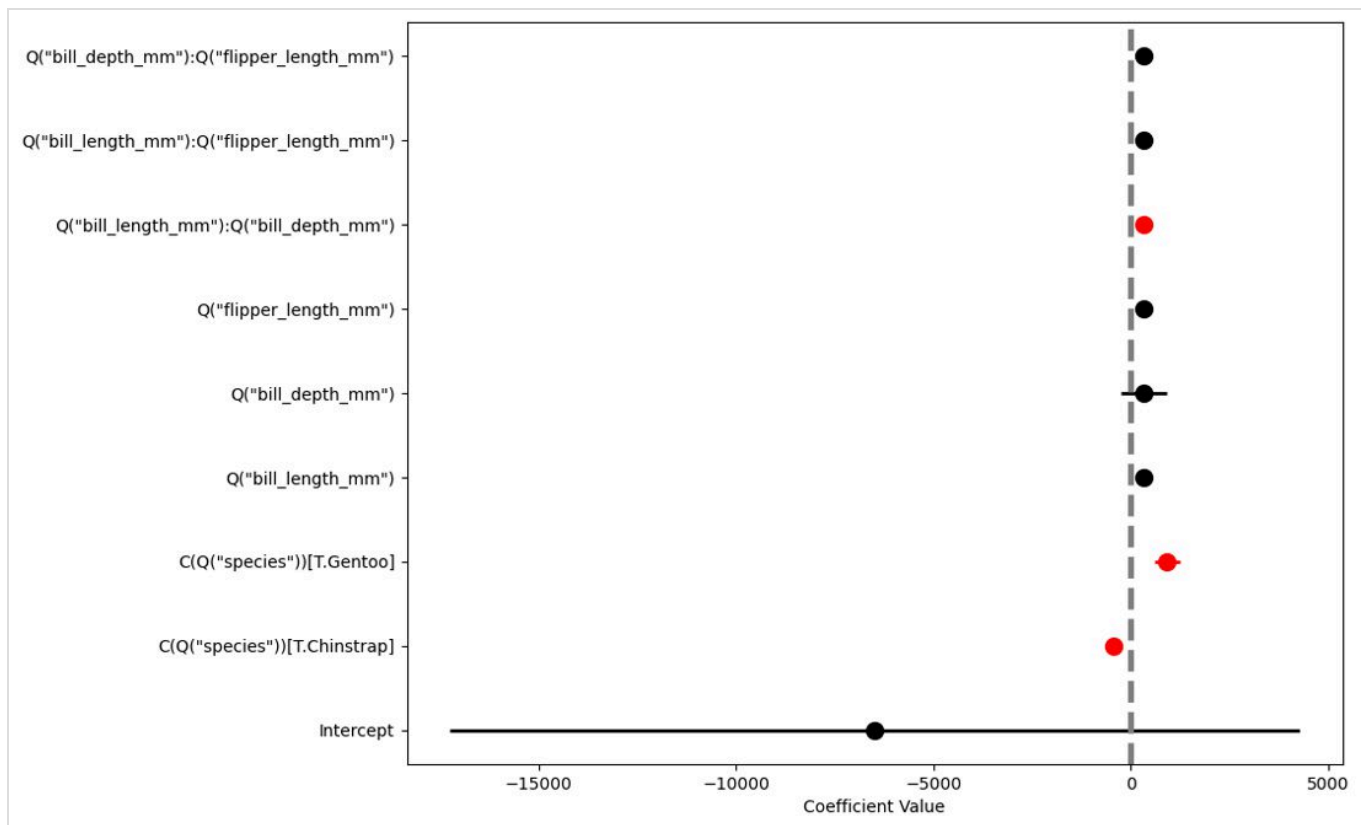
```
=====
Omnibus:                    6.344    Durbin-Watson:           2.195
Prob(Omnibus):              0.042    Jarque-Bera (JB):         6.131
Skew:                      0.321    Prob(JB):                 0.0466
Kurtosis:                   3.173    Cond. No.                 3.04e+06
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.04e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

```
my_coefplot(model_07, zero_nudge=325)
```



Model 08: Final Model (Species-Specific Slope for Flipper Length)

```
model_08 = smf.ols(formula=formula_list[8],
  data=df). \
  fit()
```

```
model_08.summary()
```



```
<class 'statsmodels.iolib.summary.Summary'>
''''
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")      R-squared:                0.827
Model:                  OLS                    Adj. R-squared:           0.823
Method:                 Least Squares          F-statistic:             259.2
Date:                   Wed, 03 Sep 2025        Prob (F-statistic):       7.91e-121
Time:                   21:28:06                Log-Likelihood:          -2408.3
No. Observations:       333                    AIC:                     4831.
Df Residuals:           326                    BIC:                     4857.
Df Model:                6
Covariance Type:        nonrobust
=====
```

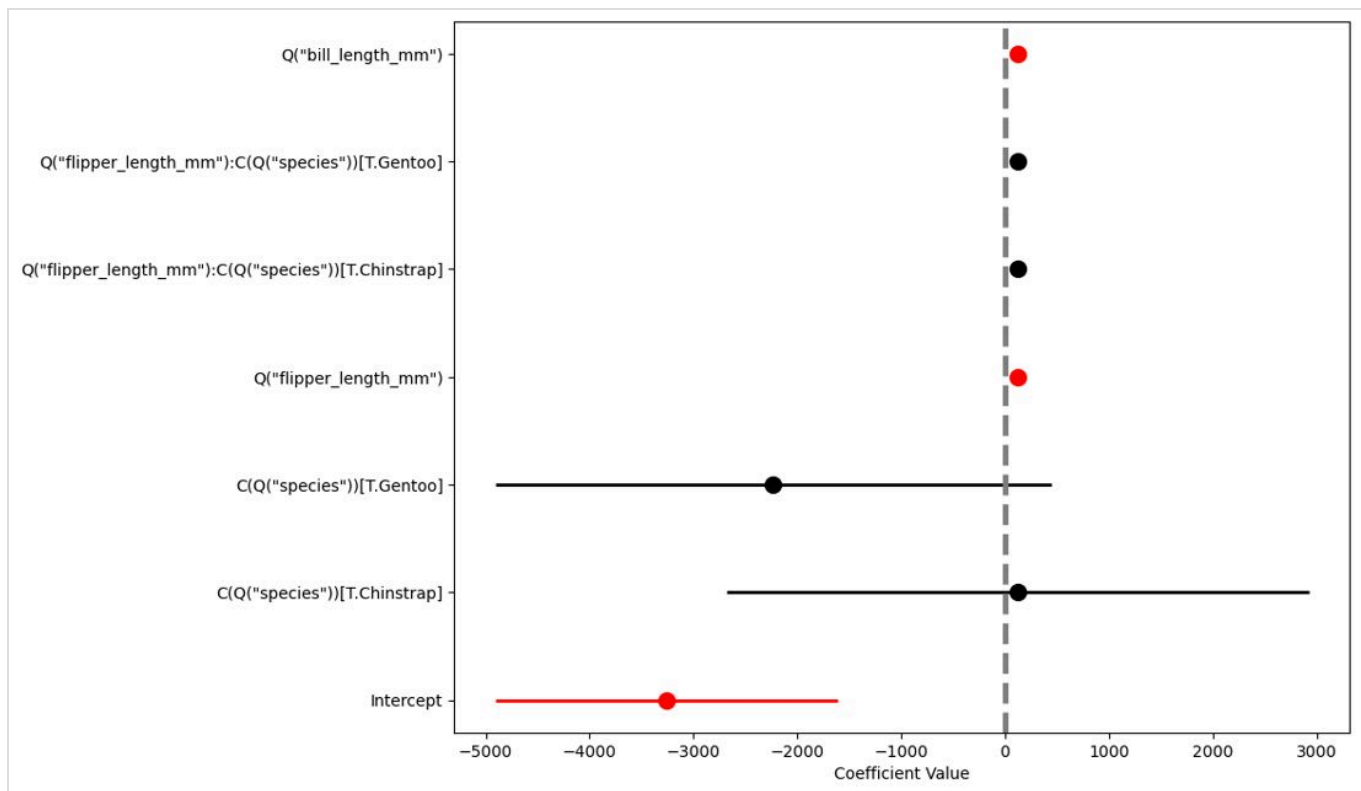
```
=====
                                coef    std err          t
P>|t|      [0.025      0.975]
-----
Intercept                    -3259.5585    824.921     -3.951
0.000   -4882.398   -1636.719
C(Q("species"))[T.Chinstrap]    -100.0775    1401.346     -0.071
0.943   -2856.901    2656.746
C(Q("species"))[T.Gentoo]     -2233.5848    1337.167     -1.670
0.096   -4864.150     396.980
Q("flipper_length_mm")          24.8689      4.421      5.625
0.000      16.172     33.566
Q("flipper_length_mm"):C(Q("species"))[T.Chinstrap]    -3.0247      7.248     -0.417
0.677    -17.282     11.233
Q("flipper_length_mm"):C(Q("species"))[T.Gentoo]       11.2368      6.528      1.721
0.086     -1.605     24.079
Q("bill_length_mm")            57.6468      7.327      7.867
0.000     43.232     72.061
=====
Omnibus:                    6.111    Durbin-Watson:           2.281
Prob(Omnibus):              0.047    Jarque-Bera (JB):         6.069
Skew:                      0.330    Prob(JB):                 0.0481
Kurtosis:                   3.029    Cond. No.                  2.13e+04
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.13e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```
''''
```

```
my_coefplot(model_08)
```



Compare Fitted Models - Training Data

```
ols_results_list = []

for model in range(len(formula_list)):
    ols_results_list.append(fit_and_assess_ols(model,
                                              formula_list[model],
                                              df))
```

```
ols_results_df = pd.concat(ols_results_list,
                           ignore_index=True)
```

```
ols_results_df
```

	model_name	model_formula	num_coefs	\
0	0	$Q(\text{"body_mass_g"}) \sim 1$	1	
1	1	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"}))$	3	
2	2	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"bi...})$	4	
3	3	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"bi...})$	6	
4	4	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"bi...})$	10	
5	5	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"})) * (Q(\text{"bill_...})$	12	
6	6	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"bi...})$	6	
7	7	$Q(\text{"body_mass_g"}) \sim (Q(\text{"bill_length_mm"}) + Q(\text{"b...})$	9	
8	8	$Q(\text{"body_mass_g"}) \sim Q(\text{"flipper_length_mm"}) + Q(\text{...})$	7	

	R-squared	RMSE
0	4.440892e-16	804.005860
1	6.744887e-01	458.714246
2	7.639367e-01	390.637086
3	8.494949e-01	311.913936
4	8.159175e-01	344.957359
5	8.559855e-01	305.114117
6	7.931653e-01	365.654520
7	8.522711e-01	309.023817
8	8.266837e-01	334.717896

Fit Linear Regression Models w/ Cross-Validation - Test Data

Functions

```
def linear_regression_cross_val_score(model, formula, init_model, df, cv):
    """
    Perform cross-validated evaluation of a linear-style model using a Patsy formula.

    This function builds design matrices from `formula` and `df` using patsy.dmatrices,
    then runs cross-validation using sklearn.model_selection.cross_val_score to compute
    per-fold
    R-squared and root-mean-squared-error (RMSE) scores. The per-fold results and some
    metadata
    are returned as a pandas DataFrame.

    Parameters
    -----
    model : str
        A human-readable name for the model; stored in the returned DataFrame.
    formula : str
        A Patsy-compatible formula string (e.g. "y ~ x1 + x2 + C(group)") used to create y
    and X.
    init_model : object
        An unfitted scikit-learn-style estimator (must implement fit/predict and be
    compatible with
        sklearn.model_selection.cross_val_score). Example: LinearRegression().
    df : pandas.DataFrame
        DataFrame containing all variables referenced in `formula`.
    cv : int or cross-validation splitter
        Number of folds (int) or a cross-validation splitter instance passed to
    cross_val_score.

    Returns
    -----
    pandas.DataFrame
        A DataFrame with one row per CV fold containing:
        - 'R-squared' : R^2 score on the test fold (from cross_val_score default scoring)
        - 'RMSE' : Root mean squared error on the test fold
        - 'fold_id' : 1-based fold identifier
        - 'model_name' : the provided `model`
        - 'model_formula' : the provided `formula`
        - 'num_coefs' : number of columns in the design matrix X
    """
    # Create the feature and output arrays using the formula
    y, X = dmatrices( formula, data=df )

    # Train and test within each fold - return the test set scores
    # R-squared
    test_r2 = cross_val_score(init_model,
                              X,
                              y.ravel(),
                              cv=cv)

    # RMSE
    test_rmse = -cross_val_score(init_model,
                                  X,
                                  y.ravel(),
```

```

        cv=cv,
        scoring='neg_root_mean_squared_error')

# Extract model metrics
results_df=pd.DataFrame({'R-squared': test_r2,
                        'RMSE': test_rmse})
results_df['fold_id'] = results_df.index + 1
results_df['model_name'] = model
results_df['model_formula'] = formula
results_df['num_coefs'] = X.shape[1]

# Return the results DataFrame
return results_df

```

Fit w/ Cross-Validation

```

kf = KFold(n_splits=5, shuffle=True, random_state=101)

```

```

lm = LinearRegression(fit_intercept=False)

```

Compare Fitted Models - Test Data

```

cv_score_list = []

for model in range(len(formula_list)):
    print('-- Formula ID %d --' % model)

    cv_score_list.append(linear_regression_cross_val_score(model,
                                                            formula_list[model],
                                                            init_model=lm,
                                                            df=df,
                                                            cv=kf) )

```

```

-- Formula ID 0 --
-- Formula ID 1 --
-- Formula ID 2 --
-- Formula ID 3 --
-- Formula ID 4 --
-- Formula ID 5 --
-- Formula ID 6 --
-- Formula ID 7 --
-- Formula ID 8 --

```

```

cv_score_df=pd.concat(cv_score_list, ignore_index=True)

```

```

cv_score_df

```

	R-squared	RMSE	fold_id	model_name	\
0	-4.526133e-02	812.420097	1	0	
1	-3.363930e-07	735.247849	2	0	
2	-6.889831e-05	792.792792	3	0	
3	-6.097962e-03	839.647092	4	0	
4	-1.400301e-02	845.708003	5	0	
5	6.381451e-01	478.008282	1	1	
6	6.301439e-01	447.146775	2	1	
7	7.303516e-01	411.664747	3	1	
8	6.918962e-01	464.649173	4	1	
9	6.414559e-01	502.888977	5	1	
10	7.606020e-01	388.801957	1	2	
11	6.965466e-01	405.022996	2	2	
12	7.292803e-01	412.481694	3	2	
13	8.160409e-01	359.035398	4	2	
14	7.528083e-01	417.558889	5	2	
15	8.554658e-01	302.101930	1	3	
16	7.868855e-01	339.422137	2	3	
17	8.310358e-01	325.868316	3	3	
18	8.922926e-01	274.725835	4	3	
19	8.375904e-01	338.459441	5	3	
20	8.081037e-01	348.098027	1	4	
21	7.827729e-01	342.681524	2	4	
22	7.949057e-01	359.022429	3	4	
23	7.490267e-01	419.363278	4	4	
24	7.793542e-01	394.501524	5	4	
25	8.614673e-01	295.763394	1	5	
26	8.023321e-01	326.890091	2	5	
27	8.253085e-01	331.345110	3	5	
28	8.846118e-01	284.352655	4	5	
29	8.414184e-01	334.446902	5	5	
30	8.012127e-01	354.292979	1	6	
31	7.207259e-01	388.551851	2	6	
32	7.618194e-01	386.899294	3	6	
33	8.527424e-01	321.229617	4	6	
34	7.679106e-01	404.602406	5	6	
35	8.526903e-01	304.988836	1	7	
36	7.951294e-01	332.792458	2	7	
37	8.344838e-01	322.526193	3	7	
38	8.937218e-01	272.897008	4	7	
39	8.285500e-01	347.751957	5	7	
40	8.393879e-01	318.461806	1	8	
41	7.781769e-01	346.287729	2	8	
42	8.005445e-01	354.052601	3	8	
43	8.481728e-01	326.175606	4	8	
44	8.173335e-01	358.946943	5	8	
			model_formula	num_coefs	
0			Q("body_mass_g") ~ 1	1	
1			Q("body_mass_g") ~ 1	1	
2			Q("body_mass_g") ~ 1	1	
3			Q("body_mass_g") ~ 1	1	
4			Q("body_mass_g") ~ 1	1	
5			Q("body_mass_g") ~ C(Q("species"))	3	
6			Q("body_mass_g") ~ C(Q("species"))	3	
7			Q("body_mass_g") ~ C(Q("species"))	3	

8	Q("body_mass_g") ~ C(Q("species"))	3
9	Q("body_mass_g") ~ C(Q("species"))	3
10	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
11	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
12	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
13	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
14	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	4
15	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
16	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
17	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
18	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
19	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
20	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
21	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
22	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
23	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
24	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	10
25	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
26	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
27	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
28	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
29	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...	12
30	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
31	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
32	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
33	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
34	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...	6
35	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
36	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...	9
...	(3 lines omitted) ...	
40	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
41	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
42	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
43	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7
44	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...	7

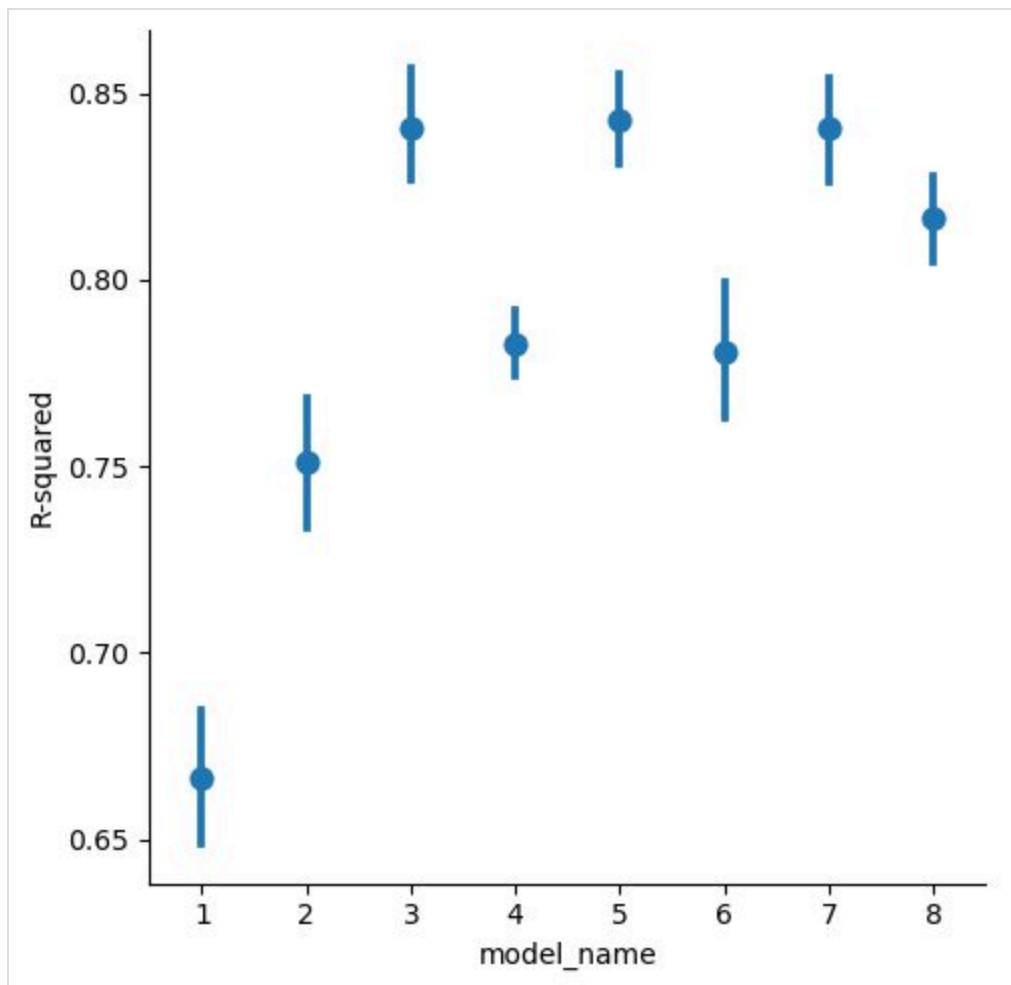
Visualize Average R-squared w/ 1 Standard Error Interval 68% Confidence Interval on the Average

```
cv_score_df.groupby(['model_name',
                    'num_coefs',
                    'model_formula']). \
    aggregate(r2_avg=('R-squared', 'mean'),
              r2_sem=('R-squared', 'sem')). \
    reset_index(). \
    sort_values(by='r2_avg', ascending=False). \
    reset_index(drop=True)
```

	model_name	num_coefs	model_formula \
0	5	12	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"})) * (Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"}))$
1	7	9	$Q(\text{"body_mass_g"}) \sim (Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"}))$
2	3	6	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
3	8	7	$Q(\text{"body_mass_g"}) \sim Q(\text{"flipper_length_mm"}) + Q(\text{"body_mass_g"})$
4	4	10	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
5	6	6	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
6	2	4	$Q(\text{"body_mass_g"}) \sim Q(\text{"bill_length_mm"}) + Q(\text{"body_mass_g"})$
7	1	3	$Q(\text{"body_mass_g"}) \sim C(Q(\text{"species"}))$
8	0	1	$Q(\text{"body_mass_g"}) \sim 1$

	r2_avg	r2_sem
0	0.843028	0.014216
1	0.840915	0.016152
2	0.840654	0.017145
3	0.816723	0.012749
4	0.782833	0.009847
5	0.780882	0.022054
6	0.751056	0.019701
7	0.666399	0.019333
8	-0.013086	0.008443

```
sns.catplot(data=cv_score_df.loc[cv_score_df.model_name > 0, :],
            x='model_name',
            y='R-squared',
            kind='point',
            linestyle='none',
            errorbar=('ci', 68))
plt.show()
```

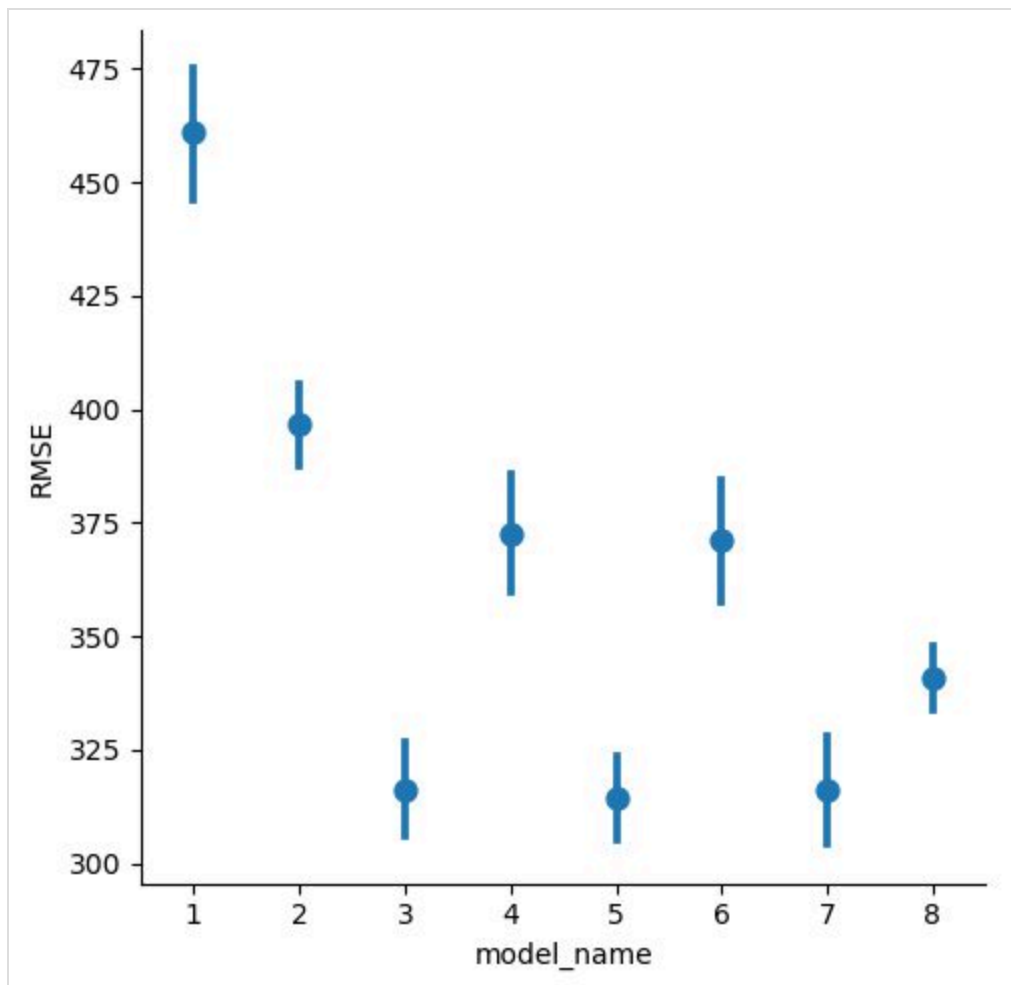

**Visualize Average RMSE w/ 1 Standard Error Interval 68% Confidence Interval on the Average**

```
cv_score_df.groupby(['model_name',  
                    'num_coefs',  
                    'model_formula']). \  
    aggregate(rmse_avg=('RMSE', 'mean'),  
              rmse_sem=('RMSE', 'sem')). \  
    reset_index(). \  
    sort_values(by='rmse_avg', ascending=True). \  
    reset_index(drop=True)
```

	model_name	num_coefs	model_formula \
0	5	12	Q("body_mass_g") ~ C(Q("species")) * (Q("bill_...
1	3	6	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
2	7	9	Q("body_mass_g") ~ (Q("bill_length_mm") + Q("b...
3	8	7	Q("body_mass_g") ~ Q("flipper_length_mm") + Q(...
4	6	6	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
5	4	10	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
6	2	4	Q("body_mass_g") ~ Q("bill_length_mm") + Q("bi...
7	1	3	Q("body_mass_g") ~ C(Q("species"))
8	0	1	Q("body_mass_g") ~ 1

	rmse_avg	rmse_sem
0	314.559630	10.234882
1	316.115532	12.343121
2	316.191290	12.867974
3	340.784937	7.899129
4	371.115229	14.909940
5	372.733356	14.741446
6	396.580187	10.569666
7	460.871591	15.302093
8	805.163167	19.908372

```
sns.catplot(data=cv_score_df.loc[cv_score_df.model_name > 0, :],
            x='model_name',
            y='RMSE',
            kind='point',
            linestyle='none',
            errorbar=('ci', 68))
plt.show()
```



Visualize Model 3 and Statistical Significance

```
model_03.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
```

```
"""
```

OLS Regression Results

```
=====
Dep. Variable:          Q("body_mass_g")    R-squared:                0.849
Model:                  OLS                  Adj. R-squared:           0.847
Method:                 Least Squares        F-statistic:              369.1
Date:                   Wed, 03 Sep 2025     Prob (F-statistic):       4.22e-132
Time:                   21:28:06             Log-Likelihood:           -2384.8
No. Observations:       333                  AIC:                      4782.
Df Residuals:           327                  BIC:                      4805.
Df Model:                5
Covariance Type:        nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
```

```
-----
Intercept                -4282.0802    497.832     -8.601    0.000   -5261.438
-3302.723
C(Q("species"))[T.Chinstrap] -496.7583    82.469     -6.024    0.000   -658.995
-334.521
C(Q("species"))[T.Gentoo]   965.1983   141.770      6.808    0.000    686.301
1244.096
Q("bill_length_mm")        39.7184     7.227      5.496    0.000     25.501
53.936
Q("bill_depth_mm")        141.7714    19.163      7.398    0.000    104.072
179.470
Q("flipper_length_mm")     20.2264     3.135      6.452    0.000     14.059
26.394
=====
```

```
Omnibus:                 7.321    Durbin-Watson:           2.247
Prob(Omnibus):            0.026    Jarque-Bera (JB):        7.159
Skew:                     0.348    Prob(JB):                 0.0279
Kurtosis:                 3.179    Cond. No.                 6.01e+03
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 6.01e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
"""
```

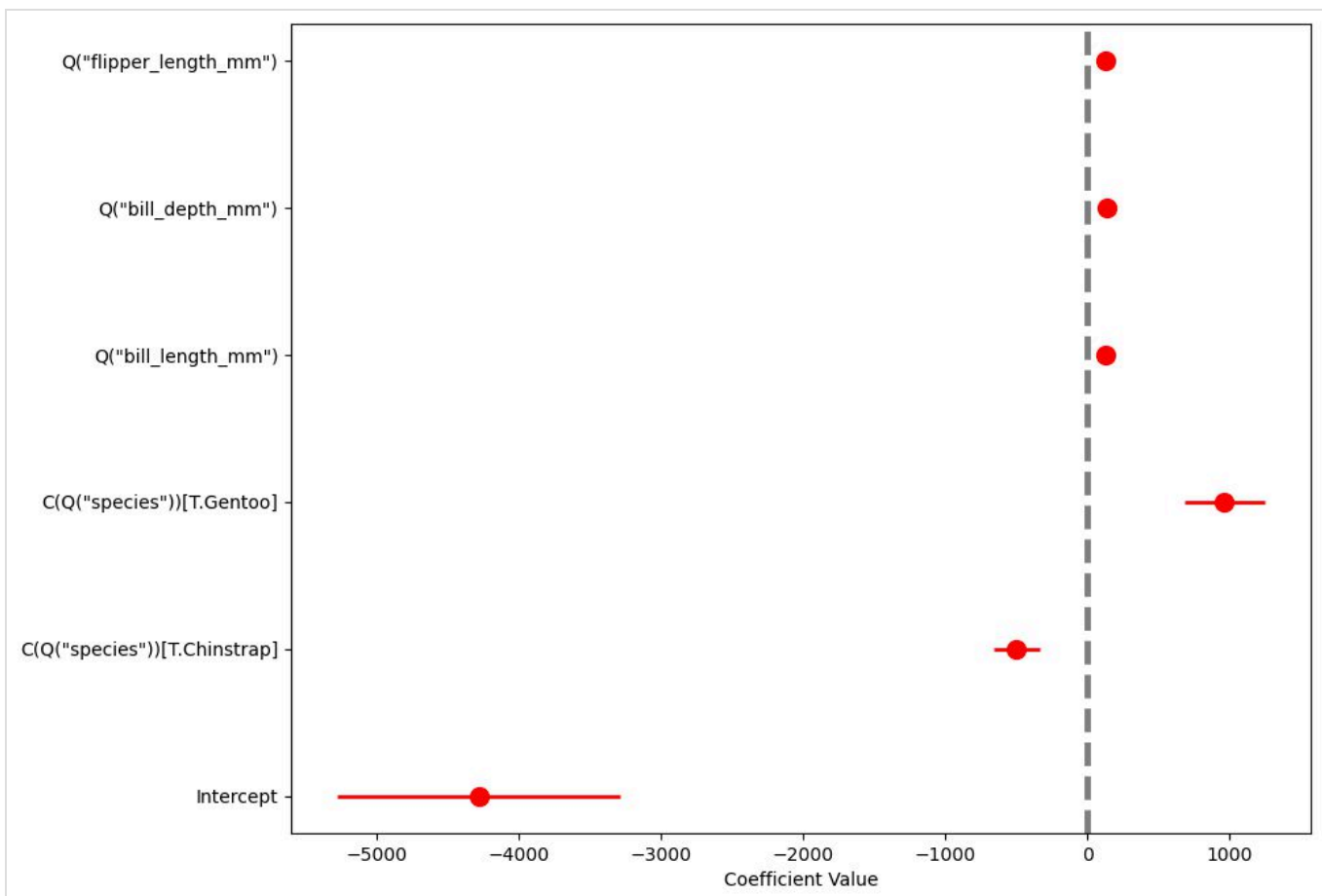
```
model_03.params
```

```
Intercept                -4282.080164
C(Q("species"))[T.Chinstrap] -496.758314
C(Q("species"))[T.Gentoo]   965.198291
Q("bill_length_mm")        39.718385
Q("bill_depth_mm")        141.771362
Q("flipper_length_mm")     20.226447
dtype: float64
```

```
model_03.pvalues
```

```
Intercept                3.332545e-16
C(Q("species"))[T.Chinstrap] 4.590481e-09
C(Q("species"))[T.Gentoo]    4.740533e-11
Q("bill_length_mm")        7.845842e-08
Q("bill_depth_mm")         1.174106e-12
Q("flipper_length_mm")     3.982016e-10
dtype: float64
```

```
my_coefplot(model_03)
```



Visualize Model 3 Predicted Response vs `bill_depth_mm` with 95% Mean CI and 95% Prediction Interval

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +
C(Q("species"))'
```

```
pd.DataFrame({'coefficient': model_03.params[model_03.pvalues < 0.05].abs(),
             'sign': model_03.params[model_03.pvalues < 0.05]. \
                 apply(lambda x: 'positive' if x > 0 else 'negative')}). \
reset_index(). \
groupby('index'). \
aggregate({'coefficient': 'first',
           'sign': 'first'}). \
sort_values(by='coefficient', ascending=False)
```

index	coefficient	sign
Intercept	4282.080164	negative
C(Q("species"))[T.Gentoo]	965.198291	positive
C(Q("species"))[T.Chinstrap]	496.758314	negative
Q("bill_depth_mm")	141.771362	positive
Q("bill_length_mm")	39.718385	positive
Q("flipper_length_mm")	20.226447	positive

```
# Sequence for bill_depth_mm (primary x1)
df_viz_3 = pd.DataFrame({'bill_depth_mm': np.linspace(df.bill_depth_mm.min() - 0.02,
                                                    df.bill_depth_mm.max() + 0.02,
                                                    num=151)})

# Fix bill_length_mm at midpoint between its own min and mean (lower half)
df_viz_3['bill_length_mm'] = 0.5 * (df.bill_length_mm.min() + df.bill_length_mm.mean())

# Hold flipper_length_mm at mean
df_viz_3['flipper_length_mm'] = df.flipper_length_mm.mean()

# Fix species at a chosen level (baseline)
df_viz_3['species'] = 'Gentoo'
```

```
# Get predictions (must pass exactly the columns expected by the model)
pred_summary_03 = model_03.get_prediction(
    df_viz_3[['bill_depth_mm',
              'bill_length_mm',
              'flipper_length_mm',
              'species']]).summary_frame()
```

```
# Create the figure and axis
fig, ax = plt.subplots()

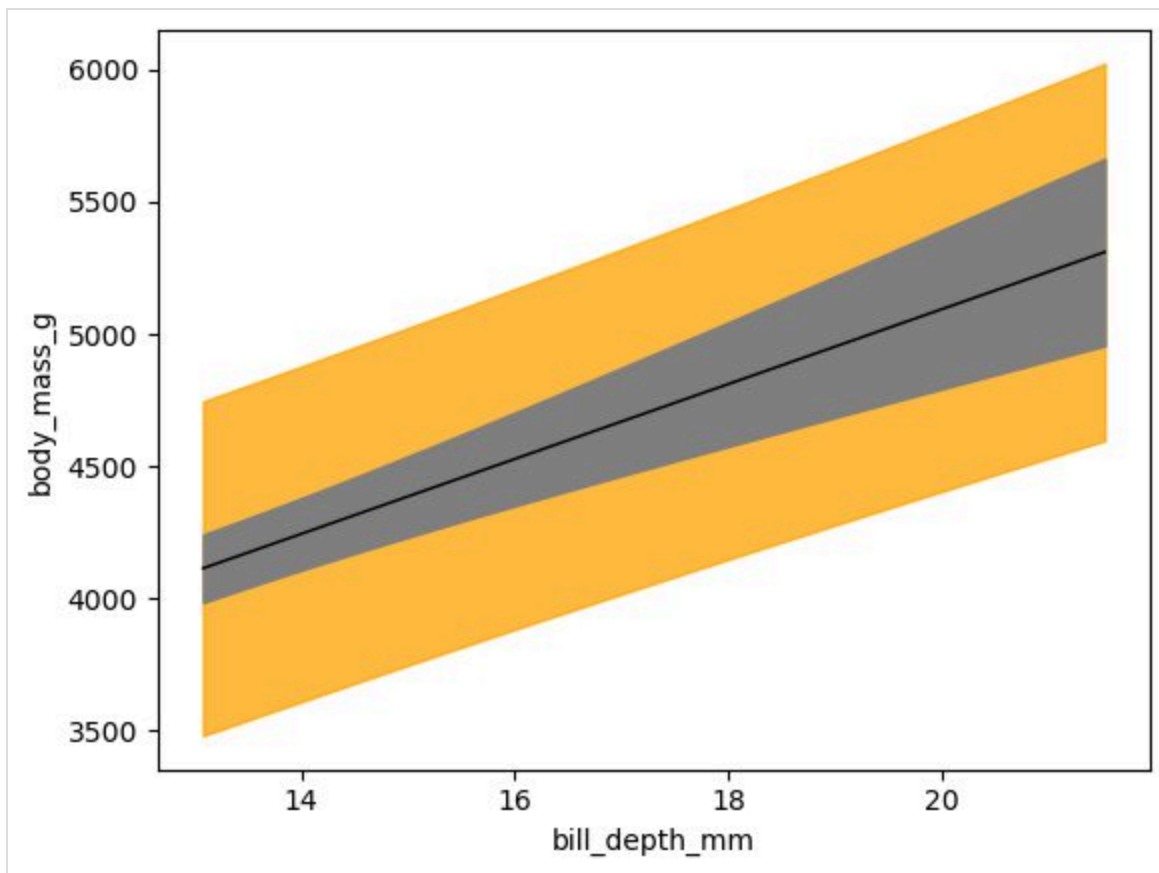
# Prediction interval
ax.fill_between(df_viz_3.bill_depth_mm,
               pred_summary_03.obs_ci_lower,
               pred_summary_03.obs_ci_upper,
               facecolor='orange',
               alpha=0.75,
               edgecolor='orange')

# Confidence interval
ax.fill_between(df_viz_3.bill_depth_mm,
               pred_summary_03.mean_ci_lower,
               pred_summary_03.mean_ci_upper,
               facecolor='grey',
               edgecolor='grey')

# Trend
ax.plot(df_viz_3.bill_depth_mm,
        pred_summary_03['mean'],
        color='k',
        linewidth=1)

# Set the labels
ax.set_xlabel('bill_depth_mm')
ax.set_ylabel('body_mass_g')

# Show the plot
plt.show()
```



Visualize Model 3 Predicted Response vs `bill_depth_mm` Across Multiple Fixed `bill_length_mm` Values (Mean Curves Only)

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```

```
# Build the varying grid for the two predictors of interest
input_grid = pd.DataFrame([(bill_depth_mm, bill_length_mm) for bill_depth_mm in
np.linspace(df.bill_depth_mm.min(),
df.bill_depth_mm.max(),
num=101)
for bill_length_mm in
np.linspace(df.bill_length_mm.min(),
df.bill_length_mm.max(),
num=9)],
columns=['bill_depth_mm', 'bill_length_mm'])

# Add fixed values for the remaining predictors
input_grid['flipper_length_mm'] = df.flipper_length_mm.mean()
input_grid['species'] = 'Gentoo'
```

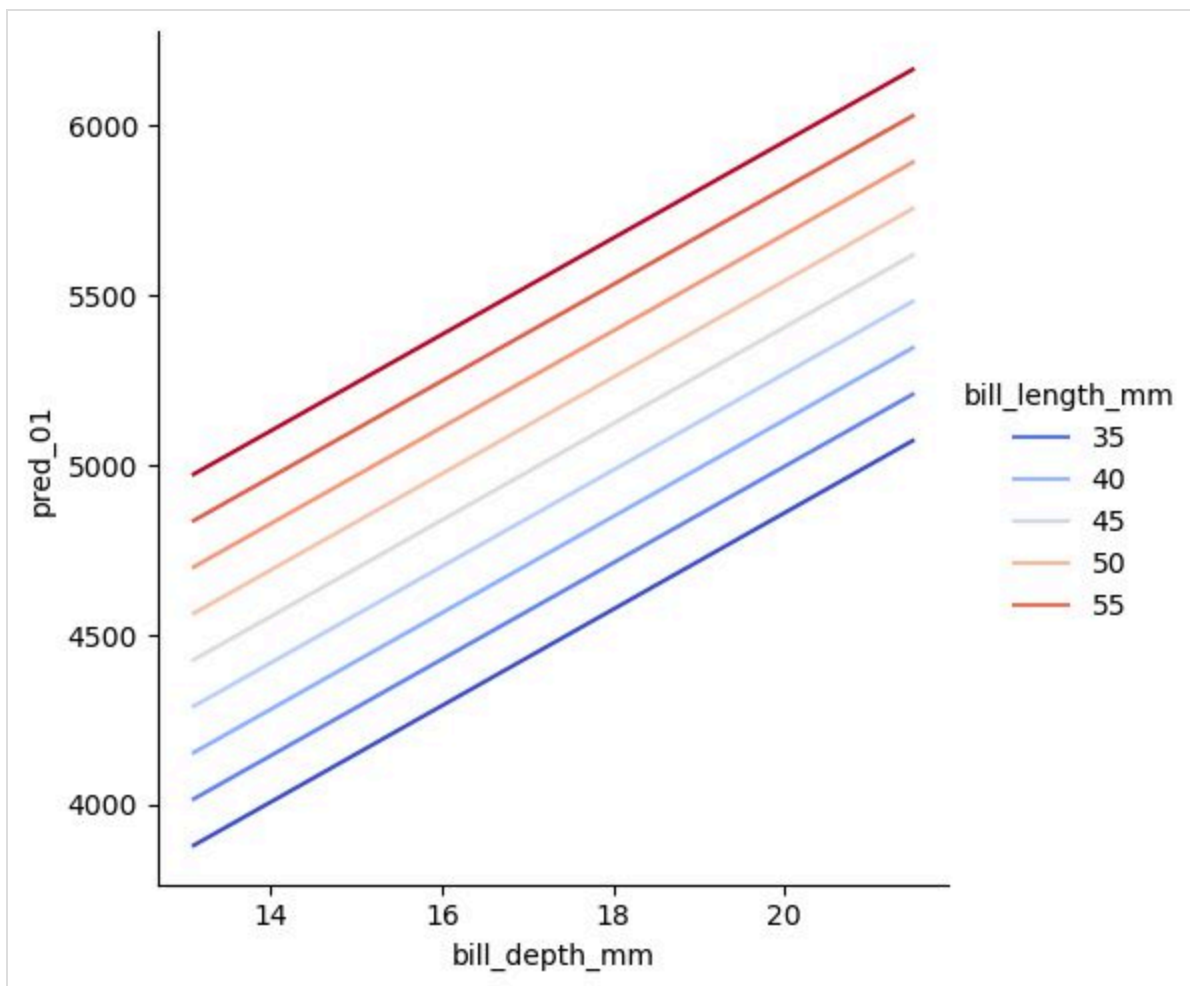


```
# Create a copy of the input grid for visualization
df_viz_1 = input_grid.copy()
```

```
# Make predictions on the varying grid
df_viz_1['pred_01'] = model_03.predict(input_grid)
```

```
# Create a line plot for the predictions
sns.relplot(data=df_viz_1,
            x='bill_depth_mm',
            y='pred_01',
            kind='line',
            hue='bill_length_mm',
            estimator=None,
            units='bill_length_mm',
            palette='coolwarm')
```

```
# Show the plot
plt.show()
```



Visualize Model 3 Predicted Response vs `bill_length_mm` Across Multiple Fixed `bill_depth_mm` Values (Mean Curves Only)

```
formula_list[3]
```

```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```

```
# Build the varying grid for the two predictors of interest
input_grid = pd.DataFrame([(bill_length_mm, bill_depth_mm) for bill_length_mm in
np.linspace(df.bill_length_mm.min(),
df.bill_length_mm.max(),
num=101)
for bill_depth_mm in
np.linspace(df.bill_depth_mm.min(),
df.bill_depth_mm.max(),
num=9)],
columns=['bill_length_mm', 'bill_depth_mm'])

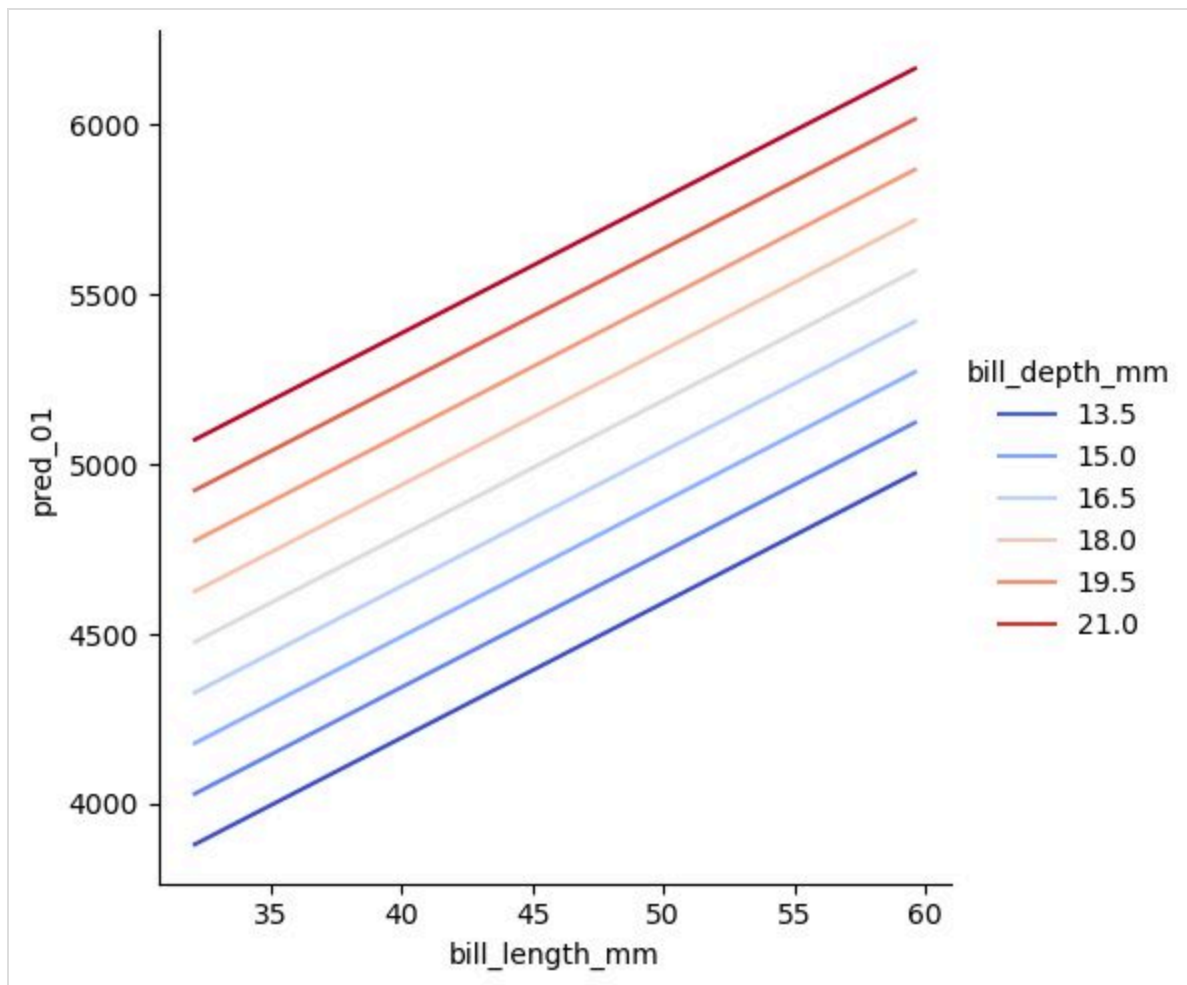
# Add fixed values for the remaining predictors
input_grid['flipper_length_mm'] = df.flipper_length_mm.mean()
input_grid['species'] = 'Gentoo'
```

```
# Create a copy of the input grid for visualization
df_viz_1 = input_grid.copy()
```

```
# Make predictions on the varying grid
df_viz_1['pred_01'] = model_03.predict(input_grid)
```

```
# Create a line plot for the predictions
sns.relplot(data=df_viz_1,
            x='bill_length_mm',
            y='pred_01',
            kind='line',
            hue='bill_depth_mm',
            estimator=None,
            units='bill_depth_mm',
            palette='coolwarm')

# Show the plot
plt.show()
```



Verify No Residual Pattern w/ Inputs

```
formula_list[3]
```

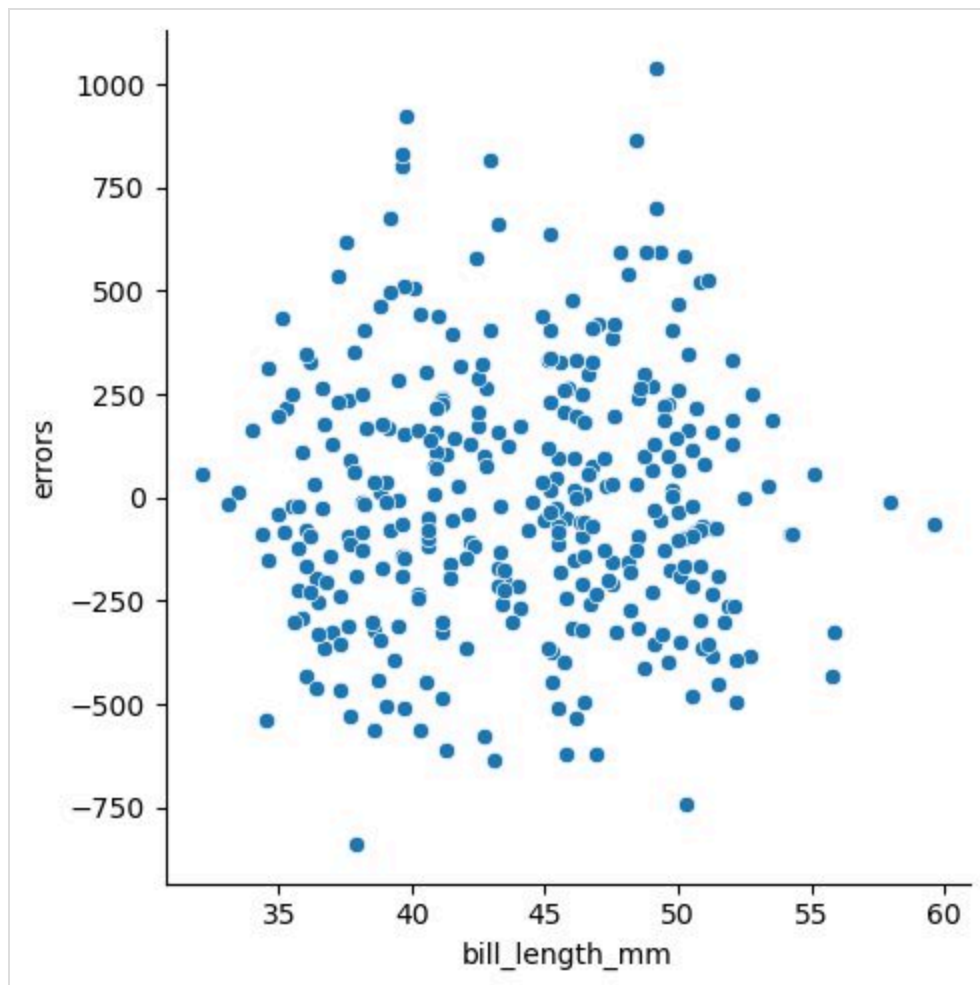
```
'Q("body_mass_g") ~ Q("bill_length_mm") + Q("bill_depth_mm") + Q("flipper_length_mm") +  
C(Q("species"))'
```

```
df['fitted'] = model_03.fittedvalues
```

```
df['errors'] = df['body_mass_g'] - df['fitted']
```

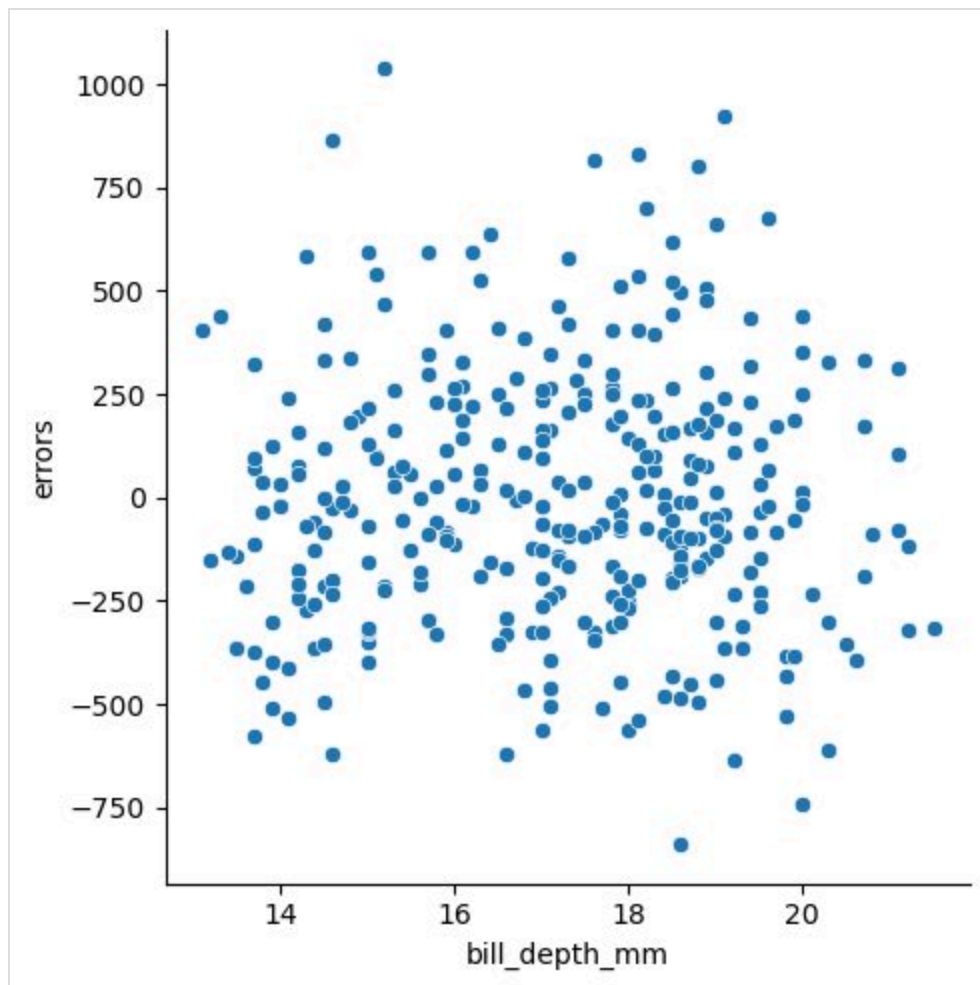
Model 3 Residuals vs bill_length_mm (No Pattern Expected)

```
sns.relplot(data=df,  
            x='bill_length_mm',  
            y='errors')  
plt.show()
```



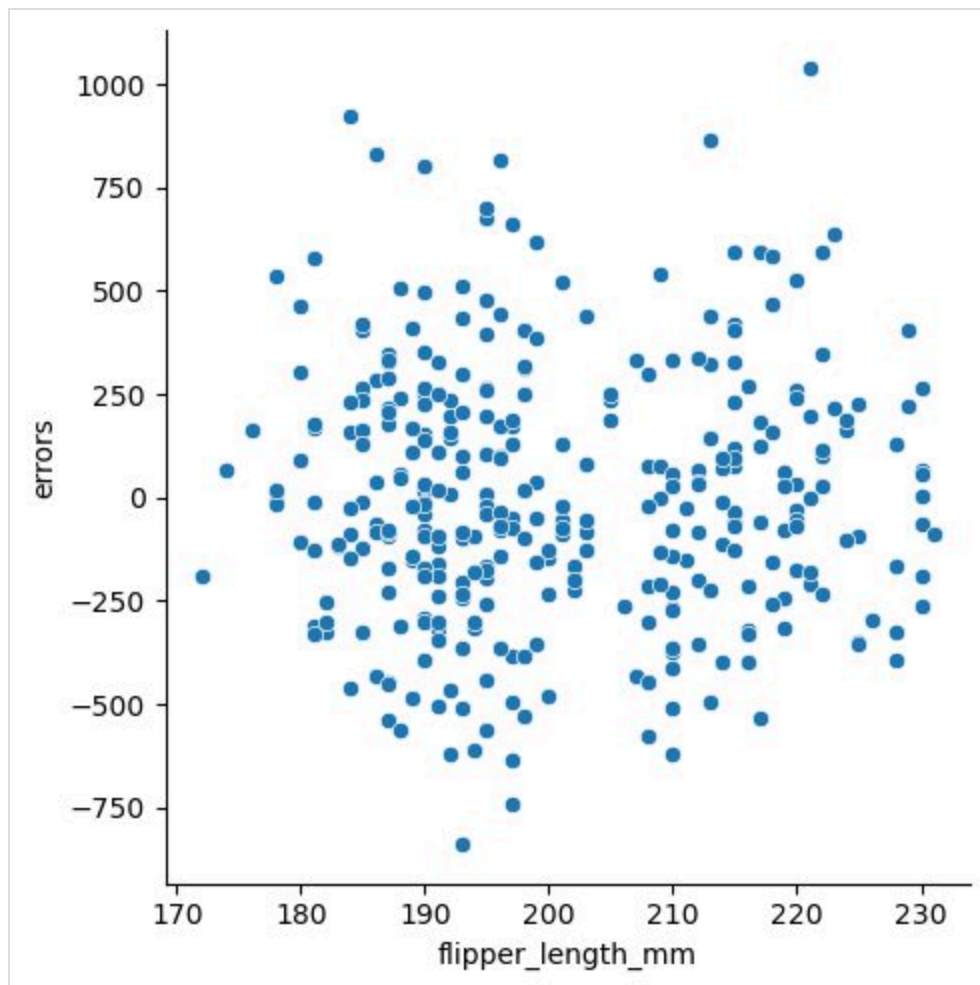
Model 3 Residuals vs bill_depth_mm (No Pattern Expected)

```
sns.relplot(data=df,  
            x='bill_depth_mm',  
            y='errors')  
plt.show()
```



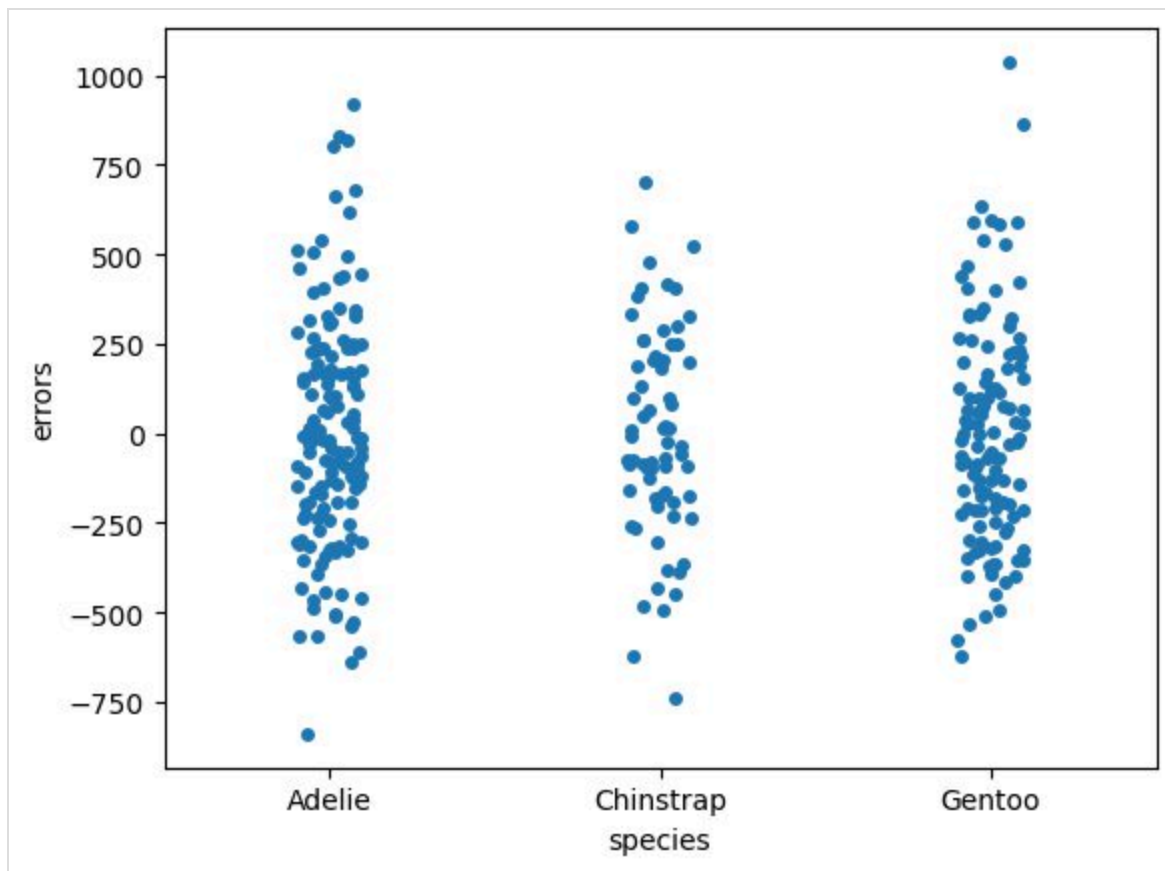
Model 3 Residuals vs flipper_length_mm (No Pattern Expected)

```
sns.relplot(data=df,  
            x='flipper_length_mm',  
            y='errors')  
plt.show()
```



Model 3 Residuals vs species (No Pattern Expected)

```
sns.stripplot(data=df,  
              x='species',  
              y='errors')  
plt.ylabel('errors')  
plt.show()
```



Save Best Model - Model 3 w/ Highest R-squared and Lowest RMSE w/ Least Coefficients

Save Model

```
with open('model_03.pkl', 'wb') as f:
    pickle.dump(model_03, f)
print('model saved to model_03.pkl')
```

```
model saved to model_03.pkl
```

Load Model

```
try:
    with open('model_03.pkl', 'rb') as f:
        model_03 = pickle.load(f)
except FileNotFoundError:
    print('model_03.pkl not found')
except pickle.UnpicklingError:
    print('failed to unpickle model_03.pkl (file may be corrupted or unsafe)')
else:
    print('model loaded from model_03.pkl')
```

```
model loaded from model_03.pkl
```

Inference on New Data

```
new_penguin_data = pd.DataFrame({
    'bill_length_mm': [22.0],
    'bill_depth_mm': [15.0],
    'flipper_length_mm': [200.0],
    'species': ['Gentoo']
})

predicted_body_mass = model_03.predict(new_penguin_data)
print(f'predicted body mass for the new penguin: {predicted_body_mass[0]:.2f} grams')
```

```
predicted body mass for the new penguin: 3728.78 grams
```


Part IV: Classification and Deep Learning with PyTorch

14-LR: Penguins Dataset - Logistic Regression

Penguins Dataset

Author: [Kevin Thomas](#)

Citation

[1] Allison Horst, <https://github.com/allisonhorst/palmerpenguins>

Import Modules

```
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from sklearn.linear_model import LogisticRegression
from patsy import dmatrices
from sklearn.model_selection import KFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Load Dataset

```
df = pd.read_csv('penguins-clean.csv')
```

```
df.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	
2	Adelie	Torgersen	40.3	18.0	195.0	
3	Adelie	Torgersen	36.7	19.3	193.0	
4	Adelie	Torgersen	39.3	20.6	190.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female
2	3250.0	Female
3	3450.0	Female
4	3650.0	Male

Logistic Regression

Logistic regression predicts the probability that a sample belongs to a class. We turn the three-class `species` target into a binary target, `is_adelie`, and model the probability that a penguin is an Adelie.

Create a Binary Target

```
df['is_adelie'] = (df['species'] == 'Adelie').astype(int)
df['is_adelie'].value_counts()
```

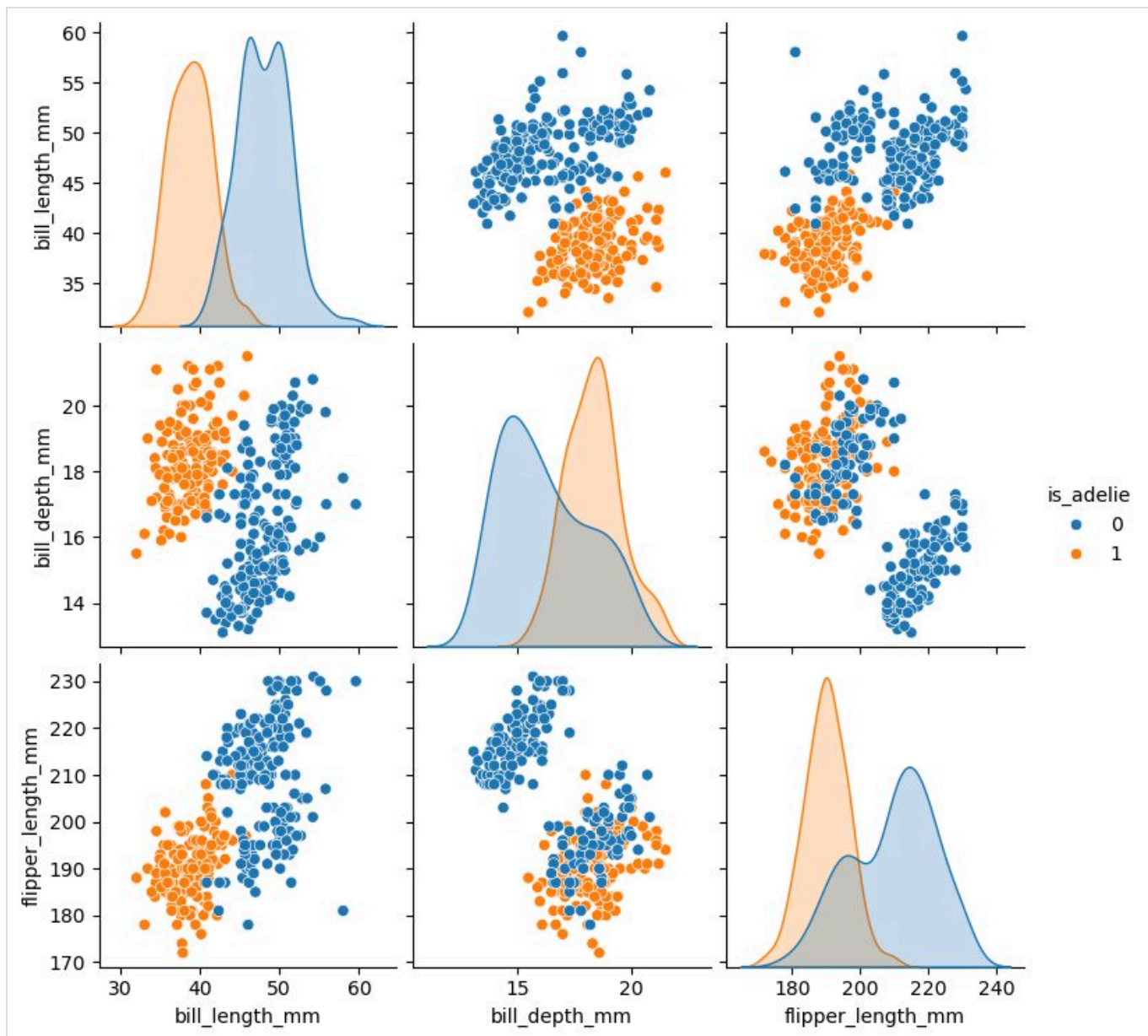
```
is_adelie
0      187
1      146
Name: count, dtype: int64
```

The classes are reasonably balanced, so accuracy is a fair starting metric, but we will still inspect ROC and AUC.

Visualize Relationship Between the Target and Each Continuous Input

Adelie penguins have shorter bills and flippers. `bill_length_mm` separates the classes most cleanly.

```
sns.pairplot(
    df,
    vars=['bill_length_mm',
          'bill_depth_mm',
          'flipper_length_mm'],
    hue='is_adelie')
plt.show()
```



Fit Logistic Regression Models - Training Data

Functions

`fit_and_assess_logit` fits a model with the statsmodels formula API and returns one row of metrics.

`predict_class` turns probabilities into class labels at a 0.5 threshold.

```
def fit_and_assess_logit(model, formula, df):
    """
    Fit a logistic regression model and return its metrics.

    Parameters:
        model (str): A descriptive name for the model.
        formula (str): A statsmodels formula string.
        df (pandas.DataFrame): The input data.

    Returns:
        pandas.DataFrame: A single-row DataFrame of metrics.
    """
    a_model = smf.logit(formula=formula, data=df).fit(dis=0)
    preds = predict_class(a_model, df)
    results_dict = {'model_name': model,
                    'model_formula': formula,
                    'num_coefs': len(a_model.params),
                    'pseudo_r2': a_model.prsquared,
                    'aic': a_model.aic,
                    'accuracy': (preds == df['is_adelie']).mean()}
    return pd.DataFrame(results_dict, index=[0])

def predict_class(a_model, df):
    """
    Predict binary class labels from a fitted model.

    Parameters:
        a_model (statsmodels.discrete.discrete_model.BinaryResultsWrapper): Fitted model.
        df (pandas.DataFrame): The input data.

    Returns:
        numpy.ndarray: Predicted class labels.
    """
    probs = a_model.predict(df)
    return (probs >= 0.5).astype(int)
```

Formulas

Island is left out of the predictors because every Gentoo penguin in this dataset lives on Biscoe island, which would leak the label.

```
formula_00 = 'is_adelie ~ 1'
formula_01 = 'is_adelie ~ flipper_length_mm'
formula_02 = 'is_adelie ~ bill_length_mm + bill_depth_mm + flipper_length_mm'
formula_03 = 'is_adelie ~ bill_length_mm + bill_depth_mm + flipper_length_mm + C(sex)'
formula_04 = 'is_adelie ~ bill_length_mm * bill_depth_mm + flipper_length_mm'
formula_05 = 'is_adelie ~ flipper_length_mm * C(sex)'
formula_05
```

```
'is_adelie ~ flipper_length_mm * C(sex)'
```

Model 00: Intercept-Only (Baseline)

```
fit_and_assess_logit('Model 00', formula_00, df)
```

	model_name	model_formula	num_coefs	pseudo_r2	aic	accuracy
0	Model 00	is_adelie ~ 1	1	7.149836e-13	458.575142	0.561562

Model 01: Flipper Length Only

```
fit_and_assess_logit('Model 01', formula_01, df)
```

	model_name	model_formula	num_coefs	pseudo_r2	aic	\
0	Model 01	is_adelie ~ flipper_length_mm	2	0.450399	254.93432	

	accuracy
0	0.822823

Model 02: All Continuous Predictors

```
fit_and_assess_logit('Model 02', formula_02, df)
```

	model_name	model_formula	num_coefs	\
0	Model 02	is_adelie ~ bill_length_mm + bill_depth_mm + f...	4	

	pseudo_r2	aic	accuracy
0	0.9658	23.614828	0.990991

Model 03: Continuous Predictors + Sex

```
fit_and_assess_logit('Model 03', formula_03, df)
```

```
/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))
```

```

model_name                                model_formula  num_coefs  \
0  Model 03  is_adelie ~ bill_length_mm + bill_depth_mm + f...      5

pseudo_r2      aic  accuracy
0      1.0  10.000157      1.0

```

Model 04: Continuous Interaction

```
fit_and_assess_logit('Model 04', formula_04, df)
```

```

model_name                                model_formula  num_coefs  \
0  Model 04  is_adelie ~ bill_length_mm * bill_depth_mm + f...      5

pseudo_r2      aic  accuracy
0  0.965802  25.613883  0.990991

```

Model 05: Flipper Length Interacting With Sex

```
fit_and_assess_logit('Model 05', formula_05, df)
```

```

model_name                                model_formula  num_coefs  pseudo_r2  \
0  Model 05  is_adelie ~ flipper_length_mm * C(sex)          4  0.482506

aic  accuracy
0  244.275095  0.840841

```

Compare Fitted Models - Training Data

Model 03 and Model 04 reach the highest accuracy with the fewest coefficients. Higher pseudo R-squared and lower AIC both indicate a better fit.

```

training_results = pd.concat([
    fit_and_assess_logit('Model 00', formula_00, df),
    fit_and_assess_logit('Model 01', formula_01, df),
    fit_and_assess_logit('Model 02', formula_02, df),
    fit_and_assess_logit('Model 03', formula_03, df),
    fit_and_assess_logit('Model 04', formula_04, df),
    fit_and_assess_logit('Model 05', formula_05, df)])
training_results.sort_values('accuracy', ascending=False)

```

```

/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))

```

	model_name	model_formula	num_coefs	\
0	Model 03	is_adelie ~ bill_length_mm + bill_depth_mm + f...	5	
0	Model 02	is_adelie ~ bill_length_mm + bill_depth_mm + f...	4	
0	Model 04	is_adelie ~ bill_length_mm * bill_depth_mm + f...	5	
0	Model 05	is_adelie ~ flipper_length_mm * C(sex)	4	
0	Model 01	is_adelie ~ flipper_length_mm	2	
0	Model 00	is_adelie ~ 1	1	

	pseudo_r2	aic	accuracy
0	9.999997e-01	10.000157	1.000000
0	9.658001e-01	23.614828	0.990991
0	9.658022e-01	25.613883	0.990991
0	4.825056e-01	244.275095	0.840841
0	4.503986e-01	254.934320	0.822823
0	7.149836e-13	458.575142	0.561562

Fit Logistic Regression Models w/ Cross-Validation - Test Data

Functions

```

def logit_cross_val_score(model, formula, init_model, df, cv):
    """
    Run cross-validated evaluation for a logistic regression formula.

    Parameters:
        model (str): A human-readable model name.
        formula (str): A Patsy-compatible formula string.
        init_model (object): An unfitted scikit-learn estimator.
        df (pandas.DataFrame): The input data.
        cv (int): Number of cross-validation folds.

    Returns:
        pandas.DataFrame: Per-fold accuracy scores.
    """
    y, X = dmatrices(formula, data=df, return_type='dataframe')
    y = np.ravel(y)
    scores = cross_val_score(init_model, X, y, cv=cv, scoring='accuracy')
    return pd.DataFrame({'model_name': model,
                        'fold': range(1, cv + 1),
                        'accuracy': scores})

```


Fit w/ Cross-Validation

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_models = [('Model 00', formula_00),
              ('Model 01', formula_01),
              ('Model 02', formula_02),
              ('Model 03', formula_03),
              ('Model 04', formula_04),
              ('Model 05', formula_05)]
cv_results = pd.concat([
    logit_cross_val_score(name, formula, LogisticRegression(max_iter=5000), df, 5)
    for name, formula in cv_models])
cv_results.head()
```

	model_name	fold	accuracy
0	Model 00	1	0.552239
1	Model 00	2	0.567164
2	Model 00	3	0.567164
3	Model 00	4	0.560606
4	Model 00	5	0.560606

Compare Fitted Models - Test Data

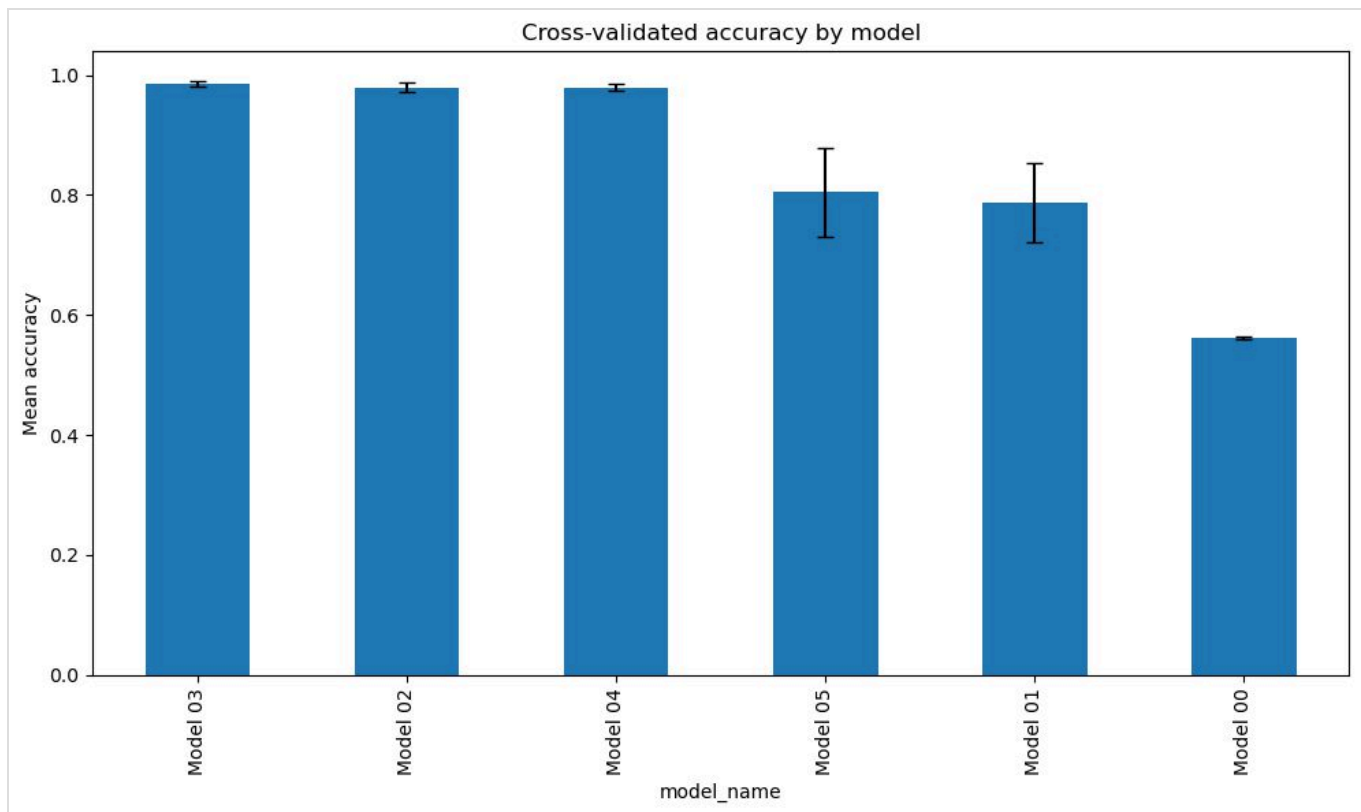
The cross-validated accuracy is the honest estimate. Model 03 is a strong choice: high accuracy with fewer coefficients than Model 04.

```
cv_summary = cv_results.groupby('model_name')['accuracy'].agg(['mean', 'std', 'count'])
cv_summary = cv_summary.sort_values('mean', ascending=False)
cv_summary
```

	model_name	mean	std	count
	Model 03	0.985029	0.010554	5
	Model 02	0.979059	0.016998	5
	Model 04	0.979059	0.013325	5
	Model 05	0.805246	0.165472	5
	Model 01	0.787291	0.148884	5
	Model 00	0.561556	0.006155	5

Visualize Average Accuracy w/ 1 Standard Error Interval

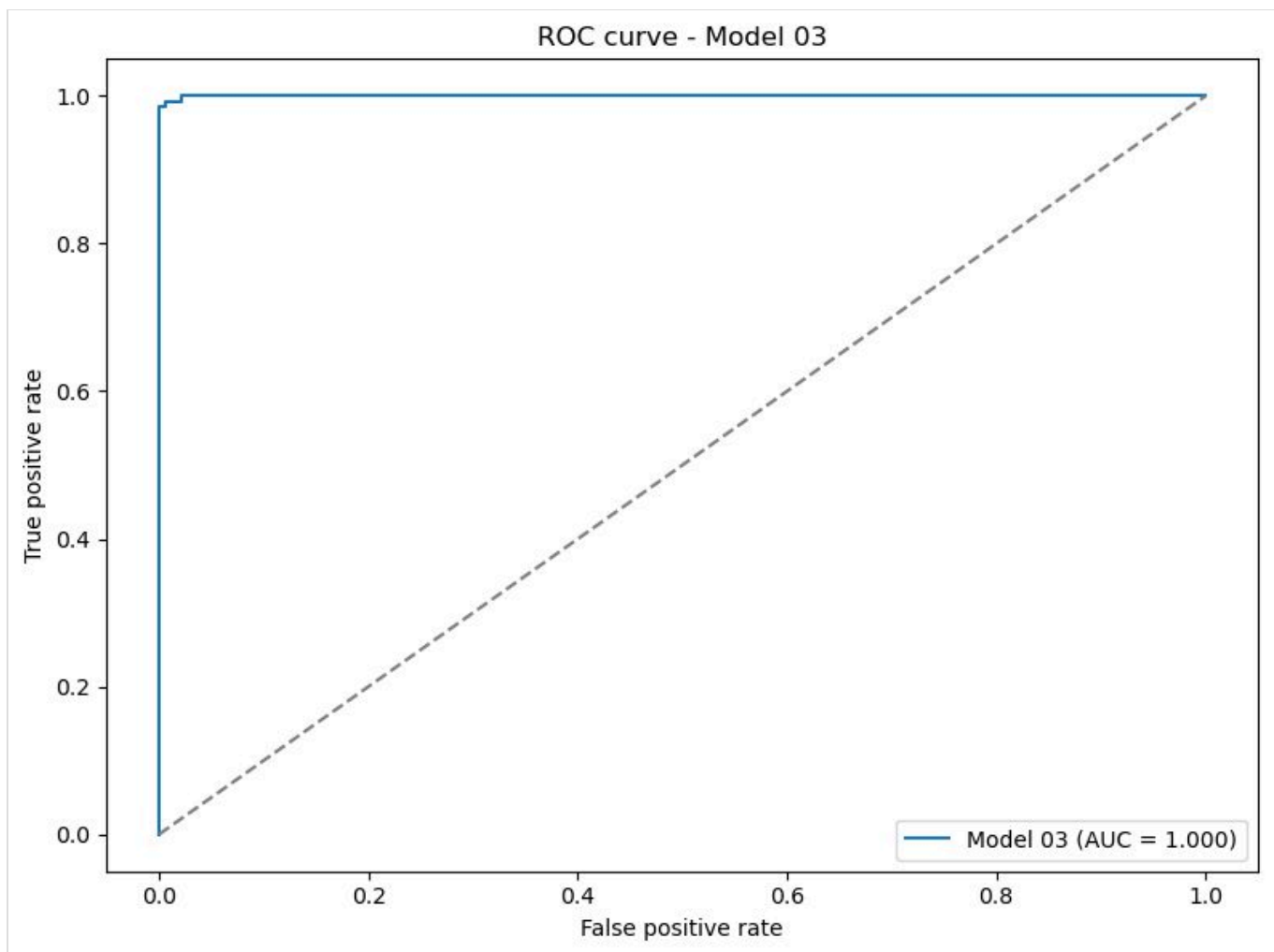
```
ax = cv_summary['mean'].plot(
    kind='bar',
    yerr=cv_summary['std'] / np.sqrt(cv_summary['count']),
    capsize=4,
    figsize=(10, 6))
ax.set_ylabel('Mean accuracy')
ax.set_title('Cross-validated accuracy by model')
plt.tight_layout()
plt.show()
```



Visualize ROC Curves

A model with a high AUC ranks Adelie above non-Adelie across all thresholds. Model 03 is close to a perfect ranker.

```
y, X = dmatrices(formula_03, data=df, return_type='dataframe')
y = np.ravel(y)
best_model = LogisticRegression(max_iter=5000).fit(X, y)
probs = best_model.predict_proba(X)[:, 1]
fpr, tpr, _ = roc_curve(y, probs)
auc_score = roc_auc_score(y, probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Model 03 (AUC = {auc_score:.3f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve - Model 03')
plt.legend()
plt.tight_layout()
plt.show()
```



Confusion Matrix and Classification Report

```
preds = (probs >= 0.5).astype(int)
print(confusion_matrix(y, preds))
print(classification_report(y, preds))
```

```
[[186  1]
 [ 1 145]]
```

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	187
1.0	0.99	0.99	0.99	146
accuracy			0.99	333
macro avg	0.99	0.99	0.99	333
weighted avg	0.99	0.99	0.99	333

Save Best Model - Model 03 w/ Highest Cross-Validated Accuracy

Save Model

```
final_model = smf.logit(formula=formula_03, data=df).fit(dis=0)
with open('logit_model.pkl', 'wb') as handle:
    pickle.dump(final_model, handle)
print(final_model.summary())
```

Logit Regression Results

```
=====
Dep. Variable:          is_adelie    No. Observations:          333
Model:                  Logit        Df Residuals:              328
Method:                 MLE          Df Model:                  4
Date:                  Wed, 23 Sep 2026    Pseudo R-squ.:            1.000
Time:                  19:47:23          Log-Likelihood:           -7.8366e-05
converged:              False          LL-Null:                 -228.29
Covariance Type:        nonrobust        LLR p-value:             1.646e-97
=====
```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	1059.3043	1.94e+08	5.46e-06	1.000	-3.8e+08	3.8e+08
C(sex)[T.Male]	117.0684	1.71e+07	6.85e-06	1.000	-3.35e+07	3.35e+07
bill_length_mm	-67.9876	7.2e+06	-9.44e-06	1.000	-1.41e+07	1.41e+07
bill_depth_mm	103.6875	1.74e+07	5.96e-06	1.000	-3.41e+07	3.41e+07
flipper_length_mm	-0.0532	8.7e+05	-6.12e-08	1.000	-1.71e+06	1.71e+06

```
=====
```

Complete Separation: The results show that there is complete separation or perfect prediction.

In this case the Maximum Likelihood Estimator does not exist and the parameters are not identified.

```
/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))
/opt/anaconda3/envs/prod/lib/python3.12/site-packages/statsmodels/base/model.py:607:
ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
/opt/anaconda3/envs/prod/lib/python3.12/site-
packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered
in exp
    return 1/(1+np.exp(-X))
```

Load Model

```
with open('logit_model.pkl', 'rb') as handle:
    loaded_model = pickle.load(handle)
loaded_model.params
```

```
Intercept          1059.304311
C(sex)[T.Male]      117.068432
bill_length_mm     -67.987582
bill_depth_mm      103.687470
flipper_length_mm  -0.053247
dtype: float64
```

Inference on New Data

Feed the loaded model a new penguin and read the probability that it is an Adelie.

```
new_penguin = pd.DataFrame({
    'bill_length_mm': [39.0],
    'bill_depth_mm': [18.5],
    'flipper_length_mm': [190.0],
    'sex': ['Female']})
probability = loaded_model.predict(new_penguin).iloc[0]
print(f'Probability of Adelie: {probability:.4f}')
print('Predicted class:', 'Adelie' if probability >= 0.5 else 'Not Adelie')
```

```
Probability of Adelie: 1.0000
Predicted class: Adelie
```

15-MLP: Penguins Dataset - MLP

[Penguins Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib seaborn scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.feature_selection import f_classif
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x124949af0>
```

Parameters

```
IN_FEATURES = 3
IN_FEATURES
```

3

H1 = 16
H1

16

H2 = 16
H2

16

OUT_FEATURES = 3
OUT_FEATURES

3

TEST_SIZE = 0.3
TEST_SIZE

0.3

MODE = "classification" # "classification" or "regression"
MODE

'classification'

Hyperparameters

LEARNING_RATE = 0.01
LEARNING_RATE

0.01

EPOCHS = 200
EPOCHS

```
200
```

```
BATCH_SIZE = 16  
BATCH_SIZE
```

```
16
```

```
DROPOUT = 0.0  
DROPOUT
```

```
0.0
```

Load Dataset

```
df = pd.read_csv('penguins-clean.csv')
```

```
df.head()
```

```
   species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \  
0  Adelie  Torgersen             39.1           18.7             181.0  
1  Adelie  Torgersen             39.5           17.4             186.0  
2  Adelie  Torgersen             40.3           18.0             195.0  
3  Adelie  Torgersen             36.7           19.3             193.0  
4  Adelie  Torgersen             39.3           20.6             190.0  
  
   body_mass_g  sex  
0      3750.0  Male  
1      3800.0 Female  
2      3250.0 Female  
3      3450.0 Female  
4      3650.0  Male
```

Clean Dataset

Map the three species to integers, keep the three continuous measurements, and standardize them so the network trains evenly.


```

SPECIES_MAP = {'Adelie': 0, 'Chinstrap': 1, 'Gentoo': 2}
df['species_id'] = df['species'].map(SPECIES_MAP)
FEATURES = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm']
X = df[FEATURES].to_numpy(dtype=np.float32)
y = df['species_id'].to_numpy(dtype=np.int64)
X_MEAN = X.mean(axis=0)
X_STD = X.std(axis=0)
X = (X - X_MEAN) / X_STD
X.shape, y.shape

```

```
((333, 3), (333,))
```

Feature Statistical Significance (P-Values)

All three features are statistically significant. `flipper_length_mm` has the strongest separation between species.

```

f_values, p_values = f_classif(X, y)
pd.DataFrame({'feature': FEATURES,
              'f_value': f_values,
              'p_value': p_values}).sort_values('p_value')

```

	feature	f_value	p_value
0	bill_length_mm	397.299805	0.0
1	bill_depth_mm	344.824921	0.0
2	flipper_length_mm	567.408508	0.0

Device

```

DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE

```

```
device(type='mps')
```

Create Model

```
class Model(nn.Module):
    """
    A feedforward neural network with two hidden layers and optional dropout.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, h2=H2,
                  out_features=OUT_FEATURES, dropout=DROPOUT):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.
            dropout (float): Dropout rate (0.0 = no dropout).

        Returns:
            None
        """
        super(Model, self).__init__()
        self.fc1 = nn.Linear(in_features, h1)
        self.fc2 = nn.Linear(h1, h2)
        self.out = nn.Linear(h2, out_features)
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        return self.out(x)
```

Instantiate Model

```
torch.manual_seed(SEED)
model = Model().to(DEVICE)
model
```

```
Model(  
    (fc1): Linear(in_features=3, out_features=16, bias=True)  
    (fc2): Linear(in_features=16, out_features=16, bias=True)  
    (out): Linear(in_features=16, out_features=3, bias=True)  
    (dropout): Dropout(p=0.0, inplace=False)  
)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)  
X_train.shape, X_test.shape
```

```
((233, 3), (100, 3))
```

Create Custom Dataset (Memory Efficient)

```

class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device, mode="classification"):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.
            mode (str): "classification" or "regression".

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device
        self.mode = mode

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        if self.mode == "classification":
            y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        else:
            y_tensor = torch.tensor(self.y[idx]).float().to(self.device)
        return X_tensor, y_tensor

```

Create DataLoaders

```
train_dataset = NumpyDataset(X_train, y_train, DEVICE, MODE)
test_dataset = NumpyDataset(X_test, y_test, DEVICE, MODE)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

```
(15, 7)
```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss for the epoch.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch + 1:3d} | loss {test_loss:.4f} | accuracy {test_accuracy:.4f}")
```

Epoch 20 | loss 0.0159 | accuracy 0.9900

Epoch 40 | loss 0.0169 | accuracy 0.9900

Epoch 60 | loss 0.0232 | accuracy 0.9900

Epoch 80 | loss 0.0270 | accuracy 0.9900

Epoch 100 | loss 0.0295 | accuracy 0.9900

Epoch 120 | loss 0.0228 | accuracy 0.9900

Epoch 140 | loss 0.0357 | accuracy 0.9900

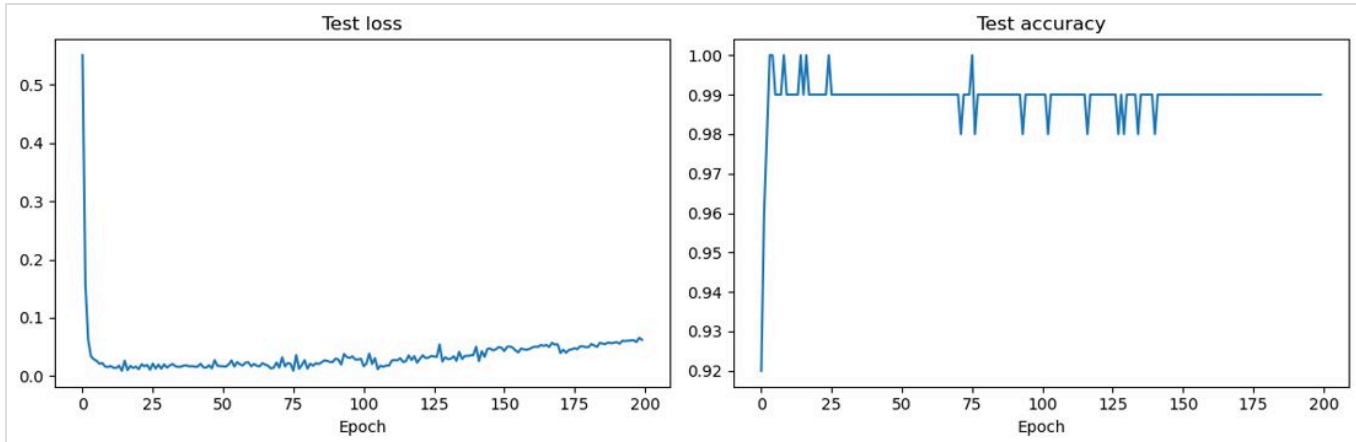
Epoch 160 | loss 0.0464 | accuracy 0.9900

Epoch 180 | loss 0.0495 | accuracy 0.9900

Epoch 200 | loss 0.0619 | accuracy 0.9900

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.9900

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.cpu().numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=list(SPECIES_MAP)))
```



```
[[43  1  0]
 [ 0 20  0]
 [ 0  0 36]]
```

	precision	recall	f1-score	support
Adelie	1.00	0.98	0.99	44
Chinstrap	0.95	1.00	0.98	20
Gentoo	1.00	1.00	1.00	36
accuracy			0.99	100
macro avg	0.98	0.99	0.99	100
weighted avg	0.99	0.99	0.99	100

Save Model

```
torch.save(model.state_dict(), 'mlp_penguins.pt')
print('saved mlp_penguins.pt')
```

```
saved mlp_penguins.pt
```

Load Model

```
loaded_model = Model().to(DEVICE)
loaded_model.load_state_dict(torch.load('mlp_penguins.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded mlp_penguins.pt')
```

```
loaded mlp_penguins.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55235/3871248278.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('mlp_penguins.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):  
    """  
    Predict the species for one standardized measurement.  
  
    Parameters:  
        model (nn.Module): Trained model.  
        features (list): Raw bill length, bill depth, and flipper length.  
  
    Returns:  
        tuple: Predicted species name and confidence.  
    """  
    model.eval()  
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD  
    with torch.no_grad():  
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)  
        probs = torch.softmax(model(X_new), dim=1)  
        confidence, predicted = probs.max(dim=1)  
    return list(SPECIES_MAP[predicted.item()], confidence.item())
```

Run Inference

```
species, confidence = predict(loader_model, [47.5, 15.0, 217.0])  
print(f"Predicted species: {species} ({confidence:.4f})")
```

```
Predicted species: Gentoo (1.0000)
```

16-CNN: Fashion-MNIST Dataset - CNN

[Fashion-MNIST Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x11c349af0>
```

Parameters

```
IN_CHANNELS = 1
IN_CHANNELS
```

```
1
```

```
NUM_CLASSES = 10
NUM_CLASSES
```

```
10
```

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
CLASS_NAMES
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 5
EPOCHS
```

5

```
BATCH_SIZE = 128
BATCH_SIZE
```

128

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

Fashion-MNIST is a real dataset of 70,000 grayscale clothing images. torchvision downloads it once into `data/` and normalizes each image.

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))]
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
len(train_data), len(test_data)
```

```
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
```

```
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz to data/FashionMNIST/raw/train-images-idx3-ubyte.gz
```

```
0%|          | 0.00/26.4M [00:00<?, ?B/s]
```

```
0%|          | 65.5k/26.4M [00:00<01:10, 371kB/s]
```

1%| | 197k/26.4M [00:00<00:33, 779kB/s]

1%|| | 393k/26.4M [00:00<00:21, 1.23MB/s]

2%|| | 557k/26.4M [00:00<00:21, 1.23MB/s]

4%|| | 1.05M/26.4M [00:00<00:10, 2.36MB/s]

6%|| | 1.57M/26.4M [00:00<00:07, 3.22MB/s]

8%|| | 2.20M/26.4M [00:00<00:05, 4.10MB/s]

11%|| | 2.79M/26.4M [00:00<00:05, 4.61MB/s]

12%|| | 3.28M/26.4M [00:01<00:05, 4.26MB/s]

15%|| | 3.90M/26.4M [00:01<00:04, 4.79MB/s]

17%|| | 4.49M/26.4M [00:01<00:04, 5.09MB/s]

19%|| | 5.11M/26.4M [00:01<00:03, 5.35MB/s]

22%|| | 5.87M/26.4M [00:01<00:04, 4.86MB/s]

25%|| | 6.68M/26.4M [00:01<00:03, 5.59MB/s]

29% |  | 7.54M/26.4M [00:01<00:03, 6.05MB/s]

32% |  | 8.42M/26.4M [00:01<00:02, 6.02MB/s]

35% |  | 9.18M/26.4M [00:02<00:03, 5.50MB/s]

38% |  | 10.1M/26.4M [00:02<00:02, 6.26MB/s]

41% |  | 10.9M/26.4M [00:02<00:02, 6.72MB/s]

45% |  | 11.9M/26.4M [00:02<00:01, 7.38MB/s]

48% |  | 12.7M/26.4M [00:02<00:02, 5.77MB/s]

50% |  | 13.3M/26.4M [00:02<00:02, 4.98MB/s]

56% |  | 14.9M/26.4M [00:02<00:01, 6.94MB/s]

60% |  | 15.7M/26.4M [00:03<00:01, 5.92MB/s]

63% |  | 16.5M/26.4M [00:03<00:01, 6.35MB/s]

66% |  | 17.4M/26.4M [00:03<00:01, 6.89MB/s]

69% |  | 18.3M/26.4M [00:03<00:01, 7.24MB/s]

72%|██████████ | 19.1M/26.4M [00:03<00:01, 7.30MB/s]

75%|██████████ | 19.9M/26.4M [00:03<00:01, 6.47MB/s]

78%|██████████ | 20.5M/26.4M [00:03<00:00, 6.42MB/s]

80%|██████████ | 21.3M/26.4M [00:03<00:00, 6.60MB/s]

84%|██████████ | 22.2M/26.4M [00:04<00:00, 7.36MB/s]

87%|██████████ | 23.0M/26.4M [00:04<00:00, 6.12MB/s]

91%|██████████ | 24.0M/26.4M [00:04<00:00, 6.91MB/s]

94%|██████████ | 24.9M/26.4M [00:04<00:00, 7.53MB/s]

97%|██████████ | 25.7M/26.4M [00:04<00:00, 7.67MB/s]

100%|██████████ | 26.4M/26.4M [00:04<00:00, 5.61MB/s]

Extracting data/FashionMNIST/raw/train-images-idx3-ubyte.gz to data/FashionMNIST/raw

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz>

Downloading <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz> to data/FashionMNIST/raw/train-labels-idx1-ubyte.gz

0%| | 0.00/29.5k [00:00<?, ?B/s]

100%|██████████| 29.5k/29.5k [00:00<00:00, 313kB/s]

Extracting data/FashionMNIST/raw/train-labels-idx1-ubyte.gz to data/FashionMNIST/raw
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz to data/FashionMNIST/raw/t10k-images-idx3-ubyte.gz

0%| | 0.00/4.42M [00:00<?, ?B/s]

1%| | 32.8k/4.42M [00:00<00:14, 309kB/s]

1%|| | 65.5k/4.42M [00:00<00:19, 227kB/s]

3%|| | 131k/4.42M [00:00<00:12, 339kB/s]

5%|| | 229k/4.42M [00:00<00:07, 537kB/s]

11%|█ | 492k/4.42M [00:00<00:03, 1.17MB/s]

21%|██ | 950k/4.42M [00:00<00:01, 1.84MB/s]

43%|██████ | 1.90M/4.42M [00:00<00:00, 3.85MB/s]

62%|██████████ | 2.72M/4.42M [00:00<00:00, 5.01MB/s]

74%|██████████ | 3.28M/4.42M [00:01<00:00, 5.00MB/s]

87%|██████████ | 3.83M/4.42M [00:01<00:00, 3.81MB/s]

100%|██████████ | 4.42M/4.42M [00:01<00:00, 3.32MB/s]

Extracting data/FashionMNIST/raw/t10k-images-idx3-ubyte.gz to data/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz to data/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz

0%| | 0.00/5.15k [00:00<?, ?B/s]

100%|██████████ | 5.15k/5.15k [00:00<00:00, 12.1MB/s]

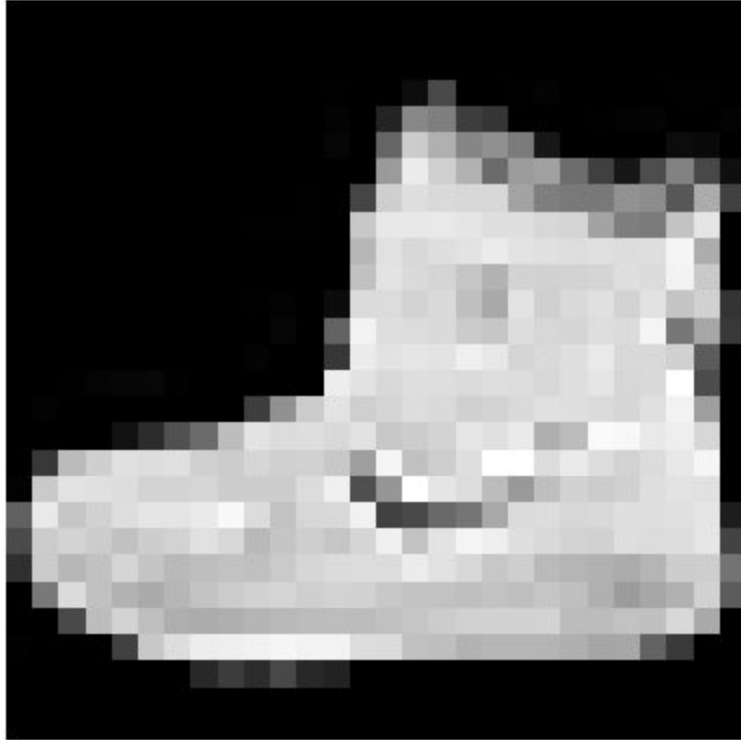
Extracting data/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz to data/FashionMNIST/raw

(60000, 10000)

Visualize a Sample

```
image, label = train_data[0]
plt.imshow(image.squeeze(), cmap='gray')
plt.title(CLASS_NAMES[label])
plt.axis('off')
plt.show()
```

Ankle boot



Create Model

```
class CNN(nn.Module):
    """
    A small convolutional neural network for 28x28 grayscale images.
    """

    def __init__(self, num_classes=NUM_CLASSES):
        """
        Initialize the convolutional and dense layers.

        Parameters:
            num_classes (int): Number of output classes.

        Returns:
            None
        """
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Output logits.
        """
        x = self.features(x)
        return self.classifier(x)
```

Instantiate Model

```
torch.manual_seed(SEED)
model = CNN().to(DEVICE)
model
```

```
CNN(
  (features): Sequential(
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=1568, out_features=128, bias=True)
    (2): ReLU()
    (3): Linear(in_features=128, out_features=10, bias=True)
  )
)
```

Create DataLoaders

```
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

```
(469, 79)
```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss for the epoch.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

Epoch 1 | train loss 0.4970 | test accuracy 0.8677

Epoch 2 | train loss 0.3266 | test accuracy 0.8844

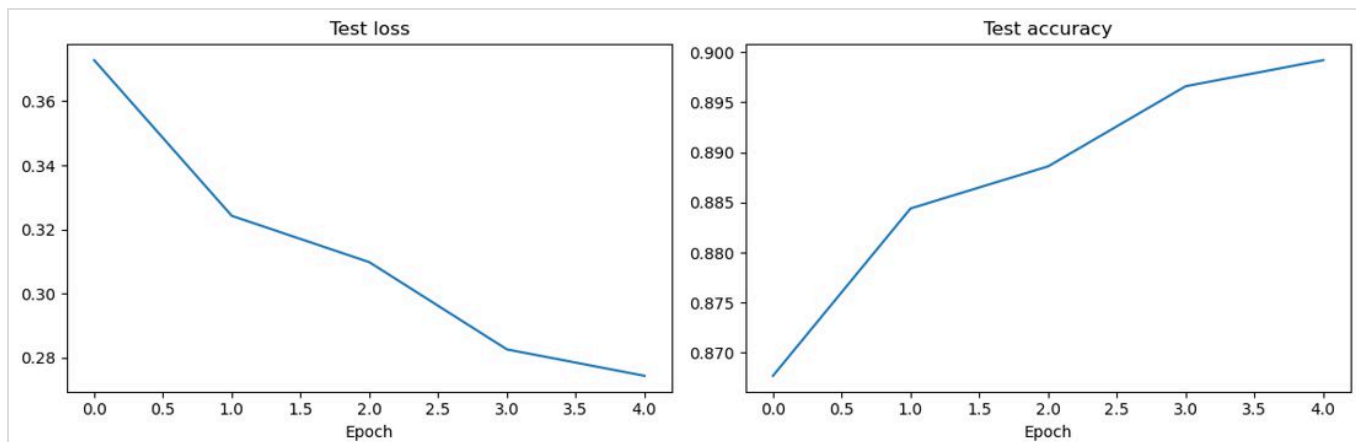
Epoch 3 | train loss 0.2806 | test accuracy 0.8886

Epoch 4 | train loss 0.2516 | test accuracy 0.8966

Epoch 5 | train loss 0.2316 | test accuracy 0.8992

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.8992

Confusion Matrix and Classification Report

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(DEVICE)
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES))
```

```
[[776   0  11  16   6   1 185   0   5   0]
 [  1 982   0  10   4   0   2   0   1   0]
 [ 12   1 773   4 119   0  91   0   0   0]
 [  7   6  12 876  50   0  48   0   1   0]
 [  1   1  20   9 919   0  50   0   0   0]
 [  0   0   0   0   0 967   0  20   0  13]
 [ 52   1  36  21  90   0 798   0   2   0]
 [  0   0   0   0   0   9   0 951   0  40]
 [  4   2   2   3   3   1   6   4 975   0]
 [  0   0   0   0   0   4   1  20   0 975]]

              precision    recall  f1-score   support

T-shirt/top      0.91      0.78      0.84      1000
  Trouser        0.99      0.98      0.99      1000
  Pullover       0.91      0.77      0.83      1000
    Dress       0.93      0.88      0.90      1000
     Coat       0.77      0.92      0.84      1000
   Sandal       0.98      0.97      0.98      1000
     Shirt      0.68      0.80      0.73      1000
   Sneaker       0.96      0.95      0.95      1000
      Bag       0.99      0.97      0.98      1000
Ankle boot      0.95      0.97      0.96      1000

 accuracy              0.90      10000
 macro avg           0.91      0.90      0.90      10000
weighted avg           0.91      0.90      0.90      10000
```


Save Model

```
torch.save(model.state_dict(), 'cnn_fashion_mnist.pt')
print('saved cnn_fashion_mnist.pt')
```

```
saved cnn_fashion_mnist.pt
```

Load Model

```
loaded_model = CNN().to(DEVICE)
loaded_model.load_state_dict(torch.load('cnn_fashion_mnist.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded cnn_fashion_mnist.pt')
```

```
loaded cnn_fashion_mnist.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55291/2230075774.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
```

```
loaded_model.load_state_dict(torch.load('cnn_fashion_mnist.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, image):
    """
    Predict the class of one normalized image tensor.

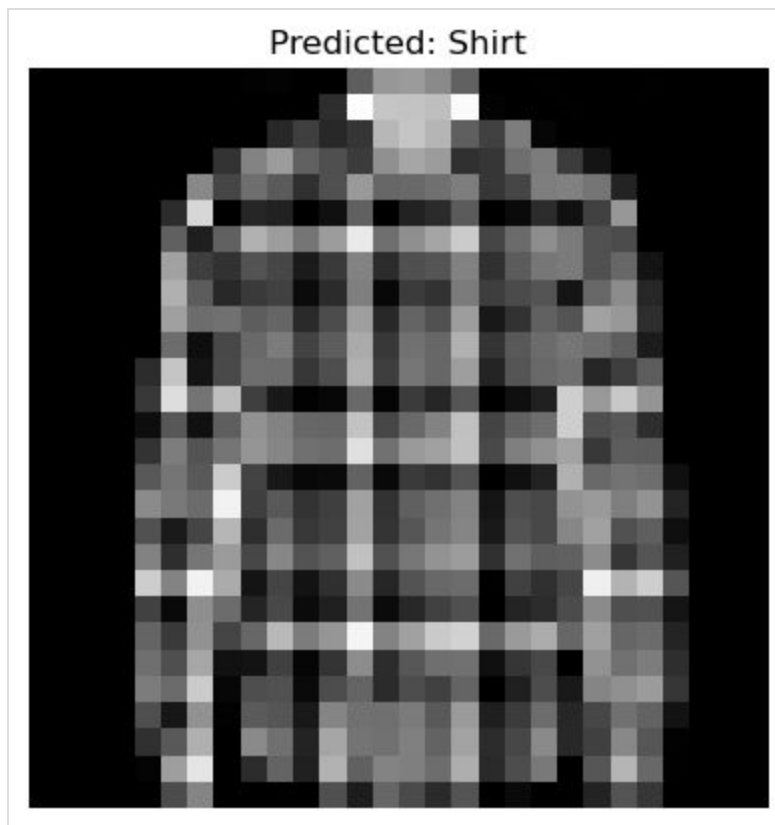
    Parameters:
        model (nn.Module): Trained model.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Predicted class name and confidence.
    """
    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0).to(DEVICE))
        probs = torch.softmax(logits, dim=1)
        confidence, predicted = probs.max(dim=1)
    return CLASS_NAMES[predicted.item()], confidence.item()
```

Run Inference

```
image, label = test_data[7]
name, confidence = predict(loader_model, image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
plt.imshow(image.squeeze(), cmap='gray')
plt.title(f"Predicted: {name}")
plt.axis('off')
plt.show()
```

```
True: Shirt | Predicted: Shirt (0.9477)
```



Part V: Sequences, Attention, and Language Models

17-RNN: Tiny Shakespeare Dataset - RNN

[Tiny Shakespeare Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x10e5b9ad0>
```

Parameters

```
SEQ_LEN = 64
SEQ_LEN
```

64

```
EMBED_DIM = 64  
EMBED_DIM
```

64

```
HIDDEN_DIM = 128  
HIDDEN_DIM
```

128

Hyperparameters

```
LEARNING_RATE = 0.002  
LEARNING_RATE
```

0.002

```
EPOCHS = 10  
EPOCHS
```

10

```
BATCH_SIZE = 128  
BATCH_SIZE
```

128

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Download Dataset

Tiny Shakespeare is a real text corpus of about one million characters. We download it once and train a character-level model to predict the next character.

```
URL = 'https://raw.githubusercontent.com/karpathy/char-  
rnn/master/data/tinyshakespeare/input.txt'  
urllib.request.urlretrieve(URL, 'tinyshakespeare.txt')  
with open('tinyshakespeare.txt', encoding='utf-8') as handle:  
    text = handle.read()  
print('length:', len(text))  
print(text[:200])
```

```
length: 1115394  
First Citizen:  
Before we proceed any further, hear me speak.  
  
All:  
Speak, speak.  
  
First Citizen:  
You are all resolved rather to die than to famish?  
  
All:  
Resolved. resolved.  
  
First Citizen:  
First, you
```

Build Vocabulary

Functions

```
def build_vocab(text):
    """
    Build character to index and index to character maps.

    Parameters:
        text (str): The training text.

    Returns:
        tuple: Character list, string to index, index to string.
    """
    chars = sorted(set(text))
    stoi = {ch: i for i, ch in enumerate(chars)}
    itos = {i: ch for i, ch in enumerate(chars)}
    return chars, stoi, itos


def encode(text, stoi):
    """
    Encode text as a tensor of indices.

    Parameters:
        text (str): The text to encode.
        stoi (dict): Character to index map.

    Returns:
        torch.Tensor: Encoded indices.
    """
    return torch.tensor([stoi[ch] for ch in text], dtype=torch.long)


def decode(indices, itos):
    """
    Decode indices back to text.

    Parameters:
        indices (list): Index values.
        itos (dict): Index to character map.

    Returns:
        str: The decoded text.
    """
    return ''.join(itos[i] for i in indices)
```

Run

```
CHARS, STOI, ITOS = build_vocab(text)
VOCAB_SIZE = len(CHARS)
data = encode(text, STOI)
print('vocab size:', VOCAB_SIZE)
print('data shape:', data.shape)
```



```
vocab size: 65
data shape: torch.Size([1115394])
```

Create Dataset

```
class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

    def __init__(self, data, seq_len):
        """
        Store the encoded text and window length.

        Parameters:
            data (torch.Tensor): Encoded text.
            seq_len (int): Length of each input window.

        Returns:
            None
        """
        self.data = data
        self.seq_len = seq_len

    def __len__(self):
        """
        Return the number of windows.

        Parameters:
            None

        Returns:
            int: Number of windows.
        """
        return len(self.data) - self.seq_len

    def __getitem__(self, idx):
        """
        Return one input window and its shifted target.

        Parameters:
            idx (int): Window index.

        Returns:
            tuple: Input and target tensors.
        """
        x = self.data[idx:idx + self.seq_len]
        y = self.data[idx + 1:idx + self.seq_len + 1]
        return x, y
```

Create DataLoader

```
split = int(0.9 * len(data))
train_data = CharDataset(data[:split], SEQ_LEN)
val_data = CharDataset(data[split:], SEQ_LEN)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)
```

```
(7843, 871)
```

Create Model

```
class RNNModel(nn.Module):
    """
    A character-level recurrent network with an LSTM.
    """

    def __init__(self, vocab_size, embed_dim, hidden_dim):
        """
        Initialize the embedding, LSTM, and output layer.

        Parameters:
            vocab_size (int): Number of characters.
            embed_dim (int): Embedding size.
            hidden_dim (int): LSTM hidden size.

        Returns:
            None
        """
        super(RNNModel, self).__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        """
        Run the forward pass over a batch of windows.

        Parameters:
            x (torch.Tensor): Input index batch.
            hidden (tuple): Optional LSTM hidden state.

        Returns:
            tuple: Logits and the new hidden state.
        """
        x = self.embed(x)
        out, hidden = self.lstm(x, hidden)
        return self.fc(out), hidden
```

Instantiate Model

```
torch.manual_seed(SEED)
model = RNNModel(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(DEVICE)
model
```

```
RNNModel(
  (embed): Embedding(65, 64)
  (lstm): LSTM(64, 128, batch_first=True)
  (fc): Linear(in_features=128, out_features=65, bias=True)
)
```

Create Loss Function & Optimizer

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        logits, _ = model(x)
        loss = loss_fn(logits.transpose(1, 2), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits, _ = model(x)
            total += loss_fn(logits.transpose(1, 2), y).item() * len(y)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss = evaluate(model, val_loader, loss_fn)
    history.append(val_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

Epoch 1 | train loss 1.4800 | val loss 1.6200

Epoch 2 | train loss 1.3361 | val loss 1.6320

Epoch 3 | train loss 1.3117 | val loss 1.6469

Epoch 4 | train loss 1.2994 | val loss 1.6530

Epoch 5 | train loss 1.2918 | val loss 1.6691

Epoch 6 | train loss 1.2865 | val loss 1.6595

Epoch 7 | train loss 1.2825 | val loss 1.6639

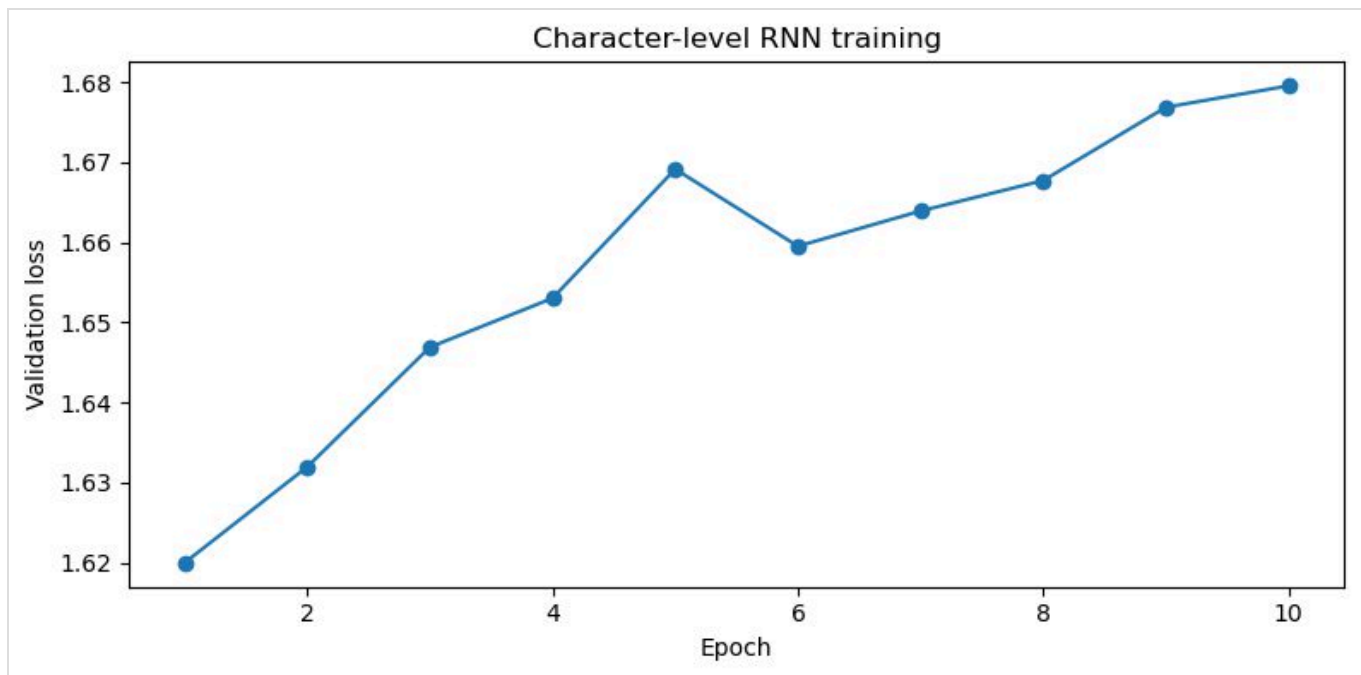
Epoch 8 | train loss 1.2796 | val loss 1.6677

Epoch 9 | train loss 1.2773 | val loss 1.6769

Epoch 10 | train loss 1.2754 | val loss 1.6795

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Character-level RNN training')
plt.tight_layout()
plt.show()
```



Generate Text

Function

```
def generate(model, start, length, temperature=0.8):
    """
    Generate text from a trained model.

    Parameters:
        model (nn.Module): Trained model.
        start (str): Seed text.
        length (int): Number of characters to generate.
        temperature (float): Sampling temperature.

    Returns:
        str: The generated text.
    """
    model.eval()
    indices = [STOI[ch] for ch in start]
    hidden = None
    with torch.no_grad():
        for _ in range(length):
            x = torch.tensor([indices[-1]]).unsqueeze(0).to(DEVICE)
            logits, hidden = model(x, hidden)
            probs = torch.softmax(logits[0, -1] / temperature, dim=0)
            next_index = torch.multinomial(probs, 1).item()
            indices.append(next_index)
    return decode(indices, ITOS)
```

Run

```
print(generate(model, 'ROME0: ', 300))
```

ROMEO: me your kingdom bad!

RIVERS:

Cloud hoteous unpriss.

VOLUMNIA:

I must have I am benefit no more
 Than man of shepherd's son's reasons,
 So far that will not say my soul is the ease.
 I was less in that which the gods my kingdom merry.

DUKE VINCENTIO:

Your grace is hence, sir.

MERCUTIO:

Then so stand

Save Model

```
torch.save(model.state_dict(), 'rnn_shakespeare.pt')
print('saved rnn_shakespeare.pt')
```

saved rnn_shakespeare.pt

Load Model

```
loaded_model = RNNModel(VOCAB_SIZE, EMBED_DIM, HIDDEN_DIM).to(DEVICE)
loaded_model.load_state_dict(torch.load('rnn_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded rnn_shakespeare.pt')
```

loaded rnn_shakespeare.pt

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55485/1614061309.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('rnn_shakespeare.pt', map_location=DEVICE))
```

Inference

```
print(generate(loader_model, 'JULIET: ', 200))
```

JULIET: soul small again.

MERCUTIO:

Why, marry, what the thumbly Camillo, now the next enduation,
My better died this paint; for I hope he doomser mock'd their!

DUKE VINCENTIO:

Though not the just and membr

18-ATT: Tiny Shakespeare Dataset - Attention

[Tiny Shakespeare Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>

[2] Ashish Vaswani et al., <https://arxiv.org/abs/1706.03762>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import math
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x1115d5ad0>
```

Parameters

SEQ_LEN = 64
SEQ_LEN

64

EMBED_DIM = 64
EMBED_DIM

64

N_HEADS = 4
N_HEADS

4

FF_DIM = 128
FF_DIM

128

Hyperparameters

LEARNING_RATE = 0.002
LEARNING_RATE

0.002

EPOCHS = 10
EPOCHS

10

BATCH_SIZE = 128
BATCH_SIZE

128

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt'
urllib.request.urlretrieve(URL, 'tinyshakespeare.txt')
with open('tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
```

```
length: 1115394
```

Build Vocabulary

```
CHARS = sorted(set(text))
STOI = {ch: i for i, ch in enumerate(CHARS)}
ITOS = {i: ch for i, ch in enumerate(CHARS)}
VOCAB_SIZE = len(CHARS)
data = torch.tensor([STOI[ch] for ch in text], dtype=torch.long)
print('vocab size:', VOCAB_SIZE)
```

```
vocab size: 65
```

Attention From Scratch

Attention lets every position look at every other position. The scores are the dot products of queries and keys, scaled by the square root of the key dimension, then turned into weights with softmax.

Scaled Dot-Product Attention

```
def scaled_dot_product_attention(query, key, value, mask=None):
    """
    Compute scaled dot-product attention.

    Parameters:
        query (torch.Tensor): Query tensor.
        key (torch.Tensor): Key tensor.
        value (torch.Tensor): Value tensor.
        mask (torch.Tensor): Optional mask of allowed positions.

    Returns:
        tuple: Attention output and attention weights.
    """
    d_k = query.size(-1)
    scores = query @ key.transpose(-2, -1) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = torch.softmax(scores, dim=-1)
    return weights @ value, weights
```

Try It on a Small Sequence

```
torch.manual_seed(SEED)
tokens = torch.randn(5, 8)
out, weights = scaled_dot_product_attention(tokens, tokens, tokens)
print('output shape:', out.shape)
print('weight row sums:', weights.sum(dim=-1))
```

```
output shape: torch.Size([5, 8])
weight row sums: tensor([1.0000, 1.0000, 1.0000, 1.0000, 1.0000])
```

Causal Mask

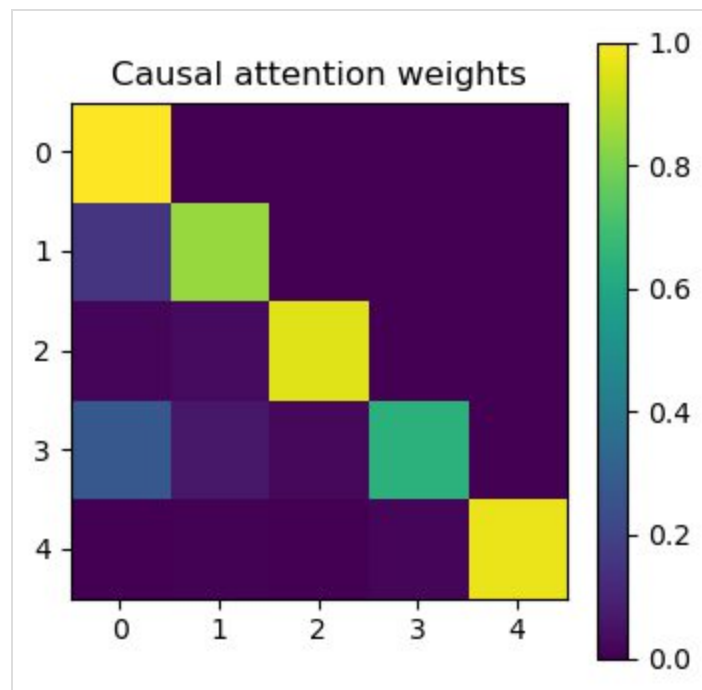
A language model may not look at future characters. A causal mask blocks every position from attending to later positions.

```
def causal_mask(size):
    """
    Build a lower-triangular causal mask.

    Parameters:
        size (int): Sequence length.

    Returns:
        torch.Tensor: A size x size mask.
    """
    return torch.tril(torch.ones(size, size)).bool()
```

```
torch.manual_seed(SEED)
tokens = torch.randn(5, 8)
mask = causal_mask(5)
out, weights = scaled_dot_product_attention(tokens, tokens, tokens, mask)
plt.figure(figsize=(4, 4))
plt.imshow(weights.detach(), cmap='viridis')
plt.title('Causal attention weights')
plt.colorbar()
plt.show()
```



Multi-Head Attention

```

class MultiHeadAttention(nn.Module):
    """
    Multi-head self-attention with a causal mask.
    """

    def __init__(self, embed_dim, n_heads):
        """
        Initialize the query, key, value, and output projections.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.

        Returns:
            None
        """
        super(MultiHeadAttention, self).__init__()
        self.n_heads = n_heads
        self.head_dim = embed_dim // n_heads
        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.proj = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        Run multi-head causal self-attention.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        batch, time, embed_dim = x.shape
        qkv = self.qkv(x).reshape(batch, time, 3, self.n_heads, self.head_dim)
        query, key, value = qkv.permute(2, 0, 3, 1, 4)
        out = F.scaled_dot_product_attention(query, key, value, is_causal=True)
        out = out.transpose(1, 2).reshape(batch, time, embed_dim)
        return self.proj(out)

```

Transformer Block

```
class TransformerBlock(nn.Module):
    """
    A single transformer block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim, n_heads, ff_dim):
        """
        Initialize attention, normalization, and feed-forward layers.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(TransformerBlock, self).__init__()
        self.attn = MultiHeadAttention(embed_dim, n_heads)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.GELU(),
            nn.Linear(ff_dim, embed_dim))
        self.norm2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        """
        Run the block with residual connections.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Output of the same shape.
        """
        x = self.norm1(x + self.attn(x))
        return self.norm2(x + self.ff(x))
```

Create Model

```

class AttentionModel(nn.Module):
    """
    A character language model with one transformer block.
    """

    def __init__(self, vocab_size, embed_dim, n_heads, ff_dim, max_len):
        """
        Initialize embeddings, the transformer block, and the output head.

        Parameters:
            vocab_size (int): Number of characters.
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            ff_dim (int): Feed-forward hidden size.
            max_len (int): Maximum sequence length.

        Returns:
            None
        """
        super(AttentionModel, self).__init__()
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        self.pos_embed = nn.Embedding(max_len, embed_dim)
        self.block = TransformerBlock(embed_dim, n_heads, ff_dim)
        self.head = nn.Linear(embed_dim, vocab_size)

    def forward(self, x):
        """
        Run the forward pass of the model.

        Parameters:
            x (torch.Tensor): Input index batch.

        Returns:
            torch.Tensor: Output logits.
        """
        positions = torch.arange(x.size(1), device=x.device)
        x = self.token_embed(x) + self.pos_embed(positions)
        x = self.block(x)
        return self.head(x)

```


Create Dataset and DataLoader

```
class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

    def __init__(self, data, seq_len):
        """
        Store the encoded text and window length.

        Parameters:
            data (torch.Tensor): Encoded text.
            seq_len (int): Length of each input window.

        Returns:
            None
        """
        self.data = data
        self.seq_len = seq_len

    def __len__(self):
        """
        Return the number of windows.

        Parameters:
            None

        Returns:
            int: Number of windows.
        """
        return len(self.data) - self.seq_len

    def __getitem__(self, idx):
        """
        Return one input window and its shifted target.

        Parameters:
            idx (int): Window index.

        Returns:
            tuple: Input and target tensors.
        """
        x = self.data[idx:idx + self.seq_len]
        y = self.data[idx + 1:idx + self.seq_len + 1]
        return x, y
```

```
split = int(0.9 * len(data))
train_data = CharDataset(data[:split], SEQ_LEN)
val_data = CharDataset(data[split:], SEQ_LEN)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)
```

```
(7843, 871)
```

Instantiate Model, Loss, and Optimizer

```
torch.manual_seed(SEED)
model = AttentionModel(VOCAB_SIZE, EMBED_DIM, N_HEADS, FF_DIM, SEQ_LEN).to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model
```

```
AttentionModel(
  (token_embed): Embedding(65, 64)
  (pos_embed): Embedding(64, 64)
  (block): TransformerBlock(
    (attn): MultiHeadAttention(
      (qkv): Linear(in_features=64, out_features=192, bias=True)
      (proj): Linear(in_features=64, out_features=64, bias=True)
    )
    (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
    (ff): Sequential(
      (0): Linear(in_features=64, out_features=128, bias=True)
      (1): GELU(approximate='none')
      (2): Linear(in_features=128, out_features=64, bias=True)
    )
    (norm2): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
  )
  (head): Linear(in_features=64, out_features=65, bias=True)
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        logits = model(x)
        loss = loss_fn(logits.transpose(1, 2), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits.transpose(1, 2), y).item() * len(y)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss = evaluate(model, val_loader, loss_fn)
    history.append(val_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

Epoch 1 | train loss 1.7006 | val loss 1.7969

Epoch 2 | train loss 1.5684 | val loss 1.7975

Epoch 3 | train loss 1.5497 | val loss 1.7955

Epoch 4 | train loss 1.5397 | val loss 1.7936

Epoch 5 | train loss 1.5331 | val loss 1.7960

Epoch 6 | train loss 1.5283 | val loss 1.7913

Epoch 7 | train loss 1.5247 | val loss 1.7884

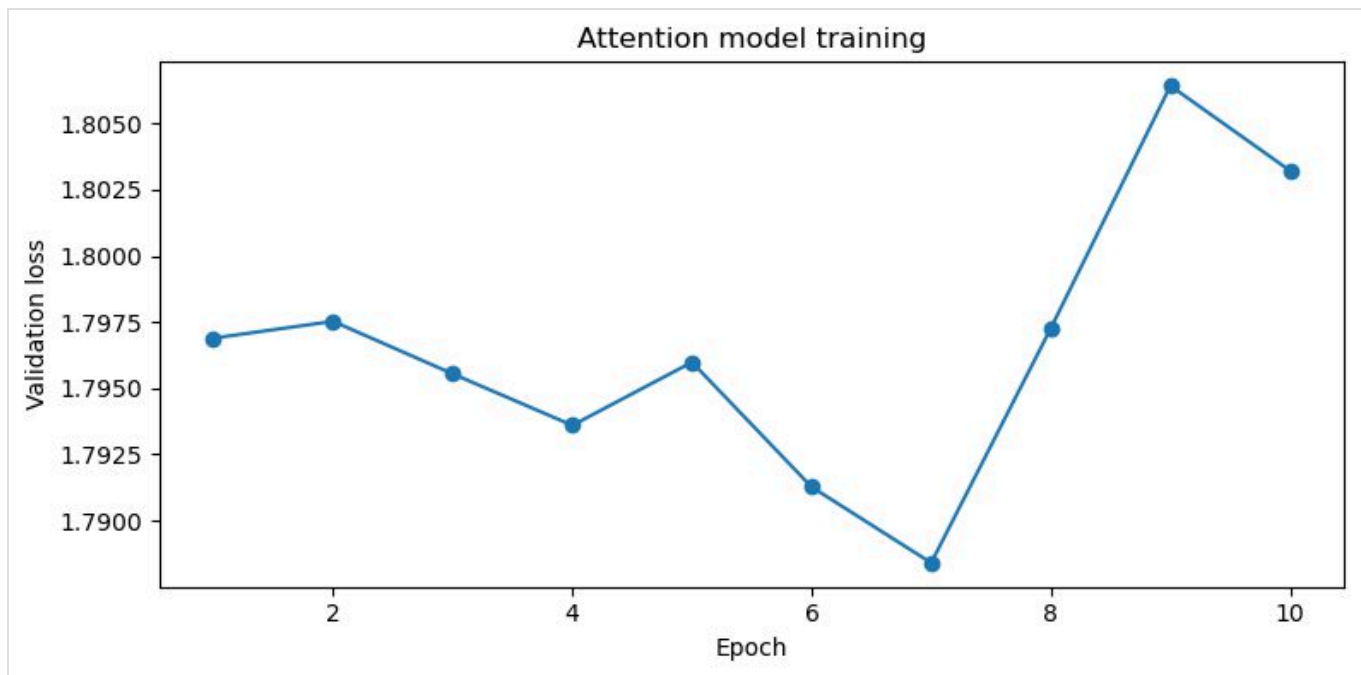
Epoch 8 | train loss 1.5219 | val loss 1.7973

Epoch 9 | train loss 1.5195 | val loss 1.8064

Epoch 10 | train loss 1.5176 | val loss 1.8032

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Attention model training')
plt.tight_layout()
plt.show()
```



Generate Text

```
def generate(model, start, length, temperature=0.8):
    """
    Generate text from a trained model.

    Parameters:
        model (nn.Module): Trained model.
        start (str): Seed text.
        length (int): Number of characters to generate.
        temperature (float): Sampling temperature.

    Returns:
        str: The generated text.
    """
    model.eval()
    indices = [STOI[ch] for ch in start]
    with torch.no_grad():
        for _ in range(length):
            window = torch.tensor(indices[-SEQ_LEN:]).unsqueeze(0).to(DEVICE)
            logits = model(window)
            probs = torch.softmax(logits[0, -1] / temperature, dim=0)
            indices.append(torch.multinomial(probs, 1).item())
    return ''.join(ITOS[i] for i in indices)
```

```
print(generate(model, 'ROMEO: ', 300))
```

```

ROMEO: wrong
That I darest, that I will make to the some see their known,
And highness uncle is not and strousands that is to pitted and him thousandly fear: I'll
shall crate the said not lead here be
He against heart as there peace merrance to remine hand;
For I consion like a should secree carmiss it say

```

Save Model

```

torch.save(model.state_dict(), 'attention_shakespeare.pt')
print('saved attention_shakespeare.pt')

```

```

saved attention_shakespeare.pt

```

Load Model

```

loaded_model = AttentionModel(VOCAB_SIZE, EMBED_DIM, N_HEADS, FF_DIM, SEQ_LEN).to(DEVICE)
loaded_model.load_state_dict(torch.load('attention_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded attention_shakespeare.pt')

```

```

loaded attention_shakespeare.pt

```

```

/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55839/4062516272.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('attention_shakespeare.pt', map_location=DEVICE))

```

Inference

```

print(generate(loaded_model, 'JULIET: ', 200))

```

JULIET: to all leave worst that the be mother prove die the part
Titus so the for to
Than dead by my day, ere I am ded ha death?

LADY CAPULET:
But master than make in in art nothings.

LADY GREY:
Mortieme! W

19-GPT: Tiny Shakespeare Dataset - TinyGPT From Scratch

[Tiny Shakespeare Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/nanoGPT>

[2] Ashish Vaswani et al., <https://arxiv.org/abs/1706.03762>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x10823db50>
```


Parameters

BLOCK_SIZE = 64
BLOCK_SIZE

64

EMBED_DIM = 64
EMBED_DIM

64

N_HEADS = 4
N_HEADS

4

N_LAYERS = 2
N_LAYERS

2

FF_DIM = 128
FF_DIM

128

Hyperparameters

LEARNING_RATE = 0.002
LEARNING_RATE

0.002

EPOCHS = 10
EPOCHS

10

```
BATCH_SIZE = 128
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt'
urllib.request.urlretrieve(URL, 'tinyshakespeare.txt')
with open('tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
```

```
length: 1115394
```

Build Vocabulary

```
CHARS = sorted(set(text))
STOI = {ch: i for i, ch in enumerate(CHARS)}
ITOS = {i: ch for i, ch in enumerate(CHARS)}
VOCAB_SIZE = len(CHARS)
data = torch.tensor([STOI[ch] for ch in text], dtype=torch.long)
print('vocab size:', VOCAB_SIZE)
```

```
vocab size: 65
```

Create Dataset and DataLoader

```
class CharDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length character windows and next-character targets.
    """

    def __init__(self, data, block_size):
        """
        Store the encoded text and window length.

        Parameters:
            data (torch.Tensor): Encoded text.
            block_size (int): Length of each input window.

        Returns:
            None
        """
        self.data = data
        self.block_size = block_size

    def __len__(self):
        """
        Return the number of windows.

        Parameters:
            None

        Returns:
            int: Number of windows.
        """
        return len(self.data) - self.block_size

    def __getitem__(self, idx):
        """
        Return one input window and its shifted target.

        Parameters:
            idx (int): Window index.

        Returns:
            tuple: Input and target tensors.
        """
        x = self.data[idx:idx + self.block_size]
        y = self.data[idx + 1:idx + self.block_size + 1]
        return x, y
```

```
split = int(0.9 * len(data))
train_data = CharDataset(data[:split], BLOCK_SIZE)
val_data = CharDataset(data[split:], BLOCK_SIZE)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)
```

(7843, 871)

Attention Head

```

class Head(nn.Module):
    """
    A single head of causal self-attention.
    """

    def __init__(self, embed_dim, head_dim):
        """
        Initialize the key, query, and value projections.

        Parameters:
            embed_dim (int): Input embedding size.
            head_dim (int): Size of this head.

        Returns:
            None
        """
        super(Head, self).__init__()
        self.key = nn.Linear(embed_dim, head_dim, bias=False)
        self.query = nn.Linear(embed_dim, head_dim, bias=False)
        self.value = nn.Linear(embed_dim, head_dim, bias=False)

    def forward(self, x):
        """
        Apply causal self-attention to the input.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        key = self.key(x).unsqueeze(1)
        query = self.query(x).unsqueeze(1)
        value = self.value(x).unsqueeze(1)
        out = F.scaled_dot_product_attention(query, key, value, is_causal=True)
        return out.squeeze(1)

```

Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    """
    Several attention heads running in parallel.
    """

    def __init__(self, embed_dim, n_heads, block_size):
        """
        Initialize the heads and the output projection.

        Parameters:
            embed_dim (int): Input embedding size.
            n_heads (int): Number of heads.
            block_size (int): Maximum sequence length.

        Returns:
            None
        """
        super(MultiHeadAttention, self).__init__()
        head_dim = embed_dim // n_heads
        self.heads = nn.ModuleList(
            [Head(embed_dim, head_dim) for _ in range(n_heads)])
        self.proj = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        Concatenate the heads and project the result.

        Parameters:
            x (torch.Tensor): Input of shape (batch, time, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        out = torch.cat([head(x) for head in self.heads], dim=-1)
        return self.proj(out)
```

Feed-Forward Network

```
class FeedForward(nn.Module):
    """
    A position-wise feed-forward network.
    """

    def __init__(self, embed_dim, ff_dim):
        """
        Initialize the two linear layers and activation.

        Parameters:
            embed_dim (int): Input embedding size.
            ff_dim (int): Hidden size.

        Returns:
            None
        """
        super(FeedForward, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.ReLU(),
            nn.Linear(ff_dim, embed_dim))

    def forward(self, x):
        """
        Run the feed-forward network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output tensor.
        """
        return self.net(x)
```

Transformer Block

```
class Block(nn.Module):
    """
    A transformer block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim, n_heads, block_size, ff_dim):
        """
        Initialize attention, feed-forward, and normalization layers.

        Parameters:
            embed_dim (int): Input embedding size.
            n_heads (int): Number of attention heads.
            block_size (int): Maximum sequence length.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(Block, self).__init__()
        self.attn = MultiHeadAttention(embed_dim, n_heads, block_size)
        self.ffwd = FeedForward(embed_dim, ff_dim)
        self.ln1 = nn.LayerNorm(embed_dim)
        self.ln2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        """
        Run the block with residual connections.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output tensor.
        """
        x = x + self.attn(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x
```

Create TinyGPT

```
class TinyGPT(nn.Module):
    """
    A small decoder-only transformer language model.
    """

    def __init__(self, vocab_size, embed_dim, n_heads, n_layers,
                  block_size, ff_dim):
        """
        Initialize embeddings, blocks, and the output head.

        Parameters:
            vocab_size (int): Number of characters.
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            n_layers (int): Number of transformer blocks.
            block_size (int): Maximum sequence length.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(TinyGPT, self).__init__()
        self.block_size = block_size
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        self.pos_embed = nn.Embedding(block_size, embed_dim)
        self.blocks = nn.Sequential(*[
            Block(embed_dim, n_heads, block_size, ff_dim) for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, vocab_size)

    def embed(self, idx):
        """
        Combine token and positional embeddings.

        Parameters:
            idx (torch.Tensor): Input indices.

        Returns:
            torch.Tensor: Embedded input.
        """
        _, time = idx.shape
        positions = torch.arange(time, device=idx.device)
        return self.token_embed(idx) + self.pos_embed(positions)

    def forward(self, idx, targets=None):
        """
        Run the forward pass and optional loss.

        Parameters:
            idx (torch.Tensor): Input indices.
            targets (torch.Tensor): Optional target indices.

        Returns:
            tuple: Logits and optional loss.
        """
```



```

    """
    x = self.blocks(self.embed(idx))
    logits = self.head(self.ln_f(x))
    if targets is None:
        return logits, None
    loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
    return logits, loss

def generate(self, idx, max_new_tokens):
    """
    Generate new tokens by sampling from the model.

    Parameters:
        idx (torch.Tensor): Seed indices.
        max_new_tokens (int): Number of tokens to generate.

    Returns:
        torch.Tensor: The extended indices.
    """
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -self.block_size:]
        logits, _ = self(idx_cond)
        probs = F.softmax(logits[:, -1, :], dim=-1)
        idx_next = torch.multinomial(probs, 1)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx

```

Instantiate Model and Optimizer

```

torch.manual_seed(SEED)
model = TinyGPT(VOCAB_SIZE, EMBED_DIM, N_HEADS, N_LAYERS, BLOCK_SIZE, FF_DIM).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())

```

79169

Train Model

Functions

```
def train_epoch(model, loader, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        _, loss = model(x, y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def estimate_loss(model, loader):
    """
    Estimate the mean loss over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            _, loss = model(x, y)
            total += loss.item() * len(y)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, optimizer)
    val_loss = estimate_loss(model, val_loader)
    history.append(val_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

Epoch 1 | train loss 1.5823 | val loss 1.6780

Epoch 2 | train loss 1.4257 | val loss 1.6574

Epoch 3 | train loss 1.4023 | val loss 1.6535

Epoch 4 | train loss 1.3897 | val loss 1.6526

Epoch 5 | train loss 1.3815 | val loss 1.6451

Epoch 6 | train loss 1.3756 | val loss 1.6348

Epoch 7 | train loss 1.3711 | val loss 1.6365

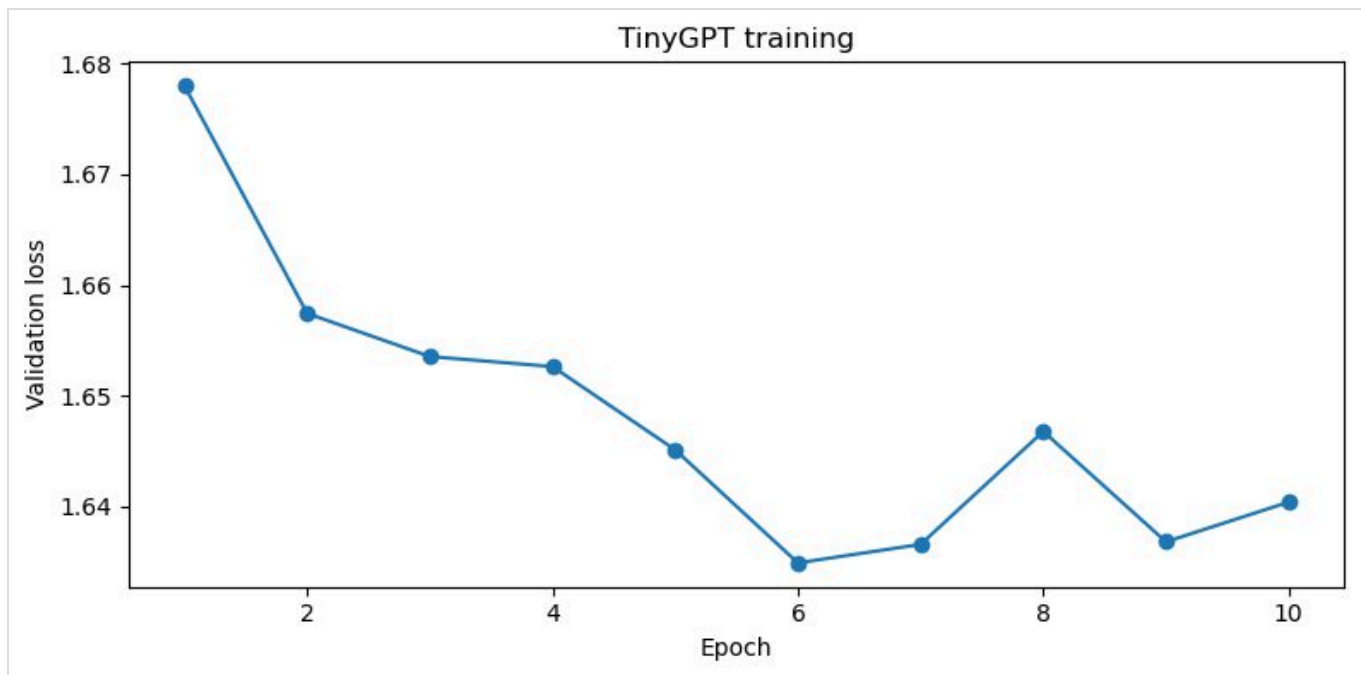
Epoch 8 | train loss 1.3677 | val loss 1.6467

Epoch 9 | train loss 1.3651 | val loss 1.6368

Epoch 10 | train loss 1.3629 | val loss 1.6403

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('TinyGPT training')
plt.tight_layout()
plt.show()
```



Generate Text

```
def generate_text(model, start, length, temperature=0.8):  
    """  
    Generate readable text from a seed string.  
  
    Parameters:  
        model (nn.Module): Trained model.  
        start (str): Seed text.  
        length (int): Number of characters to generate.  
        temperature (float): Sampling temperature.  
  
    Returns:  
        str: The generated text.  
    """  
    model.eval()  
    idx = torch.tensor([STOI[ch] for ch in start], device=DEVICE)  
    with torch.no_grad():  
        out = model.generate(idx, length)  
    return ''.join(ITOS[i] for i in out[0].tolist())
```

```
print(generate_text(model, 'ROMEO: ', 400))
```

ROME0: -Nay,
I know you, and custom calumnilling, and move
To hear it in me unhallowed unplease and Fress'd.

DUCHESS OF YORK:
What he is interprate lops.

KING RICHARD III:
ANF I am the eleven likening, and have
that my thousands: there
Is farst. Yet charge by mine own hences state
To-days with testimation.

MAMILLIUS:
You friends are monouchol't divine that I
sitten forego; sitt
That diver beggar, as I

Save Model

```
torch.save(model.state_dict(), 'tinygpt_shakespeare.pt')
print('saved tinygpt_shakespeare.pt')
```

```
saved tinygpt_shakespeare.pt
```

Load Model

```
loaded_model = TinyGPT(VOCAB_SIZE, EMBED_DIM, N_HEADS, N_LAYERS, BLOCK_SIZE,
FF_DIM).to(DEVICE)
loaded_model.load_state_dict(torch.load('tinygpt_shakespeare.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded tinygpt_shakespeare.pt')
```

```
loaded tinygpt_shakespeare.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_56325/1844433482.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('tinygpt_shakespeare.pt', map_location=DEVICE))
```

Inference

```
print(generate_text(loader_model, 'JULIET: ', 400))
```

JULIET: Say I can, I think thy sanctify,
And creates but hath but in imparlent himour
And with thou art made to sound keep liking of
shorn, she hear itself.

CLIFFORD:
I so tese my lord, England?

MENENIUS:
Tut, and leaden; much brain'd for than you
chamber?
What, snow, stribute meant and conscince at the artinace
More challen living, bear you.

ISABELLA:
And unknow
For portents by accule your very fortuna

Part VI: Regularization, Evaluation, and Generative Vision

20-REG: Breast Cancer Dataset - Regularization

Breast Cancer Dataset

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Wolberg, Street, and Mangasarian,

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x124d49af0>
```


Parameters

IN_FEATURES = 30
IN_FEATURES

30

H1 = 32
H1

32

H2 = 16
H2

16

OUT_FEATURES = 2
OUT_FEATURES

2

TEST_SIZE = 0.3
TEST_SIZE

0.3

Hyperparameters

LEARNING_RATE = 0.001
LEARNING_RATE

0.001

EPOCHS = 300
EPOCHS

300

```
BATCH_SIZE = 32  
BATCH_SIZE
```

```
32
```

```
DROPOUT = 0.3  
DROPOUT
```

```
0.3
```

```
WEIGHT_DECAY = 0.01  
WEIGHT_DECAY
```

```
0.01
```

```
PATIENCE = 20  
PATIENCE
```

```
20
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
data = load_breast_cancer()  
X = data.data.astype(np.float32)  
y = data.target.astype(np.int64)  
X.shape, y.shape
```

```
((569, 30), (569,))
```

Clean Dataset

Standardize the Features

```
X_MEAN = X.mean(axis=0)
X_STD = X.std(axis=0)
X = (X - X_MEAN) / X_STD
X.shape
```

```
(569, 30)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)
X_train.shape, X_test.shape
```

```
((398, 30), (171, 30))
```

Create Dataset and DataLoaders

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        return X_tensor, y_tensor
```

```
train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(NumpyDataset(X_test, y_test, DEVICE),
                        batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

`(13, 6)`

Create Model

Dropout randomly zeroes hidden units during training, which forces the network to avoid relying on any single unit.

```

class Model(nn.Module):
    """
    A regularized feedforward neural network with dropout.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, h2=H2,
                  out_features=OUT_FEATURES, dropout=DROPOUT):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.
            dropout (float): Dropout rate.

        Returns:
            None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(h1, h2),
            nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(h2, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)

```

Instantiate Model, Loss, and Optimizer

AdamW applies decoupled weight decay, which shrinks large weights and reduces overfitting.

```
torch.manual_seed(SEED)
model = Model().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE,
                               weight_decay=WEIGHT_DECAY)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
model
```

```
Model(
  (net): Sequential(
    (0): Linear(in_features=30, out_features=32, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.3, inplace=False)
    (3): Linear(in_features=32, out_features=16, bias=True)
    (4): ReLU()
    (5): Dropout(p=0.3, inplace=False)
    (6): Linear(in_features=16, out_features=2, bias=True)
  )
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Early Stopping

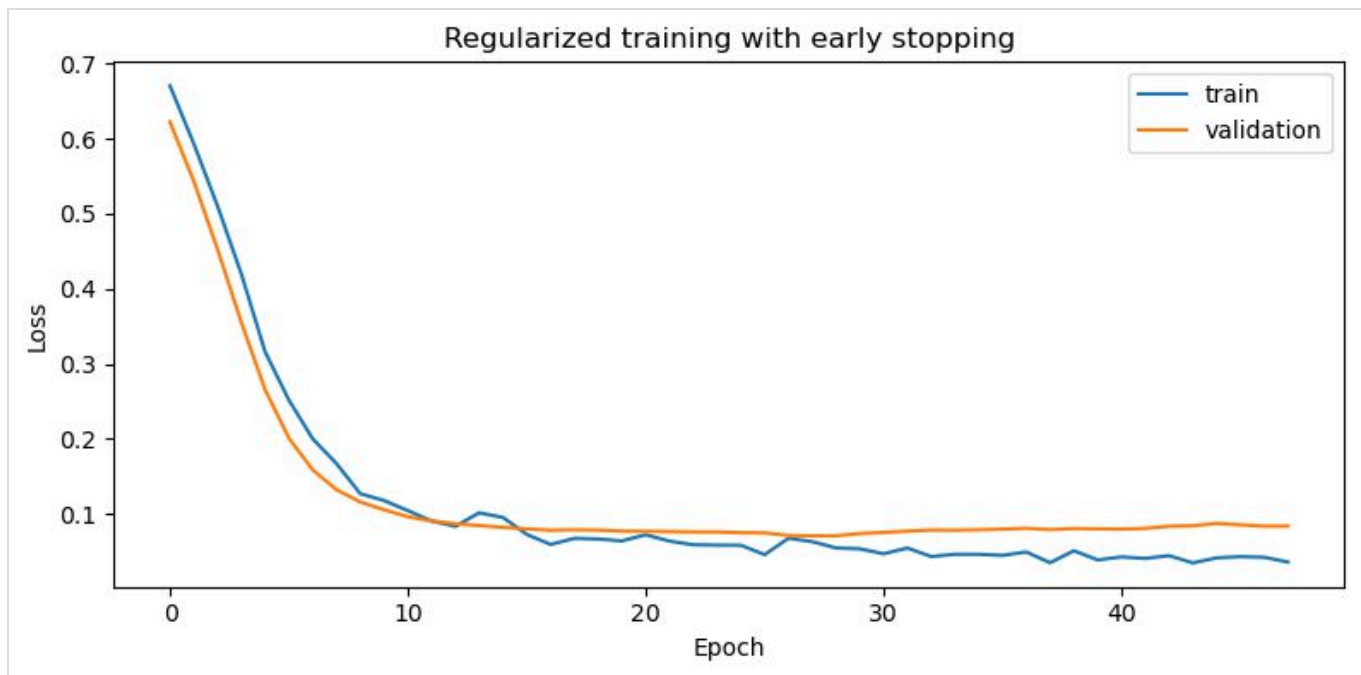
Early stopping halts training when validation loss stops improving. The best weights are restored at the end.

```
best_loss = float('inf')
best_state = None
wait = 0
train_losses = []
val_losses = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    val_loss, val_accuracy = evaluate(model, test_loader, loss_fn)
    scheduler.step()
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    if val_loss < best_loss:
        best_loss = val_loss
        best_state = {k: v.clone() for k, v in model.state_dict().items()}
        wait = 0
    else:
        wait += 1
        if wait >= PATIENCE:
            print(f"Early stop at epoch {epoch + 1}")
            break
model.load_state_dict(best_state)
print(f"Best validation loss: {best_loss:.4f}")
```

```
Early stop at epoch 48
Best validation loss: 0.0710
```

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(train_losses, label='train')
plt.plot(val_losses, label='validation')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Regularized training with early stopping')
plt.legend()
plt.tight_layout()
plt.show()
```

Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.9649

Save Model

```
torch.save(model.state_dict(), 'reg_breast_cancer.pt')
print('saved reg_breast_cancer.pt')
```

saved reg_breast_cancer.pt

Load Model

```
loaded_model = Model().to(DEVICE)
loaded_model.load_state_dict(torch.load('reg_breast_cancer.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded reg_breast_cancer.pt')
```

loaded reg_breast_cancer.pt

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55256/640927886.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('reg_breast_cancer.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):
    """
    Predict benign or malignant for one raw feature vector.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values.

    Returns:
        tuple: Predicted label name and confidence.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probs = torch.softmax(model(X_new), dim=1)
        confidence, predicted = probs.max(dim=1)
        names = ['malignant', 'benign']
        return names[predicted.item()], confidence.item()
```

Run Inference

```
label, confidence = predict(loaded_model, X_test[0])
print(f"Predicted: {label} ({confidence:.4f}) | Actual: {data.target_names[y_test[0]]}")
```

```
Predicted: benign (1.0000) | Actual: malignant
```

21-EVAL: Breast Cancer Dataset - Model Evaluation

Breast Cancer Dataset

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Wolberg, Street, and Mangasarian,

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import precision_recall_curve
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x16a390210>
```

Parameters

```
IN_FEATURES = 30  
IN_FEATURES
```

```
30
```

```
H1 = 32  
H1
```

```
32
```

```
OUT_FEATURES = 2  
OUT_FEATURES
```

```
2
```

```
TEST_SIZE = 0.3  
TEST_SIZE
```

```
0.3
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 200  
EPOCHS
```

```
200
```

```
BATCH_SIZE = 32  
BATCH_SIZE
```

```
32
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
data = load_breast_cancer()  
X = data.data.astype(np.float32)  
y = data.target.astype(np.int64)  
X_MEAN = X.mean(axis=0)  
X_STD = X.std(axis=0)  
X = (X - X_MEAN) / X_STD  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)  
X_train.shape, X_test.shape
```

```
((398, 30), (171, 30))
```

Create Dataset and DataLoaders

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        return X_tensor, y_tensor
```

```
train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(NumpyDataset(X_test, y_test, DEVICE),
                        batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(test_loader)
```

(13, 6)

Create Model

```
class Model(nn.Module):
    """
    A small feedforward classifier for tabular data.
    """

    def __init__(self, in_features=IN_FEATURES, h1=H1, out_features=OUT_FEATURES):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the hidden layer.
            out_features (int): Number of output features.

        Returns:
            None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Linear(h1, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)
```

Instantiate Model, Loss, and Optimizer

```
torch.manual_seed(SEED)
model = Model().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

Train Model

```
def train_epoch(model, loader, loss_fn, optimizer):  
    """  
    Train the model for a single epoch.  
  
    Parameters:  
        model (nn.Module): The model to train.  
        loader (DataLoader): Training data loader.  
        loss_fn (nn.Module): Loss function.  
        optimizer (torch.optim.Optimizer): Parameter update rule.  
  
    Returns:  
        float: Mean training loss.  
    """  
    model.train()  
    total = 0.0  
    for x, y in loader:  
        optimizer.zero_grad()  
        loss = loss_fn(model(x), y)  
        loss.backward()  
        optimizer.step()  
        total += loss.item() * len(y)  
    return total / len(loader.dataset)  
  
for epoch in range(EPOCHS):  
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)  
    if (epoch + 1) % 20 == 0:  
        print(f"Epoch {epoch + 1:3d} | train loss {train_loss:.4f}")
```

Epoch 20 | train loss 0.0673

Epoch 40 | train loss 0.0436

Epoch 60 | train loss 0.0308

Epoch 80 | train loss 0.0215

Epoch 100 | train loss 0.0151

Epoch 120 | train loss 0.0112

Epoch 140 | train loss 0.0084

Epoch 160 | train loss 0.0066

Epoch 180 | train loss 0.0053

Epoch 200 | train loss 0.0044

Collect Predictions and Probabilities

```
model.eval()
all_probs = []
all_labels = []
with torch.no_grad():
    for x, y in test_loader:
        probs = torch.softmax(model(x), dim=1)[ :, 1]
        all_probs.append(probs.cpu().numpy())
        all_labels.append(y.cpu().numpy())
y_prob = np.concatenate(all_probs)
y_true = np.concatenate(all_labels)
y_pred = (y_prob >= 0.5).astype(int)
print('collected', y_true.shape)
```

collected (171,)

Classification Metrics

Functions

```
def metric_table(y_true, y_pred):
    """
    Build a table of core classification metrics.

    Parameters:
        y_true (np.ndarray): True labels.
        y_pred (np.ndarray): Predicted labels.

    Returns:
        pandas.DataFrame: A single-row metrics table.
    """
    row = {'accuracy': accuracy_score(y_true, y_pred),
          'precision': precision_score(y_true, y_pred),
          'recall': recall_score(y_true, y_pred),
          'f1': f1_score(y_true, y_pred),
          'roc_auc': roc_auc_score(y_true, y_pred)}
    return pd.DataFrame(row, index=[0])
```

Run

```
metric_table(y_true, y_pred)
```

	accuracy	precision	recall	f1	roc_auc
0	0.97076	0.981132	0.971963	0.976526	0.970356

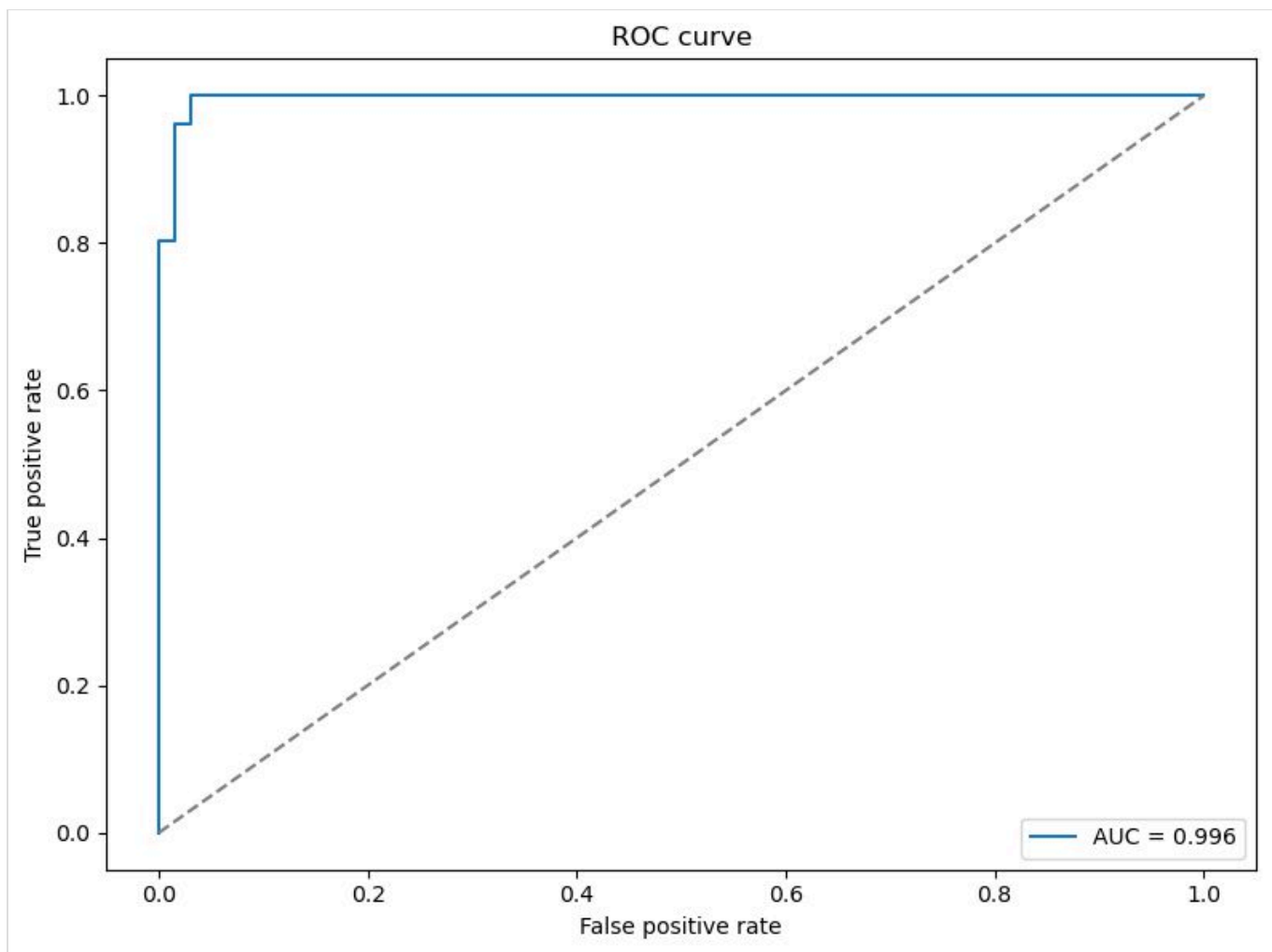
```
print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred, target_names=list(data.target_names)))
```

```
[[ 62   2]
 [  3 104]]
```

	precision	recall	f1-score	support
malignant	0.95	0.97	0.96	64
benign	0.98	0.97	0.98	107
accuracy			0.97	171
macro avg	0.97	0.97	0.97	171
weighted avg	0.97	0.97	0.97	171

ROC Curve and AUC

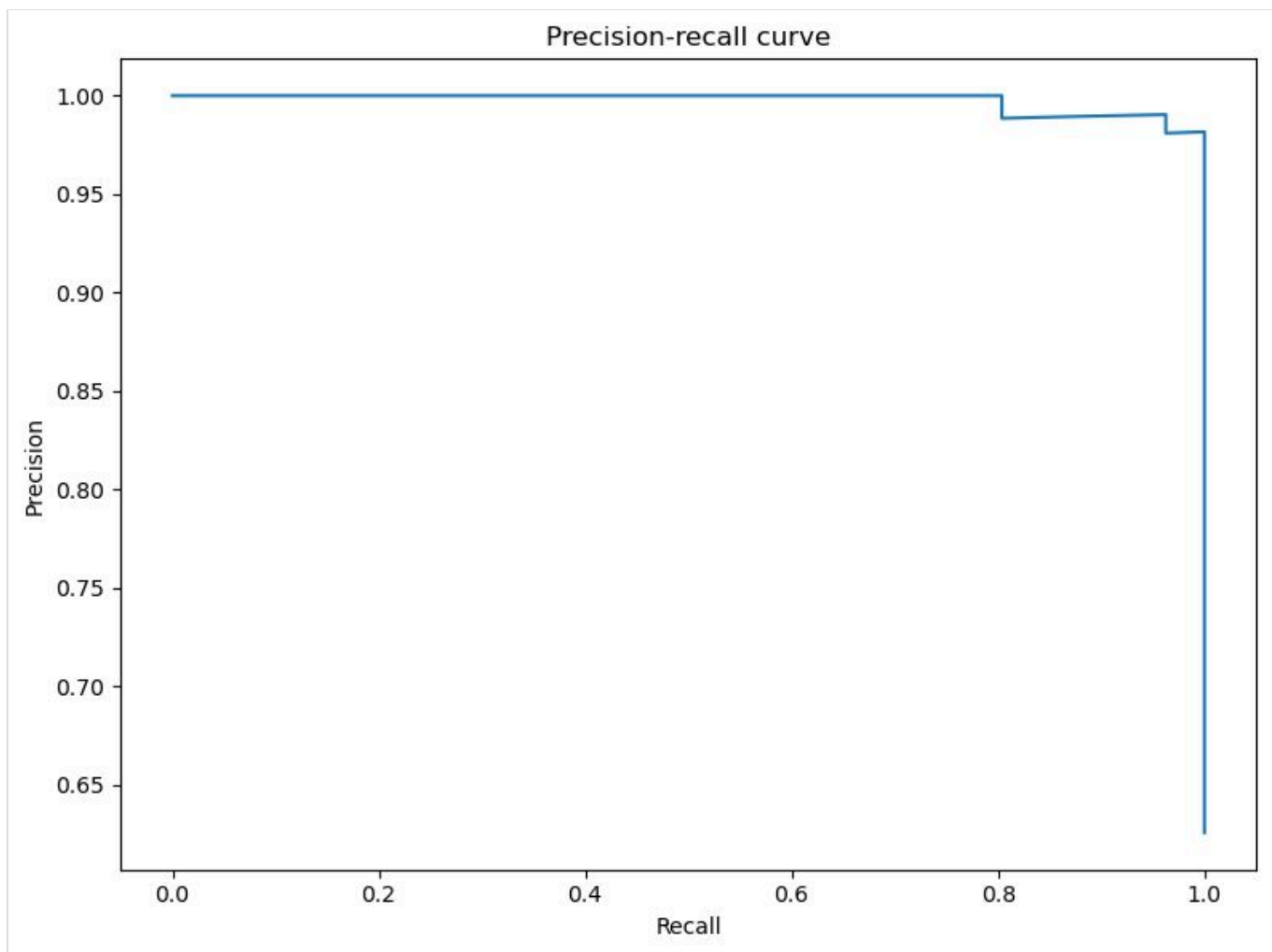
```
fpr, tpr, _ = roc_curve(y_true, y_prob)
auc_score = roc_auc_score(y_true, y_prob)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.3f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve')
plt.legend()
plt.tight_layout()
plt.show()
```



Precision-Recall Curve

The precision-recall curve is more informative than ROC when the classes are imbalanced.

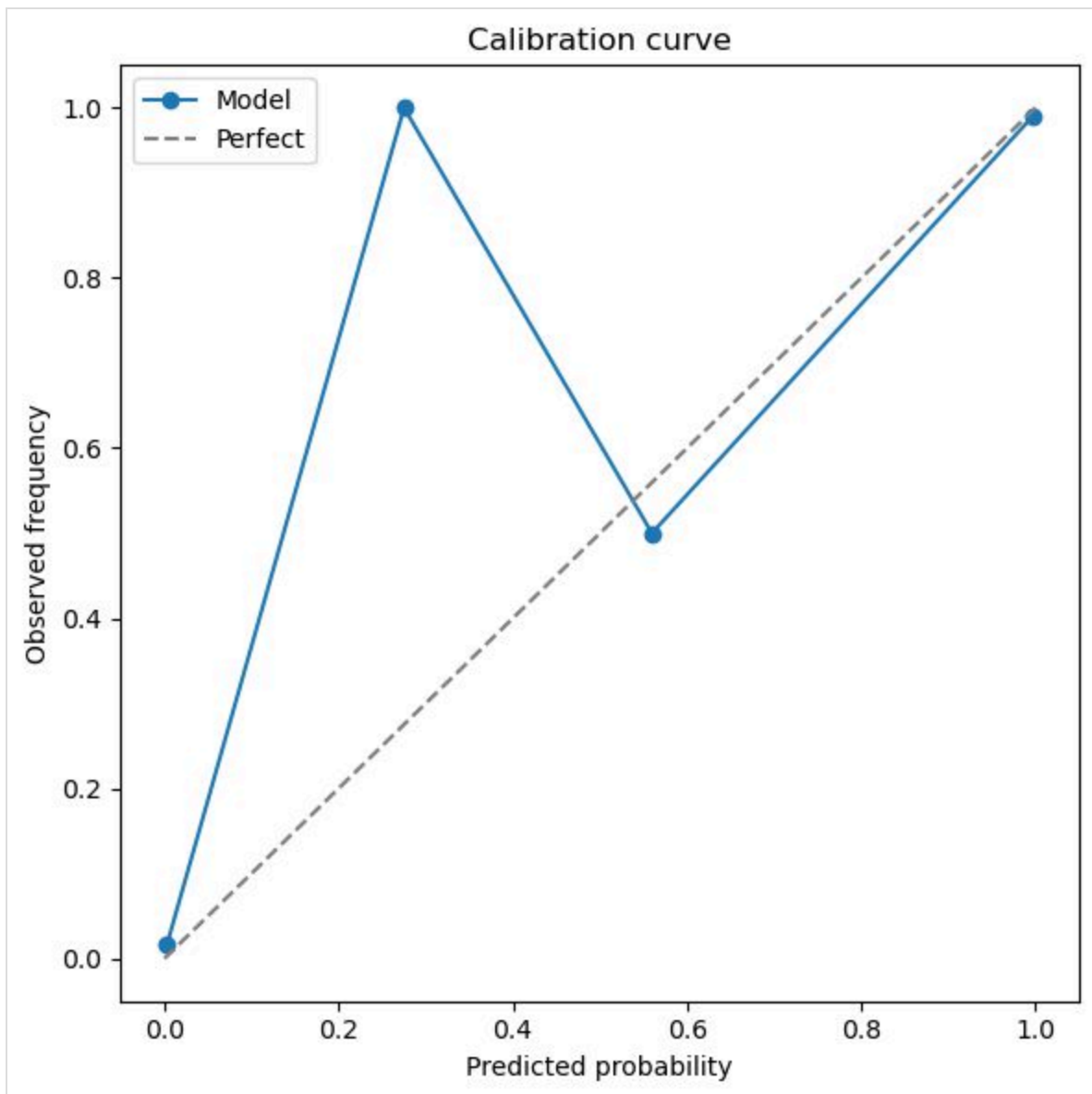
```
precision, recall, _ = precision_recall_curve(y_true, y_prob)
plt.figure(figsize=(8, 6))
plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-recall curve')
plt.tight_layout()
plt.show()
```



Calibration

A calibrated model that reports 0.8 confidence is correct about 80 percent of the time. The diagonal is perfect calibration.

```
prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10)
plt.figure(figsize=(6, 6))
plt.plot(prob_pred, prob_true, marker='o', label='Model')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Perfect')
plt.xlabel('Predicted probability')
plt.ylabel('Observed frequency')
plt.title('Calibration curve')
plt.legend()
plt.tight_layout()
plt.show()
```

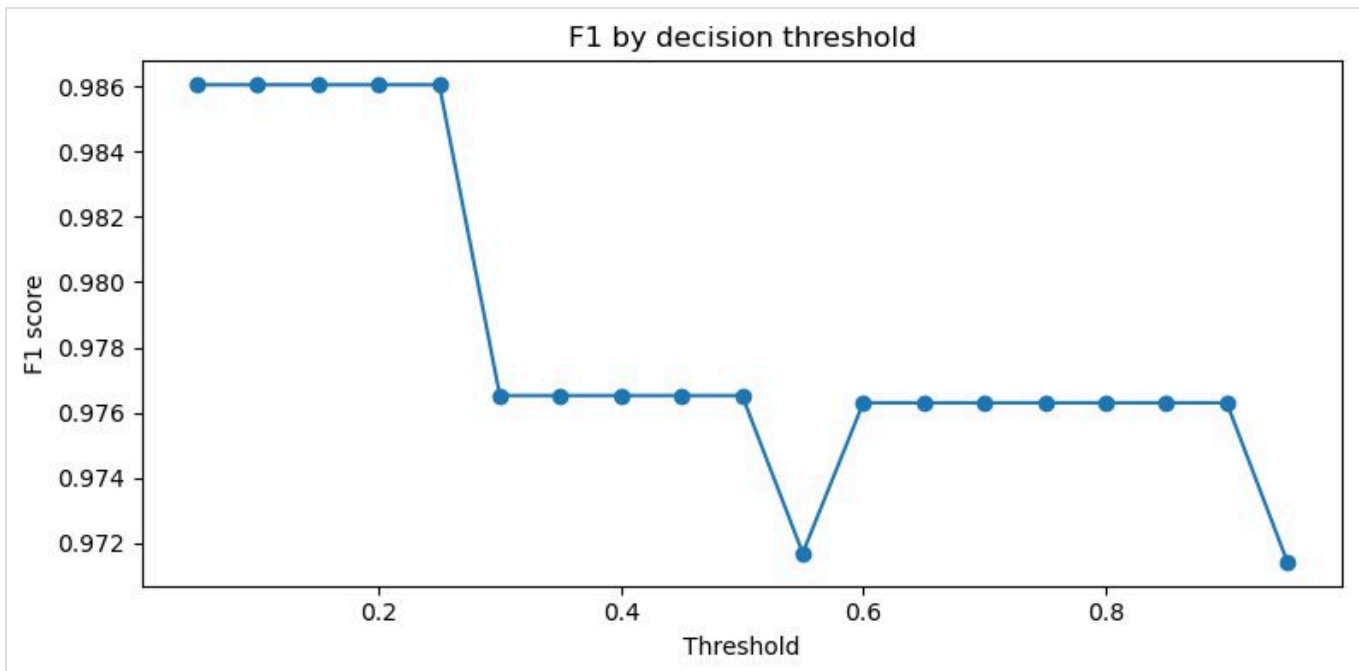


Threshold Tuning

The default 0.5 threshold is not always best. Sweep the threshold and choose the F1-optimal cutoff.

```
thresholds = np.linspace(0.05, 0.95, 19)
scores = [f1_score(y_true, (y_prob >= t).astype(int)) for t in thresholds]
best_index = int(np.argmax(scores))
print(f"Best threshold: {thresholds[best_index]:.2f}")
print(f"Best F1: {scores[best_index]:.4f}")
plt.figure(figsize=(8, 4))
plt.plot(thresholds, scores, marker='o')
plt.xlabel('Threshold')
plt.ylabel('F1 score')
plt.title('F1 by decision threshold')
plt.tight_layout()
plt.show()
```

Best threshold: 0.05
Best F1: 0.9860



Save Model

```
torch.save(model.state_dict(), 'eval_breast_cancer.pt')  
print('saved eval_breast_cancer.pt')
```

saved eval_breast_cancer.pt

Load Model

```
loaded_model = Model().to(DEVICE)  
loaded_model.load_state_dict(torch.load('eval_breast_cancer.pt', map_location=DEVICE))  
loaded_model.eval()  
print('loaded eval_breast_cancer.pt')
```

loaded eval_breast_cancer.pt

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55272/1892292173.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('eval_breast_cancer.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features, threshold=0.5):
    """
    Predict benign or malignant with a tunable threshold.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values.
        threshold (float): Decision threshold.

    Returns:
        tuple: Predicted label name and probability.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probability = torch.softmax(model(X_new), dim=1)[0, 1].item()
        label = 'benign' if probability >= threshold else 'malignant'
    return label, probability
```

Run Inference

```
label, probability = predict(loaded_model, X_test[0])
print(f"Predicted: {label} (p={probability:.4f}) | Actual: {data.target_names[y_test[0]]}")
```

```
Predicted: benign (p=1.0000) | Actual: malignant
```

22-AE: Fashion-MNIST Dataset - Autoencoder

[Fashion-MNIST Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x116f49af0>
```


Parameters

```
IMAGE_SIZE = 28  
IMAGE_SIZE
```

```
28
```

```
LATENT_DIM = 16  
LATENT_DIM
```

```
16
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 10  
EPOCHS
```

```
10
```

```
BATCH_SIZE = 256  
BATCH_SIZE
```

```
256
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.ToTensor()
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Model

The encoder compresses each 784-pixel image to a 16-number latent vector. The decoder reconstructs the image from that vector.

```

class Autoencoder(nn.Module):
    """
    A fully connected autoencoder for 28x28 grayscale images.
    """

    def __init__(self, latent_dim=LATENT_DIM):
        """
        Initialize the encoder and decoder.

        Parameters:
            latent_dim (int): Size of the latent vector.

        Returns:
            None
        """
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(IMAGE_SIZE * IMAGE_SIZE, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim))
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, IMAGE_SIZE * IMAGE_SIZE),
            nn.Sigmoid())

    def forward(self, x):
        """
        Encode and reconstruct a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            tuple: Reconstruction and latent vector.
        """
        latent = self.encoder(x)
        reconstruction = self.decoder(latent)
        return reconstruction, latent

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = Autoencoder().to(DEVICE)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```
Autoencoder(  
  (encoder): Sequential(  
    (0): Flatten(start_dim=1, end_dim=-1)  
    (1): Linear(in_features=784, out_features=128, bias=True)  
    (2): ReLU()  
    (3): Linear(in_features=128, out_features=16, bias=True)  
  )  
  (decoder): Sequential(  
    (0): Linear(in_features=16, out_features=128, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=128, out_features=784, bias=True)  
    (3): Sigmoid()  
  )  
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the autoencoder for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Reconstruction loss.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, _ in loader:
        x = x.to(DEVICE)
        optimizer.zero_grad()
        reconstruction, _ = model(x)
        loss = loss_fn(reconstruction, x.view(x.size(0), -1))
        loss.backward()
        optimizer.step()
        total += loss.item() * len(x)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the autoencoder over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Reconstruction loss.

    Returns:
        float: Mean reconstruction loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, _ in loader:
            x = x.to(DEVICE)
            reconstruction, _ = model(x)
            total += loss_fn(reconstruction, x.view(x.size(0), -1)).item() * len(x)
    return total / len(loader.dataset)
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss = evaluate(model, test_loader, loss_fn)
    history.append(test_loss)
    print(f"Epoch {epoch + 1:2d} | train loss {train_loss:.4f} | test loss {test_loss:.4f}")
```

Epoch 1 | train loss 0.0476 | test loss 0.0249

Epoch 2 | train loss 0.0221 | test loss 0.0206

Epoch 3 | train loss 0.0193 | test loss 0.0183

Epoch 4 | train loss 0.0176 | test loss 0.0170

Epoch 5 | train loss 0.0165 | test loss 0.0160

Epoch 6 | train loss 0.0156 | test loss 0.0154

Epoch 7 | train loss 0.0151 | test loss 0.0149

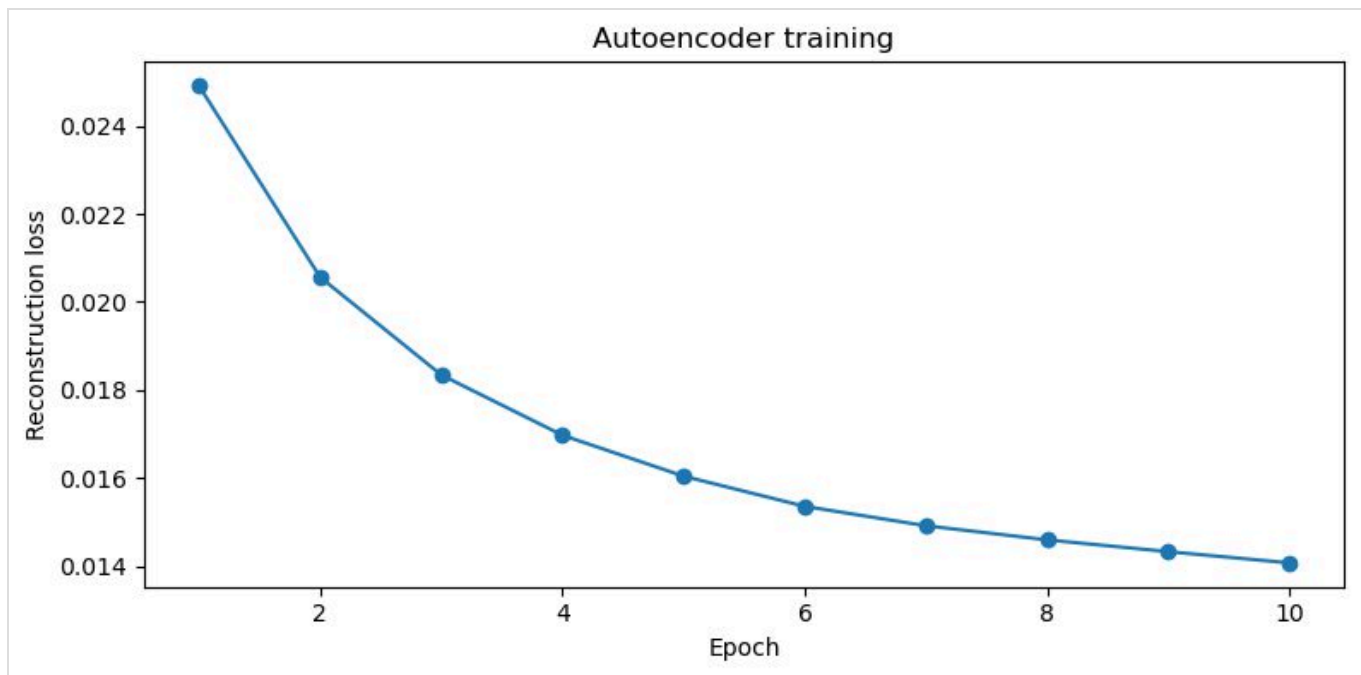
Epoch 8 | train loss 0.0147 | test loss 0.0146

Epoch 9 | train loss 0.0144 | test loss 0.0143

Epoch 10 | train loss 0.0141 | test loss 0.0141

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Reconstruction loss')
plt.title('Autoencoder training')
plt.tight_layout()
plt.show()
```



Visualize Reconstructions

Function

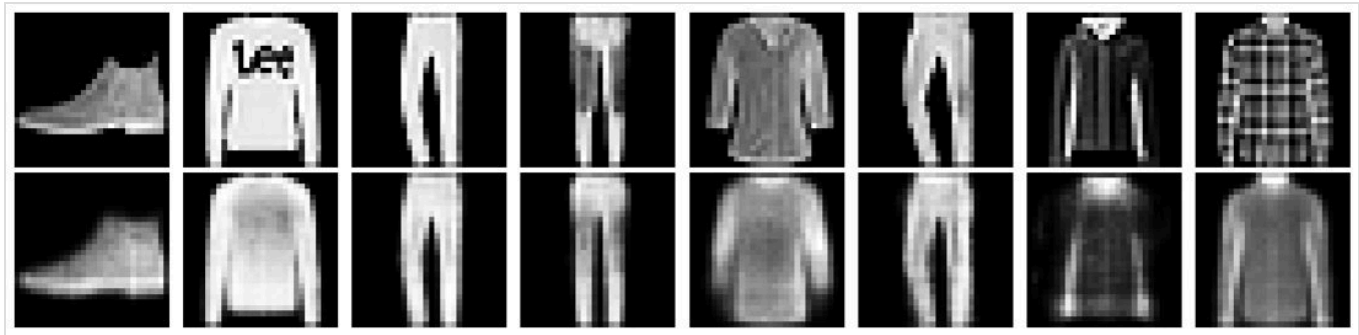
```
def show_reconstructions(model, dataset, n=8):
    """
    Plot original images above their reconstructions.

    Parameters:
        model (nn.Module): Trained autoencoder.
        dataset (Dataset): Source of images.
        n (int): Number of images to show.

    Returns:
        None
    """
    model.eval()
    images = torch.stack([dataset[i][0] for i in range(n)]).to(DEVICE)
    with torch.no_grad():
        reconstruction, _ = model(images)
    fig, axes = plt.subplots(2, n, figsize=(2 * n, 4))
    for i in range(n):
        axes[0, i].imshow(images[i].cpu().squeeze(), cmap='gray')
        axes[0, i].axis('off')
        axes[1, i].imshow(reconstruction[i].cpu().view(28, 28), cmap='gray')
        axes[1, i].axis('off')
    fig.tight_layout()
    plt.show()
```

Run

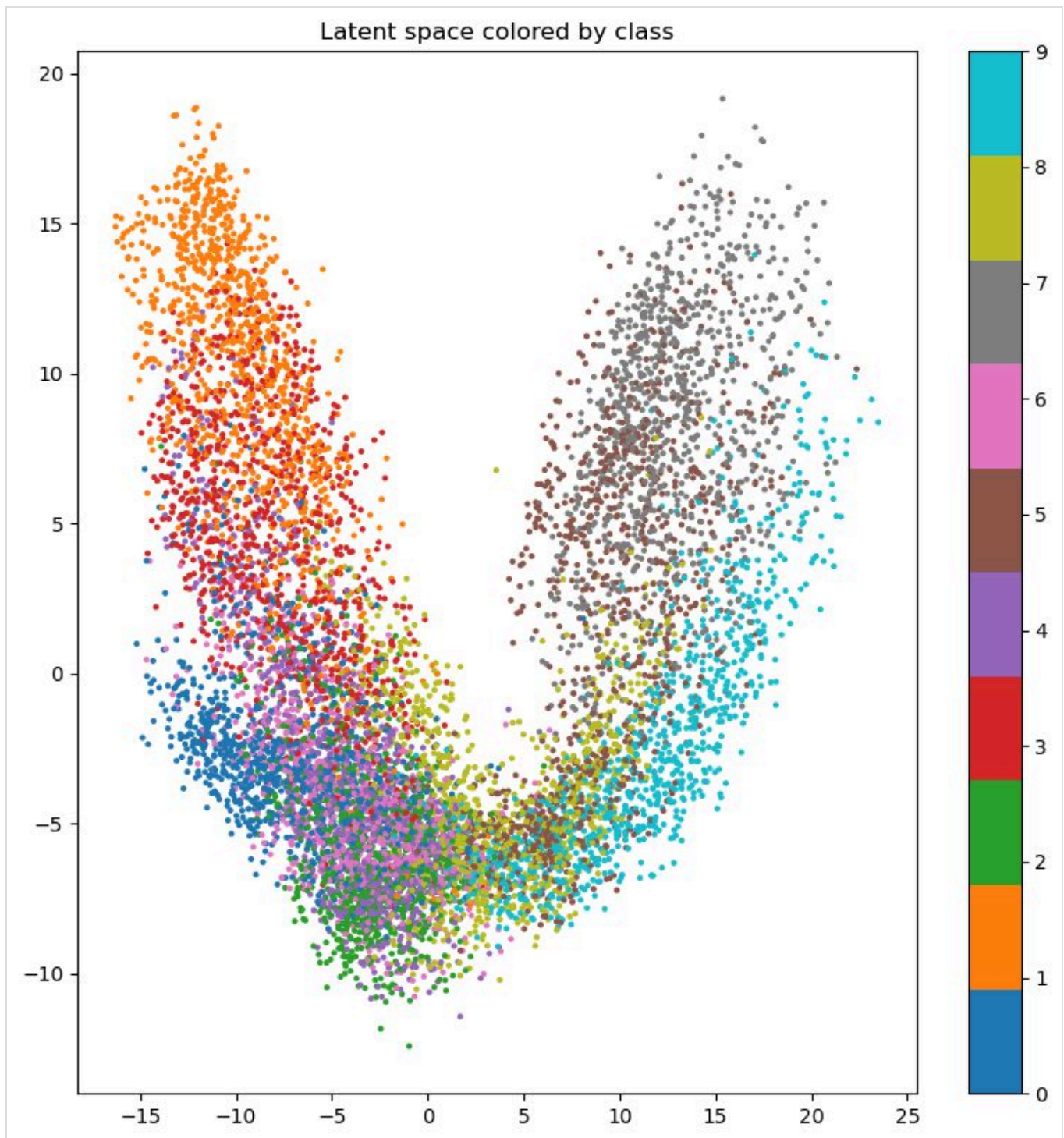
```
show_reconstructions(model, test_data, n=8)
```



Explore the Latent Space

Project the 16-dimensional latent vectors down to two dimensions with PCA and color by the true class. Similar garments cluster together.

```
model.eval()
latents = []
labels = []
with torch.no_grad():
    for x, y in test_loader:
        _, latent = model(x.to(DEVICE))
        latents.append(latent.cpu().numpy())
        labels.append(y.numpy())
latents = np.concatenate(latents)
labels = np.concatenate(labels)
coords = PCA(n_components=2).fit_transform(latents)
plt.figure(figsize=(8, 8))
plt.scatter(coords[:, 0], coords[:, 1], c=labels, cmap='tab10', s=4)
plt.title('Latent space colored by class')
plt.colorbar()
plt.tight_layout()
plt.show()
```

Anomaly Detection

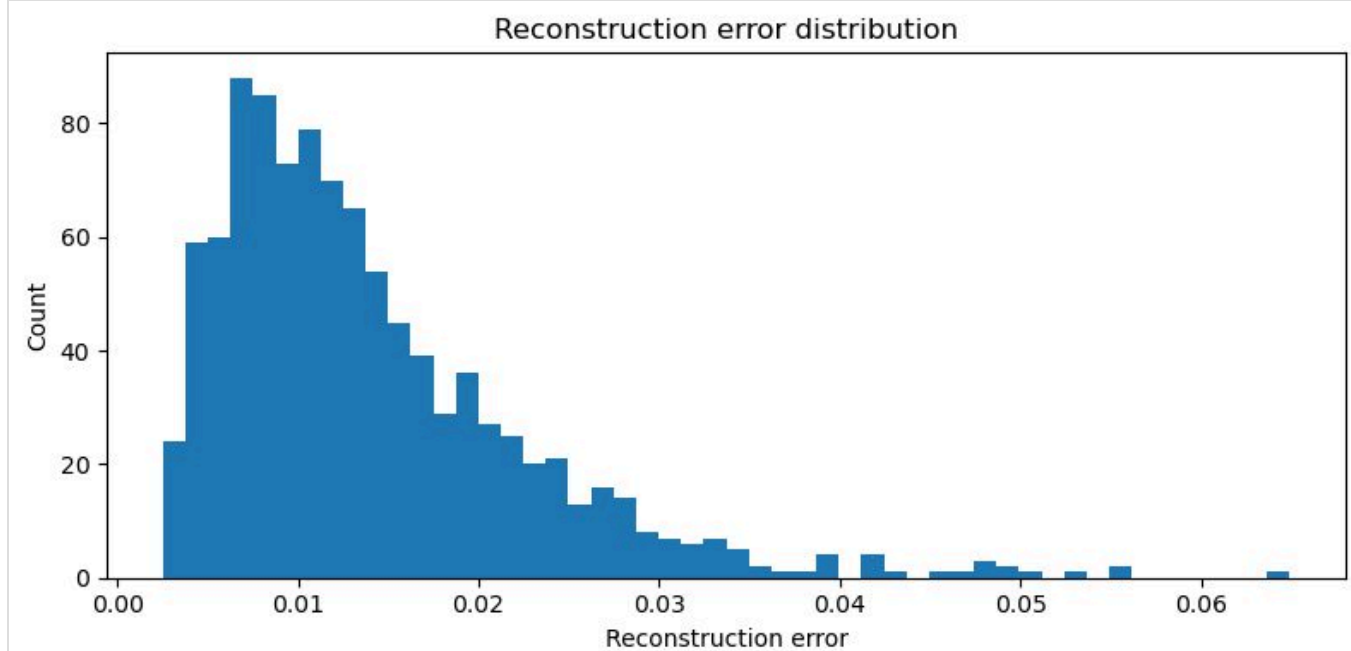
The reconstruction error is a score for how unusual an image is. Images the model has not learned to reconstruct have high error.

```
def reconstruction_errors(model, dataset, n=1000):
    """
    Compute per-image reconstruction errors.

    Parameters:
        model (nn.Module): Trained autoencoder.
        dataset (Dataset): Source of images.
        n (int): Number of images to score.

    Returns:
        numpy.ndarray: Reconstruction errors.
    """
    model.eval()
    errors = []
    with torch.no_grad():
        for i in range(n):
            image, _ = dataset[i]
            image = image.to(DEVICE).unsqueeze(0)
            reconstruction, _ = model(image)
            error = torch.mean((reconstruction - image.view(1, -1)) ** 2).item()
            errors.append(error)
    return np.array(errors)
```

```
errors = reconstruction_errors(model, test_data, n=1000)
plt.figure(figsize=(8, 4))
plt.hist(errors, bins=50)
plt.xlabel('Reconstruction error')
plt.ylabel('Count')
plt.title('Reconstruction error distribution')
plt.tight_layout()
plt.show()
```



Save Model

```
torch.save(model.state_dict(), 'autoencoder_fashion_mnist.pt')
print('saved autoencoder_fashion_mnist.pt')
```

```
saved autoencoder_fashion_mnist.pt
```

Load Model

```
loaded_model = Autoencoder().to(DEVICE)
loaded_model.load_state_dict(torch.load('autoencoder_fashion_mnist.pt',
map_location=DEVICE))
loaded_model.eval()
print('loaded autoencoder_fashion_mnist.pt')
```

```
loaded autoencoder_fashion_mnist.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55323/3630104904.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('autoencoder_fashion_mnist.pt',
map_location=DEVICE))
```

Inference

Function

```
def reconstruct(model, image):
    """
    Reconstruct one image and return its error.

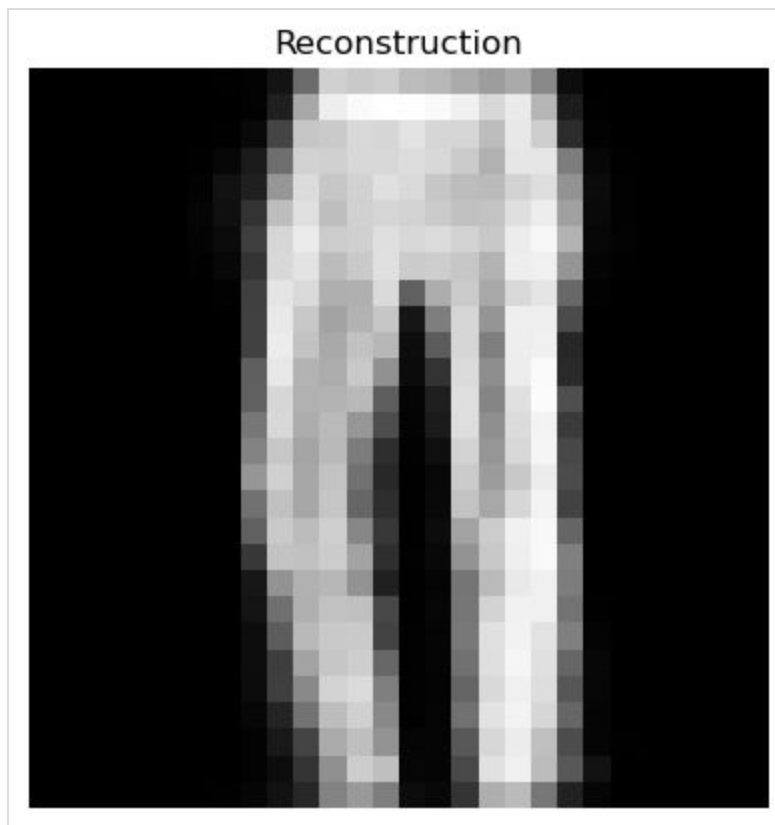
    Parameters:
        model (nn.Module): Trained autoencoder.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Reconstructed image and error.
    """
    model.eval()
    with torch.no_grad():
        batch = image.unsqueeze(0).to(DEVICE)
        reconstruction, _ = model(batch)
        error = torch.mean((reconstruction - batch.view(1, -1)) ** 2).item()
    return reconstruction.view(28, 28).cpu(), error
```

Run Inference

```
image, label = test_data[5]
reconstruction, error = reconstruct(loader_model, image)
print(f"Label: {label} | Reconstruction error: {error:.4f}")
plt.imshow(reconstruction, cmap='gray')
plt.title('Reconstruction')
plt.axis('off')
plt.show()
```

```
Label: 1 | Reconstruction error: 0.0137
```



23-GAN: Fashion-MNIST Dataset - GAN

[Fashion-MNIST Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x10fafdb10>
```

Parameters

```
IMAGE_SIZE = 28
IMAGE_SIZE
```

```
28
```

```
LATENT_DIM = 64  
LATENT_DIM
```

```
64
```

```
HIDDEN_DIM = 256  
HIDDEN_DIM
```

```
256
```

Hyperparameters

```
LEARNING_RATE = 0.0002  
LEARNING_RATE
```

```
0.0002
```

```
EPOCHS = 30  
EPOCHS
```

```
30
```

```
BATCH_SIZE = 128  
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))])
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
len(train_data)
```

```
60000
```

Create Generator

```
class Generator(nn.Module):
    """
    A generator that maps noise to 28x28 images.
    """

    def __init__(self, latent_dim=LATENT_DIM, hidden_dim=HIDDEN_DIM):
        """
        Initialize the generator layers.

        Parameters:
            latent_dim (int): Size of the noise vector.
            hidden_dim (int): Hidden layer size.

        Returns:
            None
        """
        super(Generator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(latent_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, IMAGE_SIZE * IMAGE_SIZE),
            nn.Tanh())

    def forward(self, z):
        """
        Generate a batch of images from noise.

        Parameters:
            z (torch.Tensor): Noise batch.

        Returns:
            torch.Tensor: Batch of images.
        """
        images = self.net(z)
        return images.view(z.size(0), 1, IMAGE_SIZE, IMAGE_SIZE)
```


Create Discriminator

```
class Discriminator(nn.Module):
    """
    A discriminator that scores images as real or fake.
    """

    def __init__(self, hidden_dim=HIDDEN_DIM):
        """
        Initialize the discriminator layers.

        Parameters:
            hidden_dim (int): Hidden layer size.

        Returns:
            None
        """
        super(Discriminator, self).__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(IMAGE_SIZE * IMAGE_SIZE, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, 1))

    def forward(self, x):
        """
        Score a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Logits for each image.
        """
        return self.net(x)
```

Instantiate Models, Loss, and Optimizers

```
torch.manual_seed(SEED)
generator = Generator().to(DEVICE)
discriminator = Discriminator().to(DEVICE)
loss_fn = nn.BCEWithLogitsLoss()
opt_g = torch.optim.Adam(generator.parameters(), lr=LEARNING_RATE, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(discriminator.parameters(), lr=LEARNING_RATE, betas=(0.5, 0.999))
generator
```

```
Generator(  
  (net): Sequential(  
    (0): Linear(in_features=64, out_features=256, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=256, out_features=256, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=256, out_features=784, bias=True)  
    (5): Tanh()  
  )  
)
```

Train Model

Functions

```
def discriminator_loss(disc, gen, real, loss_fn, latent_dim):
    """
    Compute the discriminator loss on real and fake images.

    Parameters:
        disc (nn.Module): The discriminator.
        gen (nn.Module): The generator.
        real (torch.Tensor): Batch of real images.
        loss_fn (nn.Module): Binary cross-entropy loss.
        latent_dim (int): Size of the noise vector.

    Returns:
        torch.Tensor: The discriminator loss.
    """
    noise = torch.randn(real.size(0), latent_dim, device=real.device)
    fake = gen(noise)
    real_logits = disc(real)
    fake_logits = disc(fake.detach())
    real_loss = loss_fn(real_logits, torch.ones_like(real_logits))
    fake_loss = loss_fn(fake_logits, torch.zeros_like(fake_logits))
    return real_loss + fake_loss


def generator_loss(disc, gen, batch_size, loss_fn, latent_dim):
    """
    Compute the generator loss from the discriminator scores.

    Parameters:
        disc (nn.Module): The discriminator.
        gen (nn.Module): The generator.
        batch_size (int): Number of images to generate.
        loss_fn (nn.Module): Binary cross-entropy loss.
        latent_dim (int): Size of the noise vector.

    Returns:
        torch.Tensor: The generator loss.
    """
    noise = torch.randn(batch_size, latent_dim, device=DEVICE)
    fake_logits = disc(gen(noise))
    return loss_fn(fake_logits, torch.ones_like(fake_logits))


def train_epoch(disc, gen, loader, loss_fn, opt_d, opt_g, latent_dim):
    """
    Train the discriminator and generator for one epoch.

    Parameters:
        disc (nn.Module): The discriminator.
        gen (nn.Module): The generator.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Binary cross-entropy loss.
        opt_d (torch.optim.Optimizer): Discriminator optimizer.
```

```

    opt_g (torch.optim.Optimizer): Generator optimizer.
    latent_dim (int): Size of the noise vector.

```

Returns:

```

    tuple: Mean discriminator and generator loss.

```

```

"""

```

```

disc.train()
gen.train()
total_d, total_g = 0.0, 0.0
for real, _ in loader:
    real = real.to(DEVICE)
    opt_d.zero_grad()
    d_loss = discriminator_loss(disc, gen, real, loss_fn, latent_dim)
    d_loss.backward()
    opt_d.step()
    opt_g.zero_grad()
    g_loss = generator_loss(disc, gen, real.size(0), loss_fn, latent_dim)
    g_loss.backward()
    opt_g.step()
    total_d += d_loss.item() * len(real)
    total_g += g_loss.item() * len(real)
n = len(loader.dataset)
return total_d / n, total_g / n

```

Training Loop

```

history_d = []
history_g = []
for epoch in range(EPOCHS):
    d_loss, g_loss = train_epoch(discriminator, generator, train_loader,
                                loss_fn, opt_d, opt_g, LATENT_DIM)

    history_d.append(d_loss)
    history_g.append(g_loss)
    print(f"Epoch {epoch + 1:2d} | discriminator {d_loss:.4f} | generator {g_loss:.4f}")

```

```
Epoch 1 | discriminator 0.4615 | generator 2.3316
```

```
Epoch 2 | discriminator 0.3134 | generator 3.7573
```

```
Epoch 3 | discriminator 0.3863 | generator 3.6871
```

```
Epoch 4 | discriminator 0.5466 | generator 3.0557
```

```
Epoch 5 | discriminator 0.5778 | generator 2.5289
```

```
Epoch 6 | discriminator 0.6026 | generator 2.6899
```

REVERSE ENGINEERING DATA

Epoch 7 | discriminator 0.6608 | generator 2.6085

Epoch 8 | discriminator 0.7435 | generator 2.3001

Epoch 9 | discriminator 0.7586 | generator 2.1638

Epoch 10 | discriminator 0.7841 | generator 2.1347

Epoch 11 | discriminator 0.7944 | generator 1.9855

Epoch 12 | discriminator 0.8158 | generator 2.0745

Epoch 13 | discriminator 0.8200 | generator 2.0426

Epoch 14 | discriminator 0.8357 | generator 1.9927

Epoch 15 | discriminator 0.8383 | generator 1.9070

Epoch 16 | discriminator 0.8470 | generator 1.9132

Epoch 17 | discriminator 0.8653 | generator 1.8377

Epoch 18 | discriminator 0.8996 | generator 1.7025

Epoch 19 | discriminator 0.9236 | generator 1.6957

Epoch 20 | discriminator 0.9409 | generator 1.6328

Epoch 21 | discriminator 0.9382 | generator 1.6664

Epoch 22 | discriminator 0.9611 | generator 1.5809

Epoch 23 | discriminator 0.9782 | generator 1.5300

Epoch 24 | discriminator 0.9820 | generator 1.5367

Epoch 25 | discriminator 0.9805 | generator 1.4951

Epoch 26 | discriminator 1.0084 | generator 1.4867

Epoch 27 | discriminator 1.0148 | generator 1.4388

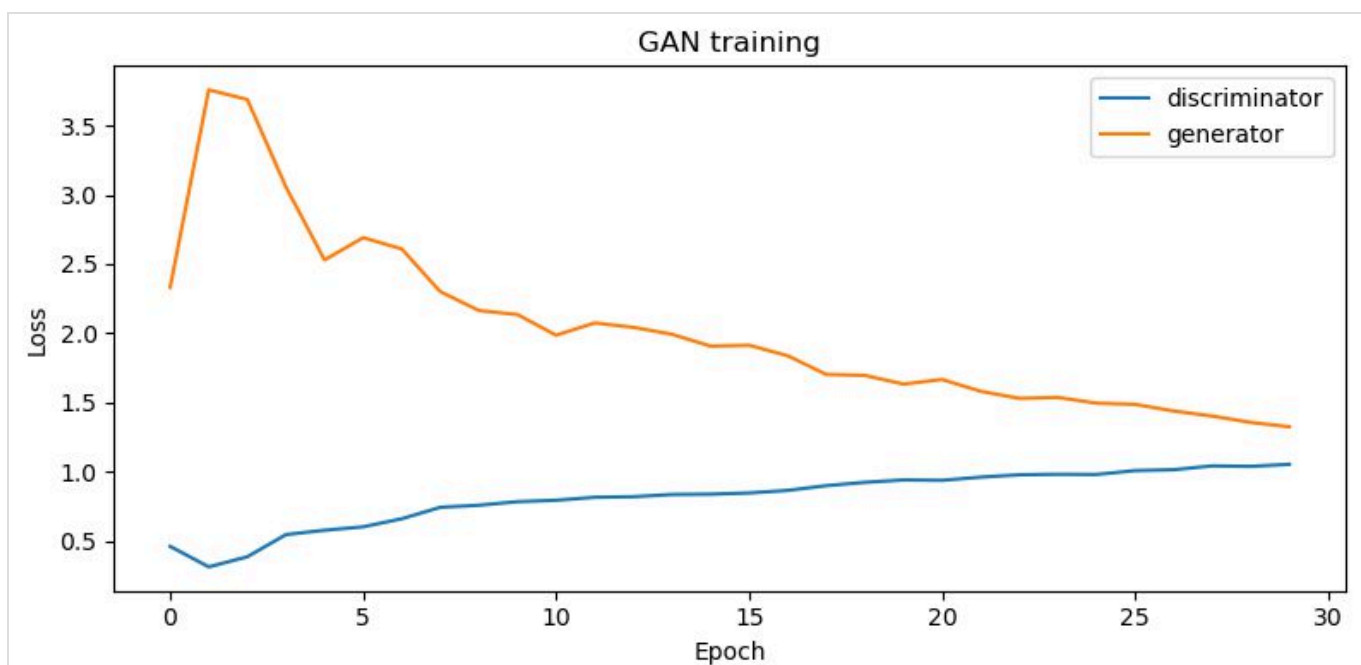
Epoch 28 | discriminator 1.0424 | generator 1.4023

Epoch 29 | discriminator 1.0389 | generator 1.3562

Epoch 30 | discriminator 1.0539 | generator 1.3248

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(history_d, label='discriminator')
plt.plot(history_g, label='generator')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('GAN training')
plt.legend()
plt.tight_layout()
plt.show()
```



Generate Images

Function

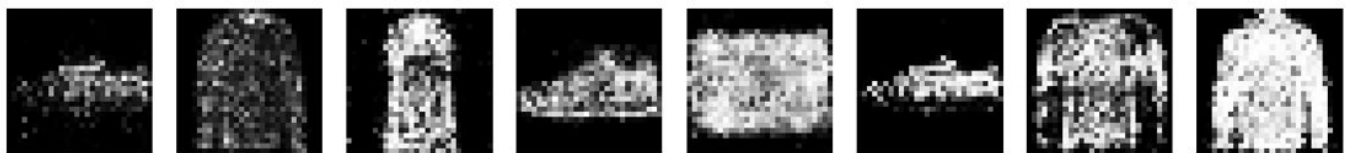
```
def show_samples(gen, n=8):
    """
    Plot a grid of generated images.

    Parameters:
        gen (nn.Module): Trained generator.
        n (int): Number of images to show.

    Returns:
        None
    """
    gen.eval()
    with torch.no_grad():
        noise = torch.randn(n, LATENT_DIM, device=DEVICE)
        images = gen(noise).cpu()
    fig, axes = plt.subplots(1, n, figsize=(2 * n, 2))
    for i in range(n):
        axes[i].imshow(images[i].squeeze(), cmap='gray')
        axes[i].axis('off')
    fig.tight_layout()
    plt.show()
```

Run

```
show_samples(generator, n=8)
```



Save Models

```
torch.save(generator.state_dict(), 'gan_generator_fashion_mnist.pt')
torch.save(discriminator.state_dict(), 'gan_discriminator_fashion_mnist.pt')
print('saved gan models')
```

```
saved gan models
```

Load Models

```
loaded_generator = Generator().to(DEVICE)
loaded_generator.load_state_dict(torch.load('gan_generator_fashion_mnist.pt',
map_location=DEVICE))
loaded_generator.eval()
print('loaded gan_generator_fashion_mnist.pt')
```

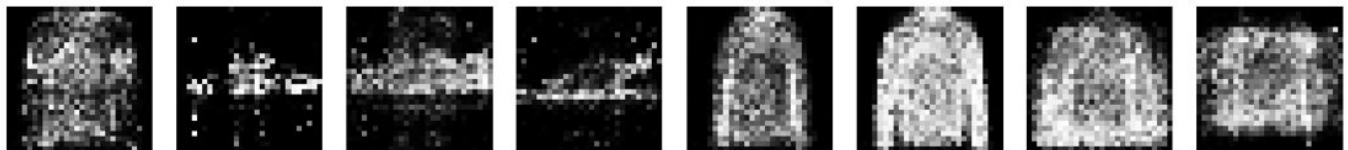
```
loaded gan_generator_fashion_mnist.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55391/1355962562.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
```

```
loaded_generator.load_state_dict(torch.load('gan_generator_fashion_mnist.pt',
map_location=DEVICE))
```

Inference

```
show_samples(loaded_generator, n=8)
```



24-VIT: Fashion-MNIST Dataset - Vision Transformer

[Fashion-MNIST Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

[2] Alexey Dosovitskiy et al., <https://arxiv.org/abs/2010.11929>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib scikit-learn
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x11da49af0>
```

Parameters

IMAGE_SIZE = 28
IMAGE_SIZE

28

PATCH_SIZE = 4
PATCH_SIZE

4

EMBED_DIM = 64
EMBED_DIM

64

N_HEADS = 4
N_HEADS

4

N_LAYERS = 4
N_LAYERS

4

FF_DIM = 128
FF_DIM

128

NUM_CLASSES = 10
NUM_CLASSES

10

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
CLASS_NAMES
```

```
['T-shirt/top',  
 'Trouser',  
 'Pullover',  
 'Dress',  
 'Coat',  
 'Sandal',  
 'Shirt',  
 'Sneaker',  
 'Bag',  
 'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 8  
EPOCHS
```

```
8
```

```
BATCH_SIZE = 128  
BATCH_SIZE
```

```
128
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.2860,), (0.3530,))])
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Patch Embedding

The image is cut into non-overlapping patches. Each patch is flattened and projected to an embedding vector by a linear layer.

```
class PatchEmbedding(nn.Module):
    """
    Split an image into patches and embed each patch with a linear layer.
    """

    def __init__(self, embed_dim=EMBED_DIM, patch_size=PATCH_SIZE):
        """
        Initialize the patch projection.

        Parameters:
            embed_dim (int): Embedding size per patch.
            patch_size (int): Side length of each patch.

        Returns:
            None
        """
        super(PatchEmbedding, self).__init__()
        self.patch_size = patch_size
        self.proj = nn.Linear(patch_size * patch_size, embed_dim)

    def forward(self, x):
        """
        Embed a batch of grayscale images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Patch embeddings.
        """
        batch, channels, height, width = x.shape
        p = self.patch_size
        x = x.reshape(batch, channels, height // p, p, width // p, p)
        x = x.permute(0, 2, 4, 1, 3, 5).reshape(batch, (height // p) * (width // p), -1)
        return self.proj(x)
```

Create Transformer Encoder

```

class Attention(nn.Module):
    """
    Multi-head self-attention built from explicit tensor operations.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS):
        """
        Initialize the query, key, value, and output projections.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.

        Returns:
            None
        """
        super(Attention, self).__init__()
        self.n_heads = n_heads
        self.head_dim = embed_dim // n_heads
        self.qkv = nn.Linear(embed_dim, 3 * embed_dim)
        self.proj = nn.Linear(embed_dim, embed_dim)

    def forward(self, x):
        """
        Run multi-head self-attention.

        Parameters:
            x (torch.Tensor): Input of shape (batch, tokens, embed_dim).

        Returns:
            torch.Tensor: Attention output.
        """
        batch, tokens, embed_dim = x.shape
        qkv = self.qkv(x).reshape(batch, tokens, 3, self.n_heads, self.head_dim)
        query, key, value = qkv.permute(2, 0, 3, 1, 4)
        out = F.scaled_dot_product_attention(query, key, value)
        out = out.transpose(1, 2).reshape(batch, tokens, embed_dim)
        return self.proj(out)

```

```

class EncoderBlock(nn.Module):
    """
    A transformer encoder block with attention and a feed-forward network.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS, ff_dim=FF_DIM):
        """
        Initialize attention, normalization, and feed-forward layers.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            ff_dim (int): Feed-forward hidden size.

        Returns:
            None
        """
        super(EncoderBlock, self).__init__()
        self.attn = Attention(embed_dim, n_heads)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.GELU(),
            nn.Linear(ff_dim, embed_dim))
        self.norm2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        """
        Run the encoder block with residual connections.

        Parameters:
            x (torch.Tensor): Input of shape (batch, tokens, embed_dim).

        Returns:
            torch.Tensor: Output of the same shape.
        """
        x = self.norm1(x + self.attn(x))
        return self.norm2(x + self.ff(x))

```

Create Vision Transformer

```

class ViT(nn.Module):
    """
    A small vision transformer for 28x28 grayscale images.
    """

    def __init__(self, embed_dim=EMBED_DIM, n_heads=N_HEADS, n_layers=N_LAYERS,
                  ff_dim=FF_DIM, patch_size=PATCH_SIZE, num_classes=NUM_CLASSES):
        """
        Initialize patch embedding, tokens, blocks, and the head.

        Parameters:
            embed_dim (int): Embedding size.
            n_heads (int): Number of attention heads.
            n_layers (int): Number of encoder blocks.
            ff_dim (int): Feed-forward hidden size.
            patch_size (int): Side length of each patch.
            num_classes (int): Number of output classes.

        Returns:
            None
        """
        super(ViT, self).__init__()
        self.patch = PatchEmbedding(embed_dim, patch_size)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed = nn.Parameter(
            torch.zeros(1, (IMAGE_SIZE // patch_size) ** 2 + 1, embed_dim))
        self.blocks = nn.Sequential(*[
            EncoderBlock(embed_dim, n_heads, ff_dim) for _ in range(n_layers)])
        self.norm = nn.LayerNorm(embed_dim)
        self.head = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        """
        Classify a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Output logits.
        """
        x = self.patch(x)
        cls = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1) + self.pos_embed
        x = self.blocks(x)
        return self.head(self.norm(x[:, 0]))

```


Instantiate Model, Loss, and Optimizer

```
torch.manual_seed(SEED)
model = ViT().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())
```

139018

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
history = {'loss': [], 'accuracy': []}
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    history['loss'].append(test_loss)
    history['accuracy'].append(test_accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

Epoch 1 | train loss 0.7988 | test accuracy 0.8157

Epoch 2 | train loss 0.4420 | test accuracy 0.8441

Epoch 3 | train loss 0.3857 | test accuracy 0.8607

Epoch 4 | train loss 0.3518 | test accuracy 0.8652

Epoch 5 | train loss 0.3254 | test accuracy 0.8710

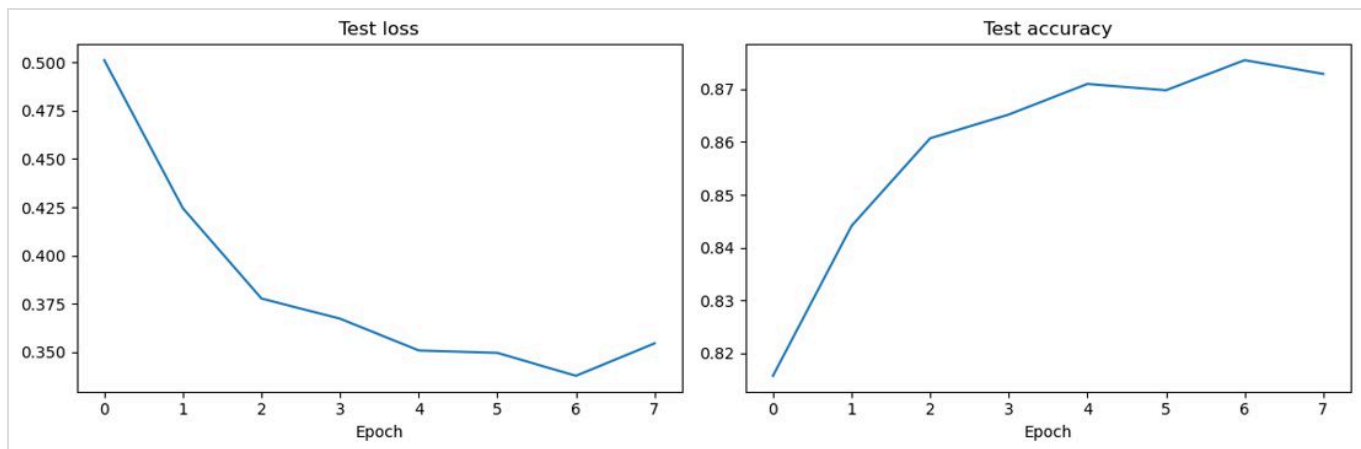
Epoch 6 | train loss 0.3059 | test accuracy 0.8698

Epoch 7 | train loss 0.2883 | test accuracy 0.8755

Epoch 8 | train loss 0.2745 | test accuracy 0.8729

Visualize Training

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history['loss'])
axes[0].set_title('Test loss')
axes[0].set_xlabel('Epoch')
axes[1].plot(history['accuracy'])
axes[1].set_title('Test accuracy')
axes[1].set_xlabel('Epoch')
fig.tight_layout()
plt.show()
```



Evaluate Model

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.8729

```
all_preds = []
all_labels = []
model.eval()
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(DEVICE)
        all_preds.append(model(x).argmax(1).cpu().numpy())
        all_labels.append(y.numpy())
all_preds = np.concatenate(all_preds)
all_labels = np.concatenate(all_labels)
print(confusion_matrix(all_labels, all_preds))
print(classification_report(all_labels, all_preds, target_names=CLASS_NAMES))
```

```

[[814  2 12  6  1  1 159  0  5  0]
 [ 4 981  1  5  4  0  4  0  1  0]
 [ 23  1 814  4 51  0 105  0  2  0]
 [ 51 13  8 805 33  0  90  0  0  0]
 [  0  0 123 18 739  0 118  0  2  0]
 [  1  0  0  0  0  0 930  1  53  7  8]
[111  3 56 10 45  0 768  0  7  0]
 [  0  0  0  0  0  6  0 979  1 14]
 [  8  2  1  4  2  3 20  3 957  0]
 [  1  0  0  0  0  3  0 54  0 942]]
      precision    recall  f1-score   support

T-shirt/top      0.80      0.81      0.81      1000
Trouser          0.98      0.98      0.98      1000
Pullover         0.80      0.81      0.81      1000
Dress            0.94      0.81      0.87      1000
Coat             0.84      0.74      0.79      1000
Sandal           0.99      0.93      0.96      1000
Shirt            0.61      0.77      0.68      1000
Sneaker          0.90      0.98      0.94      1000
Bag              0.97      0.96      0.97      1000
Ankle boot       0.98      0.94      0.96      1000

 accuracy              0.87      10000
 macro avg           0.88      0.87      0.88      10000
 weighted avg        0.88      0.87      0.88      10000

```

Save Model

```

torch.save(model.state_dict(), 'vit_fashion_mnist.pt')
print('saved vit_fashion_mnist.pt')

```

```
saved vit_fashion_mnist.pt
```

Load Model

```

loaded_model = ViT().to(DEVICE)
loaded_model.load_state_dict(torch.load('vit_fashion_mnist.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded vit_fashion_mnist.pt')

```

```
loaded vit_fashion_mnist.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55772/3325605801.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('vit_fashion_mnist.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, image):
    """
    Predict the class of one normalized image tensor.

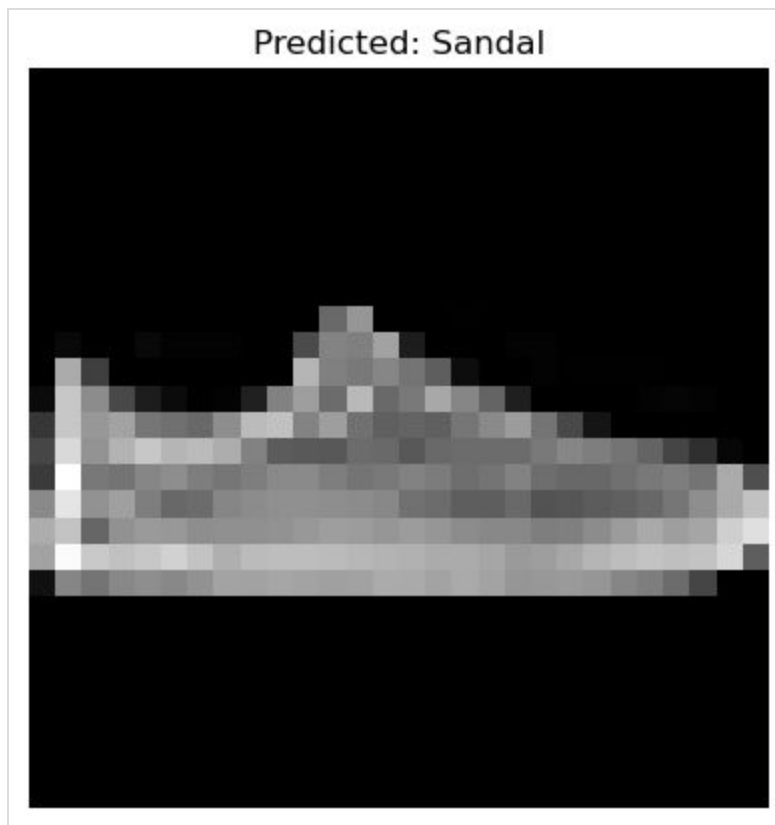
    Parameters:
        model (nn.Module): Trained model.
        image (torch.Tensor): A 1x28x28 image tensor.

    Returns:
        tuple: Predicted class name and confidence.
    """
    model.eval()
    with torch.no_grad():
        logits = model(image.unsqueeze(0).to(DEVICE))
        probs = torch.softmax(logits, dim=1)
        confidence, predicted = probs.max(dim=1)
    return CLASS_NAMES[predicted.item()], confidence.item()
```

Run Inference

```
image, label = test_data[12]
name, confidence = predict(loaded_model, image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
plt.imshow(image.squeeze(), cmap='gray')
plt.title(f"Predicted: {name}")
plt.axis('off')
plt.show()
```

```
True: Sneaker | Predicted: Sandal (0.6848)
```



Part VII: Segmentation, Reinforcement Learning, and Deployment

25-DEPLOY: Fashion-MNIST Dataset - Model Deployment

[Fashion-MNIST Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Han Xiao, Kashif Rasul, Roland Vollgraf, <https://github.com/zalandoresearch/fashion-mnist>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install -c conda-forge onnx onnxruntime
# conda install numpy matplotlib
```

Import Libraries

```
import argparse
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms
import onnxruntime
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x128749af0>
```

Parameters

```
IMAGE_SIZE = 28  
IMAGE_SIZE
```

```
28
```

```
NUM_CLASSES = 10  
NUM_CLASSES
```

```
10
```

```
CLASS_NAMES = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
CLASS_NAMES
```

```
['T-shirt/top',  
 'Trouser',  
 'Pullover',  
 'Dress',  
 'Coat',  
 'Sandal',  
 'Shirt',  
 'Sneaker',  
 'Bag',  
 'Ankle boot']
```

Hyperparameters

```
LEARNING_RATE = 0.001  
LEARNING_RATE
```

```
0.001
```

```
EPOCHS = 3  
EPOCHS
```

```
3
```

```
BATCH_SIZE = 128  
BATCH_SIZE
```

128

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Load Dataset

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.2860,), (0.3530,))])  
train_data = datasets.FashionMNIST('data', train=True, download=True, transform=transform)  
test_data = datasets.FashionMNIST('data', train=False, download=True, transform=transform)  
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)  
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)  
len(train_data), len(test_data)
```

```
(60000, 10000)
```

Create Model

```
class CNN(nn.Module):
    """
    A small convolutional neural network for 28x28 grayscale images.
    """

    def __init__(self, num_classes=NUM_CLASSES):
        """
        Initialize the convolutional and dense layers.

        Parameters:
            num_classes (int): Number of output classes.

        Returns:
            None
        """
        super(CNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Output logits.
        """
        x = self.features(x)
        return self.classifier(x)
```

Instantiate Model, Loss, and Optimizer

```
torch.manual_seed(SEED)
model = CNN().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model
```

```
CNN(  
  (features): Sequential(  
    (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): ReLU()  
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (4): ReLU()  
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (classifier): Sequential(  
    (0): Flatten(start_dim=1, end_dim=-1)  
    (1): Linear(in_features=1568, out_features=128, bias=True)  
    (2): ReLU()  
    (3): Linear(in_features=128, out_features=10, bias=True)  
  )  
)
```

Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | test accuracy {test_accuracy:.4f}")
```

Epoch 1 | train loss 0.4970 | test accuracy 0.8677

Epoch 2 | train loss 0.3266 | test accuracy 0.8844

Epoch 3 | train loss 0.2806 | test accuracy 0.8886

Save PyTorch Model

```
torch.save(model.state_dict(), 'deploy_cnn_fashion_mnist.pt')
print('saved deploy_cnn_fashion_mnist.pt')
```

saved deploy_cnn_fashion_mnist.pt

Export to ONNX

ONNX is a portable model format. Exporting makes the model usable outside PyTorch, including in services and on edge devices.

```
def export_onnx(model, path):
    """
    Export a PyTorch model to ONNX.

    Parameters:
        model (nn.Module): The trained model.
        path (str): Output ONNX path.

    Returns:
        None
    """
    model.eval()
    dummy = torch.randn(1, 1, IMAGE_SIZE, IMAGE_SIZE, device=DEVICE)
    torch.onnx.export(
        model, dummy, path,
        input_names=['input'], output_names=['logits'],
        dynamic_axes={'input': {0: 'batch'}, 'logits': {0: 'batch'}},
        opset_version=13)
```

Run

```
export_onnx(model, 'cnn_fashion_mnist.onnx')
print('saved cnn_fashion_mnist.onnx')
```

```
saved cnn_fashion_mnist.onnx
```

Verify the ONNX Model

The ONNX output must match the PyTorch output. A close match confirms the export is faithful.

```
def onnx_logits(path, image):
    """
    Run inference with an ONNX model.

    Parameters:
        path (str): ONNX model path.
        image (numpy.ndarray): A 1x1x28x28 array.

    Returns:
        numpy.ndarray: Output logits.
    """
    session = onnxruntime.InferenceSession(path, providers=['CPUExecutionProvider'])
    return session.run(None, {'input': image.astype(np.float32)})[0]
```

```
image, label = test_data[0]
array = image.unsqueeze(0).numpy()
torch_logits = model(image.unsqueeze(0).to(DEVICE)).detach().cpu().numpy()
onnx_out = onnx_logits('cnn_fashion_mnist.onnx', array)
print('max difference:', np.abs(torch_logits - onnx_out).max())
```

```
max difference: 1.9073486e-06
```


Build an Inference Engine

```
class InferenceEngine:
    """
    A small inference engine wrapping an ONNX model.
    """

    def __init__(self, path):
        """
        Load the ONNX session.

        Parameters:
            path (str): ONNX model path.

        Returns:
            None
        """
        self.session = onnxruntime.InferenceSession(path, providers=
['CPUExecutionProvider'])
        self.class_names = CLASS_NAMES

    def predict(self, image):
        """
        Predict the class for one image tensor.

        Parameters:
            image (torch.Tensor): A 1x28x28 image tensor.

        Returns:
            tuple: Predicted class name and confidence.
        """
        array = image.unsqueeze(0).numpy().astype(np.float32)
        logits = self.session.run(None, {'input': array})[0]
        probs = torch.softmax(torch.tensor(logits), dim=1)
        confidence, predicted = probs.max(dim=1)
        return self.class_names[predicted.item()], confidence.item()
```

Inference

```
engine = InferenceEngine('cnn_fashion_mnist.onnx')
image, label = test_data[3]
name, confidence = engine.predict(image)
print(f"True: {CLASS_NAMES[label]} | Predicted: {name} ({confidence:.4f})")
```

```
True: Trouser | Predicted: Trouser (1.0000)
```

Command-Line Interface

A command-line entry point turns the notebook into a small tool you can run from the terminal.

```
def build_parser():
    """
    Build the command-line argument parser.

    Parameters:
        None

    Returns:
        argparse.ArgumentParser: The configured parser.
    """
    parser = argparse.ArgumentParser(description='Classify a Fashion-MNIST image.')
    parser.add_argument('--index', type=int, default=0, help='Test image index.')
    return parser

def main():
    """
    Run the command-line inference engine.

    Parameters:
        None

    Returns:
        None
    """
    args = build_parser().parse_args()
    engine = InferenceEngine('cnn_fashion_mnist.onnx')
    image, label = test_data[args.index]
    name, confidence = engine.predict(image)
    print(f"Index {args.index} | true {CLASS_NAMES[label]} | predicted {name} "
          f"({confidence:.4f})")
```

Run the CLI

```
import sys
sys.argv = ['deploy', '--index', '3']
main()
```

```
Index 3 | true Trouser | predicted Trouser (1.0000)
```

26-SEG: Oxford-IIIT Pet Dataset - Semantic Segmentation

Oxford-IIIT Pet Dataset

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, C. V. Jawahar,
<https://www.robots.ox.ac.uk/~vgg/data/pets/>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy matplotlib
```

Import Libraries

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.utils.data import Subset
from torchvision import datasets
from torchvision import transforms
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x11f6d1b30>
```

Parameters

IMAGE_SIZE = 128
IMAGE_SIZE

128

NUM_CLASSES = 3
NUM_CLASSES

3

BASE_CHANNELS = 16
BASE_CHANNELS

16

SUBSET_SIZE = 2000
SUBSET_SIZE

2000

Hyperparameters

LEARNING_RATE = 0.001
LEARNING_RATE

0.001

EPOCHS = 8
EPOCHS

8

BATCH_SIZE = 16
BATCH_SIZE

16

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Load Dataset

Each image comes with a segmentation mask that labels every pixel as pet, background, or boundary. The raw mask values are 1, 2, and 3, so we shift them to 0, 1, and 2.

```
def mask_transform(mask):
    """
    Resize a raw mask and shift its values into class indices.

    Parameters:
        mask (PIL.Image): The raw segmentation mask.

    Returns:
        torch.Tensor: A mask with values in 0, 1, 2.
    """
    mask = transforms.functional.resize(
        mask, [IMAGE_SIZE, IMAGE_SIZE],
        interpolation=transforms.InterpolationMode.NEAREST)
    mask = torch.as_tensor(np.array(mask), dtype=torch.long)
    return (mask - 1).clamp(0, NUM_CLASSES - 1)
```

```
image_transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.ToTensor()])
full_data = datasets.OxfordIIITPet('data', split='trainval', download=True,
                                   transform=image_transform,
                                   target_types='segmentation',
                                   target_transform=mask_transform)
data = Subset(full_data, range(min(SUBSET_SIZE, len(full_data))))
len(data)
```

```
2000
```

Create DataLoaders

```
split = int(0.8 * len(data))
train_data = Subset(data, range(split))
test_data = Subset(data, range(split, len(data)))
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)
len(train_data), len(test_data)
```

```
(1600, 400)
```

Create Model

A U-Net first downsamples the image into a compact representation, then upsamples it back while reusing the encoder features to keep fine detail.

```
class DoubleConv(nn.Module):
    """
    A block of two convolutions with ReLU activations.
    """
    def __init__(self, in_channels, out_channels):
        """
        Initialize the two convolutions.

        Parameters:
            in_channels (int): Number of input channels.
            out_channels (int): Number of output channels.

        Returns:
            None
        """
        super(DoubleConv, self).__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(out_channels, out_channels, 3, padding=1),
            nn.ReLU())

    def forward(self, x):
        """
        Apply the two convolutions.

        Parameters:
            x (torch.Tensor): Input feature map.

        Returns:
            torch.Tensor: Output feature map.
        """
        return self.net(x)
```

```

class UNet(nn.Module):
    """
    A small U-Net for semantic segmentation.
    """

    def __init__(self, in_channels=3, num_classes=NUM_CLASSES, base=BASE_CHANNELS):
        """
        Initialize the encoder, bottleneck, decoder, and head.

        Parameters:
            in_channels (int): Number of input channels.
            num_classes (int): Number of output classes.
            base (int): Number of channels in the first block.

        Returns:
            None
        """
        super(UNet, self).__init__()
        self.enc1 = DoubleConv(in_channels, base)
        self.enc2 = DoubleConv(base, base * 2)
        self.bottleneck = DoubleConv(base * 2, base * 4)
        self.up = nn.ConvTranspose2d(base * 4, base * 2, 2, stride=2)
        self.dec = DoubleConv(base * 4, base * 2)
        self.head = nn.Conv2d(base * 2, num_classes, 1)

    def forward(self, x):
        """
        Segment a batch of images.

        Parameters:
            x (torch.Tensor): Batch of images.

        Returns:
            torch.Tensor: Per-pixel logits.
        """
        e1 = self.enc1(x)
        e2 = self.enc2(F.max_pool2d(e1, 2))
        bottleneck = self.bottleneck(F.max_pool2d(e2, 2))
        decoded = self.dec(torch.cat([self.up(bottleneck), e2], dim=1))
        logits = self.head(decoded)
        return F.interpolate(logits, size=x.shape[-2:], mode='bilinear',
                             align_corners=False)

```

Instantiate Model, Loss, and Optimizer

```

torch.manual_seed(SEED)
model = UNet().to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
model

```

```
UNet(
  (enc1): DoubleConv(
    (net): Sequential(
      (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (enc2): DoubleConv(
    (net): Sequential(
      (0): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (bottleneck): DoubleConv(
    (net): Sequential(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (up): ConvTranspose2d(64, 32, kernel_size=(2, 2), stride=(2, 2))
  (dec): DoubleConv(
    (net): Sequential(
      (0): Conv2d(64, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): ReLU()
      (2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (3): ReLU()
    )
  )
  (head): Conv2d(32, 3, kernel_size=(1, 1), stride=(1, 1))
)
```


Train Model

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(x)
    return total / len(loader.dataset)


def pixel_accuracy(model, loader):
    """
    Compute pixel accuracy over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Fraction of correctly classified pixels.
    """
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            preds = model(x).argmax(1)
            correct += (preds == y).sum().item()
            total += y.numel()
    return correct / total
```

Training Loop

```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    accuracy = pixel_accuracy(model, test_loader)
    history.append(accuracy)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | pixel accuracy {accuracy:.4f}")
```

Epoch 1 | train loss 0.9341 | pixel accuracy 0.5921

Epoch 2 | train loss 0.8231 | pixel accuracy 0.6015

Epoch 3 | train loss 0.7811 | pixel accuracy 0.6718

Epoch 4 | train loss 0.7528 | pixel accuracy 0.6633

Epoch 5 | train loss 0.7277 | pixel accuracy 0.6352

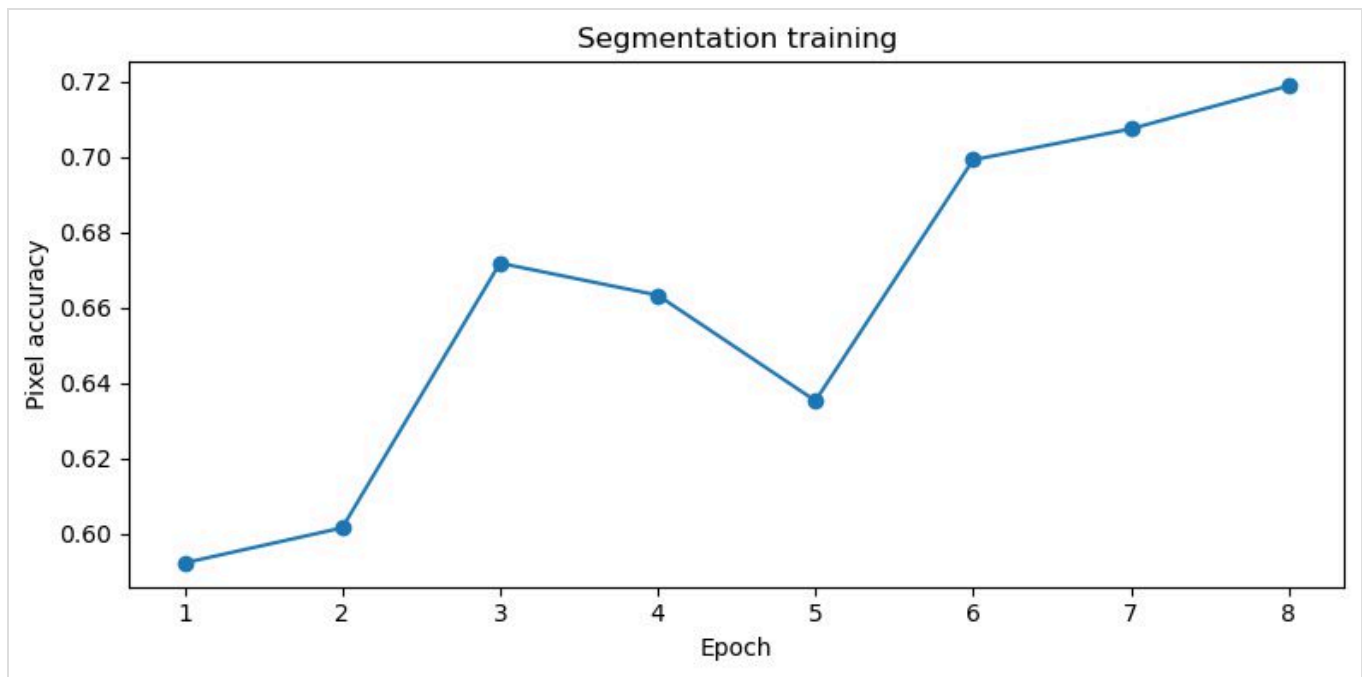
Epoch 6 | train loss 0.7030 | pixel accuracy 0.6993

Epoch 7 | train loss 0.6723 | pixel accuracy 0.7075

Epoch 8 | train loss 0.6469 | pixel accuracy 0.7189

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Pixel accuracy')
plt.title('Segmentation training')
plt.tight_layout()
plt.show()
```



Visualize Predictions

Functions

```
def plot_triplet(image, mask, preds):
    """
    Plot an image, its true mask, and the predicted mask.

    Parameters:
        image (torch.Tensor): The image.
        mask (torch.Tensor): The true mask.
        preds (torch.Tensor): The predicted mask.

    Returns:
        None
    """
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    titles = ['Image', 'True mask', 'Predicted mask']
    panels = [image.permute(1, 2, 0), mask, preds]
    for axis, title, panel in zip(axes, titles, panels):
        axis.imshow(panel, cmap=None if panel.dim() == 3 else 'viridis')
        axis.set_title(title)
        axis.axis('off')
    fig.tight_layout()
    plt.show()

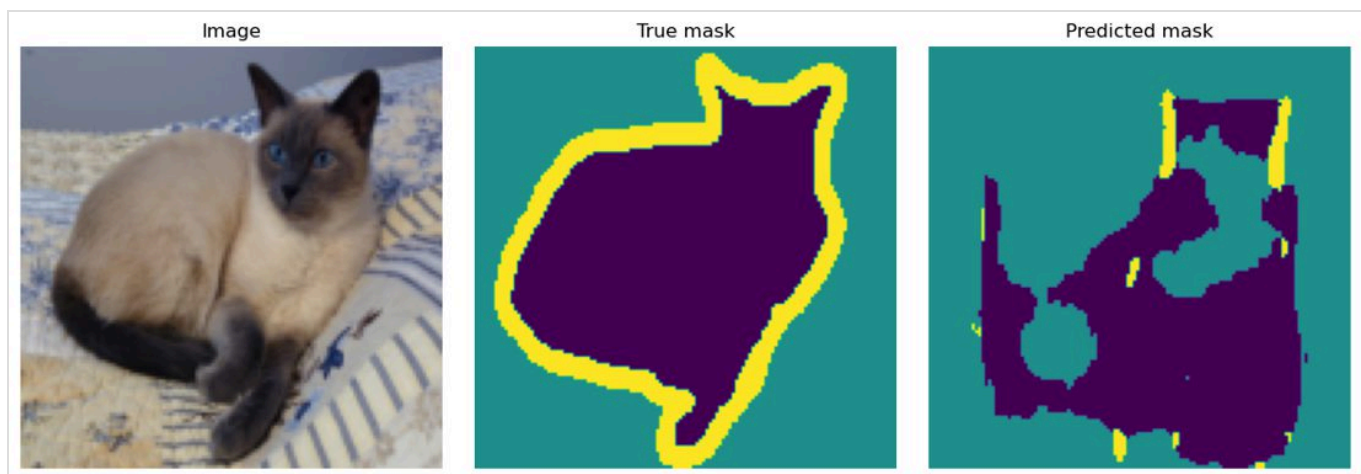
def show_prediction(model, dataset, index=0):
    """
    Plot an image, its true mask, and the predicted mask.

    Parameters:
        model (nn.Module): Trained model.
        dataset (Dataset): Source of images and masks.
        index (int): Sample index.

    Returns:
        None
    """
    model.eval()
    image, mask = dataset[index]
    with torch.no_grad():
        preds = model(image.unsqueeze(0).to(DEVICE)).argmax(1)[0].cpu()
    plot_triplet(image, mask, preds)
```

Run

```
show_prediction(model, test_data, index=0)
```



Save Model

```
torch.save(model.state_dict(), 'unet_oxford_pet.pt')
print('saved unet_oxford_pet.pt')
```

saved unet_oxford_pet.pt

Load Model

```
loaded_model = UNet().to(DEVICE)
loaded_model.load_state_dict(torch.load('unet_oxford_pet.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded unet_oxford_pet.pt')
```

loaded unet_oxford_pet.pt

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_56932/1089445556.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
```

```
loaded_model.load_state_dict(torch.load('unet_oxford_pet.pt', map_location=DEVICE))
```

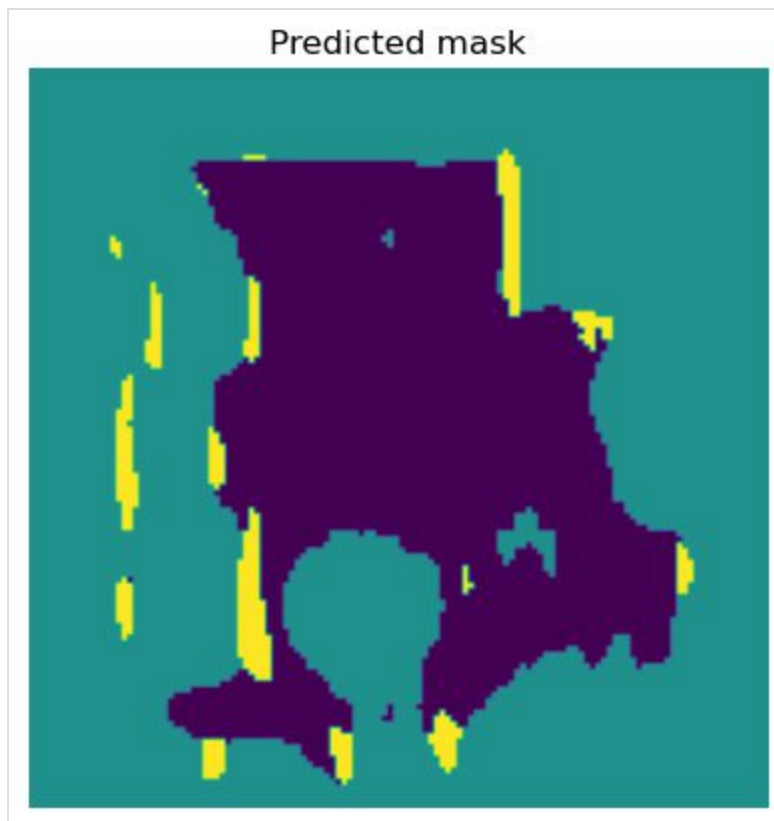
Inference

Function

```
def predict_mask(model, image):  
    """  
    Predict a segmentation mask for one image.  
  
    Parameters:  
        model (nn.Module): Trained model.  
        image (torch.Tensor): A 3xHxW image tensor.  
  
    Returns:  
        torch.Tensor: Predicted mask.  
    """  
    model.eval()  
    with torch.no_grad():  
        logits = model(image.unsqueeze(0).to(DEVICE))  
        return logits.argmax(1)[0].cpu()
```

Run Inference

```
image, mask = test_data[1]  
predicted = predict_mask(loader_model, image)  
plt.imshow(predicted, cmap='viridis')  
plt.title('Predicted mask')  
plt.axis('off')  
plt.show()
```



27-DRL: CartPole Environment - Deep Q-Network

CartPole Environment

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Volodymyr Mnih et al., <https://arxiv.org/abs/1312.5602>

[2] Gymnasium, <https://gymnasium.farama.org/>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install -c conda-forge gymnasium
# conda install numpy matplotlib
```

Import Libraries

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from collections import deque
import gymnasium as gym
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x120c49b10>
```

Parameters

STATE_DIM = 4
STATE_DIM

4

ACTION_DIM = 2
ACTION_DIM

2

HIDDEN_DIM = 128
HIDDEN_DIM

128

BUFFER_SIZE = 10000
BUFFER_SIZE

10000

Hyperparameters

LEARNING_RATE = 0.001
LEARNING_RATE

0.001

EPISODES = 300
EPISODES

300

BATCH_SIZE = 64
BATCH_SIZE

64


```
GAMMA = 0.99
GAMMA
```

```
0.99
```

Device

```
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu")
DEVICE
```

```
device(type='mps')
```

Create Environment

```
env = gym.make('CartPole-v1')
state, _ = env.reset(seed=SEED)
print('state dim:', env.observation_space.shape[0])
print('action dim:', env.action_space.n)
```

```
state dim: 4
action dim: 2
```

Create Replay Buffer

```

class ReplayBuffer:
    """
    A fixed-size experience replay buffer.
    """

    def __init__(self, capacity):
        """
        Initialize the buffer with a fixed capacity.

        Parameters:
            capacity (int): Maximum number of transitions.

        Returns:
            None
        """
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """
        Store one transition.

        Parameters:
            state (np.ndarray): Current state.
            action (int): Action taken.
            reward (float): Reward received.
            next_state (np.ndarray): Next state.
            done (bool): Whether the episode ended.

        Returns:
            None
        """
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """
        Sample a random batch of transitions.

        Parameters:
            batch_size (int): Number of transitions.

        Returns:
            tuple: Batched transitions.
        """
        batch = random.sample(self.buffer, batch_size)
        return zip(*batch)

    def __len__(self):
        """
        Return the number of stored transitions.

        Parameters:
            None

        Returns:

```

```

        int: Number of transitions.
    """
    return len(self.buffer)

```

Create Q-Network

```

class QNetwork(nn.Module):
    """
    A small Q-network that scores each action.
    """

    def __init__(self, state_dim=STATE_DIM, action_dim=ACTION_DIM, hidden=HIDDEN_DIM):
        """
        Initialize the network layers.

        Parameters:
            state_dim (int): Number of state features.
            action_dim (int): Number of actions.
            hidden (int): Hidden layer size.

        Returns:
            None
        """
        super(QNetwork, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, action_dim))

    def forward(self, x):
        """
        Score every action for a batch of states.

        Parameters:
            x (torch.Tensor): Batch of states.

        Returns:
            torch.Tensor: Q-values per action.
        """
        return self.net(x)

```

Instantiate Networks and Optimizer

```

torch.manual_seed(SEED)
policy = QNetwork().to(DEVICE)
target = QNetwork().to(DEVICE)
target.load_state_dict(policy.state_dict())
optimizer = torch.optim.Adam(policy.parameters(), lr=LEARNING_RATE)
buffer = ReplayBuffer(BUFFER_SIZE)
policy

```

```
QNetwork(  
  (net): Sequential(  
    (0): Linear(in_features=4, out_features=128, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=128, out_features=128, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=128, out_features=2, bias=True)  
  )  
)
```

Train Model

Functions

```
def to_tensor(array, dtype):
    """
    Convert a NumPy array to a tensor on the device.

    Parameters:
        array (np.ndarray): Input array.
        dtype (torch.dtype): Target dtype.

    Returns:
        torch.Tensor: Tensor on the device.
    """
    return torch.tensor(np.array(array), dtype=dtype, device=DEVICE)


def select_action(policy, state, epsilon, action_dim):
    """
    Choose an action with epsilon-greedy exploration.

    Parameters:
        policy (nn.Module): The Q-network.
        state (np.ndarray): Current state.
        epsilon (float): Exploration probability.
        action_dim (int): Number of actions.

    Returns:
        int: The chosen action.
    """
    if random.random() < epsilon:
        return random.randrange(action_dim)
    with torch.no_grad():
        state_tensor = to_tensor(state, torch.float32).unsqueeze(0)
        return int(policy(state_tensor).argmax(1).item())


def unpack_batch(buffer, batch_size):
    """
    Sample and convert a batch of transitions.

    Parameters:
        buffer (ReplayBuffer): The replay buffer.
        batch_size (int): Number of transitions.

    Returns:
        tuple: Converted tensors.
    """
    states, actions, rewards, next_states, dones = buffer.sample(batch_size)
    return (to_tensor(states, torch.float32),
            to_tensor(actions, torch.long),
            to_tensor(rewards, torch.float32),
            to_tensor(next_states, torch.float32),
            to_tensor(dones, torch.float32))
```

```

def dqn_loss(policy, target, states, actions, rewards, next_states, dones, gamma):
    """
    Compute the temporal-difference loss.

    Parameters:
        policy (nn.Module): The online network.
        target (nn.Module): The target network.
        states (torch.Tensor): Batch of states.
        actions (torch.Tensor): Batch of actions.
        rewards (torch.Tensor): Batch of rewards.
        next_states (torch.Tensor): Batch of next states.
        dones (torch.Tensor): Batch of done flags.
        gamma (float): Discount factor.

    Returns:
        torch.Tensor: The loss.
    """
    q_values = policy(states).gather(1, actions.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        next_q = target(next_states).max(1).values
        targets = rewards + gamma * next_q * (1 - dones)
    return F.smooth_l1_loss(q_values, targets)


def optimize(policy, target, buffer, optimizer, batch_size, gamma):
    """
    Run one gradient step on a sampled batch.

    Parameters:
        policy (nn.Module): The online network.
        target (nn.Module): The target network.
        buffer (ReplayBuffer): The replay buffer.
        optimizer (torch.optim.Optimizer): Parameter update rule.
        batch_size (int): Number of transitions.
        gamma (float): Discount factor.

    Returns:
        float: The loss value.
    """
    batch = unpack_batch(buffer, batch_size)
    loss = dqn_loss(policy, target, *batch, gamma)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()

```

Training Loop

```
def train(env, policy, target, buffer, optimizer, episodes):
    """
    Train the agent over many episodes.

    Parameters:
        env (gym.Env): The environment.
        policy (nn.Module): The online network.
        target (nn.Module): The target network.
        buffer (ReplayBuffer): The replay buffer.
        optimizer (torch.optim.Optimizer): Parameter update rule.
        episodes (int): Number of episodes.

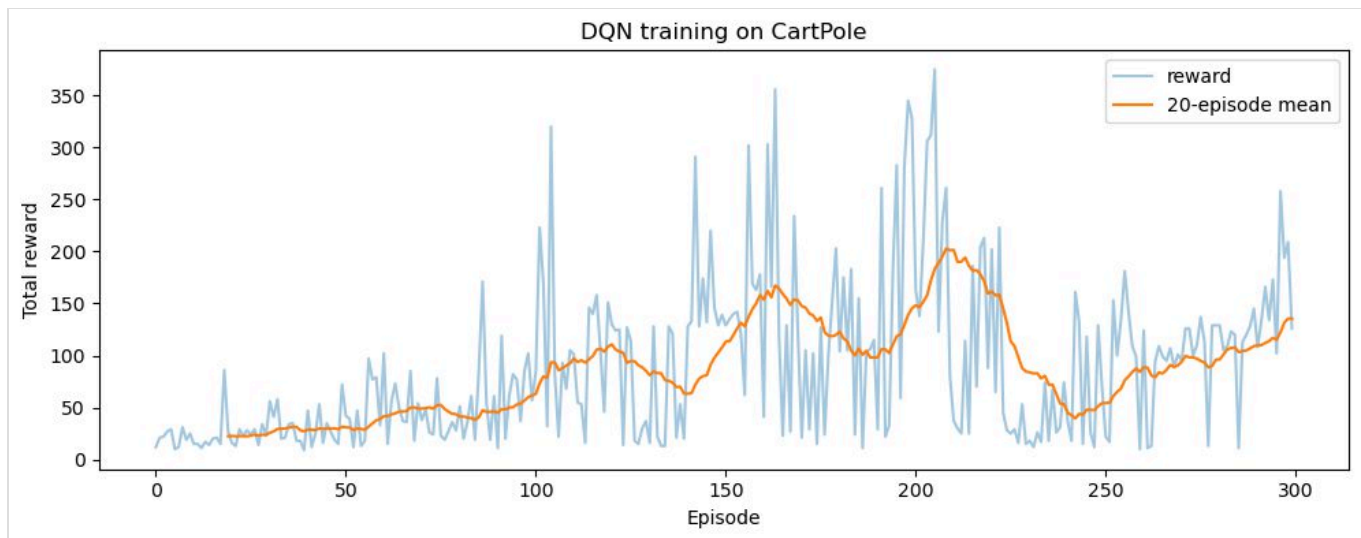
    Returns:
        list: Total reward per episode.
    """
    rewards_history = []
    epsilon = 1.0
    for episode in range(episodes):
        state, _ = env.reset(seed=episode)
        done = False
        total_reward = 0.0
        while not done:
            action = select_action(policy, state, epsilon, env.action_space.n)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated
            buffer.push(state, action, reward, next_state, done)
            state = next_state
            total_reward += reward
            if len(buffer) >= BATCH_SIZE:
                optimize(policy, target, buffer, optimizer, BATCH_SIZE, GAMMA)
        epsilon = max(0.05, epsilon * 0.99)
        if episode % 10 == 0:
            target.load_state_dict(policy.state_dict())
            rewards_history.append(total_reward)
    return rewards_history
```

```
history = train(env, policy, target, buffer, optimizer, EPISODES)
print('last 20 mean reward:', np.mean(history[-20:]))
```

```
last 20 mean reward: 135.35
```

Visualize Training

```
plt.figure(figsize=(10, 4))
plt.plot(history, alpha=0.4, label='reward')
window = np.convolve(history, np.ones(20) / 20, mode='valid')
plt.plot(range(19, len(history)), window, label='20-episode mean')
plt.xlabel('Episode')
plt.ylabel('Total reward')
plt.title('DQN training on CartPole')
plt.legend()
plt.tight_layout()
plt.show()
```



Evaluate Model

Function

```
def evaluate(policy, episodes=5):
    """
    Run the greedy policy and report mean reward.

    Parameters:
        policy (nn.Module): Trained network.
        episodes (int): Number of evaluation episodes.

    Returns:
        float: Mean reward.
    """
    policy.eval()
    env_eval = gym.make('CartPole-v1')
    rewards = []
    for episode in range(episodes):
        state, _ = env_eval.reset(seed=episode)
        done = False
        total = 0.0
        while not done:
            action = select_action(policy, state, 0.0, env_eval.action_space.n)
            state, reward, terminated, truncated, _ = env_eval.step(action)
            done = terminated or truncated
            total += reward
        rewards.append(total)
    env_eval.close()
    return float(np.mean(rewards))
```

Run

```
print(f"Mean evaluation reward: {evaluate(policy):.2f}")
```

```
Mean evaluation reward: 116.60
```

Save Model

```
torch.save(policy.state_dict(), 'dqn_cartpole.pt')
print('saved dqn_cartpole.pt')
```

```
saved dqn_cartpole.pt
```

Load Model

```
loaded_policy = QNetwork().to(DEVICE)
loaded_policy.load_state_dict(torch.load('dqn_cartpole.pt', map_location=DEVICE))
loaded_policy.eval()
print('loaded dqn_cartpole.pt')
```

```
loaded dqn_cartpole.pt
```

```
/var/folders/qg/60m3b34x32gczgr10l3665300000gn/T/ipykernel_55873/1403255126.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
  loaded_policy.load_state_dict(torch.load('dqn_cartpole.pt', map_location=DEVICE))
```

Inference

```
print(f"Mean evaluation reward: {evaluate(loaded_policy):.2f}")
```

```
Mean evaluation reward: 116.60
```

28-LLM: Tiny Shakespeare Dataset - Fine-Tuning a Pretrained LLM

[Tiny Shakespeare Dataset](#)

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Andrej Karpathy, <https://github.com/karpathy/char-rnn>

[2] Alec Radford et al., https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[3] Hugging Face Transformers, <https://huggingface.co/docs/transformers>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# pip install transformers
# conda install numpy matplotlib
```

Import Libraries

```
import urllib.request
import numpy as np
import torch
from torch.utils.data import DataLoader
from transformers import GPT2LMHeadModel
from transformers import GPT2TokenizerFast
import matplotlib.pyplot as plt
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x127b49b30>
```

Parameters

```
MODEL_NAME = 'distilgpt2'  
MODEL_NAME
```

```
'distilgpt2'
```

```
BLOCK_SIZE = 128  
BLOCK_SIZE
```

```
128
```

Hyperparameters

```
LEARNING_RATE = 0.00005  
LEARNING_RATE
```

```
5e-05
```

```
EPOCHS = 2  
EPOCHS
```

```
2
```

```
BATCH_SIZE = 16  
BATCH_SIZE
```

```
16
```

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Download Dataset

```
URL = 'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt'
urllib.request.urlretrieve(URL, 'tinyshakespeare.txt')
with open('tinyshakespeare.txt', encoding='utf-8') as handle:
    text = handle.read()
print('length:', len(text))
```

```
length: 1115394
```

Tokenize Dataset

The GPT-2 tokenizer splits text into subword tokens. We concatenate the token ids and cut them into fixed-length blocks.

```
def tokenize_blocks(tokenizer, text, block_size):
    """
    Tokenize text and cut it into fixed-length blocks.

    Parameters:
        tokenizer (GPT2TokenizerFast): The tokenizer.
        text (str): The training text.
        block_size (int): Length of each block.

    Returns:
        torch.Tensor: A 2-D tensor of token ids.
    """
    ids = tokenizer(text)['input_ids']
    n_blocks = len(ids) // block_size
    trimmed = ids[:n_blocks * block_size]
    return torch.tensor(trimmed, dtype=torch.long).view(n_blocks, block_size)
```

Run

```
tokenizer = GPT2TokenizerFast.from_pretrained(MODEL_NAME)
tokenizer.pad_token = tokenizer.eos_token
blocks = tokenize_blocks(tokenizer, text, BLOCK_SIZE)
blocks.shape
```

```
tokenizer_config.json:  0%|          | 0.00/26.0 [00:00<?, ?B/s]
```

```
vocab.json: 0.00B [00:00, ?B/s]
```

```
merges.txt: 0.00B [00:00, ?B/s]
```

```
tokenizer.json: 0.00B [00:00, ?B/s]
```

```
config.json: 0%|          | 0.00/762 [00:00<?, ?B/s]
```

Token indices sequence length is longer than the specified maximum sequence length for this model (338025 > 1024). Running this sequence through the model will result in indexing errors

```
torch.Size([2640, 128])
```

Create Dataset and DataLoader

```
class TextDataset(torch.utils.data.Dataset):
    """
    A dataset of fixed-length token blocks.
    """

    def __init__(self, blocks):
        """
        Store the token blocks.

        Parameters:
            blocks (torch.Tensor): A 2-D tensor of token ids.

        Returns:
            None
        """
        self.blocks = blocks

    def __len__(self):
        """
        Return the number of blocks.

        Parameters:
            None

        Returns:
            int: Number of blocks.
        """
        return len(self.blocks)

    def __getitem__(self, idx):
        """
        Return one block as input and target.

        Parameters:
            idx (int): Block index.

        Returns:
            tuple: Input and target tensors.
        """
        block = self.blocks[idx]
        return block, block.clone()
```

```
split = int(0.9 * len(blocks))
train_loader = DataLoader(TextDataset(blocks[:split]), batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(TextDataset(blocks[split:]), batch_size=BATCH_SIZE, shuffle=False)
len(train_loader), len(val_loader)
```

(149, 17)

Load Pretrained Model

We start from DistilGPT-2, a small pretrained language model, and fine-tune it on Shakespeare. This is far cheaper than training from scratch.

```
model = GPT2LMHeadModel.from_pretrained(MODEL_NAME).to(DEVICE)
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
sum(p.numel() for p in model.parameters())
```

```
model.safetensors:  0%|          | 0.00/353M [00:00<?, ?B/s]
```

```
generation_config.json:  0%|          | 0.00/124 [00:00<?, ?B/s]
```

```
81912576
```


Train Model

Functions

```
def train_epoch(model, loader, optimizer):
    """
    Fine-tune the model for a single epoch.

    Parameters:
        model (nn.Module): The language model.
        loader (DataLoader): Training data loader.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(input_ids=x, attention_mask=(x != tokenizer.pad_token_id).long(),
                        labels=y)
        outputs.loss.backward()
        optimizer.step()
        total += outputs.loss.item() * len(x)
    return total / len(loader.dataset)

def estimate_loss(model, loader):
    """
    Estimate the mean loss over a loader.

    Parameters:
        model (nn.Module): The language model.
        loader (DataLoader): Evaluation data loader.

    Returns:
        float: Mean loss.
    """
    model.eval()
    total = 0.0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            outputs = model(input_ids=x, attention_mask=(x !=
tokenizer.pad_token_id).long(),
                        labels=y)
            total += outputs.loss.item() * len(x)
    return total / len(loader.dataset)
```

Training Loop

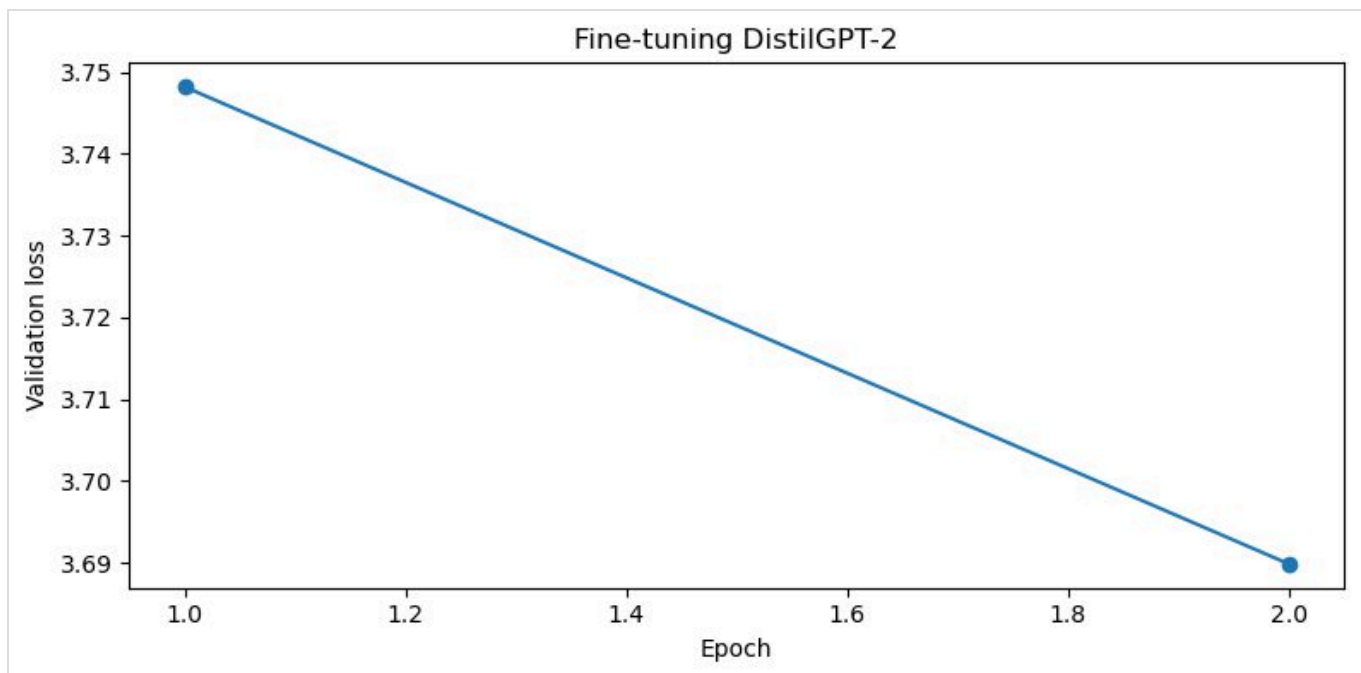
```
history = []
for epoch in range(EPOCHS):
    train_loss = train_epoch(model, train_loader, optimizer)
    val_loss = estimate_loss(model, val_loader)
    history.append(val_loss)
    print(f"Epoch {epoch + 1} | train loss {train_loss:.4f} | val loss {val_loss:.4f}")
```

Epoch 1 | train loss 4.0547 | val loss 3.7482

Epoch 2 | train loss 3.7320 | val loss 3.6898

Visualize Training

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, EPOCHS + 1), history, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Validation loss')
plt.title('Fine-tuning DistilGPT-2')
plt.tight_layout()
plt.show()
```



Generate Text

Function

```
def generate(model, prompt, max_new_tokens=100):
    """
    Generate text from a prompt.

    Parameters:
        model (nn.Module): Fine-tuned language model.
        prompt (str): Seed text.
        max_new_tokens (int): Number of tokens to generate.

    Returns:
        str: The generated text.
    """
    model.eval()
    inputs = tokenizer(prompt, return_tensors='pt').to(DEVICE)
    with torch.no_grad():
        output = model.generate(**inputs, max_new_tokens=max_new_tokens,
                                do_sample=True, top_k=50, temperature=0.8,
                                pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(output[0], skip_special_tokens=True)
```

Run

```
print(generate(model, 'ROMEO: ', max_new_tokens=120))
```

ROMEO: _____

ROMEO:
Nay, I have forgot.

ROMEO:
O, I forgot.

ROMEO:
I forgot.
COMINIUS:
I'll prove you wrong.

ROMEO:
Hear me yet?

ROMEO:
Yea, so, what is the purpose of the law?

ROMEO:
Nay, it is a law, and the law is.

ROMEO:
Hear me yet?

ROMEO:
Thou art not but a law

Save Model

```
model.save_pretrained('distilgpt2_shakespeare')  
tokenizer.save_pretrained('distilgpt2_shakespeare')  
print('saved distilgpt2_shakespeare')
```

```
saved distilgpt2_shakespeare
```

Load Model

```
loaded_model = GPT2LMHeadModel.from_pretrained('distilgpt2_shakespeare').to(DEVICE)  
loaded_model.eval()  
print('loaded distilgpt2_shakespeare')
```

```
loaded distilgpt2_shakespeare
```

Inference

```
print(generate(loaded_model, 'JULIET: ', max_new_tokens=120))
```

JULIET: ix

GLOUCESTER:

Well, I think it must be true, my lord.

JULIET:

I would, sir, I would, sir, have seen the man

Of a gentleman's face I had not seen.

GLOUCESTER:

You, sir?

JULIET:

That is the gentleman's face, sir, and that he

Is not

The man's face, sir.

JULIET:

I do remember the man; and you, sir,

What

Part VIII: A Kaggle Project from Scratch

29-KAGGLE: Kaggle Titanic Dataset - EDA and Binary Classifier

Titanic - Machine Learning from Disaster

Author: [Kevin Thomas](#)

License: MIT

Citation

[1] Kaggle, <https://www.kaggle.com/c/titanic>

[2] Data Science Dojo mirror, <https://github.com/datasciencedojo/datasets>

Install Libraries

```
# conda activate prod
# conda install -c conda-forge pytorch torchvision torchaudio
# conda install numpy pandas matplotlib seaborn scikit-learn
# pip install kaggle
```

What Is Kaggle

Kaggle is a platform for data science competitions and public datasets. A typical Kaggle project has five steps: create an account, download the data with the Kaggle API, explore the data, train a model, and submit predictions. This chapter walks through all five on the Titanic dataset, which asks you to predict which passengers survived.

Create a Kaggle Account

1. Go to kaggle.com and sign up for a free account.
2. Open the Titanic competition at kaggle.com/c/titanic.
3. Click **Join Competition** to accept the rules and unlock the data.

Get Your API Token

1. Click your profile picture, then **Settings**.
2. Under **API**, click **Create New Token**.
3. A file named `kaggle.json` downloads. Move it to `~/.kaggle/kaggle.json` and lock down its permissions.

```
# mkdir -p ~/.kaggle
# mv ~/Downloads/kaggle.json ~/.kaggle/kaggle.json
# chmod 600 ~/.kaggle/kaggle.json
```

Download the Dataset With the Kaggle CLI

With the token in place, the Kaggle CLI downloads the competition files. The `train.csv` file has labels; `test.csv` does not, because Kaggle scores your predictions on its own hidden labels.

```
# kaggle competitions download -c titanic -f train.csv -p data
# kaggle competitions download -c titanic -f test.csv -p data
# kaggle competitions download -c titanic -f gender_submission.csv -p data
```

Download With a Public Mirror

If you do not have a Kaggle token, this notebook falls back to a public mirror of the same Titanic training data so that it still runs end to end. The labeled file has 891 passengers, which we split ourselves.

Import Libraries

```
import os
import urllib.request
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Seed

```
SEED = 42
SEED
```

```
42
```

```
torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x13a5c8290>
```


Download Dataset

Function

```
def download_titanic(path='titanic.csv'):
    """
    Download the Titanic training data from a public mirror.

    Parameters:
        path (str): Destination CSV path.

    Returns:
        str: The path to the downloaded file.
    """
    url = ('https://raw.githubusercontent.com/datasciencedojo/'
          'datasets/master/titanic.csv')
    urllib.request.urlretrieve(url, path)
    return path
```

Run

```
if not os.path.exists('titanic.csv'):
    download_titanic('titanic.csv')
print('downloaded titanic.csv')
```

```
downloaded titanic.csv
```

Load Dataset

```
df = pd.read_csv('titanic.csv')
```

```
df.head()
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Exploratory Data Analysis

Dataset Shape and Types

```
print('rows:', df.shape[0])
print('columns:', df.shape[1])
```

```
rows: 891
columns: 12
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Summary Statistics

```
df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Missing Values

`Age` and `Cabin` have many missing values. `Embarked` has only two. We will impute the numeric columns with the median and the categorical columns with the mode.

```
df.isnull().sum()
```

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

Survival Rate

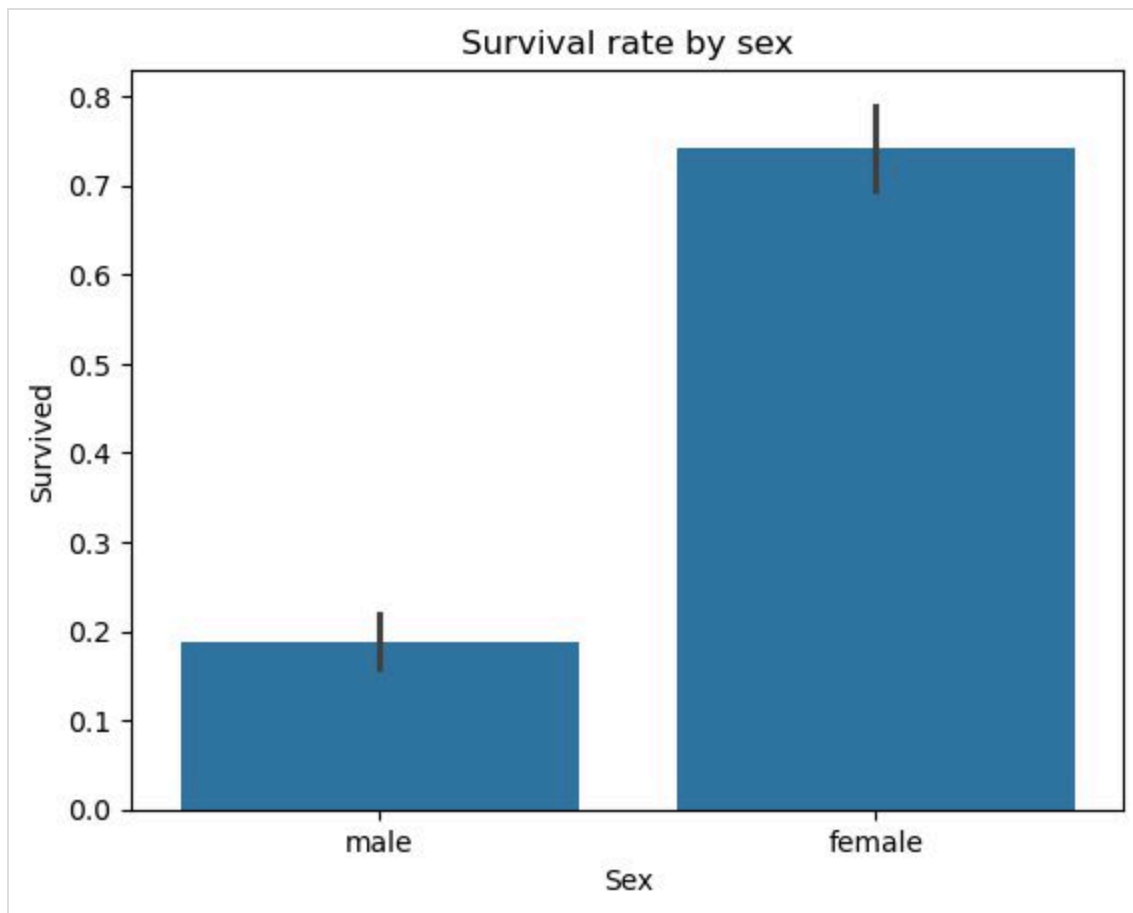
```
df['Survived'].value_counts(normalize=True).round(3)
```

```
Survived
0      0.616
1      0.384
Name: proportion, dtype: float64
```

Survival by Sex

Women survived at a far higher rate than men. **Sex** is the strongest single predictor in this dataset.

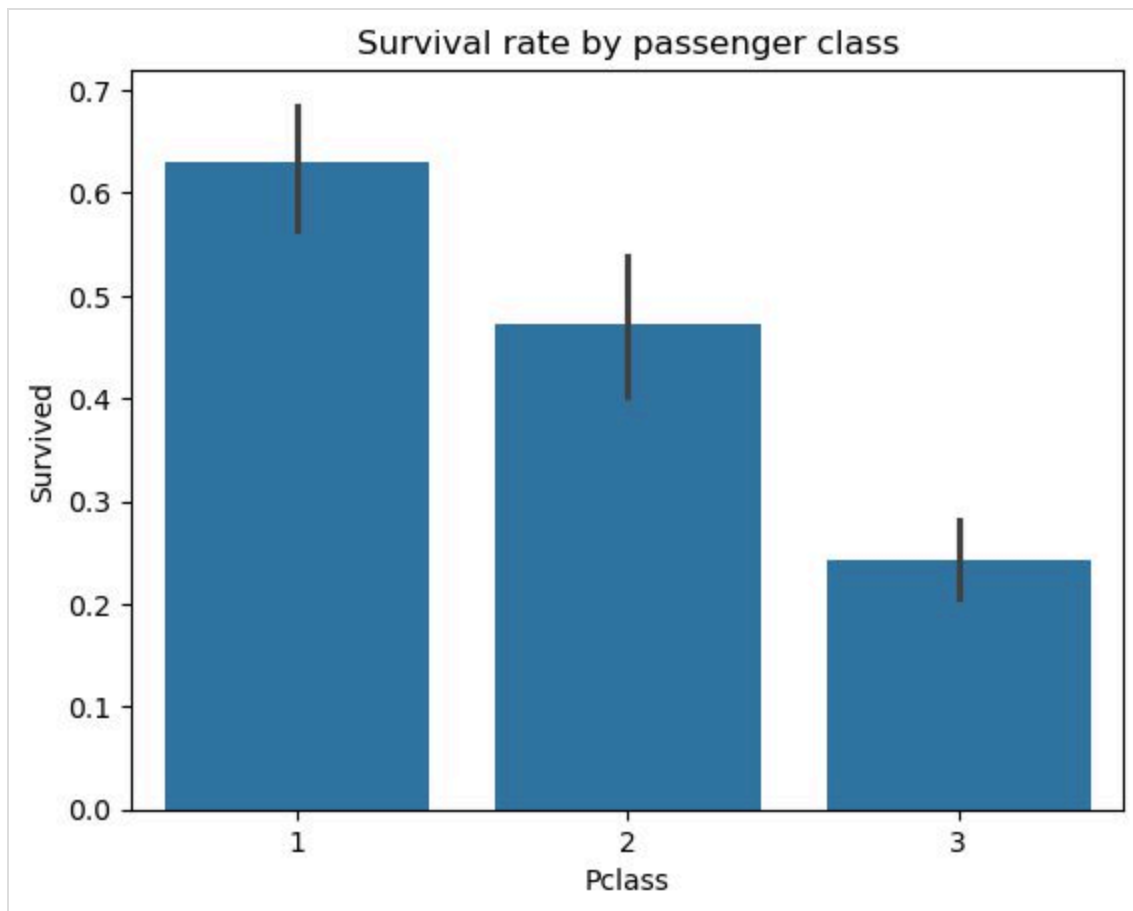
```
sns.barplot(data=df, x='Sex', y='Survived')
plt.title('Survival rate by sex')
plt.show()
```



Survival by Passenger Class

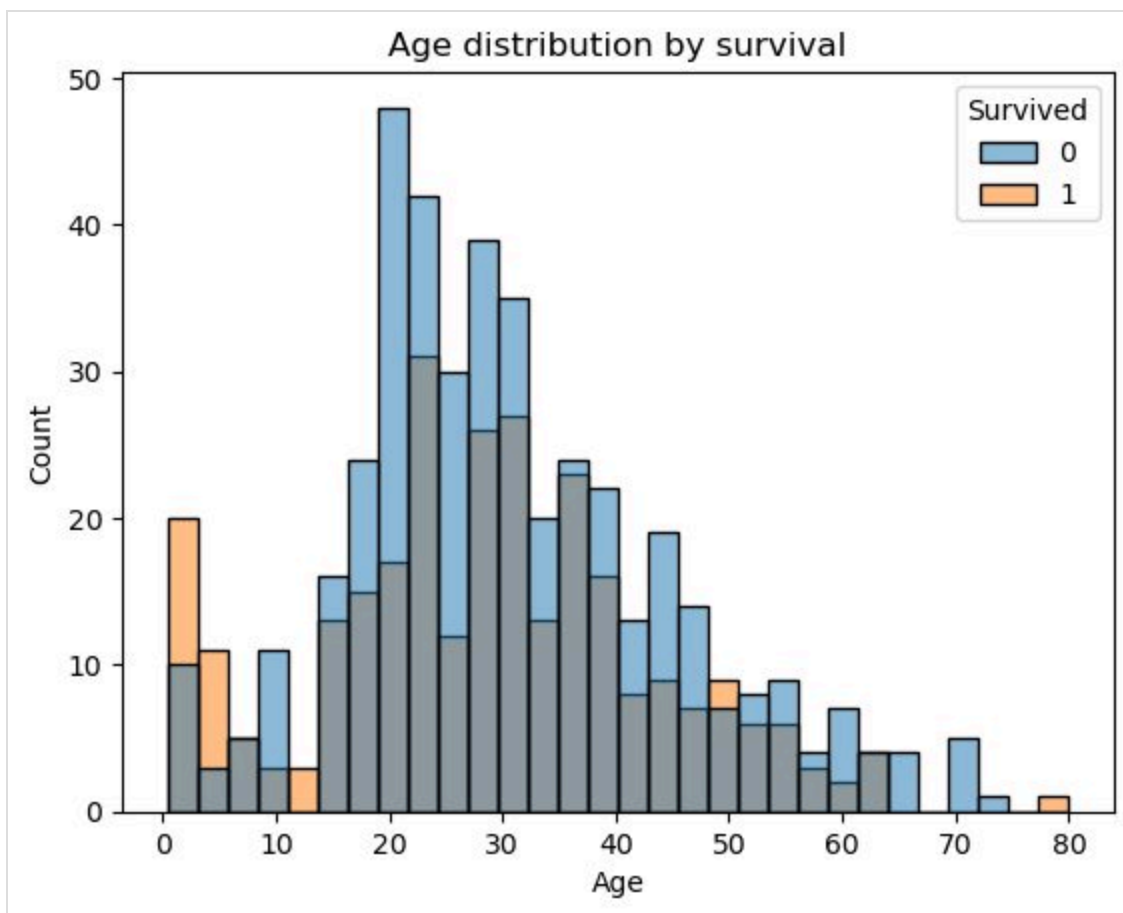
First-class passengers survived more often than third-class passengers.

```
sns.barplot(data=df, x='Pclass', y='Survived')
plt.title('Survival rate by passenger class')
plt.show()
```



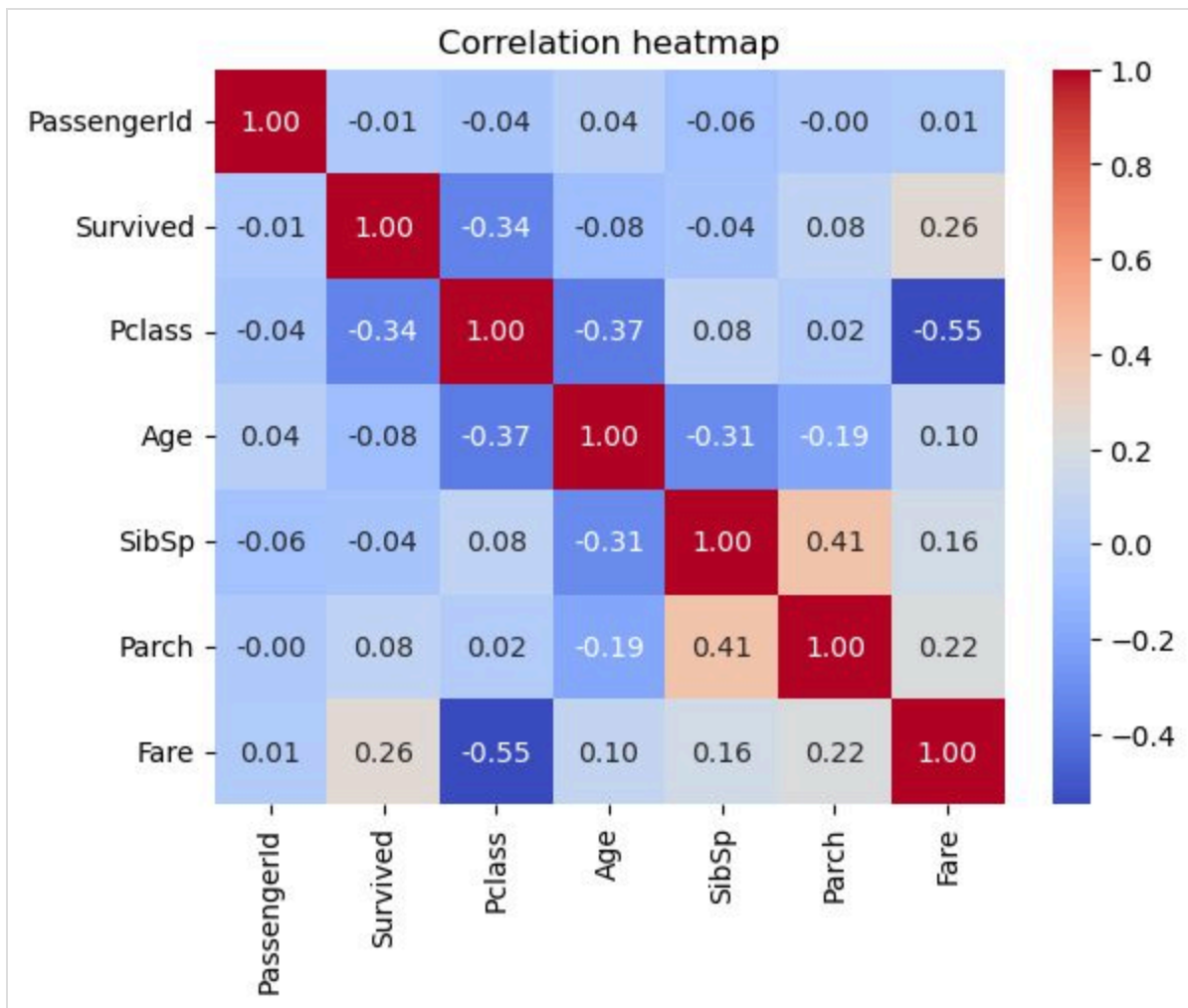
Age Distribution

```
sns.histplot(data=df, x='Age', hue='Survived', bins=30)
plt.title('Age distribution by survival')
plt.show()
```



Correlation Heatmap

```
numeric = df.select_dtypes(include='number')
sns.heatmap(numeric.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation heatmap')
plt.show()
```



Feature Engineering

Functions

```
def add_family_features(df):
    """
    Add family size and alone features.

    Parameters:
        df (pandas.DataFrame): The Titanic data.

    Returns:
        pandas.DataFrame: The data with new features.
    """
    df = df.copy()
    df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
    df['IsAlone'] = (df['FamilySize'] == 1).astype(int)
    return df


def impute_missing(df):
    """
    Fill missing values with medians and modes.

    Parameters:
        df (pandas.DataFrame): The Titanic data.

    Returns:
        pandas.DataFrame: The imputed data.
    """
    df = df.copy()
    df['Age'] = df['Age'].fillna(df['Age'].median())
    df['Fare'] = df['Fare'].fillna(df['Fare'].median())
    df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
    return df


def encode_features(df):
    """
    Encode categorical columns as numbers.

    Parameters:
        df (pandas.DataFrame): The Titanic data.

    Returns:
        pandas.DataFrame: The encoded data.
    """
    df = df.copy()
    df['Sex'] = (df['Sex'] == 'male').astype(int)
    df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
    return df
```

Run

```
df = add_family_features(df)
df = impute_missing(df)
df = encode_features(df)
df[['Sex', 'Age', 'Fare', 'FamilySize', 'IsAlone', 'Embarked']].head()
```

	Sex	Age	Fare	FamilySize	IsAlone	Embarked
0	1	22.0	7.2500	2	0	0
1	0	38.0	71.2833	2	0	1
2	0	26.0	7.9250	1	1	0
3	0	35.0	53.1000	2	0	0
4	1	35.0	8.0500	1	1	0

Prepare Features and Target

```
FEATURES = ['Pclass', 'Sex', 'Age', 'Fare', 'FamilySize', 'IsAlone', 'Embarked']
X = df[FEATURES].to_numpy(dtype=np.float32)
y = df['Survived'].to_numpy(dtype=np.int64)
X_MEAN = X.mean(axis=0)
X_STD = X.std(axis=0)
X = (X - X_MEAN) / X_STD
X.shape, y.shape
```

```
((891, 7), (891,))
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y)
X_train.shape, X_test.shape
```

```
((712, 7), (179, 7))
```

Baseline Classifier: Logistic Regression

```
baseline = LogisticRegression(max_iter=1000).fit(X_train, y_train)
baseline_preds = baseline.predict(X_test)
baseline_probs = baseline.predict_proba(X_test)[: , 1]
print('accuracy:', round(accuracy_score(y_test, baseline_preds), 4))
print('roc auc :', round(roc_auc_score(y_test, baseline_probs), 4))
```

```
accuracy: 0.7933
roc auc : 0.8462
```

PyTorch Classifier

Device

```
DEVICE = torch.device(  
    "mps" if torch.backends.mps.is_available()  
    else "cuda" if torch.cuda.is_available()  
    else "cpu")  
DEVICE
```

```
device(type='mps')
```

Create Dataset

```
class NumpyDataset(torch.utils.data.Dataset):
    """
    A memory-efficient dataset that stores data as NumPy arrays.
    """

    def __init__(self, X, y, device):
        """
        Initialize the dataset with NumPy arrays.

        Parameters:
            X (np.ndarray): Feature array.
            y (np.ndarray): Label array.
            device (torch.device): Device to move tensors to.

        Returns:
            None
        """
        self.X = X
        self.y = y
        self.device = device

    def __len__(self):
        """
        Return the number of samples in the dataset.

        Parameters:
            None

        Returns:
            int: Number of samples.
        """
        return len(self.X)

    def __getitem__(self, idx):
        """
        Return a single sample as tensors on the device.

        Parameters:
            idx (int): Index of the sample.

        Returns:
            tuple: Feature and label tensors.
        """
        X_tensor = torch.tensor(self.X[idx]).float().to(self.device)
        y_tensor = torch.tensor(self.y[idx]).long().to(self.device)
        return X_tensor, y_tensor
```

Create Model

```
class Model(nn.Module):
    """
    A small feedforward classifier for tabular data.
    """

    def __init__(self, in_features, h1=32, h2=16, out_features=2):
        """
        Initialize the neural network layers.

        Parameters:
            in_features (int): Number of input features.
            h1 (int): Neurons in the first hidden layer.
            h2 (int): Neurons in the second hidden layer.
            out_features (int): Number of output features.

        Returns:
            None
        """
        super(Model, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, h1),
            nn.ReLU(),
            nn.Linear(h1, h2),
            nn.ReLU(),
            nn.Linear(h2, out_features))

    def forward(self, x):
        """
        Run the forward pass of the network.

        Parameters:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output logits.
        """
        return self.net(x)
```

Create DataLoaders, Loss, and Optimizer

```
train_loader = DataLoader(NumpyDataset(X_train, y_train, DEVICE),
                          batch_size=32, shuffle=True)
test_loader = DataLoader(NumpyDataset(X_test, y_test, DEVICE),
                        batch_size=32, shuffle=False)
model = Model(len(FEATURES)).to(DEVICE)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
model
```

```
Model(  
  (net): Sequential(  
    (0): Linear(in_features=7, out_features=32, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=32, out_features=16, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=16, out_features=2, bias=True)  
  )  
)
```

Functions

```
def train_epoch(model, loader, loss_fn, optimizer):
    """
    Train the model for a single epoch.

    Parameters:
        model (nn.Module): The model to train.
        loader (DataLoader): Training data loader.
        loss_fn (nn.Module): Loss function.
        optimizer (torch.optim.Optimizer): Parameter update rule.

    Returns:
        float: Mean training loss.
    """
    model.train()
    total = 0.0
    for x, y in loader:
        optimizer.zero_grad()
        loss = loss_fn(model(x), y)
        loss.backward()
        optimizer.step()
        total += loss.item() * len(y)
    return total / len(loader.dataset)


def evaluate(model, loader, loss_fn):
    """
    Evaluate the model over a loader.

    Parameters:
        model (nn.Module): The model to evaluate.
        loader (DataLoader): Evaluation data loader.
        loss_fn (nn.Module): Loss function.

    Returns:
        tuple: Mean loss and accuracy.
    """
    model.eval()
    total, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            logits = model(x)
            total += loss_fn(logits, y).item() * len(y)
            correct += (logits.argmax(1) == y).sum().item()
    n = len(loader.dataset)
    return total / n, correct / n
```

Training Loop

```
for epoch in range(200):
    train_loss = train_epoch(model, train_loader, loss_fn, optimizer)
    test_loss, test_accuracy = evaluate(model, test_loader, loss_fn)
    if (epoch + 1) % 40 == 0:
        print(f"Epoch {epoch + 1:3d} | loss {test_loss:.4f} | accuracy
{test_accuracy:.4f}")
```

Epoch 40 | loss 0.4529 | accuracy 0.7989

Epoch 80 | loss 0.4894 | accuracy 0.8045

Epoch 120 | loss 0.6149 | accuracy 0.7821

Epoch 160 | loss 0.7025 | accuracy 0.7598

Epoch 200 | loss 0.8675 | accuracy 0.7709

Evaluate the PyTorch Classifier

```
_, accuracy = evaluate(model, test_loader, loss_fn)
print(f"Test accuracy: {accuracy:.4f}")
```

Test accuracy: 0.7709

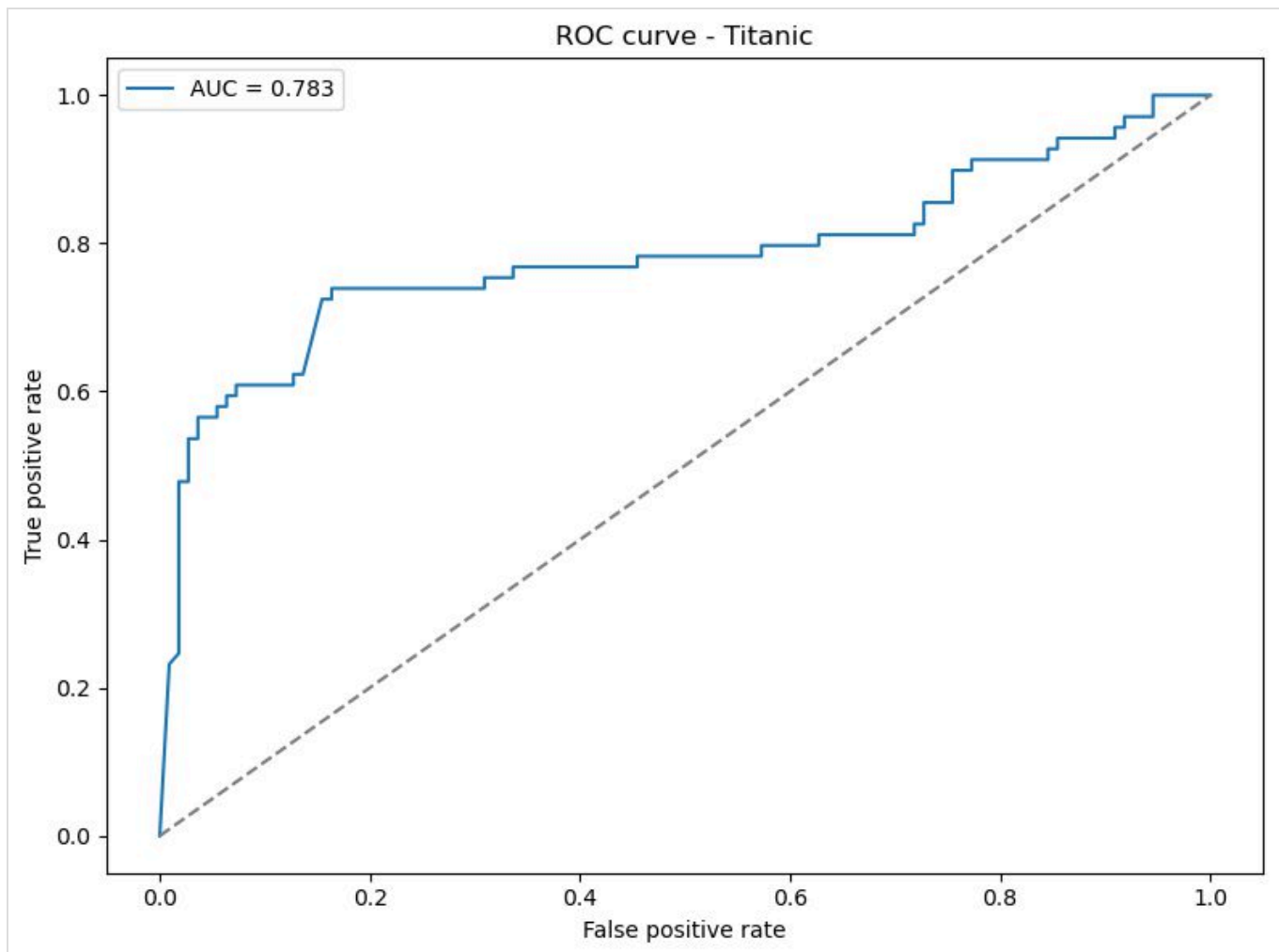
```
model.eval()
with torch.no_grad():
    logits = model(torch.tensor(X_test, dtype=torch.float32).to(DEVICE))
    probs = torch.softmax(logits, dim=1)[: , 1].cpu().numpy()
    preds = (probs >= 0.5).astype(int)
    print(confusion_matrix(y_test, preds))
    print(classification_report(y_test, preds, target_names=['died', 'survived']))
```

```
[[96 14]
 [27 42]]
```

	precision	recall	f1-score	support
died	0.78	0.87	0.82	110
survived	0.75	0.61	0.67	69
accuracy			0.77	179
macro avg	0.77	0.74	0.75	179
weighted avg	0.77	0.77	0.77	179

ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, probs)
auc_score = roc_auc_score(y_test, probs)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.3f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve - Titanic')
plt.legend()
plt.tight_layout()
plt.show()
```



Create a Kaggle Submission

A Kaggle submission is a CSV with an `Id` column and the predicted labels. On the real competition you would use the provided `test.csv`, whose `Id` column matches `gender_submission.csv`.

Function

```
def build_submission(ids, predictions, path='submission.csv'):
    """
    Write a Kaggle-format submission CSV.

    Parameters:
        ids (np.ndarray): Passenger ids.
        predictions (np.ndarray): Predicted survival labels.
        path (str): Output CSV path.

    Returns:
        str: The path to the submission file.
    """
    frame = pd.DataFrame({'PassengerId': ids, 'Survived': predictions})
    frame.to_csv(path, index=False)
    return path
```

Run

```
indices = np.arange(len(df))
_, test_indices = train_test_split(indices, test_size=0.2, random_state=SEED, stratify=y)
test_ids = df['PassengerId'].to_numpy()[test_indices]
build_submission(test_ids, preds, 'submission.csv')
pd.read_csv('submission.csv').head()
```

	PassengerId	Survived
0	566	0
1	161	0
2	554	0
3	861	0
4	242	1

Save Model

```
torch.save(model.state_dict(), 'kaggle_titanic.pt')
print('saved kaggle_titanic.pt')
```

saved kaggle_titanic.pt

Load Model

```
loaded_model = Model(len(FEATURES)).to(DEVICE)
loaded_model.load_state_dict(torch.load('kaggle_titanic.pt', map_location=DEVICE))
loaded_model.eval()
print('loaded kaggle_titanic.pt')
```

loaded kaggle_titanic.pt

```
/var/folders/qg/60m3b34x32gcgr10l3665300000gn/T/ipykernel_74107/3707992541.py:2:
FutureWarning: You are using `torch.load` with `weights_only=False` (the current default
value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipped to
`True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly
allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start
setting `weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this experimental
feature.
loaded_model.load_state_dict(torch.load('kaggle_titanic.pt', map_location=DEVICE))
```

Inference

Function

```
def predict(model, features):
    """
    Predict survival for one passenger.

    Parameters:
        model (nn.Module): Trained model.
        features (list): Raw feature values in FEATURES order.

    Returns:
        tuple: Predicted label name and probability.
    """
    model.eval()
    scaled = (np.array(features, dtype=np.float32) - X_MEAN) / X_STD
    with torch.no_grad():
        X_new = torch.tensor(scaled).float().to(DEVICE).unsqueeze(0)
        probability = torch.softmax(model(X_new), dim=1)[0, 1].item()
        label = 'survived' if probability >= 0.5 else 'died'
    return label, probability
```

Run

```
label, probability = predict(loaded_model, [3, 0, 25.0, 7.25, 1, 1, 0])
print(f"Predicted: {label} (p={probability:.4f})")
```

```
Predicted: died (p=0.4490)
```

Submit to Kaggle

1. Open the Titanic competition page.
2. Click **Submit Predictions** and upload `submission.csv`.
3. Read your score on the public leaderboard.
4. Iterate on features and models to improve it.